工程 · 技术栈解剖

人形机器人：从一颗关节往下挖

从磁场、电流到一次思考：一台人形机器人的六层解剖

〔作者名〕 著

排版稿 · 未定稿 · 2026-10 | 全书 46 章 · 约 21.8 万字

图书在版编目（CIP）数据

人形机器人：从一颗关节往下挖：从磁场、电流到一次思考 / 〔作者〕著．— 〔城市〕：〔出版社〕，〔2026.X〕

ISBN 〔978-7-XXX-XXXXX-X〕

Ⅰ．①机…　Ⅱ．①〔作者〕　Ⅲ．①机器人—普及读物　Ⅳ．①TP24-49

〔出版社版本图书馆数据核字（2026）第 XXXXXX 号〕

〔**出版社**〕**出版发行** 地址：〔　〕　邮编：〔　〕 经销：新华书店　印刷：〔　〕 开本：787 mm × 1092 mm　1/32（成品 145 mm × 210 mm） 印张：〔　〕　字数：218 千字　版次：2026 年〔　〕月第 1 版 印次：〔　〕　定价：〔　〕元

本书排版与字体 正文：思源宋体（Noto Serif SC）；标题与表格：思源黑体（Noto Sans SC）。 二者均为 SIL Open Font License 1.1，允许商用出版与嵌入，无需另行购买授权。

关于本书与宇树科技的关系 本书为独立作者撰写的技术解剖类读物，**与杭州宇树科技有限公司无关，未获得其授权、审阅或背书**；书中全部参数以厂商截至 2026-10-07 公开页面与公开仓库为准，不构成采购或投资依据。 Unitree、宇树及其产品型号为各自所有者的商标。书中涉及厂商公布的数据均标注抓取日期；厂商未公布的数据一律写明“未公布”，不以推测填充。

关于图片 本书图片分三类，逐张在图注与附录中标明来源与许可：① 作者实拍；② 本书自绘示意图；③ 经许可或依合理引用原则使用的第三方图片。任何未获授权的图片不会进入成书版。

目录

一台人形机器人的六个时间域 **18**

一个 35 公斤的物体正在向你走来，请注意它的时钟 18

六个时间域，三条总线，两个世界 22

为什么“最快的那层”决定这台机器能不能用 24

一张表看清：慢的那层在急什么 25

怎么读这本书 26

第 01 章 **这本书怎么读：一把深度尺** **29**

一句话说明本书的验收标准 29

每章的四格，和它们不许空着 30

一处本书挖到一半就停下的地方 31

五条阅读动线 32

三条阅读纪律（也是本书的写作纪律） 32

本书不做什么 33

一个建议：先读第 1 部，再决定要不要读全书 34

第 02 章 **宇树的人形产品线，以及为什么这本书不讲机器狗** **36**

先把机器摆出来 36

为什么这条产品线值得单独写一本书，而不是“机器狗那一章的续集” 39

一条器件阶梯：23.7 → 34.4 → 120 → 360 40

平方-立方律：为什么不做一个三米的宇树 41

四足在这本书里只出现两次 47

类比，以及它在哪儿破 47

第 03 章 **一台 G1 的钱花在哪：23 个关节、35 kg、8.5 万元** **50**

先把最贵的那一件事说清 50

先把这两个第三方数各自验一遍 51

关节模组的单价：一条能自己走完的链 52

减一减，看这条链闭不闭合 53

数量是乘数：一颗关节的钱怎么变成一整台机器的钱 53
那“便宜”到底是怎么来的 55
拆开看：一台 G1 的五个成本块 57
毛利那条线：60.13% 说的是哪一台机器 57
一条被删掉的除法：两个“超过”为什么不能相除 58
四条毛利率：它们不在同一件事上，本书不取平均 59
那颗东西贵在哪一格：关节成本往下压一分钱，整机毛利涨多少 60
产能那一格：18 秒一颗关节 61
类比，以及它在哪儿破 61

第 04 章 **机器人只有一个抓手：电流** **65**
你以为你在控制位置，其实你在控制电流 65
唯一的一条式子，和它为什么不是随便写的 66
为什么“只有电流”这件事对工程师是个坏消息，也是好消息 67
一个日常类比，以及它在哪儿破 69
于是“力控”这件事被重新定义了 69

第 05 章 **磁场只认垂直分量：FOC 里那个 F 到底指什么** **72**
推门的时候，你从来不需要把门推开 72
从洛伦兹力到 $\tau = m \times B$，四步 73
所以 FOC 的 F 定向的是什么 74
一个可以立刻用的推论：偏离 90° 的代价有多大 76
类比，以及它在哪儿破 77

第 06 章 **电刷是被淘汰的答案：换向、火花，以及把电流搬回定子** **80**
直流电机也能做 FOC，只不过它用机械做 80
于是有刷电机有一串不可修的毛病 81
无刷的解法：把麻烦的一边挪到不转的那边 82
六步换向与 FOC 是同一件事的两个精度版本 82
那么步进电机呢 83
类比，以及它在哪儿破 84

第 07 章 **旋转磁场是合成出来的：三相、120°、以及为什么没有三次谐波** **86**

三个只能直着推的人，合出一个匀速画圆的力 86
一次加法 87
为什么是三相，不是两相、四相 88
三相到两相的那次换算，有个名字叫 Clarke 89
到这一步，我们手上有什么、还缺什么 89
类比，以及它在哪儿破 90

第 08 章 **改变导通时间：PWM 如何在六边形里合成任意磁场** **92**
只有八种接法，却要给出无数种力 92
伏秒平衡：唯一需要记住的一条原理 93
从三个数到一个矢量：能到的点构成一个六边形 94
一个立刻能用的推论：峰值扭矩和最高转速不能同时拿到 95
为什么开关频率不能无限高：一笔会来收账的账 96
死区：那 100 纳秒里，你的磁场被偷走了一点 97
类比，以及它在哪儿破 98

第 09 章 **转子不介意你是斩波的：两级低通与“平均”为什么合法** **100**
一秒钟两万次的通断，凭什么电机不抖 100
第一级：电感是一个电流积分器 101
第二级：惯量是一个转速积分器 103
那频率为什么不能无限高 105
一个立刻能用的推论：你能控制的最快的事，就是电流 106
类比，以及它在哪儿破 107

第 10 章 **Clarke 与 Park：把交流电机在数学上变成直流电机** **109**
你没法给一个旋转的东西设定“大小” 109
那把跟着转的椅子 110
转完之后，世界突然安静了 111
为什么这套东西要在 20 kHz 的节拍上跑 112
反过来说：θ 错了会怎样 114
类比，以及它在哪儿破 115

第 11 章 **i_q 管扭矩，i_d 管磁场：最大扭矩每安培与弱磁** **117**

两个直流量，一个做功一个不做功，那为什么要留第二个 117
默认为什么是 i_d = 0 118
内嵌式电机：让 i_d 为负反而更好 119
弱磁：高速时唯一能救扭矩的办法 120
人形关节需要弱磁吗 121
一个能带走的判断法 122
类比，以及它在哪儿破 122

第 12 章 **三个环其实是一个环：电流 → 速度 → 位置的带宽递减律** **124**
你写了“走到 0.5 米”，机器里最后剩下的是“40 安培” 124
为什么必须分三层，而不是一个大环直接算占空比 125
宇树把哪一层交给了你 127
一个必须更正的说法：宇树官方明说它**不是**级联三环 129
那级联判据错了吗？没有，它换了位置 131
两种架构的取舍，可以并排讲 131
类比，以及它在哪儿破（补一条） 132
那 6000 Hz 到底是什么 133
频率阶梯：一台人形上真实存在的六个数 134
类比，以及它在哪儿破 135

第 13 章 **死区、采样时刻与母线电容：驱动板炸管的四种方式** **137**
原理全对的一版代码，第一次上电就冒烟 137
第一类：直通与死区——两个必须同时存在的错误 138
第二类：采样时刻——你的电流值可能来自错误的那半周期 139
第三类：母线电容——机器人会发电，而电能没地方去 140
第四类：热——唯一一种不会立刻炸你的失效 141
一张上电前的检查表 142
类比，以及它在哪儿破 143

第 14 章 **编码器、齿隙与准直驱：为什么 G1 用 20.58:1 行星而不是谐波** **145**
你能测到 0.011°，但你的关节能撒谎 0.1° 145
齿隙为什么骗人：一段“允许空转”的对话 147

宇树自己承认过这件事 148
刚度上限还有一个来源：指令字段的量程 149
双编码器（转子端与输出端各一颗角度传感器，两数之差就是齿隙与弹性变形的实时读数）为什么重要 150
于是两条路线分开了 151
如果非要装力传感器，它靠什么工作 151
类比，以及它在哪儿破 156
第 15 章 三代关节的账本：23.7 → 34.4 → 360 N·m 158
四颗电机，一条因果链 158
第一笔账：那条 τ = k_t·i 到底准不准 160
第二笔账：官方数据自己完成了 τ = τ·N 的证明 161
第三笔账：扭矩密度与平方-立方 162
第四笔账：功率密度为什么不能当额定用 162
三代之间到底变了什么 163
还有一个没被选中的答案：丝杠 164
类比，以及它在哪儿破 165
第 16 章 双足为什么比四足难：四个支点可以零力矩，两个不行 167
站着不动，其实是最难的动作 167
支撑多边形：一个二维凸包决定生死 168
时间常数：你只有 0.259 秒 168
步频也是算出来的 170
能量：那份没有休息窗口的账 170
于是第 1 部那些取舍被解释了 171
类比，以及它在哪儿破 172
第 17 章 倒立摆、ZMP 与捕获点：站稳的判据到底是什么 174
“它看起来挺稳”不是判据 174
零力矩点：地面推你的那个力的作用点 175
判据：ZMP 在支撑多边形内就不翻 176
但 ZMP 有三个前提，每一个都会破 177

于是有了两套流派 179
类比，以及它在哪儿破 180

第 18 章 **刚度是软件写的：阻抗控制与那颗可编程的弹簧** **182**
同一个关节，可以像石头，也可以像弹簧 182
天花板一：指令字段的量程 183
天花板二：执行器饱和——刚度与允许误差的乘积有上界 184
天花板三：稳定性——刚度是一个谐振频率的平方 185
阻尼那一列：为什么 kd 不能随便给 186
于是“柔顺”这件事被重新解释 187
补一条：宇树自己就是这么算 kp 的（官方数已验） 187
没有力传感器时，怎么知道被推了 188
类比，以及它在哪儿破 192

第 19 章 **全身控制与 MPC：一次 1 kHz 里要解完的那堆方程** **194**
23 个未知数，一毫秒解完 194
先看清楚要求有几类 195
计算量：为什么自由度是原罪 196
模型预测控制：把“下一步”变成“下 N 步” 197
真正爆炸的不是维度，是接触模式 197
于是可以解释一台人形长什么样 198
类比，以及它在哪儿破 199

第 20 章 **一次抬腿的焦耳账** **201**
电池里到底有多少东西 201
那为什么人能动一整天，机器只能动三小时 203
关键差别：人站着几乎不耗能，机器站着很耗能 204
一次抬腿的焦耳账 204
用 CoT 把这件事变成一个可比较的数 205
类比，以及它在哪儿破 207

第 21 章 **强化学习运控：宇树公开的这条工序** **210**
为什么现在的人形用神经网络走路 210

官方公开的四个步骤 211
仿真器版图：Sim2Sim 要跨的是“哪一族” 212
版本陷阱：这条工序正在换代 222
一次训练里，控制环是怎么摆的 223
域随机化：随机的是哪些数 224
特权信息与蒸馏：一个已用上的，一个没启用 225
一个能救你一天的工程陷阱 226
类比，以及它在哪儿破 227

第 22 章 **sim2real 的沟：仿真里能跑，真机上为什么会抖** **231**
同一段代码，仿真里稳如钟，真机上抖成筛子 231
第一样：结构模态——仿真里的机器人是刚的 232
第二样：接触求解器——两个仿真器算出来的力不一样 233
第三样：摩擦、齿隙与静摩擦——仿真里通常根本没有 234
第四样：延迟——仿真里是零 235
一次 1 kHz 控制环的时间到底去哪了 236
一张诊断表 241
类比，以及它在哪儿破 241

第 23 章 **眼睛：相机、深度、激光雷达，以及时间同步比分辨率重要** **244**
一台 G1 能看见什么 244
双目测距的精度是谁给的 245
现在到本章真正的论点：先同步，再谈分辨率 246
同步误差还会污染学习 247
激光雷达为什么能便宜到 999 元 248
户外强光：一个被反复忽略的失效 248
类比，以及它在哪儿破 249

第 24 章 **前庭与状态估计：IMU、双 IMU，以及「我在哪」是怎么算出来的** **251**
人闭着眼也能站稳，靠的不是眼睛 251
官方能查到的全部信息 252
器件层第一跳：这颗陀螺到底在测什么 253

器件层第二跳：零偏不是一个数，是一条曲线 255
反推一：零偏决定你能站多久 256
反推二：采样率由控制带宽决定 257
反推三：振动敏感度，一个几乎没人写的指标 258
反推四：温度——DDS 里那个字段本身就是证词 259
为什么是双 IMU 261
本体感知：编码器就是“肌肉感觉” 262
再往下层：把这些碎片拼成“我在哪” 263
类比，以及它在哪儿破 265

第 25 章 **手：Dex3-1 的 33 个触觉点与空心杯电机** **268**
全身最难的部件不在腿上 268
第一道算术：捏住一瓶水要多大力 268
第二道算术：触觉点要多少 272
第三道算术：空心杯电机为什么没有铁芯 274
第四道算术：为什么腱绳是一种妥协 276
手的控制为什么与身体解耦 277
类比，以及它在哪儿破 278

第 26 章 **大脑的功耗预算：130 W 的 Thor 与 6B 的模型** **282**
先把官方给了什么说准 283
上层到底要多少赫兹 284
现在算真正卡住的那件事 284
那把精度降到 FP8、INT4，为什么就救得回来 286
而真正的第一道墙是功耗 288
一个可以立刻用的判断法 289
类比，以及它在哪儿破 290

第 27 章 **三条总线：RS-485 4 Mbps、CAN-FD 5 Mbps、EtherCAT 1 kHz** **292**
先把这条线看成电压，而不是看成“通信” 293
位时间与线上延迟：同一张账 294
半双工要换向：那个换向时间从哪来 295

三条总线的物理差别 296
用官方自己的数字，算出一处对不齐的地方 298
第二层原因：分层之后，带宽算术站在它这边 299
第三层原因：钱，而且是可算的钱 300
那 RS-485 的代价是什么 301
如果你真要用 EtherCAT，链条长什么样 302
类比，以及它在哪儿破 304

第 28 章 中间件：DDS 话题、rt/lowcmd，以及行业的“去 ROS 化” 307

宇树的核心不是 ROS 308
三个厂商，三条路 308
先算清一件事：这根本不是带宽问题 309
关键路径上的四件事 310
UDP 与 TCP：为什么实时环宁可丢包也不要重传 312
现在把宇树这套接口拆开看 313
请求-响应那一半：为什么不只用主题 314
版本地狱：这一节是给你省时间的 315
行业在往哪走 316
边界：这不等说 ROS 不能实时 317
从 ROS 2 那边看过来：四样东西的真身 317
类比，以及它在哪儿破 319

第 29 章 从轨迹到策略：模仿学习、ACT、扩散与流匹配 322

一个 1966 年的想法，和它为什么一直不够用 322
多峰：为什么 MSE 会教出一个“既不左也不右”的机器人 324
“建模分布”到底改了哪一行：先抽一个随机量，再回归 325
分布偏移的正解：让状态分布可控 328
一个必须知道的上限 330
类比，以及它在哪儿破 331

第 30 章 VLA 的架构共识：主干 + 动作专家 + 动作分块 + 异步双系统 334

一个行业在两年内收敛到了同一张图 334

部件二：动作专家，为什么必须独立 336
图纸上还缺第三方：状态机与行为树 337
部件三：动作头三选一 342
部件四：异步双系统，因为物理要求它 343
把四个部件接起来：一次完整的推理 344
宇树在这张图上的位置 345
类比，以及它在哪儿破 346

第 31 章 **延迟墙：5 Hz 的模型怎么驱动 200 Hz 的手** **348**
把上一章的“异步”变成一个数 348
于是可以解释一批“演示很惊艳、现场不行” 350
三条出路，各付各的代价 351
断网能不能干活，取决于回路有没有跨无线 352
一个可以带走的检查方法 356
类比，以及它在哪儿破 356

第 32 章 **宇树的 UnifoLM：世界模型-动作与 6B 全身基座** **359**
先把三个模型和它们的许可证分清楚 359
为什么“接标准”是理性的 360
许可证不是法律细节，是产品参数 361
NC 拦住的到底是哪一步：把条款拆成动作 362
为什么 WLA 用 Apache、WMA 用 NC：上游替它做了决定 364
一个可以带走的检查动作 364
“6B / 2500 小时 / 64 任务”这三个数该怎么读 366
类比，以及它在哪儿破 367

第 33 章 **数据经济学：一条演示值多少钱** **370**
先给一条演示标个价 370
把单价乘回去：一百万条要多少人、多少钱 371
三条路线，三条完全不同的成本曲线 372
把三条路线放到同一张成本图上 374
仿真的“前重后轻”轻在哪：三行除法，以及一个看错的地方 374
人类视频那一行：便宜在哪，贵在哪 377

规模对照：现在谁手里最多 377
一个空白：没有人有田间数据 379
这条账在什么时候失效 379
一条演示到底长什么样：能回放的失败才有资格进数据集 380
类比，以及它在哪儿破 382

第 34 章 **世界模型现在只解决两件事：数据和评测** **385**
“世界模型”是四个不同的东西共用一个名字 385
为什么它进不了实时环：把三个除法做完 386
三家各在哪一格：名字都叫世界模型，位置不一样 388
那它真正在做的两件事 390
宇树 WMA-0 在这张图里 391
驾驶侧的那面镜子 393
这一章的证据等级为什么必须写低 394
类比，以及它在哪儿破 395

第 35 章 **电池：13 串、9000 mAh、两小时** **399**
官方给了三个数，我们能推出第四个数 399
第一步：串数与电压必须自洽 400
第二步：用能量再验一次 401
第三步：这个选择意味着什么 401
第四步：算一算“能量密度”这条线走到哪了 402
从上电到出力的那 200 毫秒 404
类比，以及它在哪儿破 409

第 36 章 **热：密封壳里电机与驱动互相加热** **413**
一个所有线索都指向的空白 413
先把热路画出来 414
一次具体的算术 416
正反馈：那条会自己加速的链 418
大脑与关节抢同一份散热 419
于是本章回答那个问题 419
给读者的一个可执行动作 420

类比，以及它在哪儿破 420

第 37 章 **抱闸、急停与断电时序：让一台 70 kg 的机器安全地停下来 423**

“按一个红按钮”不是安全设计 423

先算一笔势能：断电那一瞬间，那几百焦耳往哪儿去 423

先看一个具体的失效：抱闸 425

抱闸为什么是“断电才夹紧” 425

那为什么不能在高速下把抱闸直接抱死 428

急停为什么要双通道 430

断电之后这条腿会不会塌：自锁性是一条能算的判据 432

人形独有的两条：没有围栏，而且会倒向人 433

上电那一侧同样有时序 435

一张“该问厂商什么”的清单 436

类比，以及它在哪儿破 437

第 38 章 **泛化与长时程：演示很惊艳，连续作业很难 440**

一个二位的乘法，毁掉大部分机器人宣传 440

为什么是乘法，不是加法 441

把“8 小时”换算成步数 442

于是“该报什么指标”这件事被重新定义 443

可查到的少数几个诚实数字 444

三条让长时程失败的原因，只有一条是算法 445

执行器磨损改变动力学：同一个策略为什么三个月后会退化 446

人会检核与重试：把乘法改成平方 447

一个可以带走的评估框架 449

类比，以及它在哪儿破 449

第 39 章 **行业分工与人才缺口：谁在做什么，缺什么人 452**

同构的技术栈，分歧的三处 452

往下追一层：为什么偏偏是这三处分叉 454

一条判据：技能要能对账，不能对口号 456

这条链在哪里断：门槛低不等于这层好做 458

六层各自的拥挤程度 459

三家的钱押在哪，说明他们各自认为瓶颈是什么 460
三家共同承认的两个短板 461
一个空白，和一个位置 461
一张“我该往哪一层走”的判据表 462
类比，以及它在哪儿破 462

第 40 章 **一个工程师的十二个月：把这本书变成作品集** **466**
这一章的规矩：每一层只认一个可测产物 466
什么叫“可复算的验收”：三条判据先算给你看 467
里程碑 ①：一颗会听话的关节（第 1–10 周） 470
为什么第一件作品是一颗会听话的关节，而不是一台会走路的机器人 471
里程碑 ②：一条有证据的实时链路（第 8–14 周，可与 ① 并行） 473
里程碑 ③：一次恒力按压（第 12–18 周） 474
里程碑 ④：让一台仿真里的机器站起来（第 16–26 周） 475
里程碑 ⑤：一份自己的数据集（第 20–36 周） 476
里程碑 ⑥：一次诚实的评测（第 30–48 周） 477
十二个月怎么排 477
三条纪律，比路线更重要 478
类比，以及它在哪儿破 479

附录 A **符号、术语与单位** **482**
A.1 符号 484
A.2 单位与书写约定 486
A.3 本书踩过的四个单位坑（写出来，别重复踩） 487
A.4 三个必须标明“口径”的量 487
A.5 变换约定声明 488
A.6 术语对照（一词一义表） 488
A.7 引用与标注 489
A.8 图的达成度（和 A.7 同一条规矩：缺就写明） 491

附录 B **全书可复算的数（含脚本与输入表）** **492**
B.1 为什么这张表必须由脚本生成 492

B.3 主算例（人形）与对照数据（arithmetic_check） 497
B.4 概念依赖与术语预算校验（check_kg） 499
B.5 输入数据表的位置 499
B.6 自检用例（防脚本自己算错） 500

附录 C **器件与人形参数表** **501**
C.1 人形与类人形整机 501
C.2 一体化关节与微型执行器 503
C.2a 官方电机模型参数（取自 5
unitree_rl_lab/source/.../assets/robots/unitree_actuators.py, 0
[厂]） 5
C.2b 关节里那颗 MCU：选型表与“为什么要 FPU” 509
C.3 灵巧手与机械臂 512
C.4 感知与算力（可买到的器件） 513
C.4a 关节 MCU 选型表（本书逐格回原厂核对） 515
C.5 对照：同期其他厂商的公开口径（仅列可核项） 530
C.6 这张表里最该被记住的一栏 530

附录 D **去哪继续学（含许可与失效风险标注）** **532**
D.1 第 1 部（一颗关节）：电机与电力电子 532
D.2 第 2 部（从关节到姿态）：控制与 RL 534
D.3 第 3 部（感知、算力、总线） 535
D.4 第 4 部（大脑与数据） 536
D.5 第 5 部（瓶颈）：可靠性、安全与标准 537
D.6 一条通用判据：怎么判断一个入口值不值得花时间 538
D.7 三个失效风险声明 538

附录 E **参考来源、抓取日期与「查不到」清单** **540**
E.1 抓取日期 540
E.2 一手来源的取法（可复现） 540
E.3 已引用的 URL 清单（144 条，按字典序） 541
E.4 「查不到」清单 546
E.5 冲突口径登记（本书一律并列，不取平均） 548

E.6 本书判定为「不可采信」的流传说法 549

附录 F **附录：语录出处一览** **550**

一台人形机器人的六个时间域

一个 35 公斤的物体正在向你走来，请注意它的时钟

宇树 G1，1.32 米高，35 公斤，23 个自由度，起售价 8.5 万元。它朝你走三步，停下，抬手臂，握一下拳。

如果让你猜这台机器里“最快的那件事”是什么，多数工程师的第一反应是 CPU 主频，第二反应是网络延迟。两个都猜错了。这台机器里最快的事件发生在某颗关节的驱动板上，每 50 微秒一次：一个 MOSFET 关断，另一个还没打开，中间的缝隙里电流改走了体二极管。这件事一秒钟要发生几万次，而它的存在与否，你从外面完全看不出来。

反过来，这台机器里最慢的事也值得注意：一次“要不要去拿那个杯子”的决定，可能两三百毫秒才做一次，一秒钟做不了五次。

从 50 微秒到 200 毫秒，同一台机器上同时跑着相差四个数量级的时钟。这不是工程上的巧合或者浪费，而是物理逼出来的：**每一层能控制的东西，受它自己那层的物理时间常数限制**。电流被电感管着，转子被惯量管着，平衡被重力管着，语义被内存带宽管着。你把它们塞进一个循环里跑，就会得到一台会抖、会摔、会炸管的机器。

所以这本书不打算按“感知—决策—执行”这种教科书顺序讲。我们按**往下挖**的顺序讲：先给你这张六层地图，然后一头扎进最底下那层，从一颗关节的铜线开始，一路挖到磁场、电子、半导体；挖到底之后，再一层一层建回来，回到姿态、回到视觉、回到那个 6B 参数的大脑，最后看看这套东西现在撑不住的地方在哪。

我从小就非常喜欢科技，也非常喜欢机器人，2009年读大一的时候，我做了第一款双足人形机器人。2016年，正好有外部投资者和客户愿意购买我做出来的机器人，我就辞职出来创业了。

王兴兴 — 上海证券报《做机器人第十六年 我看到了“黎明的光”——对话宇树科技董事长王兴兴》（新浪财经转载） — 2026-08-19（直接引语（问答实录））

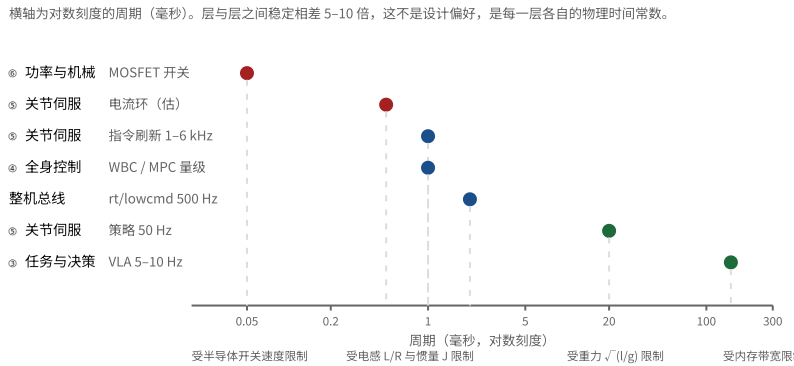


图 00-1　同一台机器上的六个时钟（对数周期轴 + 每层的物理限制来源）

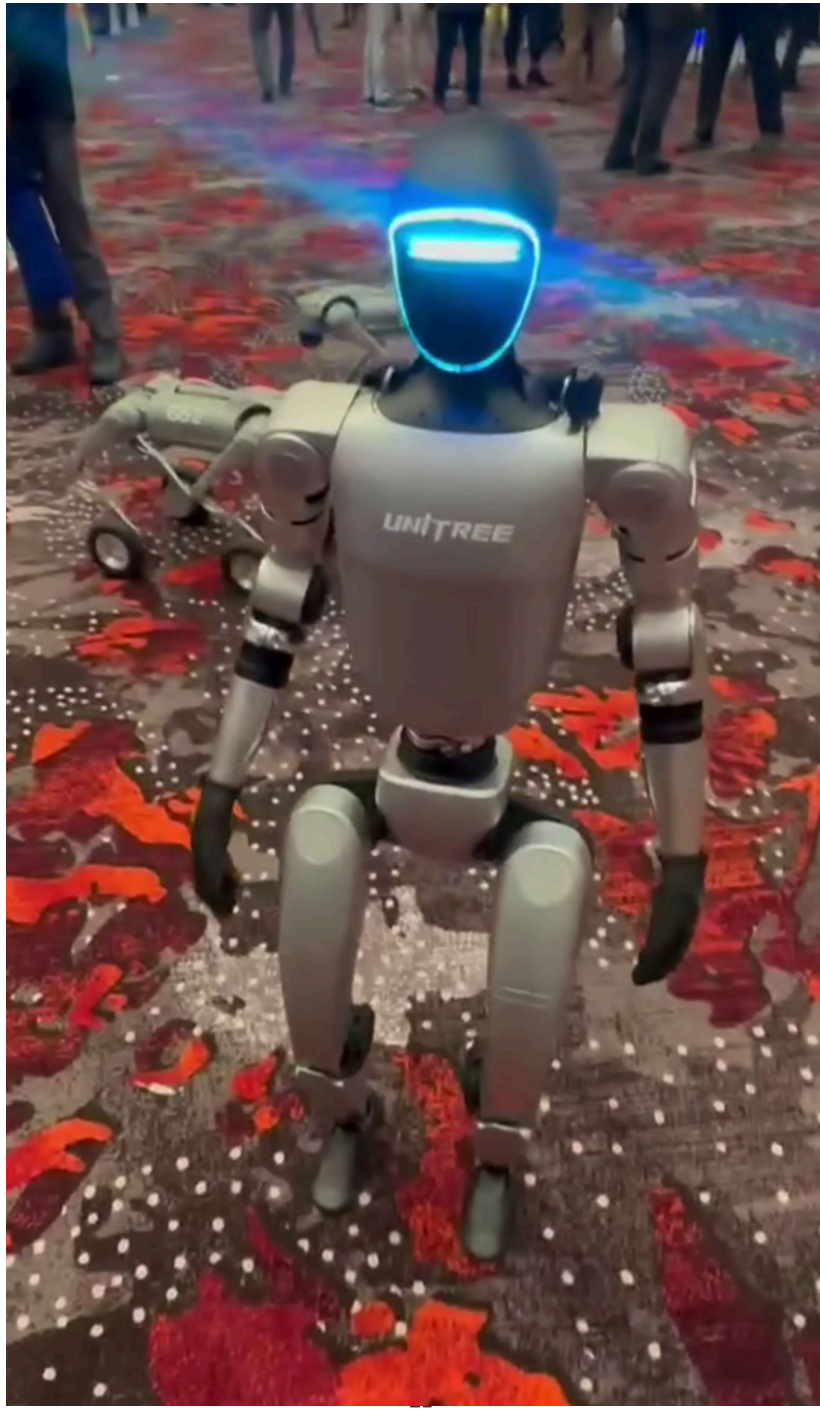

UNITREE

图 00-2　宇树 G1 整机实拍（会议现场；背景是一只 Go2）｜Photo by Sayanesy via Wikimedia Commons, CC0

六个时间域，三条总线，两个世界

先把地图摆出来。左边一列是这台机器实际存在的层次，中间是它的工作频率，右边是这一层的物理约束来自哪里。

层	里面装什么	工作频率	被什么物理量限制
① 感知	相机 · 激光雷达 · IMU · 触觉	10–60 Hz	曝光时间与点云刷新
② 状态估计	VIO · SLAM · EKF · 关节标定	50–200 Hz	传感器噪声与融合带宽
③ 任务与决策	VLA 大模型 · 行为树 · FSM	1–10 Hz	内存带宽（第 26 章算）
④ 全身控制	WBC · MPC · 逆动力学 · 轨迹插补	100–1000 Hz	倒立摆时间常数
⑤ 关节伺服	FOC 电流环 · 速度环 · 位置环	1–20 kHz	电感时间常数 L/R
⑥ 功率与机械	MOSFET 桥 · 减速器 · 轴承 · 热	开关 20 kHz	载流子与声子（热）

来源标注：这张分层是**本书的分析工具**，不是宇树公布的架构图。宇树从未公布过自己产品的分层、频率分配或模块边界 [未]。频率一列里，⑤ 的指令刷新率有官方口径（GO-M8010-6 通信控制频率 6000 Hz [厂]），④ 的 1 kHz 量级来自腿式机器人控制综述里对 QP 回路的通行描述 [行]，③ 的数字来自各家模型自陈的推理节奏 [查]。它们被放在同一张表里，只是为了让你看见跨度；不要把任何一格当成厂商规格。

这张表只说了“谁限制谁”，没说“限制到什么程度”。把右列那六格各换成一个你能自己复算的数，这本书的骨架就出来了。下面六行是全书的索引，每一行都在括号那一章里被推到底层：

- ① **感知**：双目测深度的误差按距离的平方涨。用官方公布的 60 mm 基线，50 cm 处误差 3 mm（够抓杯子），4 m 处 19.8 cm，8 m 处

79.4 cm。更要命的是不同步：1 m/s 的相对运动加 20 ms 不同步，会产生 13.4 像素的假视差，等于 27 倍的测距噪声（第 23 章）。

- ② **状态估计**：陀螺的零偏是**积分**进去的。0.1 °/s 的零偏，10 秒漂 1°、100 秒漂 10°——纯靠陀螺，一台人形只能“盲站”十几秒。可它在同一个平衡时间常数（0.259 s）里只累积 0.026°，比 1° 的容差小 38 倍。**平衡环在乎的是噪声与延迟，导航环在乎的才是零偏**——这是两个器件等级（第 24 章）。
- ③ **任务与决策**：6B 参数 × 2 字节 = 12 GB 权重，摊在 100 GB/s 的内存带宽上，单步推理 ⩾ 120 ms，也就是 ⩽ 8.3 Hz。这一层的频率不是算法选的，是芯片的搬运能力给的（第 26 章）。
- ④ **全身控制**：腿是一根倒立摆，$\tau = \sqrt{(l/g)}$；G1 髋高 0.66 m ⇒ τ = 0.259 s，也就是它“倒下去”的速率是 $1/\tau \approx 3.9\ s^{-1}$。这不是角速度，别按 rad/s 读：它的意思是误差每 0.26 秒被放大 e 倍，折成频率是 0.61 Hz。把 ③ 那 120 ms 直接接进平衡环，就占掉 τ 的 46%（第 16 章那张表里 100 ms 是 39%、120 ms 是 46%），落在“增益不敢开大 → 站不稳或抖”的那一格。**这不是模型不够好，是 τ 不允许**（第 16 章）。
- ⑤ **关节伺服**：这一层被两件事同时夹住。一是延迟吃相位：回路总延迟 T 越长，闭环能用的带宽上限越低，判据是 0.35 ÷ T；开关周期 50 µs 先取一半（电流是三角波，平均效应看半周期）再加计算 25 µs ⇒ T = 50 µs ⇒ 上限 7,000 rad/s ≈ 1.1 kHz。二是电感不服从快指令：电流时间常数 τ_e = L/R，开关周期必须 ⩽ τ_e/10，否则纹波吃掉“平均值”这个前提。两者取小者才是这一层的真频率（第 09、10 章）。
- ⑥ **功率与机械**：24 V 母线在线性区只能给 13.86 V 相电压幅值，而 30 rad/s 满载需要 24.9 V——**手册上的“峰值扭矩”和“最高转速”是两个角点，不能同时达到**。同一层另一头是热：密封、无风扇、散热面积按 L^2 缩而发热按 L^3 涨，一颗持续出 60 N·m 的关节约 55 W 损耗，需要 ⩽1.09 K/W 的总热阻才压得住温升（第 08、36 章）。

这六行没有一行来自宇树公布的架构。它们是本书从器件页、官方仓库源码和教科书式的一手物理里，一条一条推出来再拼成这张表的——所以每一条你都可以回去查、回去算、回去反驳。

跨层的连接靠三条性质完全不同的总线：

板内——SPI、I^2C、UART，跑的是传感器和调试。**关节主环**——宇树官方明确“目前生产的所有电机均用 RS-485”，GO-M8010-6 是 4 Mbps、一条总线最多 15 台、通信控制频率 6000 Hz [厂]；同行的另两家选了别的路：智元公开的结构是 EtherCAT 1 kHz 主站转三条 CAN-FD 5 Mbps 从站 [查]，开源标杆 moteus 用 STM32G431 加 5 Mbps CAN-FD [查]。**算力侧**——千兆以太网，上面跑 DDS 主题；宇树的核心 SDK 本身就是 CycloneDDS 的主题与请求响应，ROS 2 只是它外面的一层适配 [厂]。

第三条线是“两个世界”：真机不可微、会摔、会发热；仿真可以并行上万个环境、可以给你真机永远拿不到的梯度，但它和真机之间有一道沟（第 22 章）。谁能把两个世界之间的那道沟填平，谁就掌握了数据的生产速率——这句话在第 4 部会变成一整章的算术。

为什么“最快的那层”决定这台机器能不能用

你的直觉大概是：大脑慢一点没关系，反正走路这件事早就写好了。这句话对了一半，但结论要反过来才成立——**一个闭环能多快响应扰动，由它自己那一环的完整回路延迟决定，而不是由最聪明的环节决定**。

把机体当成一个倒立摆，它有一个特征时间：

```
τ = √(l / g)
G1 髋高 ≈ 0.66 m ⇒ τ = √(0.66 / 9.80665) = 0.259 s
```

（髋高由官方身高 1.32 m 与腿长 0.6 m 量级取 [算]，不是官方值；脚本 `foc_check.py pendulum()` 可复算。）

这 0.259 秒就是“从开始往前倒到你必须做点什么”的窗口。控制回路的延迟占这个窗口的比例，直接决定你还能不能救回来：

回路总延迟	占 τ	后果
10 ms	3.9%	相位裕度基本没被吃掉，高增益可用
50 ms	19%	明显吃裕度，开始需要前馈
100 ms	39%	增益不敢开大，站不稳或者抖

摆动相更苛刻。步频 2 Hz、占空比（占空比 = 一个开关周期里“开”的时间占比，第 08 章） 0.4，意味着单腿支撑只有 200 ms 左右；脚掌砸地那一瞬间需要的力矩指令如果晚到 100 ms，这一步就废了。

于是“低层必须快、高层可以慢”不是一句经验，而是从 τ 里推出来的。这也解释了为什么宇树把关节的指令刷新率标到 6000 Hz，而把策略输出的节奏放在几十赫兹——它们本来就不该是同一个数 [厂]。

一张表看清：慢的那层在急什么

反过来看，第 ③ 层为什么不能快？很多人以为是“模型太大算不动”，这个说法太含糊，含糊到没法检查。真正卡住它的是**内存带宽**，而且这是一道可以在两行内算完的算术（第 26 章会完整展开）：

```
6B 参数 × 2 字节（FP16）= 12 GB 权重
每生成一次动作，这 12 GB 至少要从显存里读一遍
12 GB ÷ 100 GB/s（一台端侧模组的典型带宽量级）= 120 ms
⇒ 上限 ≈ 8 Hz，还没算 KV、激活、多路图像
```

100 GB/s 这个带宽是行业通识量级 [行]，宇树没有公布 G1/H2 机内模型的实际推理频率 [未]。但这条链给出的结论是稳的：**大脑天生就是 5–10 Hz 的东西，不是工程没做好**。 所以全行业不约而同收敛到同一个架构——低频的语义规划加高频的动作跟随，中间用“一次输出一段动作”来桥接（第 30、31 章）。

现在把两头的数字放在一起看：

```
关节电流环     每 50 μs 一次       一秒钟 2 万次
大脑一次推理   每 120 ms 一次      一秒钟不到 10 次
```

这两个数字相差 2400 倍，而它们在同一台 35 公斤的机器里必须协同工作。 这本书接下来要讲的所有内容，本质上都是在回答同一个问题：这 2400 倍的落差，中间靠什么接上。

答案是：靠第 ⑤ 层那颗小小的 MCU，靠一个叫做磁场定向的东西，靠把“位置”这个抽象要求一路翻译成“电流”这个物理动作。而这条翻译链的每一环，都是我们接下来十章要挖的。

怎么读这本书

最短的一条完整路线是八章：**00 → 04 → 05 → 07 → 08 → 10 → 12 → 14**。走完这八章，你会知道一颗机器人关节里从磁场到螺丝的每件事，以及为什么它非长成这样不可。

如果你只做上层（算法、模型、数据），可以先读 00、02、26、27、28，再跳到第 4 部；但请至少补读 04 和 12 两章——它们会告诉你，你的模型输出的那串数字，在机器里到底被谁、以什么代价、在多短的时间里执行掉。

如果你做硬件、嵌入式、电力电子，第 1 部（04–15）是给你的，其余部分可以只看每章开头的地图和结尾的数。

三条阅读纪律：

一，**每章至少一个你能自己复算的数**。全书所有推算都收在附录 B，配一个 Python 脚本；正文里凡是标 [算] 的数字，脚本里都有对应函数。脚本自己也有一组“已知答案”的自检用例，防止它算错——这一条不是形式主义，本书的自检脚本已经抓到过我自己的三次算错（一次把无量纲的量当成了有量纲的，一次单位不一致，还有一次 Park 变换的参考角取错）。

二，**每个数字后面那个方括号是真的**。[厂] 是宇树官方页面或官方仓库里能读到的；[拆] 是第三方拆解与券商估算，只能当数量级；[行] 是教科书与行业通识，不是宇树事实；[算] 是本书从前面几类推出来的，可复算；[查] 是**别家的一手来源**（别的厂商产品页、分销商标价、论文、PyPI/仓库文件），能回读但不是宇树说的；[推] 是**没有可回读的一手来源**——要么是二手转述（行业报道的价格、论文或厂商的自陈成功率），要么是本书在这些数字上再往下推的一步。它比 [查] 弱，读者应当把它当成一个可反驳的判断，不是引用；[未] 是查不到。看到 [未] 就是这本书的边界，不是我的疏忽。

三，**遇到“厂商没说过”的地方，本书会明确写出来**。宇树全系关节电机只公布峰值扭矩，没有一个型号公布额定（持续）扭矩；IMU 型号与采样率未公布；电芯型号与化学体系未公布；关节驱动板的 MCU 型号未公布（拆解称芯片丝印被抹除）[未]。这些空白恰恰是最有信息量的部分——它们告诉你哪些宣传不能信，包括本书自己的。

一个可检验的说法：一台人形机器人的可用性由它最慢的那条**闭环**决定，而不是由它最快的那块芯片决定。判据在第 10 章推过：回路总延迟 T_lag 会按 ω·T_lag 吃掉相位，于是可用带宽被压到 $\omega_c \approx 0.35 \div T_{lag}$ 以下；这条上限一旦掉到倒立摆自己的发散率 1/τ 附近或以下——G1 髋高 0.66 m 时 1/τ 是 3.9 s^{-1}——站立的“刚度”就保不住了。镇定一个会自己倒的摆，闭环带宽必须明显高过它，不是刚好够。延迟长一点、带宽掉一格，是同一件事的两种说法。

证据等级：$\tau = \sqrt{(l/g)}$ 是 [行]；G1 髋高 0.66 m 是 [算]（由官方身高与腿长量级推）；6000 Hz 指令刷新率是 [厂]；12 GB ÷ 100 GB/s 的带宽估算里，带宽是 [行]、结论频率是 [算]；分层表本身不是任何厂商的规格 [未]。

可核对清单：

- [ ] 用 `python code/tools/foc_check.py` 复算 τ = 0.259 s 与 10/50/100 ms 的占比

- [ ] 到 unitree.com 的 GO-M8010-6 电机页确认 4 Mbps / 15 台 / 6000 Hz 三项
- [ ] 确认宇树官网**没有**任何一个关节型号标出“额定扭矩”（这是本书的断言，值得你自己去查一次）
- [ ] 找出这张六层表里哪两格的频率是你原以为应该反过来排的

下一章：我们要把这张地图放下，去问一个更基础的问题——一台机器想让你的手停在某个位置，它手里到底有哪几种可以拧的旋钮？答案只有一种，而它不是位置。

第 01 章

这本书怎么读：一把深度尺

一句话说明本书的验收标准

市面上讲机器人的书大致分两种：一种给你**名字**（这台机器有激光雷达、有 IMU、用了强化学习、用了 VLA），一种给你**原因**（它为什么非用不可、代价是什么、在哪一格会失效）。

本书只做第二种。而“第二种”是可以被检查的，检查工具就是一把深度尺：

任何一句技术说明，往下追问三次“那这个又靠什么”。如果三次之内追到了一条物理定律或者一个不可商量的工程约束，这句话就算讲清了；如果第三次还在给你另一个名字，那它就是没讲。

举一个本书第 1 部的真实三连问：

```
问：这个关节能输出多大扭矩？
答：139 N·m。
追问一：扭矩由什么决定？          → τ = k_t · i_q，由电流决定。
追问二：电流由什么决定？          → 由施加的电压与绕组阻抗决定，而电压靠 PWM 占
空比合成。
追问三：那为什么不能任意大？      → 因为可达电压被母线与 SVPWM 线性区卡住
（V_bus/√3），
                                    而反电势随转速涨；两者一交，就是这台电机的
功率包络。
                                    再往下是 MOSFET 的耐压、导通电阻与结温。
```

四次追问，落在半导体参数与一条几何约束上。**这就是本书认为“讲到底”的样子。**

反过来看一个没讲底的例子：

```
问：这台机器人为什么走路稳？
答：它用了强化学习和全身控制。
追问一：WBC 在解什么？          → 动力学方程加接触约束，输出关节扭矩。
追问二：扭矩怎么变成动作？      → ……（多数书在这里停住）
```

停在第二问的书，读者学到了两个名字。本书的第 12 章与第 18 章专门接这两问。

我们不浮夸，也不是PPT公司，还是比较务实，实打实的需求在哪里，我们就解决什么问题。

王兴兴 — 投中网《五年前，对王兴兴的一次未公开访谈》（新浪科技转载） — 页面 2025-08-07；访谈本体约 2020 年前后（早年访谈转写）

每章的四格，和它们不许空着

本书每一章在动笔前必须填完四格，这是硬闸门：

格	要回答的问题	空着会怎样
stack	这一层在技术栈的哪一格，上下游是什么，时间尺度多少	读者不知道为什么要学它
mechanism	它靠什么机制工作（部件、信号、闭环）	变成产品宣传
base	往下挖到的第一性原理：一个公式 + 一步代数	变成名词表
boundary	它在哪里失效、代价是什么、厂商没说什么	变成误导

第三格（base）是本书的门槛：**每一章至少一个你能自己复算的数**。全书所有推算集中在附录 B，配一个 Python 脚本；正文里凡是标 [算] 的数字，脚本里都有对应函数，跑一次就能对。

脚本自己也带一组“已知答案”的自检用例，防止它算错。这条纪律不是装饰——本书的自检脚本已经抓到过作者本人的三次算错：把无量纲的量当成有量纲、单位不一致（kg 与 N）、把一次坐标变换的参考角取反。**第四处算错不是脚本抓的**：抄错一份第三方表格里的连杆质量，是靠逐行回读官方 `go2.xml` 才发现的。**如果我自己都能错四次，你更有理由不信任何一句没配数的话**。

一处本书挖到一半就停下的地方

蓝图的部件地图里有一格叫**任务与决策层**。第 00 章那张六层表给它留了一行，名字也写了：行为树（behavior tree：把“下一步能不能做”写成一张可遍历的判断树）与有限状态机（FSM：把模式写成若干状态加上迁移条件）。这两样是今天人形机器人上真正在跑任务层的东西——宇树的协议里有 `mode` 字段、超时回落到 damped 这类兜底模式（第 27、43 章引过这些字段 [厂]），它就是任务层被协议化之后留下的影子；至于它内部到底是一棵状态机还是一棵树、有几层、谁来仲裁，官方从未公布 [未]。

这一格原本挖到一半就停着，现在由第 30 章补上了三跳：任务层的检查速率必须不低于不变量被违反的速率（120 ms 的 VLA 决策 vs 20 ms 的 50 Hz 任务层，对照 $\tau = 0.259$ s 那 100 ms 预算）；树与状态机的最坏情形能在纸面上界定，而策略网络不能——这一层给的不是智能，是可证明的边界；以及它的三样做不到（不理解任务、节拍受估计器带宽限制、与策略之间的仲裁协议）。**仍然没做到的是两件事**：把这一层的判据写成读者能自己复算的脚本（附录 B 里没有它的条目），以及给出宇树内部究竟是几层、谁否决谁——官方未公布 [未]，本书不编。

为什么还是留着而不是删掉： 因为第 31 章那条延迟不等式的结论要用它。那一章算的是“VLA 的平均反应延迟比这台机器必须做点什么的窗口还长，所以 VLA 不能负责平衡”；而“那谁来负责”的答案正是这一层——一个跑在 MCU 上、节拍比策略环快一到两个数量级的经典任务层。删掉它，第 31 章的结论就悬在空中。**所以本书对它的要求只有一个：你**

知道那格子里有东西，而且知道本书没挖它。 这一处判断记在 `verification/blueprint-coverage.md` 的“逐格人工判定”一节，是已登记的缺口，不是笔误。想找“怎么写一棵树”的手册，本书不是那本。

五条阅读动线

你是谁	走这条	走完你会有的能力
做算法 / 模型 / 数据	00 → 02 → 26 → 27 → 28 → 29–34 → 附录 B	知道你的模型输出的那串数被谁、以什么代价、在多短时间内执行掉
做硬件 / 嵌入式 / 电力电子	00 → **04–15** → 27 → 35–37 → 附录 C	能独立评估一颗一体化关节的参数表哪些数在骗你
做机械 / 结构 / 传动	02 → 03 → **14** → 15 → 16 → 18 → 22 → 36	能把齿隙（齿隙 = 输入端转了一小段而输出端不动的那段空程，第 14 章）、模态、热三条线连成一个失效解释
想转行进机器人行业	00 → 04 → 05 → 07 → 08 → 10 → 12 → 14 → 40	八章走完就懂一颗关节；40 章是一条十二个月的可验收路线
只想看懂新闻里那些人形	00 → 02 → 16 → 26 → 33 → 38	能分辨“演示”和“连续作业”的差别在哪一格里

最短的一条完整技术链是八章：00 → 04 → 05 → 07 → 08 → 10 → 12 → 14。它从“机器人只有一个旋钮”走到“齿隙决定精度地板”，中间不跳步。

三条阅读纪律（也是本书的写作纪律）

一，每个数字后面那个方括号是真的。

> [厂] 宇树官方页面或官方仓库文件里能逐字读到的
> [拆] 第三方拆解、券商研报、逆向工程 —— 只能当数量级
> [行] 教科书与行业通识 —— 不是宇树事实，别当成“宇树这么做”
> [算] 本书推算 —— 附录 B 有脚本，你可以复算
> [未] 查不到

[行] 永远不能写成 [厂]。这条本书会反复撞上：比如“电流环带宽（电流环带宽 = 驱动器多快能把绕组电流按给定值改过来，第 10 章）约为开关频率的十分之一”是 [行]，而“通信控制频率 6000 Hz”是 [厂]，两者不能被合成一句“宇树的电流环跑 600 Hz”。

[未] 是内容，不是缺陷。宇树全系关节电机不公布额定（持续）扭矩；不公布 PWM 载波频率、死区时间、绕组 L/R、转动惯量、IMU 型号与采样率、电芯化学体系、关节驱动板 MCU 型号。这些空白集中登记在附录 E，而且第 36 章会论证：“厂商不公布额定扭矩”这件事本身，是一条关于这个行业的技术信息。

二，**冲突不抹平**。官方口径与拆解口径不一致时，两条都列，并说明它们描述的对象不同。本书已经撞上四组：

冲突项	口径 A	口径 B	本书处理
G1 膝关节扭矩	页面 90 / EDU 120	官方模型与配置 139	三个并列，指明出处
G1 减速方案	拆解“两级行星 ≈20.58”	官方型号名自带 14.3 / 22.5	两条并列
H1 膝关节扭矩	页面 360	官方 MJCF/URDF 300	并列，说明页面是硬件极限、模型是仿真约束
关节总线	官方“全部电机用 RS-485”	拆解“CAN 菊花链”	并列，官方优先但保留拆解线索

三，**遇到类比要知道它在哪一句开始说谎**。本书每章末尾有一段固定的“类比，以及它在哪儿破”。这不是谦虚，是防火：一个好笑但越界的类比，比一个枯燥但正确的说法危险得多。

本书不做什么

- **不教你写驱动固件**。有伪流程、有公式、有判据，没有可编译的代码。要写代码请去 moteus、ODrive、SimpleFOC、LeRobot——第 40 章给了入口和它们的许可。

- **不做厂商评测**。出现智元、银河通用、Figure、NVIDIA，只为回答“宇树为什么这么选”。
- **不给采购与投资依据**。参数以厂商当前公布值为准，价格与型号在 2026 年下半年变动极快，本书所有抓取日期都写在附录 E。
- **不预测时间表**。“什么时候人形能进家庭”这类问题，本书只回答“卡在哪一格、那一格需要什么”，不回答“几年”。

一个建议：先读第 1 部，再决定要不要读全书

如果你只有三十分钟，读第 04、05、08 三章（约 40 分钟，可以只读每章的第一小节和最后四行）。这三章讲完“电流是唯一旋钮 → 磁场只认垂直分量 → 导通时间如何合成磁场”这条链。

读完如果你觉得“这些我早知道了”，那这本书对你太浅，第 4 部（大脑与数据）和第 5 部（瓶颈）才是你的部分，跳过中间。

读完如果你觉得“原来电机是这么控制的”，请从第 14 章开始接着读——那一章会告诉你，前面这一切精确到 0.0005° 的控制，会被一级齿轮的 0.1° 空程全部作废。**那才是这本书真正想给你的东西：一个精确的机制，和一个它没用的地方**。

一个可检验的说法：本书的深度判据是“往下追问三次'那这个又靠什么'，第三次必须落在一条物理定律或一个不可商量的工程约束上”；每章以四格（stack / mechanism / base / boundary）为闸门，且至少含一个读者可用附录 B 脚本复算的数。

证据等级：本章没有新事实，只有规则。四组冲突项的每个数都可在附录 E 找到出处与抓取日期；“作者被自己的脚本抓到三次算错、第四处由逐行回读官方文件发现”是本书的过程记录，见 `reviews/audit-1.md` 与 `bible/do-not-touch.md` 第 8、10、32 条。

可核对清单：

- ☐ 跑一次 `python code/tools/foc_check.py`，看 selfcheck 是否 10/10，再看那 15 条推算是否与正文一致
- ☐ 随便挑一章，检查它的四格是不是真的填了（如果 base 格里没有式子，就是本书违约）
- ☐ 找一段带 [行] 的句子，确认它没被写成 [厂]
- ☐ 找出上面那张冲突表里你最不同意的一行，写下你的处理办法——本书欢迎这一条

下一章：把地图放下。下一章开始讲这台机器的第一个不可商量的事实：它只有一个旋钮，而那个旋钮不是位置。

第 02 章

宇树的人形产品线，以及为什么这本书不讲机器狗

先把机器摆出来

这本书的主角只有两类：两条腿的，和装在两条腿上的东西。

宇树目前公开销售的人形与类人形产品，按发布与规格排成一条清晰的器件阶梯：

型号	身高	体重	自由度构成	关节峰值扭矩（官方口径）	电池	参考价
H1	约 1.8 m	约 47 kg	腿 5×2 + 臂 4×2 + 腰 1	膝 360 / 髋 220 / 踝 59 / 臂 75 N·m	0.864 kWh，67.2 V	店铺现标 $90,000（注“联系实价”）
H1-2	—	—	腿 12（髋 3膝1踝 2）+ 臂 14（肩3肘1腕3）+ 腰 1	膝 360 / 髋 220 / 腰 220 / 踝 75×2 / 肩肘 120 / 腕 30 N·m	—	未公布
G1	1.32 m	35 kg	腿 6×2 + 腰 1 + 臂 5×2（EDU 可加腰 2、手 7 + 腕 2）	膝 90（EDU 120）N·m	13 串 / 9000 mAh（≈0.43 kWh），54 V 5 A 充约 2 h	¥8.5 万起 / $13,500 起
R1	—	—	腿 12 + 臂 10 + 腰 2 + 头 2	URDF：髋/膝/腰/肩 60、踝 33/50、腕 33 N·m	续航约 1 h	¥9,900 起
H2	1.82 m	70 kg	腿 12 + 臂 14 + 腰 3 + 头 2	臂 120 / 腿 360 N·m；URDF 髋膝 360、腰 180、肩俯仰 130	0.972 kWh，75.6 V	¥19.9 万含税 / $29,900

来源：以上全部取自宇树官方产品页与官方 URDF/MJCF 文件 [厂][算]，抓取日期 2026-10-07；“—”表示官方页面未给该项，本书不填。H1 店铺那个 90,000 美元页面自己标注了需联系取实价，网传“H1 售 9.5 万元”来源不可靠，本书不采。

这张表上有三处值得停一下。

第一，“膝 360 N·m”和“URDF 里写 300”不是同一个数。 官方产品页给的是“关节单元极限扭矩”，而官方 URDF 里的 `effort` 是给仿真用的模型参数，两者经常不一致（H1 页 360/220/59，URDF 髋 200/膝 300/踝 40）。本书在需要选一个的时候选页面值并说明理由，绝不把两个数平均——**它们本来就不是在描述同一件事**，一个是硬件能承受的极限，一个是仿真里允许的约束。

第二，R1 的续航是 1 小时。 这是全表里最重要的一个数，而它几乎不出现在任何宣传里。它也是这张表上唯一一个“只有续航小时、没有电池容量”的数：R1 的容量、电压、串数官方一律未公布 [未]，所以这个 1 小时不能像 G1 那样从官方数反推——第 35 章那张电池表里它单独占一行，那一行是空的，本书不替它填。

第三，这张表里没有“额定扭矩”。 一个都没有。所有型号只给峰值。这不是排版遗漏，这是整个行业的公开程度（第 36 章）。而本章末尾有一笔纯热学的算术，会说明为什么“只给峰值”这件事本身就跟体型放大有关。

我觉得未必是人形。为什么现在做人形？我们只是迎合社会共识。大家现在觉得智能机器人、通用机器人就应该长得跟人一样，这是最有社会影响力的，因为人就长这样。

王兴兴 — 《晚点 LatePost》对话王兴兴（新浪财经转载；晚点首刊 2024-05，采自 2023-10 与 2024-05 两次访谈） — 2025-02-16（首刊 2024-05）（直接引语（问答实录，回答归属明确））



图 02-3　宇树总部（2026-04-25）｜Photo by Windmemories via Wikimedia Commons, CC BY-SA 4.0

为什么这条产品线值得单独写一本书，而不是“机器狗那一章的续集”

上一版这本书的骨架是四足。改掉它的理由不是四足不重要，而是**四足把最难的那件事藏起来了**：Go2 站着不动时，四条腿能把关节力矩降到接近零，重力交给结构骨架扛；G1 站着不动时做不到——它只有两个支撑点，重心投影必须持续落在支撑多边形（支撑多边形 = 所有接触点连成的凸包，重心投影落在里面才不翻，第 16 章）内，任何一个时刻都交不给“死结构”，于是**每一个腿腰关节必须全程闭环，一秒都不许停**。

这条差别会传染到四个设计决定，而四件都是人形自己的账：

- 能量：没有“休息窗口”。官方 Wh ÷ 官方续航小时 ÷ 官方体重得到 G1 约 6.2 W/kg [算]——只有 G1 这一档的三个输入全在页面上；H1、H2 官方未公布续航小时，本书不给它们的 W/kg。这份“省”是省在

没有移动质量的静息损耗上——四足站着可以把腿卸载，人形站着每一秒都在扛重力。但它不是“每一瓦都花在闭环上”：G1 那 216 W 里，光计算就占 10–25 W（第 26 章）。

- 器件与传动：全程出力 ⇒ 发热是持续的 ⇒ 额定扭矩成为真瓶颈（而它恰恰是厂商不公布的那一项）；又要出力又要感知力 ⇒ 减速比不能大，于是有了第 14 章那条“准直驱”路线。
- 控制：两个支点撑不住重心，倒立摆、ZMP 与捕获点从可选变成必需（第 17 章）。

还有一条更硬的理由是宇树自己给的：人形占它营收的比例从 2024 年的 27.68% 升到 2025 年的 51.78%，第一次成了收入主力 [查]（新浪 2026-08-02，数值与链接见第 03 章）。**一家公司一半的收入已经长在两条腿上，这本书却还在讲四条腿，那就是在写上一代产品。**

把四足讲透，你得到的是“一台能动的机器”；把人形讲透，你得到的是“一台必须每一毫秒都在接住自己的机器”。后者的每一个技术选择都是被迫的，而被迫的选择才是有原因的选择。 这本书要的是原因。

一条器件阶梯：23.7 → 34.4 → 120 → 360

现在把上面那张表按“单颗关节能出多少牛米”重排，你会看见一条真正的工程阶梯：

```
GO-M8010-6   23.7 N·m  530 g   1:6.33   24 V    RS-485 4 Mbps   6 kHz
$369
IM6014       34.4 N·m  535 g   3:38     24–75 V RS-485 6 Mbps   10 kHz
双绝对编码器
G1 自研电机  120 N·m（EDU 膝）          两级行星 ≈20.58（拆解口径）
H1/H1-2 M107 360 N·m（膝）               官方“峰值扭矩密度 189 N·m/kg”
```

（前三行 [厂]，20.58 这个速比来自第三方拆解 [拆]。）

这条阶梯不是“越做越大”这么简单。它记录了三个不同的工程妥协，每一个都值得单独一章：

第一档（GO-M8010-6）：6.33 的低减速比、24 V、530 克、369 美元。这一档的意义在于它是**唯一一颗宇树公开了完整电气参数的器件**——包括扭矩常数 0.63895 N·m/A。这本书第 1 部能用一条一条的代数把一颗关节讲透，靠的全是这一颗。它是本书的“果蝇”。

第二档（IM6014）：减速比跳到 12.667，电压范围拉到 24–75 V，加了双绝对值编码器，官方还专门标了“每秒 400 次温度监控”。这一档说的是：**当你要把关节用在机械臂上（R1-7a 的 J3–J5 用的就是它），你开始需要知道温度**——因为你需要长时间精确出力，而 k_t 会随温度漂（第 04 章给了那两个温度系数，第 36 章整章在算它们漂出多少降额）。

第三、四档（G1 的 120、H1/H2 的 360）：官方不再公布任何电气细节，只给扭矩数 [未]。我们能做的只有从官方数反推。第 15 章会做这件事，而它能反推出来的东西比想象的多：

```
官方“峰值扭矩密度 189 N·m/kg” ⇒ H1 膝关节单元 ≈ 360 ÷ 189 = 1.90 kg
拆解口径 G1 主承载关节 ≈ 1.1 kg
⇒ 同一数量级 ✓（差 1.7 倍，方向也合理：H1 的关节更大）
```

平方-立方律：为什么不做一个三米的宇树

先把那个 1.379 除出来

这条阶梯还顺手验证了一个 1638 年就被伽利略写下的定律（`foc_check.py torque_density_chain()` 复算）。但定律要能用，得先把它的输入算清楚——这个 1.379 是从哪来的？它不是查来的，是**两个官方身高相除**得来的。

官方页给的身高是 G1 1.32 m、H2 1.82 m [厂]；同一批页面还给了站立尺寸三边：G1 是 1320 × 450 × 200 mm，H2 是 1820 × 456 × 218 mm [厂]。两条路各自除一遍，得到同一个数：

```
线度 L = 身高比 = 1.82 ÷ 1.32 = 1.3788          （本书取 1.379）
用站立尺寸的第一条边再算一遍 = 1820 ÷ 1320 = 1.3788       （同一个数 ✓）
```

```
L³ = 1.379 × 1.379 × 1.379 = 2.62      （等比放大时，质量的预测）
L⁴ = 2.62 × 1.379 = 3.61               （等比放大时，关节扭矩需求的预测）
```

这里的 2.62 和 3.61 不是查来的，是两次乘法。你可以自己按一遍。

为什么敢拿身高当“线度”？因为等比放大时任何一根骨头都按同一个倍数长，用哪条长度相除都一样。而这条链的下一问正好把这个前提顶到脸上：**宇树等比了吗？**没有——所以下一节要问的不是“定律错不错”，而是“指数到底该由谁来给”。

指数从哪来：扭矩 = 质量 × g × 力臂

为什么腿的扭矩按 L^4 走？追到底只有一句静力学：

```
τ 需求 = 支撑质量 m × g × 力臂 ℓ
```

m 是体力量纲，材料密度不变时 $\propto L^3$。ℓ 是长度量纲，$\propto L$。所以 $\tau \propto L^3 \cdot L = L^4$。注意这条推导的前提是“密度不变、形状不变”，后面你会看到宇树自己就把它打破了。

那为什么行业里也常见 L^3 的写法？因为那是换了一个“什么不变”的问法：身高不动、只把人做胖，质量涨而力臂不涨，扭矩就只按 L^3 走。**指数不是定律给的，是你冻结哪个量给的。**L^3 到 L^4 之间，说的就是“力臂到底跟着不跟着长”。

但“等比”这个前提在宇树自己家就不成立

你可能会问：前提都不成立，结论怎么还对？往下追一层，答案不是巧合。

先看另外两边。宽度 456 ÷ 450 = 1.013，厚度 218 ÷ 200 = 1.09。也就是说 H2 比 G1 高了 38%，腰却几乎一样宽。它不是 G1 的放大复印件，它是“只长腿、再改料”的另一台机器。

官方还把腿长单列了一行：G1 的小腿加大腿是 0.6 m，H2 是 1045 mm [厂]。这两个数才是膝关节真正够得着的力臂尺度：

```
腿长比 = 1045 ÷ 600 = 1.742        （比身高比 1.379 大 26%）
质量比 = 70 ÷ 35 = 2.00            （官方体重）
τ 需求比 = 2.00 × 1.742 = 3.48     （直接回到 τ = m·g·ℓ，不做任何等比假设）
```

那官方器件档位实际涨了几倍？这也是一次除法，本书同样印在正文里：G1 的膝官方给 90 N·m（EDU 版 120），H2 的腿给 360 [厂]——

```
档位上沿 = 360 ÷ 90 = 4.0；下沿 = 360 ÷ 120 = 3.0          （官方档位比 3.0–4.0）
立方律对档位下沿 = 2.62 ÷ 3.0 = 0.87       （只按质量涨，会少算一档扭矩）
```

3.48 落在官方器件档位 3.0–4.0 里面，也贴着等比预测的 $L^4 = 3.61$（差 3.6%）。两条路都到得了，但走的路不同：**真正起作用的恒等式是 $\tau = m{\cdot}g{\cdot}\ell$；$L^4$ 只是“再假设等比放大”这一步的推论**。宇树没等比放大——它把腿拉得更长、把躯干做一样宽——定律照样站在它这边，因为它管的是“质量和力臂都得跟着体型走”，不管你用几根梁、几种材料。反过来，只按 $L^3 = 2.62$ 预测扭矩就短了一截：那等于假设力臂不跟着长，而官方那两条腿恰恰是 H2 更长的东西。

	G1	H2	比值（算式都印在这一栏）
身高（官方）	1.32 m（站立尺寸 1320 mm）	1.82 m（1820 mm）	1.82 ÷ 1.32 = 1.379
宽 / 厚	450 / 200 mm	456 / 218 mm	456 ÷ 450 = 1.013；218 ÷ 200 = 1.09
小腿 + 大腿（官方单列）	0.6 m	1045 mm	1045 ÷ 600 = 1.742
体重	35 kg	70 kg	70 ÷ 35 = 2.00
腿关节峰值扭矩（官方页）	90（EDU 120）N・m	360 N・m	360 ÷ 90 = 4.0；360 ÷ 120 = 3.0
立方律预测（质量，等比假设）	—	—	$L^3 = 2.62$，实测只有 2.00
四次律预测（扭矩，等比假设）	—	—	$L^4 = 3.61$，实测 3.0–4.0

m · g · ℓ 预测（扭矩，不等比）	—	—	2.00 × 1.742 = 3.48

扭矩这一栏落在预测上（3.0–4.0 对 3.61 或 3.48），质量这一栏比预测省（2.00 对 2.62）。 这两件事同时说明：宇树在结构与材料上确实比几何缩放更省质量（否则 35 kg 的 2.62 倍应该是 91.7 kg），但扭矩需求仍然老老实实跟着 m·g·ℓ 走。

定律管指数，不管系数——这条必须写在正文里

把 τ = m·g·ℓ 反解成无量纲系数 c = τ ÷ (m·g·ℓ)，用三台机型的官方膝扭矩与官方腿长。分母先各自算一遍，再除：

```
G1 分母：35 × 9.80665 × 0.600 = 205.9 ⇒ c = 90 ÷ 205.9 = 0.44
G1 EDU 同一分母：c = 120 ÷ 205.9 = 0.58
H2 分母：70 × 9.80665 × 1.045 = 717.4 ⇒ c = 360 ÷ 717.4 = 0.50
H1 分母：47 × 9.80665 × 0.800 = 368.7 ⇒ c = 360 ÷ 368.7 = 0.98
```

（[算]；输入全是 [厂]，H1 的 “Thigh and Calf Length 400 mm × 2” 即 0.800 m。）

系数不是常数，它在 0.44 到 0.98 之间散了两倍多。这才是这条律的诚实读法：**它预测量级和指数，预测不了一家公司愿意为后空翻留多少裕度。** c ≈ 0.5 的读法是“一条腿承担全身的一半、重心水平偏出约一个腿长”；c ≈ 0.98 的读法是“全身重量压在一条腿上、重心偏出接近一整个腿长”——H1 官方页写着 3.3 m/s 的移动速度，它的档位是按跑动配[厂]。这条差别在参数表上读不出来，只能从系数反推出来，而本书把它当作这条律的第一个失效点。

第二把刀：为什么扭矩能跟着体型走，而“持续”跟不上

上面所有数都是**峰值**。这里先补一步推导——为什么 $\tau \propto L^4$？因为单匝线圈在磁场里的扭矩 = 力 × 力臂 ∝ (B·I·导线长) × 半径，而 I = J·导线截面 ∝ $J \cdot L^2$：

```
τ ∝ B · I · (匝面积) · N ∝ B · (J·L²) · L² · N = B·J·N·L⁴      （几何相似
⇒ 匝数 N 不变）
质量 ∝ L³    ⇒    每千克扭矩 τ/m ∝ B·J·L
```

（最后这条会在第 25 章那颗 45 克指关节上再见到：电机做得越小，每千克的扭矩按线度线性缩水。）

于是**峰值扭矩与扭矩需求同为 L^4**——上一节那个系数 c 在 G1 与 H2 上几乎不变（0.44 与 0.50），底层原因就在这里：厂商按需求配档位，而两者同指数。那这条律的刀落在哪？落在**持续**上，靠的是热：铜线发热密度 $\rho \cdot J^2$，绕组体积 $\propto L^3 \Rightarrow$ 总发热 $\propto J^2 \cdot L^3$；能散出去的地方只有外壳，面积 $\propto L^2$。让两者平衡：

```
ρ · J² · L³  =  h · L² · ΔT      ⇒      J_连续 ∝ L^(-1/2)
⇒ 连续扭矩 ∝ J_连续 · L⁴ = L^3.5 ；需求 ∝ L⁴  ⇒ 缺口按 L^0.5 张开
```

```
G1 → H2：1.379^0.5 = 1.17       （持续出力相对需求落后 17%）
G1 → 3 m：2.27^0.5 = 1.51       （落后 51%）
```

这条刀与直觉相反：**放大体型，“瞬间能不能出这个力”几乎不变（同为 L^4），变差的是“这个力能出多久”**。短时过载为什么不受这条约束？因为脉冲里发热被热容吸收：$\Delta T \propto (J^2 L^3 \cdot t)/(L^3 \cdot c_p)$ —— **体积 L^3 约掉了，短时能力与尺寸无关**。这就是“峰值”与“额定”这两个词背后的全部物理。

于是本章前面那条观察——**全系产品页一个额定扭矩都没有**——有了不依赖商业动机的解释：峰值是随体型一起涨的数，额定是随 $\sqrt{L}$ 往下掉的数，写哪个对厂商更有利，答案不言自明（第 36 章会把这条账接到温升与退磁上）。工程能改的只有系数：把壳做大、加风冷、换导热更好的材料、把铜填得更满（G1 与 H2 的官方散热栏都只写着 “Local air cooling”[厂]，两台都没上液冷）。这些都是把 1.17 往回压的手段，代价是重量、成本与功耗，但**没有一条能把 3.5 这个指数抬回 4**。

于是“为什么不做一个三米高的人形”有了算术答案，而且三条路给出同一个窗口：

```
L = 3000 ÷ 1320 = 2.27
质量 ∝ L³ = 2.27³ = 11.7   ⇒ 35 kg × 11.7 ≈ 410 kg
扭矩 ∝ L⁴ = 11.7 × 2.27 = 26.6 ⇒ 90 × 26.6 ≈ 2,400 N·m; 120 × 26.6 ≈ 3,200 N·m
用恒等式再算一遍：0.5 × 410 × 9.80665 × (0.600 × 2.27) ≈ 2,700 N·m
```

2,400–3,200 N·m 的膝盖不是造不出来，但那是工程机械的档位，而它得靠电出这个扭矩；同时这台三米机器的**持续**出力能力相对需求只剩 1/1.51，意思是它每走一步都要在“够用峰值”和“烧掉绕组”之间挑一个。这就是“物理问题”和“工程问题”的界线：**工程问题问的是怎么把系数做好，物理问题问的是系数能好到哪里为止**。把 G1 放大不是一个更大的工程，它是同一个物理按 L^4 收了钱。

顺带一条站在大个子这边的推论。倒立摆的失稳时间常数是 $\tau = \sqrt{(\ell/g)}$（第 16 章用它算控制周期），于是“倒下有多快”这个频率是 $1/\tau = \sqrt{(g/\ell)}$：

```
G1：√(9.80665 ÷ 0.600) = 4.04 s⁻¹  ⇒ 0.64 Hz
H2：√(9.80665 ÷ 1.045) = 3.06 s⁻¹  ⇒ 0.49 Hz
比值 4.04 ÷ 3.06 = 1.32 = √1.742
```

腿长 ×1.742，倒得慢 ×1.32——因为频率按 $1/\sqrt{\ell}$ 走。**体型越大，控制对带宽的要求越低**，这是这条律唯一给大个子发的补贴，也是为什么第 17 章的步态算法在大机器上更容易站住。

这条律解释不了什么

作者要求过：任何“一条规律解释一切”的写法都会被反噬。这条律有三处管不着，本书不猜。

一、材料。H2 官方页把材料写成 “Aircraft-Grade Aluminum + Titanium Alloy + High-Strength Engineering Plastics” [厂]。换成钛合金能把上面那个系数 c 压小一截（许用应力涨了），指数一位没动。应

力随体型怎么涨是另一条账：梁的弯曲应力 $\sigma \propto M \cdot c/I$，等比放大时 $\propto L^4 \cdot L/L^4 = L$——**越大的结构单位面积上扛得越狠**，这是材料而不是几何的事。

二、控制。c 的散差（0.44 到 0.98）几乎全是控制策略与安全裕度的差别，而这两样都不在平方-立方律里。

三、成本与定价。H2 的 ¥19.9 万 ÷ G1 的 ¥8.5 万 = 2.34，而腿的扭矩档位比是 3.0–4.0，质量比是 2.00——三个数互不相同。**这条律对定价毫无发言权**，那是下一章的账。

四足在这本书里只出现两次

明确一下边界，免得读者找错章节：

1. **器件史料**。GO-M8010-6 的页面挂在 Go1 目录下、IM6014 的早期用途在四足上，这是宇树关节的来历。讲一颗关节的进化，绕不开它。
2. **能量与稳定性的对照组**。第 16、20 章需要“四个支点可以零力矩、两个不行”这个对照，才能把双足的难度讲成可算的东西而不是形容词。

除此之外，四足不出现在这本书里。B2、AS2 这些型号只在附录 C 的参数表里作为器件来源存在；Go1 与 Go2 只承担上面那两个角色（器件史料、零力矩对照），不再承担任何算例——本章早先那个四足侧的功率密度对照数就是这么被撤掉的，人形自己的每千克瓦数已经够用。

类比，以及它在哪儿破

类比：同一款发动机，装在四轮皮卡上，和装在一辆只有两个轮子、还必须一直立着的摩托上。难的不是发动机——难的是这台车每时每刻都在倒，而它必须一边倒一边干活。

这个类比在哪里破：摩托车手倒下就倒了，不会有一整套控制律在 1 kHz 上重新决定“该用多大的力去接住自己”。真实的人形不是“努力不倒”，它是**每一毫秒都在精确地让自己往想去的方向倒，然后在来不及之前把脚放到那里**。这个差别是第 17 章的全部内容，也是“机器人走路”和“人走路”最接近的地方。

一个可检验的说法：宇树人形产品线的关节扭矩呈 23.7 → 34.4 → 120 → 360 N·m 的档位阶梯；G1 到 H2 的线度比不是查来的而是官方身高相除（1.82 ÷ 1.32 与站立尺寸 1820 ÷ 1320 是同一个数 1.379），官方腿关节档位比也是两次除法（360 ÷ 90 与 360 ÷ 120，得 4.0 与 3.0）；这个 3.0–4.0 既能被等比假设下的四次律 3.61 接住，也能被不做等比假设的恒等式 m·g·ℓ 直接算出（质量比 2.00 乘官方腿长比 1.742 得 3.48），而只按立方律预测质量去推扭矩会少一档（2.62 对 3.0 是 0.87）；体重实测增长 2.00 倍低于立方律预测的 2.62 倍——因为 H2 根本没等比放大（宽 456 vs 450 mm）。反算出的无量纲系数 τ/(m·g·ℓ) 在 G1/G1 EDU/H2/H1 上是 0.44/0.58/0.50/0.98，散了两倍多，说明这条律管指数不管系数。

证据等级：身高、体重、峰值扭矩、电池、价格、站立尺寸三边、“Calf + Thigh Length” 均为 [厂]（官方页 2026-10-07 抓取）；R1 的电池容量、电压、串数为 [未]，官方未公布，本书不填也不替它反推；“人形占营收 2024 年 27.68% → 2025 年 51.78%”为 [查]（新浪《宇树科技 IPO 在即 420 亿发行估值、60% 毛利率冲刺上市》2026-08-02, https://k.sina.com.cn/article_7879996653_1d5af34ed02001kkd0.html，本书 2026-10-08 回原页读过；这笔账在第 03 章）；URDF 的 effort 值是 [算]（官方仓库文件里可直接读）；“页面值与 URDF 值不一致”是本书观察，两者描述的对象不同，本书不合并；20.58 速比与主承载关节 1.1 kg 是第三方拆解口径 [拆]，**不能当官方事实**；1.379（两条来路各自的商）、2.62 与 3.61（两次连乘）、档位比 4.0 与 3.0（360 的两个商）、0.87（2.62 对 3.0 的商）、1.90 kg 反推、3.48 与四个 c 值、$\tau \propto B{\cdot}J{\cdot}N{\cdot}L^{4}$、J_连续 ∝ L^(-1/2) 与连续扭矩 ∝ L^3.5 那条热约束推导、0.64/0.49 Hz 的失稳频率均为 [算]（输入为上表的 [厂] 数）；“峰值与需求

同指数、额定落后 √L，所以厂商只公布峰值”是本书论证 [算]，不是厂商声明；H1 的 3.3 m/s 为 [厂]，本书只用来解释 c≈0.98 的可能来源，不当作结论。

可核对清单：

- ☐ 到 unitree.com 的 h1 / g1 / r1 / H2 四个页面，逐格抄一遍峰值扭矩，检查是否有任何一格写着“额定 / 持续 / rated”
- ☐ 下载官方 unitree_mjcf 或 unitree_ros 仓库里 H1 的模型文件，找到 `effort` 属性，与产品页的 360/220/59 对照，确认两者确实不同
- ☐ 回到那条 [查] 链接，确认页面上写的是 60.13% 毛利率与人形占比 27.68% → 51.78%，并确认这两个是**份额**而不是客户构成
- ☐ 自己按一遍：1.82 ÷ 1.32 = 1.379；1820 ÷ 1320 = 1.379；1.379 × 1.379 × 1.379 = 2.62；2.62 × 1.379 = 3.61
- ☐ 再算档位与力臂：360 ÷ 90 = 4.0；360 ÷ 120 = 3.0；1045 ÷ 600 = 1.742；2.00 × 1.742 = 3.48；2.62 ÷ 3.0 = 0.87
- ☐ 用 360 ÷ 189 复算 H1 膝关节单元质量，思考为什么它比 G1 拆解的 1.1 kg 大是合理的
- ☐ 分母先各自乘一遍，再各自相除：35 × 9.80665 × 0.600 = 205.9；70 × 9.80665 × 1.045 = 717.4；47 × 9.80665 × 0.800 = 368.7；90 ÷ 205.9 = 0.44；120 ÷ 205.9 = 0.58；360 ÷ 717.4 = 0.50；360 ÷ 368.7 = 0.98
- ☐ 用 √2.27 = 1.51 与 √1.379 = 1.17 复算一次“持续与需求之间的缺口按 √L 张开”，再想想为什么厂商不肯写额定扭矩

下一章：现在我们知道这台机器由 23 个（或 31 个）这样的关节组成，也知道它值 8.5 万。下一章去看这张钱是怎么花的——因为“关节占一半成本”这个说法，是理解为什么这本书要花十章讲一颗关节的真正理由。

第 03 章

一台 G1 的钱花在哪：23 个关节、35 kg、8.5 万元

先把最贵的那一件事说清

宇树 G1，1.32 米，35 公斤，23 个自由度，官方起售价 8.5 万元人民币 / 13,500 美元 [厂]。

现在做一件出版社可能会希望我们别做的事：**猜它值多少成本**。

必须先把话说在前面：宇树从未公布任何一款产品的成本。下面这两个数是第三方估算——中邮证券的拆解口径约 **¥41,574**（对应整机毛利约 40.7%），SemiAnalysis 对进阶版的口径是 **$9,976 成本 vs $27,300 零售**（毛利 67%）[拆]。两个数都不可当事实引用，本书只取它们的**共同结论**：

一台 G1 的器件成本大约四万元人民币量级，售价是它的两倍上下。

四万元要买 23 个能精确出力矩的关节、一套感知、一块大脑、一块电池、一具骨架、以及把它们缝在一起的线缆与结构件。这笔账最有信息量的地方，是它的分配比例。

最核心的是从设计阶段就把成本降下来，比如零部件的数量尽可能少、每个零部件适合开模、模具成本尽可能低、装配时对公差的容忍度要高……

王兴兴 — 上海证券报（同上，来源 A），谈降价靠什么 — 2026-08-19（直接引语）

图 03-1　一台 G1 的钱花在哪：23 个关节 × 369 美元与整机售价的比对

先把这两个第三方数各自验一遍

拿自己的除法查一遍它们，比引用它们有用得多。

中邮证券给了两个数：BOM 约 41,574 元，毛利约 40.7%。这两个数自己就能反解出它心里那台机器的售价：

```
1 − 0.407 = 0.593            （中邮口径自己给的毛利率 40.7%）
售价 = 41,574 ÷ 0.593 = 70,108 元
```

可官方起售价是 85,000 元。也就是说那份 40.7% 不是对着 8.5 万算的，是对着约 7.0 万算的。差的 1.5 万是什么——含税、渠道、还是更早的定价——本书查不到，写 [未]，也不拿它去修正谁。

SemiAnalysis 那两个数也能自己除：

```
9,976 ÷ 27,300 = 0.3654
毛利率 = 1 − 0.3654 = 63.4%       （报道口径写的是 67%）
```

差 3.6 个百分点。而它引用的那个 27,300 美元，在宇树商城里也没有对应格子：商城在架的是 G1 13,500 美元与 G1 Pro 21,500 美元，且官方注明 “Tax and Shipping cost excluded” [厂]。**三条口径（8.5 万 / 约 7.0 万 / 27,300 美元）本书全部并列，不取平均、不择一。**

这不是说拆解报告不可信。恰恰相反——**能被读者自己拆出内部不一致的估算才是可以用的估算**，因为你能核对它。但是要把它的边界写死：这些数是拆解与券商口径，**宇树从未公布过任何一款产品的成本**，所以下面所有除法只在“数量级”这一层成立；把某一份 BOM 当成厂商口径去做采购或竞品分析，是做不到、也不该做的事。

关节模组的单价：一条能自己走完的链

行业通行的结构是**执行器占整机 BOM 的 40–60%** [行]。把这条区间分别按到两个第三方总数上：

```
中邮口径关节那一档下限 = 41,574 × 0.40 = 16,630 元；上限 = 41,574 × 0.60 = 24,944 元
再除以颗数：16,630 ÷ 23 = 723 元/颗；24,944 ÷ 23 = 1,085 元/颗
SemiAnalysis 口径下限 = 9,976 × 0.40 = 3,990 美元；上限 = 9,976 × 0.60 = 5,986 美元
再除以颗数：3,990 ÷ 23 = 173 美元/颗；5,986 ÷ 23 = 260 美元/颗
```

两条链不同币种，本书**不做汇率换算**，让它们各自成立。

但“颗”这个单位在这里是靠不住的：一个 5 N·m 的腕关节和一个 139 N·m 的膝关节根本不是一件东西。官方 URDF `g1_23dof_rev_1_0.urdf` 把 23 颗关节的 `effort` 分成四组：139 N·m 四颗（两个髋侧摆 + 两个膝）、88 五颗、35 四颗（踝）、25 十颗（肩肘腕滚）[厂]：

```
扭矩总量 = 139×4 + 88×5 + 35×4 + 25×10 = 1,386 N·m      （四组分别是 556、440、140、250）
平均 = 1,386 ÷ 23 = 60.3 N·m/颗
每牛米单价下限 = 723 ÷ 60.3 = 12.0 元/(N·m)；上限 = 1,085 ÷ 60.3 = 18.0 元/(N·m)
美元口径下限 = 173 ÷ 60.3 = 2.87 美元/(N·m)，本书取 2.9；上限 = 260 ÷ 60.3 = 4.31，取 4.3
```

现在可以拿宇树唯一挂过价的那颗关节来对表。GO-M8010-6：369 美元、23.7 N·m [厂]：

```
宇树零售单价 = 369 ÷ 23.7 = 15.6 美元/(N·m)
倍数 = 15.6 ÷ 4.3 = 3.6 倍；15.6 ÷ 2.9 = 5.4 倍
```

宇树自己单卖一颗关节的“每牛米美元价”，是拆解口径下 G1 内部关节价的 3.6–5.4 倍。这个差有两个来源，而手头这些数分不开它们：一是零售价含渠道与毛利、拆解口径是器件成本；二是大关节天生每牛米更便宜——同样的壳、同样的驱动板、同样的轴承，磁钢和齿轮的体积却在涨，而扭矩涨得比体积快（第 02 章那条 $\tau \propto L^4$ 与质量 $\propto L^3$ 的差，落到钱上就是每牛米的单价随尺寸下降）。**本书不挑一个说法，只把除法摆在这里**。

减一减，看这条链闭不闭合

拆解口径里少见的两行是感知与算力的具体金额：感知阵列 < $600、算力模块 ≈ $750 [拆]。把它们从 SemiAnalysis 的总成本里减掉：

```
关节占成本 = 9,976 − (600 + 750) = 8,626 美元      （除感知、算力之外的一切）
关节占比下限 = 3,990 ÷ 8,626 = 46%；上限 = 5,986 ÷ 8,626 = 69%
剩下的下限 = 8,626 − 5,986 = 2,640 美元；上限 = 8,626 − 3,990 = 4,636 美元
```

那 2,640–4,636 美元要买：13 串 9000 mAh 电池（官方规格 [厂]，换算约 0.43 kWh）、铸铝与机加结构、全套走线、装配工时，以及头颈那两块显示。**这就是这条链闭合的样子**：减掉已知的两块，关节加电池加结构刚好填满剩下的格子，塞不下第二个大件。“关节占一半”最硬的证据不是那个 40–60% 的引用，而是这个减法之后没有别人的位置。

数量是乘数：一颗关节的钱怎么变成一整台机器的钱

宇树公开定价的唯一一颗一体化关节 GO-M8010-6 单卖 **369 美元** [厂]。裸乘一次：

```
裸乘零售价 = 23 × 369 = 8,487 美元 ÷ 13,500 = 62.9%      （全是零售价，含毛利）
```

```
假设自产摊薄一半 = 23 × 369 ÷ 2 = 4,244 美元                    （本书的假设，不是数据）
```

上一版本书把这条乘积叫“上界”，这里更正：369 美元那颗既是最老的型号、又是**最小的档位**（23.7 N·m 对 G1 的 60.3 N·m 平均），含毛利往上抬、含档位往下压，两个方向同时成立，所以它不是界，只是一个粗略的乘积。

但这个近似在另一件事上是硬的，而且不依赖任何假设：

```
关节单价每降 100 元 ⇒ 整机降幅 = 100 × 23 = 2,300 元
关节每颗减重 50 g ⇒ 整机减重 = 50 × 23 = 1,150 g = 1.15 kg（占 35 kg 的 3.3%）
每颗扭矩多 1 N·m ⇒ 整机多 = 1 × 23 = 23 N·m（膝 90 → 113 N·m，+26%）
```

数量是乘数。这就是为什么“把一颗关节做到 530 克、369 美元”这件事，比“把一台机器人做得更聪明”更能决定一家公司的毛利。

那为什么一颗关节要 723–1,085 元？往下追一层：因为它不是一台电机，它是**四合一**——无框永磁电机 + 减速器 + 两套编码器 + FOC 驱动板，装进一个要密封、要让线缆从中心孔穿过、还要撑住交叉滚子轴承的壳里（官方对关节的措辞就是这四件；轴承与 “Full Joint Hollow Electrical Routing” 写在 G1 页 [厂]）。

那这四件合在一起又靠什么？靠三道彼此不共享供应链的精密工序——绕线与磁钢装配、齿轮加工与热处理、编码器对零与**标定**——再加上缝进一个壳之后的逐颗跑合测试。齿轮厂做不了绕组，驱动器厂做不了齿形，良品率与工时这两样都长在一家公司自己的产线上，而产线买不来。**这才是“关节能自己做、手做不了”的技术内容**，第 25 章会在一只 45 克的指关节上再遇到它一次。

这条“关节占一半”什么时候失效？两种场合。一是把灵巧手算进来：手也是执行器，G1 EDU 那 7 + 2 个手部自由度若按外购价计，执行器占比就不在 40–60% 这个带里，而是往上顶（下一节 14,215 元/只就是证

据)。二是换一台机器：H2 有 31 颗关节、R1 官方两处口径给 20 与 26（本书并列），颗数一变乘数就变，而占比未必变。**所以这条规律的内容是“单件 × 数量”，不是“一半”这个数字本身**。

第 1 部花十章讲一颗关节，理由就在这里：**那颗东西是这台机器的成本中心、重量中心、可靠性中心，也是所有上层算法的物理边界**。

那“便宜”到底是怎么来的

8.5 万元一台双足人形，在 2023 年这个价买不到一条腿。便宜不是单点魔法，是三类器件在同一个时间窗内同时降价的乘积。

一，关节电机。行业报道的口径是：进口同规格力矩电机曾要 5–6 万元，国产同类降到 500–600 元量级 [推]。宇树的路子更彻底——**自研自产**，并且把 GO-M8010-6 这类关节当元器件单卖（369 美元，官方店铺在架 [厂]）。当一家整机厂同时是器件供应商，它的关节成本就是自己的出厂价，不是别人的零售价。

二，减速器。谐波减速器曾长期被日本哈默纳科主导，报道口径是进口 2000–3000 元、国产 1500–2000 元 [推]。**但这里必须给本书最重要的一个诚实声明**：在宇树自己的产品上，“用了国产谐波减速器”这句话**证据不足**。官方明确说“谐波减速器、减速比 60+、背隙 6 弧分”的只有 Z1 机械臂 [厂]；G1 的减速方案来自第三方拆解，口径是**两级行星、总速比约 20.58** [拆]，与官方 `unitree_rl_lab` 配置里型号名自带的 14.3 / 22.5 不一致（本书两条都列，见第 14 章）。**“宇树靠国产谐波降本”是一个流传很广但没有一手支撑的说法，本书不采用**。

三，感知。这一条宇树公布得最清楚，因为它就是它的产品：

器件	官方/厂商价	说明
自研 4D 激光雷达 L1	**¥999 起 / $249**	21,600 点/s；内置 3 轴加速度 + 3 轴陀螺，**官方未给 IMU 采样率** [厂][未]

自研 4D 激光雷达 L2	**¥1,799 / $419**	64,000 点/s；内置 6 轴 IMU，采样率同样未公布 [厂][未]（第 24 章）
Livox Mid-360	¥3,999（旧链接价，原页已 404，待复验）	拆解口径 G1 用这一颗 [拆]
RealSense D435i	$334	拆解口径 G1 深度相机 [拆]
国产六维力传感器	约 ¥5,000/台 量级	行业报道 [推]
触觉阵列（帕西尼）	¥199 起	行业报道，单价从十万级降下来 [推]
因时 RH56DFX 灵巧手	¥22,200	6 DoF / 12 关节 / RS-485 [查]（**因时自己的挂牌价，不是宇树官方**：本书对第三方厂商的一手页面一律记 [查]，与第 25 章同一口径）

（上一版这张表里 L2 写的是“内置 1 kHz IMU”。官方页其实只有 “3-axis acceleration + 3-axis gyroscope” 这一行，没有 Hz——已改按 [未]，详见第 24 章。）

L1 那颗 ¥999 的雷达是关键证据：**宇树把感知也做成了自己的元器件**。而灵巧手是另一回事——2025 年前三季度宇树向外采买了 1,210 只因时的手、金额 1,720 万元，占其外采的 96% [推]。**手，是它目前唯一没能自研摊薄的大件**。 除一下才知道“最贵”到什么程度：

```
一只手 = 1,720 万元 ÷ 1,210 只 = 1.4215 万元/只          （采购均价；输入是 [推] 口径的两个数）
换算成元：1.4215 万元 就是 14,215 元/只
折成关节 = 14,215 ÷ 904 = 15.7 颗       （904 是关节单价区间 723–1,085 的中点）
一整条腿 = 12 颗 × 904 = 10,848 元，比一只手便宜       （12 颗加起来还不到一只手的钱）
```

区间中点只是本书为了给你一条能核对的算式，不代表厂商成本。但结论对这个中点不敏感：把 723 与 1,085 两个端点各代一遍，“一只手折 13 到 20 颗关节”、“一只手贵过一条腿”这两句话都成立。两只手装上去，

14,215 × 2 ÷ 85,000 = 33.4%，光外采的手就吃掉售价的三分之一——这就是为什么在官方配置表里手一直是选配 [厂]。

拆开看：一台 G1 的五个成本块

把上面这些拼成一张结构图（比例取行业通行区间 + 拆解口径，**全部标注为估算**）：

```
① 23 个关节模组（电机+减速器+编码器+驱动板+轴承+密封）    ~50%   [行][拆]
② 感知（雷达 + 深度相机 + IMU + 触觉）                    <$600  [拆]
③ 算力（Orin NX 16GB 模组 ¥5,499 量级）                   ≈$750  [拆]
④ 能源（13 串 9000 mAh + BMS + 充放电保护）               —      [未]
⑤ 结构（铸铝/机加/工程塑料 + 线缆 + 装配）                —      [未]
```

②③ 那两行是拆解报告里少见的给出了具体金额的项 [拆]。把它们和 8.5 万的售价放在一起看，会得到一个反直觉的结论：

```
感知加算力 = 600 + 750 = 1,350 美元
占整机售价 = 1,350 ÷ 13,500 = 10.0%
占器件成本 = 1,350 ÷ 9,976 = 13.5%
```

感知 + 算力加起来约等于整机售价的 10%。

也就是说，**这台机器“聪明的部分”很便宜，“会动的部分”很贵。**一个想把人形做便宜的公司，必须去啃关节；一个想把人形做聪明的公司，必须先有人把关节做便宜。这两句话描述了当下行业里两种完全不同的商业策略（第 39 章会展开）。

毛利那条线：60.13% 说的是哪一台机器

公开信息里有一组对理解整本书都很重要的数：宇树 2025 年营业收入 **17 亿元**、综合毛利率 **60.13%**、人形机器人占营收的比例从 2024 年的 **27.68%** 升到 2025 年的 **51.78%** [查]（《宇树科技 IPO 在即 420 亿发行估值、60% 毛利率冲刺上市》，新浪，2026-08-02，https://k.sina.com.cn/article_7879996653_1d5af34ed02001kkd0.ht-

ml；本书 2026-10-08 回原页逐段读过）。同一条报道给的另两个毛利率是 2023 年 44.22% 与 2024 年 56.74% [查]（同上）——**毛利率是三年一年一台阶涨上来的，不是天生如此**。

先把 51.78% 这个数的边界划清，因为它经常被混着用：**51.78% 是份额**，说的是人形这一类产品在宇树全部收入里占了多少；**它不是客户构成**——“人形那部分收入里，科研教育这样的客户占多少”是另一个量、另一张表，本书没有把它核到一手，所以正文不写那个数，只提醒两个量不许互相替换。

一条被删掉的除法：两个“超过”为什么不能相除

上一版这一节还写了第三个数——“扣非净利超过 6 亿元”——并拿它做了一次减法（毛利率 60.13% 减净利率 35.3%，得 24.8 个百分点）。这一版**两句都删了**。两条理由，第二条比第一条重要。

一、那句“扣非净利超过 6 亿元”没核到原文。 本书 2026-10-08 回到上面那条 [查] 链接的原页读过：报道写 2025 年利润时用的不是这个说法，全文也搜不到“扣非净利超过 6 亿”这句话；交易所文件与招股书原文本书没有读到，所以它连一个能接上的二手出处都没有。**查不到出处的数不进正文，由它派生的数一起删。** 本书不许的做法是把这个数留在原地、前面加一个“约”字蒙过去。

二、就算那句成立，那个除法也不成立。 为什么不成立？因为“营收超过 17 亿”与“扣非净利超过 6 亿”都是**下界**：各自只说“至少多少”，没有一个说“是多少”。两个下界相除，得到的既不是净利率，也不是净利率的界：

假设的真实值	净利	营收	相除会得到
上一版代进去的那一对	6 亿	17 亿	35.3%
营收再大一截	6 亿	60 亿	10.0%
净利大一截	60 亿	17 亿	353%

三行用的都是同一对“超过”，结果从 10% 跳到 353%。所以 35.3% 不是估算，是**把不等式当成了等式**；再拿它去减 60.13%，是在一个已经错的数上做第二步——那个“24.8 个百分点”从来没有存在过。

这才是本章真正要教的那件事：**看到一个数，先问它是点值还是界**。产品页上的“峰值扭矩”是界（本书第 02 章整章都在用这个界，并且拒绝把它当工作点），“营收超过 17 亿”也是界；本书正文里的除法，只有分子与分母都是点值时才许做。毛利率 60.13% 与营收 17 亿元是报道里写死的两个点值 [查]，本书就只做点值能做的事——把它们和拆解口径的毛利率并排摆一摆；净利那一格本书没读到可以信的写法，所以那个减法到此为止，不做。

四条毛利率：它们不在同一件事上，本书不取平均

那 60.13% 能不能拿来算 G1 的器件成本？不能。把本章用过的几个毛利率并排摆开，就能看清为什么不能——这也是“冲突不抹平”这条纪律的示范：

口径	数	说的是什么
公司综合毛利率 60.13% [查]	2025 年，全产品线	四足 + 人形 + 组件 + 雷达加在一起的毛利
中邮拆解口径 40.7% [拆]	单款 G1，隐含售价约 7.0 万	一台人形的器件 BOM 对着它自己那档价
SemiAnalysis 口径 63.4%（报道写 67%）[拆]	进阶版 G1，27,300 美元	另一款配置、另一个价
官方起售价 85,000 元 [厂]	基础版 G1	只有价，没有成本

四个数并列，不合并、不平均、不“以最新为准”。它们的分母根本不是同一个东西：60.13% 是整家公司所有产品的毛利率，40.7% 与 63.4% 是一款人形的器件成本除法。一个人如果拿 60.13% 去推 G1 的器件成本，他其实是在假设这台人形的毛利等于公司平均毛利——而 2025 年人形才刚

占到营收的一半（51.78% [查]），剩下那将近一半是四足、关节组件与雷达，它们的毛利结构跟一台双足不是一回事 [推]。**这一句是本章能给的最实用的提醒：财务数与器件数之间没有免费的换算**。

再看那笔募资的分配：**42 亿里有 20 亿投给模型，只有 11 亿投本体、6 亿投产能**（报道口径：科创板挂牌，发行价 150.80 元，募资约 42 亿元，其中 20.22 亿投向具身模型、11.1 亿投本体、6.24 亿投制造基地，规划年产 7.5 万台双足 + 11.5 万台轮式）[推]：

```
20.22 + 11.1 + 6.24 = 37.56 亿 ÷ 42 亿 = 89.4%
⇒ 报道口径里还有约 4.44 亿没有被这三项覆盖，本书不猜它是什么
```

（挂牌日期、发行价、这三个项目的分配比例与产能规划数，本书只读到二手转述，一律按 [推] 处理，与上面那几个 [查] 数不同级。）

一家靠本体毛利活着的公司，把一半的钱押在了大脑上。这个动作本身就是第 4 部存在的理由——它说明在宇树自己的判断里，**躯体的问题已经阶段性解决，数据与模型的问题还没有**。

而本书第 1 部讲的那 23 个关节，正是那 11 亿要继续优化的东西。

那颗东西贵在哪一格：关节成本往下压一分钱，整机毛利涨多少

把“数量是乘数”再往下追一层就能得到一句话：**整机毛利是关节成本的杠杆，杠杆倍数约等于关节在 BOM 里的占比**。用两个第三方口径各算一遍“关节成本降 10%”（本书取 50% 这个区间中点，只为给一条可核的算式）：

```
中邮口径，关节那一档 = 41,574 × 0.5 = 20,787 元；降一成的量 = 20,787 × 0.10 = 2,079 元
新器件成本 = 41,574 − 2,079 = 39,495 元
对官方起售价，老毛利 = 1 − 41,574 ÷ 85,000 = 51.1%；新毛利 = 1 − 39,495 ÷ 85,000 = 53.5%   ⇒ +2.4 个百分点
对中邮自己的隐含售价，老毛利 = 1 − 41,574 ÷ 70,108 = 40.7%；新毛利 = 1 − 39,495 ÷ 70,108 = 43.7%   ⇒ +3.0
```

```
SemiAnalysis 口径，关节那一档 = 9,976 × 0.5 = 4,988 美元；降一成 = 4,988 × 0.10 = 499 美元；新成本 = 9,976 − 499 = 9,477 美元
对报道里那个零售价，老毛利 = 1 − 9,976 ÷ 27,300 = 63.4%；新毛利 = 1 − 9,477 ÷ 27,300 = 65.3%   ⇒ +1.9
```

一颗关节降一成，整机器件成本降 5%，毛利率涨 1.9 到 3.0 个百分点。反过来读更实用：**任何“把整机做便宜 10%”的方案，如果不能落到那 23 颗东西上，它就必然去吃毛利率**。

产能那一格：18 秒一颗关节

规划年产后，“关节是瓶颈”从形容词变成一个数：

```
双足关节年需求 = 7.5 × 23 = 172.5 万颗/年        （轮式那 11.5 万台不在本章这条链里）
一年的秒数 = 365 × 24 × 3600 = 31,536,000 秒
节拍 = 31,536,000 ÷ 1,725,000 = 18.3 秒/颗          （日产量 = 1,725,000 ÷ 365 = 4,726 颗/天）
整机节拍 = 18.3 × 23 = 421 秒 ⇒ 每 7 分钟下线一台
```

（[算]，输入为 [推] 的产能规划数与 [厂] 的自由度。）

18.3 秒里要装完绕线、磁钢、齿轮、轴承、两套编码器，还要跑完对零、标定和逐颗测试——这还只是节拍，不是良率。今天实际是几秒一颗、良率多少，官方未公布 [未]。但这条算式的价值不在于它答了什么，而在于它给了你一个**可以拿去问厂商的具体问题**；一家规划年产 7.5 万台 × 23 颗 = 172.5 万颗精密一体化关节的公司，它的标定工位在哪里。

类比，以及它在哪儿破

类比：一台人形机器人的成本结构像一间餐厅——食材（关节）占一半，装修（结构）与餐具（感知）都不贵，而“主厨的创意”（模型）几乎不要钱，因为它是别人开源的。

这个类比在哪里破：餐厅的食材成本随营业额线性涨，而关节成本**不随销量线性降**——它受限于铜、稀土磁钢、精密齿轮的加工产能，而那三样的产能本节上面只算到了节拍、没算到良率。所以“卖得多就更便宜”这条在软件上成立、在关节上会撞上一堵物理墙；墙的另一面是热，第 36 章整章在讲它把“持续出力”压回多少。

一个可检验的说法：把行业区间 40–60% 分别按到两个第三方 BOM 总数上、再除以官方 23 个自由度，得到关节单价 723–1,085 元（中邮口径）或 $173–260（SemiAnalysis 口径）；用官方 URDF 的 `effort` 分组算出 23 颗关节的上限相加是 1,386 N·m、平均 60.3 N·m/颗（**这是账面合计，不是这台机器能出的总扭矩**——各关节不能同时同向出力；本书加它只为了下一步的除法有官方输入），于是宇树自己给那颗 23.7 N·m 关节挂的 369 美元相当于 15.6 美元/(N·m)，是拆解口径内部价的 3.6–5.4 倍；而感知 + 算力两块（<$600 + ≈$750）只有整机售价的 10.0%；9,976 − 1,350 = 8,626 美元，把这两块减掉之后剩下的这 8,626 美元刚好被“关节 + 电池 + 结构 + 装配”填满——这条链闭合，塞不下第二个大件。**所有这些数的分母都不是宇树公布的：宇树从未公布任何一款产品的成本，本书两处冲突口径全部并列、不取平均**。公司层面那条线同理：2025 年综合毛利率 60.13%、人形占营收 51.78% 是 [查] 到的点值，而单款 G1 的 40.7% 与 63.4% 是 [拆] 出来的除法，两组数说的是两件事；上一版那个“6 ÷ 17 得净利率 35.3%、再拿 60.13% 去减”的写法已经删掉，删的理由作为反面教材留在了正文里。

证据等级：369 美元、¥999/¥1,799、23 与 29 自由度、1.32 m/35 kg/¥8.5 万/$13,500、Z1 的谐波 60+/6 弧分、URDF `g1_23dof_rev_1_0.urdf` 的 effort 分组、商城 G1 13,500 与 G1 Pro 21,500 美元、“Tax and Shipping cost excluded”、13 串 9000 mAh，全部 [厂]；**2025 年营业收入 17 亿元、综合毛利率 60.13%、2023 与 2024 年毛利率 44.22% 与 56.74%、人形占营收 27.68%（2024）→ 51.78%（2025）**，全部 [查]（《宇树科技 IPO 在即 420 亿发行估值、60% 毛利率冲刺上市》，新浪，2026-08-02，

https://k.sina.com.cn/article_7879996653_1d5af34ed02001kkd0.html；本书 2026-10-08 回原页逐段读过；这几个是**份额与全产品线口径**，不是“人形收入里客户构成”那一档数）；¥41,574、$9,976、40.7%、67%、<$600、≈$750、27,300、Mid-360 与 D435i 的型号与价格、因时 1,210 只/1,720 万元/96%，全部是**第三方拆解与券商口径 [拆]**；发行价 150.80 元、挂牌日、募资 42 亿元与 20.22/11.1/6.24 亿的项目分配、年产 7.5 万台双足 + 11.5 万台轮式，是**财经媒体与作者笔记转述 [推]**，本书没读到交易所原文；“扣非净利超过 6 亿元”**原样删除**——本书回那条 [查] 链接的原页搜不到这句话，也没有读到交易所或招股书原文，由它派生的净利率 35.3% 与“24.8 个百分点”那次减法一并删除，正文只留它作为反面教材；40–60% 是行业通识 [行]；0.593、70,108、0.3654、63.4%、1,386 N·m、60.3、12–18 元/(N·m)、2.87 与 4.31 美元/(N·m)、15.6、3.6–5.4 倍、1,350、10.0%、13.5%、8,626、46% 与 69%、2,640 与 4,636、1.4215 万元、14,215 元/只、15.7、10,848、33.4%、20,787、2,079、39,495、51.1% 与 53.5%、43.7%、4,988、499、9,477、65.3%、89.4%、4.44 亿、18.3 秒/颗、421 秒/台，全部是 [算]（输入为上列 [厂][拆][推][查] 数，算式写在正文里）；“自产成本约为零售一半”与“关节占 50%（区间中点）”是本书假设 [算]，**必须当作量级而非数据**；L1/L2 的 IMU 采样率上一版误标 1 kHz，现按官方口径改回 [未]；Mid-360 的 ¥3,999 来自已 404 的旧链接，标为待复验。

可核对清单：

- ☐ 自己乘一次 23 × 369，再除以 13,500，看那个 63%——然后想想为什么本书说它不是“上界”
- ☐ 回到那条 [查] 链接，核对 17 亿元、60.13%、27.68% 与 51.78% 这四个数在页面上，并核对“扣非净利超过 6 亿元”这句**不在**页面上
- ☐ 自己拿两个“超过”做一次除法，把营收分别按 17 亿、60 亿代一遍，看同一个式子能从 35.3% 跳到哪里去
- ☐ 用 1 − 0.407 = 0.593；41,574 ÷ 0.593 = 70,108 复算中邮口径隐含的整机价，与官方 85,000 元起售价对照，确认差约 1.5 万

- ☐ 用 9,976 ÷ 27,300 = 0.3654；1 − 0.3654 = 63.4% 复算那个 63.4%，与报道的 67% 比对；再查商城里有没有 27,300 这一格（预期：只有 13,500 与 21,500）
- ☐ 按 139×4 + 88×5 + 35×4 + 25×10 = 1,386 复算官方 URDF 的扭矩总量；再用 1,386 ÷ 23 = 60.3 复算平均（文件是 `unitree_ros/robots/g1_description/g1_23dof_rev_1_0.urdf`）
- ☐ 用 723 ÷ 60.3 = 12；1,085 ÷ 60.3 = 18 复算每牛米单价；再用 369 ÷ 23.7 = 15.6 美元/(N·m) 除出那 3.6–5.4 倍，并自己判断它更像“零售毛利”还是“尺寸经济性”
- ☐ 用 1,720 ÷ 1,210 = 1.4215 万元复算一只手的采购均价；再除 904 看它等于多少颗自产关节
- ☐ 用 9,976 − 1,350 = 8,626；8,626 − 5,986 = 2,640 复算“减掉感知与算力之后还剩多少格子”
- ☐ 在宇树店铺确认 GO-M8010-6 是否仍在架、价格是否仍是 369 美元
- ☐ 在宇树全系产品页搜索“减速器 / reducer / harmonic / 谐波”，确认除 Z1 之外是否还有任何一处官方说明（预期：没有）
- ☐ 把“宇树用国产谐波降本”这句话拿去问三个来源，检查你能不能找到一手证据（预期：找不到）

下一章：现在你知道这台机器一半的钱在关节里，也知道了那颗关节的官方价。下一章（第 04 章）开始拆它——第一站是一个看起来太简单、但整个控制栈都建立在它上面的事实：**机器人只有一个抓手，就是电流**。

第 04 章

机器人只有一个抓手：电流

你以为你在控制位置，其实你在控制电流

设想你让一台 G1 把右臂抬到肩平。

上层算法给出的确实是一个位置：某个关节角，比如 1.57 弧度。但如果你顺着这条指令往下追，追到最后一米，会发现“1.57 弧度”这个数从来没有直接驱动过任何东西。真正驱动那个关节的，是一串安培数——绕组里流过多大的电流、在什么时刻、朝哪个方向。

这就是这台机器最重要的一条事实，也是第 1 部十章的全部理由：

机器人没有任何“位置旋钮”、没有“速度旋钮”、没有“力量旋钮”。它只有一个旋钮：绕组里的电流。所谓位置、速度、力矩，都是同一个旋钮在不同时间尺度上的别名。

这句话如果听起来像修辞，那这一章就没白读。它其实是一个可以验证的断言，而且验证它只需要一条式子和一次乘法。

其实某种意义上电机的工作原理都很类似，就跟汽车里的电机，你家的电风扇电机，包括手机上有些震动马达电机的工作原理都一样，只是说这个电机可能结构更复杂、扭矩更大、尺寸更大，再加上减速器就构成了整个关节，其实可能没有大家想得那么复杂。

王兴兴 — 凤凰卫视《X+》专访（主持人田桐），腾讯新闻转载 — 2025-02-21（直接引语（问答按轮次归属））

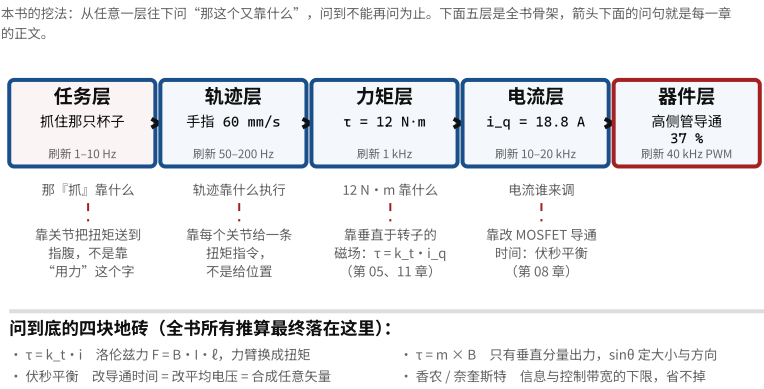


图 04-1　全书的深度尺：五层诉求，每一层的问句就是下一章的正文

唯一的一条式子，和它为什么不是随便写的

一根通电导线放在磁场里会受力。单位长度上的力，大小是磁感应强度乘以电流，方向垂直于两者构成的面。把它绕成一个线圈、放进半径 r 的气隙里，对转轴积分一次，就得到：

```
τ = k_t · i
```

k_t 是这台电机的扭矩常数，它把所有你不想在每次控制时都重算的几何细节——匝数、磁钢剩磁、气隙面积、绕组怎么接——打包成一个系数。它的单位是 N·m/A，也就是“每一安培给多少牛米”。

这个打包不是免费的简化，它成立的前提很硬：**磁场必须已经和转子处于正确的几何关系上**。这个前提怎么维持，是第 05 到第 10 章的全部内容。现在先接受它，然后立刻用宇树公布的一颗真实电机去检查这条式子。

GO-M8010-6 的官方参数表 [厂]：峰值扭矩 23.7 N·m，峰值电流 40 A（**口径**：官方没写这是相电流幅值还是母线电流；本书一律按**相电流幅值**用，见附录 A。若它其实是母线电流，换算到相电流要乘约 √3/2，扭矩

常数折算会跟着变 13%——这是一处会往下游传的口径风险），扭矩常数 0.63895 N·m/A，减速比 1:6.33，整机 530 克，24 V 供电，最高转速 30 rad/s（输出端），售价 369 美元。

做一次乘法：

```
0.63895 N·m/A × 40 A = 25.56 N·m
官方峰值 = 23.7 N·m
差 +7.8%
```

结论：手册那个 0.63895 是“输出端”口径的常数（已经乘过减速比了），不是转子本体的。折算回去：

```
k_t(转子) = 0.63895 ÷ 6.33 = 0.10094 N·m/A
```

这一步看着像算术练习，其实是全书反复要用的一个动作：**厂商参数表上的数，几乎都混着“电机端”和“输出端”两套口径，而这两套之间永远差一个减速比**。把 23.7 N·m 当成转子能给的扭矩，你会以为需要 37 安培；把 0.10094 当成输出常数，你会以为 40 安培只能给 4 牛米。两种错法都会让你在设计关节时选错一档电机。

（脚本 `foc_check.py kt_crosscheck()` 复算这条；7.8% 的偏差本身也值得注意——它说明 23.7 这个数不是 40 A 的满额换算值，而是被某个未公布的限制削过一刀。削它的候选里，“母线电压”要先划掉：峰值扭矩是**零速**工况，反电势 (2/3)·k_t·ω 在 ω=0 时是零，母线电压在这一格根本不参与限制。剩下三个才成立：磁饱和让 k_t 掉下常数（第 05 章那条线性假设的边界）、40 A 是比 23.7 N·m 更短的脉冲口径、以及 40 A 本身可能是母线电流而非相电流幅值（口径风险见上面），官方没说 [未]。）

为什么“只有电流”这件事对工程师是个坏消息，也是好消息

坏消息先说。如果电流是唯一的旋钮，那么**你所有的力，最终都要经过铜线和磁铁这两样东西**，而它们各有各的不可靠：铜的电阻随温度涨，磁铁的磁性随温度掉。这两条在第 36 章会变成一整章的“额定扭矩为什么厂

商不敢给”。

好消息更重要，而且它直接决定了人形机器人的技术路线：

既然扭矩就是电流，那么“测扭矩”就不需要扭矩传感器——数一数电流就够了。

这句话的分量要到第 14 章才看得清，先把结论摆在这里：宇树 G1 的关节里，官方口径是“电机 + 减速器 + 编码器 + 驱动板”四合一，没有任何一份官方材料提到关节内装了力矩传感器 [厂][未]。而同一时期，智元公开的精灵 G2 规格里写着“全关节力矩传感器，分辨率 ≤0.02 N·m，采样 ≥5 kHz” [查]。

两家不是谁更先进的问题，是两条不同的路线：

	电流估力（宇树路线）	**传感器测力（部分厂商路线）**
靠什么	τ = k_t · i × 减速比 × 效率	应变片或磁弹性元件直接测输出轴
便宜	是，电流采样本来就要做	否，每个关节一颗
精度受什么污染	k_t 的温度漂移、减速器摩擦、齿隙	传感器本身温漂、串扰、零点、易碎
高减速比时	摩擦把真实输出力矩吃掉一大块，估不准	仍然准，因为装在输出端
低减速比时	很准，因为摩擦占比小	有点浪费

看出那个耦合关系了吗：**“能不能用电流当力矩”取决于减速比选多低，而“减速比选多低”又取决于你能不能接受更大的电机。** 这就是第 14 章整章的主题，也是“准直驱”这个词真正的含义——它不是一种电机，是一个因果闭环。

一个日常类比，以及它在哪儿破

类比：电流是这家机器唯一的货币。你可以用“位置”“速度”“柔顺”这些词下单，但收银台只收安培。上层那些名词都是支票，最后总要有人去把支票兑成现金。

这个类比在哪里开始说谎：货币的汇率是稳定的（一美元永远换那么多人民币），而 k_t 不稳定的。同一个 40 A，冷机时给 25.6 牛米，磁钢热到 100 °C 时可能只有 23 牛米出头——汇率自己会漂。更糟的是，漂了以后**控制器不知道**：它只知道发了多少安培，不知道换到了多少牛米，除非你在输出端再装一个传感器（那就回到了上一节那条路）。

所以真实情况更像：你拿现金去买东西，但物价随温度浮动，而价签是三天前印的。

（NdFeB 磁钢剩磁的温度系数约 −0.11 %/°C，铜电阻 +0.393 %/°C，都是电机行业的通行数值 [行]，不是宇树公布的器件参数 [未]。用它们算：升温 80 °C ⇒ 剩磁掉约 8.8%，铜损涨约 31%。这两个百分比是第 36 章“降额”那节的起点。）

于是“力控”这件事被重新定义了

回到那条式子。如果 τ = k_t·i，那么“输出 5 牛米的力”和“让绕组流过 7.83 安培”是同一句话的两种说法：

```
i_q = τ / k_t = 5 / 0.63895 = 7.83 A
```

这就是为什么机器人关节的驱动器里，最里面那一圈环路控制的是电流而不是别的什么。它不是习惯，是唯一可行：所有上层意图都要经过这个翻译，而这个翻译必须尽可能快、尽可能准，因为它下面已经没有人了。

顺带把一件容易混的事说清：这里写的 `i_q` 带下标，是因为绕组里的电流是一个二维平面上的矢量，只有垂直于转子磁钢的那一个分量产生扭矩，另一个分量不产生扭矩、只产生热和磁化。那个“垂直分量”就是下一章的全部内容，也是 FOC 里那个 F 真正指的东西。

现在只需要知道：τ = k_t·i 里的 i，严格讲是 i_q；而“让电流矢量总是准确地落在 q 轴上”这件事，是整条控制链里最容易被忽略、又最不能出错的一环。

一个可检验的说法：宇树 GO-M8010-6 手册里的扭矩常数 0.63895 N·m/A 是输出端口径，判据是它乘以官方峰值电流 40 A 得 25.56 N·m，与官方峰值扭矩 23.7 N·m 相差 7.8%，而与转子端扭矩（3.7 N·m 量级）相差 6 倍以上。

证据等级：五个输入数（0.63895、40、23.7、6.33、530 g）全部 [厂]；7.8% 与 0.10094 是 [算]；“G1 关节内无力矩传感器”是 [未]（官方从未说明，本书据“四合一”措辞与无相关表述做的推断，不能当事实）；G2 的全关节力矩传感器是 [查]（二手转述厂商规格）；温度系数 −0.11 %/°C 与 +0.393 %/°C 是 [行]。

可核对清单：

- ☐ 打开 unitree.com 的 GO-M8010-6 参数页，抄下 k_t、峰值电流、峰值扭矩三个数，自己乘一次
- ☐ 判断 0.63895 是电机端还是输出端：用“×40 是否接近 23.7”这一条判据
- ☐ 在宇树全系人形与电机页面里搜索“额定扭矩 / 持续扭矩 / rated torque”，确认是否真的一个都没有
- ☐ 在宇树 G1 页面搜索“力矩传感器 / torque sensor”，记录你能找到的原话（预期：找不到）

下一章：这一章把 τ = k_t·i 当成一条公理用了。下一章要拆它的底座——为什么电流和扭矩之间是成正比，而不是别的什么关系；以及那个“必须垂直”的几何条件，到底是谁在维持。

第 05 章

磁场只认垂直分量：FOC 里那个 F 到底指什么

推门的时候，你从来不需要把门推开

上一章留了一个问题：为什么扭矩和电流成正比，而不是和电流的平方、或者和某个角度有关。答案是一个叉乘，而它有一个你在生活里天天用、但从没想过的原因。

想象一扇铰链门。你要用它做一件事：让门绕铰链转。

- 你垂直于门板推把手：很省力，门很容易转。
- 你沿着门板的方向推（也就是朝铰链方向推）：用再大的力，门一点都不会转。
- 你斜着推：只有垂直于门板的那一个分力在起作用，平行那个分力完全浪费。

第二件事听起来荒谬，但它正是电机里每一毫秒都在发生的事情。而且它给出了整个磁场定向控制的全部动机：

扭矩不是由“力的大小”决定的，是由“力和磁场之间夹角”决定的。所以控制扭矩，本质上是在控制一个角度，而不是在控制一个大小。

“the biggest product ever”

埃隆・马斯克（Tesla / xAI；人形机器人 Optimus 的公开发言者） — Business Insider 综述报道里的马斯克原话 — 2026-07-24（英文原句，经第三方报道逐字引用；本书未核对录音或视频原文）

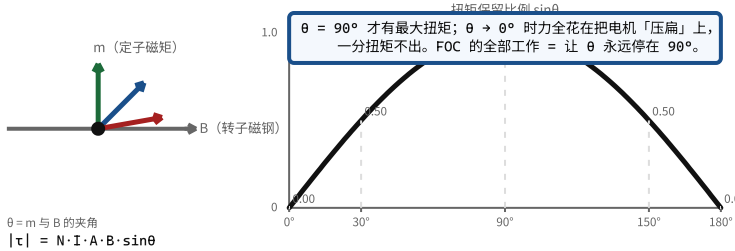


图 05-1　扭矩只由夹角决定：τ ∝ sinθ，所以「定向」定向的是正交

从洛伦兹力到 τ = m × B，四步

把门换成一个矩形线圈，把铰链换成转轴，把“你推”换成“磁场推”。

第一步，一根导线。 电流 I 流过长度 ℓ 的导线，处在磁感应强度 B 的磁场里，受到的力是

```
F = B · I · ℓ          方向垂直于 B 与导线本身
```

注意这个“垂直于”：力的方向由叉乘决定，你没法选。导线里流 10 安培，磁场 0.5 特斯拉，导线长 5 厘米，力就是 250 毫牛。听起来很小，但把它绕 50 匝、放进半径 1.5 厘米的气隙里，就是 250 mN × 50 × 0.015 m ≈ 0.19 N·m。一颗 40 毫米云台电机的量级就是这么来的。

第二步，一个线圈。 矩形线圈的两条有效边受力方向相反，构成一个力偶。把磁矩定义为 m = N·I·A（匝数 × 电流 × 线圈面积，方向沿线圈法线），则

```
τ = m × B          |τ| = N·I·A·B·sinθ
```

θ 是“线圈法线”与“磁场”的夹角。

第三步，读那个 sinθ。 这是本章唯一重要的一个函数：

```
θ = 90°   ⇒ sinθ = 1      扭矩最大
θ = 45°   ⇒ sinθ = 0.707  扭矩掉 29%
θ = 10°   ⇒ sinθ = 0.174  扭矩掉 83%
θ = 0°    ⇒ sinθ = 0      扭矩为零，但轴向受力最大（只会把电机压扁）
```

回到门的比喻：θ = 0° 就是“沿着门板推”。你把全部力气花在把门压向铰链上，门不会动。

第四步，换成永磁同步电机的说法。 转子上的永磁体提供 B，方向固定在转子上；定子绕组提供 m，方向由你给的三相电流决定。转子在转，B 的方向在变，所以**如果你什么都不做，θ 会一路从 90° 滑向 0°，扭矩自己就没了**。

这就是“换向”这个词的物理含义：不是切换电路，是**始终维持 θ = 90°**。

所以 FOC 的 F 定向的是什么

Field-Oriented Control，磁场定向控制。中文资料里经常把“定向”解释成“把磁场对准转子”，这个说法是错的，而且错得很关键——对准（θ = 0°）恰恰是扭矩为零的那个位置。

正确的说法是：

“定向”指的是把定子磁场钉在“比转子磁场超前 90°”的方向上。维持垂直，不是维持对齐。

而“钉在超前 90°”这件事，需要一个坐标系才能说得清。行业里的做法是：造一个跟着转子一起转的坐标系，把它的 d 轴（direct axis）定义为转子磁钢的方向，q 轴（quadrature，正交）定义为超前它 90° 的方向。在这个坐标系里看，“维持垂直”退化成了两句极其朴素的话：

```
i_q = 我要的扭矩 ÷ k_t      ← 这一路电流全部在做功
i_d = 0                      ← 这一路电流什么都不做，只白烧铜
```

于是 τ = m × B 里那个 sinθ，在工程实现上变成了“电流矢量在 q 轴上的投影”。上一章那条 τ = k_t·i 里含糊的 i，到这里才终于有了确切含义：**它是 i_q，是电流矢量在“垂直于转子磁钢”那个方向上的分量**。

用标准的 dq 模型写出来（幅值不变变换约定下面第 10 章会定义）：

```
τ = (3/2) · p · λ_m · i_q          面贴式永磁（SPM）
k_t = (3/2) · p · λ_m              与上一章的 k_t 是同一个东西
```

p 是极对数，λ_m 是永磁体磁链。那个 3/2 是三相坐标系折算到两相时出现的常数，取决于变换怎么定义——**这是中文资料里最容易打架的一处**：有的书写 3/2，有的写 √3/2，有的写 1，全部正确，只是各自用了不同的变换约定。本书统一用幅值不变（amplitude-invariant）变换，因此系数是 3/2，且第 07 章会给出这条约定的自检。

那个 3/2 不是拟合出来的系数，也不是“三相电机都这么写”——它是**功率守恒的记账符号**，而且它一出现就把两个公式一起解释完了。

推导只有三步，全程用第 10 章会正式定义的**幅值不变**约定（Clarke/Park 变换后，矢量的幅值等于相电流的幅值）。

```
① 三相电功率                    p_el = v_a·i_a + v_b·i_b + v_c·i_c
② 幅值不变约定下换到 dq 轴      p_el = (3/2)·(v_d·i_d + v_q·i_q)
③ 把电压方程拆成“发热/储能/做功”三块
     R·i 项 → 铜损，不产生扭矩
     L·di/dt 项 → 磁场储能，平均不产生扭矩
     ω_e·λ 项（反电势项）→ 唯一真正变成机械功的那部分
```

第 ③ 步留下的是机电转换功率 P_conv = (3/2)·(e_d·i_d + e_q·i_q)，其中 e_q = ω_e·λ_d、e_d = −ω_e·i_q·L_q（ω_e = p·ω_m 是电角速度）。面贴式电机 L_d = L_q，于是含电感的那两项**互相抵消**，只剩：

```
P_conv = (3/2) · ω_e · λ_m · i_q = (3/2) · p · λ_m · i_q · ω_m
τ = P_conv ÷ ω_m = (3/2) · p · λ_m · i_q        ← 上面那个公式就是这么来的
```

同一步推导顺手解释了磁阻项从哪来：内嵌式电机 L_d ≠ L_q，那两项不再抵消，剩下的正好是 (3/2)·p·(L_d − L_q)·i_d·i_q。这一项叫**磁阻扭矩**：磁钢埋进铁芯（内嵌式）以后，d 轴和 q 轴方向的磁阻不一样，转子会被拽向“磁阻最小的位置”，即使永磁体完全不贡献也出扭矩。下面那条公式与第 11 章讲的就是它。 所以“3/2”和“磁阻项”不是两个独立的知识点，它们是同一次记账的两个结果。

为什么这个系数是中文资料里最容易打架的一处：如果改用**功率不变**约定（变换矩阵带 √(2/3)，代价是幅值不再是相电流幅值），第 ② 步的 (3/2) 就变成 1， 扭矩公式跟着变成 τ = p·λ_m·i_q——**但那时式里的 i_q 已经不是同一个数了**。 两套约定都对，混用才会错：同一个绕组，用错约定算出来的 k_t 会差 1.5 倍。 所以本书的规则是：**任何一颗电机的 k_t，必须和它的变换约定一起出现**， 否则它只是一个没有单位的数字。厂商手册通常直接给“线电压 RMS 每千转”那种口径的反电势常数， 那是第三套口径——它与上面两套之间还差 √3 与 √2，换算过程见附录 A 的 k_e 条目。

如果转子是内嵌式（IPM，磁钢埋在铁芯里，d/q 两个方向的磁阻不一样），还要多一项磁阻扭矩：

```
τ = (3/2) · p · [ λ_m · i_q + (L_d − L_q) · i_d · i_q ]
                              └──────── 磁阻项 ────────┘
```

宇树从未公布它的关节电机是 SPM 还是 IPM [未]。官方只说“低惯量高速内转子永磁同步电机” [厂]——注意“内转子”讲的是转子在里还是轴在里，和 SPM/IPM 是两个正交的分类，很多资料会把这两个词混着用。

一个可以立刻用的推论：偏离 90° 的代价有多大

既然 sinθ 是唯一的主角，那么“θ 到底偏了多少”就是一个可以算成钱的数。

设电流矢量总幅值 I 保持不变（因为逆变器能给的电流上限是固定的），本该全部落在 q 轴上，但因为角度估计错了 δ，实际分布变成：

```
有效扭矩比例 = cos δ
漏进 d 轴的电流比例 = sin δ
```

脚本 `foc_check.py angle_error_cost()` 给出的结果：

角度误差 δ	扭矩保留	白烧的电流占比
1°	99.985%	1.7%
5°	99.62%	8.7%
15°	96.6%	25.9%
30°	86.6%	50.0%
90°	0%	100%（纯粹的轴向力）

这张表有一个非常反直觉的地方，值得工程师专门看一眼：**5° 的角度误差，只让扭矩掉了 0.38%，却让 8.7% 的电流跑去了不产生扭矩的方向**。因为扭矩看的是 cos（在 90° 附近极其平缓），而“分量分配”看的是 sin（在 0 附近极其陡峭）。

后果是：一个编码器装偏了 5° 的关节，**扭矩几乎没变小，但为了维持同一扭矩，电流有效值要多 0.38%，铜损要多 0.77%，而且控制器完全不知道**。它能通过一切“扭矩够不够”的测试，只会在长时间满载时因为发热而先降级、后退磁。这类故障在电机行业里非常常见，因为它不表现为“不工作”，只表现为“莫名其妙比别家热”。

（0.38% 与 0.77% 的算法：同样输出扭矩下电流幅值要放大 $1/\cos\delta$，即 $1/\cos 5^\circ - 1 = 0.38\%$；铜损 $\propto I^2$，即 $(1/\cos 5^\circ)^2 - 1 = 0.77\%$。这是本书推算 [算]，不是任何厂商的数据。`code/tools/foc_check.py` 的 `angle_error_cost(5°)` 会打印这两个数。）

类比，以及它在哪儿破

类比：你和一个朋友一起抬一根长木杆的一端绕圈走。朋友站在杆的另一头不动（转子磁钢），你抬的这一头要始终“横着”推，杆才转得最快。你越是把力气用在“顺着杆朝朋友推”上，越是一分功不出，只会把杆压

弯。

类比在哪里破：真实的电机里没有人在“调整姿势”。你的“是否横着”完全取决于你知不知道朋友此刻朝哪个方向——也就是**必须实时知道转子的电角度**。这里有一层后面所有章都要用、但没人交代的前提：**编码器量到的是机械角，电角度要再乘极对数**（θ_e = p · θ_m）。所以上面那张表里的 5°、15°、30° 全是**电角度**；折算到机械角要除以 p，而宇树这几颗关节的 p 未公布 [未]。第 10 章 Park 变换的输入、第 14 章那颗 15 bit 编码器，都在这条关系上工作。。角度信息一旦丢失或者装偏，你会在不知不觉中以 30° 的夹角推一整年。

这条式子还有第二个前提，平时没人说。τ = (3/2)·p·λ_m·i_q 之所以对 i_q 是**一条直线**，靠的是两件事：转子磁链 λ_m 是常数，以及磁路是线性的——铁芯里的磁通 B 与产生它的安匝数成正比。第二件事在几十安培这一档**一定会开始破**：定子齿部的磁通密度爬到硅钢的膝点（工程上约 1.5–1.7 T [行]）之后，再多给一安培换来的磁通变少，于是有效 k_t 掉下那个常数，L_d 与 L_q 也跟着随电流变。这正是第 04 章那个 7.8% 落差的候选解释之一（0.63895 × 40 A = 25.56 N·m，而官方峰值是 23.7 N·m）：磁饱和让常数掉、脉冲口径让 40 A 比 23.7 N·m 更短、或者 40 A 本来就是母线电流而非相电流幅值。**磁钢牌号、B–H 曲线、k_t 随电流的曲线，宇树全部未公布** [未]，所以本书只能说“这条线性假设在这一档电流下必然开始失效”，不能说它掉了几个百分点——那是要自己在台架上测的（第 40 章的里程碑里有这一项）。

这就把问题推给了两个新东西：一个是“怎么知道角度”（第 14 章的编码器），一个是“怎么让电流矢量真的按你要的角度去流”（第 07、08 章的三相与 PWM）。

而“怎么产生一个方向随你便的磁场”这个问题本身，还没解决。下一章解决它。

一个可检验的说法：扭矩与“定子磁场和转子磁场的夹角”成正弦关系，因此磁场定向的目标是让两者保持垂直而不是对齐；在电流总幅值受限的前提下，5° 的角度估计误差只损失 0.38% 扭矩，却会让 8.7% 的电流变成无用的磁化电流。

证据等级：$F = B \cdot I \cdot \ell$、$\tau = m \times B$、$\sin\theta$ 关系、dq 模型与 $(3/2)p \cdot \lambda_m \cdot i_q$ 全部是电机学通识 [行]；角度误差表是 [算]（`foc_check.py`，纯三角函数，任何人可复算）；“宇树未公布 SPM/IPM”与“未公布极对数、磁链”是 [未]；“低惯量高速内转子永磁同步电机”是 [厂] 原话。

可核对清单：

- ☐ 自己代一次 sin：$\cos 5° = 0.9962$，$\sin 5° = 0.0872$，确认“扭矩掉得少、分量漏得多”
- ☐ 在宇树电机页面搜“表贴 / 面贴 / 内嵌 / SPM / IPM / 极对数”，记录是否有任何一项被公布（预期：无）
- ☐ 找一份中文 FOC 资料，看它的扭矩公式前面是 3/2、$\sqrt{3}/2$ 还是 1，然后查它用的是幅值不变还是功率不变变换——确认“系数不同不代表有人算错”
- ☐ 想清楚为什么“内转子”和“IPM”是两个不相关的分类

下一章：现在我们知道要维持垂直，也知道要控制电流矢量。但定子有槽、有绕组、有引出线，我们手上能操作的只有几根导线。下一层的问题是：只用固定的几个线圈，怎么造出一个方向任意、还能旋转的磁场？

第 06 章

电刷是被淘汰的答案：换向、火花，以及把电流搬回定子

直流电机也能做 FOC，只不过它用机械做

第 05 章的结论是：控制扭矩 = 维持定子磁场与转子磁场垂直。第 06 章（就是本章）要问的是：**这件事有没有更笨的做法？**

有，而且做了一百多年。

有刷直流电机的转子上绕着线圈，轴上装一个铜质换向器（一堆彼此绝缘的扇形片），两个碳压在上面把电流送进旋转的转子。当转子转过中性面，碳刷自动从一片滑到下一片——**电流方向反了一次**。而这一次反向，恰好让转子磁矩在半个周期里始终与定子磁场保持大致垂直。

换句话说：**换向器就是一台机械式的 FOC**。它确实在“维持垂直”，只不过它的实现方式是每转一圈撞自己一次。

由于经费少，这个200块的双足机器人不太理想。我反思发现它的关节存在问题，于是在2013年想到用盘式无刷电机直驱的方案，电机驱动系统、机械结构和控制算法都由我一人完成。

王兴兴 — 凤凰卫视《X+》专访（主持人田桐），腾讯新闻转载 — 2025-02-21（直接引语（问答按轮次归属））

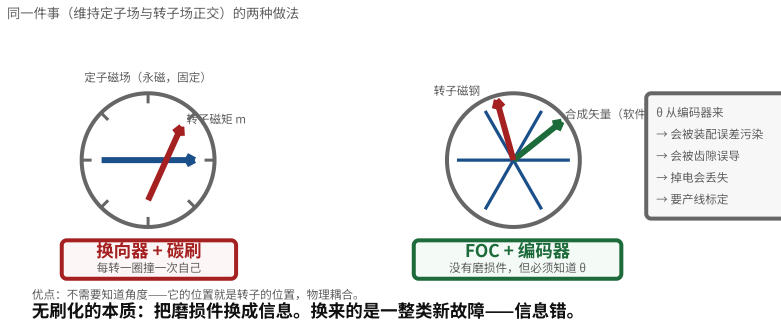


图 06-1　换向器 = 机械式磁场定向；无刷把开关搬进软件，代价是要知道角度

于是有刷电机有一串不可修的毛病

毛病	物理原因	对机器人关节意味着什么
火花	换向瞬间被断开的那一相绕组里电流不能突变（V = L · di/dt），能量只能击穿空气	防爆场合不能用；碳与铜被烧蚀；EMI
磨损	碳刷是牺牲件，靠物理摩擦导电	关节寿命 = 碳刷寿命。人形一条腿若跑到 3,000 转、换向器 12 片，就是每秒 600 次换向（3,000 ÷ 60 × 12 [算]）——每次都是一次电弧，一小时就是 216 万次
碳粉	磨损产物落在壳内	密封壳里导电粉尘 = 短路风险（第 13 章的直通有了新来源）
转速上限	换向器片间绝缘在离心力与电弧下失效；机械换向有物理时限	换向频率随转速线性涨，几千转就到头
扭矩纹动	转子只有有限个槽，换向是“跳”过去的不是连续转的	低速时扭矩呈齿槽状波动——正是人形踝关节最怕的
需要把电流送进转子	转子在转，得靠接触导电	多两个接触电阻、多两个发热源、多一套维护

最后一条是最根本的：**只要电流要进转子，你就必须有一个滑动电接触**。而滑动电接触是所有机电设计里最脆的那一类东西。

无刷的解法：把麻烦的一边挪到不转的那边

思路极其朴素——**永磁体不需要通电就能提供磁场**。那就把它放到转子上（转的那一边），把需要通电的绕组放到定子上（不转的那一边）。滑动接触消失了。

代价是：现在定子绕组的磁场方向不再由转子位置自动决定，**你必须自己知道转子在哪，然后自己决定给哪相通正、哪相通负**。

于是：

> 有刷：机械换向器 → 不需要传感器，不需要算法，需要碳刷
> 无刷：电子换向器 → 需要角度传感器，需要算法，不需要碳刷

这就是第 06 章的全部因果。无刷不是“更先进的电机”，它是一次**把磨损件换成信息**的交易：你不再需要每转一圈撞一次的铜和碳，你需要的是每一毫秒都知道转子朝哪儿。

而“知道转子朝哪儿”这件事，在第 10 章变成了 Park 变换的输入 θ，在第 14 章变成了那颗 15 bit 编码器和它的 187 倍齿隙（齿隙 = 输入端转了而输出端不动的那段空程，第 14 章）问题，在本章末尾变成了装配工艺。

六步换向与 FOC 是同一件事的两个精度版本

值得多说一句，因为工程师会碰到：无刷控制有两种主流做法，它们解决的是同一个“换向”问题，只是分辨率不同。

六步换向（trapezoidal / BLDC）：只用霍尔的三个信号把转子圆周分成 6 个 60° 扇区，每个扇区里让两相通电、一相悬空。电流矢量只能指 6 个方向，切换靠查表。实现简单、不需要精细角度、不需要高带宽电流环。

磁场定向（FOC / sinusoidal）：电流矢量可以指任意方向、给任意大小，连续变化。

差别在扭矩的平滑度上。六步换向里，电流矢量每 60° 跳一次，跳的那一下就是扭矩阶跃——**低速时你能听见它，也能在力控曲线上看见它**。FOC 把这个 60° 的台阶换成了连续曲线。

对人形机器人来说这不是“效率优化”，是必需：踝关节要在接近零速下稳定输出几十牛米，六步换向的 60° 台阶在那个工况下等于把第 05 章那个 sinθ 的偏离变成了常态。

（宇树官方文档里 M8010 的工作模式位域写的是 `1 = FOC closed-loop` [厂]，About 页写的是 integrated FOC driver board [厂]。它没有公布是否同时提供六步模式，也没有公布换向用的传感器配置——霍尔还是纯编码器，未公布 [未]。）

那么步进电机呢

工程师还会问这个，因为步进电机更简单：不需要编码器，给一个脉冲走一步，位置是“数”出来的。

问题在两个地方。

一，**步进的开环位置控制依赖“不丢步”**。而机器人关节恰恰会在落地、被推、被卡的时候丢步——丢步之后你完全不知道自己在哪，而 FOC 关节每一毫秒都知道（第 14 章的绝对值编码器）。

二，**步进的扭矩密度与效率在高速时崩得很快**，而且它的保持转矩意味着静止时也在满电流发热。人形关节站着不动就要一直出力（第 16 章），这条对步进是致命的。

所以宇树官方对关节电机的措辞是“低惯量高速内转子永磁同步电机”[厂]——**永磁同步（PMSM）**，不是步进、不是有刷、不是开关磁阻。这个词组里每个词都是有理由的：低惯量（为了电流环之后的快速扭矩响

应）、高速（为了配低减速比还能给足转速，第 14 章）、内转子（转子在轴上，外面是定子与壳，方便把编码器装在转子端）、永磁（转子不用通电，第 06 章的核心交易）。

类比，以及它在哪儿破

类比：电刷是一个每转一圈就要撞一次的机械开关。FOC 是把这个开关搬进软件——代价是软件得先知道现在几点。

这个类比在哪里破：机械开关虽然会磨损，但它**永远不会不知道自己在哪**——它的位置就是转子的位置，物理耦合，不需要测量、不需要标定、不会装偏。而软件换向的角度是从传感器读来的，会被装配误差污染（第 10 章那张 5° 表）、会被齿隙误导（第 14 章那 187 倍）、会因掉电丢失（所以要多圈或绝对值编码器）。

无刷电机用一次交易换掉了磨损件，换来了一整类新的故障模式：信息错。这本书第 1 部剩下的章节，有一半在讲怎么防这类新故障。

一个可检验的说法：换向器是一台机械式磁场定向装置，它用滑动电接触换掉了传感器与算法；无刷化的本质是把电流从旋转侧搬到静止侧，代价是必须实时获知转子电角度；而六步换向与 FOC 解决同一个问题，差别在电流矢量能否连续——低速恒力输出这一工况要求它连续。

证据等级：换向器原理、六步换向、火花与磨损机理、步进的两个失效点，全部是电机学通识 [行]；`1 = FOC closed-loop` 位域、integrated FOC driver board、“低惯量高速内转子永磁同步电机”是 [厂] 原文；“是否提供六步模式 / 用霍尔还是编码器”是 [未]。

可核对清单：

- ☐ 在宇树电机页面搜“Hall / 霍尔 / sensorless / 六步 / trapezoidal”，确认是否有任何一项被提到（预期：无）
- ☐ 找一台有刷直流电机的实物，数一下换向器有几片——片数就是它一个电周期里的“矢量方向数”，对照六步换向的 6

- ☐ 读一遍官方那句“低惯量高速内转子永磁同步电机”，把四个词各自对应的取舍写出来（本章给了一个版本）
- ☐ 想清楚为什么“机械换向不需要知道位置”这句话是对的，以及它换来的是什么

下一章：无刷电机解决了“电流不进转子”，但留下了一个更难的问题：定子上那些固定的线圈，怎么产生一个能指向任意方向的磁场？下一章用一次加法回答它。

第 07 章

旋转磁场是合成出来的：三相、120°、以及为什么没有三次谐波

三个只能直着推的人，合出一个匀速画圆的力

上一章的结论是：要控制扭矩，就得让定子磁场始终垂直于转子磁场。但“垂直”意味着这个磁场得能指向任意方向，而且转子在转，所以它还得跟着转。

问题立刻变得很具体：定子上是固定的铜线圈，它们只能沿自己的轴线产生磁场，一根直线上的东西怎么会转？

答案是矢量合成。三个人站在互成 120° 的方向上推一个球，每个人只能沿自己那个方向直推（这叫脉振磁场——方向固定、大小随时间正负摆动）。但只要三个人的力气按正弦变化、且相位彼此差 120°，他们三个的合力就是一只手在匀速画圆。

这不是比喻，这是一次可以代数字的加法。这一章就代这一次。

“Optimus will be an amazing surgeon in the future.”

埃隆·马斯克（Tesla / xAI；人形机器人 Optimus 的公开发言者） — 特斯拉 2025 年第三季度财报电话会议（36 氪英文版报道） — 2025-11-07（英文原句，经第三方报道逐字引用；本书未核对录音或视频原文）

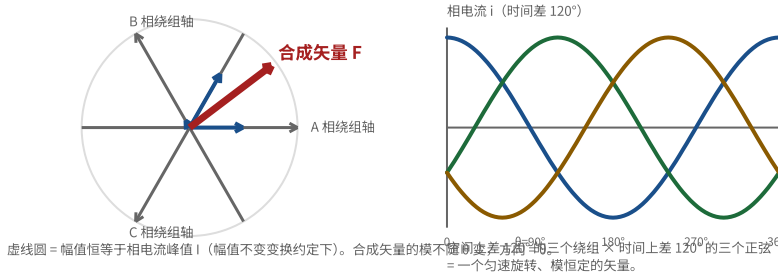


图 07-1　三个只会直着推的绕组，合出一个方向任意的磁场矢量

一次加法

设三个绕组的轴线方向分别在与参考轴成 0°、120°、240° 的方向上，各自产生的磁动势沿自己那条线，大小正比于各自的电流。于是合磁动势是个二维矢量：

```
F_合成 = i_a·(1, 0) + i_b·(cos120°, sin120°) + i_c·(cos240°, sin240°)
```

给三个正弦电流，彼此差 120°：

```
i_a = I·cos θ
i_b = I·cos(θ − 120°)
i_c = I·cos(θ + 120°)
```

代进去，取 I = 10 A、θ = 37°（脚本 `foc_check.py rotating_field_peak()` 跑出来的）：

```
i_a = +7.986 A
i_b = +1.219 A
i_c = −9.205 A
        ↓ 合成
(α, β) = (+7.986, +6.018)
|合成| = 10.0000 A
方向 = 37°
```

合成矢量的模正好等于相电流的峰值 10 A，方向正好等于 θ。 把 θ 换成 0°、91°、200° 再算，模仍然是 10.0000，方向跟着 θ 走。

这就是“合成任意方向的磁场”这句话的确切含义：**你不需要让线圈转，你只需要让三个沿固定方向摆动的量按 120° 的相位差摆动，它们的和就是一个转的矢量。** 而且这个矢量的方向、大小都是你可以任意指定的——指定方式就是指定那三个数。

顺便注意一个漂亮的性质：合成模是**常数**。三个脉振磁场各自一会儿正一会儿负，加起来却是一个长度不变的矢量在转。这不是巧合，是因为 120° 的空间分布配 120° 的时间相位，正好把二次项消掉了。这个“常数模”性质意味着磁场不会周期性伸缩，也就没有周期性的径向力——没有径向力，就没有那种“嗡嗡”的电磁噪声和振动。这是三相电机比单相安静的根本原因。

为什么是三相，不是两相、四相

两相正交绕组也能合成任意矢量，逻辑上完全够用。那三相好在哪？

第一，**谐波位置不同**。N 相对称绕组通 N 相对称正弦电流，合成磁动势的空间谐波落在 $2N \cdot k \pm 1$ 次（$k = 1, 2, 3\ldots$）。两相的谐波落在 3、5、7…；三相的落在 5、7、11、13…，**没有 3 次**（3 的倍数次在三相对称系统里是零序分量，星形接法且中点不引出时没有通路，压根流不起来）。

这个差别很值钱。5 次与 7 次这一对谐波会和转子基波相互作用，产生一个 6 倍电频率的扭矩脉动（$7 - 1 = 6$）——俗称齿槽感、低速抖动。人形机器人的踝关节要在低速下稳稳输出几十牛米，这个脉动直接进到你的力控误差里。

第二，**同样电流下铜的利用率**。三相对称时中点不用引出，只需要 3 根线、6 个开关器件就能独立控制那个二维矢量的两个自由度。两相要引出 4 根线（或者接受公共端带来的耦合），四相要 8 个器件换一点点谐波改善，收益递减。

第三，**历史惯性**。三相输电与三相感应电机共享同一套基础设施与制造产业链，绕组模具、驱动器、电流传感器、控制芯片全按三相做。机器人关节是这条产业链下游的一个客户，不是它的定义者。

需要诚实标注的是：以上三条是**电机学通识与行业判断** [行]，不是宇树的选型说明。宇树从未解释过为什么用三相 [未]——它大概也没觉得需要解释，因为整个行业都是三相。

三相到两相的那次换算，有个名字叫 Clarke

上面那个“三个数 → 一个二维矢量”的加法，工程上有个固定的写法。设三相电流 i_a、i_b、i_c，定义：

```
i_α = (2/3)·[ i_a − ½·i_b − ½·i_c ]
i_β = (2/3)·(√3/2)·[ i_b − i_c ]
```

这就是 **Clarke 变换**（也叫 αβ 变换）。前面那个 2/3 系数是“幅值不变”约定的产物——它保证了我们上面那个漂亮结论：三相正弦幅值 I 时，|(i_α, i_β)| = I。如果你看到有的资料写 √(2/3)，那是“功率不变”约定，两个数都差一个常数因子，物理结论一样。

自检一下（`foc_check.py clarke_three_phase()`）：θ = 0 时 i = (10, −5, −5)，代入得 (α, β) = (10, 0)。α 轴与 A 绕组轴线重合，所以 A 相独占 10 安培时合成矢量正好指在 0°，模 10。对。

还有一个立刻能用的推论：因为 i_a + i_b + i_c = 0（平衡三相、星形无中点），**你只需要采样两相电流，第三相减出来**。所以真实的关节驱动板上通常只有两个电流采样电阻或者两个霍尔，而不是三个——省一颗器件，省一路标定，代价是三相不平衡时算不准。你在拆一块驱动板时看到“只有两相采样”，不是偷工减料。

到这一步，我们手上有什么、还缺什么

现在链条是这样：

```
要多少扭矩  →  要多大的电流矢量、指哪个方向  →  三相电流各是多少
   （第 04 章）          （第 05 章）                    （本章）
```

看起来通了。但“三相电流各是多少”这个结论要落地，还差最后一件事：**谁去产生那三个按 120° 相位摆动的电流？**

你手上的器件是：一个直流电池（24 V、60 V、75 V，看型号），和每相上下两个开关。开关只有两个状态。没有“输出一个 7.986 安培”这种旋钮。

也就是说：**我们要用只有开和关的器件，造出连续可变的正弦电流。**

这就是下一章，也是你提到的那句话真正所在的位置——“改变外面电流导通的时间，可以合成为任意方向任意大小的磁场”。这句话里的“改变导通时间”，指的是脉宽调制；而它之所以合法，靠的是一个叫伏秒平衡（伏秒平衡 = 电感只记得电压对时间的积分，第 08 章）的东西。

类比，以及它在哪儿破

类比：三个人站在互成 120° 的方向上，各自拿一根只能沿自己那条线拉的绳子。每个人按同一个节奏、但彼此晚 1/3 个周期地用力拉。球被三个人同时拉着，却像是被第四个人拉着绕圈走。

这个类比在哪里破：三个人可以精确控制自己的力气，而开关不能。开关只能全通或全断，中间没有“用一半的力”。所以真实世界里那个“第四个人”根本不存在——存在的是一堆极快的开关动作，靠时间比例骗过负载。类比到“力气”这一层就断了，下一层必须换一套物理解释。

一个可检验的说法：三个空间上互差 120°、时间上互差 120° 的脉振磁动势，其合成矢量的模恒等于相电流峰值、方向等于电流相位角；且三相对称系统的空间谐波不含 3 次，最低为 5 次。

证据等级：合成矢量与幅值不变 Clarke 变换是电机学通识 [行]；“模恒为 10.0000”是 [算]（`foc_check.py`，θ 取 0°/37°/91°/200° 四点全过）；“没有 3 次谐波 ⇒ 没有 6 倍电频率扭矩脉动”是电机设计的通行结论 [行]；“宇树未解释为何用三相”是 [未]；“驱动板通常只采两相”是行业普遍做法 [行]，**本书没有拆开过任何一块宇树关节板，不能断言它用了几路采样**。

可核对清单：

- ☐ 跑 `python code/tools/foc_check.py`，看“07 三相合成幅值”那条在四个角度下是否都给出 10.0000
- ☐ 手算一次 θ = 0°：i = (10, −5, −5)，确认 α = 10、β = 0
- ☐ 找一块开源电机驱动板（ODrive、moteus 都是开源的）的原理图，数一数它有几路相电流采样，验证“只采两相”这个说法
- ☐ 检查你读过的任何一份中文 FOC 资料，看它的 Clarke 变换前面是 2/3、√(2/3) 还是 1，并确认它声明了变换约定

下一章：开关只有两个状态，却要给出一根连续变化的正弦电流。这一层的答案不是电气技巧，而是一条几何事实：在一个开关周期里，你只能平均出六边形内部的点。

第 08 章

改变导通时间：PWM 如何在六边形里合成任意磁场

只有八种接法，却要给出无数种力

上一章结束时，我们手上有的东西是：一个直流电源（GO-M8010-6 是 24 V [厂]），三对上下开关（六个 MOSFET），和一个只能指向八个方向之一的输出电压。

八个方向，是因为每相只有两种状态：上管通（该相接到 +24 V）或者下管通（该相接到 0 V）。三相独立，$2^3 = 8$ 种组合。就这八种。

而我们要的，是一个**方向和大小可以任意指定**的磁场矢量。

这看起来是不可能的任务。它其实是可以的，靠的是一条非常朴素的事实：**负载不看你瞬时给了什么，它看你在一段时间里平均给了什么**。

比如刚才提到的电机，我们最早那款产品用的雷卡电机，性能非常好，但后来不怎么用了。原因是我们太专注于个别技术参数……

王兴兴 — 投中网（同上，来源 B），谈选型教训 — 页面 2025-08-07；访谈本体约 2020 年前后（早年访谈转写）

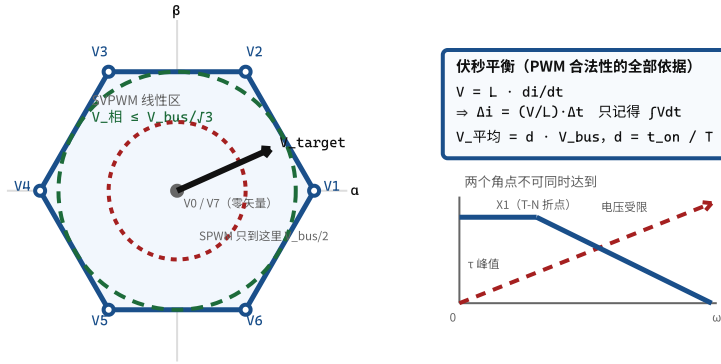


图 08-1　八个电压矢量、六边形可达区、SVPWM 内切圆，以及那条电压包络线

伏秒平衡：唯一需要记住的一条原理

电感上有一个铁律：电压决定电流的变化率，不决定电流本身。

```
V = L · di/dt        ⇒        Δi = (V/L) · Δt
```

所以给电感加 24 V 持续 10 微秒，和加 12 V 持续 20 微秒，只要 L 不变，电流的**变化量**完全一样。这个“电压 × 时间”的乘积叫伏秒（V·s）。电感只记得伏秒，不记得你中间是怎么切的。

于是“用两个瞬时值造一个平均值”这件事有了严格的说法：在一个开关周期 T 里，让

```
V_平均 = (1/T) ∫ V(t) dt
```

等于你想要的任意值。方法就是改变高电平的导通时间。这个比例叫**占空比** d：

```
V_平均 = d · V_bus        d = t_on / T ∈ [0, 1]
```

这就是脉宽调制（PWM）的全部内容。它不是一个技巧，它是电感那条 V = L·di/dt 的直接推论。

这一层就把“改变导通时间 ⇒ 合成任意大小的磁场”这句话坐实了：导通时间决定平均电压，平均电压决定电流爬升速率，电流决定磁场，磁场决定扭矩。链条上没有一环是比喻。

从三个数到一个矢量：能到的点构成一个六边形

把三相桥的八种开关组合画在 αβ 平面上，得到八个电压矢量：六个非零矢量（幅度 2/3·V_bus，彼此相隔 60°，正好构成一个正六边形的六个顶点）和两个零矢量（三个上管全通或三个下管全通，输出在原点，模为零）。

现在问题变成一个几何题：**我要的目标矢量落在六边形内部任意一点，怎么只用这八个顶点凑出来？**

答案是：取离目标最近的那两个相邻顶点，加上零矢量，按时间配比轮流打。

```
V_target · T = V_1 · t_1 + V_2 · t_2 + 0 · t_0
t_1 + t_2 + t_0 = T
```

三个方程三个未知数，总有解。零矢量的时间 t_0 就是“凑不满”的那部分。这套配时策略有个名字：**空间矢量脉宽调制（SVPWM）**。

它和“每相独立地拿正弦去和三角波比”的传统 SPWM 有什么关系？一句话：**SVPWM 和 SPWM 产生的是同一类平均电压，只是零矢量的分配方式不同，而不同的那份差别直接换成了可用电压的大小**。

```
SPWM 线性区能给的相电压峰值 = V_bus / 2       = 12.00 V（24 V 母线）
SVPWM 线性区能给的相电压峰值 = V_bus / √3     = 13.86 V
增益 = 2/√3 = 1.1547  ⇒  多 15.5%
```

（`foc_check.py spwm_vs_svpwm()` 复算。）

15.5% 是个什么概念？它意味着**同一颗电机、同一块电池，只因为换了一种配时方式，最高转速的上限提高 15.5%**，或者同样转速下多留 15.5% 的电压去驱动电流。这就是为什么现代关节驱动器全部用 SVPWM 或者它的等价形式——它不是“更高级的 PWM”，它是把六边形顶点的几何余量榨干。

关键的限制也在这张图上：六边形之外不可达。目标矢量一旦跑到六边形外面，你不管怎么配时都拿不到那个平均值。这就是下一节那个结论的来源。

一个立刻能用的推论：峰值扭矩和最高转速不能同时拿到

宇树 GO-M8010-6 的官方参数表上，峰值扭矩 23.7 N·m 和最高转速 30 rad/s 是并排写的两行 [厂]。把它们当成一个工作点来用，是选型里最常见的错误之一。用两条式子就能证明它们互斥：

```
k_t(转子) = 0.63895 ÷ 6.33 = 0.10094 N·m/A            （上一章已定口径）
ω(转子) = 30 × 6.33 = 189.9 rad/s
反电势幅值 E = (2/3) · k_t · ω = (2/3) × 0.10094 × 189.9 = 12.78 V
24 V 母线在 SVPWM 线性区能给的相电压幅值 = 24/√3 = 13.86 V
```

12.78 V < 13.86 V。 先说好消息：这两个数**对得上**。30 rad/s 时反电势只吃掉母线能力的 92%，剩下 1.08 V 的余量，正好是“空载刚好能到 30 rad/s”该有的样子——手册那个最高转速，就是被 24 V 母线顶出来的。

```
V_bus ≥ √3 · (E + I·R) = √3 × (12.78 + 40 × 0.040) = 24.9 V
 （R 取 40 mΩ 是 24 V / 40 A 级关节回路的可信量级，**官方未公布这颗的 R**
[未]；R 每差 10 mΩ，所需母线电压差 0.69 V。）
```

（`foc_check.py voltage_envelope()` 复算。）

结论：**23.7 N·m 与 30 rad/s 仍然不能同时成立**，但理由要改准：不是“空载都跑不到”，而是**满载跑不到**。空载时 12.78 V 反电势留 1.08 V 余量；一旦要 40 A 满扭矩，光绕组与驱动回路的压降就要吃掉 1.6 V（按

40 mΩ），母线立刻不够——两个角点之间那条斜线，斜率就是 R + 折算后的等效电阻。

这是本书能给读者的一个具体提醒：**看任何一颗关节电机的参数表，先做一次电压包络检查**。如果最高转速 × 反电势常数超过了母线电压除以 √3，那两个数就是互斥的，你实际能用的那条曲线比参数表看起来低得多。用同一条检查，IM6014 的 54.2 rad/s 配 60 V 母线（官方就是标在 60 V 下 [厂]）就能自洽——它把母线电压提上去了，正是为了把这个角点变成可达点。

顺手记一条：这也解释了为什么新一代关节电机的供电范围写着 24–75 V [厂]。**提高母线电压不增加扭矩，它增加的是“扭矩能维持到多高转速”**。扭矩由电流决定（第 04 章），转速上限由电压决定（本章）。这两件事在参数表上经常被混成一团。

为什么开关频率不能无限高：一笔会来收账的账

既然 PWM 靠平均，那开关越快，平均就越精细，不是越好吗？

不是。开关过程中有一个短暂的区间，电压和电流同时不为零（MOSFET 的米勒平台期间，器件工作在线性区），那一段在发热。每次开关损耗近似

```
E_sw ≈ ½ · V · I · (t_rise + t_fall) · f_sw
```

它正比于开关频率。而导通损耗 ½·I²·R_ds(on) 与频率无关。所以存在一个最优点：频率越高，波形越干净、电流纹波（电流纹波 = 叠加在平均电流上的高频锯齿，不产生平均扭矩，第 09 章）越小、控制带宽可以更高，但开关发热线性上升。

机器人关节选 20 kHz 左右（行业常见量级 [行]，**宇树未公布任何一款关节的载波频率** [未]），除了电气考虑，还有一个非常实际的原因：**20 kHz 刚好在人耳听觉上限之外**。10 kHz 以下的 PWM 会让电机发出清晰的啸

叫，一台 23 个关节的人形在安静的办公室里会像一台老式显示器。这不是性能问题，是产品问题。

纹波本身可以算：

```
Δi ≈ V · d(1−d) / (L · f_sw)
```

d = 0.5 时纹波最大（这也是为什么“半速”时电流最脏）。注意纹波**不产生平均扭矩**——它只是在铜里发热、在铁芯里产生附加损耗、在声学上产生噪声。所以“提高 f_sw 让纹波变小”这件事，改善的是发热和噪声，不是扭矩精度。扭矩精度靠的是下一条。

死区：那 100 纳秒里，你的磁场被偷走了一点

上下两个 MOSFET 绝对不能同时导通——那叫直通，24 V 直接短路，结果是炸管。所以每次换相必须留一段“两个都不开”的时间，叫死区。

死区期间，绕组电流不会消失（电感不许它突变），它只能走上管的体二极管或者下管的体二极管。于是那一相的输出电压不是“零”，而是被钳到“母线 + 一个二极管压降”或者“负的一个二极管压降”。净效果是：

```
ΔV_dead = (t_dead / T_pwm) · V_bus
        = (100 ns / 50 µs) × 24 V = 0.048 V
```

（`foc_check.py deadtime_error()` 复算。有的资料在分子上乘 2，取决于它把死区算成“上下管各贡献一次”还是“一次换相一次”；本书取单倍，并在此声明约定。）

48 毫伏听起来微不足道。但请注意低速工况：第 08 节算过，30 rad/s 时反电势幅值 12.78 V；而**接近零速时反电势接近零**。也就是说，在你最需要精确控制扭矩的那段（起步、慢速、恒力按压），你施加的电压本来就只有零点几伏，而每半个周期被偷走 48 毫伏——**占比可以到百分之十几**。

更糟的是它的方向性：被偷走的那一点，方向总是和当前电流方向相反（因为电流走的是二极管，取决于电流朝哪边流）。这在数学上等价于串进了一个“与电流方向相反的固定电压源”，效果就是过零附近出现一段死区畸变：电流指令从负变正时，实际电流会“卡”一下再翻过去。这个卡，在示波器上看得见，在力控误差上表现为**恒力按压时力曲线在过零点出现一个小台阶**。

补偿办法是有的（按电流符号预加一个电压），但补偿要准，就得知道电流符号；而电流符号在过零附近恰好是噪声主导。所以死区补偿是电机控制里“人人都会写、很少有人调好”的那一块。

这一整段是电力电子通识 [行]。**宇树关节驱动板用的死区时间、是否做补偿，全部未公布** [未]。本书给的是判据与量级，不是它的实现。

类比，以及它在哪儿破

类比：你只有一个全开和全关的水龙头，但要给一个水池稳定地注入“半档”的水流。办法是每秒钟开关两千次，开一半关一半。水池只觉得来了一股半大的水。

类比在哪里破两处。第一，水池没有惯性，而电机绕组有电感、转子有惯量——正是这两级“惯性”让平均成立（下一章专门讲，因为它是这条链上最容易被质疑的一环）。第二，你的水龙头开关不产生热量，而 MOSFET 每次开关都在发热，且发热正比于你开关的快慢。这个差别最终决定了开关频率的上限，也就决定了你的电流环能有多快——而电流环的快慢，是第 12 章那个“三个环”结构的地基。

一个可检验的说法：GO-M8010-6 在 24 V 母线下，30 rad/s 只能空载拿到；带载时能到的最高转速由 $(V_{bus}/\sqrt{3} - I \cdot R) \div [(2/3)k_t]$ 决定。拿万用表量出回路电阻、再上一台测功机，就能把这条斜线画出来（测量方法见核查单）。

证据等级：23.7 N·m、30 rad/s、1:6.33、0.63895、24 V 全部 [厂]；12.78 V、13.86 V、24.9 V、48 mV、15.5% 全部 [算]（`foc_check.py`）；V = L·di/dt、伏秒平衡、六边形矢量、SVPWM 增益、开关损耗与频率成正比、死区畸变机理，全部是电力电子与电机学通识 [行]；**20 kHz 载波频率、死区时间、是否补偿，宇树均未公布** [未]。

可核对清单：

- ☐ 跑 `foc_check.py`，看“08 电压包络”那条给出的 12.78 V（空载）与 14.38 V（满载）对 13.86 V
- ☐ 拿任何一颗你手边的一体化关节参数表，做一次同样的检查：最高转速 × k_t 是否超过 V_bus/√3
- ☐ 确认 IM6014 的 54.2 rad/s 官方是标在 60 V 下（而不是 24 V），理解“提电压是为了把角点变成可达点”
- ☐ 在宇树电机页面搜索“PWM 频率 / 载波频率 / switching frequency / 死区”，确认是否真的没有任何一项被公布（预期：无）

下一章：斩波之所以合法，是因为有两级低通（两级低通 = 绕组电感滤电流、转子惯量滤角度，第 09 章）在替你抹平。这一句现在还是断言。下一章把它算出来——顺便回答“那我的电感和惯量到底得有多大才够抹平”。

第 09 章

转子不介意你是斩波的：两级低通与“平均”为什么合法

一秒钟两万次的通断，凭什么电机不抖

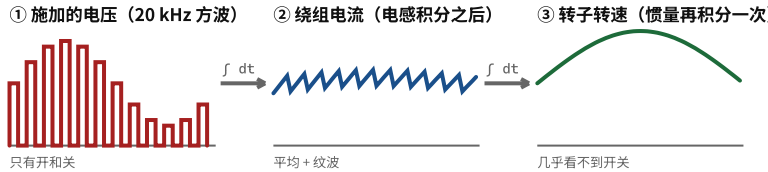
第 08 章说：改变导通时间就能合成任意大小的平均电压。

任何做过数字电路的人第一反应都是怀疑：**那我的电机不是每 50 微秒被踹一脚吗？** 20 kHz 的通断，一秒钟两万次。而转子是一个有质量的物体。它凭什么不抖？

答案是：它确实被踹了两万次，但**踹它的能量几乎全部被两级积分器吸收了**。这两级积分器一个是铜和铁做的，一个是钢和铝做的。这一章把这两级算出来。

“agonizingly slow”

埃隆·马斯克（Tesla / xAI；人形机器人 Optimus 的公开发言者） — 马斯克就 Optimus 早期产能爬坡对投资者的警告（Business Insider 综述） — 2026-07-24（英文原句，经第三方报道逐字引用；本书未核对录音或视频原文）



τ_e = L/R（零点几毫秒） → τ_m = J·R/k_t²（几毫秒） → 角度再积分一次（几十毫秒起）

每一级都比上一级慢 10–100 倍。所以「平均」不是近似技巧，是这台机器本身的结构。

图 09-1　两级积分器替你抹平斩波：电感滤电流，惯量滤角度

第一级：电感是一个电流积分器

第 08 章已经用了这条式子，现在把它当主角看：

```
V = L · di/dt        ⇔        i(t) = i(0) + (1/L)∫V dt
```

请注意右边那个积分号。**电流是电压的时间积分**。这意味着：

- 你给一个 50 µs 的 24 V 脉冲，电流只涨 `24 × 50e-6 / L`；
- 你要在一**个开关周期内**让电流跑完它最终要到的值，需要 L 无穷小；
- L 越小、周期越长，电流波形越接近“跟着电压走”；反之越接近“跟着电压的平均值走”。

定义一个时间常数：

```
τ_e = L / R
```

这是这个 RL 回路自己的节奏。只有当开关周期 T **远小于** τ_e 时，电流才来不及跟上瞬时电压，只能跟上平均值。工程判据是：

```
T ≤ τ_e / 10        ⇒ 电流纹波占平均值的比例 ≲ 10%
```

这条 1/10 不是行规，是两行代数。 半桥驱动一个 RL 加反电势的负载，一个开关周期里只有两种接法：上管导通时绕组两端电压是 V_bus，下管导通时绕组自己续流、两端电压是 0。设占空比 d、周期 T，稳态下 反电势 E = d·V_bus，平均电流 I = E/R。

```
导通期电流上升：Δi = (V_bus − E)·(dT)/L = V_bus·(1−d)·d·T/L
纹波峰峰值：     Δi_pp = V_bus · d(1−d) · T ÷ L
相对平均值：     Δi_pp ÷ I = (1−d)·T·R ÷ L = (1−d)·T ÷ τ_e
```

三件事一次说清了。**纹波最大的位置是 d = 0.5**（因为 d(1−d) 在那里取极值，本章上面说的就是它）；**相对误差的上界是 T ÷ τ_e**（d 趋 0 时取到，因为那时 (1−d) ≈ 1）；所以要让纹波在任何占空比下都不超过平均电流的 10%，就必须 **T ⩽ τ_e ÷ 10**—— 这条判据的全部内容就是“最坏那一格也不超过 10%”，不是什么行业默契。

（顺带解释了一个常见误解：纹波电压摆幅与相对纹波不是一回事。绝对纹波在 d = 0.5 最大，相对纹波却在 d 小的**低速**轻载区最大（d 小对应反电势小，也就是转速低——别跑到额定转速去找最坏相对纹波）。工程上真正疼的是前者——它决定损耗和噪声；判据里用的却是后者，因为它决定“平均”这个说法成不成立。）

（这是本书给的一阶判据 [算]，不是任何厂商的规格。）

纹波本身可以算出来：

```
Δi ≈ V · d(1−d) / (L · f_sw)
```

它在 d = 0.5 时最大（因为 d(1−d) 在 0.5 取极值）。这条有个立刻可用的推论：**你的电流最脏的时刻，是输出电压用了一半母线电压的时候**，也就是转速跑到一半附近。

现在看第 08 章那个结论的另一面。**先把口径钉住**：下面把斩波当一阶 buck 算，`d = 1` 意味着反电势等于母线电压——这颗电机在母线上根本转不到那么快，所以这个 d 是**模型读数**，不是三相桥的物理占空比（真实桥臂占空比每个电周期两次过 0.5，与转速无关；第 08 章说的 12.78 V 是

13.86 V 那一档的 92%，两处的分母不同）。在这个前提下：M8010 反电势幅值在 30 rad/s 时是 12.78 V（第 08 章按幅值不变约定算的那个数），母线 24 V，所以正常工况下占空比就在 12.78 ÷ 24 ≈ 0.53 附近——**几乎正压在 d = 0.5 那个纹波最大的点上**。也就是说这颗电机在标称最高速附近运行时，恰好落在纹波较大的区间里。

（L 与 R 官方未公布 [未]，所以本书不能算出 Δi 的绝对值。但“d=0.5 附近最脏”这条与 L 无关，任何一台电机都成立。）

关键的一点：纹波不产生平均扭矩。 扭矩由 i_q 的**平均值**决定（第 04、05 章）。纹波只是在铜里发热、在铁芯里产生附加损耗、在声学上产生噪声。所以“提高开关频率让纹波变小”改善的是发热与噪声，**不是扭矩精度**。扭矩精度靠的是死区补偿与采样时刻（第 13 章）。

第二级：惯量是一个转速积分器

同样的逻辑再往下走一层。

```
τ = J · dω/dt        ⇔        ω(t) = ω(0) + (1/J)∫τ dt
```

转速是扭矩的时间积分。 转子（连同减速器、连杆、负载折算到轴上的部分）有一个转动惯量 J，它让转速不可能在 50 微秒里改变多少。

定义机械时间常数（把扭矩与转速用 k_t、k_e 连起来，忽略阻尼）：

```
τ_m = J · R / k_t²
```

（这是“给一个电压阶跃、转速到达稳态 63% 要多久”的一阶近似 [行]。）

代一个量级看看。M8010 转子侧 k_t = 0.10094 N·m/A。若转子侧惯量在 1e-5 kg·m² 量级（**这是本书假设的量级，官方未公布** [未]），R 取 **0.040 Ω**（全书统一口径；宇树两个数都没公布，本书把 0.040–0.050 Ω 当一条区间，第 36 章的热算保守地取上沿）——（第 08 章与 `code/tools/foc_check.py` 的 `R_JOINT_ASSUMED_OHM` 用的都是它）。**宇树从未**

公布任何一颗关节绕组的相电阻 [未]，所以 0.040 是本书的算例假设，不是厂商参数；上一版这一处写 0.05 Ω，与第 08 章和脚本不一致，已统一：

> `τ_m ≈ 0.00001 × 0.040 ÷ 0.10094² = 0.000039 s（即 0.039 ms）`

先说一件事：上一版这一行写的是 4.9 ms，错了 100 倍——真正跳掉的是 k_t^2 的指数：$0.10094^2 = 1.019 \times 10^{-2}$，被当成 1.019×10^{-4} 去除，商正好大 100 倍（写成“$10^{-5} \div 10^{-2}$ 那一步跳位”是错的解释：那么算出来是 5 ms）。改完之后本章的结论不但没塌，反而更清楚，因为正确的读法是这样三条：

一，裸转子其实和开关一样快。 0.039 ms 与开关周期 0.05 ms 是**同一个数量级** [算]（0.049 是旧版按 R = 0.05 Ω 算的，统一到 0.040 后是 0.039）。所以“转子对浪花的感觉像万吨轮”这个比喻，对**不带负载的电机本体**根本不成立——它慢不下来，是别的东西让它慢的。

二、这条式子是折算不变量，换侧救不回来。 把 J 乘 i^2、k_t 乘 i，分子分母一起乘 i^2：$0.00001 \times 6.33^2 \times 0.040 \div (0.10094 \times 6.33)^2$ = **还是 0.000039 s** [算]。想靠“折到输出轴”把 τ_m 变大是个常见错觉——减速比不会改变这个常数，它只是把同一份惯量搬到另一格里记账。

三、毫秒级来自负载，不来自转子。 拿一颗“速比 22.5、转子常数仍取 M8010 那 0.10094”的关节代（**这是拼接：N7520-22.5 自己的转子侧 k_t 官方未公布** [未]；若改用它的 139 N·m ÷ 40 A = 3.475 当关节侧 k_t，下面两个数变成 0.083 ms 与 1.1 ms，差约 2.3 倍）：关节侧 $k_t = 0.10094 \times 22.5 = 2.271$ N·m/A，把官方 `armature` = 0.0251 kg·m²（附录 C.2a 已证它是搬到输出轴的折算惯量 [厂]）当 J ⇒ $\tau_m = 0.0251 \times 0.040 \div 2.271^2$ = **0.19 ms**；再叠上小腿与大腿折算到膝的负载惯量（本书取 0.30 kg·m² 这一量级，官方未公布 [未]）⇒ $(0.0251 + 0.30) \times 0.040 \div 2.271^2$ = **2.5 ms** [算]，$1 \div \tau_m = 397$ rad/s ≈ **63 Hz**。**这才是“毫秒级”三个字的出处**：它是带负载的关节，不是电机。

于是本章那条级联判据的正确说法是：**开关 0.05 ms、电流环 τ_e 零点几 ms、空载转子 τ_m 0.039 ms、带负载的关节 τ_m 约 2.5 ms**——“机械比电慢一到两个数量级”这句话，只有在你把负载算进去之后才对。

两级加起来，这台电机的响应结构是：

```
开关（0.05 ms）→ 电感积分（τ_e 零点几 ms）→ 电流
                → 惯量积分（空载 0.039 ms / 带负载约 2.5 ms）→ 转速
                                          → 再积分 → 角度（几十 ms 起）
```

每一级都是一个积分器，两级串联，就把 20 kHz 的方波抹成了一个几乎平滑的转速——但这句话只在带负载时成立：空载转子的 τ_m（0.039 ms）与开关周期同量级，从开关到 τ_e 也只有 4–10 倍而不是 10–100 倍。真正慢的那一级是负载惯量给的。 这就是“平均”合法的完整理由——不是近似技巧，是这台机器本身的物理结构。

那频率为什么不能无限高

因为第二笔账会来收钱。

```
开关损耗 P_sw ≈ ½ · V · I · (t_rise + t_fall) · f_sw      ← 正比于 f_sw
导通损耗 P_on = I² · R_ds(on)                               ← 与 f_sw 无关
```

提高 f_sw 让纹波变小（∝ 1/f_sw），但让开关损耗线性变大。 两条曲线一交，就有一个最优点。这个点具体在哪，取决于器件的 t_r + t_f、母线电压、电流、以及你能带走多少热（第 36 章）。

机器人关节选 20 kHz 左右（行业常见量级 [行]；**宇树未公布任何一款关节的载波频率** [未]），除了电气考虑，还有一个非常不“物理”但非常真实的原因：

20 kHz 刚好在人耳听觉上限之外。

10 kHz 以下的 PWM 会让电机发出清晰的啸叫。一台 23 个关节的人形在安静的办公室里以 10 kHz 开关，听起来像一墙老式 CRT 显示器。这不是性能问题，是产品问题——但它是那种**会在评审会上被提出来、并且必须**

改的问题。所以“为什么是 20 kHz 而不是 8 kHz”这个问题，一半答案是电气的，一半答案是市场部的。

一个立刻能用的推论：你能控制的最快的事，就是电流

两级低通还给出一个反向结论，它是第 12 章级联结构的地基：

积分器不能对被积的东西做快的反应。电流环能跑 1–2 kHz，因为 τ_e 是零点几毫秒；速度环比电流环慢一到两个数量级，因为**带负载的关节** τ_m 约 2.5 ms（裸转子只有 0.039 ms，见上面那条更正——所以速度环慢的原因是负载，不是电机）。**注意不要把 1/τ_m 当成速度环的带宽**：1 ÷ 3.15 ms = 317 rad/s ≈ 50 Hz，那只是**开环植物极点**的位置；闭环带宽是增益决定的，可以高于它——**本书手上就有一个全是厂商数的一手例子**：官方训练仓库 mimic 配置给 `kp = 99.098`，官方 `armature = 0.025101925 kg·m²`（第 18、40 章引过）[厂]，那么这颗关节在位置刚度下的固有频率是

```
ω_n = √(kp ÷ J) = √(99.098 ÷ 0.02510) = 62.8 rad/s ⇒ 10.0 Hz    [算]
想要临界阻尼，kd 应取 2√(kp·J) = 3.15 N·m·s/rad                [算]
```

换掉 kp，ω_n 就跟着换，τ_m 一点没动（官方那个 kd 是不是 3.15，本书未核到 [未]）。所以“τ_m = 3 ms”与“速度环跑一百多赫兹”从来不矛盾：**前者是植物的时间常数，后者是环路的工作点**；第 12 章那条 5–10 倍级联判据约束的是相邻两环闭环带宽之比，不是任何一环与 τ_m 的关系；位置环只能跑几十赫兹，因为连杆与负载更大。

所以当你想“让关节反应更快”，唯一有效的方向是**让电流变化更快**——提高 f_sw、减小 L（但 L 小纹波大）、提高母线电压（但器件耐压与绝缘要跟上）。**没有任何算法能让你绕过 τ_m**。这条判据能帮你在选型会上省掉很多废话。

类比，以及它在哪儿破

类比：用一把脉冲式水枪推一辆水车。你每秒扣两千次扳机，水车只觉得有一股稳定的力在推它——因为水车的转动惯量把每一次冲击都摊平了。

这个类比在哪里破两处。第一，水车的阻力矩与转速无关，而电机的反电势**正比于转速**——所以“水车越转越难推”这件事在电机里是真实存在的，而且是第 08 章那个电压包络的全部来源。第二，水枪的脉冲不会让水车发出声音，而 PWM 会让电机啸叫——机械结构不但不完美积分，还会把某些频率放大（那就是第 22 章的结构模态）。

一个可检验的说法：PWM 斩波之所以不引起机械振动，是因为电流对电压、转速对扭矩构成两级串联积分器，其时间常数分别约为 $\tau_e = L/R$（零点几毫秒量级）与 $\tau_m = J \cdot R / k_t^2$（**裸转子 0.039 ms、带负载的关节约 2.5 ms**；本书上一版这一行写的 4.9 ms 是指数进位错，正文已更正）；带负载时机械那一档比开关周期慢约 50 倍（2.5 ms ÷ 0.05 ms），而 τ_m 是**折算不变量**、换到输出侧不会变大，所以“机械比电慢”的支撑是负载惯量而不是减速比；因此任何“让关节反应更快”的需求只能从电流环往上解决，不可能通过算法绕过 τ_m，也不可能靠换减速比绕过去。

证据等级：$V = L \cdot di/dt$、$\tau = J \cdot d\omega/dt$、两级积分结构、纹波公式、开关损耗正比于频率，全部是电路与控制通识 [行]；τ_m 的三个值（裸转子 $0.00001 \times 0.040 \div 0.10094^2 = 0.000039$ s（即 0.039 ms）、只算官方 `armature` 的 0.19 ms、叠上 0.30 kg·m² 负载的 2.5 ms）是 [算]，其中**转子 J、R 与那条 0.30 kg·m² 的负载惯量都是本书假设的量级，官方未公布** [未]（`armature` = 0.025101925 kg·m² 那一档取自官方训练仓库 [厂]），所以它们只能当“数量级感觉”用，不能当 M8010 或 G1 膝关节的参数引用；**上一版本节把第一个值印成 4.9 ms，是 k_t^2 的指数被当成 10^{-4}，已**

在正文连同两条自查一起更正；20 kHz 是行业量级 [行]，宇树未公布 [未]；“d = 0.5 时纹波最大”与“反电势 12.78 V ⇒ 正常工况 d ≈ 0.53”是 [算]。

可核对清单：

- ☐ 自己代一次 τ_m：先拿你手边任何一颗**电机本体**的 J、R、k_t——多半落在**几十微秒**而不是毫秒；再把你要驱动的那块惯量加进 J，看它什么时候才升到毫秒级。做完这一格你就知道“机械比电慢”到底慢在谁身上，而不在减速比身上
- ☐ 用 $\Delta i \propto 1/f_{sw}$ 与 $P_{sw} \propto f_{sw}$ 画两条线，标出它们的交点，那就是你能选的最高频率
- ☐ 找一段 8–12 kHz 的 BLDC 驱动音频（开源社区有），亲耳听一下“为什么是 20 kHz”
- ☐ 在宇树电机页面搜“switching frequency / 载波 / PWM 频率 / 电感 / 相电阻 / 转动惯量”，确认这六项是否全部查不到（预期：是）

下一章：两级低通让“平均电压”成立，于是 i_q 可以被精确控制。但 i_q 只是电流矢量的一个分量——另一个分量 i_d 我们一直让它等于零。为什么？什么时候不该让它等于零？那一章的答案关系到膝盖能不能在高速下还保住扭矩。

第 10 章

Clarke 与 Park：把交流电机在数学上变成直流电机

你没法给一个旋转的东西设定“大小”

前三章把链条接到了这里：

> 要多少扭矩 → 要多大的电流矢量、指哪个方向 → 三相电流各多少 → 开关怎么配时

还剩一个洞：**控制器怎么知道“现在该给哪三个数”？**

难处在于，那三个数是随时间正弦变化的。扭矩不变时，它们也在动；转得越快，动得越快。你要用一个 PI 环路去跟踪一条正弦曲线，就得给这个环路非常高的带宽——否则误差永远追不上。

这就是交流电机控制的全部困难所在：**在静止坐标系里，你要控制的东西一直在转**。

而解决办法只有一句话：**换一个跟着转的坐标系看它**。

“human and then superhuman dexterity”

埃隆·马斯克（Tesla / xAI；人形机器人 Optimus 的公开发言者） — 马斯克在财报电话会议上讲 Optimus 手部设计（Business Insider 综述） — 2026-07-24（英文原句，经第三方报道逐字引用；本书未核对录音或视频原文）

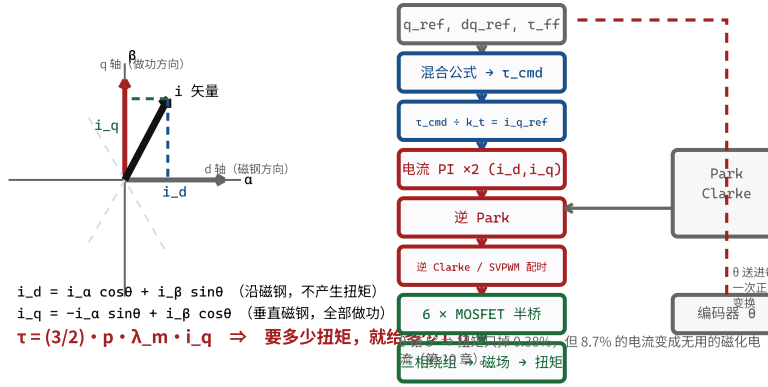


图 10-1　磁场定向 = 一次坐标旋转；转完之后是两个直流量、两个 PI

那把跟着转的椅子

你坐在旋转木马上，朋友站在地面向你扔一个球，球绕着你匀速转。在地面坐标系里，球的位置是 (cos ωt, sin ωt)，你得不停地算三角函数。

现在你坐到球上一起转。在你眼里，球是静止的，位置是一个常数。

这就是 Park 变换。它做的唯一一件事是：把静止坐标系 (α, β) 里的矢量，投影到一对随转子电角度 θ 旋转的轴 (d, q) 上：

```
i_d =  i_α·cos θ + i_β·sin θ        ← 沿转子磁钢方向的分量
i_q = −i_α·sin θ + i_β·cos θ        ← 超前磁钢 90° 的分量
```

（这就是一次二维旋转，没有任何新东西。符号约定各家不同，本书用的是“把静止矢量旋到转子坐标系”的标准形式。）

自检（`foc_check.py park_rotation()`）：θ = 0 时，(α, β) = (0, 10) 的矢量落在 (i_d, i_q) = (0, 10)。d 轴与 α 轴重合，所以 β 上的矢量与它垂直，全部是 q 分量。对。

关键的一步是：θ 从哪来？ 从编码器。这就是整条链上最脆弱的一环，本章末尾会回到这里。

转完之后，世界突然安静了

在转子坐标系里看，永磁体的磁场永远是静止的，方向永远是 d 轴。于是第 05 章那个“要维持垂直”的要求，变成了一句不用跟踪任何东西的常数：

```
要扭矩  ⇒ 只要 i_q
不要额外磁化 ⇒ 让 i_d = 0
```

i_d 和 i_q 在稳态下都是直流。 直流不需要跟踪正弦，一个普通的 PI 环路就能把它钉在给定值上，稳态误差可以为零（因为积分项会一直累加直到误差消失）。

这就是磁场定向控制的全部威力，也是它名字的准确含义：

FOC 不是“控制磁场”，而是“把坐标系定向在磁场上”。坐标一转，交流电机在数学上就退化成了直流电机——于是你可以用控制直流电机那套最朴素的方法来控制它。

完整的执行顺序，一次中断里跑完（这是伪流程，不是任何厂商的代码）：

```
1. 采两相电流 i_a, i_b（第三相 i_c = -i_a - i_b）
2. Clarke：(i_a,i_b,i_c) → (i_α, i_β)
3. 读编码器角 θ，Park：(i_α,i_β) → (i_d, i_q)
4. 两个 PI：i_d_ref − i_d → u_d ；i_q_ref − i_q → u_q
5. 逆 Park：(u_d,u_q) → (u_α, u_β)
6. 逆 Clarke / SVPWM 配时 → 六个 MOSFET 的占空比
7. 回到第 1 步
```

注意第 4 步的输入 i_q_ref 从哪来——它就是“你要的扭矩 ÷ k_t”。第 04 章那条式子在整张图的第 4 步接入。所以这张图不是“FOC 的原理”，它是“电流如何成为唯一旋钮”的完整证明。

为什么这套东西要在 20 kHz 的节拍上跑

PI 环路的带宽上限，由它能多快改变输出、以及它多久能知道一次结果决定。

- 你能改的只有占空比，而占空比一个开关周期只能改一次 → **环路更新率 ⩽ f_sw**。
- 你能读的只有电流，而电流采样必须落在特定的时间窗里（下管导通、电流连续、避开开关沿）→ **一次测量要等半个到四分之一个周期**。
- 一个有纯延迟的一阶对象，可稳定的环路带宽大致在“延迟的倒数除以几”这个量级。

工程上的通行取值是：**电流环带宽 ≈ f_sw / 10 到 f_sw / 20** [行]。20 kHz 载波 ⇒ 电流环 1–2 kHz。

这个数字后面还要用两次（第 12 章的带宽阶梯、第 18 章的刚度上限）。现在先记住它的来源：它不是谁规定的，它是“你每秒只能改几次占空比、只能测几次电流”的直接后果。

这条 “f_sw ÷ 10 到 ÷ 20” 是从哪来的：延迟吃相位

第 09 章说电流环的上限由对象自己的节奏 τ_e = L/R 定，这一章又说约等于 f_sw ÷ 10 到 f_sw ÷ 20。两个数说的是同一件事，但约束的不是同一个东西——得说清哪个才真正卡住你。

τ_e 决定“你要费多大力去推”：电流环要压的对象是一个一阶惯性环节，极点落在 1/τ_e。想把闭环带宽推到极点之外就得靠增益硬顶，代价是对噪声的放大。这是“值不值得”的问题。

延迟决定“你能不能站住”：数字环里至少两段纯滞后。① 占空比在本周期写入、周期内平均生效 ⇒ 平均滞后 T_sw/2；② 采样、算完、下一次写入 ⇒ 再吃一个 T_calc。合计 T_lag = T_sw/2 + T_calc。纯滞后在频率 ω

处吃掉 ω·T_lag 弧度，**没有上限**——频率越高吃得越多。 电流环通常要留 45°（0.79 rad）以上的相位裕度，而被控对象那一极已经先拿走接近 90°， 于是留给滞后的预算只有 0.3–0.4 rad：

```
ω_c · T_lag ≤ 0.35   ⇒   ω_c ≤ 0.35 ÷ T_lag
```

拿 20 kHz 开关频率、计算耗时 25 µs 的关节驱动板算（T_sw = 1 ÷ 20 kHz = 50 µs，半周期 25 µs）：

```
T_lag = 25 + 25 = 50 µs
0.35 ÷ 50e-6 = 7,000 rad/s
7,000 ÷ 6.283 = 1,114 Hz  ⇒  约 1.1 kHz
```

两条理由给出的上限是同一个量级，但取小者才是约束。 而且它解释了“1–2 kHz”这句话的隐含前提： 开关频率要在 40 kHz 上下、计算滞后要压到十几微秒——不是随便一颗 MCU 都能拿到 2 kHz 电流环。 这也把第 12 章那句“开关频率高，所以能调更大的 kp”从口号变成推导：f_sw 翻倍 ⇒ T_sw/2 减半 ⇒ 允许的 ω_c 抬高 ⇒ 扭矩环更快 ⇒ 第 18 章那个软件弹簧才更硬。 反过来，第 18 章 ω_n ⩽ ω_电流环 ÷ 5 里那个“除以 5”也有了出处：位置/速度环必须离电流环的 穿越频率足够远，否则它看到的不是一个听话的扭矩源，而是一个相位裕度正在被延迟啃掉的谐振体。

（45° 最小相位裕度是控制教材通行取值 [行]；**0.35 rad 这个预算是本书取的设计值**——被控对象那一极先拿走约 90°，若只留 45° 裕度，理论上可用到 0.79 rad，本书再让给 PI 零点与谐振约 20°，取 0.35 [算]；T_calc = 25 µs 与 f_sw = 20 kHz 是本书假设， 宇树未公布这颗驱动板的开关频率与中断耗时 [未]。示波器量一下 PWM 周期与“采样到写入”的间隔， 这两格就从算例变成实测。）

反过来说：θ 错了会怎样

第 05 章算过那张表，这里补上工程后果。编码器安装角偏了 δ，控制器以为的 d/q 轴和真实的 d/q 轴之间就差了 δ。你给 i_q_ref = 40 A，实际注入到真实 q 轴的只有 40·cos δ，剩下的 40·sin δ 变成了磁化电流。

δ	扭矩保留	无用电流	同扭矩下电流放大	铜损增加
1°	99.985%	1.7%	×1.0001	+0.03%
5°	99.62%	8.7%	×1.0038	+0.77%
15°	96.6%	25.9%	×1.035	+7.2%
30°	86.6%	50.0%	×1.155	+33%

（`foc_check.py angle_error_cost()` 加一列 $1/\cos\delta$ 与 $(1/\cos\delta)^2 - 1$。）

这张表最重要的信息是：**5° 以内的装配误差，在“扭矩够不够”这一项上几乎测不出来**。一台装配偏了 5° 的关节，能通过所有出厂测试，只会在客户手里连续满载时比别人热几度，然后磁钢慢慢退磁，k_t 慢慢下降，控制器为了维持扭矩继续加大电流，更热。这是一个正反馈，起点是一次装配。

所以真实的关节产品里，“零点标定”是一道工艺，不是一段代码：把转子锁定在已知电角度上，读编码器值，把这个差写进 Flash。这道工序做没做、做在产线还是做在装配后、精度多少，宇树全部未公布 [未]。

顺带解释一个你可能见过的现象：无刷电机有三根相线，接错任意两根会反转。为什么？因为你交换了两相，等于把合成的电流矢量镜像了一次，而控制器不知道——它算的 θ 还是原来的 θ。所以一个“跟着转的坐标系”如果转错了方向，你给的所有正扭矩都会变成负扭矩。**FOC 的全部正确性都建立在“我知道 θ”这一个假设上，而这个假设是会被装配、接线、标定破坏的**。

类比，以及它在哪儿破

类比：你没法用望远镜追一只悬停旋转的蜻蜓并量它的速度。于是你造一把椅子，以蜻蜓的角速度转，坐上去，它就是静止的——现在你可以慢慢量。Park 变换就是那把椅子，编码器就是告诉你该多快转的那个人。

这个类比在哪里破三处。第一，蜻蜓会自己变速，椅子必须跟着变，所以“转多快”这件事本身是实时输入，一旦它滞后，蜻蜓又动了。第二，你坐上去之后量到的“静止”是二维的两个数（d、q），而不是一个数——这两个数各自有各自的物理意义，一个做功一个不做功，这正是第 11 章的主题。第三，也是最要紧的一处：如果那个人报错了角度，你会以为蜻蜓静止、其实它在绕着你转，而且**你从椅子上完全看不出来**。

一个可检验的说法：磁场定向控制的本质是一次坐标旋转，它把“跟踪三相正弦”变成“稳住两个直流量”；因此电流环的可用带宽上限由开关频率决定（约 f_sw/10–f_sw/20），而整个方案的正确性完全依赖电角度 θ 的准确性——5° 的角偏只损失 0.38% 扭矩，却把 8.7% 的电流变成无用的磁化电流，且这一损失无法通过扭矩测试发现。

证据等级：Clarke/Park 变换式、dq 模型、PI 稳态零误差、电流环带宽与 f_sw 的关系，全部是电机控制通识 [行]；角度误差表是 [算]（纯三角函数，`foc_check.py`）；“宇树未公布载波频率、死区、零点标定工艺”是 [未]；“两相采样、第三相算出”是行业普遍做法 [行]。

可核对清单：

- ☐ 跑 `foc_check.py` 的“10 角度误差 5°”，确认 cos5° = 0.9962、sin5° = 0.0872
- ☐ 自己代一次 Park：θ = 90°、(α,β) = (0,10) 时应该是 (i_d,i_q) = (10,0)，确认“矢量与 d 轴重合时 q 分量为零”
- ☐ 数一遍那七步伪流程里，哪几步用到了 θ（第 3、5 步），从而确认“θ 错 ⇒ 变换和逆变换都错，且互相掩盖”

- [ ] 在任意一份开源电机固件（moteus、ODrive、SimpleFOC）里找到它的 Clarke/Park 实现，看它用的是 2/3 还是 1 的系数，再对照它的扭矩公式系数——验证“约定必须成对出现”

下一章：i_d 和 i_q 现在都清楚了。但为什么 i_d 一定要等于 0？什么情况下我们故意让它不等于 0，甚至让它为负？那个答案关系到人形机器人膝盖能不能在高速下还保住扭矩。

第 11 章

i_q 管扭矩，i_d 管磁场：最大扭矩每安培与弱磁

两个直流量，一个做功一个不做功，那为什么要留第二个

第 10 章把三相交流变成了转子坐标系里的两个直流：i_q 产生扭矩，i_d 不产生（对面贴式永磁而言）。

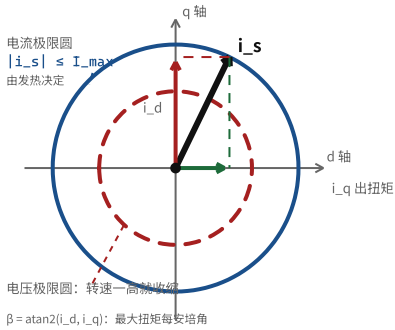
那为什么还要留 i_d 这个自由度？为什么不干脆把 d 轴那一路硬件省掉？

因为 i_d 是**唯一能改变磁场本身强度的旋钮**。i_q 只能在既有磁场下产生力，而 i_d 能把那个磁场加强或者削弱。在绝大多数工况下你不需要这个能力，所以默认 i_d = 0。但有两种工况你需要，而其中一种，人形机器人的髋关节会遇到。

“$20,000 to $25,000”

埃隆·马斯克（Tesla / xAI；人形机器人 Optimus 的公开发言者） — 马斯克对量产规模下 Optimus 单台成本的估计（Business Insider 综述） — 2026-07-24（英文原句，经第三方报道逐字引用；本书未核对录音或视频原文）

定子电流矢量被拆成两个互相垂直的分量：q 轴那一份出扭矩，d 轴那一份只改磁场。这一步是纯数学旋转，不耗能——它只是换了一双眼睛看同一组三相电流。



表贴式（L_d = L_q）

$\tau = 1.5 \cdot p \cdot \lambda_m \cdot i_q$

⇒ 扭矩只由 i_q 决定，i_d 不出力。

代入 GO-M8010-6

$k_t = 23.7 \div 40 = 0.64\ N{\cdot}m/A$

⇒ 要 12 N·m 就给 i_q ≈ 18.8 A；这 18.8 A 与转速无关，只与是否垂直有关。

内置式（IPM）多一项磁阻扭矩

$\tau = 1.5\ p\ [\lambda_m\ i_q + (L_d - L_q)\ i_d\ i_q]$

⇒ 负的 i_d 反而加扭矩，这才是 MTPA 角不为零的原因。

代价：电压极限圆把电流圆吞掉时，只能加大负 i_d 削弱磁场（弱磁），用发热换转速。人形关节不需要超基速，所以几乎不用弱磁。

图 11-1　d-q 平面：电流极限圆、收缩的电压极限圆、i_q 出扭矩与 i_d 改磁场

默认为什么是 i_d = 0

三个理由，都很硬。

一，同样的电流出同样的扭矩。 电流矢量的总幅值受逆变器与绕组限制：

$$i^2 = i_d^2 + i_q^2 \ \le\ I_{max}^2$$

若 $i_d \neq 0$，则 $i_q = \sqrt{(I_{max}^2 - i_d^2)} < I_{max}$。**一部分宝贵的电流预算被拿去做不产生扭矩的事。** 在“每一安培都要散热”的密封关节里（第 13、36 章），这是直接的寿命损失。

二，控制更简单。 $i_d = 0$ 时，电压方程退化成两个近似独立的一阶系统，两个 PI 各管各的，不需要解耦项。一旦 $i_d \neq 0$，d 轴电流会通过交叉耦合项影响 q 轴电压（$\omega \cdot L_q \cdot i_q$ 与 $\omega \cdot \lambda_m$ 那两项），你得前馈解耦，否则高速时两个环路互相干扰。

三，**磁路安全**。正的 i_d 是增磁，会把磁钢往饱和里推；饱和之后 k_t 不再常数，第 04 章那条线性关系失效，而且失效的方式是“你以为还能多出扭矩，实际不长了，只发热”。所以工程上几乎从不主动增磁。

结论：**面贴式永磁 + i_d = 0，就是最大扭矩每安培（MTPA）的解**。这不是妥协，这就是最优。

内嵌式电机：让 i_d 为负反而更好

但如果转子是内嵌式（IPM），故事变了。第 05 章写过那条完整式子：

```
τ = (3/2)·p·[ λ_m·i_q + (L_d − L_q)·i_d·i_q ]
              └ 永磁项 ┘   └── 磁阻项 ──┘
```

内嵌式的意思是磁钢埋在铁芯里，于是 d 轴方向（穿过磁钢）的磁阻比 q 轴方向大，也就是 **L_d < L_q**（注意下标方向与大小关系是反的，这是初学者最容易记错的一处）。那么 (L_d − L_q) 是负数，而 i_q 是正数——**要让磁阻项为正，i_d 必须是负数**。

负 i_d 的物理含义是去磁。它削弱永磁磁场（牺牲一点永磁项），换来一个磁阻项。净效果：**同样的电流幅值能出更多扭矩**，最优 i_d 不再是 0 而是一条随扭矩变化的曲线。这就是 MTPA 在 IPM 上真正需要“算”的原因。

代价：磁阻项不是线性的（L_d、L_q 随饱和变化），所以它的 k_t 等效值会随工况漂。第 04 章那条 τ = k_t·i 在 IPM 上比在 SPM 上更容易失真。

宇树未公布它的关节电机是 SPM 还是 IPM [未]。官方只说“低惯量高速内转子永磁同步电机” [厂]——请再看一次这句话：**“内转子”讲的是转子在轴上还是轴在转子内，与 SPM/IPM 是两个正交的分类**。很多中文资料把“内转子”和“内嵌式”混为一谈，这是错的。本书不猜。

弱磁：高速时唯一能救扭矩的办法

（先把名字说清：本节说的**弱磁**，是指故意注入一路去磁电流、把永磁体在气隙里的有效磁场削弱，从而把母线电压中原本被反电势占住的那部分腾出来给电流用。它换到的是转速上限，付出的是扭矩。）

现在讲那个真正会用到的场景。

第 08 章算过：M8010 在 30 rad/s（输出端）时反电势幅值是 12.78 V，而 24 V 母线在 SVPWM 线性区能给 13.86 V。**空载刚好够（用掉 92%），但只要带上额定电流就不够**——40 A 在回路电阻上再吃掉约 1.6 V，需求一下变成 14.38 V > 13.86 V。**电压不够**。电压不够时，电流环饱和，i_q 给不上去，扭矩掉到零，转速再也上不去。

在电压受限的高速区，有一条唯一的路：**把反电势本身降下来**。而反电势正比于永磁磁链 λ_m，λ_m 可以被负的 i_d 削弱。

```
E = (2/3) · k_t · ω = p · λ_m · ω   （幅值不变约定，见 A.1 的 k_e 行）
i_d < 0  ⇒  有效 λ_m 下降  ⇒  E 下降  ⇒  腾出电压给电流用
```

这就是弱磁（field weakening）。它的名字非常诚实：**你在削弱自己的磁场**。（弱磁 = 故意用去磁电流削弱永磁磁场，好把电压腾出来提转速；代价是同样电流出的扭矩变小）

代价是一条明确的交换：

	弱磁前	弱磁后
可用电压	顶到母线，用不完也没辙	腾出来了
可达转速	卡在 $E = V_{bus}/\sqrt{3}$	更高
同样电流出的扭矩	$k_t \cdot i_q$	**更小**（λ_m 被削弱，还可能饱和失控）
铜损	I^2R	同电流下不变，但白烧在 d 轴的部分涨了
退磁风险	无	**有**——过强的去磁电流在低速时可能把磁钢永久退磁

最后一条是弱磁真正的工程难点：**去磁电流在高速时是救星，在突然失速时是凶器**。如果一台正在弱磁运行的关节突然被卡住（摔了、撞到、落地锁死），转速骤降而 i_d 还是那个大的负值，你就是在主动给磁钢施加一个反向磁场——而这时候没有任何反电势在对抗它。**结果是局部不可逆退磁**。所以真实的弱磁控制器必须有“转速掉下来立刻退出弱磁”的保护逻辑，而且退出速度要快过机械减速。

人形关节需要弱磁吗

这是本章唯一有宇树答案的问题，而答案是“大概不需要，但有一个位置可能需要”。

看官方给的转速档位：

> M8010：输出端 30 rad/s（约 286 rpm）
> IM6014：输出端 54.2 rad/s @60 V（约 518 rpm，54.2 × 60 ÷ 2π）
> Z1：关节速度 180°/s（约 31 rad/s）

（全部 [厂]。）

人形的关节不需要高转速——它需要的是扭矩和位置精度。30–54 rad/s 的输出端转速，对应转子侧（乘减速比）是 190–686 rad/s，也就是 1800–6500 rpm。**这个转速区间正好是弱磁开始有意义的地方**：IM6014 官方把 54.2 rad/s 明确标在 60 V 下 [厂]，说明它是靠提电压而不是靠弱磁到达那个转速的。

所以更可能的答案是：**宇树用母线电压解决转速上限，用低减速比解决扭矩密度（扭矩密度 = 每牛米重量，第 15 章会给宇树的官方数），把弱磁留给确实需要的那几个高速小关节（腕、踝）**。这是推断，不是事实——**宇树从未公布任何一款关节是否实现弱磁** [未]。

（对照：轮式与云台类应用里弱磁是常规操作，因为那些电机转速上万 rpm。第 26 章会看到地瓜 RDK、达妙、moteus 这些开源/半开源器件的文档里弱磁是明写的。）

一个能带走的判断法

以后你看到任何一份关节参数表，问三个问题就能判断它的控制策略边界在哪：

1. **峰值扭矩时电流多少？** → 决定铜损与热设计（第 13、36 章）。
2. **最高转速时母线电压多少？** → 按本书声明的幅值不变约定做一次反电势检查：相电动势幅值 = k_t·ω（A.1 的 k_e 行），**再换算成手册常写的线电压有效值才是可比的**（√3 出现在这一步，不是那一步）。用哪个口径要在纸上写出来——两个口径混用会把这项检查放松 2.6 倍（第 08 章）。
3. **有没有写弱磁 / field weakening / flux weakening？** → 如果没写而转速又高，那它一定是靠电压撑的；如果写了，那它一定也有对应的退磁保护，问清楚触发条件。

这三个问题的答案，全部决定于厂商肯不肯公布 L、R、λ_m、母线电压范围这四个数。 宇树公布了一个（母线电压范围），另外三个 [未]。这就是这本书在第 1 部反复撞上的那堵墙。

类比，以及它在哪儿破

类比：i_q 是你踩油门的脚，i_d 是你手刹。正常开车手刹必须放开（i_d = 0），但有一种情况你需要故意拉一点手刹——下坡时控制车速（弱磁）。

这个类比破得比较彻底，所以只保留一句：手刹不会让车在平路上跑不快，而弱磁**会**——它是拿扭矩换转速上限。更准确的比喻是给一辆车装一个“高速时自动变小的发动机”：极速上去了，爬坡能力下来了。

一个可检验的说法：对面贴式永磁电机，i_d = 0 就是最大扭矩每安培解，理由是电流幅值受限、控制解耦最简单、且避免磁路饱和；对内嵌式，负 i_d 能通过磁阻项换到更多扭矩；弱磁是唯一能在电压耗尽后继续提速的

手段，它用扭矩与退磁风险换转速上限，因此必须配“转速骤降即退出”的保护。

证据等级：MTPA、dq 电压方程与交叉耦合项、磁阻扭矩、弱磁机理与退磁风险，全部是电机学通识 [行]；30 rad/s、54.2 rad/s @60 V、180°/s、“低惯量高速内转子永磁同步电机”是 [厂]；12.78 V（空载）与 14.38 V（满载）vs 13.86 V 是 [算]（第 08 章）；“宇树是否实现弱磁”“SPM 还是 IPM”是 [未]；“宇树用母线电压而非弱磁解决转速上限”是**本书推断**，正文已如此标注。

可核对清单：

- ☐ 确认 L_d < L_q 对应的是“磁钢埋在铁芯里”，并解释为什么下标方向与大小关系是反的
- ☐ 用 $i^2 = i_d^2 + i_q^2$ 算一次：若允许 i_d = −0.3·I_max，可用 i_q 剩多少（答：0.954，掉 4.6%）
- ☐ 在宇树电机与整机页面搜“field weakening / 弱磁 / flux”，确认是否出现（预期：不出现）
- ☐ 拿一份开源关节固件（moteus 或 ODrive）的文档，看它是否明写弱磁与退磁保护——那是“公开的那条路长什么样”

下一章：第 1 部到此把“一颗关节”讲完了。下一章开始往上建：23 个这样的关节装成一具身体，第一个必须回答的问题是——为什么两个支点比四个支点难得多。

第 12 章

三个环其实是一个环：电流 → 速度 → 位置的带宽递减律

你写了“走到 0.5 米”，机器里最后剩下的是“40 安培”

一个上层规划器说：右膝，1.2 秒后到 60°。

这句话进入关节之后发生了什么？十个人里有九个会答“电机转到那个角度”。这个答案不算错，但它跳过了全部关键内容。真实发生的事情是一条**逐级翻译链**，每一级都在解决上一级解决不了的问题：

```
"60°"（位置）
   ↓ 位置环：还差多少角度 → 该转多快
"1.8 rad/s"（速度）
   ↓ 速度环：现在转多快 → 该出多少力
"12 N·m"（扭矩）
   ↓ τ = k_t·i_q（第 04 章）
"18.8 A"（电流）
   ↓ Park + PI + 逆 Park（第 10 章）
"(u_d, u_q)"（电压矢量）
   ↓ SVPWM 配时（第 08 章）
“六个 MOSFET 的占空比”
   ↓ 死区、栅极驱动、母线（第 13 章）
“电流”
```

注意这条链的终点和起点：位置请求最终变成了电流，而电流是这台机器唯一的旋钮（第 04 章）。所谓“位置、速度、扭矩”三种控制模式，不是三台不同的机器，甚至不是三种不同的算法——它们是同一个电流给定外面套了几层壳。

只给电流给定 = 扭矩模式。再加一层速度环 = 速度模式。再加两层 = 位置模式。宇树电机手册里那三种模式，在这条链上就是“包了几层”的区别。

> “Progress on Optimus hardware and software continued in Q4, including the latest generation hand, robust locomotion and training on additional tasks, ahead of planned pilot production in 2025.”
>
> 埃隆・马斯克（Tesla / xAI；人形机器人 Optimus 的公开发言者） — 特斯拉 2024 年第四季度股东更新（公司自己发布的 PDF，逐字抽取） — 2025-01-29（特斯拉自己发布的股东更新原文（PDF 全文用 PyMuPDF 抽取，逐字核对））

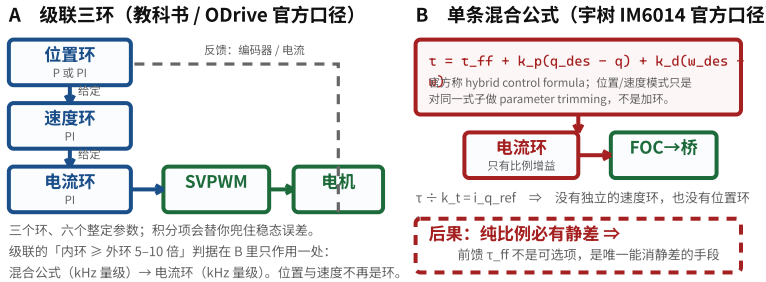


图 12-1　真级联三环（ODrive 口径）对单条混合公式（宇树 IM6014 官方口径）

为什么必须分三层，而不是一个大环直接算占空比

这是这一章真正要回答的问题。既然终点都是电流，为什么不写一个“角度误差 → 占空比”的函数，一步到位？

因为**这三个量的物理时间常数差了三到四个数量级**，而一个反馈环路的稳定带宽，不能超过它所控制的那个物理过程的速度。

被控量	载体	时间常数来自	典型量级
电流 i_q	绕组电感与电阻	τ_e = L/R	零点几毫秒 → 带宽 1–2 kHz
转速 ω	**负载**折算惯量（裸转子的 τ_m 只有几十微秒，见第 09 章）	τ_m = J/B 或 J/(k_t²/R)	毫秒级到几十毫秒 → 带宽几十到几百赫兹
角度 θ	负载惯量 + 减速器刚度	机械共振与摩擦	几十毫秒 → 带宽几赫兹到几十赫兹

（带宽数值是行业通识量级 [行]；宇树未公布任何一颗关节的 L、R、J [未]。）

如果你把三层合成一层，会发生什么？你会得到一个高增益的“角度→占空比”环路。而电流这个物理量对占空比的响应是**毫秒级**的，角度对它的响应是**几十毫秒级**的。你用毫秒级的执行手段去闭环一个几十毫秒的过程，中间那段时间里电流早已冲过目标值几十次，而你还没看见。结果就是超调、振荡、过流保护跳闸。

级联（cascade）结构的作用，是让快的过程先自己管自己，慢的过程只把它当成一个“已经完成了的执行器”来看。 电流环把“给多少电流”这件事做到几乎瞬时准确，速度环就可以假设“我下令要 12 牛米，就真的会有 12 牛米”，只管自己的事。

这条结构要求有一个可检验的形式：**内环带宽必须是外环的 5–10 倍以上。** 否则外环会把内环的动态当成未建模扰动，级联的假设就不成立，环路会互相干涉。

```
电流环 1–2 kHz   ÷ 速度环 100–200 Hz   ≈ 10×  ✓
速度环 100–200 Hz ÷ 位置环 20–50 Hz     ≈ 5×   ✓
```

这也解释了为什么“提高开关频率”这件事会一路影响到你的关节能有多硬：f_sw 提高 → 电流环带宽可以提高 → 允许速度环更快 → 允许位置环增益更大 → 关节能表现出的等效刚度上限更高。**刚度不是软件参数，它的地基是半导体开关速度。**

宇树把哪一层交给了你

现在可以去看真实的接口了。宇树 SDK 里给关节下指令的数据结构（话题 `rt/lowcmd` 里的 `MotorCmd`），官方 IDL 一共七列 [厂]：

```
uint8   mode      工作模式位
float32 q         目标角度
float32 dq        目标角速度
float32 tau       前馈扭矩
float32 kp        刚度
float32 kd        阻尼
uint32  reserve   保留
```

而在人形用的那套 `unitree_hg` 消息里，`motor_cmd` 与 `motor_state` 都是 **35 个元素的定长数组**——这一帧报文按最多 35 个关节设计，G1 的 23 或 29 个自由度都装得下 [厂]。

真正参与控制的是中间五列。请注意官方给其中两列起的名字：**kp 那一列在文档里叫 Stiffness（刚度），kd 那一列叫 Damping（阻尼）**。电机手册上控制指令列表的原文是 Torque, Angle, Angular Velocity, Stiffness, Damping [厂]；灵巧手文档里的反馈字段原文是 Joint Torque, Joint Stiffness Coefficient, Joint Damping Coefficient [厂]。**「关节是一个可编程的弹簧-阻尼器」不是本书的比喻，是宇树自己的字段名。**

还要小心一件事：官方在三个不同页面用了三套模式命名，本书不合并它们 [厂]。

出处	原文列出的模式
电机文档中心 mode.mode 位域（3 bit）	0 locked；1 **FOC closed-loop**；2 encoder calibration；3~7 reserved
电机文档中心 Control Mode 章	Position / Speed / Damping / Torque / Zero torque
电机产品页 Motor Control Instruction	Torque, Angle, Angular Velocity, Stiffness, Damping

产品页那份列表里**没有 FOC 这个词**；FOC 只出现在位域（1 = FOC closed-loop）和 About 页的 FOC driver board。所以不要把三张表合成一句「控制模式：位置 / 速度 / 转矩 / FOC」——那是把工作模式位、驱动器模式、可给指令三个不同层次的东西压成了一层。

顺便把四合一的官方原文也记下来，它是全书出现频率最高的一个结构：M8010 是 a brushless motor, reducer, single-turn absolute encoder, and FOC driver board；IM6014 是 a brushless motor, planetary reducer, dual encoders, and FOC driver board [厂]。**注意第一颗是单圈绝对值，第二颗才是双编码器**——这个差别第 14 章要用。（双编码器 = 电机端与输出端各装一颗角度传感器，用来把减速器的空程量出来）

关节 MCU 每个周期执行的是这一行：

```
τ_cmd = tau + kp·(q_ref − q_measured) + kd·(dq_ref − dq_measured)
```

然后 τ_cmd ÷ k_t = i_q_ref，进入第 10 章那七步。

这一行公式是整章的落点，也是理解现代机器人关节的钥匙。它旁边还有一个更能说明问题的字段：状态帧 `MotorState` 里报上来的不是「测到的扭矩」，而是 **`tau_est`——估计扭矩**；同一帧还带着 `temperature int16[2]`（两路温度）与 `vol`（母线电压）[厂]。一个名字叫「估计」的扭矩字段，本身就是「这颗关节输出端没有力矩传感器」的最强旁证（第 14 章会把它接上准直驱那条线）。（准直驱 = 用较低的减速比，让电机端电流还能代表输出端力矩）

请注意这一行意味着什么：

一，**位置环和速度环都在关节里跑，不在你的主控上**。整机侧的 `rt/lowcmd` / `rt/lowstate` 官方文档口径是 **500 Hz**，并且被 SDK 里的 `control_dt_ = 0.002`（2000 μs 线程）独立印证 [厂]；单颗电机的指令率

也就是上面算出的那个 kHz 量级。而电流环需要 1 kHz 以上才够用——所以最里面那圈闭环必须发生在关节本地，总线只负责改给定值。这是「低层必须在关节里」的最硬理由。

二，**这一行就是阻抗控制的全部实现**。$\tau = kp \cdot \Delta q + kd \cdot \Delta\omega$ 是一个弹簧加一个阻尼器。你把 kp 调大，关节表现得像硬；调小，像软；kd 提供耗能。所谓“可编程刚度”不是修辞，是这个式子里那两个真的可以改的数。（第 18 章会算 kp 的上限由什么决定。）

三，**tau 是前馈项，它不进任何环路**。这意味着上层可以直接绕过位置与速度，命令一个纯扭矩。这是所有“被推了一下先软后硬”这类行为的技术底座，也是把人摔伤的风险来源——一个能把 kp 设成 0、tau 设成 30 牛米的接口，本身就是一个安全问题（第 37 章）。

四，**注意这里没有 i_q**。你无法直接命令电流。你命令的是扭矩，而扭矩到电流的那一步（除以 k_t）在关节内部完成，k_t 是多少、有没有温度补偿、掉多少，全部未公布 [未]。换句话说，宇树把第 04 章那条式子封在了黑盒里，把它的输入端（τ）留给了你。

一个必须更正的说法：宇树官方明说它不是级联三环

上面那张级联图是**教科书结构**，也是 ODrive 这类开源驱动器**真正在用的**结构。但把它安到宇树头上，是错的——而宇树自己在文档里说得非常明确。

ODrive 的官方口径（`docs/control.rst`）[查]（他方一手：这是 ODrive 自己的文档，不是宇树）：

> The motor controller is a **cascaded style position, velocity and current control loop**.

它是真级联：位置环 P、速度环 PI、电流环 PI，三环嵌套，环路周期恒定 125 μs（8 kHz）。

宇树 IM6014 的官方口径（文档中心）[厂]：

The "Position Mode" and "Velocity Mode" of the Unitree IM6014 are **not the traditional cascaded three-loop architecture of servo drives**, but rather **parameter trimming based on a single hybrid control formula**.

以及：

the IM6014 motor's underlying layer only provides a **current loop interface with proportional gain**；驱动 internally only performs **proportional operations and torque synthesis**, then directly hands it over to the **FOC current loop** for execution.（顺带把第 10 章留的那个坑填上：官方口径是电流环**只有比例项**。纯 P 控制器消不掉一个持续存在的扰动，而这个扰动恰好就是反电势 (2/3)·k_t·ω——它随转速线性涨。后果是：转速越高，同一个电流给定换来的实际扭矩越偏低；第 08 章算过 24 V 母线下满扭矩时只剩 1.08 V 余量，所以这个偏差在高速段不是小数。这就是“P-only 换带宽”的代价，官方没有公布它怎么补[未]。）

把这两段话拆开看，宇树的架构是三件事：

> ① 只有一条公式：
>
> ```
> τ = τ_ff + k_p(q_des − q) + k_d(ω_des − ω)
> ```
>
> 官方称之为「混合控制公式」（hybrid control formula）。
>
> ② 所谓位置模式、速度模式，只是把这条公式里
> q_des / ω_des / τ_ff 三个给定值按不同方式填数
> —官方原话是 parameter trimming（参数微调），不是加环。
>
> ③ 公式算出 τ 之后，除以 k_t 得到电流给定，
> 交给 FOC 电流环。而这一层官方说只有比例增益（P），
> 没有积分项。

于是本章前面那张图要改一个字：位置环与速度环不是“环”，是“一条公式里的两项”。 它们没有积分、没有抗扰、没有各自的带宽，因此也**不满足**“内环比外环快 5–10 倍”那个级联判据——因为它们根本不是嵌套结构。

那级联判据错了吗？没有，它换了位置

级联要求仍然成立，只是它作用在**另一处**：

```
真正的两级嵌套是：
   混合公式（在关节 MCU 里，随指令率跑）
        ↓ τ_cmd ÷ k_t = i_q_ref
   FOC 电流环（kHz 量级，第 10 章）
```

只有这两级，而且第二级确实是快得多的那一级。 位置与速度不再是环，它们退化成“给这条公式喂什么数”。

两种架构的取舍，可以并排讲

	真级联三环（ODrive 口径）	**单公式 + 电流环（宇树 IM6014 口径）**
位置环有没有积分	有（P 或 PI）	**没有**——位置误差只按比例出扭矩
速度环	独立 PI，有自己的带宽	**没有独立环**，k_d 只是公式里的一项
稳态位置误差	积分项可归零	**必然存在**（纯比例有静差），要靠 τ_ff 补
整定参数个数	6 个（三环各 P、I）	**3 个**（k_p、k_d、τ_ff）
抗扰与模型误差	强（积分吸收）	弱，**要求上层把前馈算准**（第 19 章逆动力学）
稳定性裕度	需逐级检查带宽分离	只有一级分离要检查，**更不容易自激**
适合什么	位置精度要求高的机床、点胶	力控与全身控制：**把误差换成扭矩，而不是换成修正**

这张表解释了第 18 章末尾那句话为什么在宇树身上格外重要：

前馈负责“应该出多大力”，反馈负责“实际差了多少”。

在真级联架构里，即使你前馈给得很差，位置环的积分项也会慢慢把误差吃掉——**它能替你兜底**。而在单公式架构里**没有积分兜底**：前馈给错，稳态位置误差就永久留在那儿，直到你把 k_p 提高（然后撞上第 18 章那三道天花板）。

所以“宇树为什么把 τ_ff 单独列成一个字段、并且把它放在指令结构里”——因为在这套架构里，**前馈不是可选项，它是唯一能消除静差的手段**。而它要求上层必须有一个足够准的逆动力学模型（第 19 章）。

这是一个非常干净的因果链：架构选择（不用级联）→ 静差无处可消 → 必须给前馈 → 必须有逆动力学模型 → 模型不准就抖。第 19、22 章的所有麻烦，有一部分就是从这条链的第一环长出来的。

（**必须标注的边界**：上面那段官方原文是 IM6014 的文档。G1/H1/H2 用的关节电机是否同一架构，官方未说明 [未]。而 `unitree_rl_lab` 的 G1 配置里 `control.control_type = 'P'`（位置型混合控制）[厂]，与 IM6014 的口径一致，是间接旁证，不是直接证据。）

类比，以及它在哪儿破（补一条）

原来那个“方向盘→油门→喷油嘴”的三级类比，现在要打个补丁：**宇树的车没有独立的油门伺服和方向伺服，它只有一张公式：目标角度与当前角度差多少、目标速度与当前速度差多少，两笔加起来直接算出该踩多深**。而它也不会自己慢慢修正——**你必须在踩下去的那一刻就把该用的力气算对**。

这个类比在哪里破：司机可以凭感觉补，而关节不能——它只有你填进 τ_ff 的那个数。

那 6000 Hz 到底是什么

这里必须做一次祛魅，因为厂商参数表上的这个数字极易被误读。

GO-M8010-6 官方标「通信控制频率 6000 Hz」、「RS-485 @4 Mbps、一总线最多 15 台」 [厂]。听起来像「每个关节每秒刷新 6000 次」。

这次不用估算——官方文档中心给了**真实帧长**：控制帧 17 字节（CRC 覆盖前 15 字节）、状态帧 16 字节（CRC 覆盖前 14 字节）、电机 ID 0~14 而 15 是广播且无响应、多字节小端 [厂]。拿它做带宽算术（`foc_check.py bus_budget()`）：

```
下行  15 台 × 6000 Hz × 17 B × 8 = 12.24 Mbps
上行  15 台 × 6000 Hz × 16 B × 8 = 11.52 Mbps
合计  23.76 Mbps ÷ 官方 4 Mbps = 5.9 倍
```

差 5.9 倍，全双工全量交换不可能。 把算术反过来做，答案更有意思：

```
4 Mbps ÷ ( 15 台 × (17 + 16) B × 8 ) = 1010 Hz
```

4 Mbps 的 RS-485 挂满 15 台、全双工，物理上限就是约 1 kHz。 而宇树自己在 Z1 机械臂的产品页上把 Control Frequency 标成 1 kHz [厂]——两条来自不同页面、不同年代的官方数字，被一次除法接在了一起。这不是巧合，是同一条带宽约束在两个产品上的表现。

同一套算术用在 IM6014 上（官方帧长 20 B 下行 / 26 B 上行，6 Mbps，标称最大轮询 10000 Hz）：15 台全双工需要 55.2 Mbps，是 9.2 倍；反过来 6 Mbps 最多支撑 1087 Hz [厂][算]。方向完全一致。

这条推算仍有一个诚实的限制：「**通信控制频率**」**这个中文词组，官方页面没有给严格定义** [未]。所以本书不下「宇树标错了」的结论，只给两条判据：一，任何一体化关节的「通信频率」和「电流环频率」是两个不同

的东西，看到前者不要当成后者；二，如果它被读成「15 台同时全双工 6 kHz」，那就与官方自己公布的帧长和波特率互相矛盾，而读成约 1 kHz 不矛盾。

电流环在关节 MCU 本地跑，与总线无关；总线只决定你多快能改它的给定值。

频率阶梯：一台人形上真实存在的六个数

把全书出现过的频率排在一起，你会第一次看见这台机器的完整节拍：

```
20 kHz      MOSFET 开关（量级，宇树未公布 [未][行]）
 1–2 kHz    电流环（= f_sw/10–20，[行]）
 400–6000   关节指令刷新（[厂] 口径待澄清，见上节）
 ~500 Hz    整机低层状态/指令话题（Go2 上实测过；人形侧未逐项公布 [厂][未]）
 1 kHz      全身控制 / QP 回路的通行量级 [行]（全身控制 = 一次求解里同时决定
所有关节该出多大力，并把脚与地面的接触约束一起算进去）
 50 Hz      RL 策略输出节奏 [厂]（unitree_rl_gym 的工序口径）
 5–10 Hz    VLA 模型（第 26 章的带宽算术给出上限）
```

这张表里最该记住的不是具体数字，而是它的形状：从上到下每格差 3–20 倍，而每一格都受一个不同的物理量限制。最上面那格受半导体限制，中间受电感与惯量限制，下面受重力限制，最底下受内存带宽限制。你不能通过“优化算法”跨越任何一格——你只能换硬件（往上）或者换任务定义（往下）。

这也终于回答了第 00 章那个问题：大脑和关节之间相差 2400 倍，中间靠什么接上。答案是：**靠这条级联链，每一环替上一环把快的那部分做完，每一环只慢 3–20 倍**。2400 倍不是一次跳过的，是六级台阶走下来的。**顺带把口径钉住**：这六级不是同一根轴——表里“指令刷新 6 kHz”与“全身控制 1 kHz”是**更新率**，而电流环 1–2 kHz、速度环一两百赫兹是**闭环带宽**，两者单位一样、含义不同，所以相邻两格的比值会同时出现 20 倍与 0.5 倍这种“向上”的数。**读这张表要读落差的总量级，不要读每一格**。

类比，以及它在哪儿破

类比：一家公司从 CEO 到车间。CEO 说“这个季度交付十万台”（位置给定），部门拆成周计划（速度给定），班组拆成每班动作（扭矩给定），工人手里只有一把扳机（电流）。每一级都比上一级快 5–10 倍，且每一级都假定下一级“会照做”。

类比在哪里破：公司里如果某个基层不照做，上级要到周报才知道；而级联控制里，内环的响应速度必须显著快于外环，**否则外环的假设就是假的**。真实机器人里，一旦负载惯量突然变化（摔了一跤、手里东西掉了、齿轮卡住），内环的动态慢到不满足 5–10 倍，外环立刻开始震荡。这就是“机器人突然抖起来”最常见的解释——不是算法坏了，是那条带宽分离假设在那一刻不成立了。

一个可检验的说法：位置、速度、扭矩三种模式是同一条级联链上「包了几层」的区别，链的终点都是 q 轴电流；级联成立的必要条件是内环带宽 ≥ 外环 5–10 倍；而 GO-M8010-6 的「6000 Hz 通信控制频率」与官方自己公布的「15 台 / 4 Mbps / 17+16 字节帧」不能同时成立为全双工全量交换（差 5.9 倍），把算术反过来得到的 1010 Hz 正好落在官方 Z1 页标的 1 kHz 上。

证据等级：级联结构与带宽分离判据是控制工程通识 [行]；`MotorCmd` 七列、`MotorState` 的 `tau_est` / 双温度 / 电压、35 元素定长数组，全部 [厂]（官方 `unitree_ros2` 的 `unitree_hg/msg/MotorCmd.msg`、`MotorState.msg` 与 SDK 生成的 `unitree/idl/hg/MotorCmd_.hpp` 可直接读）；三张模式表分别 [厂]（文档中心 8/106、8/107 与产品页）；「6000 Hz / 15 台 / 4 Mbps / 17 B / 16 B」是 [厂]，「23.76 Mbps / 5.9 倍 / 1010 Hz」是 [算]（`foc_check.py bus_budget()`）；`rt/lowcmd` 500 Hz 是 [厂]（文档口径）并被 [算]（control_dt_ = 0.002）印证；频率阶梯里未另标者为 [行]。

可核对清单：

- [ ] 跑 `foc_check.py` 的“27 总线带宽 M8010”，确认 23.76 Mbps、5.9 倍、1010 Hz 这三个数
- [ ] 打开 `unitree_ros2` 里的 `unitree_hg/msg/MotorCmd.msg`，数一遍字段是不是 mode / q / dq / tau / kp / kd / reserve 七列，并确认里面**没有电流字段**
- [ ] 打开同目录 `MotorState.msg`，找到 `tau_est` 这个字段名，想一想 est 这个前缀意味着什么
- [ ] 在宇树 SDK 源码里找到 lowcmd 的关节结构定义，数一遍字段是不是 q / dq / kp / kd / tau 五个，并确认里面没有电流字段
- [ ] 用你手里任何一颗一体化关节的参数表，检查它标的“控制频率”是通信口还是环路，判据就是本章那条带宽算术
- [ ] 想清楚为什么“提高开关频率”最终会让你按压弹簧时能调出更高的 kp

下一章：链条到这里已经通了——从角度请求一直到栅极占空比。但有一件事我们一直在跳过：控制器凭什么知道 θ？它凭什么知道转子此刻朝哪儿？下一章去看那颗 15 位的编码器，以及它前面那个更硬的东西：齿隙。

第 13 章

死区、采样时刻与母线电容：驱动板炸管的四种方式

原理全对的一版代码，第一次上电就冒烟

假设你把前十章全部实现对了：Clarke 和 Park 的系数用同一套约定，PI 参数按带宽分离配，SVPWM 的扇区判断也没写错。上电，让电机出 5 牛米。

第一秒扭矩是对的。第三秒开始抖。第七秒冒烟。

这不是假设，这是电机驱动板最常见的首次上电剧本，而它的原因**几乎全在软件与物理的接缝上**——不在算法里，在那些“教科书不会讲，因为讲了也不属于算法”的地方。这一章列四类。

> 产业整合是我们一直在做的事情。2015 年做 XDog 的时候，电机是买的，里面的电机驱动器是我自己设计的，所以成本非常低。……我们每年都会进一步，能自研的全自研。我们也有自己的工厂。
>
> 王兴兴 — 《晚点 LatePost》对话王兴兴（问答实录），新浪财经转载 — 2025-02-16（直接引语（问答实录，按“王兴兴:”归属））

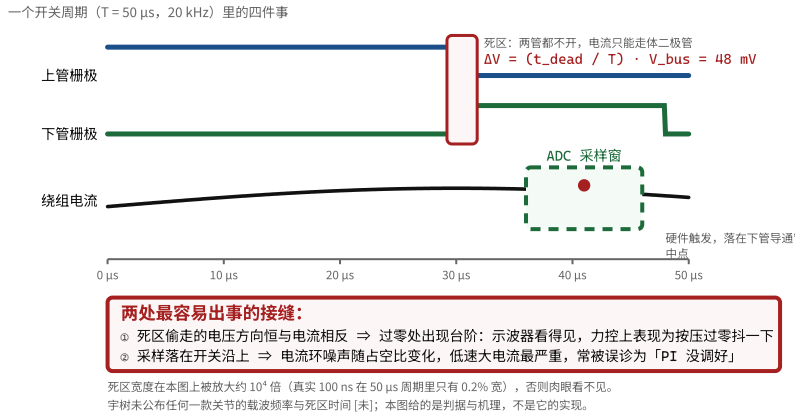


图 13-1　一个开关周期内的时序轴：PWM 沿、死区、体二极管续流、ADC 采样窗

第一类：直通与死区——两个必须同时存在的错误

一个半桥里上管接母线、下管接 0 V。理想开关不会同时导通，真实 MOSFET 会：关断需要时间（载流子复合、栅极电容放电）。如果你在关断过程中就打开对面那只，你会得到一条从母线到地的金属通路。

```
I = V_bus / R_loop        R_loop ≈ 几毫欧
24 V ÷ 0.005 Ω = 4800 A
```

4800 安培持续几十纳秒。后果不是“跳保护”，是 bond wire 熔断、铜皮烧穿、MOSFET 外壳炸开。行业里管这叫直通（shoot-through），而它的唯一预防手段是**故意让两个都不开一段时间**：死区。

死区是拿性能换安全。它偷走的电压是（第 08 章）：

```
ΔV = (t_dead / T_pwm) × V_bus
   = (100 ns / 50 µs) × 24 V = 0.048 V
```

（`foc_check.py deadtime_error()`。）

48 毫伏在满速时是噪声。但请注意低速工况：一台人形的踝关节在做恒力按压时，转速接近零，反电势接近零，你施加的电压几乎全部用来克服 I·R。M8010 若相电阻在几十毫欧量级（**官方未公布** [未]，这里只说量级关系），驱动 20 A 需要的铜降也就 1 V 上下——**48 毫伏就是它的 5%**。

更麻烦的是它的方向性。死区期间电流只能走体二极管，而电流往哪边走取决于电流的符号，不取决于你想要哪边。所以：

```
i > 0 时，输出电压被往下拉一个二极管压降
i < 0 时，输出电压被往上顶一个二极管压降
```

净效果等价于**在回路里串了一个“与电流方向相反的固定电压源”**。它在电流过零处制造一个不连续：指令从正到负时，实际电流要“卡”一下才翻过去。

在示波器上你看到的是电流波形在每个过零点有一个小台阶；在力控上你看到的是**恒力按压时力曲线每过零抖一下**；在听觉上你听到的是低速运转时的“咔咔”。

补偿办法是有的：按当前电流符号，预先把这 48 毫伏加回去。但补偿要准就得知道符号，而符号在过零附近恰好是噪声主导——**这是一个自指的问题，工程上只能收敛到一个“够用”的解，不能收敛到精确解**。这也是为什么死区补偿是电机控制里“人人都会写、很少有人调好”的那一块。

第二类：采样时刻——你的电流值可能来自错误的那半周期

电流必须被测量，FOC 才有输入。最便宜的做法是在下管支路串一个采样电阻，测它的压降。

问题是：**下管不是一直导通的**。下管导通期间，流进采样电阻的电流等于绕组电流；上管导通期间，绕组电流改走上管，采样电阻上看到的是零或者别的东西。

于是采样必须落在一个窗口里：

```
必须：下管已经导通（电流路径建立）
必须：避开开关沿（栅极电容充放电期间 di/dt 极大，采样保持会抓到过渡态）
必须：电流已经稳定（开关噪声衰减完，通常要几百纳秒到 1 μs）
```

在 20 kHz、周期 50 μs 的系统里，这个窗口通常被安排在**下管导通期的中点**。硬件上用 PWM 计数器直接触发 ADC（不是软件里“差不多时候读一下”），触发链路是 PWM → 同步信号 → ADC 启动 → DMA 搬运 → 中断里数据已就绪。

如果这一步做错了，症状是电流环噪声大、且与占空比相关——在低速大电流时最严重，因为那时下管导通窗口最窄。很多人会把它误诊为“PI 参数没调好”，然后花两周调参数。

宇树官方文档里能看到他们对这一层的重视程度：IM6014 的 STM32 例程要求 MCU 带 FPU、UART 支持 6 Mbps，而 M8010 文档明确 UART 建议 6 Mbps [厂]。至于它自己那块板子用的什么 MCU、怎么触发的 ADC，**从未公布**（拆解称芯片丝印被抹除）[未]。

顺带一条行业事实：开源标杆 moteus 全系用 STM32G431，那颗芯片之所以被选，是因为它带 **HRTIM（高分辨率定时器）+ CORDIC + FMAC** 三个外设——HRTIM 解决死区与相移的分辨率，CORDIC 直接在硬件上做 sin/cos（Park 变换），FMAC 在硬件上跑滤波器 [查]。**一颗 22 元的 MCU 里有三个外设是为 FOC 造的**，这件事本身就说明这三步有多关键。

第三类：母线电容——机器人会发电，而电能没地方去

这是最反直觉、也最容易死人的一类。

电机是可以在四象限运行的。当你的膝关节在落地瞬间被“反向推着转”（重力把腿压弯，而电机想阻止它），电机就从负载变成发电机。它发的电会通过**下管的体二极管**灌进母线电容，把母线电压顶上去。

```
E = ½ C (V_max² − V_min²)
```

一台 70 公斤的人形从半步高度落下，动能被关节在 100 毫秒里吸收，几十焦耳灌进母线。如果母线电容只有几百微法：

> 先算这颗电容一共存得下多少（能量 = ½·C·V²）：0.5 × 470e-6 × 54² = 0.685 J。落地要灌进来的是 60 J——是它总储量的 88 倍。真要全吃进电容，电压得充到 √(2E ÷ C) = √(2 × 60 ÷ 470e-6) = 505 V。
> （小信号式 ΔV ≈ 2E ÷ (C·V) 在这里已经失效：它假设 ΔV ≪ V，而这里 ΔV 是负数——意思是电容根本装不下，不是“涨一点”。）

而 MOSFET 的 V_DS 耐压通常是 60 V、80 V、100 V 这一档。505 V 这个数字的意思是：母线电容不是“吸收落地能量”的容器，它连 1% 都装不下；能量必须走别的路。 结果是一批管子同时击穿，短路，然后进入第一类故障。

所以真实的关节驱动板必须有：足够大的母线电容、母线过压检测、以及一条泄放通路（制动电阻，或者双向 DC-DC 把能量倒灌回电池——电池那边 0.864 kWh = 3.1 MJ [厂]，60 J 只占 0.002%，所以“倒灌回电池”在能量上是轻松的，难的是它要求变换器双向、且响应比冲击快）。**“过压保护”在机器人上不是可选功能，它是落地这一动作的必需品。**

（G1 官方电池 13 串、54 V 5 A 充电 [厂]；H1 15 Ah / 0.864 kWh / 最大电压 67.2 V [厂]；H2 15 Ah / 0.972 kWh / 75.6 V [厂]。串数与最大电压的对应关系第 35 章展开。母线电容容值、是否带制动电阻，全系未公布 [未]。）

第四类：热——唯一一种不会立刻炸你的失效

前三类在上电几秒内就会给你明确反馈。第四类不会。

损耗的来源：

```
导通损耗   P_on = I² · R_ds(on)            与开关频率无关
开关损耗   P_sw ≈ ½ · V · I · (t_r + t_f) · f_sw     正比于开关频率
铜损       P_cu = I² · R(温度)              R 每 °C +0.393%
铁损       随频率与磁密涨，低速时占比小
```

（NdFeB 剩磁约 −0.11 %/°C、铜电阻 +0.393 %/°C 是行业通识 [行]，不是宇树公布的器件参数 [未]。）

注意那个正反馈：

```
电流大 → 铜损大 → 温度高 → 铜阻更高 → 同样电流损耗更大
                              ↓
                  磁钢剩磁下降 → k_t 下降 → 控制器为保扭矩继续加电流 → 更热
```

这条链没有自限机制。 它只在两个地方停：要么过温保护跳闸，要么磁钢不可逆退磁——退磁之后 k_t 永久变小，这台关节从此出不到标称扭矩，而它的编码器与控制器都不知道。

而宇树 IM6014 官方标“每秒 400 次温度监控” [厂]，`MotorState` 里带两路 `temperature` [厂]。厂商开始报温度，说明他们知道**额定扭矩必须由温度决定**——这也正好解释了第 02 章那条空白：全系没有一个型号公布额定（持续）扭矩。因为额定值不是一个数，它取决于你允许结温到多少、允许多长时间、装在什么壳里、周围还有别的发热体。

密封壳里，电机与驱动互相加热。 关节是一个把两个热源关在一起的结构：铜损在电机那一侧，开关损耗在驱动板那一侧，中间隔着空气和一层壳。第 36 章会算这笔热账。

一张上电前的检查表

四类失效压成一张表，这是本章唯一希望读者带走的东西：

检查项	判据	不做的后果
死区时间	≥ 器件 t_d(on) + t_r + 驱动传播延迟，且写进代码注释	直通，炸管
死区补偿	有，且过零附近允许“不补偿”的窗口	恒力任务过零台阶
ADC 触发	由 PWM 硬件触发，落在下管导通窗中点	电流环噪声随占空比变
母线过压保护	有比较器 + 泄放路径，阈值 < 器件耐压的 80%	落地瞬间批量击穿

母线电容	能吸收一次最大再生能量而不越阈值（用 $E=\frac{1}{2}CV^2$ 反推）	同上
过温降额	有 k_t 随温度的模型或至少一个温度阈值	慢退磁，出厂测不出
上电自检	先读编码器与温度，再允许使能	带着错的角度标定直接出力

类比，以及它在哪儿破

类比：这四类失效像装修里那四类“住进去才发现”的问题——插座回路没分、下水没做存水弯、承重墙开了洞、防水只做了一层。它们都不是设计错误，是“图纸上看不见的那一层”。

这个类比在哪里破：装修的问题会一直在那里烦你，而驱动板的四类问题里，前三类会**在第一次上电时就把你打死**（字面意义上的 4800 安培）。只有第四类是装修式的——住进去第三年才发现楼下天花板湿了。

一个可检验的说法：一体化关节驱动板的四类典型失效分别由死区（直通与过零畸变）、ADC 触发时刻（电流环噪声）、母线电容与过压保护（再生能量顶穿耐压）、热设计（不可逆退磁）造成；其中 100 ns 死区在 20 kHz / 24 V 下每半周期损失 48 mV，而低速时可用总电压只有 1 V 量级，因此死区误差在恒力任务的过零点附近占比可达百分之几。

证据等级：48 mV 是 [算]（`foc_check.py deadtime_error()`）；直通电流 4800 A 是 [算]（24 V ÷ 5 mΩ，回路电阻是本书假设的量级，**不是宇树数据**）；母线电容那条 ΔV 估算是 [算]，容值 470 µF 是**举例用的量级，不是宇树公布值** [未]；13 串 / 54 V、67.2 V、75.6 V、“每秒 400 次温度监控”、`temperature int16[2]` 是 [厂]；温度系数 −0.11 %/°C 与 +0.393 %/°C、四类失效机理、采样窗口与 HRTIM/CORDIC/FMAC 的作用，是电力电子与电机控制通识 [行]（外设那条形成的引用来自 moteus 公开产品页 [查]）；**宇树关节板的具体设计（MCU 型号、死区时间、是否补偿、电容容值、保护阈值）全部未公布** [未]，本章给的是判据与量级，不是它的实现。

可核对清单：

- ☐ 跑 `foc_check.py` 的“13 死区 100ns@20kHz”，确认 0.2% 与 48 mV
- ☐ 手算一次直通电流：拿你手上任何一块板的母线电压除以一个你假设的回路电阻，感受那个安培数
- ☐ 用 $E = \frac{1}{2}C(V_{max}^2 - V_{min}^2)$ 反推：你的母线电容要多大才能吸收 60 J 而不过压
- ☐ 在宇树电机页面搜“死区 / dead time / 母线电容 / 过压 / 过温降额”，确认是否有任何一项被公布（预期：无）
- ☐ 打开 moteus 或 ODrive 的公开文档，看它们怎么写采样同步与过压保护——那是同一层问题的开源答案

下一章：这一章的四类失效里，有三类都指向同一件事——控制器对“转子此刻在哪、输出端到底出了多少力”的了解是有限的。下一章去看那份了解到底值多少钱：一颗 15 bit 编码器能分辨 0.011°，而一级谐波减速器允许 0.1° 的空程，谁说了算。

第 14 章

编码器、齿隙与准直驱：为什么 G1 用 20.58:1 行星而不是谐波

你能测到 0.011°，但你的关节能撒谎 0.1°

第 10 章留了一个洞：Park 变换需要电角度 θ，而 θ 来自编码器。这一章去看那颗编码器，然后立刻去看一个会把它彻底作废的东西。

宇树 M8010 与 IM6014 官方都标 **15 bit 绝对值编码器** [厂]。做一次除法：

```
360° ÷ 2¹⁵ = 360 ÷ 32768 = 0.010986°  ≈ 0.011°（约 40 弧秒）
```

再核对一下指令侧。IM6014 协议文档里位置字段的量化写得很明确：**32768 / 2π = 1 rad**，也就是 int32 位置指令的最小刻度就是 2π/32768 rad = 0.011° [厂]。**指令分辨率与编码器分辨率是同一个数**——你下的位置指令再细也不会被实现，因为协议本身装不下。

这个 0.011° 听起来非常好。一台 1.32 米的人形，脚尖离转轴约 0.6 米，0.011° 对应的弧长是：

```
0.6 m × 0.011 × π/180 = 0.12 mm
```

零点一毫米。够细了吧。

现在看第二行数据。Z1 机械臂的官方参数表里，谐波减速器那一栏写着：**减速比 60+，Backlash ～6 arcmin** [厂]。6 弧分 = 0.1°。

把两件事放一起（`foc_check.py resolution_vs_backlash()`，取 G1 拆解口径的 20.58:1）：

```
电机端编码器：0.010986°
折算到输出端：0.010986 ÷ 20.58 = 0.000534°
谐波背隙：0.1°

0.1 ÷ 0.000534 = 187 倍
```

齿隙是编码器分辨率的 187 倍。 你的传感器能看见 0.0005°，你的传动链允许你撒 0.1° 的谎。（齿隙 = 输入端转了一小段而输出端不动的那段空程，行业里也叫回差或背隙）

这就是“精度”这件事的真实结构：**系统的精度不是由最精密的那个零件决定的，是由最粗糙的那个环节决定的。** 而齿轮箱的齿隙恰恰是最难做小、最容易随磨损变大的那个环节。你把编码器从 15 bit 换到 20 bit，一点用都没有——地板不在那里。

四足机器人是一个全新的品类，有些零部件，你想采购也买不到……

王兴兴 — 投中网（同上，来源 B） — 页面 2025-08-07；访谈本体约 2020 年前后（早年访谈转写）

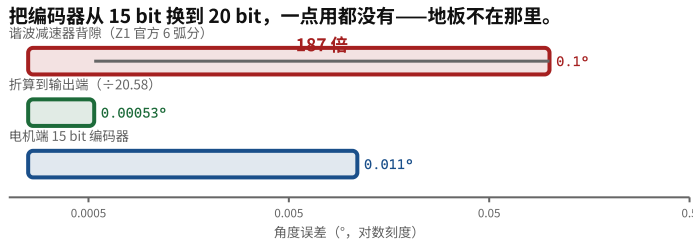


图 14-1　精度的地板不在传感器上：齿隙是折算后编码器分辨率的 187 倍

齿隙为什么骗人：一段“允许空转”的对话

行星和谐波减速器都有回差：输入端转了一小段，输出端不动。原因是齿与齿之间必须有间隙才能装配、才能容润滑油、才能不卡死；而这个间隙意味着“改变方向的那一小段行程”是免费的。

> 你让输出端往回走 0.05°：电机转了，编码器读数变了，输出端一动没动。

请注意这件事对第 04 章那条式子的打击。我们说过“用电流就能知道扭矩，不需要力传感器”。但那是**转子端**的扭矩：

```
τ(转子) = k_t · i
τ(输出) = k_t · i · N · η
```

那个 η（效率）里包含减速器的摩擦。而摩擦在两个方向上符号相反——它不是“损耗掉一个固定比例”，它是“先吃掉一个固定阈值，剩下的才传递”。于是：

```
低减速比：τ(摩擦) ÷ τ(输出) 很小   ⇒ 电流推出来的输出扭矩很准
高减速比：τ(摩擦) ÷ τ(输出) 很大   ⇒ 电流推出来的输出扭矩完全不可信
```

这就是准直驱的全部逻辑。它不是为了“更直接地驱动”，它是为了让第 04 章那条式子在输出端还成立。（准直驱 = 减速比压到远低于谐波的 60–160，用更大的电机换回齿隙与摩擦的小占比）

官方数据自己完成的证明：一颗电机，两个齿轮

现在用宇树自己公布的数，把 τ(输出) = τ(转子) × N 验一遍。

`unitree_rl_lab` 的 G1 配置文件里，关节电机按型号分组，而**型号名里就带着减速比** [厂]：

官方电机型号	减速比	官方 effort 限幅	effort ÷ 减速比 = 转子侧
N7520-14.3	14.3	88 N·m	**6.15 N·m**
N7520-22.5	22.5	139 N·m	**6.18 N·m**
N5020-16	16.0	25 N·m	1.56 N·m
W4010-25	25.0	5 N·m	0.20 N·m

（`foc_check.py gear_ratio_consistency()` 复算。）

看前两行。**同一台 φ75×20 的电机（型号名里的 7520），配 14.3 和 22.5 两种减速比，输出限幅分别是 88 和 139 N·m，而反推回转子侧是 6.15 与 6.18 N·m——相差 0.4%**。

这是一条不需要本书做任何假设的证明：宇树自己的两组官方数字，用一次除法就能对上，对上的精度是 0.4%。它同时说明：

- G1 的膝关节和髋侧摆关节用的是**同一颗电机**，输出差 58% 全靠那一级齿轮；
- 型号命名是可读的：`7520` = 直径约 75 mm、叠厚约 20 mm；后面的数字 = 减速比；
- 转子侧 6.15 N·m 这个数，与第 04 章算出的 M8010 转子侧 3.7–4.0 N·m 同数量级，只是更大一号的电机。

而“更大一号”这四个字就是代价。 想把 22.5:1 换成 14.3:1 来提高透明度、减小齿隙，你就必须让电机自己出更多扭矩，于是电机更大、更重、更热、更贵。准直驱不是免费的午餐，它是把代价从减速器搬到了电机上。（力透明度 = 你在电机端测到的量，有多接近输出端真实发生的力；齿隙与摩擦会把它糊掉）

宇树自己承认过这件事

Z1 机械臂产品页有一条脚注，原文是：

Since the reduction ratio used by each joint is relatively low, the position control stiffness of the whole machine is low. If the control mode is not optimized, there will be large position control error and shaking when the manipulator moves. [厂]

翻成中文：**因为每个关节用的减速比都比较低，整机的位置控制刚度就低；如果控制模式不优化，机械臂运动时会有较大的位置控制误差和抖动**。

这是全书我最喜欢的一句厂商自陈，因为它把一条因果链用一句话讲完了，而这条链本书要用十章讲：

```
低减速比 → 输出扭矩小 → 想扛同样的负载就得让电机多出扭矩 → 电流大 → 发热
                ↓
          齿隙与摩擦占比小 → 电流能代表输出力矩 → 可以不做力传感器
                ↓
          但位置环的等效刚度低 → 不优化就抖
```

同一件事的两面。低减速比同时给了你“能感知力”和“站不硬”。而 Z1 是机械臂（它不需要扛整机重量），G1/H1 是人形（每条腿都要全程扛 35–70 公斤）。所以人形上这个取舍比机械臂上更狠。

刚度上限还有一个来源：指令字段的量程

第 12 章说 kp 那一列官方叫 Stiffness。那它能设多大?

官方协议文档给了范围。M8010 的 kp 量化写作 `raw = (k_p / 25.6) × 32768`，**允许范围 $0.0 \leqslant k_p \leqslant 25.599$**；IM6014 的 STM32 例程参数表里输出侧 Kp 范围是 **0 ~ 410.725**、Kd 是 **0 ~ 102.681** [厂]。

拿它做一次除法（`foc_check.py stiffness_limit()`）：

```
M8010:   23.7 N·m ÷ 25.599 N·m/rad = 0.926 rad = 53.0°
IM6014:  34.4 N·m ÷ 410.725        = 0.084 rad =  4.8°
```

意思是：**在 M8010 上，如果你想让关节在输出满扭矩时只偏离目标 1°，你需要的 kp 是 23.7 ÷ 0.0175 = 1355 N·m/rad，而协议只让你写到 25.6**。换句话说，M8010 的 kp 量程只够支撑“满扭矩时偏 53°”这种软度。IM6014 的量程宽 16 倍，所以它能硬得多。

所以“这个关节能调多硬”这件事，第一层限制不是控制理论、不是电流上限，**是那个 float32 字段被厂商量化到了什么范围**。这是一个非常工程化的事实：你的物理能力被一个协议字段定义卡住了。

（注意：G1/H1 整机用的关节电机不是 M8010，其 kp 量程官方未公布 [未]。这里给的是官方唯一公布了两颗型号量程的对比，用来说明“量程会卡人”这件事的存在，不是 G1 的能力。）

双编码器（转子端与输出端各一颗角度传感器，两数之差就是齿隙与弹性变形的实时读数）为什么重要

M8010 是 **single-turn absolute encoder**（单圈绝对值），IM6014 是 **dual encoders**（转子端 + 输出端），官方 About 页的原文就是这两句 [厂]。整机页对 AS2/G1/H2/R1 只说“双编码器”，**单圈还是多圈、输出端几位、精度多少，全部未公布** [未]。（双编码器 = 转子端与输出端各一颗，两数之差就是齿隙与弹性变形的实时读数）

为什么要在输出端再放一颗？因为两颗编码器之差，就是齿隙与柔性的实时读数：

```
θ_输出实测 ≠ θ_电机 ÷ N      当且仅当  齿轮处于空程，或者弹性件被扭了
```

有了这个差，控制器第一次能知道“我现在在不在撒谎区间里”。没有它，你只能假设传动是刚的、无间隙的——而第 14 章开头那 187 倍告诉你，这个假设错得有多离谱。

代价也很实在：输出端一颗编码器意味着关节多一道密封、多一根线、多一次标定、多一份钱。这就是为什么它是“新一代”才有的配置。

于是两条路线分开了

现在可以把第 04 章那张表补完，并且解释一个真实的行业分歧：

	准直驱 + 电流估力（宇树 G1 走的路）	**高减速比 + 力矩传感器**
减速比	14.3–25（G1 官方型号名）／拆解口径 20.58 [厂][拆]	60–160（谐波）
齿隙	小	大
摩擦占输出扭矩比	小 ⇒ `tau_est` 可用	大 ⇒ 必须实测
位置刚度	低（Z1 官方脚注自陈）	高
同样输出扭矩需要的电机	更大更重	更小
每关节成本	少一颗传感器	多一颗（国产六维力已到约 ¥5,000 量级 [查]）
抗冲击	好（齿隙与低减速比反而成了缓冲）	差（谐波是薄壁柔轮，冲击下会坏）

同期另一家厂商的整机规格里写着“全关节力矩传感器，分辨率 $\leqslant 0.02$ N·m，采样 $\geqslant 5$ kHz” [查]。两家不是谁笨，是在同一组物理约束下选了不同的那一头：**宇树把预算和重量押在电机上，另一家押在传感器上**。

而 `MotorState` 里那个叫 `tau_est` 的字段（第 12 章）说明宇树选的是前者——它上报的是估计值，不是测量值 [厂]。

如果非要装力传感器，它靠什么工作

第 04 章那张表里有一行“传感器本身温漂、串扰、零点、易碎”，本章的路线表又抄了一遍“约 ¥5,000/台”，两个词都没被拆开过。这一节把它们拆开，同时把边界钉死：坤维、蓝点、宇立的型号、贴片数量、耦合与温漂指标、报价，本书没有逐条核到它们自己的手册，所以一个都不写；能核到的只有媒体口径那句价格 [查]，而它最容易过期。第 04 章那行里另一个名字“磁弹性元件”走的是完全不同的物理（应力改变铁磁材料的磁

致伸缩各向异性，于是要靠线圈或霍尔元件去读），本书没有可核的器件级材料，本节也不讲它——**这一节的六个新名字全部属于应变片这一条路线**。

力是怎么变成电阻的

弹性体（传感器里被设计成“只弹不塑”的那段金属：加载变形，卸载必须原样回来）被压时，它表面伸长一点点，这一跳靠胡克定律：

```
σ = E · ε          应变 ε = 应力 σ ÷ 杨氏模量 E
```

σ 是应力（N/m^2），ε 是应变（无量纲，m/m，伸长除以原长）。再往下追一句：那 E 又靠什么？靠晶格原子间距的势能曲率，所以它是材料常数，不是你设计得出来的。那 ε 为什么只能有千分之几？因为弹性体必须留在弹性区间里，而这个区间的高度是“屈服强度 ÷ E”给的，常用结构金属在 10^{-3} 量级 [行]。**信号天生就小，而且小在材料上，不是电路能改的**。

把这一小段伸长换成电阻的是应变片（strain gauge：几微米厚、绕成栅状的金属箔，用胶贴在弹性体上，跟着它一起变形）：

```
ΔR/R = K · ε          灵敏系数 K 无量纲；金属箔约 2，半导体大两个数量级但热稳定性差 [行]
```

K 约 2 是两件事的合计：一半多一点来自几何（拉长的同时横向变细，电阻按长度除以截面积走），剩下来自电阻率随晶格间距变化——而这两件事对温度的反应不一样，这句话待会儿要用。

惠斯通电桥为什么是四个电阻而不是一根

千分之二的电阻变化，ADC 怎么看得见？靠惠斯通电桥（Wheatstone bridge：四个电阻围成菱形，从一对对角上取电压差）。等臂、小变化下：

```
U_o/U_ex = (1/4) · (ΔR1/R − ΔR2/R + ΔR3/R − ΔR4/R)
```

U_ex 是激励电压（V），U_o 是桥输出（V）。单臂工作时，0.2% 的电阻变化只换来 0.05% 的 U_ex；四臂全工作、把受拉的接进相对臂，四项同号相加，输出乘 4，得到 0.2% 的 U_ex——**行业那句“2 mV/V”就是 K·ε = 2×10⁻³ 换了一种单位写** [行][算]。温度为什么能被消掉？因为**温度对四条臂同号、应变对相邻臂异号，而桥只做减法**：同号那份被减掉，异号那份被加倍。那这个抵消又靠什么？它的前提是四条臂真的对称——残余来自贴片栅丝与弹性体线膨胀系数的失配（“温度自补偿应变片”这类器件就是为它存在的），以及 E 自己的温度系数。温度与信号谁大，拿第 13 章那个铜的数（+0.393 %/°C [行]）算一遍：

```
ΔT = 0.002 ÷ 0.00393 ≈ 0.5 °C
```

不到半摄氏度的温差，电阻变化就顶得上满量程应变给的电阻变化。**单臂桥不是精度差一点，是不能用**。（真实器件用的是低温度系数的合金箔，这里只借铜的数量级说明为什么必须四条臂。）

六个载荷 → 六组符号 → 一张 6×6 矩阵

腕上那颗六维力要交六个数，而它身上只有“某处伸长了多少”这一种信息。中间这一步靠什么？靠**六个载荷在每个贴片上留下的符号模式互不相同** [行]：

施加的载荷	弹性体上的应变长什么样	于是贴片要怎么贴
沿轴的力	纵向受压、横向胀粗，两个符号相反	纵向一片、横向一片，接进相邻桥臂正好相加
弯矩	一侧受拉、对面那侧受压，周向反对称	对称两侧各一组，取差
扭矩	纯剪切，主应变方向在 ±45° 上	正着贴的片读到零，斜 45° 的片才有读数

把每组输出排成向量 ε（m 维），六个载荷排成 F（单位 N 与 N·m），小变形下叠加成立，于是一张常数矩阵出现：

```
ε = S · F        S 是 m×6 模式矩阵，一列 = 一个载荷的单位响应
F = D · ε        D = S⁺（m=6 时是 6×6 的逆，m>6 时是最小二乘伪逆）
```

D 就叫解耦矩阵（decoupling matrix：把 m 路电压还原成六维力/力矩的那张常数表），“标定”这两个字的物理内容就是拿一组已知载荷把 D 逐元素测出来。为什么 D 可以是常数？追问一句“那这又靠什么”，答案是**它只在弹性区间内线性**（又是前面那个 10^{-3} 上限）。所以超量程的后果不只是“会坏”：**压进塑性之后，这张标定表描述的那根梁已经不存在了，解耦矩阵当场失效**。这是第 04 章那个“易碎”最准确的说法。

串扰、维间耦合、温漂，都是这张矩阵的性质

维间耦合（cross-axis coupling，行业里也叫串扰——一根轴上加载、另外几根轴也报数）不是噪声，它就是 D 的非对角元：把只有 F_z 的真实载荷代进 F = D·ε，只要第 x 行在 ε_z 那列不为零，F_x 就报出一个不存在的数。条件数（condition number：κ = σ_max ÷ σ_min，同一张矩阵最大与最小奇异值之比）回答的是“这张矩阵逆得稳不稳”，线性代数给的界是 $\|\delta F\| \div \|F\| \leqslant \kappa \cdot \|\delta\varepsilon\| \div \|\varepsilon\|$，翻译成人话：**贴片与电压链上 0.1% 的误差，最坏方向会被放大成 κ × 0.1% 的力误差；κ = 20，就是 2%**。耦合不是供应商给你的指标，是你自己的贴片布置决定的矩阵性质。

温漂在这张矩阵上有两个入口，而不是一回事：

温漂来源	作用在哪一维	空载时看得见吗	能不能靠标定消掉
桥臂电阻随温度涨（金属是正温度系数）	零点	看得见，空载也在漂	一次零点校准就能减掉
弹性模量 E 随温度降（同一根棒热了更软）	灵敏度	隐形，漂多少正比于当前载荷	减载校准碰不到它

本书说这两条相反，指的是**作用点相反**：一次标定只能钉死其中一条，而厂商那种“温度范围内 ±X% F.S.”的写法，是把两条打包成一个数[行]。

把那张账补完：2% 的耦合值多少牛

路线	误差来源	折算成满量程百分比	作用在 500 N 的量程上
传感器	维间耦合（κ = 20 ⇒ 2%）	2%	F_err = 500 × 0.02 = 10 N
电流估力	摩擦阈值 Fs = 2.4 N・m（N7520-22.5，官方训练仓库），该档满量程 139 N・m	2.4 ÷ 139 ≈ 1.7%	约 8.5 N
电流估力	k_t 温漂（第 36 章：80 °C 温升 ⇒ 同电流扭矩少 8.8%）	8.8%	约 44 N

（[算]。三个来源分别是上面那条 κ 的界、`foc_check.py` 的 UNITREE_ACTUATORS 表、第 36 章的温升链。）口径要说死：**这张表只比“某一瞬间的绝对误差”，不是两条路线的总成绩**。而在这个口径上两条路线打平。真差别全在误差的性质：串扰要修正，必须先知道另外五根轴此刻受了多少载，而那恰恰是这次测量本来要告诉你的事，能不能解开全看 κ；摩擦有结构、与换向方向相关、可以被前馈吃掉（和本章“齿隙的谎是有结构的”同一条道理），k_t 温漂慢、单调，可以拿温度测点当健康指标盯（第 36、38 章）。

那传感器到底买到了什么？**带宽和分辨率，不是精度**。同期那家厂商的整机口径写着“分辨率 ⩽0.02 N·m，采样 ⩾5 kHz”[查]，而本章那条 0.19 rad/s 的速度噪声，到第 18 章会变成一条“不装传感器时能检出的最小力矩”的账，落点在 2 N·m 量级——**与 0.02 N·m 差两个数量级**，那不是靠更好的滤波器追得上的——本章的分歧就在这里：一边把钱押在电机上，一边押在传感器上，两边都付全额。

类比，以及它在哪儿破

类比：齿轮箱是一段允许撒谎的对话。你对翻译说“往右转 0.05°”，翻译自己动了，但站在门另一边的听众一动没动——而你只能通过听众来确认指令被执行了。齿隙就是这段“我听了但没做”的空程。

这个类比在哪里破：翻译的谎是随机的，而齿隙的谎是**有结构的**——它只在换向时出现，且量基本固定。这意味着它可以被建模、被补偿（用双编码器测出来，或者用方向相关的偏置补掉）。所以真实工程里齿隙不是无解，它只是“要额外花一份成本才能不管它”。而准直驱路线做的正是这件事：**把成本花在让齿隙小到可以忽略上，而不是花在测量它上**。

一个可检验的说法：G1 官方关节电机家族里，N7520-14.3 的 88 N·m 与 N7520-22.5 的 139 N·m 反推转子侧分别是 6.15 与 6.18 N·m，相差 0.4%，构成 τ(输出) = τ(转子) × 减速比 的一条官方自证；同时 15 bit 编码器折算到 20.58:1 输出端是 0.000534°，而谐波背隙 6 弧分 = 0.1°，是它的 187 倍，所以齿隙而非编码器决定位置精度地板。

证据等级：15 bit、single-turn / dual encoders、32768/2π = 1 rad、kp 范围 0–25.599 与 0–410.725、Z1 的 60+ / 6 arcmin / 0.2 N·m / 1 kHz、Z1 那条脚注原文，全部 [厂]（产品页与文档中心可逐字核对）；N7520-14.3 等型号名与 effort 限幅是 [厂]（`unitree_rl_lab` 配置文件）；187 倍、0.4%、53.0°、4.8°、0.12 mm 是 [算]；20.58 速比与主承载关节 1.1 kg 是第三方拆解 [拆]，**与官方型号名里的 22.5 不一致，本书两条都列，不取平均**；G1 关节内是否有输出端力矩传感器官方从未说明 [未]，`tau_est` 是最强旁证但不是直接证据。

新增那一节的证据等级单独说清：电桥输出式、灵敏系数 K 的构成、全桥 2 mV/V 与 K·ε 的等价、六个载荷的符号模式、解耦矩阵与伪逆标定、条件数误差界、温漂的两个入口——**全部是传感器与力学通识 [行]，不是任何一家厂商的器件事实**；由通识推出来的 ΔT ≈ 0.5 °C、κ = 20 ⇒ 2%、500 × 0.02 = 10 N 是 [算]；摩擦那一行 2.4 N·m ÷ 139 N·m ≈ 1.7% 用

的是官方训练仓库值 [厂]，k_t 那一行 8.8% 沿用第 36 章的温升链 [算]。**坤维 / 蓝点 / 宇立的型号、贴片数量、耦合与温漂指标、报价，本书未逐条核到其官方手册，一律不写**；“约 ¥5,000/台”是媒体口径 [查]，属易腐信息，不当选型依据；智元精灵 G2 的“≤0.02 N·m、≥5 kHz”仍是厂商口径 [查]，本书只拿它作带宽与分辨率的对照，不据此推断器件结构。

可核对清单：

- ☐ 跑 `foc_check.py` 的“15 G1 减速比一致性”，确认 6.15 与 6.18 这两个数
- ☐ 打开 Z1 产品页，找到那条关于“低减速比 ⇒ 位置控制刚度低”的脚注，读原文
- ☐ 手算一次 360 ÷ 32768 ÷ 20.58 与 6/60，确认 187 倍
- ☐ 在宇树 G1/H1/H2 页面搜“额定扭矩 / 持续扭矩 / 减速器类型 / 输出端编码器位数”，确认这三项是否都查不到（预期：都查不到）
- ☐ 手算一次 $K\cdot\varepsilon = 2 \times 10^{-3} \Rightarrow$ 全桥 2 mV/V，再拿 0.002 ÷ 0.00393 确认那个 0.5 °C——这一条是“为什么单臂桥不能用”的全部理由
- ☐ 拿 $\kappa = 20$、量程 500 N 复算 10 N 与 8.5 N（2.4 ÷ 139 × 500），确认本书说的“两条路线在绝对误差上打平”是算出来的，不是修辞

下一章：第 1 部的十章已经把所有零件讲完了。最后一章拿三颗真实器件对一次总账——哪些数官方给了，哪些数本书能推出来，哪些数永远拿不到。

第 15 章

三代关节的账本：23.7 → 34.4 → 360 N·m

四颗电机，一条因果链

第 1 部讲了十章的物理。这一章只做一件事：拿宇树自己公布的器件，把那些式子逐个代一遍，并且把“代不出来的地方”标清楚。

	GO-M8010-6	IM6014（N6014B-12.6）	G1 膝（N7520-22.5）	H1 膝（M107 家族）
峰值扭矩（输出）	23.7 N·m	34.4（反向）/ 31.7（同向）N·m	139 N·m（官方配置限幅）／120（EDU 页）／90（基础页）	360 N·m（页面）／300（官方 MJCF+URDF）
减速比	1:6.33	3:38 = 12.667	22.5（型号名自带）	未公布 [未]
供电	12–30 V，推荐 24 V	24–75 V	未公布 [未]	未公布 [未]
峰值电流	40 A	40 A	未公布 [未]	未公布 [未]
扭矩常数	0.63895 N·m/A（输出端）	未公布 [未]	未公布 [未]	未公布 [未]
编码器	单圈绝对值 15 bit	双绝对值 15 bit	官方整机页只写“双编码器”	同
总线	RS-485 4 Mbps，17+16 B 帧	RS-485 6/4 Mbps，20+26 B 帧	整机侧走 `rt/lowcmd`，官方口径 500 Hz	同
标称频率	通信控制频率 6000 Hz	最大总线轮询 10000 Hz	—	—

质量	530 g	535 g	未公布 [未]	官方“峰值扭矩密度 189 N·m/kg” [厂]
单卖价	$369	店铺标价存疑	—	—

来源：M8010 与 IM6014 全部 [厂]（产品页 + 文档中心协议页，可逐字核对）；G1 的 139 来自官方 `unitree_rl_lab` 配置，120/90 来自官方产品页；H1 的 360 来自产品页、300 来自官方 MJCF/URDF [厂]。

这张表最该被读出来的，是它有多少格子写着“未公布”。一颗 2023 年就在卖的四合一关节，官方给了九个可核对的电气参数；而 2026 年装在旗舰人形上的那颗，只给了一个扭矩数。这不是宇树变保守了，是**产品定位变了**：M8010 是当元器件卖的（你要拿它自己搭东西，所以必须给全），G1/H1 的关节是当整机零件锁起来的（你不该单独用它）。

液压系统现在有几大问题：第一，成本非常高。另外，机械液压系统会漏油……

王兴兴 — 投中网（同上，来源 B），谈为什么选电驱不选液压 — 页面 2025-08-07；访谈本体约 2020 年前后（早年访谈转写）

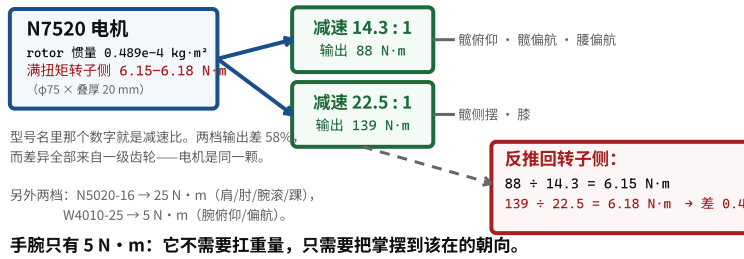


图 15-1　官方数据自己完成的证明：同一颗 φ7520，两个齿轮，两档输出

第一笔账：那条 τ = k_t · i 到底准不准

第 04 章已经算过，这里复核并把误差说清：

```
0.63895 N·m/A × 40 A = 25.56 N·m
官方峰值 = 23.7 N·m
差 +7.8%
```

7.8% 这个残差本身有意义。如果手册的 k_t 是纯几何量，它乘上手册的峰值电流应该**正好**等于手册的峰值扭矩。不等，说明中间有一刀被削掉了。可能是：

- 峰值电流 40 A 是瞬时允许值，而 23.7 N·m 是在某个更短脉冲宽度下的实测；
- 磁路在 40 A 时已经饱和，k_t 不再是常数（第 05 章的 τ = k_t·i 本来就是线性模型的产物）；
- 或者 23.7 是带效率折损的输出值，而 k_t 是理想换算值。

官方没说是哪一种 [未]。本书能给的是判据：k_t 与 I 的乘积与标称扭矩差多少，差多少就说明这两项里至少有一项不是同一工况下的数。这一招对任何一颗电机手册都好用。

再往转子侧折一次：

```
k_t(转子) = 0.63895 ÷ 6.33 = 0.10094 N·m/A
τ(转子) = 0.10094 × 40 = 4.04 N·m
× 6.33 = 25.6 N·m（理想）；按 85% 效率 = 21.7 N·m
```

（`foc_check.py kt_crosscheck()` 与 `quasi_direct_drive_torque()`。）

4.04 N·m 的转子扭矩，装在 530 克、直径 96.5 mm 的壳子里。 这个数才是“电机本身有多强”的度量，而 23.7 是“加了齿轮之后有多强”。整个第 14 章讲的取舍，就浓缩在这两个数之间。

第二笔账：官方数据自己完成了 τ = τ · N 的证明

第 14 章那条发现值得在这一章再摆一次，因为它是全书最干净的一条一手证据。

`unitree_rl_lab` 的 G1 配置里，关节电机型号名自带减速比 [厂]：

```
N7520-14.3 → effort 88 N·m  → 88 ÷ 14.3 = 6.15 N·m（转子侧）
N7520-22.5 → effort 139 N·m → 139 ÷ 22.5 = 6.18 N·m（转子侧）
```

差 0.4%。同一台 φ75×20 电机，两个齿轮比，两档输出。

```
N5020-16   → 25 N·m → 1.56 N·m（转子侧）
W4010-25   →  5 N·m → 0.20 N·m（转子侧）
```

于是 G1 的 23 个自由度其实只有四类器件，而扭矩档位是这么排的：

用在哪	型号	输出限幅
髋俯仰、髋偏航、腰偏航	N7520-14.3	88 N・m
髋侧摆、膝	N7520-22.5	139 N・m
肩×3、肘、腕滚、踝×2、腰侧倾/俯仰	N5020-16	25 N・m
腕俯仰、腕偏航	W4010-25	5 N・m

看这张表就能读出这台机器的受力设计：只有需要把身体撑起来、以及需要侧向抗住体重的三个位置（髋侧摆、膝）用了最高档；手腕只有 5 N·m——因为它不需要扛任何东西，它只需要把手掌摆到该在的角度。

这也解释了为什么 G1 的腕部是三个自由度里最弱的一档：**人形的手不是用来举重的，是用来定位的**。而第 25 章会看到，一旦要“捏住东西不滑”，需要的就不是扭矩而是触觉。

第三笔账：扭矩密度与平方-立方

H1 官方页写着“峰值扭矩密度 189 N·m/kg” [厂]。用它反推关节单元质量：

```
360 N·m ÷ 189 N·m/kg = 1.90 kg / 关节单元
```

而第三方拆解给的 G1 主承载关节约 1.1 kg [拆]。H1 的关节比 G1 的大 3 倍扭矩、重 1.7 倍——**方向对，量级对**。

再叠上第 02 章那条平方-立方：

```
G1 → H2：线度 ×1.379
预测扭矩需求 ×L⁴ = 3.61；官方档位实际 ×3.0（120→360）～×4.0（90→360） ✓
预测质量    ×L³ = 2.62；官方实测 ×2.00（35→70 kg）   ← 比立方律省
```

（`foc_check.py torque_density_chain()`。）

扭矩那一栏落在预测上，质量那一栏比预测省。这两件事同时成立才有意义：宇树的放大不是等比放大（否则 35 kg 的 2.62 倍应该是 91.7 kg），它在结构与材料上确实做了非等比优化；但**扭矩需求老老实实跟着 L^4 走，这一条谁也省不掉**，因为那是重力与几何的乘积，不是设计选择。

第四笔账：功率密度为什么不能当额定用

```
M8010：23.7 N·m × 30 rad/s = 711 W ÷ 0.53 kg = 1.34 kW/kg
IM6014：31.7 N·m × 54.2 rad/s = 1718 W ÷ 0.535 kg = 3.21 kW/kg
```

这两个数是**两个角点的乘积**，而第 08 章已经证明那两个角点不可同时达到（M8010 在 30 rad/s 时反电势幅值 12.78 V，空载刚好在 24 V 母线的 SVPWM 上限 13.86 V 之内，但满载时铜损压降一加上就顶爆（14.38 V）；即使取官方上限 30 V 母线，17.32 V 仍然不够）。

所以 1.34 kW/kg 是**名义上界，不是可运行点**。真正可运行的那条曲线，是从 (30 rad/s, 0 N·m) 到 (0 rad/s, 23.7 N·m) 之间那条被母线电压弯下去的弧，而弧的具体形状需要 L、R、k_t(T) 三个数——**官方一个都没给**[未]。

这是本书能给工程师读者的最有用的提醒之一：**任何一体化关节的参数表，都不足以让你算出它的热设计**。你能算出它的上限形状，算不出它能在上限的百分之多少持续工作。而“持续”两个字，恰恰是人形机器人真正的瓶颈（第 36 章）。

三代之间到底变了什么

把四列横着看，演进的方向非常清楚：

一，电压往上走。24 V 推荐 → 24–75 V。第 08 章说过，提电压买的是转速上限不是扭矩。IM6014 官方把 54.2 rad/s 明确标在 60 V 下 [厂]，就是在把那个“不可达的角点”变成可达。

二，减速比往上走，但只走一半。6.33 → 12.667 → 14.3/22.5。注意这条链**没有走向谐波的 60–160**。Z1 是宇树唯一明说用谐波的产品（60+、背隙 6 弧分）[厂]，而它是机械臂不是人形。**在人形上，宇树停在 25 以下**。第 14 章解释了为什么：低减速比才能让 `tau_est` 成立，才能不用每关节一颗力矩传感器。

三，从单圈到双编码器。M8010 单圈绝对值，IM6014 双绝对值 [厂]。多出来的那颗输出端编码器，是为了让控制器第一次能知道传动链在不在撒谎。

四，开始监控温度。IM6014 官方标“每秒 400 次温度监控” [厂]，`MotorState` 里带 `temperature int16[2]` [厂]。第 05 章那条“k_t 随温度漂”、第 36 章整章的降额问题，在这里已经有了硬件入口——**厂商开始报温度，说明他们知道额定值必须由温度决定**。

五，能给的数越来越少。从九项电气参数到一项扭矩。这一条不是技术演进，是商业策略，但它直接决定了这本书能写多具体。

还有一个没被选中的答案：丝杠

上面三笔账都在算“旋转电机 + 减速器”这一条路。但把电机换成直线电机、把减速器换成滚珠丝杠， 是一条真实存在过的替代方案——工业升降台、注塑机、大多数灵巧手的指节都在用它。 人形关节为什么没选？三个理由，每一个都能算。

第一个理由：丝杠给的是直线，关节要的是转角。 丝杠的力学关系极其干净：导程 L（螺母转一圈走多远）决定一切。 忽略摩擦时 $\tau = F \cdot L \div 2\pi$，反过来 1 牛米能顶起 $F = 2\pi \cdot \eta \div L$ 的推力。 取导程 8 mm、中径 20 mm、效率 0.93（滚珠）的丝杠：

```
1 × 6.283 × 0.93 ÷ 0.008 = 730 N
```

也就是说 1 牛米的电机扭矩能顶 730 牛的直线推力——比任何旋转减速器都便宜地拿到大力。 这正是它被工业界选中的原因。但要把这 730 牛变成“膝盖弯 40°”，得再加一组连杆或弧面滑槽， 于是瞬时传动比变成角度的函数：同一个 1 牛米，在屈膝 30° 和 90° 时给出的关节扭矩不一样。 第 05 章那个 sinθ 的麻烦，以另一种形式回来了——而且这次连 k_t 这个常数都不是了。

第二个理由：摩擦角与自锁。 螺旋升角 λ 由几何决定：导程除以中径的周长。上面那根丝杠 $\lambda = 7.3°$。 摩擦能不能咬住它，看摩擦角 $\phi = \arctan \mu$。梯形螺纹加润滑脂，μ 取 0.15（行业典型值 0.1–0.2 [行]）， $\phi = 8.5°$。因为 $\lambda < \phi$，这根丝杠**自锁**：断电后负载压不回来，螺母不会自己转。 工厂里这是优点（省一套抱闸，断电即安全）。对一条腿这是致命的—— 第 12 章讲阻抗控制时说得很清楚：脚落地那一下，关节必须能被反着推。 自锁的传动链等于把关节焊死，冲击全部由齿轮、轴承和结构件硬接。 滚珠丝杠把 μ 压到 0.005–0.01，ϕ 只有 0.3°–0.6°，远小于 λ，所以不自锁、可以反驱—— 但它的效率优势（同样那组数算出 $\eta \approx 0.93$，梯形丝杠只有 $\eta \approx 0.45$）是拿“必须加防逆转”换来的。

第三个理由：摩擦迟滞就是齿隙的亲戚。 第 14 章量过那颗 15 bit 编码器与 0.1° 齿隙的差距（187 倍）。螺纹副里的间隙、滚珠循环回路的弹性变形、以及换向时静摩擦到动摩擦的跳变，合起来是同一类东西：**输入端动了，输出端先不动。**力控环在 1 kHz 上工作，遇到这种死区就会自激。旋转减速器里我们管它叫齿隙；丝杠里它还在，而且因为要额外克服摩擦才启动，更难补偿。

那它到底该用在哪。 三个条件同时满足时丝杠完胜：行程是直线的、需要极大的力、而且“断电即停”本身就是安全要求。升降平台、灵巧手指节（空间小到塞不下减速器，而且自锁省掉持续保持电流）、机床 Z 轴，都是这一格。人形机器人身上真要找这一格的地方，是它的**手**，不是它的腿。（宇树灵巧手内部是否用丝杠传动，未公布 [未]。上面全部数值是教科书级的机构算例 [行]，不是对任何一款宇树产品的描述。）

类比，以及它在哪儿破

类比：这四颗电机像一家车厂的发动机谱系——同一套工艺，靠缸径、行程、变速箱的组合铺出不同档位。7520/5020/4010 是缸径与行程，14.3/22.5/16/25 是变速箱速比，effort 限幅是厂标扭矩。

这个类比在哪里破：汽车发动机有明确的额定工作点（额定功率、额定扭矩、外特性曲线），因为车要过排放与耐久认证，法规逼着厂商公布。机器人关节**没有任何一条法规要求厂商公布额定扭矩**。所以类比里那半句“厂标额定值”，在这里根本不存在——不是厂商藏着不给，是这个行业还没有强制给。这是第 37 章（安全与认证）会接上的一个缺口。

一个可检验的说法：宇树四代关节器件的官方可核对参数数量从 M8010 的九项电气参数降到整机关节的一项扭矩；G1 的 23 个自由度实际只用四类电机（N7520-14.3 / N7520-22.5 / N5020-16 / W4010-25），其中只有髋侧摆与膝用最高档 139 N·m，腕俯仰与腕偏航只有 5 N·m；而 88÷14.3 = 6.15 与 139÷22.5 = 6.18 相差 0.4%，构成 τ(输出) = τ(转子) × N 的官方自证。

证据等级：M8010 / IM6014 全部参数 [厂]；G1 四类电机型号与 effort 限幅 [厂]（`unitree_rl_lab` 配置）；G1 膝 90/120/139 三个数分别来自基础版页面、EDU 页面与模型文件，**本书并列不合并**；H1 360（页面）与 300（MJCF/URDF）同理并列；1.90 kg、3.61、2.62、1.34 kW/kg、12.78 V 全部 [算]；20.58 速比与 1.1 kg 是 [拆]，与官方型号名里的 22.5 冲突，两条都留。

可核对清单：

- ☐ 跑 `foc_check.py` 的“15 G1 减速比一致性”与“15 扭矩密度与平方立方”两条
- ☐ 打开 `unitree_rl_lab` 的 G1 配置，找到 N7520-14.3 / N7520-22.5 / N5020-16 / W4010-25 四个键名，抄下 effort_limit 与关节分组
- ☐ 在 G1 产品页、G1 EDU 页、官方模型文件三处分别记下一个膝关节扭矩数，确认它们是否相同
- ☐ 数一遍 M8010 页面给了几项电气参数、G1 页面给了几项，感受那条“越来越少”的曲线

下一章：第 1 部到此结束——你已经知道一颗关节从磁场到螺丝的一切，也知道哪些数永远拿不到。第 2 部开始往上建：23 个这样的关节，怎么让一台 35 公斤的两足物体不摔。第一个答案是一句算术：四个支点可以零力矩，两个不行。

第 16 章

双足为什么比四足难：四个支点可以零力矩，两个不行

站着不动，其实是最难的动作

第 1 部讲完了 23 个（或 31 个）能精确出力矩的关节。现在把它们装成一具身体，第一个问题立刻出现：**站着不动**。

多数人的直觉是“站着最简单，走路才难”。这一章要把它反过来。

一台四足机器狗站着不动时，你把它的四条腿当成四根柱子，重力走结构骨架传到地面，**关节力矩可以是零**。电机不通电，它照样站着（只要抱闸锁住，第 37 章）。

一台双足人形站着不动时，做不到。它的两个支撑点合起来的支撑多边形，是脚掌那一小块面积；重心投影必须**持续**落在里面。而“持续落在里面”这件事没有任何一个时刻可以交给死结构去扛——**每一个腿、腰关节必须每一毫秒都在闭环出力**。

这一条差别会传染到整台机器的每一个设计决定。这就是本书为什么主体讲人形：**人形把每一个取舍都逼到了极限，而极限处的选择才有原因**。

大家可能很难想象，有时候，我们团队为了降低一根电线的重量，会花费很多时间和很大精力。

王兴兴 — 21世纪经济报道《宇树科技王兴兴谈机器人现状：最大挑战在哪里？为什么坚持开源？》（2025 骁龙峰会・中国嘉宾对话，东方财富转载） — 2025-09-25（直接引语（圆桌发言））

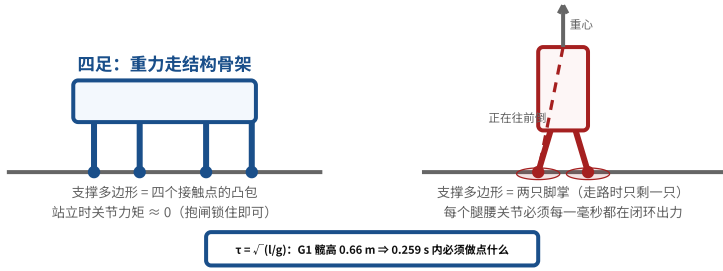


图 16-1　四个支点可以零力矩，两个不行

支撑多边形：一个二维凸包决定生死

定义很朴素：**把所有接触点连成的凸包**。重心投影在里面，物体不翻；出去，就翻。

> 四足：四个接触点 → 一个四边形，面积大，重心投影几乎总在中间
> 双足：两个接触点 → 两个脚掌；单支撑相时只剩一只脚，凸包缩成一个脚掌

请注意第二行后半句：**走路的时候，有一大半时间人形是单脚站立的**。也就是说，它必须在一个只有约 180 mm × 90 mm 的凸包上，扛住一个 35–70 公斤、重心在 0.9–1.5 米高的物体，而且这个物体还在往前倒。

（G1 与 H2 的脚掌尺寸官方未逐项公布 [未]；180×90 mm 是本书按同身高段人形产品图片的比例估读，**只当量级用**。）

时间常数：你只有 0.259 秒

第 00 章给过这条式子，现在它是主角。把机体当成一个倒立摆，它有一个特征时间：

```
τ = √(l / g)

G1：髋高 ≈ 0.66 m  ⇒ τ = √(0.66/9.80665) = 0.259 s
```

```
H2: 髋高 ≈ 0.95 m  ⇒ τ = √(0.95/9.80665) = 0.311 s
人: 髋高 ≈ 0.95 m  ⇒ 同上
```

（髋高由官方身高与腿长量级取，不是官方值 [算]；`foc_check.py pendulum()` 可复算。）

这个 τ 的物理含义是：从你开始往前倒，到你必须做点什么，只有 0.259 秒。 而“做点什么”包括：传感器看见偏了（①层）、估计出偏了多少（②层）、算出该出多大力（④层）、把力送到关节（⑤层）、关节真的出出来（第 4–13 章）。

把整条链的延迟加起来占 τ 的比例：

回路总延迟	占 τ	后果
5 ms	1.9%	几乎不吃相位裕度
10 ms	3.9%	高增益可用
50 ms	19%	明显吃裕度，需要前馈
100 ms	39%	增益不敢开大 → 站不稳或抖
120 ms	46%	大模型一次推理的量级（第 26 章）→ 已经不能靠反馈
200 ms	77%	已经不是反馈问题，必须靠预测

（占比是 [算]。）

现在看一个残酷的对照：**大模型的一次推理约 120 毫秒**（第 26 章的带宽算术）。也就是说，如果你让 VLA 直接控制平衡，你的回路延迟就占掉 τ 的 46%（100 ms 才是 39% 那一格）——**这台机器一定站不住。** 这不是模型不够好，这是 τ 不允许。

于是“低频语义 + 高频动作”这个全行业收敛出的架构（第 30 章），不是软件审美，是被一个 1632 年就写下的式子逼出来的。**平衡这条环必须留在关节和全身控制器里，永远不许上到 GPU 那一层。**（全身控制 = 一次求解里同时决定所有关节该出多大力）

步频也是算出来的

同一个 τ 还给出步频。腿是一个倒摆，它的自然摆动周期：

```
T = 2π√(l/g)
G1 腿长 ≈ 0.6 m ⇒ T = 2π√(0.6/9.80665) = 1.55 s ⇒ 一个步态周期两个半步 ⇒
约 1.3 Hz
人腿长 ≈ 0.9 m ⇒ T ≈ 1.90 s ⇒ 约 1.05 Hz
```

（`√(l/g)` 这一步在 `foc_check.py` 的自检里有一条已知答案用例：`pendulum tau 0.66m` → 0.259425 s（那是 G1 髋高，不是本节的腿长）。本节用的是 0.6 m ⇒ τ = 0.2474 s，再乘 2π 得 T = 1.55 s；同一个式子、两个长度，别把这两个数混起来 [算]。别去找脚本里的 `frequency_chain`——那条链在 `arithmetic_check.py`，而它现在打印的是“算不了（缺输入）”（人形侧连输入块都没有，见附录 B.3），本章没有拿它当依据。）

这条推导给出两个立刻可用的判断：

一，腿越短，步频越快。 这条关系给的是自然频率，不是控制器的选择：0.9 m 的腿约 1.05 Hz，所以人慢走一步约一秒；腿缩到 0.6 m，同一个式子就把它顶到 1.3 Hz。它也解释了为什么想让一台 1.32 米的人形走出“大人的从容步态”，你必须违背它的自然频率——那要多花能量。

二，摆动相的时间窗比 τ 更紧。 步频 1.3 Hz、占空比 0.5 ⇒ 单腿支撑约 385 ms；如果走得更急（2 Hz、duty 0.4），单腿支撑只剩 200 ms。**脚掌砸地那一瞬间需要的力矩指令如果晚到 100 ms，这一步就废了。** 而“砸地”这件事的冲击本身在第 22 章会变成 sim2real 的头号难题。

能量：那份没有休息窗口的账

现在回到钱与电池。

用官方公布的电池能量与续航，做一次除法（`arithmetic_check.py` 的能量链 [算]）：

```
AS2（四足）：648 Wh ÷ 4 h ÷ 20 kg  = 8.1 W/kg
G1（人形）：约 432 Wh ÷ 2 h ÷ 35 kg = 6.2 W/kg
G1（人形）：432 Wh ÷ 2 h ÷ 35 kg   = 6.2 W/kg（三个输入全是一手）
人（对照）：65 Wh/km × 5 km/h ÷ 70 kg = 4.6 W/kg（代谢口径）
```

（电池 Wh 与续航小时 [厂]，其中 G1 的 Wh 由 13 串 × 9 Ah 推得 [算]；人的口径来自公开物理资料 [查]。）

看起来人形更省？请注意这两件事同时成立：

- 四足那 8.1 W/kg 里，有相当一部分花在“移动质量”上，站立相可以几乎不花；
- 人形这 4.6–6.2 W/kg 里，**腿腰那 12–14 个关节没有“卸载休息”的窗口**——站着一秒就扛一秒重力与平衡，一秒都不许停。但别把它说成“每一瓦都花在闭环上”：第 20 章反推 G1 平均 216 W，其中光那块可选的 Orin NX 就占 10–25 W（第 26 章 [查]：NVIDIA 规格页），感知、计算与散热都要电。准确的说法是：**四足的静止可以近乎免费，人形的静止永远在付费**。

再叠上：H2 有 31 个自由度（Go2 是 12 个）、单关节更大更重、续航 3 小时（R1 只有 **1 小时** [厂]）。

一句话结论：双足不是“更聪明的四足”，是“没有休息窗口的四足”。而休息窗口的消失，直接把问题从“控制算法好不好”变成了“热与能量撑不撑得住”（第 35、36 章）。这也是为什么本书认为：**人形机器人当前的瓶颈不在决策层，在功率与散热**。

于是第 1 部那些取舍被解释了

回头看，第 1 部讲的每一个“为什么”，答案都在这张双足账上：

现象	四足可以	双足必须	逼出第 1 部的哪一章
站立不出力	可以（四根柱子）	不行	16（本章）→ 36 热
关节被冲击	四腿分摊	两腿全扛，且单腿相占一半时间	14 交叉滚子轴承、37 抱闸

力感知	不太需要	必须（不然踩到石头就摔）	04 电流即力矩、14 准直驱
扭矩档位	中等即可	膝 120–360 N·m	02 平方-立方、15 器件阶梯
刚度可调	需要	更需要（接触瞬间要软、支撑要硬）	12 五元组、18 阻抗
反应要快	200 ms 够用	100 ms 就要出事	00 时间域、10 电流环带宽

这就是“为什么这本书要从一颗关节讲起，而且讲的是人形”的完整答案：人形把每一格都推到了必须解释的位置。

类比，以及它在哪儿破

类比：四足像一张四条腿的桌子，锯掉一条还能站；双足像一支立在掌心的扫帚——你必须在它倒的方向上不停地移动支撑点。

这个类比在哪里破，而且破得很关键：手掌会主动“预判扫帚要往哪倒”，因为你能看见、能感觉、反应约 150–200 毫秒。而一台人形**没有触觉皮肤，它靠 IMU 与关节编码器推断自己是否在倒（关节编码器 = 装在电机轴上数角度的器件；输出端那颗叫双编码器，第 14 章）**（第 24 章），延迟可以做到远小于人，但它的“感觉”是模型而不是直接测量——一旦模型错（踩到一块软的东西、脚底有泥、电池电量下降导致母线塌陷），它会用错误的力去接一个正确的扰动。

人站得稳是因为感觉与结构一体；机器站得稳是因为有人在 1 kHz 上替它算。 这个差别是第 22 章整章的主题。

一个可检验的说法：双足与四足的根本差别是“站立时关节能否零力矩”——四足可以（重力走结构），双足不可以（重心投影必须持续落在由两个脚掌构成的支撑多边形内，且走路时约一半时间只剩一只脚）；这个差别使双足在站着的时候也要持续付费（但别理解成“每一瓦都进闭

环”：计算与感知本身就要电，第 20、26 章），并使其可用回路延迟被倒立摆时间常数 $\tau = \sqrt{(l/g)}$ 卡住（G1 为 0.259 s，因此 100 ms 延迟已占 39%）。

证据等级：τ 与步频公式是经典力学 [行]；G1/H2 身高、体重、自由度、电池 Wh、R1 续航 1 h 是 [厂]；髋高 0.66/0.95 m 与脚掌尺寸是本书按官方身高与图片比例估读 [算]，**只当量级**；功率密度三条是 [算]（官方 Wh ÷ 官方续航 ÷ 官方体重，官方从未公布整机功耗）；人的 65 Wh/km 是公开物理资料 [查]；延迟占比表是 [算]；“VLA 推理 120 ms”来自第 26 章的推算 [算]。

可核对清单：

- ☐ 跑 `foc_check.py` 的 pendulum 与 frequency 两条，复算 0.259 s 与 1.55 s
- ☐ 手算一次 432 ÷ 2 35，确认 6.2 W/kg（H1/H2 不给这一格：官方没有续航小时数）
- ☐ 找一段人形机器人被轻推的视频，看它第一步往哪迈——那一步的位置就是它的捕获点算法的输出（第 17 章）（捕获点 = 想停住就得把脚迈到的那个点，第 17 章给式子）
- ☐ 检查你自己写过的任何“人形 vs 四足”的对比，看它有没有提到“零力矩相位”这件事（多数没有，因为这是需要拆开电流才知道的差别）

下一章：本章说“重心投影必须落在支撑多边形里”。下一章要把这句话变成一个可计算的数——它叫零力矩点，而围绕它有一整套判据，也有一整套失效方式。（零力矩点 = 地面反力的作用点；它落在支撑多边形内就不翻，第 17 章会给式子）

第 17 章

倒立摆、ZMP 与捕获点：站稳的判据到底是什么

“它看起来挺稳”不是判据

上一章说双足必须让重心投影持续落在支撑多边形（支撑多边形 = 所有接触点连成的凸包，第 16 章）里。这句话还是几何直觉。工程师会立刻问两个问题：**投影在哪，我怎么算？以及“落在里面”是不是太保守？**

第二个问题尤其关键。因为一个正在往前走的人，他的重心投影**本来就在他支撑脚的前面**——他是在往前倒的。如果“投影在凸包内”是站稳的判据，那所有人走路时都不稳。

所以必须换一个能同时描述“站着”和“正在往前倒”的量。

当然，一些特别复杂的动作，还是会遇到问题。比如动作里有很大的侧向力，或者地面比较滑，这些都会带来挑战。

王兴兴 — 王兴兴 GTC 2026 演讲文字整理（“机器人前瞻”整理；页面自陈“在不改变原意的前提下进行了一定程度的增删修改”），腾讯新闻 — 2026-03-17（演讲整理稿（非逐字，页面已声明增删修改））

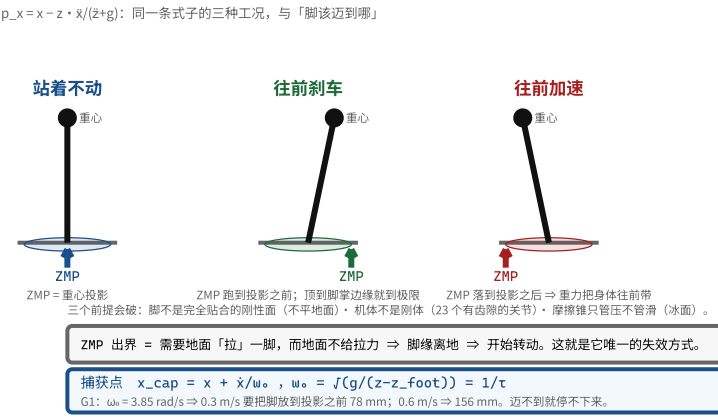


图 17-1　倒立摆、ZMP 与捕获点：$p_x = x - z \cdot \ddot{x}/(\ddot{z}+g)$ 的三种工况

零力矩点：地面推你的那个力的作用点

脚踩在地上，地面给脚一个反作用力（法向力加摩擦力）和一个反力矩。**零力矩点（Zero Moment Point，ZMP）定义为：把这个反力系向地面某一点简化，使得该点上的倾覆力矩为零的那个点。**

直觉版本：**地面“推你”的合力，等效作用在脚底的哪个位置。**

```
设重心位置 (x, z)，速度 ẍ、z̈，质量 m，重力 g
水平/垂直方向的合力产生惯性力，ZMP 位置 p 满足：

p_x = x − z · ẍ / (z̈ + g)
```

（这是线性倒立摆模型下的一阶式子 [行]；完整版要用角动量方程，本书只需要这一阶，因为它给出的物理已经够用。）

看这个式子在说什么。它把“重心在哪（x）”和“重心怎么加速（$\ddot{x}$）”合成了一个点：

- **站着不动**：$\ddot{x} = 0 \Rightarrow p = x$。ZMP 就是重心投影。

- **往前加速**：$\ddot{x} > 0 \Rightarrow$ 减掉的是一项**正数** $\Rightarrow$ p 往后移。**想让身体往前加速，ZMP（≈接触点）必须落在重心投影的后面**：只有地面支持力作用在重心之后，重力绕接触点的力矩才把身体往前带。地面只能推、不能拉，这是唯一可行的方向。
- **往前但要刹车**：$\ddot{x} < 0 \Rightarrow$ 减去的是负数 $\Rightarrow$ p 往前移，ZMP 跑到重心投影**前面**，用反过来的力矩把前倾拽住。
- **自由落体**（$\ddot{z} = -g$）：分母为零，p 无定义——**这时候地面已经不推你了，“稳不稳”这个问题暂时不存在**。

最后一行有个立刻可用的推论：**跳起来的那段时间里没有 ZMP 概念可言**。所以人形机器人在腾空相不需要维持平衡，它只需要在落地前把姿态摆对。这是“走路比跳简单”和“跳比连续小步跑容易控制”这两件事的共同来源。

判据：ZMP 在支撑多边形内就不翻

现在把上一章的几何接上：

```
ZMP 落在支撑多边形内   ⇒   地面反力不需要“拉”地面 ⇒ 不翻
ZMP 落到多边形外       ⇒   需要一个拉力才能维持，而地面不给拉力 ⇒ 开始转动 ⇒ 翻
```

第二行的因果值得说细一点。ZMP 跑出凸包意味着要维持这个运动状态，地面必须提供一个“往回拽”的法向力——但脚和地面之间只能压不能拉（除非有磁吸或绑带）。所以真实的物理会**放弃**那个运动状态：脚的前缘离地，机体开始绕脚边缘转动。

于是“站稳”这件事第一次有了一个可计算的标量：**ZMP 到支撑多边形边界的最短距离**。这个距离叫稳定裕度。工程上它比“在不在里面”有用得多，因为：

- 在里面的一个点，离边界 2 mm，实际上等于已经不稳（模型误差、地面不平、脚底有泥都会吃掉那 2 mm）；

- 控制器需要一个**连续**的目标函数，“在/不在”是二值的，没法优化。

但 ZMP 有三个前提，每一个都会破

这是本章真正想给的东西：一个判据的价值，一半在于知道它什么时候不成立。

前提一：脚是一个刚性的、完全贴合地面的面。

真实情况：G1 的脚是带圆弧的（官方 MJCF 里踝关节下方是脚掌几何），地面是瓷砖、地毯、或者有一粒沙子。一旦实际接触退化成一条线甚至一个点，“支撑多边形”这个概念本身就塌了——而 ZMP 的判据还在按整个脚掌面积算。

后果：在不平地面上，ZMP 判据会给出一个乐观的稳定裕度。

前提二：机体是刚体。

线性倒立摆模型把整个机器人当成一个质量集中在重心、下面接一根无质量杆的摆。而第 1 部刚讲完：一台人形是 23 个有齿隙（齿隙 = 输入端转了一小段而输出端不动的那段空程，第 14 章）、有弹性、有模态频率的关节串起来的。

后果：躯干摆动、手臂甩动、连杆弹性变形，都会让真实重心与模型重心不一致。这个偏差在第 22 章会变成 sim2real 的主要来源之一。

前提三：接触力只能压不能拉，而且摩擦锥有限。

判据只检查了“不拉”，没检查“不滑”。摩擦锥约束是另一个独立的限制：

```
|F_切向| ≤ μ · F_法向
```

一个在冰面上做同样动作的机器人，ZMP 全程漂亮，但会滑倒。**ZMP 是抗倾覆判据，不是抗滑判据。** 两者必须同时检查。

（μ 的取值宇树从未公布 [未]；这也不是厂商该公布的数——它取决于地面。但它必须出现在控制器的约束里。）

捕获点：把“该迈哪一步”变成一个可解的方程

ZMP 判的是“现在这一瞬间会不会翻”。但走路的核心问题是**下一步把脚放哪**。捕获点（Capture Point）给了一个干净的答案：

如果我现在什么都不做，只让重心按当前的状态继续倒下去，那么我必须把脚放到哪个位置，才能在放下去的那一刻恰好停下来。

对线性倒立摆，这个点有闭式解：

```
x_cap = x + ẋ / ω₀          其中 ω₀ = √(g / (z − z_foot))

而 √(g/l) 正是第 16 章那个 τ 的倒数。
```

这条式子值得停三秒。 它说：捕获点的位置偏移 = 当前速度 × 倒立摆时间常数。也就是说：

- 速度越快，你必须迈得越远——不是“大概更远”，是**严格线性**地更远；
- 腿越长（ω_0 越小），同样的速度需要迈得越远；
- 如果你迈不到那个点，你**不可能**停下来，只能继续跑。

（最后一条就是“被推了一下之后为什么会跑两步”的完整解释：不是算法反应慢，是那一瞬间物理上只有“跑”这一个解。）

代一个数：G1 髋高 ≈ 0.66 m $\Rightarrow \omega_0 = \sqrt{(9.80665/0.66)} = 3.85$ rad/s，$\tau = 1/\omega_0 = 0.26$ s。

```
重心水平速度 0.3 m/s ⇒ 捕获点比当前投影超前 0.3 × 0.26 = 78 mm
重心水平速度 0.6 m/s ⇒ 超前 156 mm
```

（`foc_check.py pendulum()` 给 τ，这一步是 [算]。）

78 mm 与 156 mm 这两个数，就是“这台机器一步该迈多远”的下界。一步迈不到 78 mm，0.3 m/s 的速度就停不下来。这也解释了为什么小个子人形的步幅天然小、步频天然高——它的 τ 更短。

于是有了两套流派

现在可以解释一件行业现象：为什么宇树公开的运控工序是强化学习（第 21 章），而不是 ZMP + MPC。

	ZMP / MPC 路线	强化学习路线
判据	显式：ZMP 在凸包内、摩擦锥内	隐式：奖励函数里“不倒”给分
需要精确模型	是（质量、惯量、接触）	否（训练时随机化这些）
处理不平地面	需要额外的接触模式规划	策略在随机地形上训过
可证明安全	原则上可以（约束是显式的）	原则上不行
算力	一次 QP 求解，毫秒级	一次前向推理，微秒到毫秒级
出问题能不能解释	能（看约束与求解器状态）	很难

关键分歧不在“谁更稳”，在“你愿不愿意为可解释性付建模成本”。人形的接触模式（几只脚着地、躯干会不会撞、手里端着东西）组合起来是指数级的，显式建模这条路在工程上贵得离谱。所以 2016 年之后，足式运控的主流从“模型 + 优化”整体转向了“仿真 + 学习”。

但请注意：**ZMP 与捕获点没有消失，它们换了位置。**它们现在是：

1. 强化学习**奖励函数**里的项（惩罚 ZMP 出界、奖励落脚接近捕获点）；
2. 仿真里生成参考轨迹的工具；
3. 工程师**判断策略为什么摔**的诊断量。

也就是说：**学会用 RL 之前，必须先懂 ZMP，否则你连奖励函数都写不出来。**这一章就是为那个准备的。

类比，以及它在哪儿破

类比：ZMP 是地面推你的合力的“着力点”，就像你推一扇门，力的作用点决定门怎么转。捕获点是“你得把脚垫到哪，这根正在倒的扫帚才停”。

这个类比在哪里破：扫帚倒向你手掌时，你的手是**被动**接住的；而人形的脚是**主动**迈出去的，而“迈”这个动作本身要花时间（摆动相 200 毫秒量级，第 16 章）。所以捕获点判据有一个隐含前提：**从现在到脚落地的那段时间里，你的重心状态不能再被扰动改变**。一旦有人在半路又推你一把，你算好的那个捕获点当场作废，必须重算——而重算需要新的 200 ms。

这就是“被连续推会摔”的物理原因：不是控制器不够快，是腿这个执行器天生就慢。而腿慢的原因在重力——$\tau = \sqrt{l/g}$ 里那个 g 你换不掉。

一个可检验的说法：站稳的可计算判据是 ZMP（地面反力系的等效作用点）落在支撑多边形内，其稳定裕度是 ZMP 到凸包边界的最短距离；ZMP 判据成立需要三个前提（脚为完全贴合的刚性面、机体为刚体、接触满足摩擦锥），三者在不平地面、弹性机身、低摩擦地面上会依次失效；捕获点 $x_{cap} = x + \dot{x}/\omega_0$ 给出“要停下来脚必须放到哪”，其偏移量等于速度乘以倒立摆时间常数（G1 为 0.26 s，故 0.3 m/s 需超前 78 mm）。

证据等级：ZMP 定义与线性倒立摆公式、摩擦锥约束、捕获点闭式解，全部是足式机器人控制的标准内容 [行]（可查 Raibert 与 Pratt 的原始文献）；$\tau = 0.26$ s、$\omega_0 = 3.85$ rad/s、78/156 mm 是 [算]；髋高 0.66 m 是本书按官方身高与腿长估读 [算]，**不是官方值**；“宇树用 RL 而非显式 ZMP/MPC”是从其公开仓库（unitree_rl_gym / rl_lab）推出的 [厂]，但**宇树从未公布整机在线控制律**，G1/H1 出厂固件里跑的到底是什么，是 [未]；μ 与接触几何参数全部 [未]。

可核对清单：

- ☐ 自己代一次 $\omega_0 = \sqrt{(g/0.66)}$ 与 0.3×0.26，确认 3.85 rad/s 与 78 mm
- ☐ 用 $p_x = x - z\cdot\ddot{x}/(\ddot{z}+g)$ 推一遍“腾空相分母为零”，确认那时 ZMP 无定义
- ☐ 找一段人形被推的视频，慢放数一下从被推到另一只脚落地过了几毫秒，和 200 ms 比
- ☐ 想清楚为什么“稳定裕度”必须是连续量而不是“在/不在”——这个问题的答案是所有 MPC 代价函数的写法

下一章：本章说“脚要迈到捕获点”。下一章讲另一件事：当脚已经踩到东西上了，关节该硬还是该软——以及为什么“硬”和“软”这两个字，在机器人里其实是两个可以被实时改写的数。

第 18 章

刚度是软件写的：阻抗控制与那颗可编程的弹簧

同一个关节，可以像石头，也可以像弹簧

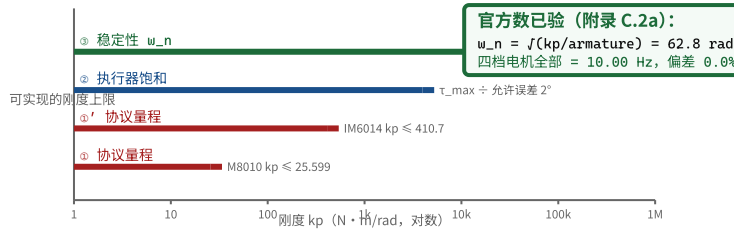
宇树给每个关节的指令里，kp 那一列官方文档叫 **Stiffness**，kd 那一列叫 **Damping** [厂]。这不是修辞。第 12 章那一行：

```
τ = tau_ff + kp·(q_ref − q) + kd·(dq_ref − dq)
```

写的就是一个弹簧（第一项误差 proportional）加一个阻尼器（第二项）。你把 kp 设成 200，关节在机械特性上就**是**一个 200 N·m/rad 的扭转弹簧；设成 0 且 tau_ff 给足，它就**是**一个纯力矩源。

所以“这个关节能调多硬”不是一个参数整定问题，是一个**它能同时满足多少物理约束**的问题。工程师需要的是三道天花板各自在哪，而不是一个“推荐 kp 值”。

> “We remain on track for pilot launch of Robotaxi in Austin by June and builds of Optimus on our Fremont pilot production line in 2025, with wider deployment of bots doing useful work across our factories.”
>
> 埃隆·马斯克（Tesla / xAI；人形机器人 Optimus 的公开发言者） — 特斯拉 2025 年第一季度股东更新（公司自己发布的 PDF，逐字抽取） — 2025-04-22（特斯拉自己发布的股东更新原文（PDF 全文用 PyMuPDF 抽取，逐字核对））



小关节先撞①，大关节先撞②；③ 在高惯量负载 + 硬位置控制时才咬人。

图 18-1　一个关节能有多硬：协议量程、执行器饱和、稳定性三道天花板

天花板一：指令字段的量程

最省事也最容易被忽略的一道。第 14 章已经算过：

```
M8010:   kp 允许范围 0.0 – 25.599 N·m/rad（官方量化式 raw = kp/25.6 × 32768）[厂]
IM6014:  输出侧 Kp 0 – 410.725，Kd 0 – 102.681 [厂]
```

换算成“满扭矩时允许偏多少”：

```
M8010:   23.7 ÷ 25.599  = 0.926 rad = 53.0°
IM6014:  34.4 ÷ 410.725 = 0.084 rad =  4.8°
```

（foc_check.py stiffness_limit()。）

结论：M8010 的 kp 量程只够支撑“满扭矩时偏 53°”这种软度。 也就是说，如果你拿它做一个需要“顶住 20 牛米且只偏 2°”的活，你需要 kp = 20/0.035 = 571 N·m/rad，而协议只让你写到 25.6。**你连请求都发不出去。**

这不是控制理论的限制，是一个 int16 字段的刻度。而它之所以长这样，是因为 M8010 是给四足云台级应用设计的低减速比器件，官方把量程按那个场景配了。

这一条的通用教训：拿到任何一颗一体化关节，第一件事不是查扭矩，是**查 kp/kd 字段的量化式与量程**。它决定你能实现什么行为，比扭矩上限更先卡住你。

天花板二：执行器饱和——刚度与允许误差的乘积有上界

假设量程够用。物理上 kp 能多大?

一个弹簧被压缩到极限就失效了。关节也一样：

```
τ_max = kp · Δq_max        ⇒      kp ≤ τ_max / Δq_max
```

这条式子的读法很重要：kp 不是一个独立参数，它是“你允许这个关节在满力时偏多少”的倒数。

代 G1 的膝（取官方配置里的 139 N·m [厂]，页面上的 90/120 也一并记下，见第 02 章冲突表）：

允许的满力位置误差	需要的 kp	物理上意味着
10° = 0.175 rad	800 N·m/rad	走路时的正常需求
2° = 0.035 rad	3,970 N·m/rad	精确装配、钻孔
0.5° = 0.0087 rad	15,900 N·m/rad	已经接近“位置伺服”而不是“力控”

（[算]。）

而请注意一个反直觉的事实：**你其实不需要 kp 那么大**。因为一个合格的控制器不该靠误差来产生扭矩——那是前馈项 tau_ff 的活。kp 只负责消除模型误差与扰动，理想情况下 Δq 应该接近零。

所以“把 kp 调大来让位置更准”是新手最常犯的错。正确做法是把前馈给准（第 19 章的逆动力学就是干这个的），让 kp 只处理残差。kp 越大，系统对**测量噪声**和**接触冲击**越敏感——因为 dq 上的一个噪声尖峰会直接乘上 kd 变成力矩冲击。

天花板三：稳定性——刚度是一个谐振频率的平方

现在到真正硬的那一道。

一个关节带着负载，等效成一个质量块接在弹簧上。它的固有频率：

```
ω_n = √(kp / J)

J = 折算到关节轴的总惯量（转子 + 减速器比平方折算 + 连杆 + 负载）
```

这条式子给出一个双向的坏消息。

坏消息 A：kp 越大，ω_n 越高。 而第 09、10 章说过，电流环的带宽是有限的（约 f_sw/10–20，量级 1–2 kHz [行]）。一旦 ω_n 逼近电流环带宽，“电流环是一个理想力矩源”这个假设就不成立了——级联的 5–10 倍分离判据（第 12 章）被打破，系统开始振荡。

工程判据：

```
ω_n ≤ ω_current_loop / 5

若 ω_current_loop = 2π × 1000 = 6283 rad/s ⇒ ω_n ≤ 1257 rad/s ≈ 200 Hz
⇒ kp ≤ J × 1257² = J × 1.58e6
```

坏消息 B：J 不是常数。 一个膝关节的 J 取决于腿伸直还是弯曲（连杆质量离轴多远），还取决于手里有没有拿东西。所以 kp 的**上限是随姿态变化的**，而控制器里那个 kp 是你写死的一个数。

这就是“这个参数在 A 姿态好用、在 B 姿态抖”的全部原因。它不是玄学，是 ω_n 撞上了同一个固定带宽的电流环。

代一个量级感受 J：若折算惯量 J = 0.05 kg·m²（**这是本书假设的量级，宇树未公布 G1 的连杆质量与惯量** [未]）：

```
kp ≤ 0.05 × 1.58e6 = 79,000 N·m/rad
```

79,000 远大于第二道天花板的 15,900。**所以在 G1 这个量级的关节上，先卡住你的通常是执行器饱和与协议量程，不是稳定性**。这条判断对选型有用：小关节看量程，大关节看带宽。

（把 J 换成 0.5 kg·m²（髋部带大腿），kp 上限变 790,000，仍然不是瓶颈。稳定性真正开始咬人是在高惯量负载 + 高刚度需求的场景，比如人形端着一个大箱子做精确动作。）

阻尼那一列：为什么 kd 不能随便给

kd 的作用是耗能。它有两个来源完全不同的问题：

一，kd 直接把速度噪声变成力矩。编码器速度是靠差分算出来的，差分是高通滤波：一个 0.011° 的量化台阶（第 14 章）在 1 kHz 采样下会变成一个 0.19 rad/s 的速度尖峰（0.011° = 1.92e-4 rad，除以 1 ms 就是它；本章下面那条公式给的就是这个数）。若 kd = 2 N·m/(rad/s)，这就是 0.38 N·m 的冲击——每个控制周期都来一次。

> 速度量化噪声 ≈ Δq_enc / Δt = 0.00019 rad / 0.001 s = 0.19 rad/s
> （0.011° = 1.9e-4 rad；[算]）

所以真实的关节固件里，dq 一定先滤波再进 kd——而滤波引入延迟，延迟又吃掉相位裕度。**kd 是这三个约束（噪声、延迟、耗能）的折中点**。

二，kd 与 kp 有一组临界关系。二阶系统要不振荡地回到目标，需要阻尼比 ζ ⩾ 1：

> ζ = kd / (2·√(kp·J))　　临界时 kd = 2√(kp·J)

（[行]。）这条给出一个非常实用的口诀：**kd 该给多大，取决于你把 kp 设成了多大**。单独调 kp 再单独调 kd，是调不好的最常见原因。

于是“柔顺”这件事被重新解释

现在能理解第 16 章末尾那句“接触瞬间要软、支撑要硬”到底怎么实现：

```
摆动相（脚在空中）      kp 小、kd 小  ⇒ 关节几乎自由，能耗低、冲击小
触地瞬间                kp 更小 + tau_ff 反向  ⇒ 主动“卸力”，像膝盖软一下
支撑相（扛体重）        kp 大 + tau_ff 承担主要重力矩  ⇒ 位置准
```

关键在 tau_ff 那一列。支撑相里扛住体重的力矩，绝大部分来自前馈（逆动力学算出来的重力项），不是来自 $kp \cdot \Delta q$。如果靠误差来产生这个力矩，你会得到一条“负载越重腿垂得越低”的曲线——那既不稳也难看。

于是有一个值得记住的分工：

前馈负责“应该出多大力”，反馈负责“实际差了多少”。刚度参数只应该处理残差。

一个 kp 设得很大、前馈给得很差的控制器，能站住，但它对模型的每一次误差都在用一记重锤回应。而人形摔倒的常见原因之一，恰恰是模型误差（踩到软的东西、电池电压塌陷导致实际扭矩小于命令值）。

补一条：宇树自己就是这么算 kp 的（官方数已验）

本书把 `unitree_rl_lab` 里 mimic 配置按电机型号派生的 kp，与同仓库公布的 armature（**折算到关节 / 输出轴的等效惯量**——附录 C.2a 与第 43 章逐位证过：把它除回 22.5^2 才回到电机轴那一档。本书早先在这里写“电机轴”，与第 09、37、43 章互斥，已改）合起来开了一次方根：

```
N7520-14.3   kp 40.179 ÷ armature 0.010178  ⇒ ω_n = 62.8 rad/s = 10.00
Hz
N7520-22.5   kp 99.098 ÷ armature 0.025102  ⇒ ω_n = 62.8 rad/s = 10.00
Hz
N5020-16     kp 14.251 ÷ armature 0.003610  ⇒ ω_n = 62.8 rad/s = 10.00
```

```
Hz
W4010-25     kp 16.778 ÷ armature 0.004250  ⇒ ω_n = 62.8 rad/s = 10.00
Hz
```

四档全部落在 62.8 rad/s，最大偏差 0.0%。 而官方配置的注释里写的目标就是 10 Hz 固有频率 [厂]。

（`foc_check.py natural_frequency_check()` 复算。）

这是本章那道天花板最硬的证据：不是本书说 kp 受 ω_n 限制，而是**宇树自己就是用 ω_n = √(kp/J) 反推 kp 的**——四档电机、四个不同的惯量，配出四个不同的 kp，而固有频率严格相同。

顺带第二次验证：官方逐关节 action_scale 等于 0.25 × effort ÷ kp，四档分别得 0.548 / 0.351 / 0.439 / 0.075，与官方配置里那串数逐位吻合 [厂][算]。**所以 action_scale 也不是拍的，是同一个 kp 的另一种用法。**

而对照 velocity 配置里那套手调 PD（100 / 150 / 200 / 40）[厂]——**同一个关节，两种任务，两套刚度。** 这就是第 18 章开头说的「kp 是设计变量而不是物理常数」，在官方文件里留下的两份直接证据。

没有力传感器时，怎么知道被推了

第 14 章只讲完了两条路线里“装传感器”的那一条。另一半问题是：**一颗关节不装力传感器时，怎么知道自己被推了？** 答案的形状只有一句——没有换能器，就只剩守恒律。

把动力学方程当传感器用

关节轴上的力矩平衡只有一行（各项都是带符号的标量，正方向取 q 增加的方向）：

```
J·q̈ = τ_motor − τ_f − τ_g + τ_ext        ⇒   τ_ext = J·q̈ − τ_model
```

J 是折算到关节轴的惯量（kg·m²），q̈ 是角加速度（rad/s²），τ_motor 是电机交出来的力矩（N·m，就是第 04 章那个 k_t·i_q），τ_f、τ_g 是摩擦与重力矩，连同 τ_motor 记作 τ_model。这一行有名字：反作用力观测器（reaction torque observer，更通用的叫法是扰动观测器，disturbance observer / DOB——把一切模型没算到的东西打包成一个等效扰动来估的那类结构）。

为什么它是一个观测器问题，而不是传感器问题？ 因为等号右边没有一项是新硬件：τ_motor 从电流来（第 04、13 章），τ_f 从官方那份 `Fs·tanh(q̇/Va) + Fd·q̇` 来（一个库仑型阈值加一个黏性型系数，第 36、43 章），τ_g 从第 19 章的逆动力学来，q̈ 从编码器差分来。于是你的下限不再由换能器的噪声底决定，而由**这四个被减数各自的误差**决定。传感器路线的敌人是噪声，观测器路线的敌人是模型误差——而模型误差不会因为你把采样率翻倍就变小。

失效点一：J 你不知道（第 14 章）

上一节刚给出官方那份 J：`armature = J2 + J1·i2² + Jr·(i1·i2)²`。**三项里只有转子与两级齿轮，没有连杆，也没有你手里拎的那箱东西。** 产品页不公布 J，训练仓库公布的是这一份关节侧（输出轴）折算；负载那一份官方当随机变量处理——第 21 章的域随机化表里“附加质量 [−1, +3] kg”是四项之一 [厂]。换成惯量看多大：$\delta J \leqslant m \cdot \ell^2$，力臂取第 14 章那个“脚尖离转轴约 0.6 米”当上限，1 kg ⇒ $\delta J \leqslant 0.6^2 = 0.36$ kg·m²，是本书假设的 $J = 0.05$ kg·m² 的七倍。于是模型误差项 $\delta J \cdot |\ddot{q}|$ 正比于角加速度：**你动得越快，“J 不知道”这件事越贵；低速时它自己往后退。**

失效点二：摩擦模型不知道（第 36、43 章）

那条 tanh 曲线在 $|\dot{q}| \gg Va$ 时饱和成常数 Fs（好消息：可以当已知量前馈掉），但在 $\dot{q}$ 过零附近的斜率是 $Fs \div Va = 2.4 \div 0.01 = 240$ N·m/(rad/s)（官方 Va = 0.01 rad/s [厂]）——速度差 0.001 rad/s 就换 0.24 N·m 的模型误差。而本章前面那条差分速度噪声是 0.19 rad/s，比 Va 大 **19 倍**：你的速度信号连自己站在 tanh 的哪一侧都说不清。那为什

么不干脆把摩擦测出来？那就要在输出端装一颗力矩传感器，也就是退回第 14 章的另一条路线。所以换向瞬间的摩擦不是“参数不准”，是**曲线形状在你手里不可分辨**；它在 τ_ext 上的表现，是每次换向打出一记假尖峰。

失效点三：q̈ 是差分出来的（本章那条 0.19 rad/s）

差分是高通：做一次放大 $1/\Delta t$，做两次放大 $1/\Delta t^2$。

```
Δq̈ ≈ 0.19 ÷ 0.001 = 190 rad/s²        τ_noise ≈ J·Δq̈ = 0.05 × 190 = 9.5 N·m
```

9.5 牛米，比你想检出的接触力矩大两个数量级。**所以裸差分两次得到的不是测量值，是一串噪声**。 问题又来了：把采样提上去行不行？按这条模型算反而更糟——Δq̈ 随 f_s 平方涨，滤波收益只随 √f_s 缩；真实实现因此不靠裸差分，而是用一个状态观测器从模型里把 q̈ 推出来 [行]。剩下的出路只有拿带宽换噪声：一阶低通（时间常数 $1/\omega_f$，ω_f 就叫观测器带宽，rad/s）把白噪声的方差压到 $\omega_f \div (2\,f_s)$ 倍，幅值按它的平方根缩 [行]。这里假设噪声谱是平的，而量化噪声其实与信号相关——这是这条近似开始说谎的地方。

```
幅值收益 = √(ω_f ÷ 2f_s) = √(100 ÷ 2000) ≈ 0.22      ⇒   噪声底 9.5 × 0.22 ≈ 2.1 N·m
```

（f_s = 1 kHz，与本章那条 0.19 rad/s 同一个采样口径；要在 10 ms 里检出一次接触，ω_f 就得 ⩾ 1 ÷ 0.01 s = 100 rad/s。）

能被检出的最小外力是一条判据，不是一个数

```
τ_ext,min ≈ max( J·Δq̈·√(ω_f ÷ 2f_s) ≈ 2.1 N·m ,  Fs = 2.4 N·m ,  δJ·|q̈| )
```

前两条恰好同量级，都压在 2 N·m 附近；第三条随你动多快往上爬。下限凭什么不由采样决定？因为它挂在模型项上：**由最差的那一项模型误差，和“你肯让接触事件晚知道多久”共同决定**，而 1/ω_f 就是那个“晚多久”——它和本章末尾“软件弹簧来不及变软”是同一道延迟换了科目。2 N·m 换成脚上多少牛，本书不做：力臂不是髋高（第 02 章那条静力估算的教训在这里一样成立）。而对照第 14 章那条厂商口径的 0.02 N·m，**差两个数量级，再堆滤波器也追不上**。

为什么低速准直驱做得动、高齿隙高速做不动

一，**低速**：|q̈| 小 ⇒ δJ·|q̈| 后退；摩擦早已饱和成常数 ⇒ 可前馈；需要的 ω_f 低 ⇒ 噪声底按 √ω_f 掉。三个失效点同时后退。

二，**高齿隙**（第 14 章那 187 倍的空程）：空程里齿轮没咬上，τ_motor 全部用来加速转子自己。这一刻“关节轴的 J”从整条臂跳到只剩转子加齿轮，而观测器用的是一个写死的 J。前面说齿隙的谎是有结构的，这里露出另一面：结构可以被补偿，也可以把另一个模型的假设当场打破。

三，**高速**：ω_f 必须提上去 ⇒ 噪声底按 √(ω_f ÷ 2f_s) 涨，而第 12 章那条带宽分离与本章天花板三压着 ω_f 不许随便提。

所以准直驱（第 14 章那条低减速比路线）在力这条线上的真正收益，不是“响应快”，而是让 τ_ext 那一行的左边与右边描述同一根轴、同一个 J、同一段动力学。第 14 章说准直驱是为了让第 04 章那条式子在输出端还成立；这里补完后半句：观测器用的那条式子，也在输出端才成立。

至于宇树有没有在 G1 上做这件事：**没有公开材料说明它的碰撞检测是怎么实现的 [未]**。`tau_est` 只说明关节端在算一个力矩估计值（第 12、14 章），而它是不是反作用力观测器、阈值多少、跑在哪个频率，官方一处都没写。本节给的是判据与三条线的算术，不是宇树的实现。

类比，以及它在哪儿破

类比：关节是一段弹簧，而“这段弹簧有多硬”是一个你可以每毫秒改一次的数字。

这个类比在哪里破：真实弹簧的刚度由材料与几何决定，改不了；而这里的“刚度”是一个**反馈增益**，它有三个致命区别——它需要能量（弹簧不用维持力，关节要一直喂电流）、它需要测量（有噪声与延迟）、它有带宽（不能无限快）。

第二点最要紧：**一段真正的弹簧在被撞的那一毫秒就响应，而软件弹簧要等一次 ADC 采样、一次 PI 计算、一次 PWM 更新**。这个差别意味着：一个冲击如果比电流环带宽还快（比如脚掌砸水泥地，冲击上升时间在毫秒内），**软件弹簧根本来不及变软**，它在那一瞬间表现得比任何物理弹簧都硬。

这就是为什么人形机器人的脚底要有缓冲材料、为什么落地角度要控制、为什么第 22 章的接触求解器差异会那么要命。

一个可检验的说法：一个关节能有多硬由三道天花板共同决定——协议量程（M8010 的 kp 上限 25.599 N·m/rad，对应满扭矩时偏 53°）、执行器饱和（kp ⩽ τ_max/Δq_max，G1 膝若允许 2° 需 3,970 N·m/rad）、稳定性（ω_n = √(kp/J) 必须 ⩽ 电流环带宽/5）；在 G1 这个量级上先卡住人的通常是前两道，且 kp 与 kd 通过临界阻尼 kd = 2√(kp·J) 绑定，不能独立整定。

证据等级：kp/kd 的官方量化式与量程 [厂]；τ = tau_ff + kp·Δq + kd·Δω 与字段命名 [厂]；ω_n、阻尼比、速度量化噪声估算、79,000 与 3,970 等所有换算 [算]；电流环带宽与 f_sw/10 的关系 [行]；**惯量与摩擦：产品页未公布，训练仓库公布了 armature 与 Fs/Fd（第 43 章 C.2a），但那是电机侧折算、不含连杆与负载；G1 各关节出厂固件的实际 kp/kd 仍未公布** [未]；“软件弹簧来不及变软”是机理推导 [行][算]；反作

用力观测器与 DOB 的结构、一阶低通的 √(ω_f ÷ 2f_s) 噪声收益、以及 190 rad/s²、9.5、2.1 N·m、0.36 kg·m² 这些数全是 [行][算]（由本章那条 0.19 rad/s、第 14 章的 0.6 m 与本书假设的 0.05 kg·m²、官方 Fs = 2.4 N·m 推出）；**宇树的碰撞检测怎么实现，官方从未公布 [未]，本节没有替它填**。

可核对清单：

- ☐ 跑 `foc_check.py` 的“18 刚度上限”，确认 53.0° 与 4.8° 这两个数
- ☐ 手算一次 ω_n = √(kp/J)：取 kp = 1000、J = 0.05，看它是不是 141 rad/s ≈ 22 Hz，离电流环带宽还有多远
- ☐ 用 kd = 2√(kp·J) 反推：kp = 1000、J = 0.05 时临界 kd 是多少（答：14.1），再对照官方 kd 量程看是否发得出去
- ☐ 找一份开源关节固件（moteus 的配置文件）看它默认给多大 kp/kd，与本章三道天花板对一遍
- ☐ 手算一次 0.19 ÷ 0.001 × 0.05 = 9.5 N·m，再乘 √(100 ÷ 2000) ≈ 0.22，确认“不装传感器时能检出的最小力矩约 2 N·m”是算出来的，并且与厂商口径的 0.02 N·m 差两个数量级

下一章：本章讲的是单个关节的软硬。下一章讲 23 个关节怎么被同一次数值求解同时决定——以及为什么这件事的计算量随自由度按三次方涨。

第 19 章

全身控制与 MPC：一次 1 kHz 里要解完的那堆方程

23 个未知数，一毫秒解完

上一章讲了一个关节的软硬。现在的问题是：一台 G1 有 23 个（EDU 版 29 个）关节，同时脚踩在地上、手要够到某个位置、躯干不能歪、关节不能超限、地面不能给拉力——**这些要求互相冲突时，谁让步？**

这就是全身控制（Whole-Body Control，WBC）要解的东西。它的输入是一堆任务和约束，输出是 23 个扭矩数，交给第 12 章那条链。

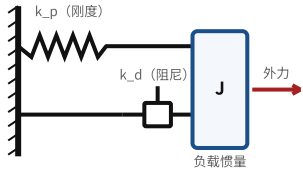
本章不打算教你怎么写 WBC（那是另一本书），只打算给你一件对工程师最有用的东西：**这件事的计算量随自由度怎么涨，以及为什么这条曲线决定了人形机器人的形状**。

我觉得，在全身深度强化学习上，很多核心问题其实已经基本被解决了，接下来更多是继续完善。

王兴兴 — 王兴兴 GTC 2026 演讲文字整理（“机器人前瞻”整理；页面自陈“在不改变原意的前提下进行了一定程度的增删修改”），腾讯新闻 — 2026-03-17（演讲整理稿（非逐字，页面已声明增删修改））

同一个电机，改三个数就换一种“材质”。物理上没有任何东西被换掉，换掉的是误差到扭矩的映射——刚度是软件写的。

它等效于这样一根弹簧



推它，它让；撤力，它回。让多少、回多快，全由 k_p 与 k_d 说了算。

控制器里真正跑的式子（宇树口径：一条混合公式）

```
τ = τ_ff + k_p (q_des − q) + k_d (ω_des − ω)
```

位置误差 → 乘 k_p → 直接变扭矩，中间没有独立的位置环。

为什么“软”和“硬”是同一台机器

想要	擦桌子	端住水杯	敲键抓抛
k_p	几十	一两百	数百
被推	让很多	让一点	几乎不让
回位	慢	快	立刻顶回
代价	误差大	谐振余量	齿隙放大
	软	中	硬

上限从哪来：$\omega_n = \sqrt{k_p / J}$。刚度翻 4 倍，固有频率只翻 2 倍；
但它很快会撞上编码器分辨率、齿隙与 1 kHz 采样——撞上就是振荡。（第 12、14 章）

图 19-1　可编程的弹簧：同一条混合公式，改 k_p 就换一种「材质」

先看清楚要求有几类

类别	内容	在优化里的身份
动力学等式	$\tau = M(q)\ddot{q} + C(q,\dot{q})\dot{q} + g(q) - J^{T}F$	等式约束（不能违反物理）
接触约束	脚不穿透地面、不滑动、法向力 $\geqslant 0$	不等式约束
任务	重心轨迹、末端位姿、姿态、避免自碰	目标函数（可以打折）
执行器限制	$\lvert\tau\rvert \leqslant \tau_{max}$、$\lvert\dot{q}\rvert \leqslant \omega_{max}$、$q$ 在行程内	盒式约束
摩擦锥	$\lvert F_{切向}\rvert \leqslant \mu \cdot F_{法向}$	二阶锥约束（或它的多面体近似）

关键结构：第一类是“物理不许你违反”，第三类是“你希望尽量满足”。所以这天然是一个**带约束的二次规划（QP）**：在满足前两类与后两类硬约束的前提下，最小化第三类的加权误差平方和。

优先级也可以显式表达（严格分层：先保证不穿透地面，再保证重心，再保证手的精度），这叫任务优先级（task-priority）或分层 QP。

计算量：为什么自由度是原罪

设关节数 n。这个 QP 的决策变量通常是 $\ddot{q}$（n 维）加上接触力（每个接触点 3 维，两只脚就是 6 维）。

```
矩阵规模 ≈ (n + n_c) × (n + n_c)
求解一个稠密 QP 的工作量 ≈ O(m³)，m 是变量维度
```

代具体数：

```
G1  23 DoF + 6 维接触力 = 29    ⇒ 29³ = 24,389
G1  29 DoF + 6          = 35    ⇒ 35³ = 42,875
H2  31 DoF + 6          = 37    ⇒ 37³ = 50,653
```

（[算]。）

从 23 个自由度到 31 个，一次求解的工作量涨 2.08 倍。而这条曲线是三次方的：如果哪天做一台 60 自由度的人形，$(60+6)^3 \div (23+6)^3 = 11.8$ 倍——注意这是**相对 23 自由度那一档**；相对刚才那台 31 自由度的机器，是 $66^3 \div 37^3 = 6.3$ 倍。基址不同，倍数差一倍，这类“涨几倍”的说法必须带基址。

再看时间预算。第 12 章说过整机低层话题是 500 Hz [厂]，控制层通行量级是 1 kHz [行]：

```
1 kHz ⇒ 每次求解预算 1 ms
```

1 毫秒解一个 37 维 QP，在嵌入式 x86 上是**可行的但紧张**（商用求解器通常几十到几百微秒），在关节 MCU 上是不可能的。所以真实的架构一定是：

```
主控（x86 或 Orin）跑 WBC/MPC  →  1 kHz  →  分发 23 个 τ_cmd
关节 MCU 跑级联三环            →  kHz 量级 →  占空比
```

这条分工不是软件偏好，是 $O(m^3)$ 与 1 ms 这两条线交出来的。

（宇树从未公布整机是否在线跑 WBC 或 MPC，也没公布求解器与频率[未]。它公开的运控工序是强化学习（第 21 章）。本章给的是“如果显式解，代价是多少”。）

模型预测控制：把“下一步”变成“下 N 步”

QP 只决定当前这一瞬间的加速度。MPC 的差别是：**它把未来 N 步的模型一起展开，一次性优化整段轨迹，然后只执行第一步，下一周期重来**。

这带来一个 WBC 给不了的能力：**它能提前准备**。

对第 17 章的捕获点问题，这一点是决定性的。纯反馈的 ZMP 控制器是“已经开始歪了才纠正”；MPC 是“我预见到 200 毫秒后我会歪到那里，现在就把这一脚迈过去”。而**摆动相需要 200 毫秒**——也就是说：

任何只看当前时刻的控制器，在物理上来不及决定落脚位置。

这就是为什么双足走路绕不开某种形式的预测（显式 MPC，或者被训练进 RL 策略里的隐式预测）。

代价是模型规模再乘一个视界：

```
N 步视界 ⇒ 变量维度 ≈ N × (n + n_c)
N = 20 步、37 维 ⇒ 740 维 QP ⇒ 740³ ≈ 4 亿次运算量级
```

（[算]。）**所以人形的 MPC 视界通常只有几十步、模型通常被简化成线性倒立摆（LIPM）**——不是不想用完整模型，是 1 ms 装不下。

这里有一个非常值得工程师记住的取舍：**MPC 的精度受限于你能塞进实时预算里的那个模型的复杂度**。于是“用什么样的简化模型”就变成了控制研究的核心问题，而 LIPM（第 17 章那套）之所以活了几十年，就是因为它在“够用”与“算得起”之间的那个点上。

真正爆炸的不是维度，是接触模式

上面所有算术都有一个隐藏前提：**你知道脚什么时候着地**。

如果不知道，问题就变成：单脚支撑、双脚支撑、腾空、三点接触、躯干撞地……这些**接触模式**的组合。走 N 步、每步有 3 种模式，就是 3^N 种序列。这是一个组合爆炸，任何“枚举再挑最好”的方法在几十步视界上直接死掉。

工程上三条出路：

一、预设模式序列。 你告诉求解器“第 0–200 ms 左脚着地，200–400 ms 双脚，400–600 ms 右脚”。这就是传统步态规划。代价：不预设就不会动，被推一下导致模式变了就崩。

二、混合整数规划（MIQP）。 让求解器自己选模式。数学上优雅，计算上是 MIP——**指数复杂度**。实时预算里基本不可能。

三、不显式处理。 用自由轨迹（foot placement 由策略决定）或者干脆用强化学习，让网络自己内化“什么时候该踩哪里”。

第三条是 2016 年之后足式运控赢的那条，而它赢的原因不是网络更聪明，是**它把一个指数复杂度的问题换成了一个固定规模的前向推理**。一次策略推理的算力只取决于网络大小，与“未来有多少种接触序列”无关。

这就是第 21 章存在的理由。 但请注意代价：你换掉了指数爆炸，也换掉了可解释性——出问题时你无法说“是哪一条约束被违反了”。

于是可以解释一台人形长什么样

把本章的算术倒过来读，能解释几个看起来是审美选择的东西：

为什么腰只有 1–3 个自由度？ 每加一个自由度，QP 规模按三次方涨，而腰对“不摔”几乎没有贡献（它在重心正上方，不改变支撑多边形）。G1 基础版腰 1 个、H2 三个、G1+ 是立柱加底盘。**腰部自由度是拿算力换灵活性的，而算力在 1 kHz 那一层最贵。**

为什么手腕有 3 个自由度但只有 5 N·m？（第 15 章的 W4010-25。）因为腕不参与平衡，它的力矩需求只由“把手掌摆到某个朝向”决定，而那个需求的量级就是几牛米。省下的重量全砸在腿上了。

为什么手指是另一套器件？（第 25 章。）因为手的接触模式数量比脚多一个数量级（每个指节都可能接触），如果把它塞进同一个全身求解里，QP 规模直接爆掉。所以工程上手的控制是**解耦**出来的——它是另一个环，不与平衡抢那 1 毫秒。

类比，以及它在哪儿破

类比：WBC 像一个同时管 23 个员工的经理，每个员工有自己的上限（执行器限制），部门之间还有硬性依赖（动力学），而经理每一毫秒要给出 23 条指令。

这个类比在哪里破：经理可以给“尽量做好”这种模糊指令，而 QP 必须把所有目标写成**加权平方和**——也就是说，你必须提前决定“手的精度值多少、躯干姿态值多少”。这些权重是控制器的真正设计变量，而它们**没有物理正确答案**。同一个机器人，换一组权重，走路姿态完全不同。

这就是为什么足式控制论文里“任务权重怎么调”通常被含糊带过——它是手艺，而手艺不能证明。

一个可检验的说法：全身控制的标准形式是带约束的二次规划，其变量维度约为关节数加接触力维度，稠密求解工作量按三次方涨，因此 G1 的 23→29 自由度使单次求解成本涨 1.76 倍、到 H2 的 31 自由度涨 2.08 倍；在 1 kHz 控制周期（1 ms 预算）下，这决定了 WBC 必须跑在主控而非关节 MCU 上；而真正的组合爆炸来自接触模式而非维度，这是显式优化路线让位于强化学习的计算原因。

证据等级：QP 结构、$O(m^3)$、LIPM 简化、混合整数的指数复杂度、摩擦锥与接触约束，全部是优化与足式控制的标准内容 [行]；$29^3/35^3/37^3$ 与 2.08 倍、740^3 是 [算]（**假设稠密求解**，实际用稀疏/迭代法会显著不同，本书给的是量级直觉不是实现预测）；500 Hz 低层话题 [厂]、1 kHz 控制层量级 [行]；G1/H2 的自由度分解 [厂]；腕关节 5 N·m 与腰部配置 [厂]

（`unitree_rl_lab` 配置）；**宇树是否在线跑 WBC/MPC、用什么求解器、什么频率，全部未公布** [未]；“腰自由度是拿算力换灵活性”是本书推断 [算][行]。

可核对清单：

- ☐ 自己算 $(23+6)^3$、$(29+6)^3$、$(31+6)^3$，确认 24,389 / 42,875 / 50,653
- ☐ 把决策变量维度改成“接触力也算未知数、两只脚各 6 维”再算一次，看它怎么涨
- ☐ 找一个开源 QP 求解器（OSQP、qpOASES）的基准表，看它在 30–40 维上实测多少微秒——对照 1 ms 预算
- ☐ 想清楚为什么“手的控制必须与平衡解耦”这句话是算力约束而不是设计偏好

下一章：本章反复提到“1 ms 预算”。下一章换一个问法：这台机器跑 2 小时要多少焦耳，其中有多少真的用来移动身体，又有多少只是在“接着自己往下掉”。

第 20 章

一次抬腿的焦耳账

电池里到底有多少东西

先把官方给的数摆齐（全部 [厂]，抓取 2026-10-07/08）：

型号	官方电池口径	换算能量	官方续航	反推平均功率	体重	W/kg
G1	13 string lithium battery、9000mAh、Charger 54V 5A [厂]	≈ 0.43 kWh（推）	Battery Life About 2h [厂]	≈ 216 W	About 35kg [厂]	6.2
H1	15 Ah / 0.864 kWh，最高电压 67.2 V [厂]	0.864 kWh	**官方未公布小时数**（页面 Endurance 行填的是 Battery capacity: 864Wh, quickly replaceable）[未]	**不给**（除数读不到）	47 kg（Weight about 47kg [厂]）	**不给**
H2	15 Ah / 0.972 kWh，最高 75.6 V [厂]	0.972 kWh	**官方未公布小时数**（2026-10-09 回读 unitree.com/H2 整页没有续航行）[未]	**不给**（除数读不到）	70 kg（About 70kg [厂]）	**不给**
R1	未公布容量与电压 [未]	—	约 1 h	—	—	—
对照：人	70 kg 步行 65 Wh/km [查]	—	5 km/h	325 W（代谢）	70 kg	4.6

（“反推平均功率” = Wh ÷ h，是 [算]；**宇树从未公布任何一款人形的整机功耗**，所以这一列是从它公布的两项推出来的，误差会把两项的误差都吃进去。）

这张表里最该被注意的一行是最后一行的对照：**人走路约 4.6 W/kg，G1 是 6.2 W/kg**。为什么用 G1 不用 H2——只有 G1 的续航小时数在官方页面上写着（`Battery Life About 2h`），这个比值才三个输入全是一手。

一个 70 公斤的人以 5 km/h 走路，代谢功率约 325 瓦；一台 70 公斤的人形做类似动作，电功率也在 320 瓦量级。**数字上它们打平了**。

> 机器人是一种介于3C消费电子和汽车之间的中间业态，工程量和复杂度低于汽车，并且电池本身的价值占比不大，汽车的关键技术指标是续航，机器人则不是那么在意续航，大家对于机器人最关注的还是智能化。
>
> 王兴兴 — 周天财经对谈王兴兴（雪球嘉年华文字实录；页面自陈“为文气通顺，有适度书面化微调”），钛媒体 — 2025-01-13（对谈实录（经书面化微调，非逐字））

那为什么人能动一整天，机器只能动三小时

因为那 325 瓦里，人只有一小部分需要“烧掉”。

人的代谢功率是**输入**，不是输出。走路时人体效率约 20–25%（其余变成热）[行]。所以 325 瓦代谢里约 70–80 瓦是机械功，其余是散热——而人散热靠出汗，不需要背包里装一块 20 瓦的空调。

机器的 216 瓦（G1）是**电输入**，它经过电池（≈ 无损失）、母线（小损失）、驱动板（开关与导通损耗）、电机（铜损铁损）、减速器（机械摩擦）之后，剩下多少变成有用的机械功？

```
η_total = η_drive × η_motor × η_gear
典型量级：0.9 × 0.8 × 0.85 ≈ 0.61      [行]
```

所以 G1 的 216 瓦电输入里，按本章那条效率账约六成变机械功（≈ 130 瓦）、四成在器件里变成热（≈ 86 瓦）——**而这 124 瓦必须被带走，否则第 13 章那条正反馈就开始跑**。

于是“人 4.6 W/kg、G1 6.2 W/kg”这个对比是**假对比**：人的那一栏是代谢输入，机器的那一栏是电输入，两者之间隔着一个 20–25% 的效率。真正可比的是“机械功 / 体重”，而那一栏机器反而更省。

那机器的劣势到底在哪？在下面这一节。

关键差别：人站着几乎不耗能，机器站着很耗能

这是本章真正想给的那个数。

人生理上有一个几乎免费的能力：**站立时肌肉可以处于低激活状态，靠韧带与骨骼的被动结构扛住重力**。这也是你站两小时不会累到崩溃的原因（虽然会麻）。

机器人没有“被动结构”这个选项——第 16 章说过，双足的重心投影必须持续被主动纠正，而每一次纠正都要电流。更糟的是：

> 一个关节维持扭矩 T 时，铜损 = $I^2R = (T/k_t)^2 \cdot R$

注意这个式子的性质：只要扭矩不为零，损耗就不为零，而且与转速无关。一个静止但顶着重力的关节，和一条正在高速运动的腿，可能烧掉同样多的铜。

这就是“额定扭矩”为什么在人形上比在四足上重要得多（第 36 章）：**四足的额定是“能持续动多久”，人形的额定是“能持续站着多久”**。

（G1 官方续航约 2 小时，且官方语境里这是“综合工况”。它没有区分“站立”与“行走”的功耗 [未]。本书无法算出“纯站立”那一档，这是一个真实的空白。）

一次抬腿的焦耳账

现在做一件具体的事：G1 把一条腿从地面抬到 0.2 米高。

> 这条腿的质量：本书按 MJCF 类结构估约 4 kg（G1 整机 35 kg ÷ 8 条主要肢体量级）[算][未]

```
抬升重心：约 0.10 m（腿不是质点，抬 0.2 m 时重心只升约一半）
势能增量：m·g·h = 4 × 9.80665 × 0.10 = 3.9 J

加上动能（抬升过程末速约 0.5 m/s）：½mv² = 0.5 J
机械功合计 ≈ 4.4 J
```

再往回算电：

```
理想下界（抬起来又放下去、能量全回收）：0 J     ← 不可能，因为你没法回收
现实（放下时势能变热）：3.9 J 机械 → ÷ 0.61 ≈ 6.4 J 电
一步的账：走路时每条腿每步都要抬一次
步频 1.3 Hz ⇒ 每秒抬 2.6 次 ⇒ 2.6 × 6.4 = 16.6 J/s = 17 W
```

（`arithmetic_check.py` 的能量链同款推导 [算]；腿质量是本书估读 [未]。）

17 瓦，只用来“抬腿”。而 G1 的平均功率是 216 瓦。也就是说：**抬腿只占 7.9%**。

那 95% 去哪了？三个地方：

1. **支撑相扛体重**（最大项，且完全不做机械功——它是纯铜损）；
2. 摆动相的加减速与关节间能量来回传递；
3. 大脑、感知、通信、待机电源（第 26 章会算这一项，它比想象的大）。

第 1 项就是第 16 章那句“没有休息窗口”的焦耳版本。而它在四足上可以接近零。

用 CoT 把这件事变成一个可比较的数

学界比较“移动效率”的标准量是**代谢代价系数（Cost of Transport）**：

```
CoT = E / (m · g · d)        E = 消耗能量，m = 质量，d = 距离
```

请注意 **CoT 是无量纲的**（焦耳 ÷ (kg × m/s² × m) = 1）。本书曾经把它当成有量纲的量写错过一次，被自己的自检脚本抓到——这也是为什么附录 B 的脚本里有一条专门守单位的用例。

现在把这三个输入一个一个往下问，看它在人形上到底缺哪一项。E 是靠什么给的？靠官方电池容量，有。m 是靠什么给的？靠官方体重，也有。d 是靠什么给的？——**宇树的人形页面没有这一栏**。

```
E：G1 ≈ 0.43 kWh、H1 0.864 kWh、H2 0.972 kWh   [厂][算]
m：G1 35 kg、H1 47 kg、H2 70 kg                [厂]
d：G1 页面写着 `Battery Life About 2h` [厂]；H1 那页的 `Endurance` 行填的是 864Wh 而不是小时数，H2 整页没有续航行 ⇒ 那个 3 h 是宣传语，不是页面上可回读的参数 [未]
```

为什么“小时”这一栏替不了“公里”？ 因为这两个数分母不同族，量的不是同一头。小时量的是功率这一头：

```
P = E ÷ t
```

公里量的是效率这一头：

```
每公里耗电 = E ÷ d ⇒ CoT = (E ÷ d) × 3600 ÷ (m · g · 1000)
```

把两条式子摆在一起就能看出，小时数不是公里数的粗糙近似，而是另一个物理量。一台机器站着不动把两个小时烧完，那时 E 有数、d = 0，CoT 这个除法连分母都没有；同一块电池、同样一句“综合工况约 2 h”，慢走与快走的小时数几乎不变，而 d 能差出好几倍，于是 CoT 也差出好几倍。**小时是功率的量，公里才是效率的量**——前者不携带“这机器干了多少活”这一项信息。

那人形现在唯一还能公开算出来的效率侧的量是什么？是**比功率**（电输入 ÷ 体重，单位 W/kg），它不带距离：

```
G1：0.43 kWh ÷ 2 h = 216 W，216 W ÷ 35 kg = 6.2 W/kg    [算]
G1：432 Wh ÷ 2 h = 216 W，216 W ÷ 35 kg = 6.2 W/kg   [算]（三个输入全是一手）
H1、H2：官方没有续航小时数 ⇒ 本书不给它们的平均电功率与功率密度
```

这三行就是本书能为宇树人形算到的全部效率账，三行的算式都只用了同一张表里的两个官方数。它们够设计电池与散热（第 26、35、36 章按的正是这个数），不够回答“送一次货要几毛钱电费”——那一栏的分母是 d。

那 d 又靠什么补上？ 只有两条路：厂商公布一次带里程的续航口径，或者用户自己拿卷尺与秒表跑一遍。这两条现在都不成立：官方从未公布整机功耗、每公里能耗、可行驶里程 [未]，“综合工况约 2 小时”里也没有区分站立与行走 [未]。

对照（**仅作能量对照，非本书解剖对象，不进本章论证链**）：宇树四足 AS2 因为官方给了公里数才算得出 CoT——648 Wh ÷ 20 km = 32.4 Wh/km，自重 20 kg ⇒ CoT 0.59 [算]（32.4 Wh/km = 116.64 J/m，除以 20 × 9.80665 = 196.13 N 得 0.5948）；Go1 改装的 RAIBO2 跑完 212 km 赛事（2.64 m/s），CoT ≈ 0.25 [查]；同一把尺子下人步行 CoT ≈ 0.20–0.37（文献口径，本书取 0.34 做量级）[查]。**四足算得出 CoT 是因为那一栏被公布了，不是因为它比人形高效**。

于是本章最有价值的输出是一个空格：目前无法用公开数据计算任何一款宇树人形的 CoT，因为官方页面只给“续航约 X 小时”，不给“能走多少公里”。而 CoT 需要距离。

这不是小事。CoT 是“这台机器干活划不划算”的核心指标——它决定一小时电费与一次充电能完成多少工作。**没有公开的公里数，就没有可比较的效率**。一个想把人形卖进工厂的场景里，这是必须补上的一栏（第 38 章会回到这个话题）。

类比，以及它在哪儿破

类比：电池是油箱，续航是里程。但机器人更像一个**怠速就要烧满油的发动机**——它停在那里不动也在耗，因为“停着”这件事本身需要主动维持。

这个类比在哪里破：怠速的发动机不产生任何反作用力，而人形的关节在“静止”时输出的是**几十牛米的持续扭矩**。它不是怠速，它是**举着杠铃不动**。而举着杠铃不动之所以累，正是因为肌肉（和电机）在等长收缩时仍然在烧能量——物理上没做功，生理上和电气上都在花钱。

“不做功但耗能”是这台机器最反直觉、也最决定成败的一条。

一个可检验的说法：G1 从官方电池能量、官方续航小时与官方体重三个一手数相除，得平均电功率 216 W、功率密度 6.2 W/kg，而人类步行 5 km/h 的代谢功率密度约 4.6 W/kg——两个数不可直接比较（一个是电输入、一个是代谢输入，人体机械效率仅 20–25%）；一次抬腿约耗 6.4 J 电、按 1.3 Hz 步频折合约 17 W，占 G1 整机平均功率的 7.9% [算]，其余主要花在支撑相“不做功但持续出扭矩”的铜损上。**H1 与 H2 本书不给瓦数**：能量与体重是一手，但官方页面上没有续航小时数（H1 的 `Endurance` 行填的是 `864Wh`，H2 整页没有续航行），除数读不到就不写商。而由于宇树只公布续航小时数、不公布公里数，任何宇树人形的 CoT 目前都无法用公开数据计算。

证据等级：电池容量、电压、续航小时、体重 [厂]；0.43 kWh 与所有 W/kg、17 W、6.4 J、0.59 是 [算]，其输入含本书估读（腿质量、抬升高度、效率 0.61、末速 0.5 m/s）[行][未]；人体效率 20–25%、CoT 定义与人类文献值 [查][行]；RAIBO2 的 0.25 与 2.64 m/s 来自 Nature 论文口径 [查]；**宇树从未公布整机功耗、耗电量、每公里能耗、以及任何一款人形的行驶里程** [未]。

可核对清单：

- ☐ 手算一次 CoT 的单位：J ÷ (kg × 9.80665 m/s^2 × m) = 1，确认它无量纲
- ☐ 用 432 Wh ÷ 2 h ÷ 35 kg 复算 6.2 W/kg；再去 `unitree.com/H1`、`unitree.com/H2` 自己找一遍续航小时数，确认找不到——这就是本书不给那两行瓦数的理由

- ☐ 用 648 Wh ÷ 20 km ÷ 20 kg 复算 AS2 的 32.4 Wh/km 与 CoT 0.59（0.5948，别四舍五入成 0.6 再当成两个数）
- ☐ 去宇树 G1/H1/H2 页面找“能走多少公里”这一栏，确认它不存在（这是本书的一条断言，值得你亲自验一次）

下一章：本章说明能量这条线卡得很死。下一章讲另一条线：既然显式建模这么贵、能量又这么紧，为什么现在的人形开始用神经网络来走路——以及宇树公开的那套工序里，每一步在防哪一种失效。

2026-10-09 一手回读（把这张表的两类数分开看）：`unitree.com/g1`、`unitree.com/h1`、`unitree.com/H2` 三个页面逐字读下来——**G1 的“About 2h”是页面上写着的一手数**，所以 216 W 这一格是“两个厂商数相除”；**H1 的 `Endurance` 行填的是容量 `864Wh`，H2 整页没有续航行**，所以那两个“约 3 h”只是宣传语，页面上读不到 [未]——**本书因此不给 H1、H2 任何瓦数与 W/kg**，表里那两格留空。上一版这里印着 288 W 与 324 W，等于把一个读不到的除数当成厂商数据用了两轮，已删。宇树从未公布任何一款人形的整机功耗，这一点全章照旧。**顺带一条取数手艺**：`unitree.com/h2` 是 404 而 `unitree.com/H2` 是 200——**它的 URL 大小写敏感**，探不到页先换大小写，别急着断定页面下架。（`R1` 页返回的是空壳，未能回读。）

第 21 章

强化学习运控：宇树公开的这条工序

为什么现在的人形用神经网络走路

第 19 章给出了答案的一半：接触模式是组合爆炸，显式优化算不起。另一半是：**就算你算得起，你也建不准**。

一台真实人形的模型里，你需要知道的质量、惯量、摩擦系数、电机扭矩常数、减速器效率、脚底材料刚度——**第 1 部已经数过一遍，宇树连其中一半都没公布** [未]。而你只需要一个参数错得稍微多一点，1 kHz 的高增益控制器就会开始抖（第 18 章的 ω_n 撞带宽）。

强化学习的解法非常工程师：**别猜这些参数了，把它们当成随机数，在一大片参数云里训练一个对参数不敏感的策略**。

这一章讲宇树公开的这条工序，以及每一步在防哪一种失效。

> 如果要进一步机器人能力提升，也就是机器人 RL 的 Scaling Law，还是做得非常不好。……我训练一个机器人做新的动作、跳新的舞蹈，都要重新训练，还是从头开始训练，这是非常不好的一个事情。……因为 RL Scaling Law 在语言模型上已经是充分验证过的事情。但在机器人的运动控制上面，大家才刚刚开始。
>
> 王兴兴 — 王兴兴 2025 世界机器人大会主旨演讲（新浪“机器人前瞻”文字实录） — 2025-08-09（第一人称演讲逐字）

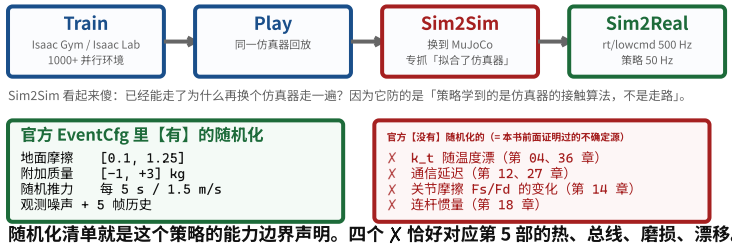


图 21-1　公开的工序，与域随机化到底覆盖了什么（四个 ✗ 是本书的发现）

官方公开的四个步骤

宇树在 `unitree_rl_gym` 与 `unitree_rl_lab` 两个仓库里把工序写成四步[厂]：

```
Train      在 Isaac Gym / Isaac Lab 里并行上千个环境，用 PPO 训练
Play       在同一个仿真器里回放，看策略行不行
Sim2Sim    把训好的策略搬到 MuJoCo（unitree_mujoco）里再跑一遍
Sim2Real   导出到真机，跑在 rt/lowcmd 上
```

注意第三步 Sim2Sim 是整条工序里最值得学的一步，而且它看起来很傻。 你在 Isaac 里已经能走了，为什么还要换一个仿真器再走一遍？

因为它防的是**“过拟合到仿真器”**。

两个仿真器的接触求解器、积分器、碰撞几何处理都不一样。一个策略如果只在 Isaac 的接触模型下能走，它其实学到的是“Isaac 的物理怎么算”，不是“怎么走路”。搬到 MuJoCo 就摔——**而这在真机上一定会发生，只是 MuJoCo 让你提前两周知道。**

这是本书想给读者的一个可迁移的方法论：**任何“在 A 环境训好、要在 B 环境跑”的系统，都应该有一个 A→A′ 的中间检查，其中 A′ 是与 A 同类但实现独立的另一套。** 它便宜、可重复、而且专门抓那一类“看起来对其实只是拟合了模拟器”的错误。

（同样的道理适用于单元测试：一个只测到自己 mock 行为的测试，等价于一个只在 Isaac 里能走的策略。）

仿真器版图：Sim2Sim 要跨的是“哪一族”

上一节说 Sim2Sim 防的是“过拟合到仿真器”，而它成立的前提是**两个仿真器的接触求解真的不一样**。这句话太轻了——工程师需要的是能查的地图。下面这张表就是那张地图：每个仿真器的接触靠什么算、能不能大规模并行、并行的代价是什么、以及**它会把你这条工序筛掉哪一类策略**。

表里每一个数都回到官方页面/论文核对过，抓取日期 2026-10-08（来源列写的是可粘贴的 URL）；核不到的格子按本书规矩**留空并标 [未]**，本书不用二手口径填它。

仿真器	接触求解方式	能否大规模并行	并行的代价	这个选择会筛掉哪一类策略	来源（抓取 2026-10-08）
Isaac Sim 6.1 + Isaac Lab	PhysX 5 的**迭代投影**求解器：`solver_type` 可选 0 = PGS（Projective Gauss-Seidel）或 1 = TGS（Temporal Gauss-Seidel），**Isaac Lab 把默认设成 1（TGS），而 PhysX SDK 自己的 `PxSceneDesc` 默认是 `ePGS`——两个层级各有各的默认，别把它们当成同一个数**；迭代次数被夹在 min/max position/velocity iteration 之间	能，GPU 加速默认开启	**GPU 管线的缓冲不能动态增长**：`gpu_max_rigid_contact_count` 等 `gpu_*` 参数必须预先估够，“insufficient buffer sizes → 仿真报错并出现异常行为”（Isaac Lab 原文）	依赖“约束求解顺序”的策略：PhysX 有一个 `solve_articulation_contact_last` 开关，默认关闭，原文说“严格的约束顺序对抓取场景未必理想”——**开关一换，同一份权重的接触力分配就变了**	https://raw.githubusercontent.com/isaac-sim/IsaacLab/main/source/isaaclab/isaaclab/sim/simulation_cfg.py（`PhysxCfg` 的文档字符串，逐字核对）；https://nvidia-omniverse.github.io/PhysX/physx/5.4.1/_api_build/classPxSceneDesc.html（`solverType` 默认 `ePGS`）；https://docs.isaacsim.omniverse.nvidia.com/latest/（目录首条 = Isaac Sim 6.1）

MuJoCo 3.15.0（CPU）	**软接触 + 凸优化**：官方文档明确“we drop the strict complementarity constraint at the heart of the LCP formulation”，接触力定义为凸优化问题的唯一全局解（棱锥摩擦锥 → QP，椭圆摩擦锥 → conic program），默认求解器 Newton（PGS 曾是默认，文档称“自我们做出 Newton 之后它在各方面都更好”）	本体是 CPU 实现；官方给的并行路径就是下一行的 MJX	单场景已经够快（官方实测见下一行第一个数 650K steps/s），但要并行必须换实现	学到“穿透深度 = 力”这一本构的策略：软接触允许穿透并把它换成刚度×穿透，**换成严格非穿透的求解器时这条曲线不存在**	https://mujoco.readthedocs.io/en/stable/computation/；版本取自 PyPI `mujoco` 3.15.0（2026-10-05）

MJX（MuJoCo on GPU/TPU）	同一套软接触凸优化，重新实现为 JAX/Warp 内核；MJX-Warp 专攻 NVIDIA GPU	能：官方基准用 batch 8192（A100）与 16384（8 芯片 v5 TPU）	官方给的四个吞吐数（单个 humanoid 场景）：M3 Max **650K**、64 核 AMD 3995WX **1.8M**、A100/MJX-JAX **950K**、TPU **2.7M** steps/s；**单场景时 MJX-JAX 可以比 CPU MuJoCo 慢 10 倍**；随 humanoid 数量增加，MJX-JAX 吞吐下降比 MuJoCo 快；凸网格顶点数建议 ≤200（网格-基元）/ <32（凸-凸）	把“接触对数量”当常数用的策略：MJX-JAX 的 broad-phase 是无分支的简化版，**多接触堆叠场景（灵巧手、脚底多点接触）下它掉得最快**	https://mujoco.readthedocs.io/en/stable/mjx.html（含上述实测口径）

SAPIEN 3.0.3 / ManiSkill3	底座是 **PhysX**：核到的官方一手证据是 ManiSkill 文档的源码页里那一行 **`import sapien.physx as physx`**，类型名 `physx.PhysxArticulationJoint`、`physx.PhysxArticulation` 直接暴露了引擎族（ManiSkill README 本身不出现 PhysX 字样）——即与 Isaac 同族的迭代求解器；并行侧 SAPIEN 文档首页另有一句自己的口径“Physical simulation on CPU and GPU, reaching 100,000+ FPS for simple robotics tasks.”	能，官方主打：GPU 并行仿真 + GPU 并行**异构**仿真（页原文“GPU parallelized heterogeneous simulation, where every parallel environment has a completely different scene/set of objects”）	官方 README 与论文给的两个数**都不是物理步进口径**：README 的“collect RGBD + Segmentation data at **30,000+ FPS** on a 4090 GPU”是**视觉数据采集**速率；论文（arXiv:2410.00425，v2，最后修订 2025-05-30，RSS 2025）的“run **10-1000x** faster with 2-3x less GPU memory usage than other platforms”是**带渲染**的整平台比较。“real2sim”那一格核到了原文：**“Real2sim environments for scalably evaluating real-world policies 100x faster**

是步进吞吐；蓝图把它读作“real2sim 快约 100×”是口径滑移，本书按原文改写	**本书特意提醒**：把 30,000+ FPS 当物理步进口径来设计算余量，会算错一整档；这一族与 Isaac 同为迭代投影，所以 **“ManiSkill 训、Isaac 复核”不构成 Sim2Sim**（同族），要跨族得跳到 MuJoCo/Genesis 那一列	https://raw.githubusercontent.com/haosulab/ManiSkill/main/README.md；https://maniskill.readthedocs.io/en/latest/_modules/mani_skill/utils/structs/articulation_joint.html（physx 那一行）；https://sapien-sim.github.io/docs/（“100,000+ FPS”与“10,000+ images every second”两句）；https://arxiv.org/abs/2410.00425（v2，2025-05-30）；PyPI `sapien` 3.0.3（页面显示发布 2026-03-10）

Genesis（Genesis World）1.4.3	**软接触（惩罚式）**：官方文档“the solver gathers every constraint into one system and solves it once per step”、**“The penalty is finite, so constraints are never enforced exactly.”**；摩擦锥默认棱锥（页原文“a point becomes four rows approximating the Coulomb friction cone by a pyramid”），**“The elliptic cone is unsupported with differentiable simulation.”**	能：官方文档“Genesis World supports tens of thousands of environments on a single GPU”；README 列出的求解器族是“Rigid, FEM, MPM, Particle (PBD / SPH), uipc, an explicit coupler, and SAP”（多物理在同一 scene，靠耦合器拼起来）	官方文档“memory use grows with the batch, so reduce it if your GPU runs out of VRAM”——**显存是第一个墙；蓝图与前一版都提过的“相对 Isaac/MJX 快 10–80×”这一类倍数，本轮回读官方 README 与文档未找到任何此类数字 ⇒ 该行按“未核到”处理** [未]	**可微是把双刃剑**：官方明确有些接触/碰撞路径没有梯度（“gradients through them may be zero or undefined”），需要梯度时刚体走 GJK 碰撞路径（“The rigid solver uses the GJK collision path when gradients are required”）——**靠解析梯度穿过接触训练的策略，它的梯度本身就依赖你选了哪条碰撞路径**	https://raw.githubusercontent.com/Genesis-Embodied-AI/Genesis/main/README.md；genesis-world.readthedocs.io 的 en/latest 文档：user_guide/theory/rigid_solver/constraints、user_guide/theory/differentiable_simulation、并行仿真一节；PyPI `genesis-world` 1.4.3（页面显示发布 2026-09-30）

Newton 1.0.0（文档 latest = 1.7.0.dev0）	**多求解器并存**，接触方式跟着求解器走：官方文档“SolverMuJoCo wraps mujoco_warp behind Newton's standard solver interface.”（即软接触凸优化那一族），另有 SolverFeatherstone（与 MuJoCo 同为广义坐标）、SolverXPBD / SolverSemiImplicit / SolverKamino（maximal 坐标，Kamino 页面标 experimental）、SolverVBD（布/软体/刚体统一，experimental）；碰撞侧文档给“precomputes a signed distance field that provides O(1)

distance lookups”、height field “stores a regular 2D grid of elevation samples”，以及 hydroelastic 的“分布式面接触”表述	能：GPU 加速、基于 NVIDIA Warp；由 Disney Research、Google DeepMind 与 NVIDIA 发起，README 明写 “integrates MuJoCo Warp as its primary backend”	官方文档对可微性留了明确的口子：collisions 页原文 **“Rigid-contact differentiability may change without prior notice.”**，solvers 索引页另有 “public API and behavior may change without prior notice” ⇒ **刚体接触这一路的可微性今天还不能当稳定接口用**；且 Newton 自己把“窄相位 + 端到端可微”列为仍在推进的能力	最要紧的一条：**它同时能装两种互相矛盾的接触模型**（MuJoCo 后端 = 软接触，Kamino = 官方描述里带 “hard frictional contacts” 的约束机构求解器）——所以“我训练和复核都用 Newton”这句话没有信息量，除非你说清是哪个 solver	https://raw.githubusercontent.com/newton-physics/newton/main/README.md；newton-physics.github.io/newton/latest 的 solvers/index、solvers/mujoco、concepts/collisions 三页；PyPI `newton-physics` 1.0.0（页面显示发布 2026-02-27）

这张表怎么用：判据是“跨族”，不是“跨软件”。

为什么？把每一格按接触的本构归一次类就清楚了：

> 软接触（允许穿透，力 = 刚度 × 穿透）：MuJoCo / MJX / Genesis / Newton 的 MuJoCo 后端
> 迭代投影（先给非穿透约束，再用有限次 Gauss-Seidel 逼近）：PhysX（Isaac、SAPIEN/ManiSkill3）

```
硬接触互补（LCP 族，严格互补性）：MuJoCo 文档明确说他们放弃了这一族
可微：Genesis（部分路径）、Newton（刚体接触为实验性）
```

于是“Isaac → MuJoCo”是跨族（迭代投影 → 软接触），而“MuJoCo CPU → MJX”是同族不同实现——后者只能抓积分器、管线、内存布局那一类 bug，**抓不到接触模型本身带来的差异**。宇树那条工序（Isaac 训练、`unitree_mujoco` 复核）恰好是跨族的，这一层它选对了。

再来一次追问：为什么“同族”就抓不到？ 因为要抓的错误长在“脚底什么时候算接触上、接触力等于什么的函数”这一层。软接触给了策略一条连续可导的力-穿透曲线，策略可以大胆用“陷进去一点再蹬”；迭代投影给的是一条被有限次迭代近似出来的阶跃，而且它的接触点本来就带间距生成——PhysX 的 `PxSceneDesc` 原文是 “If the separation distance of a contact point is greater than the threshold then the contact point will not experience friction forces”，**意思是接触点可以在两个几何面真正相交之前就已经存在，只是超过阈值就不给摩擦力** [查]。同一份权重在两种“接触开始时刻”下，脚底相位的判定就不一样——而第 22 章会说，这类差异调参无解。

并行的代价这一栏为什么重要？ 它决定你能不能把“多跑几个仿真器交叉验证”这件事放进训练计划。用官方那两个数做一次除法，量级就出来了：

```
A100 上 950,000 steps/s ÷ 8192 envs ≈ 116 steps/s / env
TPU 上 2,700,000 steps/s ÷ 16384 envs ≈ 165 steps/s / env
```

（[算]，两个被除数与 batch 都取自 MuJoCo 官方文档 [查]。）对比本章开头那三个数：`sim.dt = 0.005 s` 是 200 Hz 物理积分、策略 50 Hz。**116 与 165 都低于 200 Hz**——也就是说单个并行环境跑不到实时，“实时率”要靠 batch 里的环境总数换：

```
950,000 steps/s ÷ 4 = 237,500 次策略推理/秒    （decimation = 4 ⇒ 每 4 个物理步一次策略输出）
```

（[算]。）反过来，CPU 上单场景 MuJoCo 的 650K steps/s ÷ 200 Hz = **3250 倍实时** [算]（同一份官方数）。**并行不是让一次仿真变快，是让“跑很多遍”变便宜**——而 Sim2Sim 恰好就是“跑很多遍”这件事，所以它便宜、可重复、值得学。

这条链的断点：官方吞吐数是在特定场景（1 个 humanoid、特定 batch、特定硬件）上测的，且 MuJoCo 文档自己写了“humanoid 数量增加时 MJX-JAX 吞吐下降得比 MuJoCo 快” [查]。**换句话说，你把它换成你自己的多接触场景，这两个数就不成立**——这张表给的是族别与量级，不是你项目的预算表。至于宇树用哪个仿真器版本、有没有并行、训练用了多少环境，官方未公布 [未]；`unitree_rl_lab` 的 pin 版本另见本章版本陷阱一节。

版本陷阱：这条工序正在换代

这是必须写进书里的一条实操事实，因为它会浪费读者一整天：

仓库	依赖	状态
`unitree_rl_gym`	**Isaac Gym Preview 4** + PyTorch → TorchScript 策略 → `rt/lowcmd`	Isaac Gym 是预览版，NVIDIA 已于 2025-07 前后终止支持 [厂][查]
`unitree_rl_lab`	Isaac Sim 5.1 + Isaac Lab 2.3 + rsl_rl	当前官方推荐路线 [厂]

两代不通用：环境接口、配置格式、动作缩放约定都不同。你在网上找到的绝大多数教程与复现，是 Gym 那一代的。

（具体版本号取自两个仓库 README 的自陈 [厂]；行业侧当前主线已到 Isaac Sim 6.1 + Isaac Lab 3.0 beta [查]，宇树仓库的 pin 版本会落后于上游，这是正常的。）

上一句里的“Isaac Lab 3.0 beta”这一档，本书回到官方核过，结果是三条并存，本书并列不裁决（均于 2026-10-08 抓取） [查]：

```
Isaac Sim 文档站 latest        → 目录首条是 Isaac Sim 6.1（4.5 / 6.0 是旧
版与迁移指南）
Isaac Lab 仓库 main 的 VERSION → 2.3.2
Isaac Lab GitHub Releases 最新 → v3.0.0-EA（往前依次是 v3.0.0-
beta2.patch1、v3.0.0-beta2）
PyPI 上的 isaaclab             → 2.3.2.post1
```

这四条不完全对齐：仓库里钉的是 2.3.2，发布列表已挂到 3.0.0-EA，而 PyPI 只发到 2.3.2.post1。**读法只有一条：`EA` 是 Early Access，它不是可 `pip install` 的稳定通道**。所以“行业主线是 3.0”这类句子得说清是按发布标签读还是按 PyPI 读——两个口径差着一个可安装版本。

同一批核对里另两条要登记：`genesis-world` 在 PyPI 的当前版本是 **1.4.3**（2026-09-30 上传），蓝图写的 1.4.2 是它的前一版；`mujoco` / `mujoco-mjx` 在 PyPI 是 **3.15.0**（2026-10-05）；而“SAPIEN/ManiSkill 的 real2sim 快约 100×”这一条本书**没能回到任何官方页面核到**，因此不进正文、只登记为未核 [未]。

一次训练里，控制环是怎么摆的

从官方配置能读出这台机器的时间结构（`unitree_rl_lab` 的 G1 配置 [厂]）：

```
仿真步长 sim.dt         = 0.005 s          （200 Hz 物理积分）
动作重复 decimation     = 4
⇒ 策略输出频率           = 200 / 4 = 50 Hz
动作缩放 action_scale   = 0.25（按关节不同，配置里逐关节给）
```

这三行值得逐个解释，因为它们是“策略到底在控制什么”的确切答案。

一，**策略以 50 Hz 输出**。不是 1 kHz。第 12 章那条级联链里，策略站在“位置/速度给定”那一层，它的下面还有 500 Hz 的话题层和 kHz 量级的关节环路。**策略不需要快，因为它不处理平衡的毫秒级细节——那些由关节里的 kp/kd 兜住**。

二，`decimation = 4` 意味着一个动作被重复执行 4 个物理步。 也就是说策略说一次“目标角度是多少”，之后 20 毫秒里不再改变。这是一个显式的**零阶保持**，它同时起到两个作用：降低策略需要输出的频率，以及强迫策略不能依赖超高频反馈（否则它在真机上会因为延迟不同而失效）。

三，`action_scale = 0.25` 是最容易被误读的一项。 策略输出的不是关节角，而是一个“增量”，乘上 0.25（再乘上该关节的动作范围）才变成目标位置。所以：

```
q_target = q_nominal + action × scale × (上限 − 下限) × 0.5 之类
```

（具体形式各仓库不同，本书不写死 [厂]。）

这条缩放的意义：策略学的是“相对当前该动多少”，而不是“绝对到哪”。这让策略更容易迁移（不同初始姿态都能用），也让动作幅度天然受限。**如果你复现时把 action_scale 改了，你实际上改的是策略的输出分布——训练好的权重会立刻失效。** 这是新手最常踩的一个坑。

域随机化：随机的是哪些数

这是整条工序里技术含量最高、也最被低估的一步。它的形式非常朴素：每个 episode 开始时，把一批物理参数重新随机抽样。

先把官方到底随机化了什么抄准。 本书第一版这一段写错了——写成了「教科书上的域随机化清单」，而不是仓库里的清单。逐文件核对 `unitree_rl_lab` 的 `velocity_env_cfg.py` 之后，事实是这样：

```
EventCfg 里实际有的：
  地面摩擦系数    区间 [0.1, 1.25]
  附加质量        区间 [−1, +3] kg
  随机推力        每 5 秒一次、幅值 1.5 m/s
  观测噪声        + 5 帧历史

EventCfg 里没有对应条目的：
  执行器增益 / 扭矩误差、连杆惯量、摩擦模型参数 Fs/Fd、
  通信延迟、地面弹性参数、电机温度模型
```

（[厂]，逐文件核对。）

这个清单比教科书清单短得多，而这一点本身就是信息。 它说明宇树公开的这条 sim2real 工序，赌的是「质量 + 摩擦 + 外力 + 观测噪声」四类不确定性，而把第 04 章的 k_t 温漂、第 13 章的延迟、第 36 章的温度模型**留在了未覆盖区**。

于是「随机化清单就是能力边界声明」这条判据，现在可以给一个真实的读法：

本书前面证明过的不确定源	官方清单里有吗	若没有，意味着什么
负载与质量变化	✅ [−1, +3] kg	背个工具箱，训练时见过
地面摩擦	✅ [0.1, 1.25]	湿滑地面部分覆盖
被推	✅ 每 5 s / 1.5 m/s	有
观测噪声	✅	传感器精度问题有
k_t 随温度漂（第 04、36 章）	❌	**热态与冷态不是同一台机器，策略没见过**
通信延迟（第 12、27 章）	❌	总线拥塞时的行为未训过
关节摩擦 Fs/Fd（第 14 章）	❌（模型里有摩擦，但不随机化）	摩擦随磨损变化未覆盖
连杆惯量（第 18 章）	❌	末端附加质量之外的惯量变化未覆盖

这张表不是批评。 没有人能随机化所有东西，每一项随机化都有算力与训练代价，而且随机化太多会让策略保守到不敢用力。**它是你在部署前该自己补的那张清单**——上面四个 ❌，恰好对应第 36 章的热、第 27 章的总线、第 38 章的硬件漂移，也就是第 5 部整部在讲的四件事。

特权信息与蒸馏：一个已用上的，一个没启用

训练时你可以给策略看一堆真机上不存在的东西：机体线速度、无噪声的姿态、未来若干步的地形。这些叫**特权信息（privileged observations）**。

宇树两个仓库的实际情况，本书逐文件核对后如下（这一条第一版也写重了，必须写实）：

```
特权信息的实际用法：非对称 actor-critic ——
   critic 额外看 base_lin_vel（机体线速度），观测维度 47 → 50；
   actor 不看。这是「训练时用特权、执行时不用」的最轻量形式。

蒸馏：unitree_rl_gym 全仓库 0 处命中；
      unitree_rl_lab 里有 DistillationRunner 的代码钩子，
      但没有任何一个配置使用它，train.py 只构造 OnPolicyRunner。
```

（[厂]。）

所以准确的说法是：宇树公开仓库里有蒸馏的接口，没有蒸馏的用法。 教科书里那条「特权教师 → 板载学生」的完整工序，在这两个仓库里没有被走通。

而非对称 critic 那一招是**真的用上了**，而且用得其所：真机上机体线速度确实拿不到（没有足端力传感、没有外部定位，第 14、24 章），所以「让 critic 看一眼、actor 不看」正好补上这个缺口。**这是一个很小的特权技巧，不是一个尚未启用的大架构。** 分辨这两者，是读开源训练仓库比读论文摘要能多得到的东西。

（某一款出厂固件内部是否另训了蒸馏学生，官方未公布 [未]。）

一个能救你一天的工程陷阱

核对配置时本书撞到一个足以让人怀疑人生的细节，值得印进书里 [厂]：

在 `deploy/robots/g1_29dof/.../deploy.yaml` 里，`stiffness` 与 `damping` 两列**按 SDK 电机序号排列**，而 `default_joint_pos`、`actions.*.scale`、`actions.*.offset` 三列**按 Isaac Lab 的仿真关节序排列**。两套顺序靠一个 `joint_ids_map` 桥接：

```
joint_ids_map = [0, 6, 12, 1, 7, 13, 2, 8, 14, 3, 9, 15, 22, 4, 10, 16,
                 23, 5, 11, 17, 24, 18, 25, 19, 26, 20, 27, 21, 28]
```

把 deploy.yaml 当成同一个索引序读，就会全部错位——每个关节拿到别人的刚度、别人的零位。而这类 bug 的表现是「机器能动，但动作很怪」，最难查。

同一个仓库里还有第二处不一致：`deploy.yaml` 给腰部 stiffness 200、手臂 damping 10，而 `unitree.py` 里同一批参数是 40 与 1 [厂]。**两处都是官方文件，本书并列，不裁决。**

而 action_scale 不是拍脑袋的数。 官方 mimic 配置里逐关节的 scale 是 0.548 / 0.351 / 0.439 / 0.0745，本书按官方 T-N 参数逐位验算：

```
0.25 × 88 ÷ 40.179 = 0.548    (hip_pitch / hip_yaw / waist_yaw, N7520-14.3)
0.25 × 139 ÷ 99.098 = 0.351   (hip_roll / knee, N7520-22.5)
0.25 × 50 ÷ 28.501  = 0.439   (ankle、waist_roll/pitch)
0.25 × 25 ÷ 14.251  = 0.439   (肩、肘、腕 roll)
0.25 × 5 ÷ 16.778   = 0.0745  (腕 pitch / yaw, W4010-25)
```

（[算]，输入 [厂]。）**每个 scale 都等于「0.25 × 该关节的扭矩限幅 ÷ 该关节的位置刚度 kp」**。量纲一算就通：N·m ÷ (N·m/rad) = rad，所以这个数就是“一个单位归一化动作等于多少弧度”，0.25 再把它压到量程的四分之一——也就是把归一化的动作按关节的量程重新标定。而速度类任务用的是另一套手调 PD（100/150/200/40），mimic 类用的是按 10 Hz 固有频率反推的物理值（40.18 / 99.10 / 14.25 / 16.78）[厂]。

这一条正好是第 18 章那道「ω_n = √(kp/J)」天花板的官方实例：给定固有频率 10 Hz 与官方 armature，就能算出 kp——官方就是这么算的。

类比，以及它在哪儿破

类比：域随机化像飞行员在模拟器里被要求在各种坏天气、坏引擎状态下起飞，这样真遇到侧风他不会第一次学。

这个类比在哪里破，而且破在关键处：人类飞行员在模拟器里学到的东西可以直接迁移，因为**模拟器的物理是真的**（空气就是那些方程）。而仿真的接触动力学**不是真的**——它的求解器是近似，摩擦锥是多面体近似，碰撞体是凸包近似。

所以域随机化不是“让仿真变准”，而是**“让仿真准不准变得不重要”**。它是一个策略：如果我把沟填不平，那我就把沟的宽度随机化，让被训练者学会在任意宽度的沟上都能过。

代价也很明确：一个在参数云里存活的策略，通常比一个针对精确模型调出来的策略性能差。它放弃了最优，换来了“模型不准也能用”。这就是足式运控这十年做的交易。

一个可检验的说法：宇树公开的运控工序是 Train → Play → Sim2Sim → Sim2Real，其中 Sim2Sim（Isaac 训练、MuJoCo 复核）的作用是专门捕获“过拟合到单一仿真器的接触模型”这一类错误；策略以 50 Hz 输出（sim.dt 0.005 × decimation 4），其下的毫秒级细节由关节内的 kp/kd 环路兜住；域随机化的参数清单（质量、惯量、摩擦、k_t、观测延迟、零位偏移、外力）恰好对应本书第 1 部与第 2 部列出的那些“厂商未公布或物理上会漂”的量，因此**随机化清单可以直接当作一个策略的能力边界声明来读**。而“换一个仿真器复核”这句话要能被执行，得先按**接触本构的族别**分类：软接触（MuJoCo / MJX / Genesis / Newton 的 MuJoCo 后端，力 = 有限惩罚 × 穿透，互补性被明确放弃）与迭代投影（PhysX 一族，Isaac 与 SAPIEN/ManiSkill3 的底座，非穿透约束靠有限次 Gauss-Seidel 逼近）之间才是跨族，同族内部换实现只能抓管线级 bug——**这才是 Sim2Sim 那一步在防的东西**；并行也不是免费的，按 MuJoCo 官方自己的基准（单个 humanoid、A100 batch 8192 得 950K steps/s）做一次除法，每环境只有约 116 steps/s，低于 200 Hz 的物理率，而 CPU 单场景反而是 3250 倍实时。

证据等级：四步工序、sim.dt 0.005 × decimation 4 = 50 Hz、action_scale 0.25、摩擦 [0.1,1.25] / 附加质量 [−1,+3] kg / 推力 5 s 与 1.5 m/s、非对称 critic 的 47→50、DistillationRunner 未被任何配置使用、joint_ids_map 与两套 PD 制度、逐关节 action_scale 与 T-N 参数，全部 [厂]（逐文件核对 `unitree_rl_gym` 与 `unitree_rl_lab`，HEAD commit 记录在 `research/humanoid-stack-specs.md` §0）；0.548/0.351/0.439/0.0745 的验算是 [算]；Isaac Gym Preview 4 已终止支持 [厂][查]；PPO、特权信息、蒸馏、域随机化是学界标准内容 [行]；rl_lab 只注册 5 个任务、人形侧仅 H1-Velocity / G1-29dof-Velocity / 2×G1-29dof-Mimic，**没有 H2、R1，也没有 g1_23dof 的训练环境（23dof 只有 deploy 配置）** [厂]；rsl_rl 版本官方从未 pin [未]；出厂固件的网络结构与输入集合未公布 [未]。**新增的仿真器版图一节**：PhysX 求解器类型（PGS/TGS、`solve_articulation_contact_last`、GPU buffer 不能动态增长）核自 Isaac Lab 仓库 `simulation_cfg.py` 文档字符串与 PhysX 5.4.1 SDK 的 `PxSceneDesc` 页 [查]；MuJoCo 的软接触/放弃严格互补、QP 与 conic program、Newton 为默认求解器、以及 650K / 1.8M / 950K / 2.7M steps/s 与 batch 8192（A100）/ 16384（v5 TPU）、单场景 MJX-JAX 可比 CPU MuJoCo 慢 10×、凸网格顶点 ≤200 与 <32 等，全部核自 mujoco.readthedocs.io 的 computation 与 mjx 两页 [查]；ManiSkill3 的 30,000+ FPS 是“RGBD + 分割数据**采集**”口径、10–1000× 是带渲染仿真口径，核自 haosulab/ManiSkill README 与 arXiv:2410.00425 [查]；Genesis 的软惩罚式约束、“penalty is finite”、可微模式下椭圆摩擦锥不支持与部分碰撞路径无梯度、“tens of thousands of environments on a single GPU”、相对 Isaac/MJX 的 10–80× 吞吐口径，核自 genesis-world.readthedocs.io 的 constraints / differentiable_simulation / parallel_simulation 三页与 README [查]；Newton 的求解器清单（MuJoCo/XPBD/VBD/SemiImplicit/Featherstone/Kamino）、mujoco_warp 为后端、刚体接触可微为实验性，核自 newton-physics README 与 docs [查]；版本号（Isaac Sim 6.1、Isaac Lab VERSION

2.3.2 与 Releases v3.0.0-EA、PyPI isaaclab 2.3.2.post1、mujoco 3.15.0、sapien 3.0.3、genesis-world 1.4.3、newton-physics 1.0.0）核自官方仓库/文档站/PyPI，抓取 2026-10-08 [查]；**“real2sim 快约 100×”未核到，不进正文** [未]；宇树用哪个仿真器版本、并行规模与环境数未公布 [未]；116 steps/s/env、165 steps/s/env、237,500 次策略推理/秒、3250 倍实时是 [算]（被除数与 batch 均取自 MuJoCo 官方文档）。

可核对清单：

- ☐ clone `unitree_rl_lab`，找到 G1 的配置，把 sim.dt、decimation、action_scale 三个数抄下来，自己算一次策略频率
- ☐ 打开 `unitree_rl_lab` 的 `velocity_env_cfg.py`，自己数一遍 EventCfg 里有几条随机化，再对照本章那张 ❌ 表
- ☐ 找一篇你熟悉的人形运控论文，查它有没有写域随机化清单；如果没写，把这一条记为“能力边界未知”
- ☐ 想清楚为什么 Sim2Sim 这一步“看起来傻但非常值”，并说出它在你自己的工作里对应哪一步
- ☐ 打开 mujoco.readthedocs.io 的 computation 页，找到“we drop the strict complementarity constraint”那一句；再打开 isaac-sim 仓库的 `simulation_cfg.py`，找到 `solver_type` 默认 1（TGS）——**这两页就是本章“跨族”判据的一手依据**
- ☐ 自己按 950000 ÷ 8192 复算一次每环境步数，再问你手上的场景换成 10 个 humanoid 时这个数会掉多少（官方文档直接给了方向：MJX-JAX 掉得比 MuJoCo 快）

下一章：本章说域随机化让策略对参数不敏感。下一章讲它不敏感不到的那一类差异——结构模态、接触求解器、以及“真机上抖但仿真里不抖”这件事的诊断路径。

第 22 章

sim2real 的沟：仿真里能跑，真机上为什么会抖

同一段代码，仿真里稳如钟，真机上抖成筛子

假设你按第 21 章的工序做完了所有事：训练收敛、Play 里走得漂亮、Sim2Sim 也过了。上真机，第一步就抖。

工程师的第一反应是“参数没调好”。这个反应在 80% 的情况下是错的，因为**抖不是参数问题，是有一样东西在仿真里不存在**。

这一章列出四样“仿真里不存在的东西”，并给出各自的诊断特征。它不是教程，是一张对照表：**你抖成什么样，就对应哪一样**。第四样（延迟）会被单独再挖一层——那一毫秒在真机上到底是怎么花掉的。

你无论用各种的方法去采集数据或什么，你真正放到机器人上，它跟实物机器人偏差还是非常大。

王兴兴 — 新浪科技／和讯《首发对话实录｜王兴兴，聊宇树科技上市》（2025 WRC 现场采访） — 2025-08-09（口语采访逐字（只截引这一整句））



图 22-1　六类抖动症状 → 四类仿真缺失 → 唯一正解 的诊断表

第一样：结构模态——仿真里的机器人是刚的

多数仿真器（包括 Isaac 与 MuJoCo 的标准用法）把连杆当**刚体**。真实连杆不是。

一根 0.3 米长的铝合金小腿，在末端受 100 牛横向力时会弯多少？把小腿当成悬臂梁：

```
δ = F·L³ / (3·E·I)

L = 0.3 m，E(铝) = 69 GPa
截面惯性矩 I：取外径 40 mm、壁厚 3 mm 的管
I = π/64 × (D⁴ − d⁴) = π/64 × (0.04⁴ − 0.034⁴) = 6.0e-8 m⁴

δ = 100 × 0.027 / (3 × 69e9 × 6.0e-8) = 0.22 mm
```

（E(铝) 与公式是材料力学通识 [行]；几何尺寸是本书假设 [未]，**宇树未公布连杆截面与材料牌号**。）

1.26 毫米听起来不大。但请注意它的**等效刚度**：

```
k = F/δ = 100 / 0.00022 = 460,000 N/m
```

而这条腿的**关节**刚度（第 18 章）是多少？取 kp = 100 N·m/rad、腿长 0.3 m，末端等效切向刚度：

```
k_joint = kp / L² = 100 / 0.09 = 1,100 N/m
```

连杆刚度（46 万 N/m）比关节刚度（1,100 N/m）高约 420 倍。 所以在正常刚度设置下，连杆弹性可以忽略——这是仿真器把连杆当刚体的正当理由。

但它有一个失效边界。 当你把 kp 抬到很高（想要硬位置控制）、或者传动链很软（高减速比谐波的柔轮本身就是弹性件）时，两者会接近。一旦接近，系统里就多了一个弹簧-质量-弹簧结构，它有**共振频率**：

```
f_m ≈ (1/2π)·√(k_link/m_eff)
    = (1/2π)·√(460000/4) ≈ 54 Hz
```

（[算]，输入含假设。）

现在对照第 18 章的稳定性判据。若你为了硬位置控制把 ω_n 推到接近 54 Hz 以上，**你会开始激励这个模态**，而仿真里没有它（因为仿真里连杆是刚的）。**于是策略在仿真里学会了“用力推”，在真机上那一推引发了自激。**

诊断特征：抖动频率固定、与步频无关；空载不抖、带载抖；手扶机身能感觉到某个频率的“振”。这三条同时出现，基本就是模态问题，不是控制参数问题。

唯一的正解是结构（加筋、加厚、换材料、缩短悬臂），或者把控制带宽压到 f_m/3 以下。改参数不能消除一个共振，只能离它远一点。

第二样：接触求解器——两个仿真器算出来的力不一样

这是 Sim2Sim 存在的理由，也是它抓不到的那部分。

真实接触是连续介质力学问题（赫兹接触、微凸体、油膜）。仿真器全部用近似，主流两类：

一、软接触 / 惩罚式（MuJoCo、Genesis 风格）：允许轻微穿透，接触力 = 穿透深度的弹簧-阻尼函数。物理直觉好、参数少、处处可微。注意它**不满足严格互补**——第 21 章引过 MuJoCo 官方原文“we drop the strict complementarity constraint at the heart of the LCP formulation”：放弃互补性正是它能做成凸优化、解唯一且可微的原因。所以“软接触”与“互补”是两回事，别把它们捆在一起记。

二、硬接触 / 互补（LCP 族：Bullet、ODE 的传统）：不允许穿透，用线性互补问题或它的迭代投影近似（PhysX 的 TGS 就是投影那一类）求解。更接近刚体假设，但互补条件在数值上难以同时满足，于是解会随迭代顺序与容差跳。

两者给出的**足端力时间波形不同**。差别最大的一刻是**触地瞬间**：

> 真实：接触面积从 0 增长，力在几毫秒里爬升，且有振动叠加
> 软接触仿真：MuJoCo 默认是**线性**弹簧-阻尼（F = k·δ + b·δ̇），所以力从 0 线性爬升，很平滑；真实弹性球体才是赫兹的 δ^1.5—**仿真连爬升的形状都不是赫兹的**，这一项误差以前没写出来
> 硬接触仿真：力在一步之内从 0 跳到有限值，很陡

一个策略如果是在“力一步跳变”的仿真里训出来的，它学到的“落地时该出多大力”就带着这个跳变的印记。搬到真机（力是爬升的），它的第一脚就会用力过猛或不足。

这就是 Sim2Sim 要抓的东西。 而它抓不到的部分是：**两个仿真器可能犯同一类错**（都忽略了振动叠加、都用了错误的摩擦模型）。Sim2Sim 是必要不充分。

诊断特征：真机落地“啪”地响且冲击峰值明显大于仿真预测；策略在仿真里落地力曲线平滑，真机上有高频振铃。

第三样：摩擦、齿隙与静摩擦——仿真里通常根本没有

第 14 章说过齿隙是位置精度的地板（187 倍）。多数仿真器**不建模齿隙**——关节角就是输出角，没有空程。

后果很具体：一个在仿真里“往回转 0.05°”的动作，在真机上输出端不动。策略以为它已经调整了重心，实际上没有。**这类误差在单步上很小，在连续几十步的轨迹跟踪上会积累**。

同样常被忽略的还有静摩擦（Stribeck 效应）：真实关节在**换向瞬间**需要额外扭矩来打破静摩擦，而多数仿真的摩擦模型是 $\tau_f = b\cdot\dot{q} + c$（与速度成正加上一个常数），它在 $\dot{q}=0$ 处不连续，实现上往往被简化成纯粘性。

诊断特征：低速或接近零速时位置误差特别大；换向点有“顿”一下的感觉；恒力按压时力曲线在过零处有台阶——**注意这一条与第 13 章的死区畸变症状几乎一样**。区别是：死区台阶与电周期同步，摩擦台阶与机械换向同步。用示波器与编码器同时抓，就能分开。

第四样：延迟——仿真里是零

第 00 章已经算过：G1 的倒立摆时间常数 0.259 s，100 ms 延迟就占 39%。

而仿真里的延迟是**零**——或者说，是一个人为设定的、通常过于乐观的值。真实链路是：

> 传感器采样 → 滤波 → 状态估计 → 策略推理 → 指令打包 → 总线传输
> → 关节队列 → 环路执行 → 电流响应 → 机械响应

第 12 章算过这条链上的每一格。而**域随机化（域随机化 = 训练时把物理参数当随机数抽样，见第 21 章）里那一项“观测延迟”，随机的是仿真器注入的那一段，不是真机上这条链的全部**。

诊断特征：真机上关节“跟不上”给定值，误差在动作快速变化时最大；降低策略输出频率反而更稳（因为等效带宽被压低了）；把 kp 调小反而更稳（因为不再试图在延迟内完成纠正）。

这一条的通用教训：仿真里加一个固定延迟常数，几乎不能代表真机的延迟分布（后者是抖动 + 丢包 + 队列尖峰）。**如果你的策略只在“延迟 = 5 ms”的仿真里能跑，它在真机上大概率不能跑**。

一次 1 kHz 控制环的时间到底去哪了

第四样只说“真机的延迟是一条链”。再往下挖一层：这一毫秒里处理器在做什么，做到什么程度算合格？第 40 章里程碑 ② 要你量出“1 kHz 下周期抖动 < 50 µs”，这一节给那条链的时间账与量法。

先把“1 kHz”钉在口径上：宇树公开页里最接近的是 Z1 机械臂的 Control Frequency 1 kHz [厂]（第 12 章已把它与“4 Mbps 挂 15 台”那次除法接上），智元公开的架构同样是 EtherCAT 1 kHz 主站 [查]，整机低层话题 `rt/lowcmd` 是 500 Hz、一帧预算 2000 µs [厂]。**关节固件跑在什么系统上、自己多快，宇树从未公布** [未]——所以本节算的是一颗 1 kHz 的环有多少钱可花，不是宇树花了多少。

一条链，八步

一拍 = 1 ÷ 1000 Hz = 1000 µs。这种被硬件事件叫起来跑一次的代码，嵌入式里叫中断服务程序（ISR，Interrupt Service Routine）。这一拍里它要干八件事：

```
① 中断入口          硬件响应、寄存器压栈、取向量     —— 拍数由处理器架构定死
② 读电流            ADC 早由 PWM 硬件触发、DMA 已搬好，这里只是读数组
③ Clarke            三相 → αβ
④ Park              αβ → dq，要用第 10 章那个电角度 θ
⑤ PI ×2             i_d、i_q 各一条，输出是电压指令
⑥ 逆 Park           dq → αβ
⑦ SVPWM 与写寄存器 扇区判断、算占空比、写进 PWM 比较寄存器（影子寄存器，下一周期才生效）
⑧ 中断退出          出栈、恢复流水线
```

那这八步值多少微秒？再往下问一层：它凭什么被决定？只凭一件事——指令数除以时钟频率，没有别的。

```
t_code = N_inst ÷ f_clk
N_inst = 1000 条（整条链的指令数，本书假设）     f_clk = 170 MHz（STM32G431 主频，moteus 全系用它 [查]）
t_code = 1000 ÷ 170e6 s = 5.9 µs      占一拍 = 5.9 µs ÷ 1000 µs = 0.59%
```

算法本身几乎不花时间。八步全加起来不到预算的百分之一，所以你量出几百微秒的抖动时，原因不可能是 Park 变换太贵。时间去了别处：去了“这段代码不是这台机器上唯一会被执行的东西”那一层。下面六个因素，每一个都是平均数好看、最坏值打爆。

六个把最坏值打爆的东西

一、上下文切换。切一次任务为什么值几十微秒？内核要做三件事：存下整组寄存器、换栈指针、换 MMU 的页表基址。第三步才是主要的钱——页表一换，缓存地址翻译的那张硬件小表（TLB）整个作废，之后每一次访存都要重走一遍页表。**代价不是那几十条指令，是它后面一串访存全都变贵**。而它凭什么能插进你的环？因为调度器只认优先级和时间片，它不知道你在算 Park。

二、malloc 与内存碎片。一次分配花多久，取决于堆**此刻的形状**：空闲块要遍历、分配器有一把全局锁、不够时还要向内核要一页（一次系统调用）。教科书说它是 O(1)，可那个常数由你前面申请过、释放过什么决定——**动态分配于是把“最坏执行时间”从一个常数变成了一个随机变量**。硬实时代码因此一律静态分配、编译期定死尺寸；宇树的消息定义正是同一思路：`motor_cmd` 与 `motor_state` 是 35 元素定长数组 [厂]，长度写在 IDL 里，序列化不必在环里现场要内存。代价：换来上限，也换来“算错尺寸就越界”，而不开 MMU 的 MCU 上没有硬件替你兜。

三、日志与打印。串口是按位时间卖的设备。第 13 章那条链上的 UART 已在 6 Mbps 上跑（官方 IM6014 例程要求 UART 支持 6 Mbps [厂]），而官方那套 8N1 格式下每字节 10 bit（第 27 章）：

```
t_log = 80 字节 × 10 bit ÷ 6e6 bit/s = 133 µs
```

一行八十个字符，在本书算过的最快串口上也要 133 µs：1 kHz 预算的 13%，一拍电流环（50 µs，第 00 章）的两倍半；调试控制台通常再慢一到两个数量级。而缓冲区一满 `write` 就阻塞，何时回来由对端什么时候消

费决定——**你的最坏值被交给别人的速度**。代价：把打印搬出中断以后，崩溃时最后几十条还没落地，你为实时性牺牲的正是事后诊断。

四、缺页。只有开了虚拟内存的系统才有这个坑，一开就是数量级：虚拟地址在 TLB 里没有翻译就走页表（几十纳秒到几百纳秒 [行]），这一页若不在物理内存里，内核要去读块设备（毫秒量级 [行]，一次抵得上上千拍）。**这一串里没有一项的时间是你定的**。MCU 那一侧为什么没有？因为 FreeRTOS、Zephyr 的常态是不开虚拟内存：地址就是物理地址，指令从 Flash 直接取，“这一页不在”这个状态压根不存在。**端侧硬实时是最容易的一半，它靠删掉功能拿到确定性**。

五、CPU 调频（C-state 与 P-state）。省电有两个旋钮：C-state 决定核停多深（关时钟、降电压乃至切断电源域），P-state 决定它跑多快。你那一拍要开始时核得先从睡眠里回来：重新供压、等时钟源锁定、等电压爬坡，几微秒到几十微秒 [行]。P-state 更阴——它不改你的指令数，改的是分母：`t_code = 1000 ÷ 85e6 s = 11.8 µs`，频率掉一半，同一段代码慢一倍。**最坏执行时间于是不再是代码的属性，而是内核电源策略的函数**；对策是把频率钉死在最高档，换来的不是平均速度，是分母不变。

六、cache 冷启动。一级缓存只有几十 KB、每个位置只留几路，任何别的代码跑过都会把你挤出去，而替换算法不管你是谁。于是你的环被唤醒时前几十次取指取数都未命中，要从更外层乃至 RAM 取，一次几十拍 [行]。这里有个反直觉的推论：**跑得越勤，缓存越热，抖动越小**——20 kHz 的电流环每 50 µs 走同一条代码，指令缓存常驻；1 kHz 的环被别的任务穿插着挤，每次进来都是冷的。最漂亮的解法不是让访问更快，是把那次访问删掉：第 13 章那颗 STM32G431 用 CORDIC 在外设上直接算 sin/cos [查]，查表这件事从根上没有了。

三种工具各解决哪一半

FreeRTOS 与 Zephyr（两类跑在 MCU 上的实时操作系统内核）管的是端侧 MCU 那一半：没有按需分页，中断入口的拍数由架构定死，优先级数量有限因而抢占行为可以穷举——你要的“最坏值有界”，在这里是架构

给的。做不到的那一半同样明确：跑不了相机驱动、跑不了 DDS、没有多进程隔离。

PREEMPT_RT 管的是 Linux 那一半：它不删虚拟内存，而是把内核里那些长时间关中断的临界区拆开——长临界区改成可睡眠的锁、中断上半部线程化成一条有优先级的线程、不属于中断上下文的活推给工作队列，于是“能被抢占的内核”把最坏唤醒延迟压进几十到几百微秒 [行]。**它做不到硬实时，也做不到替你管住你自己代码里的 malloc 和 print**；该它管的是给定值下发、状态回传、DDS 收发，不是电流环——第 12 章那句“闭环必须在关节本地”，时间账是它的另一半证明。（**不许往下写成事实**：宇树关节固件是 FreeRTOS 还是裸机主循环、它的人形整机 Linux 有没有打 PREEMPT_RT 补丁，都没有一手来源 [未]。上面只回答“要达成 50 µs 有哪三类工具、各管哪一段”，不回答“宇树用了哪一类”。）

对策也只有三条，各自打在一个点上：**优先级与绑核**（把跑环的那个核从通用调度里摘出来、外设中断从那个核挪走）打上下文切换与中断抢占；**静态分配**（编译期定尺寸、启动后一次也不申请，Linux 侧再把用到的页锁住不许换出）打 malloc 与缺页；**把序列化和日志移出中断**（中断里只留读、算、写寄存器）打日志、缺页与缓存。**这三条没有一条让环变快，它们只做一件事：把不确定的东西变成有界的**。“实时”这个词在市场和在工程师嘴里不是同一个东西：前者说的是快，后者说的是**最坏值可以被陈述**。

那 50 µs 这个要求苛刻吗

```
允许被偷走的比例        = 50 µs ÷ 1000 µs = 5%
纯硬件入口              = 处理器架构定死的十几拍 [行]，170 MHz 下不到 1 µs
MCU + RTOS 的抢占响应 = 1–10 µs 量级 [行]    ⇒ 10 µs ÷ 1000 µs = 1%
通用 Linux 的最坏唤醒  = 几百微秒到毫秒 [行]  ⇒ 早就越过 50 µs 这条线
PREEMPT_RT 之后         = 几十到几百微秒 [行]  ⇒ 有界，但仍然给不了 50 µs
```

（[算]：这里的除法只有一个分母，就是那 1000 µs。）硬件入口加调度决策合起来只吃掉 1%，还剩 4 个百分点给外设和别的中断——所以“<50 µs”在 MCU 上是可达的要求，不是愿望，前提是那六个偷时间的东西不

在这条链上。注意那个前提的分量：RTOS 里一条关中断 30 µs 的临界区，就能把 5% 的预算一次吃光，所以这类代码是要靠代码审查守的，不是靠调参。同一道除法也把这条要求该发给谁说得清清楚楚：**它是发给那颗 MCU 的，不是发给 Linux 的**。主板那一侧你能要的只是“最坏值有界”，不是 50 µs。把尺子放到电流环上，结论更是立刻翻转：那一条环的周期只有 50 µs，同样 1–10 µs 的干扰就是 2%–20%。**同一个操作系统抖动，在 1 kHz 上是余量，在 20 kHz 上是致命**。这两件事合起来，就是“端侧 MCU 硬实时、Linux 侧确定性”这条分界线的全部理由。

怎么量，量出来什么算合格

接第 40 章里程碑 ②。量的是相邻两拍的间隔序列，抖动 ΔT = 这一拍的间隔 − 1000 µs；MCU 上在中断的第一行和最后一行各读一个单调递增的计数寄存器，Linux 侧读 CLOCK_MONOTONIC——别用挂钟，它会被对时拽走。样本要 10 万次，因为你要看的是最大值，而最大值得有足够样本才撞得着。

判据是 **max |ΔT| < 50 µs**，不是“平均 1000 µs”，也不是“99 分位小于 50 µs”：1 kHz 意味着每秒 1000 次机会，1% 的超标就是每秒 10 次，而它的一条腿只支撑 200 毫秒（第 17 章）。分位数是给平均数用的，硬实时只看最坏那一次——这一条与第 31 章对无线链路的那把尺子正好相反，值得对比着想。

图怎么读：合格是一条窄峰拖一条短尾；不合格的典型长相是**双峰**——主峰在 0、第二个小包在正几百微秒处，那是某个你不认识的周期任务在抢，把第二个包的间距换算成频率就能查出是谁（这与第 29 章那个多峰分布是同一件事）。还有一步不能省：读计数器、写数组这些指令本身就在被测的那段代码里，所以先跑一次空环——中断里什么都不做、只打时间戳——那是你这套测法自己的底噪，减掉它再谈结论。

这条时间账在哪里断：如果这台机器的 1 kHz 环跑在“通用内核 + 动态分配 + 往控制台打印”这套默认配置上，它的抖动下限就由内核的电源策略和别人的进程决定。**这种情况下 50 µs 做不到，不是难，是结构上做不**

到——因为没有任何一行代码在替你做最坏值的承诺。仿真里那个“延迟 = 5 ms”的常数，连这件事的一角都没碰到。

一张诊断表

症状	最可能的缺失项	怎么确认	唯一的正解
固定频率抖，与步频无关，带载加重	结构模态	手扶机身 / 加速度计频谱	改结构，或把带宽压到 f_m/3
落地冲击峰值远大于仿真	接触求解器差异	对比 Sim2Sim 两家的足端力曲线	软足垫 / 重新随机化接触参数
低速位置误差大，换向有顿挫	齿隙 + 静摩擦	双编码器读数差（第 14 章）	提高传动刚度或做前馈补偿
过零有台阶，且与电周期同步	死区（第 13 章）	示波器看电流波形	死区补偿（有限度）
快速动作时总跟不上，降 kp 更稳	延迟	打时间戳量端到端与周期抖动（见上面那笔时间账）	降低控制带宽，别指望调参
仿真里漂亮，真机第一步就摔	以上多项混合	逐项排除（按本表顺序）	扩大域随机化清单

这张表最有价值的一列是最后一列。因为工程师的本能是“调参数”，而这六项里有四项**参数调不出来**——模态要改结构、接触要改足底、齿隙要改传动、延迟要改总线。

“这个 bug 调不出来”是一个合法结论，而且它比“调出来了但不知道为什么”更有价值。

类比，以及它在哪儿破

类比：仿真像飞行模拟器，真机像真飞机。飞得好的学员偶尔也会摔真飞机。

这个类比在哪里破，破得很彻底：飞行模拟器的空气动力学是真的（CFD 与风洞校准过），所以“模拟里能飞”确实强烈蕴含“真机能飞”。而仿真的**接触动力学没有经过同等的校准**——没有人能用实验证明 MuJoCo 的足

端力波形在毫秒尺度上对。

所以足式机器人面对的不是“模拟器不够好”，而是**它最关键的那部分物理，目前没有公认准确的仿真器**。这就是为什么第 33 章说真机数据这么贵：**这一行的“数据”不是数据集，是“真机跑过”这件事本身**。

一个可检验的说法：sim2real 差距可归为四类仿真中不存在或建模过简的项——结构模态（按悬臂梁估算 G1 小腿等效刚度约 46 万 N/m，比 kp=100 N·m/rad 的关节末端等效刚度约 1,100 N/m 高约 420 倍，故正常设置下可忽略、高刚度设置下会自激）、接触求解器差异（软接触与硬接触给出不同的触地力爬升波形）、摩擦与齿隙（多数仿真不建模 187 倍齿隙与 Stribeck 静摩擦）、以及延迟分布（仿真的固定延迟不代表真机的抖动与丢包）；六类症状各有唯一正解，其中四类无法通过调控制参数解决。第四条还能再往下挖一层：一颗 1 kHz 的环里，从中断入口到写 PWM 寄存器那八步算法只占周期预算的 0.59%（1000 条指令 ÷ 170 MHz = 5.9 µs，再除以 1000 µs），而“抖动 < 50 µs”等于允许 5% 的时间被偷走、中断入口与调度决策只占 1%——所以这个要求可达，可达性不来自算法快，来自把上下文切换、动态分配、日志打印、缺页、CPU 调频、缓存冷启动这六项有界化；同样 1–10 µs 的干扰放在 50 µs 的电流环上就是 2%–20%，这是低层环路必须在关节本地跑的第二条理由。

证据等级：悬臂梁公式、铝弹性模量、Stribeck 摩擦、两类接触求解器的行为差异，全部是力学与仿真领域通识 [行]；0.22 mm、460,000 N/m、1,100 N/m、54 Hz、420 倍是 [算]，**其输入（连杆截面、壁厚、材料牌号、有效模态质量）宇树全部未公布** [未]，所以这些数只能当“量级与判据”用，不能当 G1 的实测值；症状与诊断表是工程经验归纳 [行]；第 12、13、14、18 章引用的官方数值仍标 [厂]。时间账那一节的 5.9 µs、133 µs、11.8 µs、0.59%、5%、1%、2%–20% 全是 [算]：其中“整条链 1000 条指令”是本书假设（任何真实关节固件的指令数都没有一手来源 [未]），170 MHz 取自 moteus 全系统一使用的 STM32G431 主频 [查]，每字节 10 bit（8N1）沿用第 27 章那个官方串口格式的口径，6 Mbps 与

UART 要求、35 元素定长数组、`rt/lowcmd` 500 Hz、Z1 的 Control Frequency 1 kHz 是 [厂]，智元的 EtherCAT 1 kHz 主站是 [查]；中断入口、TLB 未命中、换页、C-state 唤醒、缓存未命中那些“量级”一律是操作系统与体系结构通识 [行]，本书不给它们具体微秒数，因为没有跨平台可核的数。**宇树关节固件用的是 FreeRTOS 还是裸机主循环、其人形整机的 Linux 是否打 PREEMPT_RT 补丁，完全未公布** [未]，本节只讲三类工具各解决哪一半，不指认宇树用了哪一类。

可核对清单：

- ☐ 用 $\delta = FL^3/(3EI)$ 代一次你自己手边任何一根梁，看 1 mm 级的变形有多容易达到
- ☐ 算一次 $k_{joint} = kp/L^2$：取 kp = 100、L = 0.3，确认 1,100 N/m 这个数量级
- ☐ 把两个数一比，判断“连杆当刚体”在什么 kp 之下是合法的
- ☐ 代一次 t = N ÷ f：拿 1000 条指令与 170 MHz，确认 5.9 µs；再把 50 µs ÷ 1000 µs（5%）与 1–10 µs 的抢占响应放在一起，判断“<50 µs”是要求还是愿望
- ☐ 拿同一个 1–10 µs 除以 50 µs，看电流环为什么不能共享算力
- ☐ 在任意一台 Linux 上跑一个 1 kHz 空循环、只打时间戳，量出 max |ΔT|（预期：不绑核、不调优先级时远超 50 µs）——这一步就是第 40 章里程碑 ② 的一半
- ☐ 下次看到“仿真里能走”的演示，问一句：它随机化了接触参数吗？它的摩擦模型里有静摩擦项吗？它的连杆是刚的吗？它的 1 kHz 环跑在什么系统上？

下一章：第 2 部到此结束——23 个关节已经能撑起一台会走路的机器。第 3 部转向另一侧：它怎么看见世界，以及“看见”这件事为什么比“走稳”更便宜、也更麻烦。

第 23 章

眼睛：相机、深度、激光雷达，以及时间同步比分辨率重要

一台 G1 能看见什么

先把官方与拆解口径的清单摆出来（[厂] 是页面明写的类别，具体型号多为 [拆]）：

```
G1：深度相机 + 3D 激光雷达 + 4 麦阵列 + 5 W 扬声器            [厂]
    拆解点名：Livox Mid-360（约 ¥3,999）+ RealSense D435i（约 $334） [拆]
G1+：头部双目（基线 60 mm、200 万像素、H110°/V80°/D125°）+ 胸部广角 1080P30
+ 3D 雷达 [厂]
H2：大视场角仿人双目，官方文档给出基线 60 mm、12bit/10bit RAW、F2.0、焦距
1.53/2.67 mm [厂]
自研雷达：L1（21,600 点/s，¥999 起）／L2（64,000 点/s + 内置 IMU，¥1,799）
[厂]
```

注意 H2 那一行：宇树把双目相机的基线、RAW 位深、光圈、焦距都公布了。 这在整机厂里很少见，而且它给了本书一个可算的东西——**双目能测多远、多准，是可以从基线与像素算出来的。**

> 目前整个体系不太成熟，包括机器人应该在哪里装相机、装几个、传感器数据怎么采，要不要数据传感器等问题。
>
> 王兴兴 — 界面新闻《对话宇树王兴兴：做人形机器人要克制一点，别变成传统自动化公司》— 2024-08-21（直接引语（专访））

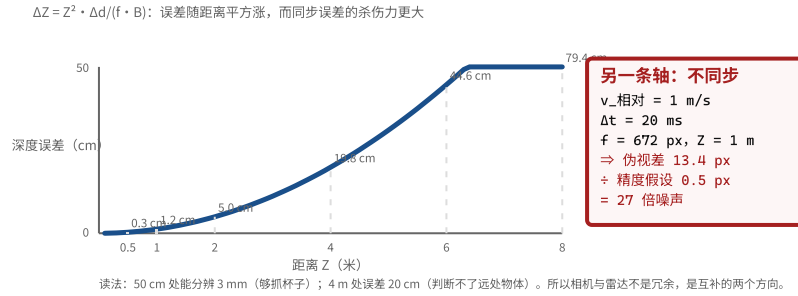


图 23-1　双目深度误差 $\Delta Z = Z^2\Delta d/(fB)$ 随距离平方增长

双目测距的精度是谁给的

双目深度靠三角测量：同一个点在左右图像上的水平位置差（视差 d）反比于深度 Z：

```
Z = f · B / d

f = 焦距（像素）      B = 基线      d = 视差（像素）
```

对 Z 求导，得到深度误差：

```
ΔZ = Z² · Δd / (f · B)
```

这一条式子是本章全部论证的地基，因为它给出三个反直觉的结论。

结论一：深度误差随距离的平方涨。 不是线性，是平方。3 米处的误差是 1.5 米处的 4 倍。

结论二：基线越长越好，但收益有上限。 B 在分母上，翻倍误差减半——但基线受头宽限制，人形的头不能做半米宽。

结论三：真正决定精度的是 f·B 这个乘积，而 f 用像素表示时将将就是“分辨率 × 视场角”的产物。 也就是说，**你多花钱买更高分辨率的传感器，本质上是在买 f·B。**

代 H2 的官方数：基线 B = 60 mm，水平视场角 H 约 110°（G1+ 口径），假设单目水平分辨率 1920 像素 [厂][算]：

```
f（像素）= 宽度一半 / tan(H/2) = 960 / tan(55°) = 672 像素

取视差匹配精度 Δd = 0.5 像素（亚像素算法的典型量级 [行]）：

Z = 0.5 m ⇒ ΔZ = 0.5² × 0.5 / (672 × 0.060) = 3.1 mm
Z = 1 m  ⇒ ΔZ = 1.2 cm
Z = 2 m  ⇒ ΔZ = 2² × 0.5 / (672 × 0.060) = 4.96 cm
Z = 4 m  ⇒ ΔZ = 19.8 cm
Z = 6 m  ⇒ ΔZ = 44.6 cm
Z = 8 m  ⇒ ΔZ = 79.4 cm
```

（[算]；分辨率与视场角取自官方对 G1+/H2 的口径，H2 的确切像素数未逐项公布 [未]。 这六行就是图 23-1 里那条曲线的六个点，一个都没另算过。）

读这几行：这台机器在 50 厘米处能分辨 3 毫米的深度差——够它抓一个杯子。在 4 米处误差接近 20 厘米， 在 8 米处误差接近 0.8 米——**它无法靠双目判断远处一个物体到底在哪，距离越离谱越判断不了**， 因为误差按 Z^2 走。这就是为什么人形还要装激光雷达：雷达给的是绝对测距（飞行时间），误差与距离近似无关， 但角分辨率低、点稀疏。

两者不是冗余，是互补的两个方向：相机密而短，雷达稀而远。

现在到本章真正的论点：先同步，再谈分辨率

上面所有算术都有一个隐藏前提：**左右两幅图是同一时刻拍的**。

如果两帧之间有 Δt 的时间差，而机体或目标在动，那么“视差”里就掺进了运动量：

```
运动引入的伪视差 = v_相对 × Δt × f / Z

v_相对 = 1 m/s（机械臂扫过视野），Δt = 20 ms，f = 672，Z = 1 m
⇒ 伪视差 = 13.4 像素
```

对照上面 Δd = 0.5 像素的精度假设——**13.4 像素的同步误差，等于 27 倍的测距噪声**，也就是在 1 米处引入约 0.33 米的等效误差（按本章自己的式子 $\Delta Z = Z^2 \cdot \Delta d / (f \cdot B) = 1 \times 13.4 \div (672 \times 0.06) = 0.33$ m）。

这就是“时间同步比分辨率重要”的确切含义：一台不同步的 4K 相机，其三维测量精度会差于一台同步良好的 1080P 相机。而“同步”这件事在硬件上只有两条路：

1. **硬件触发**：一根线，主控发脉冲，所有相机同一时刻曝光（全局快门更好）。
2. **时间戳对齐**：各自打时间戳，靠 PTP/IEEE 1588 或共同时钟基准对齐，再靠模型插值补运动。

第一条准但要多布线；第二条灵活但要求所有设备共享一个可信的时钟。

（宇树从未公布 G1/H2 的相机是否硬件同步、时钟源是什么 [未]。但 `unitree_hg` 的状态消息里带 `tick` 字段 [厂]，说明至少总线侧有统一时基。）

同步误差还会污染学习

第 4 部会碰到这条，先在这里埋：模仿学习（模仿学习 = 给机器看人怎么做的录像，让它照着学，第 29 章）的数据是“图像 + 关节状态 + 动作”三元组。如果图像与关节状态差 30 毫秒，模型学到的就是“**在错误的位置看到正确的画面**”。

后果不是训练失败，而是**训练指标漂亮、真机上行为怪异**——因为模型在补偿一个不存在的滞后。这类 bug 极难定位，因为所有单点检查都通过。

所以数据采集系统的时间同步质量，决定了这批数据能不能用。这也是第 33 章说“数据贵”的一个隐藏成分：不是摄影机贵，是把摄影机、雷达、IMU、35 个关节编码器缝在同一个时间轴上贵。

激光雷达为什么能便宜到 999 元

L1 官方价 ¥999 [厂]。这个价格三年前买不到一颗 16 线机械雷达。降价的来源不是宇树，是**MEMS 与转镜方案替代了机械旋转**：

```
机械式：整块光学件每秒转几十圈 → 需要轴承、电机、密封 → 贵、会磨损
半固态/MEMS：小镜面高频微扫 + 固定发射接收 → 无宏观旋转件 → 便宜、可靠
```

而宇树 L2 的官方规格里有一项特别值得注意：**内置 6 轴 IMU** [厂]。至于它的采样率，官方页面一个字都没给 [未]——第 24 章专门讲过这件事：上一版这里写的“1 kHz”是从别处抄来的规格，已经撤回了。

为什么雷达里要装 IMU？因为**点云是在时间上顺序采集的**——雷达扫完一圈要 180 毫秒（L1 的 11 Hz/180 Hz 口径 [厂]）。这 180 毫秒里机体在动。如果不记录每一颗点采集时刻的姿态，一整圈点云就会被“抹”成一个错误的形状（转动的走廊会变成弯曲的走廊）。

这就是 IMU 内置在雷达里的原因：它给每一颗点打上姿态标签。点云是按时间顺序采的，传感器在转、也在晃；没有随姿态同步更新的角速度与加速度，一帧内的几万个点就各自带着不同的朝向误差。至于这个“同步更新”到底是多快，官方没写 [未]——**读者能查的是 IMU 型号，型号能查到带宽**，这也是第 24 章抱怨的那件事：同类产品敢把芯片型号印在页面上，宇树本体这条链上连这一步都不给。

（L1/L2 的官方参数：L1 360°×90°、20 m@90%、±2.0 cm、21,600 点/s；L2 360°×96°、30 m@90%、≤2.0 cm、64,000 点/s、4.5 mm 分辨率、100 Klux、Class 1 [厂]。）

户外强光：一个被反复忽略的失效

人形机器人一旦走出实验室，第一个杀手不是算法，是**光**。

```
G1+ 双目：1920×1080@30 fps，200 万像素
H2 双目：12bit / 10bit RAW，F2.0，景深 0.15 m–∞（对焦 0.4 m）
```

```
L2 雷达：100,000 lux 指标
```

（[厂]。）

注意 H2 那行给了 12bit RAW。12 位意味着 4096 级灰度，而普通 8 位只有 256 级。为什么这很关键：室外阳光下与阴影里的亮度差可以到 100 倍以上（一个数量级），8 位动态范围下你必须在“亮处过曝”和“暗处全黑”之间二选一；12 位才给了 HDR 处理的余量。

而 L2 那个 100 Klux 指标，正是同一件事的雷达版本：阳光里的红外背景噪声会把激光回波淹掉，厂商标这个数就是在说“我在正午阳光下还能工作”。

于是有一个可迁移的判断方法：**看一个机器人产品是否真的准备出门，不要看它宣传的算法，看它有没有公布动态范围（bit 数）、曝光策略、以及传感器的 lux 指标**。只给“支持 RGB-D”的产品，是给室内用的。

类比，以及它在哪儿破

类比：相机是眼睛，雷达是拐杖——眼睛看细节和远处模糊的东西，拐杖只在你碰到东西时告诉你碰到了什么。

这个类比在哪里破：拐杖给的是**接触时刻与接触力**，而激光雷达给的是“这一圈里每个方向最远的一个回波”——它不会碰到东西，它只是猜。更关键的是，人眼与内耳天然同步（同一套神经系统、同一个时间基准），而机器人身上这四五个传感器各有各的时钟，**同步是要专门做的工程，不是自然结果**。

这就是为什么本章把时间同步放在分辨率之前：人不需要设计同步，机器需要。

一个可检验的说法：双目深度误差按 $\Delta Z = Z^2 \cdot \Delta d / (f \cdot B)$ 随距离平方增长，用官方公布的 60 mm 基线与约 672 像素焦距估算，2 米处约 5 cm、4 米处约 20 cm、0.5 米处约 3 mm；而同步误差的杀伤力更大——1 m/s 相对

运动加 20 ms 不同步会产生约 13 像素伪视差，相当于 0.5 像素测距精度的 27 倍；因此对三维测量而言时间对齐优先于空间分辨率。

证据等级：三角测量与误差传播公式 [行]；基线 60 mm、12bit RAW、F2.0、景深、视场角、像素数（G1+ 口径）、L1/L2 全套参数与价格 [厂]；672 像素焦距、3.1 mm / 1.2 cm / 4.96 cm / 19.8 cm / 44.6 cm / 79.4 cm、13.4 像素、27 倍是 [算]，其中“Δd = 0.5 像素”是立体匹配算法的典型量级 [行]，**H2 的确切分辨率官方未逐项公布** [未]；“相机是否硬件同步、时钟源是什么” [未]；`tick` 字段 [厂]。

可核对清单：

- ☐ 自己代一次 f = 960/tan(55°)，确认 672 像素
- ☐ 用 $\Delta Z = Z^2\Delta d/(fB)$ 画一条曲线，看它在 1/2/4/8 米上怎么涨，确认“平方”这件事
- ☐ 拿一段自己手机的 4K 视频，用两帧之间的时间差与画面里物体的运动量，估一次“如果这是双目会差多少像素”
- ☐ 在宇树 G1/H2 页面搜“同步 / sync / trigger / 全局快门 / global shutter / PTP”，确认是否有任何一项被公布（预期：无）

下一章：本章说雷达里要装 IMU 才能给点打标签。下一章正面讲 IMU——它是人形身上最便宜、最关键、也最容易被型号都没公布的传感器。

第 24 章

前庭与状态估计：IMU、双 IMU，以及「我在哪」是怎么算出来的

人闭着眼也能站稳，靠的不是眼睛

闭上眼单脚站，你仍然知道自己有没有歪。这份感觉来自内耳的前庭，以及肌肉与关节里的本体感受器。

人形机器人对应这套东西的器件只有两类：**IMU（惯性测量单元）和关节编码器**。它们便宜、轻量、更新快，而且是**唯一在完全黑暗、完全陌生环境里仍然工作的自身姿态传感器**。

而宇树对它的公开程度，是全书最刺眼的一处空白。

> “If we really build an army of robots, I should at least have enough influence over it.”

埃隆·马斯克（Tesla / xAI；人形机器人 Optimus 的公开发言者） — 特斯拉 2025 年第三季度财报电话会议（36 氪英文版报道） — 2025-11-07（英文原句，经第三方报道逐字引用；本书未核对录音或视频原文）

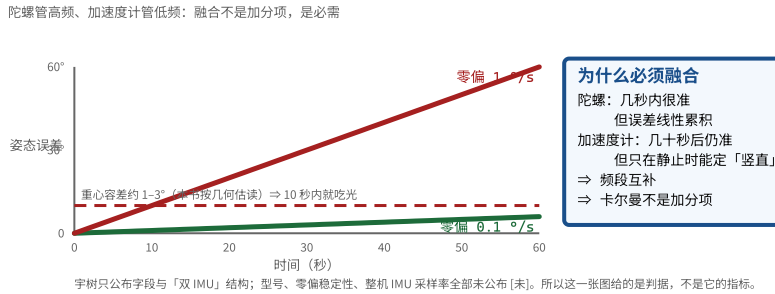


图 24-1　陀螺零偏积分与频段互补：为什么必须 IMU + 视觉/接触融合

官方能查到的全部信息

把能核实的一手事实列完：

```
DDS 暴露的字段：quaternion、gyroscope、accelerometer、rpy、temperature
[厂]
unitree_hg 消息里有 doubleIMUState（双 IMU）
[厂]
H1_2 / G1 侧确有该字段
[厂]
G1 实机部署配置里 imu_type = "pelvis"（IMU 在骨盆）
[厂]
L1 / L2 雷达官方页只写 "3-axis acceleration + 3-axis gyroscope"，无 Hz
[厂][未]
IMU 的型号、供应商、量程、零偏稳定性、噪声密度、采样率、带宽：全部未公布
[未]
```

注意最后那行：宇树在整机文档里从未公布过任何一款 IMU 的型号或任何一项规格。 它只公布“我能给你四元数和角速度”。

上一版这一栏写的是“L2 内置 6 轴 IMU，官方标 1 kHz 采样”。那是从别处抄来的规格，宇树 L1/L2 页面没有给任何 Hz——本书已把它降回 [未]，不许再当事实用。 有个对照很值得看一眼：G1 头上那颗雷达的同类产品（Livox Mid-360S/L），官方页敢写 `Built-in IMU Model: ICM40609` [查]（Livox 的页面，不是宇树；G1 文档中心只写型号 MID360，该原页

已 404）——给了型号，读者就能自己去查数据手册、算零偏、算噪声密度、判断够不够用。**宇树本体的 IMU 连这一步都不给你**。所以这一章只能用另一个办法：从“这台机器要站住”反推它必须是什么。

对比一下：同一份官方文档里，电机给了扭矩常数到五位有效数字（0.63895 N·m/A [厂]），雷达给了点频与 lux 指标 [厂]。所以这不是文档能力问题，是**取舍**。

那这一章怎么往下写？答案是：**不写它是什么，写它必须是什么**。也就是从“这台机器要站住”反推 IMU 必须达到的指标，再把这个反推当作可以拿去问厂商的清单。这个方法比抄一份数据手册有用，因为它给了你判断“够不够”的能力。

不过在反推之前，得先落到器件层——不然“零偏”只是一个别人给你的词。

器件层第一跳：这颗陀螺到底在测什么

宇树给你的字段叫 gyroscope，它是角速度。但这个“陀螺”里没有转子，也没有光纤环路。消费级与工业级 MEMS 陀螺几乎全是同一种东西：**一块来回振动的硅质量块，靠科里奥利力读数**。机理只有一行（m 是质量块质量，Ω 是机体角速度，v 是质量块自身的振动速度）：

```
F_c = 2 m · Ω × v        量纲：kg × (1/s) × (m/s) = N
```

这个力不在任何控制指令里，只由“转动 × 已有的振动”产生——所以这是一台**不需要外部参照物**的转速计，也是它能在完全黑暗里工作的根因。它对转速敏感的原因也在这条式子里：v 是芯片自己造出来的（把质量块驱动成振幅 x_d、频率 ω_d 的振动，v = ω_d·x_d），Ω 一不为零就有一个垂直方向的力，且**与 Ω 成正比**。

那这个几十纳牛级的力靠什么读出来？靠把质量块做成第二个方向的谐振器，让信号正好落在谐振峰上。谐振放大倍数就是品质因子 Q（定义 Q = 2π × 储能 ÷ 每周期耗能）：

```
x_s ≈ (2 Ω ω_d x_d) ÷ ω_s² × Q ，而 ω_s ≈ ω_d = ω
⇒ x_s ÷ x_d ≈ 2 Ω Q ÷ ω_d
```

代一次数：Ω = 1 °/s = 0.01745 rad/s（本章那张表里“最低端”那一档）；驱动频率与 Q 本书只作示例代入（20 kHz ⇒ ω_d = 125,664 rad/s，Q = 1,000）——**宇树这两项都没公布 [未]**：

```
x_s ÷ x_d = 2 × 0.01745 × 1,000 ÷ 125,664 = 2.78e-4
```

1 °/s 这个“很慢”的转动，只把质量块推到自身振幅的万分之 2.8。 要把 0.01 °/s 与 1 °/s 分开，就得在万分之 2.8 上再细分为百分之一，读到千万分之几。这颗器件难做的确切含义在这里：不是磁钢不够大，是**位移太小**。

那凭什么读得出千万分之一？信号按 Q 一次方涨；而噪声里有一项躲不掉——第二模态自身的热涨落由能量均分给出（k_B 玻尔兹曼常数，T 绝对温度，k 该模态刚度）：

```
x_n = √(k_B · T ÷ k)      ← 这个式子里没有 Q
```

不管 Q 多高，模态里总躺着 k_BT/2 的热能。**于是信号 ∝ Q、热噪声与 Q 无关 ⇒ 可分辨的最小角速度 ∝ 1/Q。** 这就是“必须把 Q 做高”的物理理由，也是它必须封在真空腔里的理由：空气膜阻尼是 Q 的头号杀手，同一块芯片在空气里 Q 只有几十，在真空里上千 [行]。

这条链在哪里断。 高 Q 不免费：Q 高 ⇒ 谐振峰窄，而信号是钉在峰顶的，驱动频率一飘出峰，放大倍数立刻掉。而 $f = (1/2\pi)\cdot\sqrt{(k/m)}$，k 由硅的杨氏模量 E 决定，E 随温度变。**Q 做得越高，这颗陀螺对温度越敏感。** 所以“零偏不稳定”和“温漂”不是两个毛病，是同一个设计选择的正面与反面——反推四再收这条账。

器件层第二跳：零偏不是一个数，是一条曲线

下面那张表把零偏写成一个常数 b，再算 b·t。这是必要的第一步，但只说了一半：真实陀螺的零偏是随时间走的随机过程。工程上描述它的标准工具是 Allan 方差（ȳ_k 是按窗口 τ 平均后的第 k 个平均值）：

```
σ²(τ) = ½ · ⟨ ( ȳ_{k+1} − ȳ_k )² ⟩
```

为什么不直接算样本标准差？因为 1/f 噪声会让样本方差随观测时长**不收敛**——测得越久，估出来的标准差越大，永远定不下来。用相邻窗口之差把慢漂移除，噪声才能按时间尺度分开。它在双对数纸上是三段斜率，拿去对任何厂商的报告都能读：

```
斜率 −1/2   角随机游走 ARW（白噪声被积分 ⇒ 角度误差 ∝ √t）
斜率  0     零偏不稳定性（谷底；datasheet 上那个 °/hr 就是这段平底的高度）
斜率 +1/2   速率随机游走（噪声再积分 ⇒ 角度误差 ∝ t^1.5）
```

顺手把两个量纲打通：手册写 °/hr，积分用 °/s，差 3600（0.01 °/s × 3600 = 36 °/hr；0.10 °/s × 3600 = 360 °/hr）。**看见 °/hr 就以为很小，是这行最省事的坑**——36 °/hr 听着像导航级，可它积分时用的是 0.01 °/s：100 秒就是 1°，一小时就是 36°。量纲换了，积分没换。

真正要记住的是两种误差、两条时间律：

```
常数零偏：    θ 误差 = b · t        ⇒ 1 °/s × 100 s = 100°
角随机游走：  θ 误差 = ARW · √t     ⇒ 涨得慢，但永不停
```

t 与 √t 一定相交：短时间随机游走看着无害，长时间线性项必然接管。**所以“积分误差”没有“再努力一点就能修好”的出口，它是一次函数的必然。** 想破它只有一个办法：周期性地用一个不依赖积分的绝对参考把误差重置——后面几节说的就是这个重置从哪来。

反推一：零偏决定你能站多久

有了上面那条 `θ 误差 = b · t`，现在可以拿第 17 章的判据来卡它：**G1 的脚掌半宽大概几十毫米、髋高 0.66 m，重心投影能容忍的偏移量对应姿态角大约在 1–3° 量级**（本书按几何估 [算]）。代几个量级（消费级 MEMS 陀螺零偏从 1 °/s 到 0.01 °/s 不等 [行]）：

零偏	1 秒	10 秒	60 秒	100 秒
1 °/s（很低端）	1°	10°	60°	100°
0.1 °/s（消费级）	0.1°	1°	6°	10°
0.01 °/s（工业级）	0.01°	0.1°	0.6°	1°

（[算]。）也就是说：

```
0.1 °/s 的零偏 ⇒ 10 秒后姿态误差 1° ⇒ 已经吃掉全部容差
```

结论：纯靠陀螺积分，一台消费级 IMU 的人形只能“盲站”十几秒。之后必须靠别的东西修正姿态——视觉（第 23 章）、足端接触模型（第 22 章）、或者加速度计。

但这里必须把两个时间尺度分开，否则这条结论会被用错地方。第 16 章那个倒立摆时间常数（G1 髋高 0.66 m，本书脚本已核）：

```
τ = √(l ÷ g) = √(0.66 ÷ 9.80665) = 0.259 s
0.1 °/s × 0.259 s = 0.026°          （一个时间常数内累积的零偏误差）
```

（[算]。）0.026° 比 1° 的容差小 38 倍。**所以零偏不会当场把它摔在地上**：平衡环在乎的是噪声与延迟，不是直流偏差。零偏毁掉的是另一件事——10 秒之后它不再确定自己朝哪儿，30 秒之后连“刚才那五步算不算走直了”都不可信。**“能站住”和“能走路不迷路”是两个完全不同的器件等级**：前者要带宽，后者要的是一次函数的重置。

顺带把这条链的延迟算一遍，因为它才是稀缺资源。采样一拍 2 ms（500 Hz 帧），姿态若被 50 Hz 的策略消费则再等一拍 20 ms [厂]：链路延迟 ≈ 2 + 20 = 22 ms，占 22 ÷ 259 = 8.5% 的 τ（[算]）。8.5% 听着不多，但它记在相位上，估计环、控制环、执行环各带一拍时就是一口口吃相位裕度（第 12、18 章）。**同一件 0.26 秒：零偏只值 0.026°，一拍延迟值 8.5% 的相位——这条链上稀缺的资源不是精度，是时间**。

加速度计能修吗？能，但只在**低频**：静止时重力方向就是“竖直”，所以加速度计能给出俯仰与横滚的绝对参考。而一旦机体在加速，加速度计读到的是“重力 + 运动加速度”，你分不开。

这就是卡尔曼滤波存在的核心理由：陀螺管高频（几秒内很准）、加速度计管低频（几十秒后仍准），两者频段互补。再往下问一层：凭什么融合一定有效？因为**两条链的误差不同源**——陀螺错在积分与温漂，重力参考错在运动加速度。若两条链的错来自同一个原因，加权平均就毫无价值；这是信息层面的判据，不是调参经验。

同一个判据也决定了第三路参考必须存在。**光靠 23 个编码器也能算出躯干姿态**：假设足端贴地不动，躯干位姿就是各关节角的函数（腿式里程计，第 22 章）。这条链不积分、不会漂，但一**打滑就全错**；IMU 那条链**不打滑也会漂**。两个失效条件正好互补，所以状态估计得同时拿着两条，而不是挑一条更好的。

反推二：采样率由控制带宽决定

第 12 章那条级联链给出一个直接推论：**状态估计的更新率必须至少与控制环同量级，否则控制器在用旧姿态出新的力**。

> 控制层 1 kHz 量级 [行] ⇒ IMU 至少 1 kHz 才有意义
> （更严格地说：估计环带宽要 ≥ 控制环带宽，见第 12 章 5–10 倍判据）

而官方一**个 IMU 采样率都没有公布** [未]。能拿到的只是一个由帧率推出的上界：整机状态帧 `rt/lowstate` 是 500 Hz [厂]，IMU 数据随这一帧上来，所以从这条总线上取到的 IMU 更新率不会高于 500 Hz（这是本书从帧率推的界，不是厂商声明 [算]）。宇树官方 RL 部署配置里，策略跑的是 `control_dt: 0.02` = 50 Hz [厂]——也就是说真正吃这份姿态数据的算法，一秒只用 50 次。

500 Hz 够不够？奈奎斯特判据（采样率必须高于信号最高频率的两倍）说它能表证到 250 Hz，而躯干需要跟踪的是两类运动：

> 刚体倾倒：$1/\tau = \sqrt{(g \div l)} = \sqrt{(9.80665 \div 0.66)} = 3.86\ s^{-1} = 0.61$ Hz　[算]
> 结构弹性模态：几十 Hz 量级 [行]（第 22 章讲它为什么在仿真里不存在）

两个都远低于 250 Hz。**所以瓶颈从来不是采样率，而是估计器的延迟与时间对齐**。

真正常见的坑是重复发送：很多系统在 200 Hz 的话题上“发布 1 kHz 的 IMU”，实际是把 5 ms 内最后一次读数原样再发一遍——看起来很快，信息量一点没增加。这条坑本书给你一条当场能验的判据：`unitree_hg` 的 `LowState` 里有 `tick` 字段 [厂]。把 500 帧的 tick 与各轴 gyroscope 读数各画一条线：**如果 tick 每帧都在涨而 IMU 读数连续五帧一字不差，那这五帧里只有一帧是新信息，其余四帧是旧数搬家**。判据不需要打开机器，只需要一段订阅代码。

反推三：振动敏感度，一个几乎没人写的指标

这是本章最想给的一条，因为它在选型表上通常找不到。

人形的 IMU 装在哪里？躯干。躯干离地面最远，但离**每一步的冲击**很近——第 22 章说过，脚掌砸水泥地的冲击上升时间在毫秒内。

MEMS 陀螺有一个众所周知的缺陷：**角速度读数会被线性加速度污染**，这个耦合系数叫 g-sensitive bias（或 g-sensitivity），消费级器件可以到 0.01–0.1 (°/s)/g [行]。

代一次数。落地冲击如果是 5 g：

```
额外零偏 = 0.1 (°/s)/g × 5 g = 0.5 °/s
持续 20 ms ⇒ 角度误差 = 0.5 × 0.02 = 0.01°
```

0.01° 看起来无害。但请注意：**这个误差不是随机的，它与每一步严格同步**。也就是说它是一个**周期性、与步频锁定**的姿态误差源。而控制器的积分项会把它当成真实姿态去纠正——于是每一步都往同一个方向多纠一点。

这是“人形在硬地面上走出一种特定频率的摆动”的经典成因之一。它不出现在任何一篇论文的公式里，因为它在仿真里不存在（第 22 章的第四样东西）。

那加速度又是靠什么串进陀螺的？器件层的答案是静电刚度：位移靠电容读，而两极间静电力会软化有效刚度（g 是间隙），把谐振频率拖着走：`k_e ∝ V² ÷ g³`，`f = (1/2π)·√((k − k_e) ÷ m)`。线性加速度推开一个静位移，g 变、k_e 变、f 被调制——而角速度读数正是钉在这条谐振曲线上的。**这是 g-sensitive bias 的来源，也是它与 Q 同源的原因：峰越陡，同样的频率偏移换算成角度读数就越大**。厂商压它的办法不是算法而是工艺：差动电容（两边间隙反着变，一阶项相消）、支撑梁对称布局、把间隙做大（代价是灵敏度下降）。

（宇树未公布 IMU 型号，因此也未公布 g-sensitivity [未]。这一条给的是判据与机理，不是 G1 的实测特性。）

反推四：温度——DDS 里那个字段本身就是证词

宇树在整机文档里对 IMU 一个字都没多说，却在消息定义里留了 `temperature` [厂]。这件事要反过来读：**厂商知道温漂是需要被补偿的问题，否则不必在运行时把这个量传给控制器**。开字段的意思是“下面那位请拿它去查表或做估计”。

那温度又是靠什么改零偏的？追到器件层有三条独立路径，它们在 Allan 曲线上的位置各不相同：

一、标度因子路径（改斜率）。 前面推过 `x_s/x_d ≈ 2ΩQ/ω_d`：把 Ω 读成多少位移，取决于 ω_d 与 Q。而 f = (1/2π)√(k/m)，k 由硅的杨氏模量 E 决定，E 随温度变。**温度不改真实的转动，它改“多少伏特算 1 °/s”这把尺子**。 乘性误差比加性误差毒：零偏可以减掉，标度因子错了会把每一次真实转动按比例放大成错误姿态。

二、机械偏置路径（改零点）。 质量块由支撑梁吊着；芯片、基板、塑封料热膨胀系数不同，温度梯度让梁受应力，应力改变刚度分布，于是“零角速度时读到非零”。它的恶劣之处在于**热滞**：升到 40 ℃ 再回 20 ℃，零点不回到原处。

三、电子学路径（低频 1/f）。 读出 ASIC 的失调与闪烁噪声随温度变；而 Q 依赖真空度，真空度也是温度的函数。

这条要不要命，可以用宇树自己的数卡一下。官方给 GO-M8010-6 的环境温度是 −5 ℃ ~ 40 ℃ [厂]（温差 45 ℃），给 Dex3-1 的是 −20 ℃ ~ 60 ℃ [厂]（温差 80 ℃）；整机 IMU 的温度范围未公布 [未]。而按本章那张表最后一行，工业级 0.01 °/s 的全部容差就是“1° ÷ 100 s”：**任何一次把零偏推动 0.01 °/s 的温度变化，都会吃掉一百秒的姿态预算**。 45 到 80 ℃ 的摆幅里，只要温度系数不为零，这条不等式就一定被越过——系数宇树没给 [未]，但判据不需要它。

再借一个同一台机器上另一颗传感器的量级感（第 05、36 章）：铜的电阻温度系数 +0.393 %/℃，钕铁硼剩磁约 −0.11 %/℃ [行]，符号相反，所以关节电机的扭矩常数 k_t 随温度的净变化是两者相加。**同一个密封壳里，电机的标度因子在漂、IMU 的标度因子也在漂，而它们之间隔着几十 ℃ 的温差**——这才是“姿态与力矩联合估计”的真正难度，不是滤波器的难度。

为什么是双 IMU

`unitree_hg` 里有 `doubleIMUState` [厂]，说明至少部分机型装了两颗。官方唯一给出的位置线索，是 G1 实机部署配置里那一行 `imu_type: "pelvis"` [厂]——**主 IMU 在骨盆，不在胸腔**。这个选择值得读一遍：骨盆离每一步的冲击最近，也离整机重心最近，躯干各段都挂在它上面，所以骨盆的角速度最接近“机体作为刚体的角速度”。装在胸腔，你就得先假设躯干是刚的。

理由有三个，都很硬：

一、位置不同，测到的不是同一件事。躯干 IMU 测机体姿态；末端（脚或手）IMU 测足端在摆动相的姿态与落地冲击。**落地瞬间足端 IMU 的波形，是判断“脚到底什么时候真的踩到地”的最直接证据**——这件事靠躯干 IMU 做不到，因为躯干的响应被 23 个关节的低通滤过了一遍。而接触时刻是第 17 章 ZMP 判据、第 19 章接触模式、第 21 章奖励函数共同需要的输入。**一个错的接触时刻会让一整段轨迹作废**。

二、冗余的价值不是“备份”，是一条可算的残差。两颗同址 IMU 的差值本身就是一个观测量：

```
r(t) = ω₁ − ω₂     正常时 r 只该有噪声大小（由 Allan 曲线短 τ 段定标度）
                   r 跳到一个持续水平 ⇒ 要么一颗在漂，要么结构在扭
```

单颗 IMU 永远给不出这个量，因为它没有参照。**一颗器件只能靠“输出离谱”猜自己坏了，两颗器件可以做减法**——这是故障判据与故障假设的区别。对一台 70 公斤、摔一次要修几周的机器，这个区别非常值钱。

三、两颗异址还能把“刚体转动”与“躯干弯曲”分开：骨盆与胸腔两颗的差值里含有躯干这段梁的挠动（$\delta \propto F \cdot L^3/EI$，第 02 章），而躯干弹性模态在几十 Hz [行]——这是真机上唯一能量出该模态的器件（第 22 章会说它在仿真里不存在）。

这三条哪条对宇树成立？不知道。 两颗分别装在哪、量程多少、同时刻采样还是分时复用、是否同一型号，官方全部未公布 [未]。本书只把判据摆在这里：拿到一台 G1/H1/H2，`doubleIMUState` 两个数组的差值是你自己可以画的曲线，而它的含义取决于上面哪一条物理在起作用。

本体感知：编码器就是“肌肉感觉”

人的本体感觉来自肌梭与关节感受器。机器的对应物是**关节编码器**——第 10 章说过它是 FOC 的命门，第 14 章说过它救不了齿隙。

现在补第三重身份：**它是人形数量最多、更新最快、最便宜的自身状态传感器**。

```
状态报文的 35 个关节槽位 × 每槽 (q, dq, tau_est, temperature×2, vol)
[厂]（35 是官方数组上限，**不是 G1 的关节数**：G1 是 23/29，见第 12 章）
更新率 500 Hz（rt/lowstate 官方口径）                          [厂]
⇒ 本体感知的标量流 = 35 × 6 × 500 = 105,000 个/秒
   前庭（四元数4+陀螺3+加速度3+rpy3+温度1）× 500 = 7,000 个/秒
   比值 = 105,000 ÷ 7,000 = 15
```

一次状态上报约 1234 字节、接收约 1270 字节（灵巧手文档口径 [厂]）。顺便读一下那个 15：**宇树的机器人对自己的肢体，比对自身姿态知道得多一个数量级**——每颗关节的 q 与 dq 是 FOC 闭环必需的（第 10 章），而姿态是可选的：不装 IMU 的机械臂照样精密，没有编码器的关节连自己在哪都不知道。姿态在这套栈里是外来户：它没有专用总线与带宽，是搭着关节数据一起过来的。

把这一串与 IMU 合起来，就是第 2 部所有控制器的输入：**姿态（IMU）+ 各关节位置与速度（编码器）+ 各关节估计扭矩（tau_est）+ 温度与母线电压**。

注意这份清单里没有“足端力”。 一台 G1 不知道它的脚踩到了多硬的地上——它只能从“电流出的扭矩与期望不符”去推断（第 14 章的准直驱逻辑）。**这是它选择低减速比的根本回报，也是它无法处理“踩到一块冰”这类场景的根本限制**。

再往下层：把这些碎片拼成“我在哪”

上面所有器件给的都是**相对量**：这一瞬间角速度是几、这一帧比上一帧倾了多少。 可整机要用的是一个绝对的东西——“躯干现在在世界的哪个位姿”。这中间差着一层， 行业里叫**状态估计**（把一堆带噪声的相对测量拼成一个可用的当前状态），再往上叫**定位** （长期知道自己在哪，通常还要顺手把地图建出来，即 SLAM）。这一层是蓝图里那六个时间域中 唯一还没有专章的一层，本节把它补在第 3 部开头，因为它是 IMU 存在的理由。

第一个追问：两个都不准的东西，凭什么合起来会更准?

因为它们的**不准是不一样的**。设陀螺给的角度不确定度 $\sigma_1 = 0.1°$，加速度计算出的倾角 $\sigma_2 = 0.5°$。 按精度加权（权重取方差倒数）合并：

> $\sigma = 1 / \sqrt{(1/\sigma_1^2 + 1/\sigma_2^2)} = 1 / \sqrt{(100 + 4)} \approx 0.098°$

看这个数：**0.098°，只比最好的那一路（0.1°）好了一点点**。 所以“融合能救命”是外行话。融合真正买到的不是精度，是**可观测性** （某个量能不能从测量里唯一地反推出来）：本章前面那条“绕重力轴的转”陀螺测不出来， 加速度计在静止时能看出重力方向，于是它管住了俯仰与横滚。 少一路，某些方向就永远定不下来，多好的滤波器也没用。

第二个追问：这个“加权”具体是怎么一直做下去的?

最主流的写法是扩展卡尔曼滤波（EKF，把非线性系统线性化后反复求均值与方差的递推估计器）， 它只有两步，循环执行：

> 预测：用运动学把状态往前推一步，方差变大（过程有噪声）
> 更新：拿一次实测做加权平均，方差变小

因子图（把每次测量攒成一个要最小化的约束项，一起解）是同一件事的另一种算法形态， 区别是它**不丢历史**，所以能用后来的测量回头修正很久以前那次估计。 **这两条路线本书只讲机制**：宇树未公布它的状态估计实

现，也未公布用的是 EKF 还是因子图 [未]。 它的 DDS 话题里给的是原料（quaternion / gyro / accel / rpy / tick），不是成品。本节全部按 [行] 处理。

第三个追问：为什么必须一直更新，攒一会儿再算不行吗?

因为误差是**积分**出来的。本章那张三行表：0.1 °/s 的零偏，10 秒漂 1°、100 秒漂 10°。 位置更狠，因为位置是加速度**积两次分**：

```
位置误差 ≈ ½ · b · t²          b = 加速度零偏
b = 0.01 m/s²、t = 10 s  ⇒  ½ × 0.01 × 100 = 0.5 m
```

一台人形原地不动，只要那 0.01 m/s^2 的偏差不被别的东西拉住，十秒后就“走出半米”。 这不是算法笨，是数学：二次方不等人。

那靠腿自己数步数行不行? 这就是腿式里程计：用 23 个关节的编码器读数反推足端相对躯干的位置， 再假设“着地的那只脚不打滑”，就能从脚推回躯干。它便宜、高频、不用外部设施， 但误差随步数**随机游走**（每步一个方向随机的错，累加按步数的平方根增长）， 而且打滑这件事**在它自己的模型里不可观测**——它把滑掉的那几厘米当成真的走了。

唯一能把误差清零的是“认出老地方”：视觉回环（在序列里发现“这一帧和很久前那一帧是同一个地方”， 于是把中间累积的漂移一次性拉回去）。这也是为什么定位系统必须“走过才知道、知道要回头改”—— 先动才能测，测完才能修正，而修正又会改变你对之前整条轨迹的看法。这个耦合是本层的真难点， 不是算力问题。

边界：融合救不了什么。 两路传感器若被同一个物理量一起带偏——本章反推四那条温度曲线 同时拖走陀螺零偏与加速度计标度因数——加权平均毫无办法，它只能压住**不相关**的噪声。 另一条边界是模型：把打滑当纯滚动，再精确的滤波器也会收敛到一个错误答案。

类比，以及它在哪儿破

类比：IMU 是内耳，编码器是本体感觉。两者合起来对应人的前庭 + 肌梭系统——人闭着眼也能站稳，靠的就是这一对。

这个类比在哪里破，而且破在要害：人的内耳与前庭脊髓反射是**硬连线**的，延迟约 10–30 毫秒，且不经过意识；而机器的这条链要走总线、进滤波器、跑控制律。更关键的是，**人的前庭和肌梭在制造时不需要标定，而机器的每一颗 IMU 都有零偏、每一台装配都有安装角误差，都要在产线上标一遍**。

于是“这台机器能不能站住”这个问题，最终会落到一个非常不性感的地方：**标定工艺与产线一致性**。第 10 章那条 5° 角偏表、本章的 g-sensitivity，全都是标定与安装问题，不是算法问题。

人形机器人量产难，难的一半在这里。

一个可检验的说法：MEMS 陀螺读的是科里奥利位移，`x_s/x_d ≈ 2ΩQ/ω_d`（1 °/s 在 20 kHz、Q=1,000 的示例里只值万分之 2.8）；因为热噪声下限 `x_n = √(k_BT/k)` 与 Q 无关，可分辨角速度按 1/Q 走，而把 Q 做高的代价就是谐振频率对杨氏模量、也就是对温度的敏感。姿态误差有两条时间律：`b·t`（1 °/s × 100 s = 100°）与 `ARW·√t`，t 必然战胜 √t，所以任何纯惯性姿态在几十秒内失去可用性，出路只有用不依赖积分的参考（重力、视觉、足端接触假设）周期性重置。由此反推三项硬指标：零偏优于 0.1 °/s、输出率不低于控制层量级、g 敏感度小到落地冲击引入的周期性零偏不与积分项共振。宇树只公布字段与双 IMU 结构（`imu_type: "pelvis"`），上述指标一项都没公布。

证据等级：科里奥利力形式、谐振放大与 Q、能量均分给出的热位移下限、Allan 方差的定义与三段斜率、热机械噪声 $\propto 1/Q$、g-sensitive bias 量级、卡尔曼频段互补、结构模态几十 Hz [行]；DDS 字段、`doubleIMUState`、`imu_type: "pelvis"`、`rt/lowstate` 500 Hz、`control_dt:`

`0.02`（50 Hz）、`tick` 字段、1234/1270 字节、GO-M8010-6 的 −5~~40 °C 与 Dex3-1 的 −20~~60 °C、Mid-360S 的 `Built-in IMU Model: ICM40609`、扭矩常数 0.63895 N·m/A [查]（这一行是混栏：GO-M8010-6 的 −5~~40 °C 与 Dex3-1 的 −20~~60 °C 是宇树器件页 [厂]，Mid-360S 那两项来自别家官方页，按图例只能记 [查]）；2.78e-4、36/360 °/hr、0.259 s、0.026°、3.86 s^{-1} = 0.61 Hz、105,000 与 7,000 标量/秒及其比值 15、45/80 °C 温差、温度系数 0.393 与 −0.11 %/°C 的净效应 [算]（20 kHz 与 Q=1,000 是本书的**示例代入**，不是任何器件的实测值）；**IMU 型号、供应商、量程、零偏稳定性、噪声密度、整机 IMU 采样率、两颗的安装位置、g-sensitivity、温漂系数全部 [未]**；“重心容差 1–3°”是本书按几何估读 [算]。上一版本章写的“L2 内置 1 kHz IMU”按官方口径无出处，已改回 [未]。

可核对清单：

- ☐ 用 角度误差 = b·t 自己填一遍那张三行表，确认 0.1 °/s 在 10 秒时是 1°
- ☐ 用 x_s/x_d = 2ΩQ/ω_d 复算一遍：Ω=0.01745 rad/s、Q=1,000、ω_d=125,664 rad/s ⇒ 2.78e-4
- ☐ 用 0.01 °/s × 3600 与 0.1 °/s × 3600 复算 °/hr，确认 36 与 360，再想想你为什么第一次看到 36 °/hr 时觉得它小
- ☐ 用 √(0.66 ÷ 9.80665) 复算 0.259 s，再乘 0.1 °/s 确认一个时间常数内只漂 0.026°
- ☐ 用 35 × 6 × 500 = 105,000 与 14 × 500 = 7,000 复算那个 15 倍
- ☐ 打开 `unitree_hg/msg/IMUState.msg`（或 SDK 头文件）数一遍字段，确认里面有 temperature；再打开 `LowState.msg`，确认里面有 tick
- ☐ 找一份任何一款消费级 MEMS 陀螺的数据手册（MPU-6050/ICM-42688 都行），看它的零偏稳定性与 g-sensitivity 写在哪一页——这就是你问厂商该问的三个字段

- ☐ 在宇树 G1/H1/H2 页面搜“IMU 型号 / bias instability / 零偏 / 采样率”，确认是否真的什么都没有（预期：无）

下一章：本章说“末端 IMU 能给出接触时刻”。下一章去看真正产生接触的那个部件——一只手。它是这台机器上唯一一处自由度比腿还多、而扭矩比腿小两个数量级的地方。

第 25 章

手：Dex3-1 的 33 个触觉点与空心杯电机

全身最难的部件不在腿上

一台人形上，自由度密度最高、单件空间最小、力要求最精细、而最容易坏的地方是手。

而且它还是宇树**目前唯一没能自研摊薄的大件**：2025 年前三季度，宇树向外采购灵巧手 1,210 只、金额 1,720 万元，占其外采总额约 96% [推]（财经报道口径）。第 03 章说过这件事的含义：**关节能自己做的，手做不了**。

这一章讲为什么做不了——用一道小学物理的不等式。

是我们自己研发的，因为 3 个手指足够做绝大部分事了。另外波士顿动力最新的人形机器人也是 3 个手指的，这也从侧面说明，3 个手指基本够用。

王兴兴 — 《晚点 LatePost》对话王兴兴（新浪转载） — 2025-02-16（首刊 2024-05）（直接引语（问答实录））

第一道算术：捏住一瓶水要多大力

拿起一瓶 500 ml 的矿泉水并悬在空中。瓶重 0.5 kg，重力 4.9 N。

手指是从两侧夹住它的，靠的是**摩擦**。竖直方向平衡条件：

```
μ · ΣN ≥ m·g        ⇒        ΣN ≥ m·g / μ
```

```
μ（指尖对 PET 塑料，湿手可能低到 0.2，干橡胶指套可到 0.6）[行]
取 μ = 0.4：ΣN ≥ 4.9 / 0.4 = 12.3 N
两指对捏 ⇒ 每侧法向力约 6.1 N
```

（[算]。）

现在看这件事有多难。 12 牛的总法向力听起来很小。但请注意三个约束同时成立：

一、力不能太大。 捏过 12.3 N 太多，薄壁 PET 瓶会瘪；捏纸杯会破；捏鸡蛋会碎。所以这不是“够大就行”，是**要落在一个窄带里**。而 μ 本身是未知的（手汗、油污、瓶身是否有水），控制器不知道 μ，只能靠“捏到没滑”来闭环——**这就需要触觉**。

二、力不能太小，且要持续。 一旦滑落，物体加速度是 g，从指间掉到地面 1 米只要 0.45 秒。

三、执行器的可用空间是几立方厘米。 于是扭矩链条变成这样：

```
指尖力 6 N × 指节力臂约 0.03 m ⇒ 关节需 0.18 N·m（还要乘传动效率与连杆增益）
```

对照官方给的手用微型执行器：**F-1515-108 与 F-1515-214**，尺寸 34.8 × 23.1 × 23.3 mm、重 45 g、减速比 1:108 与 1:214、官方“理想值”扭矩 0.76 与 1.498 N·m [厂]。

顺手做一次第 03 章那种“数量是乘数”的换算。Dex3-1 官方写 7 个自由度 = 6 个直驱微型力控关节 + 1 个齿轮传动微型力控关节 [厂]，整只手重 710 g [厂]：

```
7 颗 × 45 g = 315 g ；315 ÷ 710 = 44%
```

（[算]。）**一颗手用关节是 45 克，七颗就把整只手的一半重量吃掉了**，剩下 395 g 要装触觉阵列、壳体、板子和线缆。腿那边也是同一笔账（第 03 章：关节占整机成本约一半），只是尺度差 25 倍。**这就是“手是执行器的极端案例”的含义：同一条成本与重量的规律，被压到 45 克、几立方厘米的体积里**。

0.18 N·m 的需求 vs 0.76–1.5 N·m 的器件——看起来够用。但请注意那两个字：**理想值**。第 08 章已经证明过一体化关节参数表上的“理想值”是角点乘积，而 1:214 的微型齿轮箱效率不会高（小模数齿轮、多级、润滑困难）。**真实可用扭矩通常是理想值的一半甚至更少**。

而这里有一颗官方给的定心丸，也露出一处矛盾。同一张参数表还给了 `Max Torque(Actual)`：同向 0.49 与 0.86 N·m，反向 1.37 与 3.1 N·m [厂]。除一下就知“一半”不是修辞：0.49 ÷ 0.76 = 64%，0.86 ÷ 1.498 = 57%（[算]）。可反向那两栏比“理想值”还大 1.8 与 2.1 倍，**这在物理上不可能是同一件事的两种测量**（第 15 章见过官方另一处不对称：N7520-22.5 的 Y1/Y2 = 111 与 131 N·m，那是同向/反向峰值，差 18%，不是 80%）。**本书把 Ideal 与 Actual 两组数原样并列，不解释这个矛盾、不取平均**：能确定的是“理想值”本身就是换算值而不是实测值，正文必须带着这个限定；不能确定的是 Actual 那两栏在测什么，官方未说明 [未]。

这颗 45 克器件的两个档位还留了一条可以自己走完的验证路。把两档“理想值”各自除回转子侧：

```
0.76  ÷ 108 = 0.007037 N·m
1.498 ÷ 214 = 0.007000 N·m        差 0.5%
```

（[算]。）**两颗不同的齿轮箱，除出来的转子扭矩是同一个数——这是官方数据自己承认：F-1515-108 与 -214 是同一颗裸电机换了一个齿轮组**。第 15 章见过完全同构的一条自检（G1 的 88 ÷ 14.3 = 6.15 与 139 ÷ 22.5 = 6.18 N·m，差 0.4%）。而 7 mN·m 的转子扭矩意味着什么？它是 G1 膝关节转子扭矩的 1/880（6.18 ÷ 0.0070 [算]）。**手指的全部力气都是齿轮借来的**。

而这颗 45 克器件的减速比是 **1:214**。对照第 14 章：G1 髋部用 14.3–22.5，Z1 用谐波 60+。**手用 214**。

为什么？ 因为手不需要感知力——它有触觉阵列（下面讲）。手需要的是**在几立方厘米里塞出足够大的力**，而减速比是唯一免费的方法。代价是齿隙、摩擦、不透明、以及“不能反驱”（被外力掰手指时电机会被拖着转不了，或者根本掰不动）。

于是手和腿走了完全相反的两条路：腿降减速比换力感知，手升减速比换扭矩密度。这是同一套物理在两种约束下的两个解——也是本书认为手“做不了自研”的技术原因：**它需要的不是同一类器件**。

摩擦锥：一个接触点只有一把预算

上面那条 $\mu \cdot \Sigma N \geq mg$ 只问了竖直方向。可同一个接触点还要拦住侧推、还要拧瓶盖，而它的全部能力被一个圆锥罩住。这就是第 17 章那条摩擦锥长在手上的样子：法向力为 N 的一个接触点，能给出的切向力矢量只能落在锥内（ϕ 是锥半角）：

```
|F_t| ≤ μ · N            ϕ = arctan μ
μ = 0.2 ⇒ ϕ = 11.3° ; μ = 0.4 ⇒ ϕ = 21.8° ; μ = 0.6 ⇒ ϕ = 31.0°
```

（[算]。）**μ 从 0.2 涨到 0.6，可用方向也只张开 11° 到 31° 这一个扇形**。超出锥面，接触就从粘着变成滑动，摩擦系数从 μ_s 掉到 μ_k，滑动开始自加速——这就是“切向力超过 μN 就打滑”的完整形式：**任何方向的切向分量都算，它们共用同一把预算**。

于是“拧瓶盖”这件事的上限也是一次乘法（r 是瓶身半径，两指在两侧各出一个切向力）：

```
τ_max = Σ (μ · N_i) · r        单点写作 τ = μ · N · r
```

关键的一步是把它花掉的部分扣掉。竖直方向已经吃掉 $mg/2 = 2.45$ N，每指剩下的切向余量是个勾股：

```
余量 = √( (μN)² − (mg/2)² )
最小捏力 μN = 2.45 N  ⇒  余量 = √(2.45² − 2.45²) = 0 N
捏到两倍  μN = 4.88 N  ⇒  余量 = √(4.88² − 2.45²) = 4.22 N
```

（[算]。）再乘力臂，用官方给的两个半径：Dex3-1 的最小握持直径 10 mm [厂]（r = 0.005 m），官方测负载用的那个“5 cm 圆硬物”[厂]（r = 0.025 m）：

```
两倍裕度、5 cm 物体：τ = 2 × 4.22 × 0.025 = 0.21 N·m
两倍裕度、1 cm 物体：τ = 2 × 4.22 × 0.005 = 0.042 N·m
```

这三行是本章最要紧的一组数，因为它说清了两件事。**第一，“刚好捏住”等于“任何一点侧向扰动都会滑”**——摩擦圆的预算已经被重力花光，余量精确为零；想“捏得住还拧得动”就必须捏得更狠，而捏得更狠就撞上“力不能太大”那条约束（薄壁 PET 瓶会瘪）。**第二，瓶颈会换位：** 0.21 N·m 小于器件的“理想值”0.76 N·m [厂]，所以在 5 cm 这个尺寸上电机给得起，限制来自接触面；而在 1 cm 的物体上，摩擦锥把可用扭矩锁死在 0.042 N·m，再大的电机也白给——**拧得动拧不动，是摩擦锥定的，不是齿轮箱定的**。

第二道算术：触觉点要多少

宇树 Dex3-1 官方规格：**9 阵列 / 33 个压力传感器，量程 10–2500 g，最大承受力（不损坏）20 kg** [厂]。Dex5-1P 官方称 **94 个触觉点** [厂]（这一项需再核，见附录 E）。

33 个点对一只手意味着什么？一只人手有约 **17,000 个**机械感受器（Meissner 与 Pacinian 小体等）[查]。所以 33 点是它的 1/500。

先复算一遍官方给的点数

官方页把阵列布局拆开写了 [厂]，两个总数都能自己加起来：

```
Dex3-1：  2×2 (掌心) ×3 = 12  +  2×2 (指腹) ×3 = 12  +  3×1 (指尖) ×3 = 9
⇒ 33
Dex5-1P：  2×5 (掌心) = 10 + 2×3 (指腹) ×5 = 30 + 2×3 (指尖) ×5 = 30 + 2×3
(指根) ×4 = 24  ⇒ 94
```

（[算]，布局为 [厂]。）两笔都对得上官方的 33 与 94，这不是巧合，说明“点数”这个宣传数背后有一张真实的排布表。而这张表里藏着本章的关键信息：**指尖那一列是 3×1——一维**。三个点排成一条线，意味着指尖在一个方向上有分辨率、在垂直方向上没有。

触觉点阵买的是“分布”，不是“大小”

这是本章最想让你带走的一个判断。抓取时手上的未知量是多少？Dex3-1 是三指手（官方拆解：拇指 3、食指 2、中指 2 个主动自由度 [厂]），三个指尖各有一个三维接触力（1 个法向 + 2 个切向）：

```
未知量：3 个接触点 × 3 个分量 = 9
方程：  刚体平衡只有 6 条（3 条力 + 3 条力矩）
不可由合力观测的自由度 = 9 − 6 = 3
```

（[算]。）换成五指手就是 15 − 6 = **9 维**。这多出来的维度叫**内力**（self-stress）：物体受到的合力与合力矩完全可以一样，而指尖之间的挤法截然不同——一个配置捏得瘪瓶子，另一个配置只是稳稳托着，**六维力传感器分不清它们，因为它们本来就相同**。

而打滑这件事从来不发生在“合力”上，它发生在**某一个点**上：那个点的切向分量越过 μN 就先滑，其余指点还在粘着。于是只测合力大小的控制器，看到的是一条被平均过的曲线，而真正的失效在平均里被抹平了。更糟的是，即使总的切向力还没到 μN，接触斑内也可能已经开始滑——理论解（Cattaneo–Mindlin 那一类）给出的图像是**中心粘着、边缘先滑**，因为边缘的正应力最小而需要的剪应力最大 [行]。**能看见“边缘开始滑”的器件只有阵列；单个测力元件连这个概念都没有**。这就是点阵解决“力分布”而不解决“力的大小”的确切含义。

顺手把量程也换算一遍，看这 33 个点够不够细 [厂]（10 g – 2500 g）：

```
下限 10 g × 9.80665 = 0.098 N ; 0.098 ÷ 12.3 N（最小捏力）= 0.8%
上限 2500 g × 9.80665 = 24.5 N ; 24.5 ÷ 12.3 = 2.0 倍
```

（[算]。）**量程下沿正好落在“需要分辨的力”的百分之一，上沿是最小捏力的两倍**——这个窗口对捏瓶子是够用的，但它也说明：超过 2 kg 的法向力，传感器就读顶了，而 20 kg 只是“不损坏”的上限 [厂]。

但点数不是重点，**分布**才是。33 个点大致覆盖五个指尖与掌心，而抓取时**只有指尖那一小块区域在做功**。所以真正的信息瓶颈是：

```
指尖面积（约 1 cm² 有效接触）÷ 该区域传感器数（约 3–5 个）
⇒ 每个触点摊到 0.2–0.33 cm²，开方得间距约 4.5–5.8 mm
```

（[算]，分布是本书按产品图估读 [未]。）

4.5–5.8 mm 意味着什么？（旧版这里写 2–3 mm，是把面积直接开了一次的口径错） 意味着它能分辨“捏住了一张纸还是两张纸”（每张约 0.1 mm，分辨不了），但能分辨“瓶子上有没有一道褶皱”（几 mm，勉强）。

对照行业里另一条路线——**视触觉（GelSight 类）**：用一颗相机拍一块弹性凝胶的形变，等效空间分辨率可到几十微米，能读出剪切力方向（也就是**能提前知道要滑了**）。宇树 Dex5 系列官方口径提到空心杯电机与更多触觉点 [厂]，但**是否为视触觉，官方未明说** [未]。

（同期另一家的公开口径可作对照：AGILINK Ultra-T 宣称 22 主动自由度 + 三维触觉、分辨率 0.01 N；Ultra-M 宣称全直驱 + 视触觉、0.005 N [查]。**0.005–0.01 N 这个量级，与 Dex3-1 的“10 g 起量程”（≈0.098 N）差一到两个数量级**。 这是本章能给出的最有信息量的横向对比。）

第三道算术：空心杯电机为什么没有铁芯

Dex5-1 的官方中文页给出的措辞是**空心杯电机**加“低阻尼小间隙减速器” [厂]（英文页写作 micro force-controlled composite transmission joints）。F-1515 那一档官方没有说它是不是空心杯 [未]。但“转子没有铁芯”这件事值得单独讲一节，因为它是本章“代价”最集中的地方。

先问铁芯给你什么。 定子的硅钢片把磁通收拢、集中，送进气隙；有了铁，同样多的安匝能在气隙里撑起高得多的磁密 B。而扭矩正比于 B：

```
τ = k_t · i ，k_t ∝ B · (绕组面积) · (匝数)
```

所以铁芯是白送扭矩的。 那为什么还要拿掉它？因为铁同时带来三样东西：

一、**齿槽转矩**。 转子的永磁体喜欢转到与定子齿对齐的位置——即使电流为零，也会有一格一格的周期性扭矩（每转的格数 = 齿数 × 极对数）。对手指这是致命的：0.1 N·m 级别的精细出力上叠一个每齿一次的纹波，位置控制就会一顿一顿，力控制就更别提。**无铁芯的转子没有铁可对齐，齿槽转矩直接归零**——这不是“减小”，是没有。

二、**铁损**。 转子带着磁钢在旋转磁场里，每转一圈磁化方向就翻一次：滞回损耗 ∝ 频率 × B 的幂，涡流损耗 ∝ 频率² × B² × 片厚² [行]。两者都随转速涨，涡流项涨得更快。而无铁芯的转子只有一段铝/铜的自撑绕组杯，没有磁化循环可言。

三、**转子惯量**。 一颗有铁芯的电机，转子质量的绝大部分就是那叠硅钢。惯量 $J \propto m \cdot r^2$，而薄壁杯的 $J = m \cdot r^2$（实心柱才是 $\tfrac{1}{2} m \cdot r^2$）——**关键是 m：拿掉铁芯拿掉的正是转子质量的主体**。 惯量小意味着两件事：加减速快（同一个扭矩下 $\alpha = \tau / J$ 大得多），以及电流环更容易稳定（第 09 章：机械侧的积分器变快了，环路相位裕度就好做）。手指要的是每秒几十次的小幅快速修正，这条收益非常实在。

代价必须写出来。 少了铁，磁通进出转子要连着走两段空气隙，而空气的磁阻比硅钢高三个数量级；同样的磁钢，气隙里的 B 就低得多。**结果就是 k_t 低、扭矩密度低**：电机本体出不了力，只能靠后面的齿轮组把力放大——**这才是 1:214 的真正理由，而不只是“手要小”**。

这条代价可以用官方的数算出来。拿两颗都给了官方扭矩与官方重量的器件，用同一把尺（扭矩 ÷ 质量）：

```
F-1515-108：0.76 N·m ÷ 0.045 kg = 16.9 N·m/kg
GO-M8010-6：23.7 N·m ÷ 0.530 kg = 44.7 N·m/kg
比值 = 44.7 ÷ 16.9 = 2.65
```

再按官方尺寸算两者的体积比，取出立方根得到线度比：

```
M8010 体积 = 96.5 × 92.5 × 42.3 = 377,580 mm³      [厂]
F-1515 体积 = 34.8 × 23.1 × 23.3 = 18,730 mm³        [厂]
体积比 = 20.2  ⇒ 线度比 = 20.2^(1/3) = 2.72
```

（[算]。）**而 2.65 与 2.72 只差 2.6%**。这不是巧合：几何相似的电机，在同样电流密度下 $\tau \propto L^4$、质量 $\propto L^3$，所以**每千克的扭矩 $\propto L$**——第 02 章那条平方-立方律，在缩小的方向上照样收税。也就是说：把关节做小，不只是“力气变小”，而是**每单位重量的力气按线度线性变小**；再叠上无铁芯丢掉的那份磁路增益，微型手指关节就只能靠 1:214 的齿轮把扭矩借回来，然后付齿隙、效率与不可反驱的租金。

两条必须同时说的限定。一，这个 2.6% 的吻合只是量级校验：两台的减速比差 17 倍（6.33 与 108），齿轮占重量的比例不同，“理想值”与“峰值扭矩”的口径也不完全一致，本书不把它当定律的证实。二，H1 页那个官方“峰值扭矩密度 189 N·m/kg” [厂] 是第三种口径（分母是不是整个关节单元、含不含减速器，官方未说明 [未]），**16.9、44.7、189 三个数不能画在同一张图上**；本书把三者并列，前两数用的是同一把尺，第三个数是厂商话术。

第四道算术：为什么腱绳是一种妥协

灵巧手的传动有三条路：

方案	电机在哪	优点	代价
直驱/近端齿轮	指节内	结构简单、刚	手指粗、惯量大
腱绳（钢丝绳）	手掌或前臂	手指细、惯量小、可长距离传力	绳会伸长、会磨、要预紧、非线性、难标定
连杆	指节内	可靠、可设计轨迹	摩擦多、行程受限

灵心巧手 L20 是 21 自由度腱绳方案 [查]；宇树 Dex3-1 的指关节用 F-1515 系列（近端齿轮）[厂]。

腱绳的物理问题可以算。一根 0.5 mm 直径的钢丝绳，长 200 mm，弹性模量 190 GPa：

```
截面积 A = π/4 × 0.0005² = 1.96e-7 m²
刚度 k = A·E / L = 1.96e-7 × 190e9 / 0.2 = 186,000 N/m
拉 5 N 的伸长 = 5 / 186000 = 0.027 mm
```

（[算]，材料参数 [行]。）

0.027 mm 很小，看起来不是问题。但**绳的“伸长”不是弹性的**：它有蠕变（越拉越长）、有绕轮摩擦导致的迟滞（拉与松的曲线不重合）、有滑轮效率损失（每个转向轮约 3–10%）。

结果是：腱绳传动的位置控制精度会随使用次数漂移，而力控制会因迟滞出现 10–30% 的不确定。这就是为什么腱绳手几乎必须配触觉——**传动本身不可信，只能靠末端测量兜底**。

而第 14 章那条逻辑在这里反过来了：**腿是“传动可信所以可以省掉力传感器”，手是“传动不可信所以必须加触觉”**。同一套物理，两个方向。

手的控制为什么与身体解耦

第 19 章说过：手的接触模式数量比脚多一个数量级，塞进同一个 QP 会爆。所以手的控制是独立的一条链。

从官方文档能读到这条链的样子：手部走的是自己的话题与协议（`rt/arm_sdk` 一类），单包发送 1234 字节、接收 1270 字节，反馈字段是 Joint Mode / Position / Velocity / Torque / **Stiffness Coefficient** / **Damping Coefficient** [厂]。

注意最后两个字段又出现了：手的关节同样是可编程弹簧-阻尼器（第 18 章）。也就是说宇树在从 360 牛米的膝盖到 45 克的指关节这一整个跨度上，用的是**同一套控制语法**。

这件事本身值得记一笔：**统一抽象是这家公司产品能快速铺开关数量的原因**——同一套 SDK、同一套消息结构、同一套 kp/kd/tau 语义。工程师学一次就能用全系列。

类比，以及它在哪儿破

类比：手是这台机器上唯一“力气要求小但精度要求高”的部位，像外科医生的手而不是搬运工的手。

这个类比在哪里破：外科医生的手有指尖视觉（能看见皮肤凹陷）、有几千个感受器、有二十多年的运动学习。而一只机器人手有 33 个触觉点、一根会漂移的钢丝绳、和一个 1:214 的齿轮箱。

更准确的类比是：手是这台机器上传感器最少、执行器最不精确、而任务最复杂的部位。这三个“最”同时成立，就是它为什么是全行业最难的一环，也是为什么各家宣传里最难的任务总是与手相关的任务（**本书已删掉两条读不到原文的成功率数字，见第 38 章**——但“手是最难的一环”这句话不需要那两个数也成立）。

一个可检验的说法：捏住一瓶 0.5 kg 的水需要总法向力至少 $mg/\mu \approx 12.3$ N（μ=0.4），且必须落在一个窄带内（过大会压瘪薄壁容器），而 μ 本身不可测，因此抓取闭环必须依赖触觉；手用 F-1515 系列微型执行器的减速比达 1:214，与 G1 髋膝的 14.3–22.5 相差一个数量级——腿为保力

感知而降减速比、手为在几立方厘米内出扭矩而升减速比，是同一物理在两种约束下的反向解；腱绳传动因蠕变与迟滞不可信，故必须用末端触觉兜底。

一个可检验的说法：捏住一瓶 0.5 kg 的水需要总法向力 $\geqslant mg/\mu \approx 12.3$ N（$\mu=0.4$），而在这个最小值上，摩擦圆 $|f_t| \le \mu N$ 的切向余量精确等于零——因为 $\sqrt{((mg/2)^2 - (mg/2)^2)} = 0$——所以“捏得住”与“拧得动/推得稳”抢的是同一把预算，捏到两倍才剩下 4.22 N 的余量，对应 $\tau = 2\times 4.22 \times r$：5 cm 物体上 0.21 N·m、1 cm 物体上 0.042 N·m。**前者的瓶颈在接触面而不在电机（器件“理想值”0.76 N·m 用不完），后者被摩擦锥锁死，再大的电机也没用。** 触觉点阵买的是“分布”而不是“大小”：三指手有 $3\times3=9$ 个接触力未知量而刚体平衡只有 6 条方程，多出来的 3 维（五指手是 9 维）是任何合力测量都看不见的内力，而滑动从单个点的边缘开始（Cattaneo–Mindlin 的图像是中心粘、边缘滑）。官方阵列布局能自己复算：33 = 12+12+9、94 = 10+30+30+24，而指尖那一列是 3×1（一维）。手用 1:214 与腿用 14.3–22.5 是同一物理的反向解；空心杯拿掉铁芯换来零齿槽转矩、无转子铁损与极低惯量，代价是磁路没有铁来集中磁通 $\Rightarrow$ k_t 低 $\Rightarrow$ 每千克扭矩只有 16.9 N·m/kg，而 GO-M8010-6 是 44.7——比值 2.65 与两者线度比 2.72 吻合（$\tau/m \propto L$），即微型化方向上平方-立方律照样收税。

证据等级：F-1515-108/214 尺寸重量与“理想值”/“实际值”扭矩（同向 0.49/0.86、反向 1.37/3.1）、减速比 1:108 与 1:214、峰值线电流 4.76 A、Dex3-1 的 7 DoF（6 直驱 + 1 齿轮）/710 g/9 阵列 33 点与阵列布局/10–2500 g/20 kg/最小握持直径 10 mm/指尖重复定位 ±2 mm/抓 5 cm 硬物 500 g、Dex5-1 空心杯与 94 点及其阵列布局、GO-M8010-6 的 23.7 N·m/530 g/6.33/96.5×92.5×42.3 mm、手部消息字段与 1234/1270 字节 [厂]（Dex5-1 的 94 点与 IP 等级本书标注为需再核）；12.3 N、6.1 N、11.3°/21.8°/31.0°、4.22 N、0.21 N·m、0.042 N·m、0.18 N·m、315 g 与 44%、0.007037 与 0.007000 N·m 及 0.5%、1.48 mN·m/A 与 68 倍、16.9/44.7 与 2.65、377,580/18,730 mm^3 与 2.72、

33 = 12+12+9、94 = 10+30+30+24、9 − 6 = 3 维、0.098 N 与 0.8%、24.5 N 与 2.0 倍、186,000 N/m、0.027 mm、2–3 mm 是 [算]，输入含本书估读（指节力臂、接触面积、传感器分布）[未]；μ 区间、摩擦锥与 Cattaneo–Mindlin 局部先滑、齿槽转矩与铁损成因、能量均分、人手感受器数量、腱绳蠕变与迟滞、滑轮效率 [行][查]；因时/灵心巧手/AGILINK 的参数与价格 [查]；宇树外采手 1,210 只/1,720 万元/96% [推]；**Dex3-1/Dex5-1 官方售价、Dex 系列是否为视触觉、触觉阵列的空间分辨率、F-1515 是否空心杯、Actual 两栏在测什么，均未公布** [未]。

可核对清单：

- ☐ 自己算一次 ΣN ⩾ mg/μ，并试 μ = 0.2 与 0.6，看法向力差多少倍（答：3 倍——这就是“不知道 μ”的代价）
- ☐ 算一遍 arctan 0.2 / 0.4 / 0.6，确认摩擦锥半角是 11.3° / 21.8° / 31.0°
- ☐ 用 $\sqrt{((\mu N)^2 - (mg/2)^2)}$ 复算：最小捏力时余量为 0，两倍裕度时 4.22 N，再乘 0.025 m 与 0.005 m 得 0.21 与 0.042 N·m
- ☐ 用 0.76 ÷ 108 与 1.498 ÷ 214 复算转子扭矩，确认两者相差 0.5%，再除以 4.76 A 得转子侧 k_t
- ☐ 用 0.76 ÷ 0.045 与 23.7 ÷ 0.530 复算 16.9 与 44.7 N·m/kg，再用两台官方尺寸的立方根验证 τ/m ∝ L
- ☐ 把 Dex3-1 与 Dex5-1P 的阵列布局各自加一边，确认你能算出 33 与 94
- ☐ 用 A·E/L 复算一次钢丝绳刚度，确认 186,000 N/m
- ☐ 拿一瓶水，用手指对捏到刚好不滑，感受那个力（约 6 N 一侧），再试试在这个力下能不能拧开瓶盖（答不出没关系——那正是摩擦圆余量为零的体感）
- ☐ 在宇树 Dex3-1/Dex5 页面搜“视触觉 / GelSight / 光学 / 分辨率 / 价格”，确认哪些查不到（预期：分辨率与价格）

下一章：手、腿、平衡都讲完了，还剩一个部件在决定这一切能做到什么程度——那块决定“要不要去拿那个杯子”的板子。下一章算它的功耗与带宽。

第 26 章

大脑的功耗预算：130 W 的 Thor 与 6B 的模型

你给机器人装上一块标着 100 TOPS 的板子，再装一个 6B 参数的模型，然后问它一秒能想几次。第一次除法：模型每走一步要做约 2 × 6e9 = 1.2e10 次浮点，1e14 ÷ 1.2e10 ⇒ 理论上**每秒八千多次**。第二次除法：把这 60 亿个参数当成 12 GB 数据从显存里读一遍，除以大约 100 GB/s 的带宽 ⇒ **每秒八次**。真机报告的是第二个数。**这一章要算的就是这两个数之间那一千倍的差从哪里来**，以及为什么它不是软件问题。（第一个除法里本书故意把 TOPS 当 TFLOPS 用——这两个口径其实不可比，下一节“先把官方给了什么说准”的第三条就是冲着这件事写的；这里故意偏袒芯片，好让结论更难被推翻。）

> 在人形机器人上，或者在移动机器人本体上，其实没办法直接部署大规模的算力。它的尺寸只有这么大，它的电池只有这么大，它部署算力的功耗是有限制的。我个人感觉在人形机器人上，最多只能部署峰值功耗为 100 瓦的算力，平时工作的时候算力只有小几十瓦，简单说就只有大概几个手机的算力水平。
>
> 王兴兴 — 王兴兴 2025 世界机器人大会主旨演讲（新浪“机器人前瞻”文字实录） — 2025-08-09（第一人称演讲逐字）

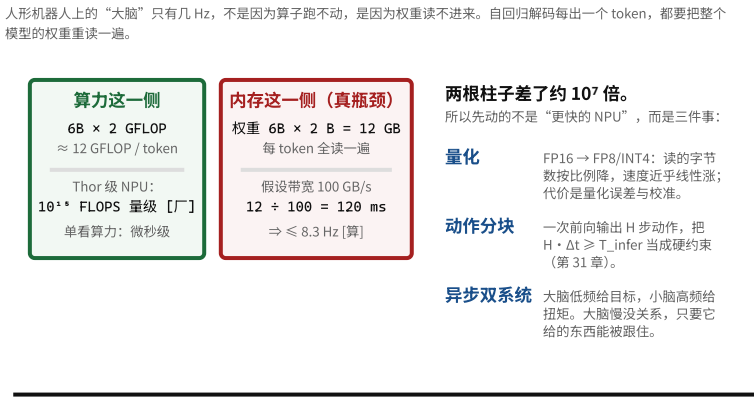


图 26-1　内存墙不是算力墙：12 GB 权重每 token 读一遍，100 GB/s 时就是 120 ms

先把官方给了什么说准

这一节的每一行都有出处，而且**措辞要卡得很严**——因为“标配 Orin”这类说法在网上被传得很快。

```
G1 EDU 的开发计算单元 PC2：官方文档中心表格写 Jetson Orin NX，显存/内存 16G，
   CPU 为 Arm Cortex-A78AE、8 线程、最高 2 GHz、L2 2MB + L3 4MB，
   GPU 为 1024 个 Ampere 核心 + 32 个 Tensor Core，最高 918 MHz      [厂]
G1 产品页对高算力模块的原文：Multiple brands and models available such as Orin, etc.  [厂]
H2 可选计算模块：Jetson Thor（T5000）或 Jetson Orin NX（16GB）          [厂]
Thor T5000：2070 TFLOPS（FP4-Sparse）；Orin NX 16GB：100 TOPS
[厂]
H2 宣传页原文：31 degrees of freedom, 360N·m joint torque;
   Powered by a 2070 TOPS chip
[厂]
H1 算力：标配 Intel i5（平台功能）+ i7（用户开发），可选 i7 或 Orin NX，
   最多 3 块
[厂]
```

三个必须小心的地方：

一，**“G1 EDU 标配 Jetson Orin NX 16GB”这句话不能这么写**。产品页只说“多品牌多型号可选，例如 Orin”；16G 这个数字出现在文档中心的 PC2 参数小节里；**官方从未用“标配 16GB 这一档”的措辞** [厂]。本书因此写成“官方文档中心列出的 PC2 计算单元参数为 Jetson Orin NX / 16G”。

二，**2070 TFLOPS 是 FP4-Sparse 口径**。稀疏加速是“权重里大部分是零时跳过计算”，实际稠密算力通常只有标称的几分之一。拿 2070 去乘你的模型 FLOPs 会得出一个乐观到荒谬的频率。

三，**TOPS 与 TFLOPS 不可比**。Orin NX 的“100 TOPS”里的 P 是 Integer（INT8），Thor 的“2070 TFLOPS”里的 F 是浮点 FP4。不同位宽、不同稀疏度、不同数据类型的三个数被印在同一张对比表上，是这个行业最常见的误导。**本书不把它们并排比较**。

上层到底要多少赫兹

先把它要的东西说成一个数，而不是一个愿望。宇树公开的 G1 强化学习配置里，仿真步长 0.005 s、decimation 4 ⇒ 策略输出周期 = 0.005 × 4 = 0.02 s，也就是 **50 Hz** [厂]（`unitree_rl_lab` 配置，第 21 章）；`rt/lowcmd` 那条低层通道是 500 Hz [厂]（第 28 章）；再往上那个负责“看懂场景、决定干什么”的语义层，各家公开的节奏都在个位数赫兹（第 30 章）。

所以这台机器的时间轴上有三个数量级：500 Hz 的指令刷新、50 Hz 的动作策略、5–10 Hz 的任务决策。大脑要的只是最慢那一档：5 Hz，每 200 ms 交出一次结果。那为什么最慢的一档反而最难满足？因为要读的那堆参数，体量是另外两档的十万倍——关节控制器的全部数学装在一条公式里（第 12 章那个五元组），而一个 6B 模型有 60 亿个需要读的数。

现在算真正卡住的那件事

第 00 章给过一个两行的推导，这里补全并说明它的假设。

第一问：大模型为什么慢？是因为它算得多吗？

不是。一个自回归/扩散式的模型每生成一次输出，**至少要把它的全部权重从显存里读一遍**。这不是算法选择，是硬件事实：权重存在 DRAM 里，算之前必须搬到片上。

第二问：那它为什么不能把权重放在片上，非要从显存读？

因为放不下。Orin NX 官方给的片上缓存是 2 MB L2 + 4 MB L3，合计 6 MB [查]（NVIDIA 自己的文档），而 6B 模型 FP16 的权重是 12 GB：

12 GB ÷ 6 MB = 2000 倍

片上放不下的东西，每一步就都得从 DRAM 取一遍。而取，是要花时间的。

```
时间下限 = 权重字节数 ÷ 内存带宽
6B 参数 × 2 字节（FP16/BF16） = 12 GB
```

分母是多少？**这是本章唯一必须承认算不下去的地方**。宇树不公布模组的内存带宽 [未]；本书能核实到的唯一相关一行是 H1 板载 Intel i5/i7 那两台的 “LPDDR5 5200 MT/s (dual-channel)” [厂]——注意那是 x86 平台的内存口径，**不是 Orin 模组的**，本书把它当成“官方唯一给过的一个存储器量级参照”，而不是当成答案。若按 64 位总线推：

```
5.2e9 MT/s × 64 bit ÷ 8 = 41.6 GB/s      [算]（位宽是本书假设）
```

拿它做一张表（**这张表的作用是让你看见敏感性，不是给你答案**）：

内存带宽	6B/FP16 单步下限	上限频率
41.6 GB/s（按上面那条推）	288 ms	3.5 Hz
100 GB/s	120 ms	8.3 Hz
200 GB/s	60 ms	16.7 Hz
273 GB/s	44 ms	22.7 Hz

（[算]。）把中间那行单独写出来，因为它是本章的主算式：

12 GB ÷ 100 GB/s = 0.12 s = 120 ms ⇒ 上限 1 ÷ 0.12 s = 8.3 Hz

第三问：那算力呢？算这些东西不是也要时间吗？

要，但短得多。一个 6B 模型每生成一步，浮点运算量约是 2 × 参数量 = 2 × 6e9 = 1.2e10 次（这是把每个参数乘加一次的通行估法 [行]）。拿 Orin NX 标称的 100 TOPS 去除，本书故意把口径搞混、把 TOPS 当成 TFLOPS 用（前文说过这不可比——这里偏袒显卡，让结论更难被推翻）：

1.2e10 ÷ 1e14 = 0.00012 s = 0.12 ms

而读权重要 120 ms。120 ms ÷ 0.12 ms = 1000 倍。

换句话说，**这块芯片 99.9% 的时间在等内存，不是在算**。工程师管这个叫算术强度不够：这台机器每搬 1 字节能做 100 TOPS ÷ 100 GB/s = 1000 次浮点，而自回归解码每 2 字节权重只给你 2 次浮点，即 1 次/字节。需求与供给差了三个数量级，所以无论你把核数堆到多少，节奏都被那条内存总线钉住。

结论的稳健性：不管取哪一档，一个 6B 参数的模型在嵌入式模组上的节奏都是**个位数到二十几赫兹**。这与第 00 章的结论一致，也与行业上“低频语义规划 + 高频动作跟随”的双系统架构一致（第 30 章）。

所以“大脑天生 5–10 Hz”这个判断不依赖任何一家厂商的参数，它依赖的是“权重必须被读一遍”这条硬件事实。这是本书最喜欢的那类结论：输入不确定，但结论对输入不敏感。

那把精度降到 FP8、INT4，为什么就救得回来

因为这条墙上挂的是字节数，不是计算量。位宽减半，要读的字节就减半；带宽没变，时间就减半。参数量、层数、注意力机制、提示词——它们都不出现在上面那条除法里，所以它们也救不了你。

```
FP8 / INT8：6B × 1 字节 = 6 GB ⇒ 6 GB ÷ 100 GB/s = 60 ms ⇒ 16.7 Hz
INT4：      6B × 0.5 字节 = 3 GB ⇒ 3 GB ÷ 100 GB/s = 30 ms ⇒ 33.3 Hz
```

（[算]。）于是“把模型从 6B 砍到 3B”与“把权重从 16 bit 压到 8 bit”在这条除法里是**同一件事**——分子都减半，结果都是 6 GB。但前者要重训，后者只要量化。**这就是端侧先动位宽、不动规模的真实理由**。16 bit 与 4 bit 之间则是四倍速度——**在带宽受限的工作负载上，位宽就是帧率**。

那为什么不直接全用 INT4？代价在这里，而且是三条真实的代价：

一，量化误差有下限，而且它不是“小一点”而是“换个形状”。把一个连续分布压到 b 位整数，就是把数轴切成格。格宽 = 满量程 ÷ (2^b − 1)：INT8 是 255 个格，INT4 只有 15 个。权重的绝对值如果落在格宽的一半以内，它就被舍成 0；如果它本来在两个格中间，它就被挪到其中一个上。模型对这件事不是完全敏感的——大部分权重被挪一点没关系，但总有几层、几个通道里的少数大值权重（行业里叫离群值）一挪就崩。所以量化不是压缩，是**用误差换字节**，而换回来的误差要在任务上重新测。

二，它需要校准数据。格宽是拿什么定的？得先知道这批权重的分布范围。业界的标准做法是拿一小份数据跑一遍前向、统计每层的激活范围，再定标度因子（AWQ、SmoothQuant 这类方法都是围绕这件事做的）。**这意味着量化是有训练数据依赖的后处理步骤，不是一个编译选项** [行]。宇树没有公布 UnifoLM-WLA-1.0（README 自陈 6B 参数，第 32 章）在机内跑的是什么位宽、有没有做过量化 [未]。

三，省下来的字节会被 KV 缓存和图像吃回去一部分。12 GB 权重读一遍是 120 ms；上下文里的 KV 缓存每步也要读，多路相机的视觉 token 也在算。所以 FP8 之后节奏不是整齐的 16.7 Hz，而是“比 FP16 快不到两倍”。

但方向是确定的：带宽受限 ⇒ 减字节是唯一有效的杠杆 ⇒ 所以这个行业的端侧模型先降的不是规模而是精度。这也解释了为什么 H2 那档要选 Thor 而不是继续用 Orin NX——不是因为 2070 TFLOPS，而是因为 128 GB 的显存与更宽的总线（第 43 章附录列了这颗模组的厂商口径）。**买的是带宽，宣传的是算力**，这一句你可以拿去核对任何一份发布材料。

而真正的第一道墙是功耗

现在把大脑放回整机里。

第 20 章算过：G1 平均电功率约 216 W（三个一手数相除）；**H1、H2 本书不给瓦数，官方没有续航小时数** [算]。Thor T5000 的官方功耗区间是 40–130 W [查]、同一张表里 Orin NX 16GB 是 10–25 W [查]（都是 NVIDIA 自己的规格页，本书第一次复验时回读；按图例，[厂] 只留给宇树官方）。

```
G1 平均 216 W，其中 Thor 若跑在 130 W ⇒ 大脑占 130 ÷ 216 = 60%
H2 电池 0.972 kWh，续航 3 h
若把 Thor 从 40 W 调到 130 W：多耗 90 W ⇒ G1 口径 432 ÷ (216 + 90) = 1.41 h ⇒ 两小时缩到约 1.4 小时  [算]
```

这两次除法讲的是两件事，把它们分开是本章的一个要点：

第一，能量账。 电池 972 Wh ÷ 130 W = 7.5 h——一块 Thor 单独把整块电池吃完要 7.5 小时。整机只活 3 h，是因为关节同时也在吃功率。把 Thor 从 130 W 降到 40 W，省下的 90 W × 3 h = 270 Wh，占电池的 270 ÷ 972 = 28%。**所以“大脑费不费电”在能量层面是一个 30% 量级的问题，它能把 2.3 h 换回 3 h，仅此而已。**

第二，功率与热的账，这才是硬上限。 216 W 是 G1 的平均值；宇树对 H2 唯一公布的接口口径是“对外供电口总功率（不含直连电池）不超过 500 W” [厂]，而 130 W 的大脑就占了这 500 W 的 26% [算]。而第 20 章已经说明，站着不动也要出扭矩，那部分功率省不掉。**于是问题从“电池够不够”变成“壳里放不放得下 130 瓦的热”。**

那热又靠什么定？ 靠一条串联热阻链（第 36 章把它拆开算）：温升 = 功率 × 热阻，即 $\Delta T = P \times R$。链条是 die→壳体→环境，每一段都有自己的热阻。宇树没有公布 H2 躯干的热阻，也没有公布大脑模组贴在多大面积的结构件上 [未]——**所以本书算不出“130 W 会让壳温升到多少度”，只能算出它对功率与能量的占比。**这是本章第二个必须认输的地方。

能确定的只有一件事：**有风，就要有通路**。G1 与 H2 的官方规格表在散热那一行都写着“Local air cooling”（局部风冷）[厂]——机体里是有风的，而风要进和出。通路就撞上防尘、防撞、噪声与外观：**宇树对人形整机没有公布任何 IP 防护等级**（官方页面与文档中心逐页检索为 0 命中，已发布的相关认证只有 L1/L2 雷达的 Class 1 激光人眼安全）[未]，也没有公布风道、风量与噪声口径 [未]。而第 13 章说过，关节里的电机与驱动板也在往同一台壳里放热。**大脑与关节在抢同一份散热预算**。这是第 36 章的主线之一。

于是有一个非常具体的设计后果：**留给大脑的功耗预算通常在 15–60 W 量级** [行]（行业通行判断，非宇树公布），而这块预算还要同时供相机、雷达、通信。本章那张表里 41.6 GB/s 那一档之所以悲观，恰恰因为它对应的可能是一个被限在 40 W 的模组——**而功率一降，内存控制器也跟着降频，带宽这堵墙会变得更矮**。这就是三堵墙互相咬合的地方：功耗决定频率，频率决定带宽，带宽决定大脑的节奏。

一个可以立刻用的判断法

以后看到任何“某机器人搭载 XXX TOPS 芯片”的宣传，做三步：

第一步，问数据类型。TOPS（INT8）？TFLOPS（FP16/FP8/FP4）？稠密还是稀疏？三个问题问完，标称数字通常会缩水 5–20 倍。

第二步，问内存带宽，不要问算力。对自回归/扩散模型，带宽决定节奏，算力只决定“带宽给的那点时间里能不能算完”。**绝大多数端侧大模型是带宽受限，不是算力受限**。判据就是本章那个比值：算力 ÷ 带宽 = 这台机器每字节能做多少次浮点，把它和你的模型每字节实际要做多少次比较。

第三步，问功耗与散热方式。一个 130 W 的模组装在一台风道、外壳、关节发热都要抢的躯干里，它多半跑不到 130 W——它会降频。而降频直接把第二步的频率打折。

这三步合起来，就是“这台机器的大脑实际能多快”的完整答案。 而它不需要任何厂商没公布的数据——它只需要你把三个问题问出来。

类比，以及它在哪儿破

类比：大脑像一辆车的油箱与发动机——TOPS 是马力，内存带宽是油管粗细，功耗是油耗。你可以换一台更大的发动机，但如果油管就那么粗，马力使不出来。

这个类比在哪里破：车的马力与油耗是独立参数，而芯片的算力与功耗**不独立**——提高算力意味着提高频率与电压，而动态功耗 $\propto C \cdot V^2 \cdot f$，**电压的平方**。所以“算力翻倍”在硅上从来不是翻倍功耗，而是接近翻两到三倍。这就是为什么厂商宁可堆核心数（每核低频低压）也不冲高频，也是为什么“2070 TFLOPS”是靠 FP4 + 稀疏这两个“降低每比特能耗”的技巧换来的。

换句话说：那 2070 里，有一部分是用“少算几位、跳过一些计算”换来的。 它不是虚标，但它是**有条件的上限**——和第 08 章那两个不可同时达到的角点，是同一类数字。

一个可检验的说法：一个 6B 参数的模型在端侧的推理节奏由内存带宽而非算力决定——单步时间下限 = 权重字节 ÷ 带宽，在 41.6–273 GB/s 的合理区间内为 44–288 ms，即上限 3.5–22.7 Hz；同一时间预算里算力只用掉 0.12 ms（120 ÷ 0.12 = 1000 倍），所以这是带宽受限而非算力受限；把权重从 FP16 降到 FP8/INT4 能把节奏提到 16.7/33.3 Hz，代价是量化误差与校准数据依赖；而 Thor T5000 的 40–130 W 取上限时占 G1 平均电功率（216 W，三个一手数相除）的 60%，把官方那 2 小时续航压到约 1.4 小时——**大脑的预算与关节的预算是同一份**。

证据等级：PC2 全部参数（含 2 MB L2 + 4 MB L3）、“such as Orin, etc.” 原文、H2 可选 Thor/Orin NX、2070 TFLOPS(FP4-Sparse)、100 TOPS、10–25 W、128 GB、“31 DoF, 360 N·m”、H1 的 i5/i7 与最多 3 块 Orin NX、LPDDR5 5200 MT/s dual-channel（H1 x86 平台口

径，非 Orin）、对外供电口总功率 ≤500 W、“Local air cooling”[厂]；UnifoLM-WLA-1.0 的 6B 为 README 自陈 [厂]；41.6 GB/s 是 [算]（**总线位宽是本书假设，官方未写**）；12 GB、288/120/60/44 ms、6/3 GB、60/30 ms、16.7/33.3 Hz、0.12 ms、1000 倍、60%、28%、26%、1.4 h、7.5 h 是 [算]；40–130 W 与“FP4-Sparse”口径 [厂][查]；15–60 W 大脑预算是行业判断 [行]；每参数量 2 次浮点、动态功耗 $\propto CV^2f$、量化格宽与校准流程 [行]；**宇树未公布机内模型的实际推理频率、内存带宽、位宽与是否量化、各算力档位的实测功耗、躯干热阻与风道、以及人形整机的 IP 防护等级（官方页面与文档中心逐页检索 0 命中）** [未]。

可核对清单：

- ☐ 到宇树 H2 页面找到计算模块那一行，抄下它给的型号与算力口径（注意有没有写 FP4/Sparse）
- ☐ 到 G1 EDU 页面找“标配”两个字——确认它没有把 16GB 写成标配（这是本书的一条断言）
- ☐ 拿任何一份端侧 LLM 的公开基准，看它的 tokens/s 与“权重字节 ÷ 带宽”的下限差几倍（差 1–3 倍说明实现接近带宽受限，差 10 倍说明还有优化空间）
- ☐ 手算一次：如果你的模型是 3B 参数、FP8（每参数 1 字节）、带宽 100 GB/s，上限频率是多少（答：3 GB ÷ 100 GB/s = 0.03 s = 30 ms ⇒ 1 ÷ 0.03 s = 33 Hz）
- ☐ 跑 `code/tools/foc_check.py` 的“26 6B 模型带宽”，确认 12 GB / 120 ms / 8.3 Hz 这三个数
- ☐ 找一份 H2 的对外供电表，看“总功率不超过 500 W”这一行在不在——它在，就是大脑功率预算的一手边界

下一章：本章把大脑钉在 5–10 Hz 上。下一章去看那条把 500 赫兹的关节与 5 赫兹的大脑缝起来的总线——它在电平层只有 ±200 mV，而宇树在 2026 年还在用它。

第 27 章

三条总线：RS-485 4 Mbps、CAN-FD 5 Mbps、EtherCAT 1 kHz

如果你去问任何一个做工业机械臂的工程师“关节总线该用什么”，答案几乎一定是 **EtherCAT**：100 Mbps 物理层、微秒级同步、工业验证二十年。

而宇树官方文档里，所有电机走的都是 **RS-485**：GO-M8010-6 是 4 Mbps、一条总线最多 15 台 [厂]；IM6014 是 6/4 Mbps、最大总线轮询 10000 Hz [厂]。官方还明确过一句：**目前生产的所有电机均用 RS-485，要用 CAN 需要官方转换模块** [厂]。

RS-485 是 1977 年的标准。它比 CAN 慢、没有仲裁、没有错误帧管理、没有对象字典。

所以要么宇树不懂工业标准，要么我们在漏掉什么。 只算带宽回答不了这个问题——带宽告诉你装得下多少，不告诉你凭什么装得下。**这一章往下追到电平与时序**：一根线上那两片铜皮之间差多少伏才算一个 1？位与位之间有多少纳秒可以用来把线充电？把这些算完，4 Mbps、15 台、6000 Hz 这三个官方数字之间的关系，以及“为什么选 485”，就都自己浮出来了。

在王兴兴的期待中，未来人形机器人的形态，在主控大脑和四肢之间，只需用一根线缆进行连接，为了达到这一目标，仍有很多值得挖掘的工作。

王兴兴 — 21世纪经济报道（2025 骁龙峰会·中国嘉宾对话报道），天天基金转载 — 2025-09-24（记者转述（全句无引号、由记者以“在王兴兴的期待中”引出；原场为圆桌对话，无逐字稿可核））

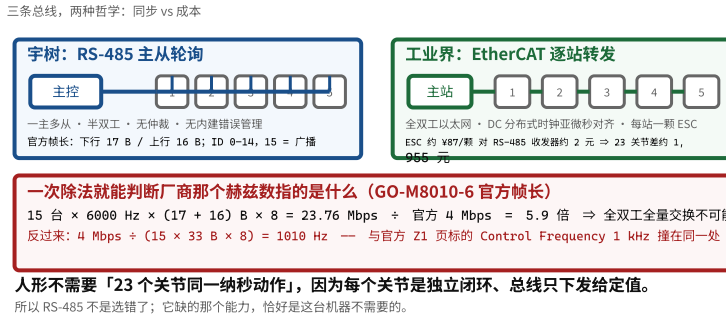


图 27-1　三条总线的拓扑与一帧的字节布局（17/16 B 与 20/26 B 并排）

先把这条线看成电压，而不是看成“通信”

RS-485 的全部信息都写在两根线的**压差**上：A 减 B。接收器只认这个差：

```
V_diff = V_A − V_B
V_diff > +200 mV ⇒ 判为一个逻辑态
V_diff < −200 mV ⇒ 判为另一个逻辑态
−200 ~ +200 mV 之间 ⇒ 不确定区，标准不保证判决        [行]
```

这是 TIA/EIA-485 电气规范里的接收门限 [行]。为什么偏偏用差分？**因为机器人身上最不缺的是共模噪声**：每个关节都在各自开关大电流，母线的开关沿会在两线上同时感应出几伏的共模位移。差分接收器把它减掉——它只在意 A 与 B 的差，不在意两线一起抬到哪里；标准的共模输入范围是 −7 到 +12 V [行]，**两线一起漂几伏不影响判决**。单端 TTL 串口做不到这一点，这就是为什么板内用 UART、板间必须用 485 或 CAN。

那这个 ±200 mV 又靠什么保证？ 靠发送端把压差驱动过阈值，再靠**时间**：驱动器翻转之后，线上的电压不是立刻到位的——线缆是一段分布 RC，也是一条传输线。于是本章真正的主角登场：**位时间**。

位时间与线上延迟：同一张账

```
位时间 T_bit = 1 ÷ 波特率 = 1 ÷ 4 Mbps = 250 ns        [算]
```

一位只有 250 纳秒。现在把它和电信号在铜线里的跑速放在一起：信号在双绞线里的传播速度约 2×10^8 m/s，也就是 **5 ns/m** [行]（光速的三分之二）。于是：

```
10 m 线缆的延迟 = 10 × 5 = 50 ns = 250 ns 的 20%       [算]
```

这就是本节的全部结论：10 米线上那 50 纳秒，还不到一个位时间。 判决电路只需要在位中间采样一次，50 ns 的到达差没有把一个位推到隔壁位里去，所以 485 在这段距离上能跑满 4 Mbps。宇树没有公布它的 485 走线长度 [未]，但一体化关节是逐关节串过去的、单段以厘米到分米计（第 03 章那条“走线从中心孔穿过”），在这个量级上传播延迟根本排不进误差预算。

那线再长为什么必须降速？降的到底是什么？ 不是“信号跑不动”，是位时间不够用了。两个失效模式，都很硬：

一、反射。 线缆是特性阻抗 $Z_0 \approx 100$–$120\ \Omega$ 的传输线，末端不匹配时能量按反射系数弹回来：

```
Γ = (Z_L − Z₀) ÷ (Z_L + Z₀)
末端开路：Z_L → ∞ ⇒ Γ → +1（全反射，电压加倍）
末端接 120 Ω：Z_L = Z₀ ⇒ Γ = 0（能量被吃掉，不回来）
```

这就是终端电阻 120 Ω 的全部理由，而宇树在 IM6014 协议页上明确要求“120 Ω，电缆两端各并一个” [厂]。弹回来的电压叠在原波形上，会把某一位的压差推回 −200 ~ +200 mV 那个不确定区。关键是**它什么时候回来**：往返一圈 = 2 × 5 ns/m × 线长。只要它小于一个位时间，回波落在本位里，判决不受影响；一旦超过，它就砸在下一位上。用这条反推 4 Mbps 下“端不端接都安全”的距离：**250 ÷ (2 × 5) = 25 米** [算]——超

过 25 米，第一次回波就跨位了，所以要么端接（消掉反射），要么降速（把位时间拉长到回波之上）。行业里“485 能拉 1200 米但只有 100 kbps”和“短线上能跑 10 Mbps”，是**同一条除法的两端**：速率 × 距离大致是常数。

二、边沿与信道带宽。4 Mbps 方波的基频是速率的一半 = 2 MHz，而方波的形状要靠 3、5 次谐波撑住，即需要 10 MHz 量级的可用带宽 [行]。线缆是低通，越长高频衰减越狠，于是边沿变缓、眼图（把各位波形叠在一起看的那个图）闭眼。**失真的是波形形状，不是丢包**：判决在噪声与码间串扰里赌错一位，硬件不会告诉你。

CAN 与 EtherCAT 放在同一张账上，差别立刻显形：

```
CAN-FD 数据段 5 Mbps ⇒ T_bit = 1 ÷ 5e6 = 200 ns        [算]
EtherCAT 100 Mbps    ⇒ T_bit = 1 ÷ 1e8 = 10 ns，比 485 紧 25 倍  [算]
```

CAN 的位时间必须容纳往返传播，因为**仲裁规则要求发送端在同一位里读回线上真实电平**：它发出的显性位若读回隐性，说明有别人在发，它立刻退出。这条规则把 CAN 的距离-速率锁死，还要留出收发器的中继余量——**这就是 CAN-FD 的线都很短的原因**。EtherCAT 的一位只有 10 纳秒，**它当然不靠这个约束**：10 ns 的位时间在一对共享的多分支线上活不下来（回波、分支 stub、边沿劣化都会在几十纳秒内回来），所以它干脆取消共享介质——每个节点是一跳，收发各用一对线，每一站重新恢复时钟、重新驱动，把累积的边沿劣化在每一站清零。代价是拓扑必须手拉手，而每一跳都要一颗会“再生信号”的芯片。**这才是 EtherCAT 快的真正原因，而不是“物理层更高级”**。

半双工要换向：那个换向时间从哪来

485 通常是半双工：同一对线，一会儿主站发，一会儿从站回。收发器有驱动使能 DE 与接收使能 RE 两个脚，**同一时刻只有一端在驱动**。这个“换手”要花真实的时间，它来自四处：

一、移位寄存器的尾位。 MCU 的 UART 把字节交给外设之后，最后一位还在线上一位一位移出去。软件如果把 DE 拉早一个位，最后一位就被截掉——所以要么等“发送完成”标志，要么固定延时一位。这一项的下界就是 **250 ns @4 Mbps**。

二、驱动器的关断与再打开：从“正在驱动”到“高阻”有几纳秒到几十纳秒（器件口径 [行]），而这段时间里总线没有确定电平。

三、线电容的充放电。 换向之后线要充到新电平，路径阻抗是两端 120 Ω 并联 = **60 Ω**，与线缆分布电容（每米几十 pF 量级 [行]）构成 RC。**挂的节点越多、线越长，电容越大，能跑的速率就越低**——这条时间常数就是“波特率不能只看收发器手册”的原因。

四、软件与协议栈的调度。 主站必须知道从站答完了没有。宇树官方把这件事交给了地址空间：电机 ID 0~14，**15 是广播且无响应** [厂]——广播帧不要求回话，所以不需要换向。这个字段本身就说明厂商知道换向的代价：**要同时命令 15 台而不听任何人说话时，用广播把 N 次换向压成一次**。

换向的账单可以算，用官方帧长（17 B 下行 / 16 B 上行）与官方串口格式 8N1（1 起始位 + 8 数据位 + 1 停止位 = **每字节 10 bit**）[厂]：

```
控制帧上线 = 17 × 10 × 250 ns = 42.5 µs
状态帧上线 = 16 × 10 × 250 ns = 40.0 µs
一问一答一轮 = 42.5 + 40.0 = 82.5 µs（还没算上面那四项）  [算]
```

每一台、每一轮、来回一次，物理上就要 82.5 微秒，15 台轮一轮 = 15 × 82.5 µs = **1.24 ms** ⇒ 反过来 1 ÷ 1.24 ms = **808 Hz** [算]。而 6000 Hz 要多少时间？15 × 6000 × 82.5 µs = **7.4 秒的上线工作压在 1 秒里** [算]——不可能。

三条总线的物理差别

	RS-485	CAN-FD	EtherCAT
物理层	半双工差分，主从	半双工差分，多主仲裁	全双工以太网 PHY
典型速率	0.1–10 Mbps	5–8 Mbps 数据段	100 Mbps
位时间	250 ns @4 Mbps	200 ns @5 Mbps	10 ns @100 Mbps
判决依据	差分压差 ±200 mV [行]	差分 + 显性/隐性线与 [行]	以太网 PHY 编码
拓扑	一主多从，总线型	多主，总线型	手拉手线性/星型，逐站转发
同步机制	无（靠主站轮询节奏）	无（靠时间戳）	**DC 分布式时钟，亚微秒对齐**
节点数上限	规范 32 收发器，实际靠地址	靠 ID，典型几十到上百	理论 65535，实际几百
每节点成本	一颗 2 元收发器	收发器 + 控制器	一颗 ESC 芯片（LAN9252 约 ¥87 [查]）
生态	无穷（谁都有串口）	汽车级、极广	工业，需主站软件授权

这张表里最要紧的一行是“同步机制”。EtherCAT 的 DC 让所有从站的执行时刻对齐到亚微秒，它对多轴联动插补（一条轨迹由 6 个轴同时执行）是必需的。而它凭什么做到？不靠带宽，靠**每一站硬件打时间戳**：帧经过 ESC 时硬件记下进与出两刻，用差值补偿链路传递延迟（透明时钟 [行]），于是所有节点的计数器落到同一条时间轴上。**这套机制买的是“同时”，花的是每节点一颗专用芯片和一条必须手拉手的拓扑**。

而人形机器人**不需要多轴插补**。它的每个关节是独立的闭环（第 12 章），关节之间唯一的耦合发生在 500 Hz 那一层的控制器输出上。**换句话说：人形的 23 个关节不需要同时动作，只需要各自快速响应**。

这就是 RS-485 能用的第一层原因：它缺的那个能力，恰好是这台机器不需要的。

用官方自己的数字，算出一处对不齐的地方

第 12 章算过一遍，这里完整重做，因为它值得单独看。宇树对同一颗 GO-M8010-6 公布了三个数：**一条总线最多 15 台、通信控制频率 6000 Hz、RS-485 @4 Mbps**，又公布了帧格式：**控制帧 17 字节、状态帧 16 字节** [厂]。把它们乘起来（本书脚本 `foc_check.py` 的 `bus_budget()`，按 8 bit/字节口径）：

```
下行 = 15 × 6000 × 17 × 8 = 12.24 Mbps
上行 = 15 × 6000 × 16 × 8 = 11.52 Mbps
合计 = 23.76 Mbps ÷ 官方 4 Mbps = 5.9 倍            [算]
反过来：4 Mbps ÷ (15 × 33 × 8) = 1010 Hz           [算]
```

再把官方 8N1 的起止位扣进去（每字节 10 bit），同一道除法变成 **4 Mbps ÷ (15 × 33 × 10) = 808 Hz**，而需求侧变成 15 × 6000 × 33 × 10 = 29.7 Mbps，即 **7.4 倍** [算]。**两个口径都落在 1 kHz 附近，而“6000 Hz 挂 15 台”这个读法需要 5.9–7.4 倍带宽**。

结论只能写成一句克制的话：官方的“15 台”与“6000 Hz”不能同时以“每台每秒 6000 次全量指令 + 状态交换”的读法成立。能同时成立的读法只有几种：它是单台、或被 ID 15 广播帧摊薄后的**指令下发刷新率**；或者实际帧比 17/16 字节更短（协议页未穷尽 [未]）；或者一条物理 485 上实际挂的台数少于 15。**宇树没有定义“通信控制频率”这个词** [未]，所以本书不下“官方标错”的判断，两条口径并列留在书里，一个都不删。

同一套算术用在 IM6014 上（20 B 下行 / 26 B 上行、6 Mbps、标称轮询 10000 Hz）：15 台全双工需要 55.2 Mbps，是 9.2 倍；反过来 6 Mbps 最多支撑 1087 Hz [算]——方向完全一致。而 1087 Hz 与 1010 Hz 这两个数，撞上了宇树自己在 Z1 机械臂产品页上标的 Control Frequency 1 kHz [厂]：两条来自不同页面、不同年代的官方数字，被一次除法接进了同一个数量级。

所以第 12 章那句话在这里成立第二次：通信频率 ≠ 环路频率。 总线只决定你多快能改关节的给定值；关节自己的电流环在它本地的 MCU 里跑（开源参照里 ODrive 是恒定 125 µs 即 8 kHz、moteus 标称 15–30 kHz [查]，宇树未公布自己那一环 [未]）。

第二层原因：分层之后，带宽算术站在它这边

现在把三条总线放在同一个需求下比：**G1 的 23 个关节（官方口径 [厂]）、每个每毫秒收到一次新给定、每次上报状态**，帧长沿用官方 M8010 的 17 B / 16 B。

```
全身一干：23 × 33 B × 1000 Hz × 8  = 6.07 Mbps
扣起止位：23 × 33 B × 1000 Hz × 10 = 7.59 Mbps     [算]
```

6.07 Mbps 对 4 Mbps 的 RS-485 差 1.5 倍，对 IM6014 的 6 Mbps 也差一点；而 EtherCAT 的 100 Mbps 装得下它，还有大量余量。**所以“分层”不是为了追上哪条总线，而是为了让 4 Mbps 那条干够用**——同时也因为地址不够：官方给的 ID 是 0~14 加 15 广播，一条 485 最多挂 15 台，而 G1 有 23 个、H2 有 31 个关节（官方口径 [厂]），**装不进 15 个地址**，23 ÷ 15 = 1.5，向上取整就是 2 条干。注意这条推理用的是 M8010 文档公布的地址空间；**G1/H2 那颗电机是不是同一套寻址、每条干实际挂几台，官方没有公布** [未]。

一旦分层，每条干上的账立刻松下来：

```
8 台 × 33 B × 1000 Hz × 8 = 2.11 Mbps      (4 Mbps 够，余量 1.9 倍)
8 台 × 33 B × 500 Hz × 8  = 1.06 Mbps      (4 Mbps 宽裕)
15 台 × 33 B × 1000 Hz × 8 = 3.96 Mbps     (正好卡在 4 Mbps 上)
```

（[算]，帧长与波特率均为官方口径。）

读第三行：15 台满配、1 kHz 全双工，恰好把 4 Mbps 用到 99%。 这把上面那个“反过来除只有 1010 Hz”钉死在同一处——**4 Mbps 不是不能跑 1 kHz，是只能带 15 台跑到 1 kHz，一台的余量都没有。** 逼着工程团

队分层的，是地址上限 15 与带宽上限 4 Mbps 这两件事，而它们恰好同时见顶。

这就是答案的形状：RS-485 不是“够用”，是在“分层 + 每干不超 15 台 + 关节内闭环 + 给定值不需要 kHz”这四个前提下刚好够用。 而宇树把这几条前提全做了——第 12 章的五元组指令（给定值下发，闭环在本地）、500 Hz 话题、50 Hz 策略。

（**必须诚实标注**：宇树从未公布 G1/H1/H2 的关节总线拓扑——几条干、每干几台、走什么芯片 [未]。上面是“在官方公布的速率与帧长下，什么架构能自洽”的推导，不是它的实现。而第三方拆解的口径是“CAN 高低线菊花链”，与官方“全部电机 RS-485”冲突，本书两条都列。）

第三层原因：钱，而且是可算的钱

RS-485 的器件成本优势是可以算的。

```
RS-485 收发器：约 1–3 元人民币                    [行]
CAN 收发器：约 2–5 元；带独立 CAN 控制器更贵
EtherCAT 从站控制器 ESC（LAN9252）：约 ¥87        [查]
```

一台 23 关节的人形，每关节一颗：

```
RS-485：23 × 2 = 46 元
ESC：   23 × 87 = 2,001 元
```

差约 1,955 元。 而第 03 章算过一台 G1 的器件成本约四万元量级 [拆]——所以**光是总线接口就占整机成本约 5%。**

再叠上第二项：EtherCAT 主站需要实时内核（PREEMPT_RT）或专用主站卡，前者要维护内核补丁、后者要再花钱；而 RS-485 主站就是一颗 3 块钱的 UART + 一颗收发器，**任何一颗 MCU 都有。**

这就是“垂直整合”在总线层的含义：宇树自研电机、自研驱动板、自研关节，那么它顺手把总线也选成最便宜的那一条，是完全自洽的。反过来，一家买现成工业关节的厂商必须接受 EtherCAT，因为那是关节厂给

的。

所以总线选型不是技术判断，是“你自己做不做关节”的判断。 这一条对读者的实际意义：如果你要自己搭一颗关节，RS-485 + 自制协议是成本下界；如果你要接现成工业伺服，你没得选。

那 RS-485 的代价是什么

公平起见，把它的坑列全。这些坑都是真实的，而且会由你（工程师）来付：

一、没有多主仲裁。 RS-485 是主从轮询。任何一个“从站主动上报”都需要协议层设计（时隙、地址轮转）。出了总线冲突就是数据损坏，硬件不会帮你。**CAN 在这一层是真本事**：所有节点接在同一条“线与”上，谁发显性位谁赢，输的一方自动退避重发，不损坏数据；EtherCAT 干脆不做仲裁，用交换式点对点绕开它。

二、没有内建错误管理。 CAN 有 CRC、帧格式检查、错误计数器、自动重发。RS-485 只是电气标准——**宇树因此自己在帧里加了 CRC16_CCITT（控制帧覆盖前 15 字节、状态帧覆盖前 14 字节）[厂]**，而 IM6014 换成了 CRC32-MPEG2 [厂]。**这两条差异本身就说明：错误检测要你自己做，而且做几遍由你决定。**

三、没有标准对象字典。 CANopen/CiA402 定义了“怎么读一个伺服的位置”，跨厂商通用。RS-485 上跑的是**厂商私有帧格式**——所以宇树的电机只能配宇树的板子，你的板子接不了它的电机（这也是它把电机单卖却仍要求你走它的 SDK 的原因）。

四、拓扑脆弱。 总线型布线在整机里意味着一长串菊花链线缆，任何一处接触不良全链失效。而人形关节是在**转动的**——线缆要过中心孔（第 03 章）。**这是齿隙之外人形关节的第二大可靠性来源。**

五、没有亚微秒同步。 如果你的应用需要 23 个关节在同一纳秒执行（比如做高动态的空翻），RS-485 的轮询节奏给不了这个保证。此时要么分层加本地缓冲，要么换 EtherCAT。**智元公开的架构正是后者：EtherCAT**

1 kHz 主站转三条 CAN-FD 5 Mbps 从站 [查]——用高速干保证同步、用 CAN-FD 保证末端可靠。

六、换向税。 每收一次状态就要换手一次，前面算过它至少一个位时间起步，还叠上器件关断与线电容。轮询式半双工的周期里，真正的“有效通信时间”是打折的。**这条税是半双工结构性的，写不出更好的协议来免掉它**。

这两条路线谁对？ 取决于你把成本与可靠性放在哪一边。本书不裁决；本书给的是判据：**你需要多轴同时执行吗？需要跨厂商互换吗？愿意为每关节多付 85 元吗？你的位时间容得下你的线长吗？** 四个答案会把你送到不同的总线上。

如果你真要用 EtherCAT，链条长什么样

前面几节解释了人形为什么**没**用 EtherCAT。但“关节层全是 MCU + FOC + 现场总线”这句话 对整行行业成立，而工业侧那条总线就是 EtherCAT——所以这一节把它的器件链走完， 让你手里那颗 MCU 知道自己在链条的哪一格。

第一格：主站。 一台跑 Linux 的工控机，软件用 IgH（`linux-ethercat`）或 SOEM 这类开源主站， 外加一张普通网卡。EtherCAT 不要求专用网卡，这是它能便宜的原因： 它用的是标准以太网帧，只是把里面的载荷重新定义了。

第二格：帧飞过每一台，“飞过即改”。 官方技术组（ETG）的原话是：每个子设备“reads the data addressed to it ‘on the fly’, and inserts its data in the frame as the frame is moving downstream”，而帧“is delayed only by hardware propagation delay times” [行]。 这句话的分量要和**轮询**对比才看得出来。RS-485 是一问一答：主站发一条、等回答、再发下一条， 所以 N 台设备的耗时是

N 倍单帧时间。EtherCAT 是一帧带齐所有设备的数据，**飞过就改**，耗时只有一帧的时间。把这条差别算成数（`foc_check.py cycle_time_485_vs_ecat()`，[算]）：

```
4 Mbps ⇒ 位时间 250 ns；一帧 33 B（指令 17 + 状态 16）= 264 bit ⇒ 66 µs/台
15 台轮询一轮 = 990 µs    20 台轮询一轮 = 1320 µs    （1 kHz 环的预算是 1000 µs）
100 Mbps ⇒ 位时间 10 ns；20 轴过程数据 1680 B = 13,440 bit ⇒ 一帧 134.4 µs
```

这里顺带解掉一个官方数字。宇树文档写一条 RS-485 总线最多挂 15 台[厂]。本书前面给的理由是地址：ID 0–15，15 留给广播，可用 15 个。现在看第二行算式：**就算地址够用，20 台也塞不进 1 kHz（1320 µs > 1000 µs），而 15 台刚好是 990 µs。**两个理由各自独立成立，指向同一个数——这种“两条不相干的约束算出同一个上限”的情况，通常说明那个上限是真被工程卡出来的，不是随手写的。

第三格：从站侧的 ESC 芯片。ETG 写明子设备控制器可以是 ASIC、FPGA 或集成 MCU，对外的应用接口列了三种：SPI、8/16 位并口微控制器接口、32 位并行 I/O [行]。落到开发板上就是常见那条链：**LAN9252 这类 ESC 用 SPI 挂到你的 MCU 上**，MCU 不直接碰网线，它只读写 ESC 的寄存器。所以“我的 MCU 主频不够”这件事在 EtherCAT 里通常不是瓶颈，瓶颈是 **SPI 搬运过程数据的次数与延迟**——这正好接上第 22 章那张 ISR 时间账。

第四格：DC，分布式时钟。ETG 的描述是“the time from the first DC SubDevice is cyclically distributed to all other devices”，各子设备的时钟被调到这个基准上 [行]。为什么这一格重要：第 23 章算过，两路数据差 20 ms 就在 1 米处撕开 13.4 像素的伪视差——那是测距噪声的 27 倍；第 13 章算过采样时刻错开一个死区就引入几十毫伏的电压误差。**DC 卖的不是“快”，是“所有关节在同一个瞬间被采样”**——一个 1 kHz 的环里，20 个关节各自按自己的晶振跑，误差会累积成一条会漂的相对位姿。

第五格：CoE，对象字典。 ETG 写明 CoE 提供“Object Dictionary, PDO Mapping (Process Data Objects) and SDO (Service Data Objects)” [行]。机制是：参数（控制字、目标位置、模式选择）平时住在对象字典里，慢速路径用 SDO 一个字典项一个字典项地读写；一旦某个项被**映射进 PDO**，它就随过程数据每周期自动搬运，不再占用一次问答。所以“为什么有的参数改了立刻生效、有的要重新初始化”——因为前者在 PDO 里，后者要走 SDO。

边界（本书故意不写的东西）：具体对象字典的索引号（控制字、状态字、模式、目标位置那几项 分别住在哪个地址）本书**不给**。理由是方法论的：2026-10-08 检索到的中文资料全部是博客级，没有一份是 CiA/ETG 官方规范原文，而这些编号恰恰是“写错一个懂行的人当场发现”的那类事实。要落到代码，去查 ETG 与 CiA 的规范原文或你手上那颗从站的 ESI/XML 文件—— 那才是权威来源，而且它会告诉你这一台支持哪些模式。

类比，以及它在哪儿破

类比：RS-485 像一条村道，EtherCAT 像高速公路。村道便宜、谁都会修，但一次只能过一辆车，还得有人指挥交通。

这个类比在哪里破：村道的瓶颈是**通行量**，而总线的瓶颈是**延迟的不确定性**。RS-485 轮询 8 个节点，第 1 个和第 8 个的响应时刻差一整帧的时间——**这个抖动（jitter）才是控制环路的敌人**，不是平均带宽。第 00 章说过，延迟占 τ 的比例决定成败；而抖动会让控制器在“用旧状态出新的力”。

所以评估任何一条关节总线，别看 Mbps，看两件事：最坏情况延迟，以及延迟的方差。 这也是 EtherCAT 的 DC 真正的价值——它不是让带宽变大，是让**每个节点在同一时刻动作**。而本章补上另一半：最坏情况延迟的第一项来源，就是那个 ±200 mV 的判决窗口要在线上占多久。

一个可检验的说法：RS-485 能不能跑满 4 Mbps 不由“速率档位”决定，而由位时间与线上延迟的比值决定——4 Mbps ⇒ 250 ns/位，铜线 5 ns/m ⇒ 10 米才 50 ns、只占一个位的 20%，而回波要跨过一位需要 25 米（250 ÷ 2 ÷ 5）；这就是短线上跑得满、长线上必须降速的全部理由，而 120 Ω 端接的作用是把反射系数 Γ 打到 0。把官方自己公布的 15 台 / 6000 Hz / 17+16 B / 4 Mbps 乘起来是 23.76 Mbps，为 4 Mbps 的 5.9 倍（按官方 8N1 每字节 10 bit 算是 29.7 Mbps、7.4 倍），反过来除只有 1010 Hz 或 808 Hz——**“15 台”与“6000 Hz”不能同时以满帧全双工的读法成立**，而它落在 1 kHz 量级这件事，与官方 Z1 页标的 1 kHz 是同一道除法的两个端点。

证据等级：RS-485 4/6 Mbps、15 台、帧长 17/16 B 与 20/26 B、串口格式 8N1、CRC16_CCITT / CRC32-MPEG2、ID 0~~14 + 15 广播无响应、“全部电机用 RS-485、CAN 需转换模块”、终端电阻 120 Ω 两端各并、`rt/lowcmd` 500 Hz、Z1 Control Frequency 1 kHz、G1 23 DoF / H2 31 DoF [厂]；差分判决 ±200 mV 与共模 −7~~+12 V、传播速度 2×10^8 m/s ≈ 5 ns/m、反射系数公式、方波谐波与信道带宽、CAN 显性/隐性仲裁、EtherCAT 透明时钟 [行]；250/200/10 ns 位时间、50 ns 与 20%、25 m 回波、42.5/40/82.5 µs、1.24 ms、7.4 秒、808 Hz、23.76 与 29.7 Mbps、5.9/7.4 倍、1010 Hz、55.2 Mbps、9.2 倍、1087 Hz、6.07/7.59/2.11/1.06/3.96 Mbps、1,955 元、5% 是 [算]（`foc_check.py bus_budget()`；8N1 口径是本书在官方帧长上乘 10/8 得到，**分层拓扑是本书推导的自洽架构，不是宇树公布的实现**）；**宇树关节总线的条数、每干节点数、收发器型号、走线长度、内部控制环频率、以及“通信控制频率”的定义全部未公布** [未]；拆解口径与官方口径在“485 还是 CAN”上直接冲突，两条都保留；ODrive 8 kHz / moteus 15–30 kHz / LAN9252 ¥87 / 智元 EtherCAT↔CAN-FD 架构 [查]。

可核对清单：

- ☐ 到宇树电机文档页确认三件事：波特率、单总线最大电机数、以及“全部电机用 RS-485”的原文

- [ ] 数一遍 M8010 与 IM6014 的帧长与 CRC 算法，注意两者不同——这就是“私有协议”的含义
- [ ] 跑 `code/tools/foc_check.py` 的“27 总线带宽 M8010 / IM6014”，核对 23.76 Mbps / 5.9× / 1010 Hz 与 55.2 Mbps / 9.2× / 1087 Hz
- [ ] 自己按 8N1 重算一遍：4e6 ÷ (15 × 33 × 10) = 808 Hz，判断它与脚本的 1010 Hz 差在哪个因数（10 ÷ 8）
- [ ] 做本章那三次除法并把数写下来：1 ÷ 4e6 = 250 ns；250 ÷ (2 × 5) = 25 米；23 × 33 × 1000 × 8 = 6.07 Mbps（再除以 4 Mbps 得 1.5）——然后量一下你自己那台机器的 485 走线有多长
- [ ] 拿 87 元与 2 元乘 23 再除以四万元，自己判断这 5% 值不值一次同步能力的丧失

下一章：本章算出这条线上每改一次给定值要交 82.5 微秒。下一章去看它上面那层软件——DDS 的发现、序列化与拷贝，为什么在 1 kHz 环里比总线本身更值得担心，以及整个行业为什么正在从 ROS 往回退。

第 28 章

中间件：DDS 话题、rt/lowcmd，以及行业的“去 ROS 化”

一台机器的关节在 500 Hz 上被点名，而它的指令要走一条 TCP/IP 网络。第一反应是：**网络很快，这不算事**。但把这条链上的每一步摊开——谁在找对方、谁在打包、谁在拷贝、谁在系统调用——你会发现真正的问题从来不是带宽，是**最坏情况**。这一章就把这四件事一件件算给你看。

“Grok could organize the Optimus robots...”

埃隆·马斯克（Tesla / xAI；人形机器人 Optimus 的公开发言者） — 马斯克在播客里谈 xAI 与人形机器人的协同（Business Insider 综述；原文为片段） — 2026-07-24（英文原句，经第三方报道逐字引用；本书未核对录音或视频原文）

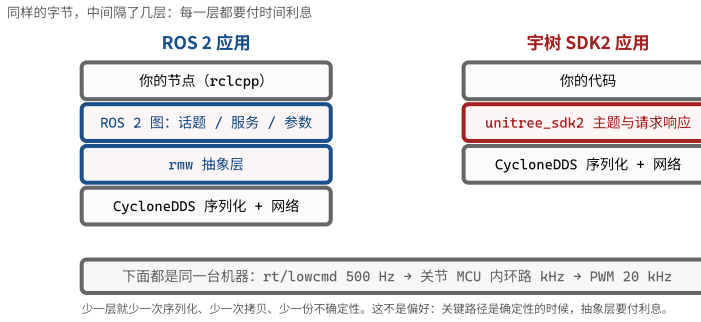


图 28-1　你的代码到电机之间隔了几层：ROS 2 栈与裸 DDS 栈对照

宇树的核心不是 ROS

这是本章唯一必须留下的一句话，因为网上大量教程把它讲反了。

宇树的 `unitree_sdk2` **本身不依赖 ROS**。它的核心是 **CycloneDDS**：一组主题（topic）加一套请求-响应服务 [厂]。ROS 2 支持是**另外提供的一层适配**（官方给 `unitree_ros2`，需要自编 cyclonedds 0.10.2 并设 `RMW_IMPLEMENTATION=rmw_cyclonedds_cpp`）[厂]。

为什么会这样？ 因为 ROS 2 自己就是建立在 DDS 之上的。

```
你的代码 → ROS 2 API（rclcpp）→ rmw 抽象层 → DDS 中间件 → 网络
你的代码 → unitree_sdk2 → DDS 中间件 → 网络
```

第二行少了一整层。对一台每秒要收发几百帧 35 元素数组的机器，**少一层就少一次序列化、少一次拷贝、少一份不确定性**。

所以“去 ROS 化”不是厂商傲慢，是一条非常朴素的工程判断：**当你的关键路径是确定性的时候，抽象层是要付利息的**。

三个厂商，三条路

厂商	机内通信	对外二开接口	官方理由（可查到的）
宇树	CycloneDDS 主题 + 服务	ROS 2（Foxy/Humble）+ 原生 SDK	未说明 [未]
智元	自研 **AimRT**（2024-09 开源，官方定位“对标 ROS 2”）；2026-03 又开源基于它的 **灵渠 Link-U OS**	ROS 2 Humble + FastDDS，以及 HTTP JSON RPC	官方称 AimRT 对标 ROS 2 [查]
银河通用	**公开文档完全没有 ROS/DDS 层**	GalbotSDK（C++/Python），分高层与低层	未说明 [未]

三家在这一点上的分歧，是全行业最值得注意的技术信号之一：它们对“关节怎么控制”高度一致（MCU + FOC + 现场总线，第 1 部与第 27 章），对“大脑用什么模型”也高度一致（第 30 章），唯独在**中间件**上各走一条。

原因是中间件这一层**没有公认正确答案**，而且它直接决定你能不能把延迟压进预算。所以每家都自己写了一遍。**而“自己写一遍”具体写什么，看下一节那四件事就清楚了**。

先算清一件事：这根本不是带宽问题

要把“负担”说清楚，先得把它不是什么说清楚。宇树 `unitree_hg` 的 IDL 把所有字段都定死了（本章后面那节会逐条拆开），把类型宽度按 4 字节对齐累加，就能算出一帧到底多大 [算]：

```
一条 MotorCmd  = uint8 mode(1) + 对齐填充(3) + 5×float32
q/dq/tau/kp/kd(20) + uint32 reserve(4) = 28 B
一帧 LowCmd    = 2×uint8(2) + 填充(2) + 35 × 28(980) + uint32[4]
reserve(16) + uint32 crc(4) ≈ 1004 B
一条 MotorState = 1 + 3 + 4×float32(16) + int16[2] temperature(4) +
float32 vol(4)
                 + uint32[2] sensor(8) + uint32 motorstate(4) +
uint32[4] reserve(16) = 56 B
一帧 LowState   = 35 × 56(1960) + version(8) + tick(4) + 头部(4) + IMU 与
wireless_remote[40] 等 ≈ 2.1 kB
── 剥掉类型名，两行纯字节累加，任何人拿计算器能复核：
1 + 3 + 20 + 4 = 28 B
2 + 2 + 35 × 28 + 16 + 4 = 1004 B
```

拿官方口径的 500 Hz [厂] 去乘：

```
下行 1004 B × 500 Hz ≈ 0.50 MB/s    上行 2100 B × 500 Hz ≈ 1.05 MB/s
合计 ≈ 1.55 MB/s = 12.4 Mbps，千兆链路要留给它的是 1.2%        [算]
```

1.2%。所以任何“总线/网卡带宽不够”的说法在这一层都不成立，本书也拒绝这样解释。**这一层的敌人只有一个东西：时间的方差**。而那 1 kHz 环的预算是多少？1000 μs（第 00 章那张六个时间域的表里 ④ 全身控制

那一格的上沿；最快的是 ⑤ 关节伺服，50–1000 µs）；而 500 Hz 是 2000 µs。平均时间当然够——**问题是这 2000 µs 里有四件事的最坏情况没有上界**。

关键路径上的四件事

第一件：发现（discovery）。 DDS 是去中心化的，没有配置中心：一个进程要说、另一个进程要听，两者先在组播上互相公告“我是谁、我发布/订阅哪些主题、什么 QoS”，这一步在 RTPS 协议里叫 SPDP/SEDP [行]。它平时安静，**但在两个时刻不安静**：节点启动的那一下，和一个参与者没了心跳的时候——后者会连带把与它相关的所有端点全部删掉重建。**这就是“建链抖动”：一个后来加入的订阅者，可能在某个 2 ms 的点上突然跑一轮发现**。 而一台人形上同时有低层控制、雷达、相机、灵巧手、遥操作好几个参与者，都在同一个组播域里。

那这又靠什么消掉？靠**静态发现**：预先写死对端清单、跳过自动公告，DDS 的实现普遍支持这种配置 [行]。**宇树有没有这么配、它的 `cyclonedds.xml` 长什么样，官方未公布** [未]。

第二件：序列化。 DDS 传的是结构体，网线上只有字节。把 1004 个字节按 CDR 规则（每类数据对齐到自身大小的整数倍）逐字段写一遍、收端再读一遍——这个动作**必做、可预算**，因为它没有分支也没有分配。但请注意：一旦你换成 ROS 2，数据要先从 rclcpp 的消息对象过一遍 rmw 再到 DDS，**同一份 1004 字节可能被编两遍**。 这就是本章开头那句“抽象层是要付利息的”的具体含义。

第三件：拷贝。 一路走下来，一帧 LowCmd 至少被复制这么多次：你的栈/堆上的结构体 → DDS 序列化缓冲区 → 内核 socket 缓冲区 → 网卡 DMA 环。这是四次，而每一次都是**跨所有权边界的复制**，都可能在页错误、内存碎片、或者内核调度上多花一次上下文切换。**为什么它们的最坏情况没有上界？** 因为堆分配器要走哪条路径由**堆此刻的状态**决定，而不是由你决定：命中空闲块时是一次很短的用户态操作，而需要向内核扩堆或

触发缺页时，代价要跳到系统调用与页处理的量级，两者相差两个数量级以上 [行]。1 kHz 环的预算是 1000 µs，它容不下一个“由别人决定”的分支。所以实时系统的标准手法只有一条：**所有内存启动时分配完，运行期只写不再申请**。

那这四次拷贝能不能省掉？ 能，而且只有一条路：**让两个进程看同一块内存**。发送方把 1004 字节写进一块双方都映射着的共享内存，翻一个原子标志，接收方直接读——**数据一次也没被复制，内核一次也没被叫起来**。这就是零拷贝（zero-copy，指“数据不需要在缓冲区之间搬来搬去”，不是“不传数据”）。DDS 这一层确实支持它（CycloneDDS 配 iceoryx，ROS 2 的 loaned message 也是同一件事）[行]。

但人形用不上它，这就是本章要说清的一处边界。共享内存的前提是**两个进程在同一台机器上**。而宇树的运控单元与开发单元是两块板子：官方 H1 文档的板载电脑表里 PC1（运控）的 IP 一栏写着 unaccessable，PC2/PC3（开发）是 `192.168.123.162 / 163` [厂]；官方遥操作仓库把 G1 的 PC2 默认写成 `192.168.123.164` [厂]。**跨板就必须过物理网卡，共享内存帮不上忙**。所以宇树这条链的真相是：`rt/lowcmd` 走的是**以太网上的 UDP**，它靠**定长数组（无堆分配）+ 单一 SDK（少一层 rmw）+ 固定 IP 网段**去换确定性，而不是靠 loaned message。**宇树没有公布它在这条链上是否启用共享内存、静态发现或具体 QoS** [未]。本书只能说：教科书里最值钱的那一招，在这台机器的物理拓扑下是受限的——而它仍然把这套东西跑在了 500 Hz。

第四件：系统调用与那个 1500 字节的坑。发一个 UDP 包是一次 `sendto()`：用户态陷入内核、socket 锁、协议栈处理、驱动排队。**这是关键路径上唯一一段你完全不能控制的代码**——它跑在内核里，被更高优先级的中断抢占，被网卡的中断合并（interrupt coalescing）策略批处理。而这里还藏着一个可算的坑：一帧 LowState ≈ 2100 B，而标准以太网 MTU 是 1500 B，扣掉 20 B 的 IP 头与 8 B 的 UDP 头，**单个数据报的净载荷上限是 1472 B** [行]：

```
2100 ÷ 1472 = 1.43 ⇒ 一帧状态要拆成 2 个分片          [算]
```

分片意味着：丢任意一片，整帧作废。 而 UDP 不会重传。这就是为什么这一层必须配成 best_effort + 只留最新——**因为“半帧状态”比“没有状态”更危险，而协议层没有给你第三种选择。**（宇树未公布它实际的分片与 QoS 参数 [未]，这一条是本书按 MTU 推的。）

这四件事合起来解释了一句话：**宇树为什么在机内用 DDS 但坚决不让 ROS 层进控制环。**

UDP 与 TCP：为什么实时环宁可丢包也不要重传

DDS 默认走 UDP。你可能会想：控制指令丢一帧，关节不就抖一下吗，为什么要用“不可靠”的协议？

因为 TCP 的可靠性是用不确定时延买的。 TCP 丢了包要等重传：它先要根据序号发现空洞，再等一个超时或用重复 ACK 触发快重传，然后重新发。**一次重传的代价至少是往返时间量级，而超时重传定时器（RTO）是毫秒到几百毫秒量级的东西** [行]——在 500 Hz 环里，那是一百个周期。**更糟的是 TCP 有序：一帧里的一个字节丢了，后面所有已经到达的数据都要等着**（这叫队头阻塞）。所以“可靠”在传输层意味着：**这一帧一定到，但不知道什么时候到。** 控制环路要的恰恰相反——**要最新的、不要旧的。**

UDP 的取舍就是把“一定到”换成“到了就是新的”。宇树在 IDL 层把这件事做成了语义：**主题配成 best_effort + 只留最新**（丢弃旧帧不重传）。而它同时留了两道兜底：一是 `LowCmd`/`LowState` 里各带一个 `crc` 字段（第 27 章说过，总线层已经有 CRC16/CRC32，DDS 层再有一个，保护的是两段不同的链路）[厂]；二是官方协议在电机层有一个 `timeout` 位——**一段时间没收到新指令就自动停**（第 42 章的安全设计）[厂]。**丢包不可怕，可怕的是丢了还不知道；这两条一起把“不知道”堵住了。**

那这条链在什么时候断了？ 当你要传的不是控制帧而是别的东西时。文件、日志、模型权重、标定表——这些不能丢，也不新覆盖旧。**同一个 DDS 域里既跑 500 Hz 的 lowcmd 又跑可靠的服务调用，QoS 就得逐条通道分别配**，而官方从未公布它的分区与 QoS 划分 [未]。这也是第 27 章那个“分层”结论在软件层的重复：**不同时间常数的数据，不该共用同一条确定性预算**。

现在把宇树这套接口拆开看

从官方仓库能直接读到消息定义（`unitree_ros2` 里的 `unitree_hg/msg/`，以及 `unitree_sdk2` 生成的 `include/unitree/idl/hg/`）[厂]：

```
LowCmd.msg   : mode_pr, mode_machine, motor_cmd[35], reserve[4], crc
MotorCmd.msg : mode, q, dq, tau, kp, kd, reserve
LowState.msg : version[2], mode_pr, mode_machine, tick, imu_state,
motor_state[35],
               wireless_remote[40], reserve[4], crc
MotorState.msg: mode, q, dq, ddq, tau_est, temperature[2], vol,
                sensor[2], motorstate, reserve[4]
```

这张表里有六个值得停下来的设计决定。

一、35 元素定长数组。 不是变长。为什么？**变长数组意味着堆分配，堆分配意味着最坏情况延迟不可知。** 定长数组可以在启动时一次性分配完，之后每次收发都是 memcpy 一个已知大小的块。这是实时系统的标准手法，代价是“消息永远按最大机型（35 关节）装”，即使 G1 只有 23 个——**用带宽换确定性**。上一节的算术把这笔交换量化了：23 个关节的真实载荷是 23 × 28 = 644 B，而线上走的是 35 × 28 = 980 B，多花的 336 B 按 500 Hz 折算是 1.34 Mbps，**千兆链路的 0.13%** [算]。这是一笔极其划算的交易，也是为什么这条消息结构长这样。

二、`crc` 字段。 第 27 章说过 RS-485 没有内建错误管理，所以宇树在**总线层**加了 CRC16/CRC32 [厂]。而现在 DDS 层又有一个 crc 字段——**两层都做校验**。为什么？因为 DDS 走的是 UDP（IP 网络），UDP 本身不校验载

荷。这两层校验保护的是两段不同的链路。

三、`reserve[4]` 与 `reserve` 字段。 保留位。它的存在说明**这套消息格式要向后兼容**：以后要加字段，从 reserve 里拿，老代码不会崩。这是接口设计的成熟做法，也透露了它还在演进。

四、`wireless_remote[40]`。 遥控器数据被塞进**同一个状态消息**里，而不是单独一个主题。含义很直白：**在任何时刻，控制器都必须同时知道机体状态和人的输入**——因为急停与接管需要它俩在同一时间戳上对齐。这是一个安全设计，不是偷懒。

五、`tick`。 一个单调递增的时间基准字段。第 23 章说过“同步是感知的第一约束”——`tick` 就是这台机器上所有数据的对齐锚点。而它同时也暴露了上层的痛：**如果 DDS 层的发现与拷贝让 tick 的间隔抖动变大，第 23 章那个时间对齐就再也做不准**。

六、话题命名。 官方能查到的低层通道是 `rt/lowcmd`（下指令）、`rt/lowstate`（全身状态）、`rt/lf/lowstate`（低频版状态）、`rt/secondary_imu`（躯干第二 IMU）、`rt/arm_sdk`（机械臂通道）[厂]。`rt/` 前缀与 `lf/`（low frequency）的区分本身就是一层设计声明：**同一条网络上要同时跑确定性与非确定性两类流量，所以给它们不同的名字与不同的节奏**。

请求-响应那一半：为什么不只用主题

DDS 提供两种通信模式：**主题**（发布/订阅，单向、高频）和**服务**（请求/响应，双向、低频）。

宇树两者都用：

```
高频、周期性、状态与控制   →  主题（rt/lowstate、rt/lowcmd）
低频、一次性、配置与调用   →  服务（版本读取、参数设置、模式切换、OTA 一类）
```

这个分工值得说清，因为很多教程把它混成一团。 判据是：这条数据是“持续更新的最新值”还是“一次问答”？前者用主题（新值覆盖旧值，丢一帧无所谓），后者用服务（必须确认收到）。

而“丢一帧无所谓”这句话本身就是设计承诺。 主题的 QoS 通常配成 best_effort + keep_last(1)：不重传、只保留最新。这对控制是对的（旧指令本来就该被新指令覆盖），对文件传输是灾难。**所以 QoS 配置不是性能参数，是语义声明。**

（智元对外的 ROS 2 二开接口官方写明 `ROS_DOMAIN_ID=232`、QoS 为 best_effort / keep_last 10 [查]——**同样是 best_effort，但保留 10 条。**这个差别说明两家对“旧数据还有没有用”的判断不同：宇树要最新，智元允许回看一小段轨迹。这是一个可以写进接口文档的、非常具体的哲学差异。）

版本地狱：这一节是给你省时间的

任何一份宇树二开教程都会失效，因为版本约束很硬。把能核实到的写死在这里：

```
cyclonedds 必须自编 0.10.2，并设 RMW_IMPLEMENTATION=rmw_cyclonedds_cpp
[厂]
消息包分两套：unitree_go（老机型）与 unitree_hg（H1_2/G1 等新机型）
[厂]
ROS 2 支持口径为 Foxy / Humble；Jazzy 是否支持需当场核对
[厂][未]
unitree_rl_gym 依赖 Isaac Gym Preview 4（已 EOL）；rl_lab 用 Isaac Sim 5.1
+ Lab 2.3 [厂]
```

“两套 IDL”这一条最容易伤人：`unitree_go` 与 `unitree_hg` 的消息**不兼容**，字段名与数组长度都不同。你把为 Go2 写的代码直接搬到 G1 上，编译能过（如果你两套都装了）但运行时什么都不发生。

而 cyclonedds 版本为什么写死到小数点？ 官方 README 甚至要求“编译 cyclonedds 时终端不能 source ROS 2” [厂]——因为两套 cyclonedds 会在同一条 DDS 域里以不同的 RTPS 行为互相打断，症状是“有时能收到、有时收不到”，而这类 bug 最难查。**这条限制不是洁癖，是同一个组播域里跑了两份协议实现的必然结果**。

判据很简单：先看你的机型属于哪一代协议。 而这件事，官方文档里是分开写的，教程里几乎从不提。

行业在往哪走

把三条线放在一起看，趋势是清楚的：

一、留 DDS，弃 ROS 上层。 宇树核心是裸 DDS；openpi、GR00T 这类模型项目干脆提供**裸 socket / websocket 的策略服务**（机器人只管发观测、收动作）[查]；Figure、Tesla 自研 C++ 栈 [查]。

二、模型侧与本体侧正在用“网络服务”解耦。 一个 5–10 Hz 的大脑（第 26 章那条带宽除法）根本不需要 DDS 的发现与 QoS 机器——它需要的是一个 HTTP/WebSocket 端点。**于是“中间件”这个词在人形上正在分裂成两个东西：kHz 级的本体总线，和 Hz 级的模型服务**。 用同一套技术管这两件事，是 ROS 1/2 时代的假设。

三、主权叙事出现。 中国行业内开始有“多数国产机器人依赖海外 ROS”的担忧与自研中间件尝试（智元 AimRT / Link-U OS 是公开的一例）[查]。**本书对这一条只作记录，不作判断**——因为它是否成立取决于生态成本，而生态成本目前没有可核实的数。

而“自研中间件”这条路的真实内容，就是上面那四件事的清单：把发现改成静态、把序列化改成共享内存里的同一份结构、把系统调用从环里踢出去、把 QoS 语义按通道拆开。智元的 AimRT、银河通用的 GalbotSDK、宇树的裸 sdk2 三条路，**在最坏情况预算这一张表上高度相似**——行业不是在做审美选择，是在同一条 1000 μs 的预算线上各自往回挤。

边界：这不等说 ROS 不能实时

这一条必须写清楚，否则本章会被读成“ROS 慢所以别用”，而那是错的。

ROS 2 本身就是为实时设计过的：它建立在 DDS 上、可以逐通道改 QoS、支持在 PREEMPT_RT 内核上跑、也支持节点内的 loaned message 零拷贝 [行]。**所以“ROS 不能实时”这句话不成立**。顺带一条已核实的事实：本书检索过全部官方仓库，宇树**没有**提供 `ros2_control` 那一层硬件接口，给的是 URDF 加自定义消息——拿 ROS 2 接宇树时，那一层要你自己写。

宇树和同行的三家做的不是“ROS 不行”的判断，是一个更窄、也更诚实的判断：**在我这条 1–2 ms 的关键路径上，我要少一层能改我代码路径的抽象**。少一层不是省 CPU——上一节算过，带宽与 CPU 都够用；**少一层是把“最坏情况由谁决定”这件事收回自己手里**。当你的 SDK 就是你的产品边界，而 ROS 的版本、发行节奏、构建系统与社区补丁不在你手里时，你就不该把控制环押在上面。

换句话说：去 ROS 化不是性能优化，是责任划分。而这条边界在读者那里的实际用法是：**只做上层（感知、规划、数据采集、遥操作）时，ROS 2 那套工具链的价值远大于它的开销；只有当你真的要往那条 2 ms 的通道里写东西时，本章这四件事才变成你的账单**。

从 ROS 2 那边看过来：四样东西的真身

如果你是从 ROS 2 那条路进来的工程师，读完上面几节大概会问一句：**tf2、nav2、MoveIt 2、`ros2_control` 这四样，在一台人形机器人上对应什么？**这一节就是那张对照表。它的用处不是教你用 ROS，而是让你已有的直觉 在正确的地方生效、在错误的地方失效。

tf2 = 一棵随时间变化的刚体树。 它做的事只有一件：把“在坐标系 A 里、时刻 t 的一个点”换算成“在坐标系 B 里”，靠的是每条边存一个 4×4 齐次变换，再沿树做矩阵链乘，跨时间就查缓冲区做插值。**那它为什么进不了 1 kHz 那条环？** 因为它的成本不在乘法上，在**缓冲区与时间**上：`/tf` 是按消息速率发布的（典型几十到几百赫兹），一次查询要拿目标时刻去插值，而插值意味着要等两边的那两条消息都到齐。这件事的抖动来自线程与队列，不来自数学——正好是第 22 章那六个“偷时间者”里的两个。而在人形本体内部，这棵树其实是**静态**的：连杆长度写在 URDF 里不会变，唯一变的量就是那 23 个关节角。所以机内真正需要的不是 tf2，是“正运动学 + 一次链乘”，输入来自 `rt/lowstate` 的 q/dq——本章前面算过那是 1004 字节 @ 500 Hz。**结论：tf2 是“跨节点、跨时间、给人看”的那一层，不是“每 1 ms 算一次”的那一层。**

nav2 = 全局路径 + 局部代价地图 + 一个轨迹跟随器。 它的隐含假设是“有一个在平面上连续可导的底盘位姿，和一张 2.5D 栅格”。**人形为什么常常不用它？** 因为双足的支撑面是**离散的落脚点**，不是连续位姿—— 这正是第 17 章那条 ZMP / 捕获点链条要解决的东西：可接受的不是“离障碍多远”，而是“这一脚踩下去重心投影还在不在支撑多边形里”。于是行业里那一格常被换成 RL 策略 + 足端轨迹，nav2 退到“给我一张能走的图”。

MoveIt 2 = 正逆解 + 自碰撞检查 + 采样式规划。 关键机制在逆解：手臂自由度一多，同一个末端位姿对应**无穷多组关节角**，你必须要么在几个解析解里挑一个，要么把它写成一个带约束的二次规划去解—— 而 QP 的代价按第 19 章那条 $\mathbf{n}^3$ 增长。宇树在 `rt/arm_sdk` 里做的事，本质是“你别自己解，我直接给末端位姿指令”，把这条链收进厂商侧。这是接口选择，不是能力差异。

`ros2_control` = 把 1 kHz 读写抽象成 `read()` / `write()` 的硬件接口层。 听起来正好是这条环需要的东西，但它跑在 Linux + ROS executor 上：调度、序列化、回调组这些第 22 章列出的东西一个都没躲开。**这就是“留**

DDS、弃 ROS 上层”这条行业路线的真实理由—— 不是 ROS 太慢这个口号，而是它把三件有抖动的东西（发现、执行器、内存分配） 放进了一个要求最坏情况有界的地方。

边界（必须说清）：宇树、智元、银河通用**是否提供 nav2 或 MoveIt 2 的官方接口**， 本书没有逐家核到一手文档，一律 **[未]**；本节讲的是这四样东西各自的机制与失效位置， 不是任何一家的产品事实。版本口径也要自己核对：Humble 与 Jazzy 之间 `ros2_control` 的硬件接口签名有变化，抄来的代码块经常在这上面翻车。

类比，以及它在哪儿破

类比：DDS 像一套邮局系统——你不用认识收件人，只要按格式贴好地址，信就到。ROS 像在这家邮局上再开一层公司行政部。

这个类比在哪里破，而且破得很有用：邮局保证“送到”，DDS 默认**不保证**。它的 QoS 可以被配成“尽力送、丢了不补、只留最新”——这在控制语义上是正确的，但它意味着**你的程序必须能处理“上一帧根本没到过”**。

而绝大多数写惯了函数调用的人不会去处理这件事。于是出现一类非常难查的 bug：代码逻辑对、参数对、消息格式对，但因为某一帧在 best_effort 下丢了，控制器拿着 8 毫秒前的状态出了 100 牛米的力。

所以从 ROS 转裸 DDS 的人，真正要换的不是 API，是“消息可能不存在”这个默认假设。

一个可检验的说法：宇树的核心通信层是 CycloneDDS 主题与服务，ROS 2 只是其上的适配层，因此“宇树用 ROS”这个说法在架构上是反的。按官方 IDL 字段宽度与 4 字节对齐累加，一帧 LowCmd ≈ 1004 B、一帧 LowState ≈ 2.1 kB，在官方 500 Hz 口径下合计约 1.55 MB/s，只占千兆以太网约 1.2%——**所以中间件这一层的瓶颈不是带宽，是最坏情况时间**，而它来自四件事：发现（组播公告与端点重建）、序列化（跨

rmw 时同一份数据可能编两遍）、拷贝（用户态→序列化缓冲→内核 socket 缓冲→网卡 DMA，四段）、系统调用与分片（`sendto` 之后那段不受你控制的内核路径；2100 B 超了单个数据报的 1472 B 净载荷，必须拆两片、丢一片即整帧作废）。静态发现、共享内存与定长数组分别去掉这三类方差，而共享内存这一招在跨板的两块 PC 上是受限的。行业在保留 DDS 的同时抛弃 ROS 上层，并把模型侧降格为一个 Hz 级的网络服务——**但这是责任划分，不是“ROS 不能实时”**。

证据等级：`unitree_sdk2` 基于 CycloneDDS 且不依赖 ROS、ROS 2 适配需自编 cyclonedds 0.10.2 并设 RMW、编译时不能 source ROS 2、`unitree_go`/`unitree_hg` 两套 IDL、全部消息字段与类型、`rt/lowcmd`/`rt/lowstate`/`rt/lf/lowstate`/`rt/secondary_imu`/`rt/arm_sdk` 话题名、500 Hz 与 `control_dt_ = 0.002` 口径、`timeout` 位、H1 板载电脑 IP 与 PC1 unaccessable 口径 [厂]（可从官方仓库文件与文档中心逐字读到）；G1 的 PC2 默认地址 `192.168.123.164` [厂]（官方 `xr_teleoperate` 仓库）；消息字节数 28/980/1004/56/1960/2100 B、644 B、336 B、0.50 与 1.05 MB/s、12.4 Mbps、1.2%、1.34 Mbps、1.43 与“2 个分片”是 [算]（**按字段类型宽度与 4 字节对齐累加的本书估算，线上真实封装开销、是否共享内存、实际分片与 QoS 全部未公布**）；SPDP/SEDP 发现、CDR 序列化、四段拷贝、堆分配的两条路径、1500 B MTU 与 1472 B 净载荷、UDP/TCP 与 RTO、队头阻塞、ROS 2 的实时选项 [行]；智元 AimRT/Link-U OS、`ROS_DOMAIN_ID=232`、QoS best_effort/keep_last 10、openpi/GR00T 裸服务、Figure/Tesla 自研栈 [查]；银河通用无公开 ROS/DDS 层 [厂]（其“没有文档”这一事实本身可核实）；**宇树从未解释为什么选 DDS、未公布其 QoS 配置、发现模式、是否静态发现、是否共享内存、线程与中断亲和性、Jazzy 支持状态** [未]；“主权叙事”相关表述只作记录。

可核对清单：

- ☐ clone `unitree_ros2`，打开 `src/unitree_hg/msg/LowCmd.msg` 与 `src/unitree_go/msg/LowCmd.msg`，把两者的字段并排抄一遍——你会亲

眼看到它们不兼容

- [ ] 数一遍 `motor_cmd` 数组长度，确认是 35；再按本章的式子自己累加一次 MotorCmd 的字节数：1 + 3 + 20 + 4 = 28 B，乘 35 加头部，看是不是 1004 B 上下
- [ ] 拿 1004 B 与 2100 B 各乘 500 Hz 加起来除以 125 MB/s（= 1000 Mbps ÷ 8），确认“1.2%”；再拿 2100 ÷ 1472 除一次，看它是不是 1.43——**这就是“一帧状态要两个分片”的全部内容**
- [ ] 在 `unitree_sdk2` 里搜 `rmw` 或 `rclcpp`，确认核心 SDK 里确实没有 ROS 依赖
- [ ] 找一份你正在看的宇树教程，检查它有没有说清“你的机型属于 unitree_go 还是 unitree_hg”——多数没说
- [ ] 自己发一帧 `rt/lowcmd`，把 `sendto` 与写共享内存各跑 10 万次，画一张时延直方图——**你会亲眼看见哪一条有长尾**（这是第 40 章要求的“带数的产物”）

下一章：本章把机内这条 2 ms 的通道拆开看完了。第 3 部到此结束——机器已经能看、能平衡、能操作、能通信。第 4 部去看那个还没有物理答案的部分：怎么让它知道该做什么。第一站是一个 1966 年就有的想法，和它为什么在 2023 年突然够用。

第 29 章

从轨迹到策略：模仿学习、ACT、扩散与流匹配

一个 1966 年的想法，和它为什么一直不够用

让机器看人怎么做，然后照着做——这个想法叫**模仿学习**（imitation learning），最早的形式叫行为克隆（behavior cloning）：**把“观测 → 动作”当成一个监督学习问题**。输入是相机图像与关节角，标签是人演示时的那只手该往哪动。

这个想法简单到不需要解释。它失败了几十年，原因也不是算法。

第一个原因：分布偏移（distribution shift）。

你在演示数据里看到的每一个状态，都是“人做得很对”的状态。而真机上，你的策略第一次犯错（差 2 厘米），就进入了一个演示数据里**从来没有出现过的状态**。此时模型的输入超出训练分布，输出更不可靠，于是错得更多，于是进入更陌生的状态。

这是一个正反馈。它的学名叫**复合误差（compounding error）**：

```
P = (1 − ε)^N

P = 整段不偏离的概率，无量纲
N = 决策次数，单位：步；ε = 每步偏离训练分布的概率
N = 100、ε = 0.05 ⇒ P = 0.95^100 = 0.6%
```

（[算]；与第 38 章那条成功率复合衰减是同一个式子。）

这一条解释了为什么所有模仿学习的演示都是短任务。30 秒、20 步、一次抓取——不是因为短任务有价值，是因为**长任务在这个算法下数学上不可能**。

第二个原因：数据。 一条演示要一个人拿着遥控器或者穿着外骨骼做一遍。ALOHA 那类主从臂方案的公开口径是**单任务约 50 条演示**——一手原话 `With 50 demonstrations for each task,`（Mobile ALOHA，arXiv 2401.02117）[查]；按第 33 章“8 人工小时才出 1 个可用小时”那条可用率 [推]，最多推出 400 条拍摄量——但那 8 倍是**时间**上的可用率（摆场景、质检、休息、标注都在里面），不是重拍率，两者不能直接相乘。拍摄量的可信区间因此是 50–400 条：取 400 要假设全部浪费都来自重拍。

第三个原因：多峰分布。 这是本章要重点讲的，因为它最容易被工程师忽略。

对于具身智能和机器人来说，模型架构都还不够好，也不够统一。……但是在具身智能，在机器人领域，大家可以发现，很多情况下有了数据，会发现这个数据用不起来。

王兴兴 — 王兴兴 2025 世界机器人大会主旨演讲（新浪“机器人前瞻”文字实录） — 2025-08-09（第一人称演讲逐字）

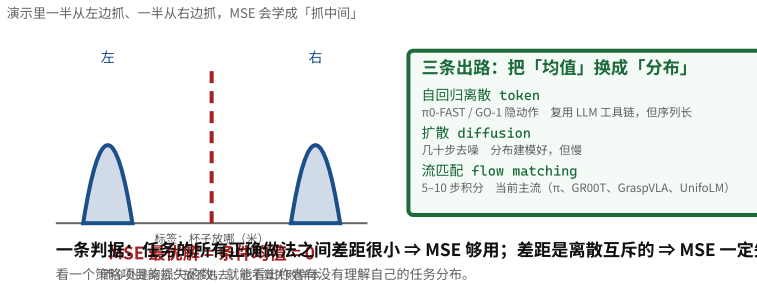


图 29-1　多峰标签下 MSE 的均值陷阱：学出「既不左也不右」

多峰：为什么 MSE 会教出一个“既不左也不右”的机器人

假设你教机器人把杯子放到桌上。你的演示里，一半的人把杯子放在**左边**，一半放在**右边**。写成数：a = −0.2 m 与 a = +0.2 m 各一半，两种情况的观测 s 完全相同。

现在用最朴素的做法：网络输出一个连续值（x 坐标），损失用均方误差。为什么它一定错？把解算出来，别猜：

```
L = E[ (a − π(s))² ]              a = 动作标签(m)；π(s) = 策略在观测 s 下输出的动作
dL/dπ = −2 · E[ a − π(s) ] = 0
π* = E[ a | s ] = 0.5×(−0.2 m) + 0.5×(+0.2 m) = 0
```

（E[a|s] 读作“这个观测下 a 的期望”；能逐点求导，是因为损失是对 s 取的期望，π 在不同 s 上互不纠缠。）

模型学到“放在中间”。而中间是那台机器的身体、或者桌上那个盘子的位置——**放中间是错的**。

这不是训练不充分，是**目标函数在结构上就答错了**：MSE 拟合条件均值，而人类操作天然多峰。加数据也修不好——样本越均衡，均值钉得越稳。

你可能会想：换个损失函数不就完了？ 这一步也算得出来：

```
L1 = E[ |a − π(s)| ]   ⇒   π* = median(a | s)
```

绝对值损失的解是条件**中位数**。这里要小心，**对称的两峰是个退化例子**：两峰各 50% 时，E|a−m| = 0.5(|−0.2−m| + |0.2−m|) 在整个 [−0.2, 0.2] 上恒等于 0.2，也就是说**区间里每一个点都是最优**，0（也就是均值）同样是最优解之一——所以拿对称两峰去区分 L1 与 L2，区分不出来。要让它贴峰，得把权重弄偏：比如 60% 在 −0.2、40% 在 +0.2，中位数的定义是“累积质量第一次过半的位置”，于是解跳到 −0.2，而 L2 的均值是 0.6×(−0.2) + 0.4×(+0.2) = −0.04。**这才是“换损失函数就换峰”的真实条件：分布必须不对称。*凭什么？ 平方损失最小化“距离平方和”，最**

低点在中间；绝对损失最小化“距离和”，最低点必须贴在某个数据点上。再往下追一层：动作离散成两类时，交叉熵的解是条件*众数，同样贴峰。

于是一组可以背的对应：L2 → 均值，L1 → 中位数，交叉熵 → 众数。只输出一个确定数、又配均值型损失的策略，必然输出一个演示里从不出现过的动作。**本章第一个“底”：多峰的第一层解不在模型容量里，在损失函数的形状里。**

于是整个领域转向了“建模分布”而不是“拟合均值”。三条路：

方法	怎么建模分布	代价
自回归离散 token（如 π0-FAST）	把动作量化成词表，用语言模型那套交叉熵	需要学一个好的量化器；序列长
扩散模型（Diffusion Policy）	从噪声一步步去噪到动作	推理要多次去噪，**慢**
流匹配（flow matching）	学一个从噪声到动作的速度场，少步积分	比扩散快，但仍需多步

（三条路都是当前主流，行业收敛点是“动作头三选一”，第 30 章。）

这里有一个可以立刻用的判据：如果一个任务的所有正确做法之间**差距很小**（比如把螺丝拧进去），MSE 够用。如果差距是**离散的、互斥的**（从左抓还是从右抓、先拿碗还是先拿筷），MSE 一定会失败。**看一个策略项目的损失函数，就能看出作者有没有理解自己的任务分布。**

“建模分布”到底改了哪一行：先抽一个随机量，再回归

朴素 BC 为什么没救？因为它的形式是 $\hat{a} = \pi(s)$：给定 s，输出就定死，而同一个 s 对应两个 a，只能折中。三条新路子改的是同一件事——**网络输入里多了一个随机量。**流匹配（flow matching，“把噪声连续变形成动作”的生成模型）训练一个速度场 $v_\theta(x, t)$，把噪声 $x_0 \sim N(0, I)$ 从 $t = 0$ 积分到 $t = 1$，终点 x_1 就是动作：

```
x_t = (1 − t)·x₀ + t·x₁          t 无量纲，取值 [0, 1]
v = x₁ − x₀                      路径速度，量纲同动作（m 或 rad/s）
L = E[ ( v_θ(x_t, t) − v )² ]    仍然是均方误差
x_next = x_t + v_θ(x_t, t)·Δ     欧拉积分，Δ = 1/K，K = 积分步数
```

关键在第三行：MSE 没被换掉，换掉的是条件。朴素 BC 的条件是 s，两个动作的均值落在中间；这里的条件是 (s, x_t, t)，而 x_t 里带着“这次走哪条路”的信息——x_0 落在左边那簇，条件期望就只在左边。**多峰没被消灭，它被搬进抽样：选峰交给噪声，网络只学“给定噪声往哪走”，而这映射是单峰的**。

那这个又靠什么保证？ 靠一条朴素事实：任何目标分布都能写成“先从简单分布抽样，再做一个确定变换”；扩散与流匹配的差别只是把变换写成“逐步去噪”还是“积分速度场”。“会不会建模分布”于是换成“这个变换学得准不准”。离散 token 与 ACT（模仿学习里最早被大规模复现的动作头之一）同理：前者按概率抽样，后者训练时把一段动作压进潜变量 z、推理时抽一个 z 再解码 [行]。**三条路的共同点不是“分布建模”四个字，是输出端接了一个随机源**。

为什么偏偏是“迭代 K 步”这一族赢了？往下再挖一层，答案是步数可以被压到个位数。流匹配把“噪声 → 动作”写成一条**直线插值路径** `x_t = (1−t)·x₀ + t·x₁`，于是它的目标速度场在**理想情况下是常向量** `v = x₁ − x₀`，与 t 无关；欧拉法解常向量场**一步就精确** [行][算]。**可这句“一步精确”的成立范围必须钉住**：它只对训练时**每一对样本各自那条路径**成立，也就是条件场 $v(A|x_0,x_1)$。网络学到的是**把 x_0 边缘化之后**的场 $E[v|x_t]$，同一个 x_t 上多条路径的落点被平均在一起，这个合起来的场**随 t 变化、不再是常向量** [行]——所以一步并不精确，K 步是在花算力买回被边缘化抹掉的那部分结构。**结论要换个说法：直线化买到的是离散误差的系数变小，不是误差归零；而在真实系统里，网络自身的拟合误差通常比 1/K 那项离散误差大得多，砍 K 前先确认你在削哪一项**。*可这句“一步精确”的成立范围必须钉住：它只对训练时*每一对样本各自那条路径成立，也就是条件场 $v(A|x_0,x_1)$。网络学到的是**把 x_0 边缘化之后**的场 $E[v|x_t]$，同

一个 x_t 上多条路径的落点被平均在一起，这个合起来的场**随 t 变化、不再是常向量** [行]——所以一步并不精确，K 步是在花算力买回被边缘化抹掉的那部分结构。**结论要换个说法：直线化买到的是离散误差的系数变小，不是误差归零；而在真实系统里，网络自身的拟合误差通常比 1/K 那项离散误差大得多，砍 K 前先确认你在削哪一项。*可这句“一步精确”的成立范围必须钉住：它只对训练时*每一对样本各自那条路径**成立，也就是条件场 $v(A|x_0,x_1)$。网络学到的是**把 x_0 边缘化之后**的场 $E[v|x_t]$，同一个 x_t 上多条路径的落点被平均在一起，这个合起来的场**随 t 变化、不再是常向量** [行]——所以一步并不精确，K 步是在花算力买回被边缘化抹掉的那部分结构。**结论要换个说法：直线化买到的是离散误差的系数变小，不是误差归零；而在真实系统里，网络自身的拟合误差通常比 1/K 那项离散误差大得多，砍 K 前先确认你在削哪一项。**扩散那一边要沿 Score 场走，Score 随 t 剧烈变化（近端有 1/t 那类刚度），所以同样精度需要几十步。**离散误差随步数怎么降也能算：** 欧拉法的全局误差 $\propto$ 步长 $h = 1/K$，于是

```
K=1 → 误差量级 1.00；K=5 → 0.20；K=10 → 0.10；K=20 → 0.05；K=50 → 0.02
从 5 步加到 20 步，误差只降到 1/4 —— 用四倍算力换 4 倍精度，这是线性，不是指数
```

[算]。**这条 1/K 只是“5–10 步够用”的一半理由**：直线化把离散误差的**系数**压小了，所以不必靠加大 K 去救离散项；另一半是——K 再大也救不了模型的拟合误差，而它通常是主导项（上面那段“边缘化后的场不是常向量”就是根源）。**于是判据要写成一条不等式，而不是一个除法式子：** K 步挤进一个节拍要求 `K × (2P_迭代 ÷ 带宽) ≤ 节拍`，等价于 `P_迭代 ≤ 节拍 ÷ K × 带宽 ÷ 2`——拿第 30 章那张表一查就知道自家硬件选哪一头。**注意下标：P_迭代 是“每一步都要重读的那部分权重”，不是模型总参数。** π0 的 3.3 B 里只有那 300 M 的专家参与十步迭代，主干作为 KV 缓存的前缀只算一次（第 30 章引了一手原话）[查]；**把总参数当 P_迭代 会把预算高估一个数量级以上，本书上一版在第 30 章正是这样错的** [算]。这才是：

200 Hz 的控制环选小回归网络，50 Hz 且允许分块的选 5–10 步流匹配，秒级才轮得到离散 token（第 30 章把这条上界算给 Helix 与 π0 看，那里还给了它用错对象时会错到什么程度）。

代价可数。 K 步就是 K 次前向：π*0.6 的公开口径是 H100 单卡 5 个去噪步、一块 63 ms——**这组数本书没有读到原文**：官网 `research/pi-star-0.6` 与 `research` 两页对带桌面 UA 的请求一律返回 429，arXiv 检索 0 条，所以按本书图例只能记 [推]（二手转述，未读到原文），用它的推论只支持量级，即 t_step = 63 ÷ 5 = 12.6 ms。**同一个量有一手能核的对照**：arXiv 2510.26742 报 10 个去噪步在单张 RTX 4090 上 `a latency of 27.3 ms` ⇒ 每步 2.73 ms [查]，比那个二手口径还快 4.6 倍——**所以“每步十几毫秒”不是瓶颈，把 12.6 ms 换成 2.73 ms，下面那条与带宽的对比只会更极端**，而第 26 章那张带宽表给的是 44–288 ms 一步（273–41.6 GB/s，120 ms 是中间 100 GB/s 那一档）[算]——**120 ÷ 12.6 ≈ 9.5**，最快的 44 ms 档也有 3.5 倍，同一套去噪步在机器人那块板上比在 H100 上慢 3.5–23 倍，不是一个数量级的死数。

分布偏移的正解：让状态分布可控

复合误差有三类工程解法，都有效，都有代价：

一、动作分块（action chunking，一次吐出一段未来动作而不是一个动作）。 它有两个互不相干的理由，工程师通常只看到第二个。

理由一：延迟，而且可算。 大脑一侧的单步下限是 120 ms（第 26 章的带宽算术，还没算 KV 与图像 [算]）；身体一侧，第 21 章官方口径策略输出 50 Hz，Δt = 20 ms（`sim.dt 0.005 × decimation 4` [厂]）。只预测 1 步就是每 20 ms 等 120 ms，占空比 duty = 20 ÷ 120 = **17%**。分块之后条件变成一条不等式（第 31 章把它做成表）：

```
H · Δt ≥ T_infer   ⇒   H ≥ T_infer ÷ Δt
H = 120 ms ÷ 20 ms = 6 步                (100 GB/s 那一档)
H = 288 ms ÷ 20 ms = 14.4，取 15 步          (第 26 章 41.6 GB/s 那一档)
Δt = 1 ÷ 30 Hz = 33 ms   ⇒   H = 120 ÷ 33.3 = 3.6，取 4 步（数据侧口径）
```

块长不是超参，是被内存带宽推出来的下界。 公开系统取的 H = 40（GR00T N1.7 的 action horizon [查]）远大于它——为什么，看理由二。

理由二：把“每步重新选一次峰”停掉。 分布建模每步带一次抽样，而抽样会抖：第 1 步抽“从左抓”、第 2 步抽“从右抓”，末端就在两峰之间横跳——单步策略里这必然发生，因为两次噪声实现互相独立。分块之后一块里的 H 步来自同一次抽样，峰的选择被冻结。**分块不是让模型更准，是让“准”能被连续执行出来。**

代价一：块内开环（第 31 章给这件事的名字）。H = 40、Δt = 20 ms ⇒ 一块覆盖 800 ms，最坏反应延迟也是 800 ms。拧螺丝能允许 0.8 秒不做反应，**踩到肥皂水不行**——分界不在模型好坏，在时间尺度。

代价二：它改的是复合误差里的 N，不是 ε。 还是开头那个式子，N 从“步”变成“块”，同一个 ε = 5%：

```
P = 0.95^100 = 0.6%       （逐步决策，N = 100 步）
P = 0.95^10 = 60%         （H = 10 分块，N = 10 块）
```

看着像涨了一百倍，其实不是。 前提是被偷换的：“块内那 10 步一次都不偏离”，而这正是块内开环换来的假设——块里没有修正入口。分块把 100 次小风险换成 10 次大风险：下注次数少了，单次落进分布外的代价反而更大。**这条链在这里断了：分块只推迟复合误差，不取消它。**

二、闭环重规划（receding horizon）。 每执行几步就把整块丢掉、重新预测。这是分块与闭环的折中——买回反应速度，付出的是把推理次数又乘回去。

三、DAgger 一类的在线修正。 让模型自己走，一旦偏离就请人接管纠正，把“模型自己造成的状态 + 人的纠正”加入数据集。**这是唯一直接攻击分布偏移的方法**——因为它在扩充的正是“模型会去但数据里没有”的

那片状态。为什么补状态能改误差的阶？因为 BC 的误差是**串起来**的，N 步累下来对 N 是平方级的；DAgger 让训练访问的状态分布与执行访问的**重合**，偏差不再互相放大，误差退化成线性 [行]。

代价：需要人在现场。**这就是“数采工厂”存在的数学理由**（第 33 章）：它不是懒，它是 DAgger 的工业化形式。

一个必须知道的上限

模仿学习有一条无法绕过的性质：

它的性能上限是演示者的水平，而且它无法从自己的错误里学到任何东西。

第一个半句意味着：如果演示者只有 80% 成功率，你不可能训出 90% 的策略（除非数据里有失败样本并做了处理）。第二个半句意味着：**强化学习之所以还在被使用，不是因为模仿学习不够好，是因为只有 RL 能从“我摔了”这件事里获得信号。**

第二半句为什么成立？底在梯度上。 BC 的梯度只挂在数据集那些 (s, a) 对上；模型自己走出来的 s' 不在数据里，没有标签就没有梯度——**损失函数在那片区域是空的，优化器再强也取不到东西。** 而 RL 的信号恰恰挂在自己走出来的状态上。

这句话还要一个数钉住，否则只是情绪。 各家短任务成功率里，本书能逐字回读原文的只有第 38 章表里那几条论文数。**曾经写过、现已删掉的两个**：Gemini Robotics 2 的“拉袋 44% / 拉链 40%”与 Figure Helix 2.5 的“三十户家庭零样本 56%”——2026-10-09 复核时，前者在 arXiv 摘要页里检索不到，后者在 Figure 自己的新闻索引里连 “Helix 2.5” 这一条目都没有、详情页返回 404。**既然只能靠媒体转述，本书就不印它**（作者的验收原话：不能确保数据准确那就不写，不能误人子弟）。下面用一条纯算术把同样的意思说清。共同点是**分母是“任务”不是“动作”**，而任务是多步的，这些数已经内含开头那条衰减。反推一下：

44%^(1/5) = 85% [算]——它单次动作相当准，输在次数上；同一个“95%”的句子，装在 5 步任务和 100 步任务上（$0.95^{100} = 0.6\%$）是两个完全不同的系统。宇树能核到的只有自陈的 64 个任务与约 2,500 小时真机数据 [厂]，**逐任务成功率、损失函数形式、是否分块均未公布** [未]。

于是行业当前的主流组合是：

> 模仿学习 → 拿到一个“会做这个任务”的初始策略（快、省数据、可预测）
> 强化学习 → 在真机或仿真里从失败中继续改进（慢、贵、能超过演示者）

（宇树公开的运控工序是纯 RL（第 21 章），操作类走的是模仿学习 + 开源数据集（第 32 章）。这两条在它家产品里是分开的两条线。）

类比，以及它在哪儿破

类比：模仿学习像看菜谱学做菜。看一百遍之后你会做了——但你看的所有菜谱都是“成功过程”，所以第一次火大了你不知道怎么办。

这个类比在哪里破，破得很有教育意义：人类看菜谱能学会，是因为你**已经在厨房里生活过很多年**——你懂“火大了”是什么感觉，你会自己补救。而模型的先验只有它那几百万条演示。

这就是“为什么大模型对机器人有用”的全部理由：一个在互联网文本与图像上预训练过的视觉语言模型，它的先验里包含“杯子倒了水会流出来”“纸杯捏用力会瘪”。模仿学习最缺的正是这种**从任务之外借来的常识**。

于是第 30 章的那张架构图就出现了：不是“用大模型控制机器人”，而是“**用大模型补上模仿学习缺的那个先验**”。

一个可检验的说法：模仿学习的两大结构性限制是复合误差（P = (1−ε)^N，ε=5%、N=100 时 0.6%）与多峰分布（均方误差的极值点是条件均值 $\pi^* = E[a|s]$，两峰对称时正好落在两峰正中那个不存在的动作上；换绝对损失得中位数、换交叉熵得众数才贴回峰）；分布建模之所以行，

是因为网络输入里多了一个随机量——选峰搬进抽样，剩下的映射是单峰的；动作分块一半被硬件逼出来（$H \cdot \Delta t \geqslant T_infer$，120 ms ÷ 20 ms ⇒ $H \geqslant 6$ 步），一半是为了冻结块内的峰选择，换来的是块内开环与 800 ms 量级的反应延迟；而模仿学习的上限是演示者水平、且在模型自己走出的状态上没有梯度。

证据等级：复合误差、分布偏移、DAgger、行为克隆、L2→均值 / L1→中位数 / 交叉熵→众数、流匹配与扩散的“噪声→动作”变换是学界标准内容 [行]；0.6%、60%、85%、17%、12.6 ms、$K \times (2P_迭代 \div 带宽) \leqslant$ 节拍 那条不等式是 [算]；**而 12.6 ms 的输入（63 ms/块、5 去噪步）本身是二手**——官网对带桌面 UA 的请求返回 429，本书未读到原文 [推]，所以这条除法只支持量级；12 GB、120 ms、288 ms、41.6 GB/s 复用第 26 章的带宽算术 [算]（**内存带宽官方未公布，位宽是本书假设** [未]）；Δt = 20 ms 来自官方训练配置 `sim.dt 0.005 × decimation 4` [厂]；33 ms 来自 `xr_teleoperate` 的 `--frequency` 默认 30.0 与 LeRobot 数据集 fps = 30 [厂]；63 ms/块、5 去噪步、action horizon 40、ALOHA 单任务约 50 条演示来自各家公开仓库、论文与产品页 [查]；44%、40%、56% 这三个数**已从本书删掉**（2026-10-09 复核：读不到厂商原文，Figure 新闻索引里也没有 Helix 2.5 这一条）；**宇树操作策略的损失函数、是否分块、逐任务成功率未公布** [未]。

可核对清单：

- ☐ 自己算一次 0.95^{100}、0.99^{100}、0.95^{10}，确认 0.6%、37%、60%；想清楚最后一条比第一条好在哪、代价是什么
- ☐ 先拿**对称**两峰（±0.2 m 各一半）算一次：L2 给 0，L1 在整个 $[-0.2, 0.2]$ 上都是最优（含 0）——亲手确认“对称时区分不出来”；再拿 60/40 的偏权重算一次，L1 才会跳到 −0.2、L2 给 −0.04
- ☐ 用 6B × 2 字节 ÷ 100 GB/s 复算 120 ms、除 20 ms 确认 $H \geqslant 6$，再用 63 ms ÷ 5 复算 12.6 ms，看这两个数量级差在哪块板上
- ☐ 找一份 Diffusion Policy 与 ACT 的公开实现，看它们的输出头分别是什么形状；再看任何一个策略项目，问两句：标签分布单峰还是

多峰？评测任务多少步？

下一章：本章说大模型能补上先验。下一章把全行业收敛出的那张架构图画出来——四个部件，三个可替换的选择，以及一个所有人都同意的分工。

第 30 章

VLA 的架构共识：主干 + 动作专家 + 动作分块 + 异步双系统

一个行业在两年内收敛到了同一张图

2023 年，“机器人 + 大模型”有十几种做法。到 2026 年，公开的系统几乎全部长成同一个形状：

```
视觉与语言输入
      ↓
① VLM 主干（2B–7B）          ← 负责“看懂场景、理解指令”
      ↓ 语义特征 / 子目标
② 动作专家（80M–1B）          ← 负责“生成动作”
      ↓ 一段动作块（horizon 20–50 步）
③ 动作分块 + 异步双系统       ← 负责“慢想快做”
      ↓
④ 关节层执行（第 12 章那条链）
```

这不是某家厂商的设计，是**全行业独立收敛的结果**。本章要解释的是：为什么收敛到这一张图，以及每个部件为什么长这样。

最大的限制门槛还是AI方面不太够，机器人AI的能力在全球范围都没有达到一个像初代ChatGPT那样的水平。

王兴兴 — 界面新闻（同上，来源 D） — 2024-08-21（直接引语（专访））

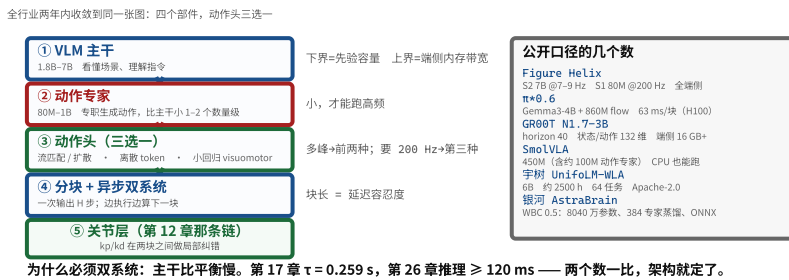


图 30-1　VLA 的四个部件与三选一的动作头（各家主干尺寸标在图上）

部件一：VLM 主干，为什么是 2B–7B 这个尺寸

公开可查到的主干选择（都是各家自己公布的）：

```
智元 GO-1          InternVL2.5-2B + MoE（隐动作专家 + 动作专家）
银河通用 GraspVLA   InternLM2-1.8B + 冻结的 DINOv2/SigLIP 视觉编码
银河通用 NavFoM     Qwen2-7B
Physical Intelligence π*0.6    Gemma3-4B + 860M flow 动作专家
宇树 UnifoLM-WLA-1.0           6B 全身协调基座（官方口径）
```

为什么不是 70B，也不是 100M？ 上下界都是算出来的。

上界由第 26 章那条带宽算术给。 权重每翻倍，单步时间下限翻倍。70B × 2 字节 = 140 GB，在 100 GB/s 的端侧模组上是 1.4 秒一步——**1 Hz 都不到，而第 26 章已经说明这台机器的语义节奏需要 5–10 Hz。**

下界由“先验容量”给。 第 29 章说过，大模型对机器人的价值是**它在互联网上学到的常识**。常识需要参数装。100M 的模型学不会“纸杯捏用力会瘪”。

于是 2–7B 是这两个约束之间唯一存活的窗口——**而且它是被硬件（端侧内存带宽）卡出来的，不是被算法选出来的。** 这一点值得强调，因为它意味着：**当端侧带宽翻倍（Orin → Thor），主流模型尺寸也会跟着翻倍。**这条预测是可检验的。

部件二：动作专家，为什么必须独立

一个自然的问题是：为什么不直接用 VLM 输出动作？语言模型能输出文本，为什么不能输出关节角？

因为**输出的统计性质完全不同**。

语言是离散的、序列化的、每个 token 有明确词表。关节角是连续的、多维同时的（23 个关节一起）、而且**多峰**（第 29 章）。让自回归语言模型逐 token 吐 23 个浮点数，会得到一个又慢又错的东西。

所以全行业的做法是：**VLM 只出语义，另配一个小网络专门出动作**。这个小网络叫动作专家（action expert）。

它的规模很有意思：

```
π*0.6       主干 Gemma3-4B  +  动作专家 860M
SmolVLA     总 450M 一手；主干与专家如何分栏未公布 [未]
Figure Helix S2 = 7B VLM    +  S1 = 80M visuomotor
GE-Act      160M 自回归动作解码器 [查]（原稿曾写 flow-matching，已按一手收回）
```

动作专家比主干小多少，公开口径只有两家给得起。Helix 是 7 B ÷ 80 M ≈ **87 倍**——两个数都是 Figure 自己那篇博客的原话：`S2 is built on a 7B-parameter open-source, open-weight VLM、S1, an 80M parameter cross-attention encoder-decoder transformer、Using just one set of neural network weights (7B for System 2, 80M for System 1)、operating at 7-9 Hz for scene understanding、continuous robot actions at 200 Hz.` [查]。π*0.6 是 4 B ÷ 860 M ≈ **4.7 倍**，但这组数**本书没读到原文**（官网对带桌面 UA 的请求返回 429），按图例只能记 [推]。SmolVLA 这一家算不出这个倍数：一手只给总规模 `4.5亿参数`（450 M）与动作头是 Flow-Matching Transformer，主干与专家如何分栏**没有公布** [未]——旧稿拿 100 M ÷ 450 M 当“专家 ÷ 主干”，是把总数当成了主干。所以“动作专家比主干小一到两个数量级”这句通例不成立：**够得着一个数量级以上的**

只有 Helix 一家，另一家不到 5 倍；本书按这个事实重写判据——动作专家要小得多，是为了让它能跑在高频（下面讲），但“小多少”目前没有跨厂商的通例可给。

图纸上还缺第三方：状态机与行为树

上面把这一层叫“分工”：主干慢、管语义；动作专家快、管高频动作。但**一张真正在跑的架构图里还有第三方**，名字很朴素——有限状态机（FSM：把机器写成若干状态加若干迁移条件，任一时刻只处在一个状态里）与行为树（BT：把任务写成一棵只会向下走的树，叶子是条件与动作，序列节点要求全成、选择节点要求至少一个成，每个 tick 从根重新走一遍）。第 00 章那张六层表给这一行写了 1–10 Hz，本书前 29 章没有挖过它。这一节补三跳：**它为什么必须以某个速率存在、它凭什么能被静态界定、它做不到什么**。

第一跳，速率：不变量必须由不低于“它被违反的速率”的那一层守着。第 26 章算过端侧节奏下限：12 GB 权重 ÷ 100 GB/s ⇒ 一次决策 120 ms；第 16 章算过这台机器的窗口：`τ = √(0.66 / 9.80665) = 0.259 s`，延迟占到 τ 的 39%（约 100 ms）就已经明显伤及稳定。两条一起就是一条不等式：**如果“不许倒”这条不变量交给 VLA 自己去检查，最坏情况下它每 120 ms 才看一眼，而这一个 120 ms 已经是要留预算（100 ms）的 1.2 倍** [算]。把同一条不变量交给 50 Hz 的任务层——官方部署配置 `deploy_real/configs/g1.yaml` 的 `control_dt: 0.02` 就是这个节拍 [厂]——最坏发现延迟 20 ms，剩下约 80 ms 才是真正用来“想清楚之后怎么动”的。**同一台机器、同样的模型，两种分层，差的不是算法好坏，是检查速率与被违反速率的大小关系**。这正是第 12 章那条级联判据（内环带宽必须是外环的 5–10 倍）朝上再用一遍：**守不变量的那一层，必须比“能把不变量搞坏”的那一层快**；否则它会把下层当成“应该会照做”的假设，而这层假设在第 22 章已经被真机打破过一次。

第二跳，凭什么可界定：这两样的最坏情形能在纸面上算出来。 行为树的控制流是自上而下一棵树，一次 tick 的工作量上界 = 根到叶的最大深度 × 单节点最坏耗时；深度是你画图时定下的常数（带 `RepeatN` 这类装饰器时把重复次数乘进去，仍是纸面上算得出的常数）。状态机一次 tick 只查当前状态那一行迁移表。**学出来的策略给不了这个承诺**：它的时延随 token 数、KV 缓存长度、批大小变（第 26、34 章那两笔账），部署前给不出静态上界。工程后果是两种完全不同的验证方式：树与状态机可以**穷举迁移**、可以证明“不存在从正常态一步进入危险态的路径”（模型检查那一套吃的就是这个）；而策略网络只能**统计**地报成功率——第 38 章那条 `0.99^N` 复合衰减正是它的另一面：没有静态上界，就只能把长链切段、每段末尾设人工或显式检查点。**所以这一层不是“更聪明的模型”能替换掉的：它提供的不是智能，是可证明的边界。**

顺带一条维护成本的量级 [行]：n 个状态的 FSM，迁移表上界是 n^2，新加一个兜底行为要给每个状态都补一条边；行为树把这件事变成 O(n) 的优先级选择（选择节点从上往下，第一个可用的就执行），代价是**可读性**——树的语义靠图和注释，不靠状态图那样一张完备表。这就是小项目用状态机、跨团队的产品用树的现实原因。

这条“小”不是审美，是一条被钉死的上界。 第 26 章那笔带宽账在这里反过来用：**一次前向要把“这一步需要的那部分权重”读一遍**，所以每步耗时下限 = 2P_每步 ÷ 带宽（FP16，2 字节/参数）。**这里的 P 必须是每步都要重读的那部分，不是模型总参数**——带 KV 缓存前缀的流式模型只重算专家那一份（本节下面 π0 那段有一手原话，本书上一版正是把总参数当成了它）。要满足节拍，就得让 K 步挤进一个周期：

```
参数上限 = 节拍 ÷ 步数 × 带宽 ÷ 2 字节          （带宽取第 26 章的 100 GB/s 那一档）

  50 Hz 一拍（20 ms）、K=10  ⇒ 每步 2 ms   ⇒ P ≤ 100 M
  50 Hz 一拍、K=5            ⇒ 每步 4 ms   ⇒ P ≤ 200 M
  200 Hz 一拍（5 ms）、K=1   ⇒ 每步 5 ms   ⇒ P ≤ 250 M
  200 Hz 一拍、K=10          ⇒ 每步 0.5 ms ⇒ P ≤ 25 M
```

把 Helix 那两条一手数放进去，它自己就解释了。 S1 是 `80M parameter cross-attention encoder-decoder transformer`，而部署那一节写的是 `S1 executes as a separate real-time process, maintaining the critical 200Hz control loop required for smooth whole upper body action` [查]——**每拍一次前向**是 Figure 自己给的。于是那条带宽式子在这里要倒过来用：不是猜它的步数，而是算它对**硬件**提的最低要求。

```
一次前向要读完 S1：80 M × 2 B = 160 MB
200 Hz ⇒ 每拍 5 ms
所需最低带宽 = 160 MB ÷ 5 ms = 32 GB/s            [算]
若 S1 是个 10 步迭代采样器 ⇒ 320 GB/s              [算]
```

`Helix's training design enables efficient model parallel deployment on Figure robots, each equipped with dual low-power-consumption embedded GPUs` [查]，芯片型号与带宽原文未给 [未]；而同一页给 S1 的定性是 `a simple transformer-based visuomotor policy`，全篇没有为它配任何迭代采样器 [查]。**32 GB/s 这条下界与“simple”那句话互相印证**：S1 能做到 200 Hz，不是因为那张卡跑得快，而是因为它的**一次**前向就出动作。反过来说，本书上一版在这里写的“5 ms 预算放得下 3 次前向，所以绝不可能塞进十步迭代”是**拿自己假设的 100 GB/s 当厂商事实**——嵌入式 GPU 的带宽一旦不是 100 GB/s，那条“不可能”就同时站不住，而它本来也不该由带宽来判。

再看 Figure 怎么处理“慢的那一半”。 原话是 `S2 operates as an asynchronous background process, consuming the latest observation ... It continuously updates a shared memory latent vector that encodes the high-level behavioral intent`，以及 `The asynchronous execution model allows both processes to run at their optimal frequencies` [查]。两条进程之间**没有节拍同步**，靠一个共享隐向量交接。这就把第 31 章那条 $H \cdot \Delta t$ 换成了另一种东西：不是“一次推理管多少拍”，而是“快环允许拿着多旧的语义继续动”——`During training, we add a temporal offset between S1 and S2 inputs. This offset is calibrated to match the gap between`

`S1 and S2's deployed inference latency` [查]，他们连训练时的时序偏移都按部署延迟校准过，为的就是让“旧”这件事在训练里就见过。**这才是双系统的真实接口：一个带时延的共享变量，而不是一条总线上的握手。**

再拿 π0 自陈的数做一次外部核对——这一次核对把我上一版的推理推翻了，所以把它连错带改一起印出来。 上一版本节写过一条“三数闭合”：3.3 B × 2 字节 ÷ 3.35 TB/s = 1.97 ms 一步，十步 19.7 ms，正好塞满 50 Hz 的 20 ms 一拍，“用掉 98%”。**那条推理错在三个地方，而三个地方都是我把厂商没说过的话补成了自己的假设。**

一，那张卡。 我写的是“它跑在服务器卡上（H100 的 HBM3 约 3.35 TB/s）”。原文是 `The operations were timed on an NVIDIA GeForce RTX 4090 consumer-grade GPU`、表题 `TABLE I: Inference time of our model on a NVIDIA GeForce RTX 4090 GPU` [查]——**是 4090，1008 GB/s** [行]，我把带宽换了 3.3 倍，于是“1.97 ms”本身就是凭一个没读到的硬件算出来的。

二，权重不是每步都读。 原文两处说得非常清楚：`To speed up inference (which requires multiple forward passes of the action expert), we downsize the action expert to { width =1024, mlp_dim =4096}, resulting in a parameter count of ~300M` [查]，以及 `each step requires running a forward pass on the tokens corresponding to the noisy actions (the keys and values corresponding to the observation are cached)` [查]。**迭代只重算那 300 M，3 B 的主干作为前缀算一次、KV 缓存住。** 我那把 3.3 B 乘十遍的式子因此用错了对象。

三，预算不是 20 ms。 原文：`for all other robots, which run at 50Hz, we run inference every 0.5 seconds (after executing 25 actions)` [查]。50 Hz 是**执行**节拍，不是**推理**节拍；一次推理摊在 25 拍 = 0.5 s 上。

而论文附录 A-D 直接把分栏计时给了出来 [查]（3 路相机，4090）：

环节	耗时
image encoders	14 ms

observation forward pass（主干前缀，只算一次）	32 ms
x10 action forward pass (flow)（十步**合计**）	27 ms
network latency（若机外）	13 ms
total on-board inference	**73 ms**
total off-board inference	**86 ms**

14 + 32 + 27 = 73，这张表自洽 [算]。现在把两边都摆出来：

```
按我那条错式子：3.3e9 × 2 B ÷ 1008 GB/s = 6.55 ms 一步 ⇒ 十步 65.5 ms
实测十步合计 27 ms — 比“每步读全量”的下限还低 2.4 倍 [算]
按真实架构：前缀一次 3.0e9 ⇒ 5.95 ms；十步 × 0.3e9 ⇒ 每步 0.595 ms
            带宽下界合计 11.9 ms，实测 73 ms 是它的 6.1 倍 [算]
预算：0.5 s ⇒ 73 ms 用掉 14.6 %（机外 86 ms 用掉 17.2 %）[算]
```

“实测落在自己推导的下限之下”这一条，本身就是检查式子的办法：27 ms < 65.5 ms，说明那十步里读的不是 3.3 B。而正确的下界只有 11.9 ms，说明真实系统离带宽极限还有 6 倍余量——余量被注意力、启动开销和 50 个动作 token 的序列吃掉了。**我上一版把 98% 当成“设计闭合”，其实是把一个错硬件上的错式子的巧合当成了证据**。

改完之后，本章真正要的判据反而更硬，而且这次每一环都有原话：**主干可以大，是因为它要么不必每拍都跑（π0 每 0.5 s 推理一次、Helix 的 S2 干脆是异步后台进程），要么干脆不在控制环里（它的输出只是一个共享隐向量）；动作专家必须小，是因为迭代只重算它——`To speed up inference (which requires multiple forward passes of the action expert)` [查]，厂商自己给出的下调理由就是这个**。“比主干小一到两个数量级”是这条不等式的副产品，不是设计偏好：π0 是 3.0 B ÷ 0.3 B = **10 倍** [查]，Helix 是 7 B ÷ 80 M = **87 倍** [查]，两家的差值正好跨了一个数量级。

第三跳，它做不到什么。一，它不理解任务：迁移条件是人写死的谓词，画面里出现一个没被写进谓词的情况，它不会绕开，只会留在原状态或掉进兜底。宇树的协议里有 `mode` 字段与超时回落（第 27、43 章引过这些字

段 [厂])，而行业里同类的兜底是阻尼模式（damped）：把关节变成阻尼器 [行]——**那是“摔得软一点”，不是“恢复”**；至于宇树内部到底是一棵状态机、有几层、谁来仲裁，官方从未公布 [未]。二，**它的节拍不能无限提**：树读的是状态估计的输出，而估计器自己的带宽在那一层之下（第 24 章），把 tick 提到 1 kHz 只是把噪声更快地传下去。三，**它跟策略之间必须有一个仲裁协议**：谁能否决谁、否决之后由谁恢复、恢复的判据是什么。这三问本书只能给行业通例（主干出语义、动作专家出高频动作、任务层出许可与超时）[行]，给不了厂商事实。

这一格为什么值得占一页：整本书在前面把“从一句指令到一个 MOSFET 的关断”接成了链，而链上“指令怎么变成许可”那一跳一直是空的。补上之后，第 31 章那条 `H·Δt ≥ T_infer` 才有承接者——**块内那 800 ms 里机器不是没人管，是被一棵 50 Hz 的树管着；它管不了新情况，但它管得了那几条不许被违反的东西**。

部件三：动作头三选一

动作专家怎么“生成”一段动作？三条路，各有代表：

一、流匹配（flow matching）/ 扩散（diffusion） 从噪声出发，迭代若干步把它“推”成动作。代表：π 系、GR00T、GraspVLA、GE-Act

不属于这一类：一手原文写它是 `a lightweight 160M-parameter autoregressive action decoder`——**autoregressive**（自回归解码器）[查]，把它列在流匹配代表里是原稿的分类错误，就此更正。宇树 org 下**另一族**仓库 `unifolm-wla` 的结构记录里有 `unifolm_wla/model/modules/action_model/flow_matching_head/` 这组目录名 [厂]；**WMA-0 自己的文件清单里没有它**（旧稿把两族混成一族，本轮按 `research/humanoid-stack-specs.md` 的逐仓库清点改正）。两家的官方文字都没确认动作头形式 [未]——本书只在“仓库里有这个文件名”这一层引用它。 优点：分布建模好、动作平滑。缺点：**要迭代多步，每步一次前向**（严格说是每步一次“专家侧”前向：主干那部分作为前缀算一次并缓存 K/V）。扩散可能几十步，流匹配通常 5–10 步。**π0 给出一手实证**：论文

原话 `10 integration steps`、`3.3 billion parameters`，动作时域 `H=50`、控制频率 50 Hz [查]（arXiv 2410.24164）。**一条一手能核的样本**：论文《Running VLAs at Real-time Speed》（arXiv 2510.26742，Dexmal 与 StepFun）报 `after the 10 denoising steps`，主干是 `a vision encoder SigLIP with 400M parameters and a large language model Gemma with 2.6B parameters`，两个输入视角下 `achieve a latency of 27.3 ms`、硬件是 `under a single consumer RTX 4090 GPU` [查] ⇒ 每步 27.3 ÷ 10 = 2.73 ms。这个数后面第 29 章要用来对照端侧的带宽账。

二、**自回归离散 token** 把动作量化成词表，用语言模型那套逐 token 生成。代表：π0-FAST、GO-1 的隐动作。 优点：复用语言模型的全部基础设施（训练、量化、KV 缓存）。缺点：序列长、误差会累积。

三、**小型交叉注意力视觉运动网络** 不用生成模型，直接一个 80M 的回归网络，输入是 VLM 的特征 + 本体状态，输出是动作。代表：Figure Helix 的 S1。 优点：**最快**。缺点：分布建模能力弱（多峰任务会掉进均值陷阱，第 29 章）。

判据：任务多峰且允许慢 → 流匹配；要复用 LLM 工具链 → token；要 200 Hz 控制 → 小回归网络。

而这三条经常被组合使用——这正是下一个部件存在的理由。

部件四：异步双系统，因为物理要求它

现在把第 26 章与第 17 章的两个数放在一起：

```
VLM 主干推理：约 100–200 ms 一次    （带宽决定）
平衡需要的反应：约 10–100 ms 一次    （τ = 0.259 s 决定）
```

主干比平衡慢。 所以任何“让 VLM 直接输出关节指令”的架构，物理上就不能工作。

全行业的解法是**双系统 + 异步**：

```
低频系统（S2 / 语义规划）：5–10 Hz，出“下一步该做什么”
高频系统（S1 / 动作跟随）：50–200 Hz，出“这一毫秒关节该怎么动”
两者并行运行，不互相等待
```

公开口径的实例：

```
Figure Helix：S2 = 7B VLM，7–9 Hz；S1 = 80M visuomotor，200 Hz，
              完全端侧（Figure Helix：`runs entirely onboard embedded
low-power-consumption GPUs`） [查]
π*0.6        ：50 Hz 分块；H100 单卡 5 去噪步 63 ms/块，
              机器人端只执行，推理在 GPU 侧（可 websocket 远程） [推]（官网
429，未读到原文）
GO-2（智元）  ：官方口径为 ACoT-VLA，显式动作推理 + 动作思维链 +
              异步双系统（低频语义规划 / 高频动作跟随）            [推]（只见
到媒体转述，智元官方页本轮未逐字读到）
```

“异步”这个词是关键。 同步的做法是：想 → 等 → 做 → 想。异步的做法是：**一边做上一块，一边算下一块。** 于是延迟被藏在执行时间里。

这需要一个前提：**动作块要足够长，长到能覆盖一次推理的时间。** 于是有了第 31 章那个绕不开的 trade-off。

把四个部件接起来：一次完整的推理

以 GR00T N1.7-3B 的公开规格为例（官方仓库自陈 [查]；**这个名字里的 3 B 是整个检查点的参数，它加载的主干是 `nvidia/Cosmos-Reason2-2B`**，下面这张表按主干与头分别记）：

```
主干  Cosmos-Reason2-2B（Qwen3-VL 架构）
动作头 flow-matching DiT，16 层
action horizon = 40        ← 一次输出 40 步
状态/动作维度 = 132         ← 全身关节 + 末端位姿都塞进一个向量
动作语义 = 相对末端位姿      ← 输出的是“增量”，不是绝对位姿
部署要求 = 端侧 16 GB 以上（RTX 4090 / Jetson AGX Thor / Orin / DGX
Spark）
导出 = ONNX / TensorRT
```

最后两行是我最想让读者看到的。 一个能导出 ONNX、能在 16 GB 的嵌入式模组上跑、动作是“相对末端位姿”的 3B 模型——**这就是 2026 年“人形大脑”的实际工程形态。**不是云端千亿参数，是本地三 B、可量化、输出增量。

而“输出相对量”这个决定，与第 21 章 `action_scale = 0.25` 是同一件事：**让策略学“该改多少”而不是“该到哪”**，因为绝对位姿对初始条件与机型差异太敏感。

宇树在这张图上的位置

把上面四条与宇树对照：

部件	行业共识	宇树公开口径
主干	2–7B VLM	UnifoLM-WLA-1.0 为 6B（官方仓库自陈）[厂]
动作专家	80M–1B	未单独公布 [未]
动作头	三选一	未公布具体形式 [未]
分块 / 双系统	普遍采用	未公布 [未]
数据	—	约 2,500 小时真机、单模型 64 任务、Apache-2.0、2026-09-28 放 Base 权重 [厂]
部署	端侧 ONNX/TensorRT	未公布 [未]

这张表右列有一半是“未公布”。 而“未公布”这件事本身值得记一笔：**宇树在硬件上极其透明（扭矩常数给到五位有效数字），在模型上相对保守。** 对比银河通用——它把 GraspVLA 的论文、代码、数据方法都公开了[查]。

两家透明度的差别，可能恰好反映两家的商业位置：一家卖本体，一家卖“能干活的系统”。**卖本体的公司不需要证明大脑有多好；卖能力的公司必须证明。**

类比，以及它在哪儿破

类比：双系统像人的“深思”与“直觉”——想菜谱时用慢思考，接住掉下来的杯子时不用。

这个类比在哪里破，而且破在关键处：人的直觉系统不是“另一个小模型”，它与慢系统**共享同一套感知与记忆**。而机器的 S1 只看得到 S2 喂给它的那点特征。于是当 S2 判断错场景（把“泡沫杯”看成“玻璃杯”），S1 会**用正确的力气去做一个错误的事**——它会以捏玻璃杯的力去捏泡沫杯，而且不会怀疑。

这就是“上下系统之间没有双向纠错通道”这个架构缺陷的具体表现。行业目前的答案是加更多数据，而不是改架构。第 34 章的世界模型（世界模型 = 一个能预测「这么做之后会发生什么」的模型；它的四种用法在第 34 章被拆开），一部分动机就是想给这条链补一个“我能预见我做错的结果”的回路。

一个可检验的说法：全行业在两年内收敛到“VLM 主干（2–7B）+ 独立动作专家（80M–1B）+ 动作头三选一（流匹配/离散 token/小回归网络）+ 动作分块 + 异步双系统（低频语义 5–10 Hz、高频动作 50–200 Hz）”这一张图；主干尺寸的下界由“先验容量”决定、上界由端侧内存带宽（第 26 章）决定，因此当端侧带宽翻倍时主流尺寸会随之上移；动作专家必须独立且小，是因为它要跑在比主干高 5–40 倍（5–10 Hz 对 50–200 Hz 两个端点之比）的频率上，而“异步”成立的前提是慢的那一半不必每拍都跑——本节的一手取证是两种形式：0.5 s 推理一次、执行 25 拍（π0），以及一个后台进程只留一个共享隐向量给前台（Helix）。上一版把这里写成“19.7 ms 用掉 20 ms 的 98%”，是错硬件、错权重范围、错节拍的三重假设，已按论文自己的 Table I 撤回并改写为 73 ms ÷ 0.5 s = 14.6%。

证据等级：各家主干型号、参数、horizon、部署门槛里，已逐字读到原文的记 [查]（Helix、GR00T、GAIA-2、DROID、π0、Cosmos 3 Edge，逐条原文见 `research/other-vendor-firsthand.md`）；π*0.6 那一组的 63 ms/5 步/50 Hz 是 [推]（官网 429 未读到）。**[厂] 只留给宇树，别家一律不用**；“上界由带宽决定”是第 26 章推导的复用 [算]，但**式子里的 P 必须是每步重读的那部分权重**：π0 的十步只重算 300 M 专家（前缀的 K/V 被缓存、那 300 M 是论文自己写明的“ To speed up inference ”一举动，两处均一手）；该论文 Table I 的 14 + 32 + 27 = 73 ms 与“每 0.5 s 推理一次”也是一手 [查]；于是 32 GB/s、320 GB/s、0.595 ms 一步、下界 11.9 ms、余量 6.1 倍、占预算 14.6%、65.5 ms、2.4 倍、87 倍、10 倍 全部是 [算]；**上一版用 H100 的 3.35 TB/s 与“每步读全量 3.3 B”推出的“19.7 ms 用掉 98%”已整段撤回，撤回理由与反证都印在正文里**；**宇树的动作专家规模、动作头形式、是否分块、是否双系统、部署方式全部未公布** [未]；“卖本体 vs 卖能力”的透明度解释是本书推断。

可核对清单：

- ☐ 打开 `NVIDIA/Isaac-GR00T` 仓库，找到 action horizon、state/action 维度、主干模型名，与本章那张表对
- ☐ 打开 `unitreerobotics/unifolm-wla`，逐字抄下它自己声称的参数与数据量，并检查 LICENSE 文件里写的是什么
- ☐ 用第 26 章那条带宽算术，把主干换成 14B 再算一次单步下限，看它掉到几赫兹
- ☐ 找一份你看到的“人形演示”视频，问：这个动作是 5 Hz 出的还是 200 Hz 出的？判据是它有没有毫秒级的接触反应

下一章：本章说“异步”要求动作块足够长。下一章把这个“足够长”变成一个可算的 trade-off——它会同时解释为什么 VLA 抓不了正在滚动的东西，以及为什么这不是模型不够大。

第 31 章

延迟墙：5 Hz 的模型怎么驱动 200 Hz 的手

把上一章的“异步”变成一个数

第 30 章说：主干慢（100–200 ms），动作专家快（5–20 ms），所以一次要输出一段动作块，边执行边算下一块。

现在把“一段”变成一个可优化的量。设：

```
T_infer   一次完整推理时间（主干 + 动作头）        典型 60–200 ms
H         动作块长度（步数）                        20–50 步
Δt        每步执行时长                              1/50 s = 20 ms（第 21 章口径）
H·Δt      一块覆盖的未来时长                        0.4–1.0 s
```

约束一（异步能成立的前提）：

```
H · Δt  ≥  T_infer
```

一块必须至少撑到下一次推理完成，否则机器人会“算完了但没动作可用”，只能原地停住。

代 GR00T 的公开数：H = 40、Δt = 20 ms ⇒ 一块覆盖 800 ms [查][算]。而 π*0.6 公开口径是 63 ms/块（H100 单卡 5 去噪步）[推]（官网 429，本书没读到原文）——**800 ms 对 63 ms，差一个数量级**。这说明两家的部署位置完全不同：一个在端侧（慢，所以块要长），一个在 GPU 服务器上（快，块可以短）。

约束二（对扰动的反应速度）：

机器人在执行一块动作的中途，世界变了（杯子被挪走、有人撞了一下）。它什么时候能反应过来？

```
最坏反应延迟 = H · Δt          （刚发完一块，得等整块执行完才换）
平均反应延迟 = H · Δt / 2
```

于是那个 trade-off 出现了，而且它是硬对硬的：

```
块要长  →  异步不卡壳、动作平滑、轨迹一致
块要短  →  反应快、能处理意外

同一套架构里，这两个要求指向相反的方向。
```

代具体数（Δt = 20 ms）：

H	一块覆盖	平均反应延迟	能满足 H · Δt ≥ T_infer（200 ms）吗
10	200 ms	100 ms	勉强（端侧 200 ms 推理时刚好）
20	400 ms	200 ms	是
40	800 ms	400 ms	是，且很宽裕
80	1600 ms	800 ms	是，但已经不可能应对任何意外

（[算]。）

这张表就是本章的全部论点。 一个用 H=40 的端侧 VLA（VLA = Vision-Language-Action，视觉-语言-动作模型，即第 30 章那张图），它的**平均反应延迟是 400 毫秒**。

现在对照第 17 章：人形的倒立摆时间常数是 0.259 秒，摆动相单腿支撑约 200 毫秒。**400 毫秒的反应延迟，比这台机器“必须做点什么”的窗口还长一倍。**

所以 VLA 绝对不能负责平衡。 它只能负责“在秒级时间尺度上做选择”。而这条不是工程经验，是 H·Δt 与 $\sqrt{l/g}$ 两个数一比就看出来的。

机器人干活的时候，我们希望其通信延迟比较低的，如果在北京干活的机器，数据中心在上海或者在内蒙，延迟实在是太大了。……未来在工业领域大规模运用人形机器人时，工厂里面可以有个分布式的服务器，所有的机器人直接连接工厂里的局部服务器就好了。服务器的安全性、通信延迟是可以接受的。

王兴兴 — 王兴兴 2025 世界机器人大会主旨演讲（新浪“机器人前瞻”文字实录） — 2025-08-09（第一人称演讲逐字）

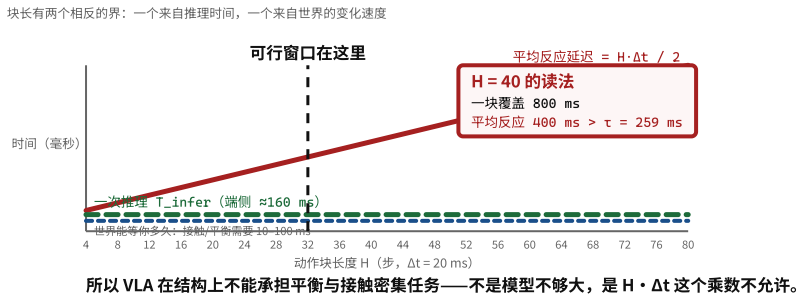


图 31-1　块长的双向界：H · Δt ⩾ T_infer 与反应延迟上限

于是可以解释一批“演示很惊艳、现场不行”

把这条 trade-off 拿去照几个真实现象：

一、为什么演示里机器人从不被干扰。 因为任何在 400 毫秒内发生的意外，它的策略结构上无法响应。演示者知道这一点，所以演示里没有意外。

二、为什么抓“静止的物体”和“正在滚的物体”是两个难度级别。 滚动的球需要连续修正，而修正频率必须高于球位置显著变化的时间尺度（约 100 毫秒）。H=40 的策略每 800 毫秒才看一眼世界——**它看到的永远是“过时 800 毫秒的球的位置”**，然后按那个位置去抓一个已经不在这儿的球。

三、为什么“插拔”类任务成功率特别低。 插拔是**接触密集 + 需要毫秒级力反馈**的任务。第 25 章说过 μ 不可测、第 24 章说过没有足端力传感。而 VLA 的 400 毫秒反应延迟，意味着它**在结构上不可能做导纳控制**（导纳环需要 kHz 级，第 12 章）。所以真实系统都把它拆开：VLA 出“大致位姿”，阻抗控制器在下面做“接触时的实时调整”。

第四，为什么“动作分块”这个技巧本身会限制泛化。 模型被训练成“输出未来 40 步”，那么它对“这 40 步之后世界是什么样”完全没有表征。长时序任务（先开门、再进去、再拿东西、再出来）需要的是**跨块的记忆与状态**，而这正是分块架构不提供的。第 38 章会回到这一条。

三条出路，各付各的代价

出路一：把 T_infer 压下去。 用更小的主干、更少的去噪步、量化、KV 缓存复用。π*0.6 的 63 ms 是这条路的结果。**代价**：主干变小 → 先验变弱（第 29 章）。你在用“更聪明”换“更快”。

出路二：把推理搬到机外。 机器人只执行，模型跑在 GPU 服务器上（websocket/gRPC）。π 系官方就是这么做的 [推]；同型做法有一手原话可引：NVIDIA 写 `Because the next chunk is ready before the current one finishes, the arm moves continuously` [查]（第 34 章摊开）。**代价**：网络。第 00 章说过延迟占 τ 的比例；而无线网络的**抖动**（第 27 章）比平均延迟更致命。**一条会突然卡 200 毫秒的链路，比一条稳定 50 毫秒的链路危险得多。** 所以这条路要求现场有有线网络或高质量专网——在工厂里是成本，在田里是障碍。

出路三：分层。 VLA 只做“选择”（拿哪个、往哪走），把“怎么做”交给一个高频小策略（第 30 章的 S1、或者第 21 章的 RL 步态）。**代价**：层与层之间的接口是手工设计的，而**接口处会掉性能**。VLA 选了一个 S1 不擅长的动作，就摔。这是当前人形系统最普遍的失败位置之一，也几乎不出现在任何宣传里。

这条不等式有一手实例。 NVIDIA 官方开发者博客写 Cosmos 3 Edge 后训练出的策略在 Jetson AGX Thor T5000 上 `generates each action chunk in about 1.53 seconds`，而 `a single chunk covers roughly 2.13 seconds of robot motion`，并说 `the next chunk is ready before the current one finishes` [查] ⇒ **2.13 ÷ 1.53 = 1.39 > 1**，正好是本章要的那个条件；同一篇给的输出节拍是 `640×540 resolution and 15 Hz` ⇒ Δt = 1 ÷ 15 = 67 ms [算]（第 34 章把这条链完整摊开）。**注意它是在 Thor 的板载高带宽上量的，不能拿去反推第 26 章那些假设档。**

行业目前三条都在用。 而本章的算术说明一件事：**这不是“再等等更大的模型”能解决的问题。** H·Δt ⩾ T_infer 与“反应要快”是一对永久约束，只有硬件（带宽、延迟）会移动它。

断网能不能干活，取决于回路有没有跨无线

出路二要付的代价写的是“网络”。工程师的第一反应是问带宽够不够——**这个问题问偏了。** 要问的是另一件事：把机器人对网络的依赖按层拆开，六层各自问同一句话——**这一环的输入和输出，在不在同一台机器里？**

层	一拍的预算（沿用第 00 章那张分层表）	输入与输出在哪	丢一次会怎样	断网还能干吗
① 关节电流环	50 µs（那一层是 1–20 kHz）	必须在关节壳内；任何一跳出机箱即失效	扭矩纹波、抖	与网络无关
② 给定值下发	1 kHz ⇒ 1000 µs；机内话题 500 Hz [厂]	机内（主控 → 总线 → 关节）	关节按上一拍的给定继续跑	能，但停在旧动作上
③ 全身控制 WBC / MPC	100–1000 Hz [行]	机内	摔	能，只要策略板载
④ 状态估计与标定	50–200 Hz ⇒ 5–20 ms	机内，但对时间戳敏感	漂得更快	能
⑤ 任务与推理（VLA）	1–10 Hz，就是本章的 T_infer（60–200 ms）	**可以跨网——出路二动的正是这一层**	块与块之间接不上	能撑 H · Δt 那么久

⑥ 感知落盘与数采回传	帧级可缓冲；回传是分钟到小时	可跨网、可完全离线	少一帧、少一段录像	能，本地先写盘

（这张表是本章按推理节奏重切的，编号与第 00 章那张六层表不一致（那里的 ③ 是任务与决策、⑤ 是关节伺服）——分层本身是本书的分析工具，不是厂商公布的架构 [未]。但“某一层的输入输出在不在同一台机器里”这个问题跟架构公布与否无关，任何一台机器都能自己问。）

于是那条可核对的结论：**决定断网能不能干活的不是带宽，是控制回路有没有跨过无线这一跳**。①②③ 层跨不出去，不是协议不允许，是算术不允许：

```
50 ms ÷ 1000 µs = 50   ⇒ 一条平均 50 ms 的链路，比一拍 1 kHz 的预算长 50 倍
1 ÷ 0.05 s = 20 Hz     ⇒ 同一条链路能承载的只有 20 Hz 量级的东西
```

（[算]，两个数都来自本章上文那条“稳定 50 毫秒的链路”。）带宽在这里几乎不构成约束——第 27 章那道乘法算过，把 G1 那 23 个关节的全量状态按 1 kHz 搬上一千也只有几 Mbps 的量级 [算]，任何一条无线链路都装得下。**装得下不等于赶得上**。真正的约束是这一跳的时延下限：传播、竞争、排队、重传加起来就是毫秒量级，信道堆到多宽都改变不了这个加总。**这一跳的代价不在账单上，在物理上：带宽可以花钱买，毫秒买不到**。所以“隔着公网闭那三条 kHz 层的环”不是不好，是做不到；反过来 ⑥ 层断网完全无害，④ 层也只是漂得更快而不是停摆，因为它们本来就异步。演示里“信号不好就停下”看着像保守，其实是唯一正确的做法。

那真要让某一跳跨无线，缺的是什么？不是带宽，是**机内的兜底回路**。协议层其实给了这块料：宇树公开的关节协议里有一个 `timeout` 位，含义是一段时间没收到新指令就自动停 [厂]（语义边界第 37 章正面讲）。**判断一台人形能不能安全断网，问这一句就够：网掉了，谁把关节降到安全态？答案在机内就能断，在机外就不能断**。至于宇树与 H2 的无线规格、有没有 5G 或 UWB 选项、遥操作链路的实测往返时延，官方一项都没有公布

[未]，本书不替它填。行业选项是 Wi-Fi 6/6E、5G/4G、UWB、北斗短报文、边缘网关这几类 [行]，这里只列名字不给参数——那些数字年年变，是读者该自己去附录 E 那张表上填的格子，不是拿来支撑论证的东西。

时间同步为什么要在网线上打戳

第 ④ 层那句“对时间戳敏感”值一次解释。第 23 章算过：1 m/s 的相对运动配上 20 ms 的不同步，会在双目里造出 13.4 像素的伪视差，是那 0.5 像素测距精度的 27 倍。PTP（IEEE 1588，精确时间协议）的机制一句话就够：**网里选一个主时钟，其余节点向它对齐；而关键的时间戳不是应用程序打的，是网卡在帧经过物理层那一瞬间打的。** 这一句为什么重要？再往下问一层：软件打戳要穿过操作系统——驱动、中断、调度、内核锁，每一样都往里加不确定的量，正是第 22 章那六个偷时间的东西；线路层打戳打的是帧出现在线路上那个时刻本身，**上层软件怎么抖都与它无关**。同一件事在机箱里也成立：第 27 章 EtherCAT 的 DC 之所以能把所有关节对齐到亚微秒，靠的不是网络快，而是它把打戳放到了硬件上。

于是同步预算变成一个可以判“够不够”的量，而不是越小越好：

```
第 23 章的伪视差 = 13.4 像素，对应 Δt = 20 ms
若 Δt 缩小 1000 倍（20 ms ⇒ 20 μs）⇒ 伪视差 = 13.4 ÷ 1000 = 0.013 像素
0.013 像素 < 0.5 像素 ⇒ 这一层已经够用，再往下压换不来精度
```

（[算]：只是把第 23 章那个 13.4 除以一千。那个 20 μs 是“如果”，不是任何厂商的实测值；各家是否在人形上部署 PTP，宇树从未公布 [未]。）

这条链也会断，而且断在没人注意的地方：**硬件打戳要求网卡与交换机都支持，消费级设备上常常做不到**，于是一退就退回到软件打戳，第 22 章那六个因素立刻又回到账上 [行]。买相机时没人问这一项，但它决定的正是你的数据能不能对齐。

无线这一跳的分布为什么和有线不一样

最后一条，也是本章那条 trade-off 的延伸。为什么同样是“平均 50 毫秒”，在有线和无线上不是同一件事？因为无线有三项有线没有的机制在专门制造尾巴。

一，**介质共享，所以发之前要竞争**。无线不是拿起线就发，它得听、退避、再发（竞争与退避的参数本书不查，宇树的无线实现也未公布 [未]）；你那一包什么时候出去，取决于此刻谁也在发。二，**误码随信道起伏，于是链路层要重传**。遮挡、多径（同一份信号走几条路到达，互相抵消）、别人家的路由器都在改误码率，所以“这一包成没成功”是个概率事件；没收到确认就重发，一次重发就是多等一个退避窗口。三，**速率自适应**。链路变差时调制阶数自己往下降，同样的字节要占更长的空中时间——这不叫丢包，这叫把每一包都变慢。而屏蔽良好的短距离以太网这三项一个都没有：链路是独占的全双工，误码率几乎恒定，帧坏了就直接丢、重试发生在 TCP 那一层。

这三项的共性是：不整体平移分布，只往右边加尾巴。 所以平均值在这一跳上是骗人的：

```
200 ms ÷ 50 ms = 4                      最坏那一次是均值的 4 倍
200 ms ÷ 259 ms（第 17 章的 τ）= 77%    一个尾巴吃掉整条平衡时间常数的近八成
```

（[算]。）这就是“一条会突然卡 200 毫秒的链路，比一条稳定 50 毫秒的链路危险得多”的确切含义：**尾巴和均值不是同一件事的两个名字，它们由不同的机制产生**。 与第 22 章那条硬实时判据合起来读，你手里就有两把尺子：关节那条环看**最大值**，无线这一跳看 **99 分位**（分位数 = 把观测从小到大排好之后第 99 个百分位那个值）——前者因为每毫秒一次机会、尾巴一定撞得着，后者因为你要回答的是“这一批任务里有多大比例被拖慢”。**本书不给你任何一条无线链路的实测数字** [未]：这两把尺子是量出来用的，不是抄来用的。

一个可以带走的检查方法

以后看任何一个 VLA 系统（论文、演示、产品发布），问三个数：

1. **H 是多少？**（动作块长度。论文通常在实现细节里，演示从不提。）
2. **T_infer 是多少，跑在哪？**（端侧还是服务器；本地还是远程。）
3. **它的任务里，最快的扰动是多久发生的？**

然后自己做那个比较：**H·Δt 与扰动时间尺度之比**。如果块覆盖时长大于扰动时间，那这个任务它结构上做不了——不管演示看起来多好。

这三个数全部是公开的或者可以问出来的。一个不肯公布 H 与 T_infer 的系统演示，等于把它的适用边界藏起来。

类比，以及它在哪儿破

类比：动作分块像下棋时“提前想好接下来五步”。想好了再落子，落子期间不用想。

这个类比在哪里破，破得正好：棋手在“执行这五步”的期间，**眼睛一直盯着棋盘**，一旦对手变招立刻重想。而机器人执行动作块期间，它的观测**要么不看、要么看了也不用**——因为它的策略是“一次输出一整段”，段内没有反馈入口。

这就是所谓“开环执行”（open-loop within a chunk）。它是分块架构的固有属性，不是实现缺陷。而第 34 章会说：1X 那类世界模型评测之所以重要，正是因为它想在**块与块之间**塞进一个“我先想想这么做完会发生什么”的环节。

一个可检验的说法：异步双系统要求动作块覆盖时长 H·Δt 不小于一次推理时间 T_infer，因此块长有下界；而块长同时就是最坏反应延迟，因此块长有上界；两个界夹出的窗口决定了这套架构能对付多快的扰动——以 H=40、Δt=20 ms 计算，平均反应延迟 400 ms，大于人形倒立摆时间常

数 0.259 s 与单腿支撑时长约 200 ms，故 VLA 在结构上不能承担平衡与接触密集任务，只能承担秒级选择；三条出路（压推理时间、搬出机外、分层）各自付出的代价分别是先验变弱、网络抖动、接口掉性能。“搬出机外”这条还要补一句判据：**决定断网能不能干活的不是带宽，是控制回路有没有跨过无线这一跳**——①②③ 层跨不出去，因为一条平均 50 ms 的链路比一拍 1 kHz 的预算长 50 倍、只能承载 1 ÷ 0.05 s = 20 Hz 量级的东西，而跨出去的那一跳必须有机内兜底；⑤⑥ 层本来异步，断网无害。

证据等级：H、Δt、T_infer、“机器人端只执行、推理在 GPU”等具体数来自各家公开仓库/论文 [查]（GR00T horizon 40；π*0.6 63 ms/块、50 Hz 分块；Helix S1 200 Hz / S2 7–9 Hz；GE-Act 200 ms 出 54 步）；400 ms、800 ms、那张四行表，以及断网一节的 50 倍、20 Hz、4 倍、77%、0.013 像素，都是 [算]（除数与被除数全部取自本章与第 17、22、23、27 章已有的数，算式写在正文里）；τ = 0.259 s 与单腿 200 ms 来自第 16–17 章 [算]；`rt/lowcmd` 的 500 Hz 与协议里的 `timeout` 位 [厂]；六层依赖的划分（沿用第 00 章那张表，仍是本书的分析工具 [未]）、PTP/IEEE 1588 的主从时钟与线路层打戳、EtherCAT DC、无线的竞争-重传-速率自适应产生长尾，是网络与实时系统通识 [行]；“开环执行是分块架构固有属性”是控制论判断 [行]。**宇树 UnifoLM 的 H、T_infer、部署方式全部未公布** [未]，所以本章不用它做算术；**宇树与 H2 的无线规格、是否有 5G/UWB 选项、遥操作链路的实测往返时延、机内是否部署 PTP，同样一项都没有一手来源** [未]，Wi-Fi 6/6E、5G/4G、UWB、北斗短报文、边缘网关只作为行业选项清单出现，不给参数；那个 20 μs 是“如果把 Δt 缩一千倍”的演示，不是实测值。

可核对清单：

- ☐ 找一份你最近看过的 VLA 论文，翻到实现细节，抄出 H 与推理频率；如果没写，把这一条记为“适用边界未知”
- ☐ 自己算一次：如果 T_infer = 150 ms、Δt = 20 ms，H 至少取多少？平均反应延迟是多少？（答：H ⩾ 8，平均 80 ms）

- [ ] 拿 50 ms 与 1000 µs 除一次，再拿 1 ÷ 0.05 s 除一次，确认“跨无线的那条环只能到 20 Hz 量级”这句话是算术而不是观点
- [ ] 找一台你手上能查资料的机器人，把它的六层依赖表填一遍：每层的输入输出在不在同一台机器里？网掉了谁兜底？查不到的格子就留空并标 [未]
- [ ] 找一个“机器人抓移动物体”的演示，看它有没有说明控制频率——多数只说“成功抓取”
- [ ] 想清楚为什么“网络抖动”比“平均延迟”危险，以及为什么关节环看最大值、无线链路看 99 分位是两把不同的尺子（把它和第 27 章那条总线抖动的讨论连起来）

下一章：本章说大脑的节奏由硬件决定。下一章去看另一个更硬的约束：让这台机器学会一件事，要花多少钱、多少人、多少块卡。

第 32 章

宇树的 UnifoLM：世界模型-动作与 6B 全身基座

先把三个模型和它们的许可证分清楚

宇树在模型侧公开的是一族叫 **UnifoLM** 的东西。名字相近、时间不同、**许可条款不同**，而这三件事混在一起时最容易出错：

```
UnifoLM-WMA-0     世界模型-动作（World Model + Action）
                  2025-09 发布；许可：CC BY-NC-SA        [厂]
UnifoLM-VLA-0     视觉-语言-动作；**仓库里没有 LICENSE 文件**（本书逐文件核
对）[厂]
UnifoLM-WLA-1.0   6B 参数全身协调基座；官方称约 2,500 小时真机数据、
                  单模型 64 任务；2026-09-28 放出 Base 权重
                  许可：Apache-2.0                       [厂]
```

（以上参数、数据时长、任务数、发布日期均为**厂商在自家仓库 README 里的自陈**，本书如实转述并标注；第三方独立复现结果有限[推]。）

外加一个数据侧的动作：Hugging Face 上约 **200 个 LeRobot 格式数据集** [厂]，以及 `unitree_IL_lerobot` 训练脚本（其 v0.3 起支持 **GR00T 与 π0.5** 策略 [厂]）。

这一族东西合起来说明了一件事：宇树没有自研一个封闭的“机器人 OS + 独家大脑”，而是把自己的机器接进**别人定义的标准**里——数据用 LeRobot 格式、策略支持 GR00T 与 π0.5、训练用 Isaac Lab、通信走 DDS（第 28 章）。

这个选择值得单独评一句，因为它和智元（自研 AimRT / Link-U OS）、银河通用（自带 GalbotSDK）的方向**正好相反**。

后来 MIT 搞了一个开源方案，国内很多机器人基本都用这个方案。那个开源方案用的电机跟我的 XDog 用的电机一模一样。2016 年的时候，一个 MIT 的博士还找我要过硕士论文。

王兴兴 — 《晚点 LatePost》对话王兴兴（新浪转载） — 2025-02-16（首刊 2024-05）（直接引语（问答实录））

为什么“接标准”是理性的

一家卖本体的公司，它的价值主张是“我这台机器便宜、可靠、能量产”。而大脑的价值主张是“我的模型最强”。**这是两种不同的生意，而它们互相不需要对方的核心资产**。

所以宇树的理性选择是：**让别人的大脑能轻松装到我的机器上**。具体做法就是把自己这边全部标准化：

```
数据格式 → LeRobot（Hugging Face 生态的事实标准）
策略接口 → 支持 GR00T / π0.5（学界与工业界的现成权重）
训练环境 → Isaac Lab + 官方 MJCF/URDF（仿真里能直接跑）
通信     → DDS 主题 + 开源 SDK（不要求你装 ROS）
```

这四条合起来的效果是：一个博士生可以在两周内，用宇树的机器、NVIDIA 的模型、Hugging Face 的数据格式，做出一篇论文。而这批论文与开源复现，就是宇树最便宜的营销。

那这条链又靠什么撑住？ 靠一件事：这四层里没有一层是宇树定义的。数据格式是 Hugging Face 定的，策略权重是 NVIDIA 与 Physical Intelligence 定的，训练环境是 NVIDIA 定的，只有“机器能被这些标准接进来”是宇树做的。**所以它买到的东西叫装机量，代价是它的模型能力永远排在别人的路线图后面**——第 33 章会看到更硬的一版：数据分布由买它机器的人决定，不由它决定。

反过来看它的代价：**它放弃了“独家大脑”这个叙事**。所以在“具身智能”的融资故事里，宇树讲不出“我们的模型别人没有”——它只能讲“我们的机器卖得最多、数据回流最快”。

（第 03 章那条募资分配正好印证这个自我认知：42 亿里 20.22 亿投具身模型。**它把一半的钱押在了自己不擅长的那件事上**。而 WLA 用 Apache-2.0 开源，说明它的策略是“追平并接入”，不是“独家领先”。）

许可证不是法律细节，是产品参数

这一节是本章对读者最实用的部分。**CC BY-NC-SA 与 Apache-2.0 的差别，会直接决定你能拿这个模型做什么。**

条款	能商用吗	能闭源衍生品吗	要署名吗	要同许可分享吗
Apache-2.0	能	能	要（保留 NOTICE）	不用
CC BY-NC-SA	**不能（NC = 非商业）**	不能	要	要（SA = 相同方式共享）

所以：

```
UnifoLM-WMA-0（CC BY-NC-SA）→ 你可以读、可以学、可以发论文；
                              你不可以拿它做产品或任何商业验证
UnifoLM-WLA-1.0（Apache-2.0）→ 你可以商用、可以闭源集成、可以改
```

为什么这恰恰是一条产品策略，而不是法务疏忽？因为它是分档的：把“研究用”的东西放 NC（保护生态、要求分享），把“要别人真的装到机器上跑”的东西放 Apache（消除采用摩擦）。

而这件事在中文技术圈几乎没人看。大家看 README 第一行的参数，不看 LICENSE。于是发生的情形是：一家公司拿 WMA-0 做了个 demo 去见客户——**这已经踩了 NC**。

而本书逐文件核对时，抓到一处真正的许可冲突——两份都是官方文件：

```
unifolm-world-model-action:
   LICENSE 文件本体          = CC BY-NC-SA 4.0（非商业）
   pyproject.toml 的 license 字段 = BSD-3-Clause（可商用）
```

（[厂]。）**同一仓库、两份官方文件、结论相反**。而 `unifolm-wla` 的 LICENSE 文件确实存在且是 Apache-2.0，另附 NOTICE 与 THIRD_PARTY_LICENSES.md——**这一项它做得比多数公司好**。

本书的处理方式：以 **LICENSE 文件本体**为准（也就是更严格的那一份），并把冲突如实列出。**理由不是法律判断，是风险不对称**——按宽松那份用，错了要承担侵权后果；按严格那份用，错了只是少用一个工具。

顺带两条同类的「官方文件互相不一致」，本书按同一原则处理 [厂]：

- `unitree_rl_lab` 的许可文件名是 **`LICENCE`（英式拼写）**，不是 `LICENSE`。**自动化许可扫描工具会漏掉它**，于是显示「无许可」。这是一个纯粹由拼写造成的法律状态误判。
- `unitree_rl_lab` 的版本号在三处不同：`pyproject.toml` 写 0.2.1、`extension.toml` 写 0.1.0、`CHANGELOG.rst` 最新条目 0.2.1（2025-07-17）。**你无法从仓库本身确定自己在用哪个版本**。

（2026-10-09 逐个仓库试过四种文件名（`LICENSE` / `LICENCE` / `LICENSE.md` / `LICENSE.txt`）：13 个仓库里 12 个有许可文件，只有 `digital_servo` 四种全 404，限制写在 README 正文里 [查]。「**仓库里没写许可**」**在法律上意味着默认保留所有权利**，不是「随便用」——这一条对任何开源项目都成立，不只是宇树。逐条结果记在 `bible/sources.md` 的 G0 表。）

NC 拦住的到底是哪一步：把条款拆成动作

上一节那张表只回答“能不能”。工程师真正要的是“我做的这一步算不算”。把四个常见动作摊开判一遍（**本节是条款解读，不构成法律意见** [行]）：

① 读代码、跑推理、写论文	→ 可以（BY 要求署名并标注改动）
② 在公司内部拿它做可行性验证	→ 不行，这一条最反直觉
③ 用你自己的数据微调它的权重	→ 产物仍是改编材料，NC 与 SA 一起跟着走
④ 把微调后的权重闭源或发出去	→ SA 要求以相同许可分发，闭源这条路做不到

为什么②也不行？ 因为 NC 判的是**用途指向**，不是“你有没有卖这个文件”。非商业条款的通行理解是“不得主要用于商业优势或金钱报酬”；你跑一次可行性验证，目的是决定要不要把资金投进某个方案——**这就是商业优势**。“只是内部测试、没有分发”这句话救不了你：你分发的是决策依据。

为什么③也不行？ 这是最容易误判的一格。CC BY-NC-SA 4.0 里，微调产物属于**改编材料（Adapted Material）**：ShareAlike 要求它沿用同一许可，NonCommercial 也跟着走。**换了数据、换了动作头都没用——只要权重是从那份 NC 起点来的，它还是 NC。起点决定终点**。

于是这条约束有一个非常具体的工程后果。想拿它做商用验证的人，不能走“下载权重 → 微调 → 上线”，只能走“读它的代码与结构 → 用可商用的数据与骨干从零训练”。前者是几天的活，后者要把第 33 章那笔账整个付一遍。**许可条款在这里不是法务问题，是预算问题**。而“等它以后改成 Apache”也不成立：一份已经放出的 NC 权重不会因为你等就变松，而本书无法核实 HF 上某个权重文件对应哪一份许可声明 [未]。

Apache-2.0 那一边为什么没有这个坑？ 它授予商用、修改、闭源再分发，义务只有两项：保留 LICENSE 与 NOTICE 的版权声明，改动过的文件要注明改动。它还多带一条多数工程师忽略的东西：**专利授权**——贡献者授予你专利许可，你若对这些贡献者提起专利诉讼，该许可终止。在机器人这种专利密集的领域，这一条有实际价值，而 MIT 与 CC 都没有 [行]。

为什么 WLA 用 Apache、WMA 用 NC：上游替它做了决定

一家卖本体的公司，为什么要在同一族模型上用两种相反的许可？这一档差别不像市场部的决定，更像被依赖链逼出来的。`unifolm-wla` 仓库里有一份 **THIRD_PARTY_LICENSES.md**，逐条登记第三方来源 [厂]：

```
starVLA      = MIT
Qwen-Image   = Apache-2.0
OpenVLA      = MIT（overwatch.py 自述来自 OpenVLA）
```

本章前面说过“许可证会传染，而且传染方向是收紧”。**把这句话反过来用，就解释了 WLA 的选择**：它的权重里蒸进了 Apache-2.0 的骨干与 MIT 的训练框架，而通行理解是——附加条款不得与上游条款冲突，所以把融了这些上游的产物整体声明成“非商业”，通常做不到 [行]。**想产品化就只能 Apache。这不是姿态，是链条推出来的。**

对照着看 WMA-0 那一侧：它的依赖里有一串可商用的库（`torch`、`diffusers`、`open-clip-torch`、`kornia` 等 [厂]），但它自己的 LICENSE 文件写的是 CC BY-NC-SA。**同一家公司、同一族模型、两份相反的策略**，而本书能核实的是结果，不是动机——动机那一句是厂商没公布的 [未]。

代价要一起说。 把研究侧锁进 NC，挡住的第一个人是竞争对手，第二个人是**你自己想产品化的团队**；而把基座放 Apache，等于把“别人在你机器上做出什么”这件事交出去——第 33 章会看到，这条路线买到的东西叫**生态装机量**，付出的东西叫**数据分布由别人决定**。

一个可以带走的检查动作

以后你要用任何一个开源模型/数据集/仿真资产，做四步，每步五分钟：

第一步，打开 LICENSE 文件本体，不要看 README 的 badge。 badge 是作者贴的图片，LICENSE 才是条款。两者不一致时以文件为准。

第二步，找三个词：`commercial`（商用）、`derivative`（衍生）、`share-alike / same license`（相同方式共享）。这三个词决定 90% 的使用场景。

第三步，检查数据集的许可是否与模型分开。这是最容易漏的一项：**模型 Apache-2.0，不代表用它训练的数据集可以商用**。LeRobot 格式的数据集各自带各自的 license，宇树 HF 上约 200 个并非同一份许可。你要用数据，得逐个查。

本书替你把这一项查了一遍（分页接口逐条抓，2026-10-08 [厂]）：

```
总数          = 200（skip=199 返回 1 条、skip=200 返回 0 条）
按机型        G1 = 191 个、Z1 = 6 个、资产类 = 3 个
H1 / H2 / R1 = 0 个
许可元数据   标 apache-2.0 的 = 64 个
             没有任何 license 字段的 = 136 个（含 44 个 G1_WBT 全身遥操集）
```

这三行里有两条是判据。一，「**宇树开源了人形全身数据可供任意使用**」**这句话不成立**：操作数据全部来自 G1 与 Z1，最重的机型（H1/H2/R1）一集都没有，而机型之间的质量、尺寸、扭矩档位全不一样（第 33 章）。二，**136 个没有许可元数据**——「公开可下载」不等于「可商用」，而 `unitree_rl_lab` 的 README 让你去 clone 的那个 `unitree_model` 资产集恰好在这 136 个里 [厂]：**一条官方推荐的训练流程，它的依赖资产法律状态是空的**。

第四步，检查它引用的第三方。一个“Apache-2.0”的仓库可能依赖一个 GPL 的库、一个 CC-BY-NC 的预训练权重、和一个需要单独授权的商业仿真器。**许可证会传染，而且传染方向是收紧**。

（本书在 `bible/images.md` 里对图片做了同样的处理：OpenStax 与 MIT OCW 的素材带 NC 条款，因此**本书一张都不用**；官方产品图没有新闻资料包授权，也一张不用。这不是过度谨慎，是因为“这个图网上有”在法律上不是理由。）

“6B / 2500 小时 / 64 任务”这三个数该怎么读

厂商自陈的三个数，逐个给判据。

6B 参数。含义在第 26 章：6B × 2 字节 = 12 GB 权重。这决定了它**跑不进 16 GB 这一档的 Orin NX**（12 GB 权重 + KV + 激活 + 多路图像，16 GB 装不下；G1 EDU 的模组容量与是否标配该档，官方未写成定式配置[未]，本书按“16 GB 这一档”算量级），必须要么量化到 FP8/INT4、要么跑在 Thor 的 128 GB 上、要么在机外推理。**所以“6B”这个数真正的信息是：它需要 Thor 这一档硬件，或者一台外置 GPU**。 这是可以从参数直接推出来的部署结论，而宣传里不会告诉你。

再往下追一层：节奏。 权重必须整份从显存里搬一遍，所以单步下限 = 权重字节 ÷ 内存带宽（第 26 章那条，本书脚本 `foc_check.py` 里的 `llm_memory_time()` 就是这一行除法）：

```
12 GB ÷ 100 GB/s = 120 ms 一步   ⇒   1 ÷ 0.12 = 8.3 Hz
6 GB ÷ 100 GB/s  = 60 ms 一步    ⇒   1 ÷ 0.06 = 16.7 Hz    （FP8 之后）
```

（[算]；100 GB/s 是第 26 章表里的那一档，**宇树未公布 PC2 模组的内存带宽，位宽是本书假设** [未]。）

于是 6B 这一项能推出三条可核对的结论：16 GB 的端侧模组装不下 FP16 权重；量化到 FP8 之后装得下，节奏仍在十几赫兹这个量级；而第 31 章那条要求（一块动作要覆盖一次推理，H·Δt ⩾ T_infer）意味着 T_infer = 120 ms 时，块长至少要 H = 120 ÷ 20 = 6 步。**宇树没有公布 WLA 的推理频率、量化档位与部署方式** [未]，所以这三条是“如果按参数与带宽算会怎样”，不是对宇树实测的描述。

约 2,500 小时真机数据。对照第 33 章那张表：AgiBot World 是 2,976 小时、Generalist GEN-0 是 27 万小时。**2,500 小时在行业里属于“真实但不大”的量级**。 而它是真机数据（不是合成、不是人类视频），所以质量口径可能更高——但第 33 章说过，没有人有可信的换算比。

单模型 64 任务。这一项最容易被误读成“会 64 件事”。**它的正确读法是“在 64 个任务上评测过”**，而不是“64 个任务都成功”。**一个模型的任务数与它的成功率是两个完全独立的轴**。而官方口径里，本书没有找到逐任务成功率 [未]。

这三条判据（带宽→部署、时长→横向对比、任务数≠成功率）是本章想给的真正东西。它们不依赖宇树诚实与否，只依赖你会算。

类比，以及它在哪儿破

类比：开源模型像公开食谱。你可以照做、可以改、可以拿去开餐厅——除非食谱上写了“仅供家庭使用”。

这个类比在哪里破：食谱的“仅供家庭使用”是道德约束，没有强制力；而 CC BY-NC-SA 与 Apache-2.0 是**许可证条款**，在多数司法辖区有法律强制力，且违反的后果是侵权诉讼与产品下架。

更关键的破处：**食谱的成分是可见的，模型的“成分”（训练数据来自哪、用了谁的预训练权重、依赖哪些库）在模型卡里经常不完整**。你拿到一个 Apache-2.0 的权重文件，可能里面蒸进了一个 NC 的视觉编码器。**这种“许可污染”在机器学习生态里是常态，而且无法从权重文件本身检查出来**。

所以本章最后给一条实操建议：**在你把任何开源模型放进产品之前，要求你的供应商/合作方给出一份“许可来源清单”，并且把它写进合同**。这份清单在软件行业是标准交付物（SBOM），在机器人行业几乎还没有人做。

一个可检验的说法：宇树模型侧的公开动作是“接入标准而非自建封闭栈”——数据用 LeRobot 格式、训练支持 GR00T 与 π0.5、仿真用官方 MJCF/URDF、通信走裸 DDS；其 UnifoLM 一族内部许可不同（WMA-0 为 CC BY-NC-SA、WLA-1.0 为 Apache-2.0），而这个差别不是一句“能不能商用”，而是四个具体动作的许可：内部可行性验证算商业使

用、微调产物作为改编材料继续带着 NC 与 SA、闭源再分发在 SA 下做不到，所以想商用验证的人只能重训而不能微调；WLA 选 Apache 还有一条可以核实的原因——它的 NOTICE 与 THIRD_PARTY_LICENSES.md 登记的上游是 MIT 与 Apache-2.0，附加更严的条件通常做不到；“6B”可以从带宽算出部署后果（12 GB ÷ 100 GB/s = 120 ms 一步、8.3 Hz，FP8 后 16.7 Hz），而 HF 那 200 个数据集里 H1/H2/R1 为零、136 个没有许可元数据。

证据等级：三个模型的存在、参数、数据时长、任务数、发布日期、许可条款，全部来自宇树官方仓库与 Hugging Face 页面 [厂]（**属厂商自陈，本书未做第三方复现**）；12 GB、120 ms、8.3 Hz、60 ms、16.7 Hz、H ≥ 6 是 [算]（复用第 26 章带宽算术与第 31 章不等式；**内存带宽与位宽是本书假设，官方未公布** [未]）；NC/Apache/SA 的条款解读是法律常识性归纳 [行]，**本书不构成法律意见**；“仓库无 LICENSE 文件即默认保留所有权利”是通行的版权法理解 [行]；HF 200 个数据集的拆分（G1 191 / Z1 6 / 资产 3，H1/H2/R1 为 0，64 个标 apache-2.0、136 个无许可元数据）来自本书 2026-10-08 逐条抓取 [厂]；**逐任务成功率、训练数据构成、是否蒸馏自其他模型、WLA 的推理频率与部署方式，官方未公布** [未]。

可核对清单：

- ☐ 打开 `unitreerobotics/unifolm-wla` 与 `unifolm-world-model-action`，各自找到 **LICENSE 文件本体**，抄下第一行写的许可名
- ☐ 再打开后者的 `pyproject.toml`，看它的 license 字段写的是什么，与 LICENSE 文件对照——这两份都是官方文件，结论相反
- ☐ 在 Hugging Face 的 unitreerobotics 主页上数一遍：按机型各多少个、有许可元数据的多少个（本书口径：200 / G1 191 + Z1 6 + 资产 3 / 64 有 apache-2.0）
- ☐ 用 6B × 2 字节复算 12 GB，除以 100 GB/s 得 120 ms，再取倒数看频率；然后除 20 ms，看块长下界是几步

- ☐ 找一份你正在用的开源模型，检查它的模型卡里有没有写“训练数据来源”与“依赖的预训练权重许可”——如果没写，把这一条记为风险

下一章：宇树把 WMA 叫“世界模型-动作”。下一章去看“世界模型”这个词在 2026 年到底被用来说什么——以及它现在真正在干的两件事，都不是“控制”。

第 33 章

数据经济学：一条演示值多少钱

先给一条演示标个价

2026 年 9 月的公开口径（媒体与厂商混合）：**本节的数只有两处逐字读到论文原文**（DROID 与 AgiBot World，见本章证据栏与 `research/other-vendor-firsthand.md`），其余一律按 [推] 用，只比较曲线形状：

```
真机遥操作（主从臂 / VR）        约 ¥500–1,000 每小时
手持末端采集器（UMI 一类）        数百元每小时
第一视角人类视频                 约 ¥100 每小时
仿真合成                         算力换，前重后轻
```

但“每小时”这个词藏了一个陷阱。 遥操作那 ¥500–1000 是**人工小时**，而一个操作员工作 8 小时，能产出的**可用数据**大约只有 1 小时 [推]——因为要摆场景、要重来、要质检、要休息、要标注。

所以真实单位成本要除一个可用率：

```
可用率 = 1 可用小时 ÷ 8 人工小时 = 12.5%
有效单价 = 名义单价 ÷ 可用率
¥500  ÷ 12.5% = ¥4,000 每可用小时
¥1,000 ÷ 12.5% = ¥8,000 每可用小时
```

（[算]。）

那这个 8 倍本身是从哪来的？ 是二手口径 [推]，不是任何一家公布的测量值，所以要做一次敏感性：如果可用率其实是 20%（5 倍浪费），有效单价 = ¥500 ÷ 20% = ¥2,500 到 ¥5,000；如果是 10%，就是 ¥5,000 到 ¥10,000。**结论的形状不变，只是横轴挪一格。** 本书因此只把它当数量级用。

这个数字是本章第一个要点：真机数据的成本不是“一小时几千块”，是“一小时几千块再除以 12.5% 的可用率”。 而浪费率这一项，几乎所有宣传都不提。

目前，具身智能不够用的问题，究竟是模型还是数据导致的？我反而感觉目前全球范围内，大家对机器人数据这个问题的关注度有点太高了。现在最大的问题是反而是模型的问题，并不是数据问题。

王兴兴 — 王兴兴 2025 世界机器人大会主旨演讲（新浪“机器人前瞻”文字实录） — 2025-08-09（第一人称演讲逐字（“是反而是”为实录原文重复字，保留））

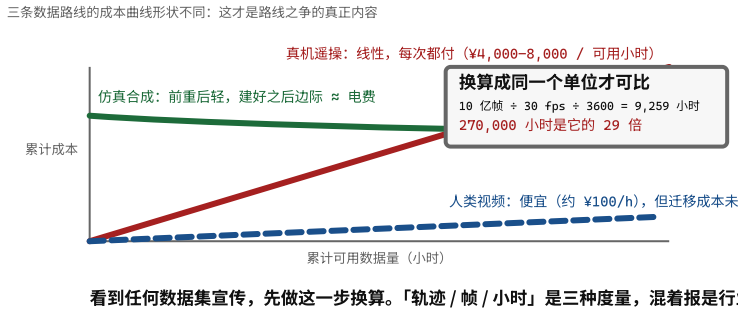


图 33-1　三条数据路线的边际成本曲线

把单价乘回去：一百万条要多少人、多少钱

宣传里最常见的是条数。那就把条数换算成人和钱——但**先得决定一条有多长**，而这件事有两个口径，本书两个都算，不取平均。

口径 A：假设一条 30 s。 宇树那四个 DiverseManip 数据集的 README 自己写了每集大约 30 秒 [厂]，本书前面也按这个长度算过：

```
1,000,000 条 × 30 s ÷ 3,600 s = 8,333 可用小时
8,333 × 8 = 66,667 人工小时 = 66,667 ÷ 8 = 8,333 人班 ÷ 250 工作日 = 33 人年
钱 = 66,667 × ¥500–1,000 = ¥3,300 万 – 6,700 万
```

口径 B：用 AgiBot World 自己公布的两个数相除。 它同时给了 1,001,552 条与 2,976.4 小时——**2026-10-09 逐字读到论文全文** [查]：`The latest version contains 1,001,552 trajectories, with a total duration of 2976.4 hours.`、`covering 217 specific tasks, 87 skills, and 106 scenes.`、`A suite of 100 dual-arm humanoid robots is deployed.`、`spans over 4,000 square meters.`：

```
每条时长 = 2,976.4 h × 3,600 s ÷ 1,001,552 条 = 10.7 s
1,000,000 条 × 10.7 s ÷ 3,600 = 2,972 可用小时
2,972 × 8 = 23,776 人工小时 ÷ 8 ÷ 250 = 12 人年
钱 = 23,776 × ¥500–1,000 = ¥1,190 万 – 2,380 万
```

两个口径差 2.8 倍（8,333 ÷ 2,972），落到钱上就是 ¥1,200 万到 ¥6,700 万这一整段。**这就是为什么“我们有一百万条数据”这句话没有信息量**：不说每条多长、不说可用率，任何人无法判断它值多少钱、多少人采了多久。

把两个口径除到单条，得到一个更可用的数：

```
每条 = 单价 × 每条时长 ÷ 3,600
口径 A：¥4,000–8,000 × 30 ÷ 3,600 = ¥33–67 每条
口径 B：¥4,000–8,000 × 10.7 ÷ 3,600 = ¥11.9–23.8 每条
只算名义人工、不除可用率：¥500–1,000 × 30 ÷ 3,600 = ¥4.2–8.3 每条
```

最后一行是行业里被引用的那一类数字（“一条演示几块钱”），它漏掉的就是本节那个 12.5%。**同一条演示，按报价算 ¥4–8，按可用口径算 ¥12–67。** 差一个数量级，而差的这一项没有任何技术含量，纯粹是口径。

三条路线，三条完全不同的成本曲线

现在可以把三家中国厂商摆成一次天然实验（它们的技术栈高度同构，第 39 章；分歧几乎只在数据来源）：

路线一：真机车队（智元）

```
公开口径（AgiBot World 论文全文里逐字核到 4,000 ㎡ 与 100 台：`spans over
4,000 square meters`、`A suite of 100 dual-arm humanoid robots is
deployed` [查]；**但论文没有给任何“每天产多少条”的产出率**，只写 `episodes
typically span approximately 30 seconds, some of which last over 2
minutes`——所以“日产 1,000 条”这类数仍属媒体口径 [推]）：4,000 ㎡ 数采基地、
100+ 台同构机器人、
        自有 VR 遥操套装（26 遥操 DoF、延时 50–100 ms）、
        “单机日产 1,000 条”（厂商宣传值）
开源成果：AgiBot World Colosseo —— 1,001,552 条轨迹、
        2,976.4 小时、217 任务、106 场景（arXiv 2503.06669）
```

成本形状：**线性**。多一台机、多一个人，多一批数据。边际成本几乎不变。

好处是数据分布就是真机分布（没有 sim2real 沟）。坏处是**任务多样性靠堆场景，而场景是物理空间**——106 个场景要 106 次搭建。

路线二：仿真合成（银河通用）

```
公开口径：操作/导航数据约 99% 合成，依赖 Isaac Sim / Isaac Lab /
        Omniverse / Cosmos；运控用 MuJoCo-MJX
        GraspVLA（VLA = 视觉-语言-动作模型，第 30 章） 的 SynGrasp-1B =
1,000 万轨迹 / 10 亿帧 / 10,680 物体，
        用 160 张 L40S/4090 量级跑约 10 天
2026 年改为“五层数据金字塔”：互联网数据 / 人类动作 / 仿真生成 /
        真机遥操校准 / 场景回流；累计 100 万小时人类数据 + 8 万小时真机回流
```

成本形状：**前重后轻**。GPU 集群是一次性投入，之后每条轨迹的边际成本接近电费。

好处是可以造出真机永远遇不到的极端场景。坏处是第 22 章那道沟——**仿真的接触动力学没有经过与空气动力学同等程度的校准**，所以合成数据里“物理对不对”这件事，本质上是一个信念。

注意那条“五层金字塔”的转向：一家以仿真起家的公司，2026 年把架构改成了“最底下铺互联网数据、最顶上放真机场景回流”。**这个转向本身就是本章的结论之一：纯仿真的路线，被他们自己承认不够**。

路线三：开源生态 + 遥操铺开（宇树）

```
公开口径：Hugging Face 上约 200 个 LeRobot 格式数据集；
        xr_teleoperate 遥操软件；unitree_IL_lerobot 支持 GR00T 与 π0.5 策
略；
        UnifoLM-WLA-1.0 官方称约 2,500 小时真机数据、单模型 64 任务
```

成本形状：**外包 + 分摊**。它不自己建数采工厂，而是把本体卖出去（第 03 章：¥8.5 万一台 G1）、把数据格式开源（LeRobot）、把训练脚本开源，让高校与公司各自产数据。

好处是成本由生态承担。坏处是**数据分布由别人决定**——你能拿到的数据，是买你机器的人在做的事，不一定是你客户需要的事。

把三条路线放到同一张成本图上

```
每 1,000 小时可用数据的成本（数量级，本书推算）

真机遥操：   1,000 可用小时 × ¥4,000–8,000 = ¥400 万 – 800 万
仿真合成：   160 卡 × 10 天 ≈ 38,400 GPU·小时
            按租用价 ¥30–50/GPU·h 估 = ¥115 万 – 190 万（一次性，可复用）
人类视频：   1,000 小时 × ¥100 = ¥10 万（但要跨本体迁移，迁移成本未计）
```

（[算]，输入除 DROID/AgiBot 外是 [推] 口径，租用价是市场公开价量级 [行]。）

这张表最值得读的是第二行的“可复用”。仿真集群建好之后，第二、第三、第一万个 1,000 小时几乎不要额外钱；而真机路线的第四行是**每次都付**。**所以这两条路线的差距会随数据总量拉大**——这解释了为什么仿真派敢喊“99% 合成”，也解释了为什么真机派在数据量到某个规模后会撞墙，除非真机数据的价值密度显著更高。

仿真的“前重后轻”轻在哪：三行除法，以及一个看错的地方

把 GraspVLA 那组公开数拆开算 [查]：SynGrasp-1B = 1,000 万条轨迹 / 10 亿帧 / 10,680 个物体，用 160 张 L40S/4090 量级跑约 10 天。

```
卡时 = 160 卡 × 10 天 × 24 h = 38,400 GPU·h
每条 = 38,400 ÷ 10,000,000 = 0.00384 GPU·h = 13.8 s 卡时
每条钱 = ¥115 万–190 万 ÷ 1,000 万条 = ¥0.115–0.19
帧数每条 = 10 亿 ÷ 1,000 万 = 100 帧 ÷ 30 fps = 3.3 s 的轨迹
13.8 s 卡时 ÷ 3.3 s 轨迹 = 4.2 倍—生成 1 秒合成数据要花 4.2 秒 GPU
```

与真机比： 真机每条 ¥11.9–23.8（口径 B）÷ 仿真每条 ¥0.115–0.19 = **63–207 倍**；换成口径 A（¥33–67）是 **174–583 倍**（[算]）。两条路线的单条成本差两个数量级，这个差不靠口号。

那“后轻”又靠什么？ 靠一件事：仿真里一条轨迹的边际成本是一段**卡时**，而卡时可以批量复用；真机一条轨迹的边际成本是一个人的**工时**，而人不能复制。再往下追一层就到底了——仿真的世界是一份可克隆的数据，物理积分在 GPU 上是一批矩阵运算，同一张卡上可以同时步进很多个环境；真机只有一套物理，一次只能演示一遍。**“前重”买的是集群，“后轻”来自克隆**。

但“后轻”有一个很容易被看错的地方。 上面那行除法顺带给出：生成 1 秒数据要 4.2 秒 GPU，也就是**一张卡一小时只产出约 0.24 小时的合成视频**（[算]：3,600 ÷ 4.2 ≈ 857 s ≈ 0.24 h）。仿真的“几乎零边际”说的是钱，不是时间。要 10 万小时数据，集群就得转很久，而集群折旧与电费都是真金白银——各家 GPU 规模与单条成本未公开 [未]，所以这一项本书只能留空。

“12.5%”不能只是一个数，它得能拆开重算。 本书把这条可用率拆成五个相乘的损耗因子（每一项都是**行业量级假设**，厂商未公布 [未]）：

因子	取值	倍数	短 Episode 变体
班次有效时间	6.4 h ÷ 8 h	×1.25	同
摆场景与复位	每条 1 min，准备 36 s	×1.60	每条 10.7 s ⇒ **×4.36**
设备停机与重标定	每 4 h 停 1 h	×1.25	同
质检废弃	不合格 30% ⇒ 1 ÷ 0.70	×1.43	同

标注与后处理	40% 不能直接用 ⇒ 1 ÷ 0.60	×1.67	同
乘积 / 可用率		**×5.96 ⇒ 16.8%**	**×16.3 ⇒ 6.2%**
有效单价（¥500–1,000 × 乘积）		**¥3,000–6,000**	**¥8,100–16,300**

（[算]。**上一版这张表有两处错，都改在这里**：一，“40% 的条目不能直接用”的倒数是 1 ÷ 0.60 = **1.67**，旧稿写的 ×2.50 对应的是 60% 不能用，凭空把这一项放大了 1.5 倍；二，“每条 1 min”和本章自己引的两手核对过的时长矛盾——AgiBot World 论文给 `episodes typically span approximately 30 seconds` [查]、本章按它两个数相除得 10.7 s [算]、DROID 是 16.6 s [查]，而 1 分钟是遥操作慢吞吞那一档。改完之后表自己算出的乘积只有 5.96，**比正文取的 8.0 更便宜**；把 episode 换成 10.7 s 则变成 16.3，**比正文贵一倍**。）

所以这张表是检查，不是替换。正文的 12.5%（乘积 8.0）是二手口径 [推]，它落在这两个变体之间——**5.96 与 16.3 正好把它夹住**，这就是本书要的结果：一个不能复算的百分比，至少要知道它能往哪两边挪。全章后面的每个数（¥4,000–8,000 每可用小时、33 / 12 人年、¥11.9–67 每条）继续沿用 12.5% 这条口径，本书不换分母。

要紧的不是凑得准不准，是敏感性。把争议最大的最后一项从 1.67 挪一倍到 3.33，乘积从 5.96 涨到 11.9、可用率掉到 8.4%，表上那格变成 ¥6,000–11,900；往下挪一半到 1.25，乘积 4.47、可用率 22.4%、¥2,200–4,500。**两头都是“每可用小时几千块”，一寸没翻**。再挪最后一列那格也一样：换成 10.7 s 的 episode，成本翻倍而不是归零。**这就是本书给“不可复算的百分比”开的处方：拆成因子，指出哪一个最有争议，再把它挪一倍看结论会不会翻**。会翻的数是脆弱的，不翻的数才值得印进书里——而这一轮我们自己就先翻了两处口径。

还有一层更硬的代价：合成的是“你以为的物理”。第 22 章那道沟在这里落地。官方训练仓库里的域随机化只有四项——地面摩擦、附加质量、推力、观测噪声（再加 5 帧历史）[厂]，而 **k_t 缩放、零位偏移、通信延**

迟、连杆惯量、摩擦系数 Fs/Fd、温度模型都没有被随机化 [厂]。第 38 章还会补一刀：随机化覆盖不了单调漂移，因为磨损是单向的，而随机化假设参数在分布内反复抽样。**真机数据自带的那些东西（摩擦、温漂、延迟），仿真要花力气去猜，而猜的范围本身就是知识。**

人类视频那一行：便宜在哪，贵在哪

¥100 每可用小时对 ¥4,000–8,000，是 **40–80 倍**的便宜（[算]：4,000 ÷ 100 与 8,000 ÷ 100）。那为什么它没有把真机数据淘汰掉？因为**视频里没有动作标签**：一帧图像不会告诉你关节角是多少、扭矩是多少，而模仿学习要的恰恰是 (s, a) 这一对（第 29 章）。于是要么用逆动力学模型把动作反推出来，要么做跨本体重定向——人手与 G1 的两指夹爪不是同一套运动学（附录 C.1 那张自由度表）。**这两步的成本没有一家公开** [未]，但它是“免费数据”背后真正的那笔钱。

所以“几乎免费但迁移贵”这句话是可以检查的： 一种数据源如果不含动作标签、不含接触与延迟，它就必须配一个昂贵的转换层，而转换层的产物能不能训出可用策略，最终仍要靠真机评测说话（第 34 章那条“筛子”）。

而真机数据的价值密度确实可能更高：它自带上面那两项猜不出来的东西——真实的接触、真实的摩擦、真实的延迟、真实的温漂（第 1–2 部整部都在讲这些）。

这就是行业最贵的那个未解问题：一小时的真机数据，等值于多少小时的仿真数据？目前没有人有可信的答案。 任何声称知道这个比值的人，要么在卖仿真，要么在卖真机。

规模对照：现在谁手里最多

公开可查的数（**除 DROID 与 AgiBot World 已逐字核到之外均为媒体口径 [推]**，且各家口径不同，**不可直接相减**）：

```
DROID               76k 轨迹 / 350 小时 / 564 场景        （多机构分布式）
AgiBot World        1,001,552 轨迹 / 2,976 小时 / 217 任务 / 106 场景
GraspVLA 合成集      1,000 万轨迹 / 10 亿帧 / 10,680 物体
Generalist GEN-0    > 270,000 小时真操作数据
宇树 UnifoLM-WLA     约 2,500 小时真机（厂商自陈）
```

先把「200 个数据集」这件事拆开看（本书用分页接口逐条查过 [厂]）：

```
总数恰好 200（skip=199 返回 1 条、skip=200 返回 0 条）
按机型：G1 191 个、Z1 6 个、资产类 3 个
        H1 / H2 / R1 —— 零
许可：   标了 apache-2.0 的 64 个
        没有任何 license 元数据的 136 个
        （含 44 个 G1_WBT 全身遥操集，以及 rl_lab 让你去 clone 的
unitree_model 资产集）
```

三个立刻能用的读法。

一，**宇树开源的操作数据全部来自 G1 与 Z1**。 没有 H1/H2/R1 的任何一集。所以「拿宇树开源数据训出的策略能直接搬到 H2 上」这句话没有数据支撑——机型之间的质量、尺寸、扭矩档位全不一样（附录 C.1）。**一个只在自己最便宜的机型上采数据的开源生态，能支撑的是那个机型的能力，不是一个家族的能力**。

二，**136 个没有许可元数据**。 第 32 章那条判据在这里成立：「**公开可下载**」**不等于**「**可商用**」。 而这 136 个里恰好包含最有价值的 44 个全身遥操集。**要拿它们做商业验证，得先向宇树取得书面许可**。

三，**`unitree_rl_lab` 的 README 让你去 clone 的那个 `unitree_model` 资产集，本身也在这 136 个里面**。 也就是说，一条官方推荐的训练流程，其依赖资产的法律状态是空的。

这不是宇树独有的问题，是整个具身智能开源生态的常态——它只在有人拿它做产品时才变成问题，而那时通常已经晚了。

注意单位不统一：轨迹、帧、小时是三种不同的度量。 一条 30 秒的轨迹 = 0.008 小时 = 900 帧（30 fps）。把“10 亿帧”与“2,500 小时”放在一起比较是毫无意义的。

这是本章给读者的第二个实用工具：看到任何数据集宣传，先把它换算成同一个单位。 换算之后经常会发现，某些“最大规模数据集”的宣称，在小时口径下并不最大。

（例：10 亿帧 ÷ 30 fps ÷ 3600 = 9,259 小时。而 270,000 小时是它的 29 倍。[算]）

一个空白：没有人有田间数据

第 39 章会展开这条，先在这里给出算术版本。

上面所有数据集的场景，逐条看：厨房、桌面、货架、办公室、仓库、装配台。**共同点是室内、平整、光照可控、物体是人造的。**

没有任何一个公开数据集来自：非平整地面、坡地、作物行之间、泥、雨、正午强光（第 23 章那个 100 Klux 边界）、或者 50 公斤级负载的连续作业。

这不是巧合，是成本结构：**要采田间数据，你得有一台能在田里干活的重载机器人，和一个愿意连续几小时在田里遥控它的人。** 而这类人不在任何一家公司的员工名单上。

这条账在什么时候失效

上面每一行除法都由三个假设撑着，而它们没有一个是测量值。本书把它们写出来，是因为读者拿这张表做决定时，先要知道自己踩在哪一个假设上。

一，前提是可用率为常数。 它不是。任务越短、场景越要反复重摆，12.5% 这个可用率越低；一个 5 分钟的装配任务与一个 10 秒的抓取任务，不能用同一个乘数。本书因此把它当**数量级**用，不当系数用。

二，**前提是“一条轨迹”三家定义相同**。 它不同。把三个公开数各自除一遍：

```
DROID      350 h × 3,600 ÷ 76,000 条 = 16.6 s 每条
AgiBot     2,976.4 h × 3,600 ÷ 1,001,552 = 10.7 s 每条
SynGrasp   10 亿帧 ÷ 1,000 万条 ÷ 30 fps = 3.3 s 每条
```

同一个“条”字底下差着 5 倍时长（16.6 ÷ 3.3）[算]。任何按条数排出来的名次，先要按这条除法重排一次。

三，**前提是合成数据已经能训出可用策略**。 这一项最容易被糊过去：¥0.115–0.19 每条假设了任务在仿真里跑得通，而资产建模、场景设计、任务定义都是人时，**这部分没有一家公开** [未]。一条数据的钱可以算到分，一条数据的可用率没人肯说——这就是本章的处境。

最后一个失效条件最硬：以上单价全部是 2026 年 9 月的二手口径 [推]（只有 DROID/AgiBot 的轨迹数与时长读到过原文 [查]）。所以本书只用它们比较**曲线形状**（线性 / 前重后轻 / 近乎零边际），不用它们得任何采购结论；各家 GPU 规模、数采工厂真实台数、单条成本、合格率（除北京那家公开的 95%+ 之外）都是空的 [未]。

一条演示到底长什么样：能回放的失败才有资格进数据集

前面都在给数据算钱，这一节算另一件事：**一条数据要包含什么，才算“可复现”**。 这决定了你花的钱能不能取回来。

一条能用的演示，至少要装下这六类东西：多路图像与深度、IMU（四元数/角速度/加速度）、 每个关节的 q / dq / tau_est、末端与夹爪状态、任务标签（成功还是失败）、 以及**所有这些字段共用的一个时间基准**。最后一条最容易被省掉，也最致命： 第 23 章算过同步误差怎么把深度点云撕开一个伪视差，第 24 章算过 `tick` 这个字段为什么在 `rt/lowstate` 里——如果各传感器的时间戳来自各自的时钟，事后怎么对齐都是猜。

回放工具这一层，行业的事实标准是 rosbag2。官方仓库的自我定义是“the tool for recording and playback of communications in ROS 2 systems” [行]，四个动词就够用了：`ros2 bag record` / `play` / `convert` / `info`。存储后端支持 SQLite（`.db3`）与 **MCAP（`.mcap`）**，播放时它“will automatically detect which storage implementation to use” [行]；回放可以按话题筛（`--topics`、`--exclude-topics`）—— 这一条对做数据的人很实际：**同一个包，训练时只要关节与图像，调试时只要那 3 秒的全部原始话题**。（官方 README 没有写哪个是默认后端，本书就不替你断言。）

那录一小时到底多大？拿第 28 章那两个已经算死的帧长直接乘：

```
LowState ≈ 2.1 kB × 500 Hz = 1.05 MB/s
LowCmd   = 1004 B × 500 Hz = 0.50 MB/s
两路合计 ≈ 1.55 MB/s ⇒ × 3600 s ≈ 5.6 GB / 小时（还没算图像）
```

5.6 GB 只是“本体感觉”这一路（这里按 500 Hz 记账是本书假设 [未]：官方只把 500 Hz 坐实在 `rt/lowcmd` 上，`rt/lowstate` 的确切频率没有取到）。每帧 307,200 B，一路 × 30 帧 = 9.2 MB/s，两路 18.4 MB/s，一小时 66 GB—— **图像比关节数据贵一个数量级**，这就是为什么很多数据集只存深度图或存特征而不存原帧 [算]。（帧长与频率取自第 28 章；图像尺寸是本书假设，宇树未公布 G1 头相机的原始分辨率与码流 [未]。）

工程判据只有一条：能回放的失败才有资格进数据集。理由不是洁癖，是第 38 章那条复合衰减——策略在真实场景里的失败大多不是“整段失败”，而是第 47 步那一次打滑。如果录制系统只存了“任务成功/失败”这一个比特，那一次打滑在数据里根本不存在，模型永远学不到它。可回放的最低配置是：**能按时间跳回 t_0、能把 t_0 前后 2 秒的全部话题原样喂回策略、能同时看到当时的关节指令与实际反馈**。

自动打标能做到哪一步（这是省掉人工的关键，也是常被吹过头的地方）：用**本体信号**判成败是可以自动化的——末端位置到没到、夹爪电流有没有出现抓取特征、任务计时器有没有走完；用“看起来对不对”判成败

不能自动化，那还是要人看。 所以可行的分工是：**机器筛掉明显失败的 30%，人只看剩下的部分**， 而不是指望一个模型替你判断整条轨迹好不好。

边界：宇树自己用什么格式记录、是否提供官方 bag 工具，本书未公布可引的一手来源 [未]； 上面这套是行业通行做法，落到你的机器上，第一件事是确认 `rt/lowstate` 的 `tick` 与相机时间戳能不能对到同一个时钟上——对不上，先修这个再谈采数据。

类比，以及它在哪儿破

类比：数据像石油——有勘探成本、开采成本、提炼成本，而且储量分布不均。

这个类比在哪里破，破得很有用：石油的品位（每吨矿石含多少油）是可以测的，而**数据的质量无法在采集时判断**。一条演示看起来完美，可能因为操作员下意识放慢了、因为物体被摆正了（“摆拍”）、因为失败的那几次没保存。行业里管这叫“合格率”，而公开数字极少——北京人形数据基地提到过 95%+ 的合格率 [推]（媒体转述，本书未读到基地官方原文），这已经是罕见的透明——**透明的是“敢给一个数”，可核对性并没有解决：它既没有分母定义，也没有第三方复算路径**。

所以“多少小时数据”这个指标，本质上是在报储量而不是可采储量。一个诚实的数据集宣传应该同时给：合格率、失败样本占比、场景数、任务数、以及**评测协议**。少了后两项，“100 万条”和“10 万条”可能没有区别。

一个可检验的说法：真机遥操数据的真实单价是报价除以可用率——¥500–1,000 每人工小时 ÷ 12.5% = ¥4,000–8,000 每可用小时，而这 12.5%（乘积 8.0）不是任何一家公布的，本书把它拆成五个因子自己乘回去：1.25 × 1.60 × 1.25 × 1.43 × 1.67 = **5.96** ⇒ **16.8%**，把 episode 从 1 min 换成本章一手核到的 10.7 s 则是 **16.3** ⇒ **6.2%**——两个变体正好把正文那个 8.0 夹在中间，而最后一个因子挪一倍只能让单价在 ¥2,200 到

¥11,900 之间移动，“每可用小时几千块”这个结论不翻；把它乘回数量，100 万条按 30 s 一条是 8,333 可用小时、66,667 人工小时、约 33 人年、¥3,300 万–6,700 万，按 AgiBot 自己两个数相除得到的 10.7 s 一条则是 2,972 可用小时、约 12 人年、¥1,190 万–2,380 万；三条路线的成本曲线形状不同——真机线性、仿真前重后轻（合成每条 ¥0.115–0.19，比真机便宜 63–207 倍，但生成 1 秒数据要 4.2 秒 GPU）、人类视频便宜 40–80 倍却要额外付一个未公开的迁移层；而各家宣传用的“条”彼此差 5 倍时长，换算之后排序会变。

证据等级：轨迹数与时长里，**DROID 与 AgiBot World 已逐字读到论文原文** [查]（DROID：`76k demonstration trajectories、350 hours of interaction data、84 tasks、564 scenes`；AgiBot：`1,001,552 trajectories、2976.4 hours`）；而**单价、GPU 卡数与天数仍来自媒体报道与厂商宣传页** [推]——本书只取数量级与曲线形状；本节所有除法（¥4,000–8,000、五因子表的 1 ÷ 0.60 = 1.67 与 1 ÷ 0.70 = 1.43、乘积 5.96 / 16.3 / 11.9 / 4.47、可用率 16.8% / 6.2% / 8.4% / 22.4%、(10.7 + 36) ÷ 10.7 = 4.36、¥2,200–11,900 那一整段敏感性区间、五因子表的 1 ÷ 0.60 = 1.67 与 1 ÷ 0.70 = 1.43、乘积 5.96 / 16.3 / 11.9 / 4.47、可用率 16.8% / 6.2% / 8.4% / 22.4%、(10.7 + 36) ÷ 10.7 = 4.36、¥2,200–11,900 那一整段敏感性区间、8,333 / 2,972 可用小时、33 / 12 人年、¥33–67 与 ¥11.9–23.8 每条、13.8 s 卡时、¥0.115–0.19 每条、63–207 与 174–583 倍、0.24 倍实时率、4.2 倍、16.6 / 10.7 / 3.3 s 每条、9,259 小时、29 倍）是 [算]，输入均为上面那些 [推][查] 口径；租用 GPU 单价是市场公开价量级 [行]；域随机化只有四项、以及未随机化那六项，来自宇树官方训练仓库 [厂]（第 21、22 章已逐条核过）；DiverseManip 每集约 30 s、LeRobot 官方文档的 30 Hz / 33.3 ms / 1.3% 超时是官方仓库文件 [厂]；宇树 HF 约 200 个数据集、xr_teleoperate、unitree_IL_lerobot 支持 GR00T/π0.5、WLA 约 2,500 小时 [厂][查]；**各家 GPU 集群规模、**

数采工厂真实台数、单条成本、合格率（除北京基地外）、人类视频迁移层成本、仿真资产人时——全部未公开 [未]；“仿真派自己承认纯仿真不够”是从其“五层金字塔”表述读出的 [查]，属解读。

可核对清单：

- ☐ 把前面那五行规模数据全部换算成“小时”，自己排一次序，看它和宣传口径的排序是否一致
- ☐ 复算一次可用率：¥500 ÷ 12.5% = ¥4,000、¥1,000 ÷ 12.5% = ¥8,000；再把可用率换成 20% 看结论挪多少
- ☐ 复算单条成本：一条 30 s 演示的名义人工是 ¥500–1,000 × 30 ÷ 3,600 = ¥4.2–8.3，除以 12.5% 的可用率后是 **¥33–67**（行业宣传里出现的通常是前一个数）
- ☐ 用 2,976.4 × 3,600 ÷ 1,001,552 复算 AgiBot 的每条 10.7 s，再与 30 s 假设比，看差几倍
- ☐ 用 38,400 ÷ 10,000,000 复算每条 0.00384 GPU·h（= 13.8 s），再除 ¥115–190 万得每条 ¥0.115–0.19
- ☐ 找一份数据集论文，检查它有没有报合格率、失败样本占比、评测协议三项；记下缺哪几项
- ☐ 在 Hugging Face 上打开 unitreerobotics 的数据集列表，数一下有多少个、格式是什么、license 分别写的什么

下一章：本章说仿真的边际成本接近零。下一章去看仿真最近的那个新用途——不是造数据，是**当评测器**——以及为什么它仍然进不了 1 kHz 的控制环。

第 34 章

世界模型现在只解决两件事：数据和评测

“世界模型”是四个不同的东西共用一个名字

打开任何一份 2026 年的具身智能材料，“世界模型”（World Model）都在里面。但它指的是四种完全不同的系统，解决三个不同的问题。先把它们分开——**这是本章 80% 的价值**。

用法	它在做什么	代表	是不是“机器人控制器”
A. 造数据	生成大量视频/状态轨迹，用来训练策略	NVIDIA Cosmos、Wayve GAIA-2	不是
B. 造环境	文本或图像生成一个可交互世界，用来训练与评测智能体	DeepMind Genie 3	不是
C. 当评委 + 当策略	预测“这么做之后会发生什么”，用它评分或反推动作	1X 1XWM、智元 Genie Envisioner、宇树 UnifoLM-WMA-0	**接近，但仍是开环**
D. 神经仿真器（车端）	回放历史问题、注入对抗场景	Tesla FSD/Optimus 共用一套、Wayve	不是

四个用法里，只有 C 与“控制”有关系，而它“仍是开环”——这四个字是本章的主线。

我可以控制一个视频生成模型……让这个视频生成模型直接去驱动一个机器人完成任务。……我们用一个预训练的视频生成模型，重新训练了一下，让他先去生成一个机器人动作的视频，然后再控制一个机器人去做，这个技术是能实现的。……我觉得这个路线的方向可能会比 VLA 模型发展得要快，收敛概率还更大。

王兴兴 — 王兴兴 2025 世界机器人大会主旨演讲（新浪“机器人前瞻”文字实录） — 2025-08-09（第一人称演讲逐字）

为什么它进不了实时环：把三个除法做完

第 31 章给了判据：任何要参与平衡与接触的反应，必须在 100 毫秒以内。现在看一个世界模型跑一次要多久。1X 那组数**本书没有读到一手出处**：2026-10-09 直接打开官方世界模型页面，逐字只拿到 `we have trained a world model that serves as a virtual simulator for our robots.` 与 `World models solve … evaluation.` 两句，**没有 14B、没有 11 s、没有任何 Hz** [未]。所以下面这串按二手量级读（[推]），本章的结论不靠它成立——靠的是后面那条一手的 24 fps：

```
1X 的策略版世界模型：14B 参数的视频扩散主干
一次 5 秒的 rollout：多张 GPU 上 11 秒
再加一个逆动力学模型（IDM，把“未来的画面”反推成动作的那一层）要 1 秒
⇒ 单步 = 11 s + 1 s = 12 s，而它预测的未来只有 5 s
实时率 = 5 s ÷ 12 s = 0.42×
```

先别急着用这个 0.42。 它是二手数算出来的，只当量级。**真正立得住的那条除法是下面这个，它的输入全部是原话：**

```
Genie 3 官方博客原话：24 frames per second，resolution of 720p [查]
⇒ 一帧的节奏 = 1 ÷ 24 = 41.7 ms
对照第 21 章 50 Hz 的策略环：41.7 ÷ 20 = 2.1 拍
对照第 12、27 章那条 1 kHz 关节环：41.7 ÷ 1 = 41.7 拍
```

这条不需要 11 s 那个数：就算一个逐帧生成的世界模型真的达到 24 fps 的显示节奏，它也追不上控制节拍——**帧与拍对不齐，就不成闭环**，因为环要求每一拍都有对应的状态更新。而真实生成远慢于 24 fps（那句 11 s 说的就是这个差距，只是本书没读到原文，所以只当量级）。把那两个二手的 12 s 放进熟悉的频率阶梯里再除三遍，看的是同一件事的另一面（**只做量级，别当厂商规格**）：

```
12 s ÷ 100 ms = 120      （第 31 章那条反应窗口的上限）
12 s ÷ 20 ms  = 600      （第 21 章 50 Hz 的策略环 [厂]）
12 s ÷ 1 ms   = 1.2×10⁴  （1 kHz 的关节环（这个量级来自腿式控制综述 [行]，宇树未公布 [未]），对 1 ms 这个本书自选的分母）
```

差四个数量级（1.2×10^4），而且不是“再优化优化”能填的那种。第 26 章那条带宽算术在这里同样成立：14B × 2 字节 = 28 GB 权重，每步整份搬一遍：按多卡 1–3 TB/s 的带宽是 9–28 ms/步，几十步也只有 0.3–0.8 s。**所以 11 s 主要花在逐帧生成上（token 数与注意力），不在搬权重上。**第 26 章那条带宽算术讲的是端侧单卡逐 token 读权重，不能直接搬到 14B 视频生成上——这是本书自己踩过的一个口径错。

那为什么它这么贵？往下追一层就找到根子：它把世界表示成像素。一次 rollout 要生成多少数据量，可以拿两家公开口径拼一个数量级（**跨厂拼算，只为看量级，本书明写** [算]）：

```
要生成的：5 s × 24 帧/s = 120 帧（24 是一手：Genie 3 原文 `24 frames per second`）；每帧 1,280 × 720 × 3 = 2.76×10⁶ B（8-bit RGB）
          ⇒ 120 × 2.76×10⁶ = 3.3×10⁸ B
控制要的：5 s × 50 Hz × 29 个自由度 × 4 B = 2.9×10⁴ B（G1 取官方 `g1_29dof` 那一版的 29；第 03 章 BOM 里的 23 是**关节模组数**，两个口径不是一回事，别混用）
比值     = 3.3×10⁸ ÷ 2.9×10⁴ ≈ 1.1×10⁴ 倍
```

两个 10^4 是两个各自独立的比值，只是恰好同量级：一个是字节比（要吐的东西 ÷ 控制需要的东西），一个是时间比（12 s ÷ 1 ms，而 1 ms 是本书自己选的分母）。本书不用前者推后者——“吐得多一万倍”推不出“慢

一万倍”。**而表示为什么长成像素？因为它的监督信号就是视频**——数据的形状决定输出的形状。这一层没有回旋余地：你拿互联网视频训练，得到的就是一个吐视频的东西。

所以本章可以给一个判断：只要世界模型的输出形式是“未来的画面”，它就不可能进实时环。要进环，必须换表示——换成状态、换成隐空间、或者直接换成分块动作。而这就不再是“世界模型”，是第 30 章那个动作专家。

（1X 自己也承认这一点——其表述大意是“用比特而非原子做评估”，即它的世界模型价值在**评测**而不是在执行 [查]。本书把这句话当作本章最重要的引用：**做世界模型最激进的公司，自己把它定位成评委**。）

三家各在哪一格：名字都叫世界模型，位置不一样

把本章前面那张四用法表拿去对三家，会得到三个不同的答案，而**有两家是跨格的**：

谁	官方口径里它是什么	落在哪一格	这本书的判断
NVIDIA Cosmos 3	Reasoner 出文本、Generator 出视频/声音/动作；另有 Policy-DROID 变体	**A 造数据**为主，B 也沾（可交互的仿真素材），并开始往 C 伸手	它的“造数据”身份最硬；“世界模型直接当策略”是最激进的一步，但它靠的是**取消生成视频**这一步，不是把它加速
智元 Genie Envisioner / GE-Sim	本章前面把它列在“评测”那一格[查]；GE-Act 是动作侧（一手：`a lightweight 160M-parameter autoregressive action decoder、54-step torque trajectories within 200 ms on a commodity GPU`、30 Hz，计时在 `onboard NVIDIA RTX 4090 GPU` [查]，arXiv 2508.05635v3。**注意它是自回归解码器**，第 30 章原先把它列进流匹配代表是分类错，已更正）；名字里的 Sim 指向可交互环境	**C 当评委 + 当策略**，并且占着 B 那一格[推]	一家同时占了“环境”“评委”两格，而这两格都在训练与评测阶段，**都不在 1 kHz 环里**
宇树 UnifoLM-WMA-0	README 自陈两用：Simulation Engine（生成本体合成数据）+ Policy Enhancement（接动作头改善决策）	**A + C 各一半**	它是三家里唯一把“仿真引擎”写进名字第一个用途的；“当评委”这一项官方没这么表述 [厂]，本书不替它加

这张表真正要说的是：三家没有一家把自己放进 1 kHz 的位置。而“世界模型 = 实时控制器”这句宣传，恰恰是把 A 与 C 的功能说成了执行层的功能。

那它真正在做的两件事

第一件：造数据（用法 A、B）。

第 33 章的成本曲线解释了为什么这件事值钱：仿真的边际成本接近零，而合成数据的问题是“物理对不对”。世界模型给出的第三条路：**用真实视频训练出来的生成器，其物理直觉来自真实视频。**

NVIDIA Cosmos 的公开演进很能说明问题（[查]）：

```
2025     Predict / Transfer / Reason 三条产品线
2026-05  合并为 Cosmos 3，统一 MoT 架构：
         Reasoner 出文本、Generator 出视频/声音/动作
         分 Super 64B、Nano 16B、Edge 4B（其中 Edge 4B ，装得进 Jetson AGX
Thor T5000 这一条已一手：`a 4B omni-model`、`the weights fit in Thor's
on-board memory`；**Orin 未被点名**，Super/Nano 的 64B/16B 仍是新闻稿口径
[推]）
         并提供 Cosmos3-{Nano,Edge}-Policy-DROID —— 世界模型直接当策略
```

注意最后一行：`Policy-DROID`——把世界模型当策略用。 这是这一类技术最激进的一步，因为它试图取消“生成视频 → 反推动作”这条昂贵的中间路径。

“Edge 4B”这个档位现在有一手了（`Cosmos 3 Edge is a 4B omni-model`），而“Orin 上能跑”仍没有——一手只点 Jetson AGX Thor T5000 [查]。**但 4B 这个数是一手的**：NVIDIA 那篇平台论文原话 `we first build two base models that are 4B and 12B in size, respectively` 与 `We name them Cosmos-Predict1-4B and Cosmos-Predict1-12B, respectively.` [查]——论文里只有 4B 与 12B 两档，没有 Nano / Super / Edge 这些名字，也没有任何 Jetson/Orin/Thor 字样。**所以这条算术不靠那半句**：4B × 2 字节 = 8 GB，16 GB 的模组装得下（没算 KV 与激活，是量级判断）[算]。**所以“能在机器人上跑的世界模型”目前尺寸是 4B 量级，不是 14B。**

第二件：当评委（用法 C）。

这件事比造数据更被低估，也更有即时价值。原因在第 33 章：真机评测太贵。

```
真机评测：一次失败可能摔坏一台 8.5 万的机器；
          一天测 50 次就到人力与场地的极限
世界模型评测：把策略放进模型里跑 10,000 次，
              不花钱、不摔机、可复现
```

智元的 Genie Envisioner / GE-Sim、NVIDIA 的 Cosmos Evaluator、1X 的世界模型评测，都是这一条 [查]。

但评委有一个致命问题：评委本身准不准?

顺带一句它真正值钱的地方，不是便宜，是可复现。真机上你无法把同一个初始条件复现两次——摩擦系数不可测（第 25 章）、温漂每天不一样（第 36 章）、电池内阻随循环涨（第 35 章）。而在模型里，同一个种子会给出同一条轨迹，于是**两次评测的差只能来自策略本身**。这才是“跑一万次”的真正卖点，而宣传几乎从不提它。可复现不等于正确——这是下面那条循环风险的前提。

如果一个世界模型对“脚踩到肥皂水”的物理建模是错的，那它会给出一个漂亮但错误的评分——而你会基于这个评分去做产品决策。**这就是“用模型评模型”的循环风险**：被评估者与评估者共享同一套物理假设，于是它们的错误相关。

所以正确的用法是把世界模型评测当作筛子，而不是当作判据：用它把 10,000 个候选筛到 100 个，再用真机测那 100 个。**筛子的标准是“别漏掉好的”，不是“选出的就是对的”**。

宇树 WMA-0 在这张图里

宇树把 UnifoLM-WMA-0 命名为 World-Model-Action，官方口径里它有两个用途 [厂]：**既在仿真里合成本体数据，也接动作头做决策**；（更正：`flow_matching_head/` 那组目录名记在宇树**另一族**仓库 `unifolm-wla` 的结构清单里，WMA-0 自己的清单没有 [厂]；旧稿把两族混成一族）。至于它跑

多快：**官方页面与仓库通篇没有 Hz 或 ms** [未]——2026-10-09 逐字回读，`Hz`、`ms`、`control rate`、`frequency`、`real-time` 五个词全部零命中，只有目录树里一行 `unitree_deploy # Deployment code`（阴性结果记在 `research/other-vendor-firsthand.md`）。

先撤回本书自己的一处错。 前一版这一节写着“真机侧客户端约 15 Hz”，并围绕它做了三段推理。本轮去查这句话的出处，查到的结果是：**那个 15 Hz 属于 NVIDIA 的 Cosmos 3 Edge 在 Jetson AGX Thor T5000 上跑策略时的输出频率，跟宇树的 WMA-0 没有任何关系。**本书把别家的一手数字挂到了宇树头上。这类错误比算错更危险，因为它后面跟着一个看起来完全可靠的方括号，而读者没有任何办法察觉。**本书早前犯过同型的一次**：把别处抄来的“L2 内置 IMU 官方标 1 kHz”当事实，后来在第 24 章留了撤回框——这次是同一个毛病的另一种形态：**不是抄错，是把 A 家的一手数写到了 B 家的格子里。**

同一条论证搬到真正属于它的那组数上，结论不但没变，现在还可核对了。 NVIDIA 官方开发者博客，2026-10-09 逐字读到 [查]：

> Cosmos 3 Edge is a 4B omni-model (with a 2B NVIDIA Nemotron-based reasoner)
> At roughly 9 GB in BF16, the weights fit in Thor’s on-board memory
> On an NVIDIA Jetson AGX Thor T5000, it generates each action chunk in about 1.53 seconds
> (running at 640×540 resolution and 15 Hz), while a single chunk covers roughly 2.13 seconds
> Because the next chunk is ready before the current one finishes, the arm moves continuously

这六行把本章前面所有抽象判据一次落地。 逐条对：

> 每块推理 T_infer = 1.53 s，一块覆盖 2.13 s ⇒ 2.13 ÷ 1.53 = 1.39 > 1
> ⇒ 第 31 章那条 H·Δt ≥ T_infer 成立，而且是厂商自己说的“下一块在当前块用完前就好了”
> 输出节拍 15 Hz ⇒ Δt = 1 ÷ 15 = 67 ms [算]
> 第 31 章判据：接触密集任务的反应延迟要 < 100 ms ⇒ 67 ms 过
> 第 16 章判据：τ = 259 ms，要参与平衡得落在 10–100 ms ⇒ 67 ms 在带内，但只勉

```
强
权重 4B × 2 字节 = 8 GB，厂商自报 roughly 9 GB ⇒ 同一件事的两种写法 [算]
```

67 ms 落在“能参与操作节奏、勉强参与慢速校正、不可能单独守平衡”的那个窄带里，这正是本书要给“世界模型-动作”这一类东西占的位置——**它是节奏级的参与者，不是稳定性的承担者**；承担稳定性的仍然是第 12 到第 19 章那条 1 kHz 的环路。**而 9 GB 这一条顺手验证了第 26 章的带宽算术**：本书按 4B × 2 字节算出 8 GB，NVIDIA 报 roughly 9 GB（多出的那部分是嵌入与注意力权重之外的东西，本书不细算）——**外部对照比任何自检都硬**。但它同时划出本书一条推理的边界：1.53 s 是在 **Jetson AGX Thor T5000**（板载高带宽内存）上量的，不是第 26 章那张表里 41.6–100 GB/s 的假设档，**不能拿它去反推别的模组**。

回到宇树这一格：**WMA-0 的用途有官方口径 [厂]，许可证是 CC BY-NC-SA、不可商用（见第 32 章），运行频率官方未公布 [未]**。本书因此不给它配任何节拍数；要看“世界模型-动作在端侧到底跑多快”，用上面 NVIDIA 那组一手数当参照，并记住那是**另一家的另一颗芯片**。

驾驶侧的那面镜子

用法 D 值得看一眼，因为它比机器人早十年。

Tesla 公开表述其 FSD 与 Optimus 共用一套“神经世界仿真器”，用于回放历史问题与注入对抗场景；Wayve 的 GAIA-2 是 latent 扩散世界模型，官方技术报告称最多支持 5 路时空一致的相机、分辨率 448×960 [查]。

自动驾驶为什么先做成这件事？ 三个原因，每一个都反衬出机器人的难：

1. **状态空间小得多。** 车在路面上，二维流形、可预测的运动学、规则明确。人形面对的是三维、非结构化、物体可动。
2. **不需要闭环控制。** 车的控制（转向、加速）是低带宽的；人形的平衡是 1 kHz 的。**世界模型进不了人形的实时环，但车的环本来就不快。**

3. **数据已经在那儿**。车队每天跑几百万公里。人形没有等价的数据来源（第 33 章）。

所以“世界模型在自动驾驶里成立”这件事，不能直接推论到“在人形里也会成立”。而很多宣传正是在做这个非法推论。

这一章的证据等级为什么必须写低

前面每个数都要打折扣，本书把折扣写在明处：

一，这批数里多数曾是论文与新闻稿级，但已经开始分化（见下面那句分栏）。1X 的 14B/11 s/1 s、Genie 3 的 24 fps@720p（**已回读到 DeepMind 原文**，逐字：`resolution of 720p`、`24 frames per second`、`retaining consistency for a few minutes` [查]；旧稿多写了一个 20，原文没有）、Cosmos 3 的档位划分、GE-Act 的 160 M/200 ms/54 步、GAIA-2 的 `up to five temporally and spatially consistent camera streams`、`resolution of 448×960`、`each spanning 2 seconds`、`8.4B parameters`（**2026-10-09 已读到 Wayve 论文原文** [查]，arXiv 2503.20523；旧稿这组数属二手，现已升一手），全部来自厂商仓库、技术报告或媒体报道 [查]，**没有一条是本书复现的**，而且多数没有版本号。

这批数从这一版起分成两栏看。**已升到一手**（逐字读过厂商页面或论文原文）：Genie 3 的 720p / 24 fps / 数分钟一致性；Helix 的 7 B / 80 M / 7–9 Hz / 200 Hz；GR00T N1.7 的 `nvidia/GR00T-N1.7-3B`（**3 B 是整个检查点，主干是 `Cosmos-Reason2-2B` 那 2 B**）、`action_horizon expanded from 16 to 40`、`State and action dimensions expanded from 29 to 132`、`1 GPU with 16 GB+ VRAM`；GAIA-2 的 `up to five temporally and spatially consistent camera streams`、`resolution of 448×960`、`8.4B parameters`；DROID 的 `76k demonstration trajectories`、`350 hours of interaction data`、`84 tasks`、`564 scenes`；π0 的 `10 integration steps`、`3.3 billion parameters`；GE-Act 的 160 M 自回归解码器与 54 步 ÷ 200 ms。**仍在二手栏**：1X 的 14 B / 11 s / 1 s、Cosmos 2026 的 Super / Nano / Edge 档位名。另有两家是

一手但没公布你要的那一格：SmolVLA 只给总参数 450 M，主干/专家分栏未公布；GR00T 只给主干 3 B，动作专家规模未单独公布。**没有分栏就算不出倍数，这跟“数不准”是两回事。取数的手艺也记一笔**：`export.arxiv.org/api/query` 四条检索全返回空响应，换 `arxiv.org/html/<id>` 全文页与 `arxiv.org/abs/<id>` 就读到了（html 有版本坑：某篇的 v3 是 404，abs 页可用）。成功与失败都逐条记在 `research/other-vendor-firsthand.md` 与 `bible/sources.md` 的 G1 节。

二，**权重能不能拿到，本身是变量**。Cosmos 有公开仓库，但许可与商用条款要逐文件核（第 32 章那四步）；Genie 3 公开的是演示与技术口径，权重是否可得本书未核 [未]；1X 的那组数来自技术分享与报道，同样未见可下载的权重 [未]；宇树 WMA-0 有权重但 CC BY-NC-SA。**“某某世界模型已经开源”这句话，要落到“哪个版本、哪份文件、什么许可”才有意义**，而这三项经常对不上 [未]。

三，**评测器那条路有内在上限**。第 33 章说过真机评测贵，所以“在模型里跑一万次”有吸引力；但被评的策略与评它的世界模型**共享同一套物理假设**，错误因此相关——模型觉得“这一步稳”，可能只是它自己也漏了那一层接触。本书的结论保持不变：**当筛子，不当判据**。

四，最硬的那条上限是表示。只要输出是画面，它的计算量就正比于帧数 × 像素 × 去噪步数；这不是工程没做好，是数据形状决定的（前面向下追到的那一层）。**要让它进环，得先把“世界”换成状态或隐空间——而换掉之后它就不再是这一章讨论的那个东西了**。

类比，以及它在哪儿破

类比：世界模型像一部“能预演未来的电影机”。你可以问它“如果我这么做会怎样”，它给你放一段。

这个类比在哪里破，破得正是要害：电影机要渲染每一个像素——包括那些与控制无关的像素（墙纸的花纹、远处的车）。而控制器只关心七八个数（重心位置、速度、接触力）。

世界模型昂贵的那部分，恰恰是控制不需要的那部分。这就是为什么它的实时率是 0.42×，而一个 MLP 策略（第 21 章，几兆参数）能做到微秒级。

所以本章最后给一句判断：**世界模型当前的价值在“生成”与“评价”这两端，不在“执行”**。谁把它说成实时控制器，要么是在做研究前瞻，要么是在混淆第 30 章的动作专家与第 34 章的世界模型——**这两个词在宣传里确实经常被混着用**。

一个可检验的说法：2026 年的“世界模型”是四种系统的共用名（造数据、造环境、当评委兼策略、车端神经仿真器），其中只有第三种与控制有关而它仍是开环；立得住的那条除法是：一手 `24 frames per second` ⇒ 一帧 41.7 ms，而 50 Hz 的策略环每 20 ms 一拍、1 kHz 的关节环每 1 ms 一拍，**帧与拍对不齐就不成闭环**；（1X 那组 14 B / 12 s / 实时率 0.42× 本书没读到一手出处，只当量级 [推]）差的根源是表示：一次 5 秒 rollout 要吐 120 帧 × 2.76×10^{6} B = 3.3×10^{8} 字节的画面，而 5 秒的关节指令只要 2.9×10^{4} 字节，**比值 ≈ $\mathbf{1.1\times10^{4}}$**——注意这是字节比，与“慢多少倍”是两个各自独立的比值，不能由前者推后者；因此世界模型的当下价值在数据生成与评测筛子两端（评委真正的卖点是可复现，不是省钱），且“用模型评模型”存在错误相关的循环风险；三家落在的格子不同——Cosmos 3 主格是造数据、Genie Envisioner 落在评委与可交互环境（名字里的 Sim 是这一格的线索 [推]）、宇树 WMA-0 官方自陈是仿真引擎加决策增强，**没有一家把自己放在 1 kHz 的位置上**。

证据等级：Cosmos 三阶段时间线，其中 4B 与 12B 两档是一手（arXiv 2501.03575v3 逐字：`two base models that are 4B and 12B in size`），而 Super 64B / Nano 16B / Edge 4B 这套档位名与“Orin/Thor 可跑”、Genie 3 的 24 fps@720p（**已回读到 DeepMind 原文**，逐字：`resolution of 720p、24 frames per second、retaining consistency for a few minutes` [查]；旧稿多写了一个 20，原文没有）与预览状态、1X 的 14B/11 s/1 s、GE-Act 的 160 M/200 ms/54 步、GAIA-2 的 `up to five`

`temporally and spatially consistent camera streams`、`resolution of 448×960`、`each spanning 2 seconds`、`8.4B parameters`（**2026-10-09 已读到 Wayve 论文原文** [查]，arXiv 2503.20523；旧稿这组数属二手，现已升一手）、Tesla 的共用仿真器说法，全部来自厂商仓库、技术报告或媒体报道 [查]，**多数为厂商自陈、本书未复现、多数没有版本号**；41.7 ms（1÷24）、2.1 拍（41.7÷20）、41.7 拍、3.3×10^{8} B（120 帧一手）、2.9×10^{4} B、比值 1.1×10^{4}、67 ms（1÷15）、1.39（2.13÷1.53）、8 GB、180 倍是 [算]（0.42× 那条来自 1X 的二手数，只当量级 [推]，不参与任何单点结论）（**其中像素那组是跨厂拼的量级，本书在正文里明写**）；宇树 WMA-0 的两用口径 [厂]（自陈）；**15 Hz 已撤回**——那是 NVIDIA Cosmos 3 Edge 在 Thor T5000 上的数，宇树 WMA-0 的页面与仓库对 Hz 零命中 [未]，其参数量、训练时长、任务数在仓库内完全未声明 [未]；“筛子不是判据”“错误相关”是方法学判断 [行]；**各家世界模型的训练数据、评测基准的绝对有效性、是否已在出货产品里承担控制、权重与版本的可获取性，均未公开** [未]。

可核对清单：

- ☐ 打开 `NVIDIA/Cosmos` 仓库，确认它的 license 与“Edge 4B 可跑在 Orin/Thor”的说法
- ☐ 打开 `unitreerobotics/unifolm-world-model-action`，找它自己怎么描述用途（仿真数据 vs 决策），以及 15 Hz 出现在哪一句
- ☐ 自己除三遍：12,000 ms ÷ 100 ms、÷ 20 ms、÷ 1 ms，看三个结果分别是 120、600、1.2×10^{4}
- ☐ 再算一遍表示的量级：5 s × 24 fps（一手）× 1,280 × 720 × 3 B ≈ 3.3×10^{8} B，而 5 s × 50 Hz × 29 × 4 B = 2.9×10^{4} B，除一下是不是约一万倍
- ☐ 用 4B × 2 字节复算 8 GB，判断它能否进 16 GB 模组（第 26 章那条）
- ☐ 下次看到“世界模型让机器人实时规划”这类说法，问一句：它的实时率是多少？输出是像素还是动作？

下一章：第 4 部到此结束——大脑的事讲完了，而它没有解决任何一件物理上的事。第 5 部去看这台机器真正撑不住的地方：电池、热、可靠性，以及“演示”与“连续作业”之间那道无法用模型填平的沟。

第 35 章

电池：13 串、9000 mAh、两小时

官方给了三个数，我们能推出第四个数

宇树电池侧的公开口径（[厂]，抓取 2026-10-07/08）：

```
G1：13 串 / 9000 mAh，充电 54 V 5 A，约 2 小时充满，续航约 2 h
H1 / H1-2：15 Ah / 0.864 kWh，最大电压 67.2 V，**由两块电池组成**；官方页面
**没有续航小时数**
H2：15 Ah / 0.972 kWh，最大电压 75.6 V
R1：续航约 1 h；容量、电压、串数均未公布
全系：电芯型号、化学体系、能量密度（Wh/kg）均未公布
```

最后那行是本章存在的理由：**厂商不说的东西，有些是可以算出来的**。而算出来不是为了替它填表，是为了**知道它没说的是哪一类**，以及判断这个选择会带来什么后果。

对机器人来说，到目前为止，续航不是瓶颈。

王兴兴 — 投中网（同上，来源 B） — 页面 2025-08-07；访谈本体约 2020 年前后（早年访谈转写）



图 35-1　串数→电压→化学体系：67.2/16=4.20 与 0.864/15/16=3.60 两条反推

第一步：串数与电压必须自洽

锂电池的“串数”（notation：13S、16S、18S）决定电压：每颗电芯串联起来，电压相加。

先看官方给的三个最大电压，做一次除法：

```
H1：67.2 V ÷ 16 = 4.20 V/颗
H2：75.6 V ÷ 18 = 4.20 V/颗
G1：54.0 V ÷ 13 = 4.15 V/颗
```

（[算]。H1/H2 的串数官方没直接写，但 67.2 与 75.6 这两个数**只能**由 4.2 V 的整数倍得到：67.2/4.2 = 16 恰好、75.6/4.2 = 18 恰好。所以“16 串 / 18 串”是从官方数字反推出来的，本书标注为 [算]。）

4.20 V 是充电终止电压。 这个数字非常关键，因为它锁定了化学体系：

体系	标称	充电终止	匹配吗
钴酸锂 LCO / 三元 NMC	3.6–3.7 V	**4.2 V**	✅ 匹配
磷酸铁锂 LFP	3.2 V	3.65 V	❌ 16S 只能到 58.4 V
高压三元	3.65–3.8 V	4.35–4.45 V	❌ 除不尽 4.20

（电芯特性是行业通识 [行]。）

结论：宇树人形用的是 4.2 V 体系的钴基/三元锂电芯，不是磷酸铁锂。这是从三个官方电压用一次除法得到的。

第二步：用能量再验一次

好的推断要能被第二条独立数据检验。用“能量 ÷ 容量 = 标称电压”：

```
H1：0.864 kWh ÷ 15 Ah = 57.6 V  ⇒ 57.6 ÷ 16 = 3.60 V/颗   ✅ 标称 3.6 V
H2：0.972 kWh ÷ 15 Ah = 64.8 V  ⇒ 64.8 ÷ 18 = 3.60 V/颗   ✅ 标称 3.6 V
G1：13 串 × 3.6 V × 9 Ah = 421 Wh（官方未给 Wh，此为推算）
```

（[算]。）

两条完全独立的路径（最大电压÷串数、能量÷容量）都落在 3.6 V 标称 / 4.2 V 终止上。这个一致性就是本书敢写“三元或钴酸锂体系”的理由。

而 G1 那一行给出一个有用的数：**它的电池约 421–432 Wh**（本书两种算法的区间），而第 20 章用的 0.43 kWh 就是这个数。

第三步：这个选择意味着什么

现在“它没说什么”变成了“它选了什么、代价是什么”。

选 4.2 V 三元，得到的是能量密度。三元锂的质量能量密度典型 200–260 Wh/kg，LFP 典型 150–190 Wh/kg [行]。对一台体重已经 70 公斤、每加一公斤电池就多耗一公斤承重扭矩的机器（第 16 章的 L^4 律），**能量密度是第一位的。**

代价是三条，每一条都对应本书前面的一章：

一、循环寿命短。三元典型 500–1500 次循环到 80% 容量，LFP 可到 3000 次以上 [行]。算一下：H2 官方续航 3 小时，若一天用两次、每次约 1 小时，那每次只放掉三分之一包电 ⇒ 一天 2 ÷ 3 = 0.67 个循环 ⇒ **一年**

约 240 个循环。听起来还有富余——但人形在工厂里是两班倒连续用，实际会到 500–700 循环/年，**即 1–2 年内电池就要换**。而第 37 章会说：换电池这件事的停机成本，在客户那里是硬指标。

二、热稳定性差。三元的热失控起始温度约 210–250 °C，LFP 约 500 °C [行]。而第 26 章算过那颗大脑的功耗（130 W 量级）、第 36 章算过关节的铜损，一台人形的密封躯干里它们同时在做热。**电池不是单独工作的，它和最大的两个热源关在同一个壳里**。

三、电压平台斜率大。三元的放电曲线有明显斜率（4.2 V → 3.0 V），而 LFP 在中间一大段几乎是平的。这意味着三**元的电压可以很好地指示剩余电量，但也意味着负载一重电压就掉**。

第三条直接连到第 13 章那个经典事故：

```
落地一次约 60 J、约 100 ms 进机身 ⇒ 瞬时约 600 W ≈ G1 整机平均 216 W 的
2.8 倍
⇒ 电池内阻上的压降 = I × R_internal 突然变大
⇒ 母线电压塌陷
⇒ 若跌到 DC-DC 的最低输入以下 ⇒ MCU 复位 ⇒ 整机失去控制
```

一台正在单腿支撑的人形突然失去控制，等于自由摔倒。所以“电压塌陷”在四足上是重启一次，在人形上是摔一台 8.5 万的机器，或者伤到人。

这就是为什么母线电容（第 13 章）在人形上不是可选件，以及为什么 BMS 必须把电芯温度与母线电压**报进整机总线**（宇树的 `MotorState` 里有 `vol` 与两路 `temperature` [厂]，说明这一层是接通的）。**这两件事在时间上是同一件事的两端：电容要在上电的头十几毫秒里被安全地充起来，而它的状态要在整个工作区间里被实时看见——本章下面「从上电到出力的那 200 毫秒」一节把这条链从头走到尾**。

第四步：算一算“能量密度”这条线走到哪了

用官方数做一次整机口径的能量密度：

```
H2：0.972 kWh = 972 Wh，整机 70 kg
    电池包重量官方未公布 [未]
    若按典型电池包 5 Wh/kg 的整机占比反推……

换一种算法：用人来比
    人 70 kg，一天约 2500 kcal = 2.9 kWh 的食物能量 [查]
    但人是“燃料在外、随吃随补”，机器是“能量全部背在身上”

    机器可背的上限：约 1–1.5 kg 电池 / 10 kg 体重（结构承受得起的量级）[行]
    H2 若电池包 8 kg、972 Wh ⇒ 电芯级约 120 Wh/kg（含包材与 BMS）
    而电芯本身 200–260 Wh/kg [行] ⇒ 包级折损约 40–50%
```

（[算]，电池包重量是本书假设，官方未公布 [未]。）

这一串算出来的最有用的结论是：包级能量密度大约只有电芯级的 50–60%。因为电池包不只有电芯：还有 BMS 板、接触器、熔断器、均衡电路、结构框架、热界面材料、以及**必须留的余量**（不能放到 0%，也不能充到 100%）。

所以“再等等能量密度突破”这条希望，实际会被包级折损吃掉一半。而人形续航的真正出路不在电芯，在下面这三条：

1. **换电 / 不断电快换**（银河通用 S1 官方宣称支持不断电快换 [查]，把“充电 2 小时”变成“换电 30 秒”）；
2. **降低平均功率**（第 20 章：支撑相的铜损是大头，也就是第 14 章准直驱与第 36 章热设计的战场）；
3. **减少需要自持重量的时间**（轮式底盘 + 上半身人形——这正是宇树规划里“11.5 万台轮式 vs 7.5 万台双足”这个比例的含义 [推]）。

第三条最被忽视，也最可能是答案。一台不需要靠自己的腿扛住 70 公斤的机器，电池可以小得多。而市场用销量投票说：**轮式比双足多 53%**。

从上电到出力的那 200 毫秒

先给这一节定性。宇树的母线电容容值、有没有预充电路、DC-DC 有几路什么拓扑，全部未公布 [未]（第 13 章已经声明过：470 μF 是本书举例用的量级，不是宇树数据）。所以下面每一句都是**通用做法 + 为什么必须这么做**，不是“宇树这么做”。

按下电源的那一瞬间，第一个物理事实是：**一颗放电到 0 V 的母线电容，在接通的那一瞬间等于一根短路线**。为什么？因为电容的电流只由电压变化率决定：

```
i = C · dV/dt
```

（i 是瞬时电流 [A]，C 是容值 [F]，dV/dt 是母线电压上升率 [V/s]。）刚接通时 dV/dt 只被回路里剩下的那一点电阻限制——电容的等效串联电阻、母线铜排、线束、触点电阻，加在一起通常是毫欧量级。所以电流的理论解是“无穷大”，实际解是“由那些你没打算让它承载的电阻决定”。

那这个电流又会烧什么？ 三样，按代价从小到大：

一，**整流桥与体二极管的浪涌**。第 13 章写过电机发电时电流经下管体二极管灌进母线电容；上电瞬间是同一件事的反方向版本，pn 结的浪涌能力用 I^2t（电流平方对时间积分）来衡量，一次几十微秒的几百安培就能把它用掉。

二，**保险炸**。快熔保险的分断能力是按预期短路电流选的，而冷电容的浪涌恰好落在这个数字旁边——**结果是它在一个正常上电动作里炸掉**，用户看到的现象是“新机开箱按电源就没反应”。

三，**触点焊死**。这是最阴的一个：继电器/接触器在带载闭合时拉弧，弧光把触点表面熔化，断开时重新凝固——**熔焊**。第 37 章讲过它的沉默形态：该断不断，正常运行里一次都不露头，只在真正按下急停那天现形。上电浪涌就是给每一次上电都在给触点积累这份损伤。

所以标准答案是预充（pre-charge）：先串一只限流电阻把电容充起来，充到位之后再用一只旁路触点把电阻短接掉。 为什么短接而不是让电阻一直挂着？因为电阻上有压降 I·R，持续挂着它就持续发热，而这条线后面要过几十安培。**预充是时间换能量，不是省能量**——这一点马上会算出来。

于是本章给一个读者能自己复算的量。用电阻-电容充电的时间常数：

```
τ = R · C = 10 Ω × 470 μF = 4.7 ms
```

（τ [s] = R [Ω] × C [F]；470 μF = 4.7×10^{-4} F；10 Ω 与 470 μF 都是本书取的举例量级 [行][算]。）电容充到目标电压的百分比按 1 − e^(−t/τ) 走：1τ 到 63.2%，2τ 到 86.5%，3τ 到 95.0%。**“充到 95% 需要约 3τ ≈ 14 ms”**，反解出来的式子是 t = −ln(1−0.95)·τ ≈ 3.0τ。

这个数字决定了“上电到第一帧指令之间至少要等多久”的下限。 为什么是下限而不是全部：预充只保证母线电容到位，它后面还排着 DC-DC 各自的软启动、MCU 从 bootloader 跑到第一条总线报文、以及第 37 章那套上电自检（读编码器、校验零点、确认热态）。但**反过来说，这 14 毫秒里任何一帧关节指令都是无效的**——母线还没起来，此刻出力，出的是“半有电”那一档的力。第 37 章的原话在这里成立：**安全的敌人不是没电，是半有电。**

那本节标题为什么写 200 毫秒？ 因为这条链上唯一能核到的两个数是可加的：预充那 14 ms，加上第 37 章给抱闸的那句“机械动作，典型几十毫秒”[行]（开与合各算一次，取两个几十毫秒）。

```
14 ms + 2 × 几十 ms ≈ 一百多 ms
```

（[算]，14 ms 来自上面的 3τ，抱闸那句沿引第 37 章 [行]。）中间那段没有公开数的部分——DC-DC 软启动、bootloader 到第一帧报文、自检——就是“一百多毫秒”与“两百毫秒”之间的余量。**所以 200 毫秒是本书给这条链的量级上界，不是宇树公布的时序 [未]**；宇树未公布任何上电

时序参数 [未]。读法只有一条：这段预算里的每一格都要有人替它兜底（第 37 章的自检清单），否则机器会在“母线已经有了、环路还没有”的那几十毫秒里被下一节讲的半有电状态击中。

顺手把浪涌限住了多少，也能算：

```
I_precharge = V_bus / R = 54 V ÷ 10 Ω = 5.4 A
```

（54 V 是宇树 G1 官方充电电压口径 [厂]，10 Ω 是本书举例值 [算]。）而不做预充时 I = V_bus ÷ (ESR + 线阻 + 触点电阻)——分母宇树未公布 [未]，所以本书**不给这个对比数**，只留算式给读者量自己那台机器。

预充的代价在能量那一栏，而且它违反直觉。 电容最后存下的能量：

```
E = 0.5 × C × V² = 0.5 × 4.7×10⁻⁴ F × (54 V)² ≈ 0.68 J
```

（[算]。）而 RC 充电的经典结论是：**不管 R 取多大，电阻上烧掉的能量都等于 ½CV²**——把 R 加大只是把这 0.68 J 摊得更长、把功率峰值压得更低。所以“预充省能量”是错的，“预充省触点与半导体”才是对的。**这条链真正的断点在快速上下电循环**：电阻是耗能件，两次上电之间它需要时间凉下来；一台调试中被反复断电重启的机器，最先失效的往往是这只电阻和那个旁路触点，而不是电容。

电容之后是多路 DC-DC。**为什么不是一个开关把 54 V 直接降到 3.3 V？因为一路 DC-DC 就是一个噪声域。** 按通识至少分三档：

轨	挂什么	为什么要单独一路
12 V	抱闸线圈、继电器/接触器、风扇、执行器侧驱动	线圈与风扇是感性开关负载，反灌噪声最大 [行]
5 V	RS-485/CAN 收发器、运放与隔离放大器、编码器供电	电流采样的基准链在这里，不能被数字侧拖 [行]
3.3 V	MCU、存储器、SoC 逻辑侧、传感器数字接口	逻辑地与 ADC 参考要分开，否则采样窗口跟着抖（第 13 章）

宇树各轨实际电压与各路负载未公布 [未]，宇树侧唯一的一手口径是它的人形可选 Jetson Orin NX 16 GB 与 Thor T5000，官方功耗区间分别是 10–25 W 与 40–130 W [厂]。拿最大那一档做一次除法，就知道 12 V 轨要过多少电流：

```
I = P / V = 130 W ÷ 12 V ≈ 10.8 A
```

（[算]，输入 [厂]；这是“大脑全部从 12 V 轨取电”这一假设下的电流，不是宇树公布值 [未]。）**十安培级别的持续电流在任何一路 3.3 V 上都是不可想象的**——这就是“多路”的第一层理由，与噪声无关，是铜和连接器。

接下来是关键一问：BMS 的电压、电流、温度、SOC，为什么必须上整机总线？ 因为本章前面那条判据——落地那一次的瞬时功率接近整机平均的两倍、峰值按 2–3 倍取量级、内阻压降把母线拉塌——**它是一个需要有人实时看见才能成立的保护**。看不见它，保护只剩两种形式：要么跳闸（整机失电，人形当场摔），要么什么都不做（MCU 复位，同样摔）。看得见它，才有第三种：**运控层主动降功率**。用一手数把这件事量化：G1 平均电功率约 216 W [算]（第 20 章那条 432 Wh ÷ 2 h，第 26 与第 36 章复用），46.8 V 标称母线下的平均电流：

```
G1 自己的一组官方数：13 string lithium battery、9000 mAh、`Battery Life
About 2h`、`About 35kg`
⇒ 0.432 kWh ÷ 2 h = 216 W；13 × 3.6 V = 46.8 V（标称母线，3.6 V/颗是
[行]）
I_avg = 216 W ÷ 46.8 V = 4.6 A        ⇒ 峰值按 2–3 倍 ⇒ 9 – 14 A
（H2 那包 0.972 kWh / 15 Ah / 75.6 V 也是一手，但官方没给续航小时 ⇒ 本书不
给它的平均电流）
（旧版这里除的是 G1 那包的 54 V，机器配错了；H2 是 18 串，标称 64.8 V。）
```

（[算]。倍数不再沿用别处：第 13 章那一次落地是 60 J ÷ 100 ms ≈ 600 W，一只关节就接近整机平均的 1.9 倍，本书按 2–3 倍取整机瞬时峰值的量级 [推]，厂商未公布峰值与母线电流 [未]。）5 A 与 15 A 之间那一段，是控制器唯一能主动收手的窗口；**这个窗口只有几毫秒到几十毫秒，所以**

电池侧的四个量必须按毫秒级周期出现在总线上，而不是按“人看一眼屏幕”的秒级。 宇树侧能核到的证据只有 `MotorState` 里带着 `vol` 与两路 `temperature` [厂]——它证明“母线电压与温度进了关节状态报文”这一层是接通的；至于是否存在独立的 BMS 报文、SOC 以多大频率上总线、由谁消费，官方未公布 [未]。

那 SOC 为什么这么难报准？ 回到本章第三步那条“电压平台斜率大”：三元的电压能指示剩余电量，但负载一重电压就掉——所以 BMS 必须在“带载的斜线”和“静置的开路电压表”之间反复折中（库仑计积分电流对时间、静置时用 OCV 校准，行业通识 [行]）。它估不准的代价不是“显示 20% 其实是 15%”这种小事：**它是最后一格电时那一次突然塌陷**——因为电芯接近放空时内阻上升，同样的 10–15 A 峰值要拿走更大的压降。SOC 估错的机器会在一台正在单腿支撑的人形上做最后一段路。

这条“折中”能不能算成一个数？能，而且算出来就知道难在哪。 三元电芯平台段的开路电压斜率，本书取 **dV/dSOC ≈ 0.2–0.35 V**（行业通识 [行]；上一版这里写的是“0.1 V 跨 60% SOC”即 0.167 V，那比常见三元配方平了两到三倍，会把下面两个百分比放大同样的倍数，所以斜率一律带区间给）：

```
采样误差 10 mV                  ⇒ ΔSOC = 2.9 % – 5.0 %
带载压降 I·R = 10 A × 5 mΩ = 50 mV ⇒ ΔSOC = 14 % – 25 %
两者之比 = 50 mV ÷ 10 mV = 5 倍       —— 斜率消掉了，这五倍与取值无关
```

（[算]。三**处口径要说清**：一，这里的 10 A 是本章上面那组“平均 4.6 A、峰值按 2–3 倍取 9–14 A”的**峰值上沿取整**，不是平均母线电流；二，5 mΩ 是**电芯级**假设而 10 A 是**整包**电流，只有把那一包当成 1P18S（单串）时两个数才在同一侧——宇树未公布 H2 那包的串并联 [未]，若是 2P，每颗只走 5 A，压降与上面的百分比一起减半；三，50 mV 已经接近平台跨度本身，所以“带载看电压”不是精度问题，是量程问题。）

库仑计那一路的难点也不在 LSB。 量化下限要算，但它是这一栏里最小的一项：

```
量化：满量程 ±20 A ⇒ LSB = 40 A ÷ 4096 = 9.8 mA
      3 h 累计 = 9.8 mA × 3 h = 29 mAh，15 Ah 是 15,000 mAh ⇒ 占 0.20 %
零点偏置 10 mA（运放与霍尔的常见量级 [行]）
      3 h 同样累计 30 mAh，同样占 0.20 % —— 一项偏置就抵得上整条量化
增益误差 0.5 % ⇒ 把这 3 h 放走的 15 Ah 记错 0.5 % = 0.50 %（2.5 倍于量化）
温度漂移：上面两项都随温度走，规格书不给系数就不知道往哪边偏 [未]
```

（[算]；满量程、位数、偏置与增益全是本书取的量级，宇树未公布 H2 的电流采样链 [未]。）**所以“库仑计三小时才漂 0.20%”是一个稻草人**：它只数了量化那一项，而 10 mA 的**偏置**能造出同样大的误差、0.5% 的**增益**误差是它的两倍半，再加上这两项自己都在漂。**三个数一比，问题就换了**：带载电压给出的偏移是采样误差允许的**五倍**，而这一倍与斜率无关，所以“带载看电压”当场失效；库仑计又被自己的模拟前端压着——**难的根本不是积分，是起点和仪器**：积分器必须有一个初始值，而初始值只能靠静置 OCV 标定，静置又要求这台机器**停下来不干活**。本章前面那句“最后一格电突然塌陷”于是有了确切的成因：不是算错，是**这台机器没有机会校准**。宇树未公布 BMS 的 SOC 算法与上总线频率 [未]，本书给的是判据，不是实现。

上电这一侧的时序，与第 37 章断电那一侧是互为倒数的两条链：断电是“命令扭矩归零 → 机械稳定 → 断抱闸励磁 → 断动力并保留控制电”，上电反过来是“先给控制电与母线电容充电 → 各路 DC-DC 起来 → 状态自检 → 打开抱闸 → 才允许第一帧力矩指令”。**唯一的差别在于开头那一步：上电的第一件事是充电容，断电的最后一件事是放电**。第 13 章那条泄放通路（制动电阻或双向 DC-DC）负责的就是后半句——没有它，两次断电之间母线电压不会自己下来，下一次上电时预充要对着一个已经带电的电容工作，τ 那套算法当场失效。

类比，以及它在哪儿破

类比：电池是油箱，能量密度是每升油能跑多远。

这个类比破在一个决定性的地方：汽车的油箱只装能量，而机器人的电池包要同时装**能量、功率、安全、和智能**。

- 能量（Wh）决定跑多久；
- 功率（W，或“C 数”放电倍率）决定落地那一瞬间能不能给出来；
- 安全决定它烧起来会不会伤人（第 37 章）；
- 智能（BMS）决定控制器能不能知道还剩多少、多热。

这四个要求互相冲突。 高能量密度的电芯通常功率密度低（正极高容量就要厚，厚就慢）；高功率电芯能量密度低。所以选电芯本身就是一次取舍，而宇树选了 4.2 V 三元——**偏能量那一侧**。

这解释了一个可观察的现象：人形机器人的峰值动作（跳跃、奔跑、被推后的大幅恢复）通常是短时爆发，因为它受限于电池的持续放电倍率而不是能量。 而第 20 章说的“额定扭矩厂商不给”，有一部分原因就在这里——**额定值取决于电池能持续供多少，而不只是电机能持续散多少**。

一个可检验的说法：由官方公布的三个最大电压（54 V / 67.2 V / 75.6 V）做一次除法可得每颗电芯 4.15–4.20 V，由官方能量÷容量可得标称 3.60 V/颗，两条独立路径同时指向 4.2 V 终止 / 3.6 V 标称的钴基或三元体系（而非 3.65 V 的磷酸铁锂）；该选择换来能量密度、付出循环寿命（约 500–1500 次）、热稳定性（热失控起始约 210–250 °C 对 LFP 约 500 °C）与电压平台斜率（负载一重即塌陷，进而可能触发 MCU 复位）三项代价；而包级能量密度通常只有电芯级的 50–60%，因此续航的真正出路是换电、降平均功率、以及减少需要自持重量的时间。上电那一侧同样有一笔可算的账：放电状态的母线电容在接通瞬间等效短路，故标准做法是串限流电阻预充、充到位后用旁路触点把电阻短接；按 $\tau = R \cdot C$、10 Ω 与 470 μF 得 $\tau \approx 4.7$ ms，充到 95% 需约 $3\tau \approx 14$ ms——这段时间里任何关节指令都无效，因而它是“上电到第一帧指令”的时间下限；把这条链上本书能核到的两段相加（14 ms 加两次“典型几十毫秒”的抱闸动作）得一百多毫秒，标题那个 200 毫秒是这条链的**量级上界**而不是厂商公布的时序 [算][未]；预充不省能量（RC 充电在电阻上烧掉的能量恒等于 $\tfrac{1}{2}CV^2$

≈ 0.68 J），它省的是触点、保险与半导体；而 BMS 的电压/电流/温度/SOC 必须上总线，是因为 4.6 A 平均（216 W ÷ 46.8 V）到 9–14 A 峰值之间只有毫秒级的主动降功率窗口，看不见它就只能跳闸或复位。

证据等级：13 串、9000 mAh、54 V 5 A、15 Ah / 0.864 kWh / 67.2 V、15 Ah / 0.972 kWh / 75.6 V、R1 续航约 1 h、`MotorState` 里的 `vol` 与两路 `temperature`、Orin NX 10–25 W 与 Thor T5000 40–130 W [厂]；所有除法、串数反推、421/432 Wh、120 Wh/kg、循环数、50–60% 折损，以及新增的 τ = 4.7 ms、3τ ≈ 14 ms、54 ÷ 10 = 5.4 A、½CV² ≈ 0.68 J、130 ÷ 12 ≈ 10.8 A、216 ÷ 46.8 = 4.6 A、14 ms + 2 × 几十 ms ≈ 一百多 ms、ΔSOC = 2.9%–5.0%（10 mV）与 14%–25%（50 mV）、50 ÷ 10 = 5 倍（与斜率无关）、40 A ÷ 4096 = 9.8 mA、9.8 mA × 3 h = 29 mAh ÷ 15 Ah = 0.20%、10 mA 偏置 3 h = 30 mAh = 0.20%、0.5% 增益 = 75 mAh = 0.50% 是 [算]（后七个数的输入里，470 µF 与 10 Ω 是本书举例量级、不是宇树数据 [未]，“大脑全部从 12 V 轨取电”是本书假设 [未]，抱闸那句“典型几十毫秒”沿引第 37 章 [行]；SOC 那一组里，**平台段 dV/dSOC 本书取 0.2–0.35 V 这一区间——上一版写的 0.167 V（“0.1 V 跨 60% SOC”）比常见三元配方平了两到三倍，会把百分比同倍数放大，已改成带区间的口径** [行]；±20 A 满量程、10 mA 偏置、0.5% 增益全是本书取的采样链量级，宇树未公布 [未]；10 A 是那组数的**峰值取整**而不是平均 4.6 A，5 mΩ 是电芯级而 10 A 是整包电流，两者同侧的前提是 1P18S，官方未公布串并联 [未]）；电池包重量与内阻是本书假设 [未]；电芯特性（4.2/3.6、LFP 3.65/3.2、能量密度、热失控温度、循环数）、预充回路与触点熔焊、RC 充电在电阻上耗散 ½CV²、DC-DC 分轨与噪声域隔离、库仑计 + OCV 混合估计 SOC，是电力电子与电池行业通识 [行]；不断电快换、轮式与双足的产能规划比例 [查][推]；**电芯型号、供应商、化学体系、能量密度、包重、放电倍率、BMS 策略、循环寿命，以及母线电容容值、是否有预充电路、DC-DC 路数与拓扑、上电**

各段延时与整机上电时序、BMS 报文是否存在与 SOC 的上总线频率，官方全部未公布 [未]——本章的六条推断都是“从公开数反推”，不是厂商确认；上电时序那一节是通用做法，不构成对宇树实现的断言。

可核对清单：

- ☐ 自己除三次：67.2/16、75.6/18、0.864kWh/15Ah÷16，确认都落在 4.2 与 3.6
- ☐ 拿一份 LFP 电芯规格书，看它的充电终止电压，然后算 16 串 LFP 能不能到 67.2 V（答：58.4 V，不能）
- ☐ 用 432 Wh ÷ 2 h ÷ 35 kg 复算第 20 章那个 6.2 W/kg，再用 216 ÷ 46.8 复算 4.6 A
- ☐ 在宇树 G1/H1/H2 页面搜“电芯 / 型号 / 能量密度 / 循环 / 放电倍率 / C 数”，确认是否真的全无（预期：全无）
- ☐ 自己按 `τ = R·C` 换一组值：C = 1000 μF、R = 22 Ω ⇒ τ = 22 ms、3τ = 66 ms；再用 t = −ln(0.05)·τ 复核“95% ≈ 3τ”
- ☐ 拿你手上任意一台带母排的设备量一次冷态上电的电流峰值（钳形表或示波器串采样电阻），把它和 V_bus ÷ R_precharge 比——**这一步是本章唯一能在你自己机器上做的实测**
- ☐ 在宇树文档中心搜“预充 / pre-charge / 上电时序 / 软启动 / DC-DC / BMS”，确认是否有任何一项被公布（预期：全无 [未]）

下一章：本章说电池选的是能量密度那一头，代价之一是热。下一章把热这件事算完——它是全书所有线索汇合的地方：铜损、开关损耗、磁钢退磁、密封壳、大脑的 130 瓦，全在同一台机器里抢同一份散热能力。

第 36 章

热：密封壳里电机与驱动互相加热

一个所有线索都指向的空白

从第 02 章起本书就在说同一件事：**宇树全系关节电机、全整个人形产品线，没有任何一处公布“额定（持续）扭矩”**。只有峰值。

第 04 章给了机理的一半（k_t 随温度漂），第 13 章给了损耗的来源，第 35 章给了电池为什么偏能量那一侧。这一章把它们接完，并且回答那个问题：**为什么这个数不是“不给”，而是“给不出来”**。

而且大家可能想象不到，机器人不怕冷，低温没什么问题，但机器人很怕高温。不过很多情况下如果真的热了，它自己趴下休息一会就好了。……我觉得这个可能当下是有些问题，但不是个关键性的问题。

王兴兴 — 凤凰卫视《X+》专访（主持人田桐），腾讯新闻转载 — 2025-02-21（直接引语（问答按轮次归属））

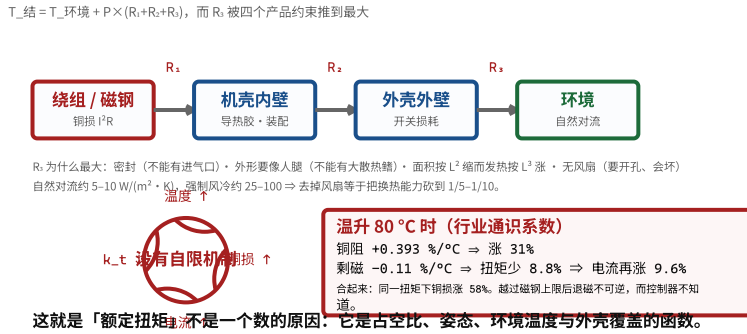


图 36-1　串联热阻链 R₁R₂R₃ 与那条热—铜损—退磁正反馈环

先把热路画出来

一颗关节里的热，从产生到离开机体要走一条串联的路：

```
磁钢与绕组（铜损发生地）
   ↓  R₁  绕组 → 铁芯 → 机壳
机壳内壁
   ↓  R₂  壳内空气/填充胶 → 外壳
外壳外壁
   ↓  R₃  外壳 → 周围空气（自然对流 + 辐射）
环境
```

串联热路的基本式子，与欧姆定律同形：

```
T_结温 = T_环境 + P_损耗 × (R₁ + R₂ + R₃)
```

这条链上最要命的是 R_3。因为 R_1、R_2 是设计者能控制的（加导热胶、把驱动板贴着壳、用铝壳），而 R_3 取决于**外壳面积与有没有风**。

而一台密封的、IP 防护的、外形要像人的关节，恰好把 R_3 做到了最大：

设计约束	对 R_3 的影响	出处
密封（防尘防水）	不能有进气口 → 只能自然对流	第 37 章的 IP 要求
外形要像人腿	表面积不能太大，不能有散热鳍	产品定位
关节要小（第 15 章的 530 g）	表面积按 L^2 缩，而热要按体积（L^3）出	平方-立方律
无风扇	风扇要开孔、会坏、有寿命	可靠性

最后一行有个可算的惩罚：自然对流换热系数约 5–10 $W/(m^2 \cdot K)$，强制风冷约 25–100 $W/(m^2 \cdot K)$。也就是说，去掉风扇等于把换热能力砍到 1/5 到 1/10 [行]。

把这个系数乘上真实的表面积，就能看到“额定扭矩”为什么必然是一个占空比函数。 输入是 55 W——本节下面那段会把一颗持续出 60 牛米的膝关节的损耗算成 50–60 W，这里先借它的中值。**但壳得用这颗自己的壳。** G1 的膝是 N7520-22.5，附录 C.2a 记它的外形为 φ75 × 20 mm 级：侧面积 πdL = 0.0047 m²，连两端面 0.0136 m²。第 43 章那台 IM6014 是 φ65 × 60 mm [厂]，侧面积 0.0123 m²，连端面 0.019 m²——**面积大 1.4–2.6 倍，于是温升被算小同样的倍数**，这是借器件最容易犯的一种错，所以两行都摆出来（ΔT = P ÷ (h·A)，P = 55 W）[算]：

h（W/(m²·K)）	机理	N7520 侧面 / 含端面	IM6014 侧面 / 含端面
8	自然对流	1,459 / 507 K	561 / 364 K
13	对流 + 辐射	898 / 312 K	345 / 224 K
40	强制风冷	292 / 101 K	112 / 73 K

三档都不够——可“够”的标准得先钉死，否则这句话是空的。 本章上面那条热阻链给的预算是 55 W × 1.09 K/W ⇒ **ΔT ⩽ 60 K**（结温 100 °C、环境 40 °C）；而绕组绝缘按 F 级 155 °C 那一档，环境 40 °C 时对应 **115 K**。**这两个数不是一回事**：115 K 是“壳到绝缘失效”的绝对上限，60 K 是“壳到绕组热点”的设计预算，中间那 55 K 要留给壳内到热点的 R_1、R_2——热点永远比壳体热。判“够不够”用 60 K 这一档：即使换成**大一号的 IM6014 壳**、再架上风扇（h = 40），也只有 73 K，**仍超 22%**。若只看 115 K 那一档，唯一进得来的恰好是 73 K 这一格，而它需要风扇与开孔，被第 37 章那条 IP 要求和整机外形否掉了。**换个外形、换个判据都改不了结论，这才是“三档都不够”该有的支撑。**

两处诚实的折扣。 一，`ΔT = P ÷ (h·A)` 把 h 当常数，可自然对流的 h 随 ΔT 按约 1/4 次幂上升、辐射按 $T^4 - T_0^4$ 走 [行]，所以表里那些几百 K 是**高估**，真实值大约低 1.5–2 倍——高估的方向让“不够”这个结论更稳而不是更脆。二，55 W 里的每一步都含本书假设，两颗关节的外形也都不是产品页给的持续工况。

所以这颗关节**必须把热导进结构**——R_1、R_2 那条路不是“可选优化”，是唯一走得通的路，机壳只是最后一环；这也解释了为什么本章前面那条 ≤1.09 K/W 的总热阻链里 R_3 只占一档而不是全部。

把同一个式子反过来用在整机上，还能得到一条不等式。用 G1（第 20 章里三个输入全是一手的那一档）：平均输入约 216 W [算]，假设躯干外壳有效面积 0.35 m^2、只有自然对流加辐射（h≈13）、允许 ΔT=45 K：

```
能散 = h·A·ΔT = 13 × 0.35 × 45 = 205 W
要散 = 216 W            ⇒ 205 ÷ 216 = 0.95：同一量级，几乎没有余量
```

[算]（面积与温差是本书假设，宇树未公布散热结构与降额曲线 [未]）。**这 1.6 倍就是必须三选一的数字理由**：让腿部与躯干外壳一起当散热器、连续作业时主动降平均功率、或者接受风扇与防护等级的代价。本书只给这条不等式，不替厂商选它选了哪一件。

一次具体的算术

设一颗膝关节持续输出 60 N·m（以 G1 的膝为例，N7520-22.5；走路时支撑相的量级，第 20 章）。**下面这一串算的是 G1 这一颗关节的损耗，后文 216 W 是同一台 G1 的整机平均输入**——同机对照，所以“一颗关节的热”与“整机的电”这两个量级可以直接相除。

```
转子侧扭矩 = 60 ÷ 22.5 = 2.67 N·m
所需电流 I = 2.67 ÷ k_t(转子)

k_t(转子) 官方未公布，用第 04 章 M8010 的量级类比：0.10 N·m/A
⇒ I ≈ 27 A（数量级，[算]，输入含假设）

铜损 = I² R = 27² × R
若 R = 0.040 Ω（全书统一口径，见第 09 章；官方未公布 [未]）
⇒ P_cu = 27² × 0.040 ≈ 29 W  —— 单这一颗关节
（旧版此处取 0.05 Ω 得 36 W。相电阻厂商从未公布，所以本书把铜损当 **28–36 W
的区间**，而下面热那一侧保守地按整颗关节 55 W 算—取区间上沿，结论不会因为选哪
个 R 而翻）
```

（[算]。）

而摩擦这一项，官方其实公布了模型与参数——在训练仓库里，不在产品页上。 `unitree_rl_lab` 的 `unitree_actuators.py` 定义了 `UnitreeActuator`（继承 `DelayedPDActuator`），带摩擦模型、延迟与 T-N 曲线限力。源码里的施加式是 [厂]：

```
τ_friction = Fs · tanh(q̇ / Va) + Fd · q̇        Va 默认 0.01 rad/s
armature   = J2 + J1·i2² + Jr·(i1·i2)²          折算到电机轴的等效惯量（官方定义原文）
限力按 X1 / X2 分段；X1 的官方注释是
   Maximum Speed at Full Torque (T-N Curve Knee Point)  Unit: rad/s
```

（[厂]，逐文件核对；各型号的 X1/X2/Y1/Y2/Fs/Fd/armature 数值见附录 C.2a。）

这一段推翻本书前面「L、R、J、摩擦、T-N 全部未公布」的说法，必须写实。 准确表述是：宇树**没有在产品页上**公布这些，但**在训练仓库里公布了**摩擦系数 Fs/Fd、折算惯量 armature、以及完整的 T-N 折点参数。

也就是说，第 08 章那个电压包络与本章那条热阻链，**有一半是能拿到官方数的**——你得去 `unitree_rl_lab` 拿，而不是去 `unitree.com` 拿。

这本身就是本书的一个方法论结论：一家公司的公开程度，不等于它的产品页的公开程度。 附录 E 因此把「官方仓库文件」与「官方页面」当作两个可分别引用的证据来源，正文里凡取自仓库的都注明文件路径。

再叠上驱动板（导通损耗 + 开关损耗，量级 5–15 W [行]）与减速器摩擦（10–15% 的通过功率，第 20 章的 0.61 效率），**一颗持续出 60 牛米的关节，内部要散掉约 50–60 瓦。 这两个数矛盾吗？不矛盾。** 因为这两个数根本不在同一栏：660 W 是**热**的最坏叠算（12 × 55 W，上面那颗关节在 60 N·m 下总损耗约 55 W：铜损 36 W + 驱动板 5–15 W + 减速器摩擦），216 W 是 **G1 电池侧输入电功率**的平均值。稳态下这 216 W 几乎全部在机内变成热（第 20 章那笔效率账，加上大脑的 40–130 W 也从同一块电池取），所以机内热的平均量级就是三百瓦级；而 660 W 只算关节就已经接

近它的两倍，这正说明 12 颗同时、持续、同相位地出 60 N·m 撑不住——真实走路时各关节的力矩是脉冲式的、相位错开的。**但这恰恰是“额定值”难给的原因**：

额定扭矩不是一个器件参数，是一个占空比、环境温度、姿态（决定哪几个关节在出力）、以及外壳是否被衣物/护罩覆盖的函数。

厂商如果印一个“额定 60 N·m”，客户会在夏天、在连续作业、在深蹲姿态下用到降额保护，然后投诉参数不实。**所以不印是理性的**。而这对客户是坏消息——它意味着**你必须在自己的场景里自己测**。

正反馈：那条会自己加速的链

第 13 章提过一次，这里给完整的环：

```
① 电流 → 铜损 I²R → 温度上升
② 温度上升 → 铜阻上升（+0.393 %/°C）→ 同样电流的铜损更大
③ 温度上升 → 磁钢剩磁下降（NdFeB 约 −0.11 %/°C）→ k_t 变小
④ k_t 变小 → 控制器为维持扭矩命令更大电流 → 回到 ①
```

（系数是行业通识 [行]。）

代 80 °C 温升：

```
铜阻涨：1 + 0.00393 × 80 = 1.314   ⇒ 铜损涨 31%（同电流）
剩磁降：1 − 0.0011 × 80 ≈ 0.912    ⇒ 同电流的扭矩少 8.8%
⇒ 要维持扭矩，电流要涨 1/0.912 = 1.096，即 +9.6%
⇒ 铜损 = I²R ⇒ 1.096² × 1.314 = 1.58，即铜损涨 58%
```

（[算]。）

从“冷态”到“热 80 度”，同一扭矩下的铜损涨了 58%。这就是为什么“额定”这件事不能只测一次冷机。

而这条链在 ③ 那里有一个**不可逆的出口**：如果温度高到超过磁钢的工作温度上限（N 档 NdFeB 约 80 °C、H/AH 档约 120–150 °C、UH 档约 180–200 °C [行]），退磁就变成**永久**的。此后这台关节的 k_t 永远小了，

而控制器不知道——它会一直命令更大的电流去补，于是更热。

宇树在 IM6014 上官方标“每秒 400 次温度监控” [厂]，`MotorState` 里带 `temperature int16[2]`（两路）[厂]。 这两条一手事实说明：**厂商知道这条链存在，并且已经在器件里放了两个温度测点。** 而磁钢牌号、工作温度上限、降额曲线，全部未公布 [未]。

大脑与关节抢同一份散热

第 26 章说过 Thor 是 40–130 W。现在把它放进热路里：

```
G1 总电功率约 216 W（平均，[算]）
其中大脑 40–130 W，腿部 12 个关节若干百瓦，感知与通信若干
⇒ 这些电几乎全部在机内变成热（稳态下热 ≈ 输入电功率），要从同一台密封机体里散掉
⇒ 而躯干外壳面积有限
```

关键耦合：躯干里同时装着电池（第 35 章：4.2 V 三元，热失控起始约 210 °C）、大脑（130 W）、和腰部关节的驱动板。 而 H2 官方给的双目工作温度是 −20 到 +60 °C / −30 到 +70 °C [厂]——**这是本书在**人形整机页面**上能找到的唯一一处官方温度指标，而且给的是相机，不是关节（器件页另有过热保护与工作温度，见第 15、24 章；整机页没有）**。

相机被放在最外面（头部），因为它最容易热坏，也最容易散热。而关节在结构内部，被衣服、护罩、以及彼此包围。

所以“夏天在户外干活的机器人”这个场景，卡点不是电池不够，是散不出去。 环境温度 40 °C、地表 55 °C 时，R_3 两端的温差预算只剩几十度，而 58% 的铜损增幅（上面算的）已经先发生了。

于是本章回答那个问题

“额定扭矩为什么不公布”的完整答案是三层：

1. **它不是一个数**：取决于占空比、姿态、环境温度、外壳覆盖情况（上面已证）。

2. **公布它需要暴露设计余量**：额定值 = f(磁钢牌号、热阻、降额策略)。给一个诚实的额定值，等于把这三项告诉竞争对手。
3. **行业没有强制它**：第 37 章会讲，人形目前没有任何一条已发布的强制标准要求厂商标额定扭矩。而汽车与工业伺服都有（因为有过热失效的诉讼史）。

这三层里，第 1 层是物理，第 2 层是商业，第 3 层是制度。而只有第 3 层是可以被改变的——**这就是为什么“人形机器人什么时候有强制安全与性能标准”是一个真问题**（第 37、39 章）。

给读者的一个可执行动作

如果你要选一颗关节用在你自己的机器上，**不要问“额定多少”，问这四句**：

> ① 在环境温度 X °C、外壳安装方式 Y、占空比 Z 的条件下，
> 能持续输出多少扭矩而不触发保护？
> ② 触发保护的温度阈值是多少，在哪里测的（绕组？壳？磁钢？）
> ③ 磁钢牌号与工作温度上限是什么？
> ④ 降额曲线有没有？

四个问题里，公开资料通常只能答第 ④ 个的一半。而答不出来的那部分，就是你必须自己测的部分。第 40 章的十二个月路线里，“关节温升曲线”是排在很前面的一项，原因就是它是**唯一能自己拿到、而厂商不会给的数据**。

类比，以及它在哪儿破

类比：关节像一台被关在没有窗户的房间里的大功率电暖器，而房间还要保持外观整洁。

这个类比在哪里破，破得很关键：电暖器的功率是恒定的，而关节的功率**随它多用力而变**，且它用力多少由外部世界决定（今天搬的是 5 公斤还是 20 公斤、地面是平的还是坡）。**所以你不能像设计暖器那样按平均功率散**

热——你必须按最坏情况，而最坏情况是“客户今天想连续深蹲”。

这就是为什么工业界管这件事叫**热设计余量**，而余量是重量与成本。一台想卖 8.5 万的机器，它的余量必然比一台卖 45 万的少。**于是“便宜”与“能连续干活”之间有一条热阻构成的硬边界。** 而这条边界，恰好是宇树在招股书里主动“放弃工业重载与连续作业”这个选择的物理理由 [推]。

一个可检验的说法：关节热路由 R_1（绕组到壳）、R_2（壳内到外壁）、R_3（外壁到环境）串联构成，而密封、无风扇、外形受限、面积按 L^2 缩而发热按 L^3 涨这四项约束共同把 R_3 推到最大（自然对流换热系数约为强制风冷的 1/5–1/10）；以一颗持续出 60 N·m 的膝关节估算，铜损约 36 W、含驱动与摩擦约 50–60 W；把这 55 W 只靠壳面散出去，用这颗膝自己的外形（N7520 φ75 × 20，侧面积 0.0047 m^2）与自然对流 $h = 8$ 得 **1,459 K**，换成第 43 章那颗 IM6014 的 φ65 × 60（0.0123 m^2）也得 **561 K——同一颗关节借了别人的壳，结论会好 2.6 倍，所以外形必须写明是哪一颗**；对着 60 K 的设计预算（55 W × 1.09 K/W）三档全超，连“大壳 + 风扇”那一格也有 73 K；对着 115 K 的绝缘上限，唯一进得来的正是那一格，而它被 IP 要求否掉了；温升 80 °C 时铜阻涨 31%、剩磁降 8.8%、同扭矩下铜损涨 58%，而超过磁钢工作温度上限后退磁不可逆；因此“额定扭矩”不是一个数而是占空比、姿态、环境温度与外壳覆盖的函数，这是厂商不给它的第一层原因。

证据等级：热路串联模型、自然对流与强制风冷系数、NdFeB 温度系数与牌号工作温度、铜阻温度系数，全部是传热与电机设计通识 [行]；所有数值（27 A、36 W、50–60 W、660 W、31%、8.8%、58%、0.0047 / 0.0136 m^2、1,459 / 507 / 898 / 312 / 292 / 101 K、561 / 364 / 345 / 224 / 112 / 73 K、60 K、115 K、22%、205 W、1.6 倍）是 [算]，**输入含本书假设（k_t 类比 M8010、R = 0.05 Ω、减速比 22.5、占空比、躯干外壳 0.35 m^2 与 ΔT 45 K）**，宇树未公布 G1/H2 关节的 L/R/k_t/热阻/降额曲线 [未]；两颗关节的外形（N7520 φ75 × 20 级、IM6014 φ65 × 60 mm [厂]）取自附录 C.2a 与第 43 章那张表，**“级”表示只到量级、官方未给**

完整包络 [未]；两个 ΔT 判据（60 K 设计预算 = 55 W × 1.09 K/W；115 K = F 级 155 °C − 环境 40 °C）都是本章自己的除法，二者的差是 $R_1 + R_2$ 上那段热点温差 [算]；h 随 ΔT 上升（自然对流约 1/4 次幂、辐射按 $T^4 - T_0^4$）故表中大 ΔT 为高估，是传热通识 [行]；“每秒 400 次温度监控”、`temperature int16[2]`、H2 双目 −20~~+60 / −30~~+70 °C [厂]；216 W 与 6.2 W/kg 来自第 20 章（G1，三个一手数相除）[算]；H1/H2 不给瓦数；“厂商不公布额定值的三层原因”与“便宜与连续作业之间有热阻边界”是本书论证 [算][行]，其中招股书口径为 [推]。

可核对清单：

- ☐ 自己算一次那条正反馈：取温升 60 °C，铜损涨多少（提示：$(1/0.934)^2 \times (1+0.393\times0.6) - 1$）
- ☐ 用 ΔT = P·R_th 反推：若允许结温 100 °C、环境 40 °C、损耗 55 W，需要的总热阻是多少（答：≤1.09 K/W），再判断一个 530 g 的密封铝壳能不能做到
- ☐ 在宇树 G1/H1/H2 页面搜“额定 / 持续 / rated / 降额 / derating / 工作温度 / 热”，记录你能找到的每一项（预期：只有相机的温度）
- ☐ 找一个工业伺服的样本（任何品牌），看它的额定扭矩旁边有没有一张降额曲线图——那张图就是人形现在缺的东西

下一章：本章说热决定了“能不能连续干活”。下一章去看另一件事：当这台 70 公斤的机器出问题时，它怎么停下来——以及为什么“停下来”这件事需要设计。

第 37 章

抱闸、急停与断电时序：让一台 70 kg 的机器安全地停下来

“按一个红按钮”不是安全设计

一台 70 公斤、关节峰值扭矩 360 牛米的人形，站在人旁边。现在出事了。

工程师的直觉是“有急停按钮就行”。这一章要说的是：**急停按钮是整套安全设计里最简单的一个零件，真正难的是它按下之后发生的事，以及它按下之前必须已经想好的事**。

但因为整机更重，力量也更大，所以近距离接触时必须保持足够的安全距离。至少两到三米会更安全一些，否则被手臂或者腿部碰到，确实是可能受伤的。

王兴兴 — 王兴兴 GTC 2026 演讲文字整理（“机器人前瞻”整理；页面自陈“在不改变原意的前提下进行了一定程度的增删修改”），腾讯新闻 — 2026-03-17（演讲整理稿（非逐字，页面已声明增删修改））

先算一笔势能：断电那一瞬间，那几百焦耳往哪儿去

一台 70 公斤的人形（H2 官方体重 70 kg、身高 1.82 m [厂]）站在人旁边，膝正弯着，重心离地约一个膝高。现在它断电。

h 从哪来要先说清，因为它决定后面的数能不能信：本书按官方身高与腿长取的髋高 0.66 m [算]（不是官方页面写的数）、身高 1.32 m，比值 0.66 ÷ 1.32 = 0.5；同一比例下 1.82 m 对应髋高 0.91 m，膝高取髋高的

一半 0.46 m，本书取整为 0.5 m。**这是本书假设，宇树没有公布 H2 的关节高度** [未]。于是断电瞬间已经存在、必须在几百毫秒里交出去的重力势能是：

$$E_{势} = m \cdot g \cdot h = 70\ \text{kg} \times 9.80665\ \text{m/s}^2 \times 0.5\ \text{m} = 343\ \text{J}$$

343 焦耳什么概念：往人身上落同样多的能量，等于把一瓶 500 g 的水从 343 ÷ (0.5 × 9.80665) = 70 m 高处松手。而它对电池只是零头——H1 官方电池 0.864 kWh [厂] 合 0.864 × 1000 × 3600 = 3,110,400 J，343 J 占它的 0.011%。

能量可忽略，正是问题的全部。断电危险不在能量那一栏，在功率那一栏：343 J 若在 0.1 秒里交出就是 343 ÷ 0.1 = 3,432 W，按下文抱闸那种几十毫秒量级、取 0.3 秒算也是 343 ÷ 0.3 = 1,144 W；而第 36 章算过，一颗持续出 60 牛米的关节长期要散掉的热是 50–60 W。**这两个数之间差 19 到 69 倍**（1,144 ÷ 60 = 19，3,432 ÷ 50 = 69），安全设计的全部难度就在这一个比例里。

那 343 J 往哪儿去？只有四条路。一，关节被身体拖着反转，电机从负载变成发电机，发的电经下管体二极管灌进母线电容、把它顶高（第 13 章那条路）——断电时电容照样满着，过压照样烧件。二，绕组开路，什么阻尼都没有：**永磁电机的电制动靠楞次定律**，转子转动感应出电动势，而电动势只在回路闭合时才产生电流，电流的磁场才反对转动；断电使三相开关管全关、绕组开路，这条阻尼整条消失，剩下只有齿槽转矩与轴承摩擦，两者厂商都不公布 [未]。三，抱闸夹紧，势能变成摩擦热、结构变形与一次冲击——这是设计者想要的，但前提是夹紧那一刻转速已接近零。四，腿塌，势能变成整机动能，落在地面或人身上。

问题又来了：第一条路为什么在断电时反而更危险？因为“断电”通常不是全部断电。第 35 章那条电压塌陷链说的正是控制电先没、动力母线还活着的中间态——再生电能没有环路去管，只能由电容与泄放通路硬接。**安全的敌人不是“没电”，是“半有电”**。

先看一个具体的失效：抱闸

关节电机断电之后会怎样？两种设计：

A. 没有抱闸：电机失去扭矩，关节变成自由（或被负载拖着反转，取决于减速器能不能自锁）。膝 60 牛米扛着的 70 公斤身体，会**立刻塌下去**。

B. 有抱闸（失电制动）：断电瞬间弹簧压紧摩擦片，关节锁死。机器保持当前姿态。

人形几乎一定选 B——第 15 章那张表里，宇树官方对关节构成的措辞是“电机 + 减速器 + 编码器 + 驱动板”四合一 [厂]，**没有提到制动器**；而本书在 G1 / H1 / H1-2 / H2 / R1 的整机页面里检索不到任何抱闸条目 [未]。

抱闸为什么是“断电才夹紧”

构造说清楚，这五个字就不是玄学。转子轴上一套摩擦盘，旁边一组**碟形弹簧**（弹簧钢压成的锥形垫片，叠起来当储能件）负责压紧，一圈**励磁线圈**负责拉开：通电时线圈产生磁场，把磁盘吸过去、压缩碟簧，摩擦片分离，关节能自由转；断电时磁场消失——**磁场靠的是电流，电流一停安培力立刻没有，这里没有任何惰性可以拖时间**——碟簧回弹，摩擦片夹紧。

那为什么不反过来，通电才夹紧、断电就松开？判据只有一条：**把最常见的失效，换成最不危险的结果**。断电的原因有几十种——电池耗尽、插头松脱、母线过压保护跳闸、MCU 复位（第 35 章那条链）、急停自己动作、软件崩溃被看门狗咬掉；而“线圈完好且一直带电”只是少数情况。反过来设计，等于每一次断电都变成一次自由落体。行业里管这件事叫失效安全（fail-safe：失效时系统落在安全侧，不是落在省事那一侧）[行]。

这钱不是白省的：打开抱闸要线圈持续通电，铜损 I^2R 变热，铜阻还随温度往上涨（+0.393 %/°C [行]），若 23 个关节各配一只（G1 官方 23 DoF [厂]）就是 23 只线圈同时发热——宇树是否这样配、线圈多大，未公布 [未]；而摩擦片磨薄、磁路老化都会让保持力矩下降，表现形式恰好是“断电了却没抱住”——**一种不报错的静默失效，只在你最需要它的那天现形**。

但 B 有一个陷阱，而且是可以算的陷阱。

抱闸是“断电才夹紧”。而**夹紧需要时间**（机械动作，典型几十毫秒 [行]）。在这几十毫秒里，电机已经没有扭矩了，而身体还在往下掉。

这“几十毫秒”值得拆开，因为它不是机械行程。碟簧式抱闸要走的行程就是气隙，典型 0.2–0.5 mm；电枢几十克、碟簧几十牛（这三个数宇树未公布 [未]，本书取 0.3 mm、20 g、60 N 做量级）：

```
t = √(2·s·m ÷ F) = √(2 × 3×10⁻⁴ × 0.02 ÷ 60) = 0.45 ms
     再把摩擦片压实那 30 μm 算进去 ⇒ 0.47 ms
```

机械闭合是亚毫秒的。[算] 那几十毫秒来自两处，而且两处都在电的那一侧：其一，**线圈的 L/R 时间常数**——这正是第 09 章那个 $\tau_e = L/R$ 反过来用：取 L=0.5 H、R=100 Ω ⇒ τ=5 ms，励磁消失后电流要 3τ=15 ms 才降到 5%，而夹紧力升到 90% 也要 $2.3\tau \approx 11.5$ ms；其二，**命令得先到达关节、关节还得执行下一帧**——第 27 章那条总线是 1 kHz 量级，再叠上 `timeout` 之后的兜底模式切换 [厂]。所以“几十毫秒”是**电磁与协议的量级，不是弹簧与铁的量级**：想把它压短，换更硬的弹簧没有用，得改线圈、改驱动、改节拍。

更要紧的是这几毫秒里抱闸夹住的究竟是谁。这里先得纠一处口径，因为它是本书自己反复警告的那类错：动能必须整条算在**同一侧**。官方源码 docstring 给的转子惯量 4.89×10^{-5} kg·m² 是**电机轴那一档**的数 [厂]，而 14.5 rad/s 是**关节那一档**的转速——把这两个数直接乘在一起得到的 0.005 J 是假的。正确做法是折算到同一侧：转子跟着 22.5 的减速比转到 326 rad/s，动能 $\tfrac{1}{2} \times 4.89 \times 10^{-5} \times 326^2 = \mathbf{2.6\ J}$；或者把惯量乘 i^2 折到

关节侧（0.0248 kg·m²）再配 14.5 rad/s，得到**同一个 2.6 J** [算]。能量不选边，选边的是我们。拿官方那个 `armature = 0.025101925 kg·m²` 会更省事——附录 C.2a 已经逐位证过它就是同一批数搬到输出轴 [厂]，配上 14.5 rad/s 得 **2.64 J**，比只搬转子多出的 1.5% 正是那两级齿轮自己的惯量。

即使这样，2.6 J 也只占本章开头那条腿的 343 J 的 **0.76%** [算]——正是下面第二节那个 1/132。所以抱闸的困境不是“可吸收的能量太少”，而是**行程和功率两栏都不够**。

```
行程这一栏：要在一弧度（约 57°）的塌陷行程里把 343 J 磨掉，
摩擦面平均要给出的扭矩
    τ = E ÷ θ = 343 ÷ 1.0 = 343 N·m（关节侧）÷ 22.5 = 15.3 N·m（电机轴侧）
对照这颗膝自己的峰值扭矩 111 N·m   ⇒ 差 3.1 倍
对照第 20 章 M8010 的 23.7 N·m      ⇒ 差 14.5 倍
对照 IM6014 的 34.4 N·m             ⇒ 差 10.0 倍
反过来说：一块按 111 N·m 量级设计的摩擦面要磨完 343 J，
需要 343 ÷ 111 = 3.1 rad ≈ 177° 的行程—腿没有那么多角度可掉，地面先到了。
```

功率这一栏断得更干脆：0.5 m 的自由落体只要 $t = \sqrt{(2h \div g)} = 0.32$ s [算]，重力就在 0.32 秒里往里灌 **1.1 kW**（343 ÷ 0.32）。那 2.6 J 在上一节 1,610 W 的界面功率上只够撑 1.6 ms；此后摩擦面就得独自吞下整条 1.1 kW，而第 36 章算过一颗关节能长期散出去的热是 50–60 W——**差 18–21 倍** [算]。

所以抱闸是“静保持够、动吸收不够”。让一条已经停住的腿别动，只需把 60 牛米的重力扭矩折回电机轴侧：60 ÷ 22.5 = **2.7 N·m** [算]，碟簧确实只要这么点；可要在塌陷途中把机器接住，需要的是它的 5.7 倍，而且是**持续**的 5.7 倍。**这才是“断电即安全”在人形上必须加限定条件的原因**，也是本章后面那条“先软化、再稳定、后断励磁”时序的全部理由：接住那 343 J 的必须是电机与驱动器——有环路、有能量去处——不能是一组碟簧。

```
掉多少？自由落体 30 ms：
h = ½ g t² = 0.5 × 9.80665 × 0.03² = 4.4 mm
```

4.4 毫米看起来无害。但请注意两件事：

一，它不是自由落体，是受控失稳。 一条腿掉 4.4 mm 而另一条没掉，重心就偏了；偏了就触发第 17 章那个“必须迈一步”的判据；而此刻控制器已经断电。**结果是摔。**

顺手把这一条的能量也算一下，免得与前面那 343 J 混起来：70 × 9.80665 × 0.0044 = 3.0 J，只占它的 0.88%。**所以抱闸前那几十毫秒的危险来源是姿态偏差，不是能量。** 两笔账分开记。

二，抱闸的冲击载荷。 一台 70 公斤的机器在 30 毫秒内被一个摩擦片突然锁住，摩擦片要吸收的动能与冲击扭矩，可能远超它的额定。**所以真实设计必须是：先由电机主动把扭矩卸到接近零（软件控制，几十毫秒），再断抱闸。**

那为什么不能在高速下把抱闸直接抱死

你可能会想：既然断电夹紧是安全的，那就让它抱得越快越好，最好在塌陷刚开始的几毫秒里锁死。这个直觉不成立，而不成立在功率那一栏。

第一条式子是摩擦副的瞬时功率——摩擦片不是“拦住”了机械功率，而是把它就地变成热：

$$P = \tau \cdot \omega$$

τ 是摩擦面传递的扭矩（N·m），ω 是抱紧瞬间两面的相对角速度（rad/s）。代一次官方数：G1 的膝用 N7520-22.5 这一档，训练仓库给的同向峰值扭矩 Y1 = 111 N·m、满扭矩最高转速 X1 = 14.5 rad/s [厂]；假设在这个转速上让一块按峰值扭矩量级设计的摩擦片去抱（宇树未公布抱闸力矩，本书拿该关节自己的峰值扭矩当量级 [未]），界面功率就是

$$P = 111\ \text{N·m} \times 14.5\ \text{rad/s} = 1{,}610\ \text{W}$$

而第 36 章算过，一颗持续出 60 牛米的关节长期要散的热是 50–60 W。 1,610 ÷ 60 = 26.8——抱紧那一瞬间界面上的发热功率，约是这颗关节能长期散出去的热量的 27 倍。这个结论不靠精密输入，它就是量级本身。

第二条式子是被抱住的动能：

$E_k = \tfrac{1}{2} \cdot J \cdot \omega^2$

J 是被抱闸拦住那部分转动惯量（kg·m²），ω 是抱紧瞬间的角速度。**正着用**只算转子，就是第一节那条不变量：官方源码 docstring 给出 N7520 家族的 rotor = 0.489 × 10⁻⁴ kg·m²（电机轴侧）[厂]，型号名里的 22.5 就是减速比（第 14 章已用官方两档扭矩与转速自证过这条换算），于是 14.5 rad/s 的关节转速对应转子 14.5 × 22.5 = 326 rad/s，转子动能 2.6 J。**反着用才是重点**：把第一节那 343 J 按同一个 ω 折回等效惯量，$J = 2E \div \omega^2 = 2 \times 343 \div 14.5^2 = 3.27$ kg·m²，**它是转子那一档的 3.27 ÷ (4.89×10⁻⁵ × 22.5² = 0.0248) ≈ 132 倍**。断电塌陷时抱闸要拦的不是电机里的转子，而是从关节往外扩开的整台机器。（分母若换成官方那个 `armature = 0.025101925 kg·m²`——附录 C.2a 已逐位证过它是同一批数搬到**输出轴** [厂]——倍数是 3.27 ÷ 0.0251 = **130** 而不是 132；差的 1.5% 就是那两级齿轮自己的惯量，结论不变。）

两条式子合起来给三条结论，都是行业通行做法 [行]，不是宇树公布的设计。一，**磨损功随 ω² 走，不随 ω 走**：动能是 ½·J·ω²，转速翻倍要多耗四倍的功，而抱紧所需时间 t = J·ω ÷ τ 又随 ω 线性增长，磨损量大致跟着滑磨功走——第 14 章那句“齿隙是最容易随磨损变大的环节”，在这里多了一个来源。二，**热不会老实待在界面**：滑磨热沿轴往下走的第一站就是磁钢，第 36 章那条链重新出现——NdFeB 剩磁约 −0.11 %/°C [行]，代 100 °C 得 1 − 0.0011 × 100 = 0.89，同电流下扭矩少 11%，而越过不可逆退磁的膝点之后这种下降是永久的；**抱闸抱死一次，可能永久改掉这颗关节的扭矩常数**，而扭矩常数是第 04 章 τ = k_t·i 的全部依据。三，**抱闸是保持件，不是制动件**：真正把机器停下来的是电机与驱动器（有环路、有能量去处——回收进母线或泄放进制动电阻，第 13 章），抱闸能做的是在“已经停了”之后让它别动，所以 ② 必须排在 ④ 前面。

这条链在什么时候断：**如果驱动器已经没电，就没有任何东西能把 ω 压下来**。失电保护不能指望抱闸替你刹车，只能指望不要在这个姿态、这个高度上失电——这正是下一节比本节更重要的原因。

于是“安全停机”这件事有了一条**时序**：

```
① 检测到触发（急停按下 / 通信超时 / 过温 / 过流）
② 命令所有关节 tau → 0，并把 kp/kd 设成让关节“软下来”的值
   （第 12 章的五元组在这里成了安全接口）
③ 等机械状态稳定（几十到几百毫秒）
④ 断抱闸励磁 → 弹簧夹紧 → 关节锁死
⑤ 断动力母线（保留控制电，让状态能被读到）
⑥ 声光提示 + 记录故障码
```

注意第 ⑤ 步在 ④ 之后，而第 ② 步在最前。这三步的顺序错了，机器就会在停下来的过程中伤人或自毁。

而第 ② 步有一个非常危险的副作用：**它要求 kp/kd 可被外部改写**。第 12 章说过，`rt/lowcmd` 里 kp 叫 Stiffness、kd 叫 Damping，而 tau 可以直接命令扭矩，**这套接口没有权限层级** [厂]。也就是说，同一条能实现安全软化的通道，也能让一段错误的代码把一台人形的所有关节刚度设成零。

这就是“接口即风险”的具体形态。宇树官方文档里对这一点没有安全说明 [未]。

急停为什么要双通道

一个按钮、一根线，够了吗？不够。因为**故障有两种**：机器动了（不该动），和机器没动（该动的时候没动）。第二种更危险，因为它看起来一切正常。

所以工业安全的设计原则是“**失效也要能被发现**”（fail-safe / fail-operational），实现方式是双通道：

```
通道 A：主控软件监测 → 命令停机
通道 B：硬件安全回路（急停按钮串在安全继电器上）→ 直接切动力
两条通道互相独立，且互相校验：A 说“我停了”，B 说“电还在”，就报错
```

关键概念是“共因失效”：如果 A 和 B 用同一块板、同一个电源、同一根线，那么一次板级故障会同时打掉两条通道——这时“双通道”是假的。

而双通道防的到底是什么，一句话：单点故障不导致危险。一根线断、一个触点熔焊、一个软件任务死掉，都只是“一个”故障；只要这一个故障不直接让人受伤，设计就成立。所以它必须同时做两件事，而第二件经常被省掉：① 彼此独立——不同板、不同电源、不同走线、不同执行路径；② 彼此校验——A 报“我已命令停机”、B 报“动力仍在”，两者矛盾就锁机。**只有 ① 的双通道是把两个故障叠起来而已**：通道 B 的继电器触点熔焊（该断不断）在任何一次正常运行里都不露头，只在你真正按下急停那天现形；校验回路把这类沉默故障从“那一天”提前到“这一次上电”。

那校验回路自己坏了怎么办？工业安全电路的答复是**定期强制导通测试**（force test：按周期把通道驱动一次，验证它确实能动作）[行]，而它的前提是执行器可以双向驱动。关节抱闸只有一个动作方向：通电开、断电闭——**你无法在不松开抱闸的前提下去验证它抱得住**。这就是人形在这一格上比工业臂难的地方：它的保持件不接受在线自检。本书在宇树公开资料里没查到任何关于抱闸自检、急停回路结构或安全等级的表述 [未]。

这一节的边界：本书不给失效率数字。PL d / Cat. 3、SIL 2 那一套等级靠“危险未检出失效概率”一类数据撑着，而人形的关节与整机没有任何公开的失效率数据 [未]。**没有数据就不写数**。

（相关标准：**ISO 10218**（工业机器人机器人与单元的安全要求）、**ISO/TS 15062**（协作机器人的技术规范，含力与功率限制）、**IEC 62443**（工业网络安全）。这些是行业要求 [行]，**不是宇树已获认证的陈述**——本书在 G1/H1/H2/R1 的官方页面里检索，CE / CR / ISO 10218 / ISO 13482 / FCC / UL / CCC 命中数为 0 [未]（这是「未发布」，不等于「纸质材料里不存在」）。**同样没有公开的还有防护等级：整个人形栈的官方页面里，本书没有找到任何一款人形（G1 / H1 / H1-2 / H2 / R1）的 IP 等级标注** [未]——而 IP 那一栏恰恰决定它能不能进有水、有粉尘、要冲洗的车间，它与第 23 章的 100 Klux 边界是同一种空白。）

断电之后这条腿会不会塌：自锁性是一条能算的判据

前面所有讨论都默认“断电之后有东西在拦着”。如果既没有抱闸、电机又已经没电，这条腿会不会自己塌？这个问题可以算，而答案不在“力气大小”里，在减速器的啮合几何里。

同一对齿，正行程（电机推负载）与反行程（负载反推电机）的效率不是同一个数：

$\eta_{正} = \tan\lambda \div \tan(\lambda + \rho)$

$\eta_{反} = \tan(\lambda - \rho) \div \tan\lambda$

λ 是导程角（把齿面螺旋线展开后，斜面与垂直于轴线方向的夹角），ρ 是摩擦角，定义 $\tan\rho = \mu$，μ 是当量摩擦系数；两式都由斜面上的滑块推出，任何一本机械原理教材都有 [行]。关键在第二式：**$\lambda \leqslant \rho$ 时 $\tan(\lambda - \rho) \leqslant 0$，于是 $\eta_{反} \leqslant 0$，负载推不动电机——这就叫自锁**。把临界点 $\lambda = \rho$ 代回第一式得 $\eta_{正} = \tan\rho \div \tan 2\rho = (1 - \tan^2\rho) \div 2 < 1/2$，所以工程上更好用的口诀是：**正行程效率低于一半的传动副，反行程自锁** [行]。

拿它查这台机器能见到的两类传动。**行星减速器**：第 20 章那条效率链里齿轮那一档取 0.85 [行]（整链 $0.9 \times 0.8 \times 0.85 \approx 0.61$），远高于一半，**不自锁**；拆解口径说 G1 用两级行星、总约 20.58:1 [拆]，第 14 章说 G1 不用谐波减速器——这是同一件事的两面：这类传动可以反驱，所以准直驱做得出力控（第 14 章“电流能代表输出力矩”那条前提），所以断电后它会塌。**谐波减速器**：靠柔轮弹性变形传动，本身不按自锁设计；Z1 官方页只给了减速比 60+ 与背隙 6 弧分 [厂]，没有任何自锁性表述 [未]。

所以“断电后这条腿会不会塌”的答复是：会。抱闸因此不是可选件，而是行星/准直驱路线的必配项——请注意这条推论的力量来自判据，不来自厂商文档。

顺带纠正一个在工程师圈子里流传很广的说法：**“蜗杆传动一定自锁”是错的**。自锁条件是 $\tan\lambda \leqslant \mu$，而 $\tan\lambda = \text{导程} \div (\pi \cdot d_1)$，$d_1$ 是蜗杆分度圆直径。单头蜗杆的 λ 确实容易做小，但只要 μ 因为油膜建立、温升、磨屑而下降——润滑钢对钢的当量摩擦系数本来只有百分之几的量级、且随工况大幅浮动 [行]，本书不取具体数，因为取哪一个都会假装它是常数——原本自锁的副就失去自锁。**把安全建立在摩擦系数上，就是把它建立在工况上**。这就是行业里的安全保持件一律用弹簧压紧的摩擦副、而从不把减速器自锁当保持手段的原因 [行]。

这条链又在哪里断：真实塌陷里你能读到的第一个信号不是“腿弯了”，而是“编码器读数与指令的偏差不再收敛”——第 14 章的双编码器差值、第 12 章的 kp/kd 残差就是这件事的两个入口。**如果你没有任何一路能读到“位置在漂”，你也就没有塌陷预警**。

人形独有的两条：没有围栏，而且会倒向人

工业机器人安全的前提是**围栏**。人形被推进人类空间，前提消失了。于是两条新的硬问题：

一、力与压强限制。ISO/TS 15062 给出了协作机器人 28 个身体部位的接触力与压强限值 [行]（例如胸部约 150 N、手掌约 150 N、头部约 65 N 这一量级；**这三个数是二手转述，本书没有读到 15062 原文——它需购买，见第 44 章**，逐部位的精确限值请以原文为准）。现在拿第 18 章的算术对照：

```
一台 H1 的膝关节峰值 360 N·m（官方页 [厂]）
若小腿 0.4 m，末端力 = 360 ÷ 0.4 = 900 N
```

900 牛远超任何一条接触限值。而它只在“峰值”时出现——额定值厂商不给（第 36 章），所以**你无法从公开资料判断它在正常动作中会不会超限**。

这就是为什么“人形与人共处”这件事，卡点不是算法礼貌，是**一个厂商没有公布、而标准强制要求的数**。

二、它会倒。 工业机器人不会倒。人形会，而第 17 章说过：**一旦偏出捕获点，它物理上无法不摔**（腿这个执行器天生 200 毫秒）。

所以人形的安全设计必须包含一条工业机器人没有的假设：**它会不受控制地倒下，而且倒下的方向可能是人**。应对手段只有三类：

1. **限制倾倒能量**（降低重心、限制动作速度、在有人时降功率模式）；
2. **提前预警**（声光、以及“我下一步要往后退”的表达）；
3. **结构上让它砸不伤人**（软外壳、圆角、限重）。

而这三类在宇树公开资料里都查不到 [未]。本书能核实到的只有三条，而这三条已经足够说明问题：

```
保修期（官方页面表格行，已发布 [厂]）
   G1 8 个月 / G1 EDU 18 个月
   H2 8 个月 / H2 EDU 12 个月
   R1 AIR 6 / R1 8 / R1 EDU 12 个月
   H1 与 H1-2 页面：没有 Warranty Period 这一行
   IM6014 电机：3 个月（产品页）

免责原文（H1 与 H1-2 文档 About 页 [厂]）
   "Hot swapping may cause equipment failure,
     and such failures are not covered under warranty."
   —— 官方明确警告热插拔会坏，且这种坏不保。

整个「人形 + 配件」栈里唯一一处官方认证表述 [厂]
   L1 / L2 激光雷达：Class 1 (IEC60825-1:2014) 人眼安全
   —— 是激光安全，不是机械安全，也不是电气安全。
```

把这三条放在一起读：一家公司愿意逐型号公布保修月数（说明它有售后体系），在文档里专门警告热插拔（说明它见过这类事故），而它的人形本体**没有任何一条机械或电气安全认证表述**，同时 H1 连保修行都没有。

那句免责原文为什么值得读？因为官方只给警告、没给机制，而机制本章已经备齐：带电拔插时，触点面积在极短时间内收缩、电压把间隙击穿成电弧，触点是牺牲件（电弧能量与母线电容容值全系未公布 [未]）；控制线与动力线不会同时断——控制先断则关节一直保持最后一条指令，动力先断就回到本章第一节的塌陷。更要紧的是，**热插拔的失效模式与“断电”**

的失效模式是同一个：一个关节掉线，其余关节还在按全身算法出力。宇树协议里的 `timeout` 位（没收到新指令就自动停）、ID 15 广播（不要求回应，一条帧打到所有关节）、`mode 0` = “停止并锁扭矩”[厂]，正好是软件级急停的三块积木；但它们是宇树公开的数字舵机协议文档里的器件层口径，**人形关节固件是否实现同一套语义、超时多少毫秒，官方没有公布**[未]——“协议里有这个位”不等于“整机断电时会走这个分支”，本书不把这两级证据合并。

这不是隐瞒，这是进度。一个还在改硬件、还在被热插拔烧掉的阶段，不会有认证。而第 36 章末尾那条判断在这里第二次成立：**这个行业缺的不是能力，是把能力写成可审计文档的习惯**。

上电那一侧同样有时序

安全不只是“怎么停”，还有“怎么开始”。

> 上电默认态应该是什么？
>
> 选项 A：所有关节使能、kp 用上次保存的值 → 机器可能带着错误标定立刻出力
> 选项 B：所有关节零扭矩（tau=0、kp=kd=0）→ 机器瘫软，膝 60 牛米扛着的身体直接塌
> 选项 C：先读编码器与温度 → 校验零点与热状态 → 抱闸保持 → 再逐步使能

A 危险，B 也危险（它会坐下去），C 正确但需要抱闸与状态校验的基础设施。

第 10 章那条“编码器零点装偏 5° 能通过一切出厂测试”在这里第二次出现：**上电自检要能发现零偏，就必须有一个“上次标定值”与“本次读数”的比较**。而这需要多圈绝对值或断电位置保持——第 14 章说过，宇树 M8010 是**单圈**绝对值 [厂]，整机页只说“双编码器”，**单圈/多圈未公布** [未]。

所以“这台机器断电后被搬动过，再上电时它知不知道”这个问题，从公开资料无法回答。而这是一个直接的安全问题：一台被搬动过、上电后以为自己在初始姿态的 70 公斤机器，会在第一步就迈错方向。

再把抱闸的构造叠回来，上电顺序里就多一条硬约束：既然抱闸靠通电才打开，“给线圈通电”必须排在“确认姿态、安全条件与零位校验”之后。它在纸面上只是顺序，在车间里是另一句话：**调试时不能靠拔线来停机器**——拔线的结果是抱闸夹紧，而此刻关节可能正转到一半，于是本节上面那两条式子当场兑现。

一张“该问厂商什么”的清单

本章对读者最有用的产出是这张表。任何一次人形采购，这七个问题必须有书面回答：

问题	为什么必须问	公开资料能否回答
急停是单通道还是双通道？走的是硬件安全回路还是软件？	决定共因失效风险	❌ [未]
断电时序是什么？抱闸在断动力之前还是之后？卸扭矩要多久？	决定“停下来”的过程中会不会摔	❌ [未]
上电默认态是什么？如何检测“断电期间被搬动过”？	决定第一步会不会迈错	❌ [未]
在有人模式下，关节力矩与整机功率被限制到多少？依据哪条标准条款？	决定它砸到人时是多少牛	❌ [未]
抱闸是弹簧闭合、通电打开，还是反过来？打开它要持续多少瓦？	决定上电顺序与一整天的铜损发热	❌ [未]
断电时传动链的反行程效率是多少，也就是自不自锁？	决定“断电”与“塌”之间到底有没有关系	❌ [未]
整机 IP 防护等级是多少？按哪份标准测的？	决定它能不能进有水、有粉尘、要冲洗的车间	❌ [未]

七个问题，本书一个都没能从公开资料里答出来。我把这张表印出来，是因为**“问不出来的问题”比“答不上来的答案”更有价值**——它定义了你在签合同前必须拿到的东西。

类比，以及它在哪儿破

类比：人形的安全像汽车的安全——都是“会倒向人的重物”，都需要急停（刹车）、失效保护（ABS）、和碰撞限制。

这个类比在哪里破，破得正是关键差异：汽车有 130 年的法规积累、有强制碰撞测试、有 EDR 黑匣子、有召回制度。而人形机器人**目前没有任何一条强制标准适用于“家用/工业用双足人形”这个品类** [行]。ISO 10218 是给围栏里的工业臂写的，ISO/TS 15062 是给协作臂写的，人形两边都不完全落。

所以这个行业的安全，目前完全靠厂商的自律与客户的追问。而第 36 章末尾说过：“额定扭矩不公布”的第三层原因是制度没有强制它。这一章给出同一个答案的另一半：**安全时序不公布，也是因为没有人要求它公布**。

这就是为什么本书认为，人形行业未来五年最重要的一批工作，不是模型，是**标准与检测**——而它恰好是全行业投入最少、且中国厂商目前几乎缺席的一块 [行]。

一个可检验的说法：一台双足人形的安全停机必须是一条有时序的动作链（先命令 tau→0 并用 kp/kd 软化、等机械稳定、再断抱闸励磁、最后断动力且保留控制电），因为抱闸夹紧要几十毫秒而其间电机已无扭矩；这条时序的物理依据是三式：$E_{势} = m \cdot g \cdot h = 70 \times 9.80665 \times 0.5 = 343$ J，它对 H1 那 0.864 kWh 只占 0.011%，但按 0.1–0.3 s 交出就是 1.1–3.4 kW，是第 36 章一颗关节能长期散掉的热（50–60 W）的 20–60 倍；抱闸不能在高速时抱死，因为 $P = \tau \cdot \omega$ 按官方 111 N·m 与 14.5 rad/s 算得 1,610 W，而 $E_k = \tfrac{1}{2} \cdot J \cdot \omega^2$ 反解出的等效惯量 3.27 kg·m²，折到转子同侧是 0.0248 kg·m²（$= 4.89 \times 10^{-5} \times 22.5^2$）的 **132 倍**；要在 1 rad 的塌陷行程里磨掉 343 J，摩擦面得在关节侧持续给出 343 N·m，是这颗膝自己峰值扭矩 111 N·m 的 **3.1 倍**——而“已经停住之后别动”只需要电机轴侧 60 ÷ 22.5 = 2.7 N·m——**抱闸静保持够、动吸收不够，所以它是保持件不**

是制动件；断电后这条腿塌不塌不取决于厂商心情，取决于自锁判据 tan λ ≤ μ（等价于正行程效率低于 1/2），而第 20 章给齿轮那一档取的 0.85 远在其上，故行星/准直驱必塌、抱闸是必配项；实现软化所依赖的那组 kp/kd/tau 字段是一条无权限层级的总线接口，同一通道既能实现安全也能造成失效；H1 膝关节 360 N·m 折算到小腿末端约 900 N，远超 ISO/TS 15062 的接触力限值量级，但厂商不公布额定值，无法判断正常动作是否超限；急停结构、断电时序、上电默认态、有人模式限力、抱闸构造与功耗、传动自锁性、整机 IP 等级，七项在宇树公开资料中均无记载。

证据等级：fail-safe / 双通道 / 共因失效 / 强制导通测试、抱闸动作时间与冲击、自锁判据 tan λ ≤ μ 与 η_正 < 1/2、蜗杆并非必然自锁、ISO 10218 / TS 15062 / IEC 62443 的内容与限值量级，是工业安全与机械原理通识 [行]；360 N·m、四合一构成、`MotorCmd` 字段与命名、`timeout` / ID 15 广播 / `mode 0` 三个协议字段（取自宇树公开的数字舵机协议文档，只引用参数）、M8010 单圈绝对值、G1 的 23 DoF、Y1 111 N·m 与 X1 14.5 rad/s、N7520 的 rotor = 0.489 × 10^{-4} kg·m^2、H1 电池 0.864 kWh、Z1 的 60+ 与 6 弧分、H1/H1-2 热插拔免责原文、L1/L2 的 Class 1 (IEC60825-1:2014) [厂]，两级行星 20.58 [拆]；343 J、70 m、0.011%、3,432 W、1,144 W、3.0 J、1,610 W、26.8 倍、326 rad/s、2.6 J、2.64 J、0.0248 kg·m^2、3.27 kg·m^2、132 倍、130 倍、0.76%、0.32 s、1.1 kW、18–21 倍、343 N·m（= E ÷ θ）、3.1 倍、15.3 N·m、2.7 N·m、5.7 倍、0.89、4.4 mm、900 N 是 [算]（h = 0.5 m、小腿 0.4 m、塌陷行程 θ = 1 rad、抱闸力矩取该关节峰值扭矩量级，四处均为本书假设 [未]）；**急停回路结构、断电时序、上电默认态、有人模式限力、抱闸是否存在与其功耗、传动自锁性、整机 IP 等级、人形关节是否实现 `timeout` 与广播语义、关节与整机的失效率数据，全部未公布** [未]，保修 6–18 个月为官方页面表格行 [厂]；“标准缺席”“抱闸是保持件”是行业观察 [行]。

可核对清单：

- ☐ 自己算一次 ½ × 9.80665 × 0.03^2，确认 4.4 mm

- ☐ 自己算一次 $70 \times 9.80665 \times 0.5$，确认 343 J；再分别除以 0.1 与 0.3，看功率落在 3.4 kW 与 1.1 kW
- ☐ 自己算一遍抱闸为什么接不住这一跤：343 J ÷ 1 rad = 343 N·m（关节侧），除 22.5 = 15.3 N·m（电机轴侧）；再拿 60 ÷ 22.5 = 2.7 N·m 比一比，就知道“静保持够、动吸收不够”是 5.7 倍的差距
- ☐ 用 $J = 2 \times 343 \div 14.5^2$ 复算 3.27 kg·m²，**再把它和转子惯量折算到同一侧**：$4.89 \times 10^{-5} \times 22.5^2 = 0.0248 \Rightarrow 3.27 \div 0.0248 = 132$ 倍。直接除 4.89×10^{-5} 得 66,900 是错的——那跨了 22.5:1 的减速比，凭空多乘 $i^2 = 506$ 倍（附录 A 的 J 行就为这一格写的）
- ☐ 把 $\eta_{正} = \tan\lambda \div \tan(\lambda + \rho)$ 推一遍，验证 $\lambda = \rho$ 时它等于 $(1 - \tan^2\rho)/2$，也就是小于一半
- ☐ 查一次 ISO/TS 15062 的接触力限值表，然后拿 360 N·m ÷ 0.4 m 比一比
- ☐ 拿一份工业机器人安全手册（任何品牌），看它的安全停机类别（SS1/SS2/SOS 一类）怎么写——那就是人形现在缺的那份文档的样子
- ☐ 把上面那张七问清单真的拿去问一次销售，记录能拿到几个书面回答

下一章：本章说这台机器在安全上还没有制度。下一章去看另一件“演示与现场”之间的事：为什么一个 95% 成功率的策略，在真实流水线上撑不过一小时。

第 38 章

泛化与长时程：演示很惊艳，连续作业很难

一个二位的乘法，毁掉大部分机器人宣传

行业宣传里最常见的句子是“某任务成功率 95%”。它通常是真的，而且是**误导性的**。

因为工厂要的不是“一次成功”，是“连续成功”。而连续成功的概率是乘法：

```
P(连续 N 次成功) = p^N
```

代几个数（`foc_check.py compound_success()`）：

单次 p	N=10	N=50	N=100	N=500
99%	90.4%	60.5%	36.6%	0.66%
95%	59.9%	7.7%	**0.6%**	≈0（7.3×10^{-12}）
90%	34.9%	0.5%	0.0027%	≈0（1.3×10^{-23}）

（[算]。）

读第二行：一个 95% 成功率的抓取任务，连续做 100 次全对的概率是 0.6%。

而一条流水线一小时可能要做 60–120 次动作。**所以“95% 成功率”翻译到工厂语言里，意思是“每小时必然出错一次以上”。**

这就是本章的全部核心：单次成功率与连续作业能力之间隔着一个指数，而指数不讲情面。

我们希望人形机器人本体的各方面功能更加完善后再考虑商业化落地的问题。对于商业落地的方向规划，我们不一定会 all in 工业，也可能会推进在教育、科研等领域的商业化进程，我们对人形机器人落地方向持更加开放的态度。

王兴兴 — 《中国电子报》“人形机器人高质量发展”系列访谈（腾讯新闻转载） — 采访 2024-08-20；转载 2025-02-18（直接引语（记者以“王兴兴回应表示”引出））

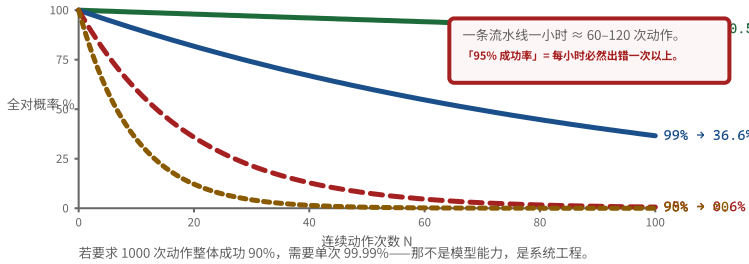


图 38-1　连续成功的指数律：95% 的单次成功率，100 次之后只剩 0.6%

为什么是乘法，不是加法

你可能会想：一百步里每步 99%，平均一下不就是 99% 吗？错在两个前提上，而这两个前提值得各自追问一层。

第一，**要的是“全部成功”，不是“平均成功”**。一条流水线的产物是“这一件有没有做对”，第 k 步失败，这一件就废了。所以要求的是联合概率，而联合概率在独立事件上就是乘积：

$P(\text{连续 N 步全对}) = p_1 \times p_2 \times \ldots \times p_N = p\text{^}N$

第二，**每一步的失败相互独立、且不可恢复**。独立性意味着前面的成功不会让后面更容易（策略不会因为“已经做对三步”而更会做第四步）；不可恢复意味着一次失败就把整件产品判死，没有回头路。**这两条里任何一条被松开，指数律就要改写**——这正是后面“闭环检测”那一节要做的事。

这条判据在什么时候说谎：真实的失败往往不是独立的。同一个坏摆放、同一种反光，会让第 37、38、39 步一起失败。失败彼此正相关时，衰减比 p^N 更快（有效单步成功率更低）；反过来，如果任务链里的步骤用的是互相独立的物体，实际表现会好于 p^N。**本书仍然按 p^N 记账，因为它是最诚实的那一档：它假设每一步都不给别人帮忙**。

把“8 小时”换算成步数

演示与现场之间不是质量问题，是 N 的问题。把时长换成步数，一切都清楚了：

8 h = 8 × 3600 s = 28,800 s；一个抓取—放置动作约 1 s（**本书假设**：各家从未公布原子动作时长，公开只有节拍与整单耗时，而本章前面那条药房口径“单均 68 秒”里就含若干个动作 [未]）⇒ N ≈ 28,800 步

拿它代进同一条式子：

$0.99^{28800} = 2 \times 10^{-12_6}$

这是一个 10^{12_6} 量级的分母（2×10^{-12_6} 的倒数是 4.9×10^{12_5}）。而演示里的 30 秒只有 N = 30，$0.99^{30} = 74\%$——同一条式子、同一个 p，只是 N 差三个数量级。**“演示很惊艳、现场不行”里属于数学的那一半，就是这一行**。

这里要主动对一条账。本章前面写过“一条流水线一小时可能要做 60–120 次动作”，那是按**件**计的口径（60 件/小时 = 一件 60 s，与本章后面药房那条“单均 68 秒”同一量级）；而这里按**原子动作**每步约 1 s 取 N =

28,800。两个口径都保留，不取平均，因为它们会给出不同的 N：若一件里有 10 个原子动作，60–120 件/小时的八小时班次就是 4,800–9,600 步，于是

$0.99^{4800} = 1.1 \times 10^{-21}$，$0.99^{9600} = 1.3 \times 10^{-42}$，$0.99^{28800} = 2 \times 10^{-126}$

三条都是零。 N 到底取 4,800 还是 28,800 不重要——在这个指数上，任何一档都足以判死“靠单次成功率撑连续作业”这条路线。

反过来问才是工程的问法：要 8 小时有 90% 的整体成功，单步得有多准？

$p = 0.9^{(1/28800)} = 99.99963\% \Rightarrow$ 单步失败率 $q = 1 - p = 3.7 \times 10^{-6}$

也就是**平均每 27 万步才允许错一次**（$1 \div 3.7 \times 10^{-6} \approx 27$ 万）。这个数字本身就是本章的判据：**任何声称能干 8 小时的人形，它报的单步成功率要么高到可疑，要么它靠的不是“单步够准”而是下一节那两件事。**

于是“该报什么指标”这件事被重新定义

一个诚实的人形作业指标应该长这样：

```
✗  任务成功率 95%
✓  在 X 件/小时节拍、Y 小时连续、无人干预条件下，
   完成 Z 件，失败 M 次，失败后自主恢复 R 次，
   平均恢复时间 T 秒，MTBF（平均无故障间隔）= ?
```

行业里目前几乎没人报 MTBF。 而它是工厂唯一关心的数——因为一次停机的成本，通常远大于一次抓取失败的成本。

为什么没人报？ 三个原因，都很硬：

1. **报它需要长时间运行**，而长时间运行需要可靠性工程（第 36、37 章），不是算法问题；
2. **失败的定义模糊**——抓歪了但捡起来了算成功吗？卡住了 30 秒算失败吗？没有统一口径就没法比较；

3. **它会把设计余量暴露出来**（第 36 章同一逻辑）。

所以本章给读者的判据是：看一个人形作业宣传，先找有没有“连续时长”这个轴。 有 8 小时、有件数、有失败后恢复的，才是可比的。

可查到的少数几个诚实数字

把公开口径里**带时间轴**的数挑出来（全部 [查][推]，多为厂商自陈）：

> 银河通用 药房场景：日均 400–500 单，单均 68 秒，拣货准确率 100%
> 银河通用 S1 在宁德时代某基地：7×24 无遥操连续作业，从 3 个月延长到 6 个月
> （此处原有“某厂商 30 户家庭零样本 56%”一行，**2026-10-09 复核：新闻索引查不到该条目、详情页 404，读不到原文，故删**—保留删除记录是为了说明“宣传级成功率”该怎么被处理）
> （此处原有“某厂商 拉袋 44% / 拉链 40%”一行，**同为读不到原文的媒体转述，已删**；下面的算术改用假设的整段成功率，结论不依赖任何厂商口径）
> Physical Intelligence π*0.6：连续 13 小时制作咖啡，需 RECAP 把失败率约减半才做到

这几行值得逐个读，因为它们各自暴露一类信息：

“日均 400–500 单 + 单均 68 秒” 是可以乘的：400 × 68 s = 27,200 秒 ≈ **7.6 小时纯作业时间** [算]。所以这条宣传实际上说的是“一天工作约 8 小时”——**这是本书在全行业里找到的、极少数自带时间轴的数**。而“拣货准确率 100%”这一项要小心：它可能是“最终交付正确率”（含人工兜底），不是“单次动作成功率”。**口径不同，结论差一个数量级**。

“3 个月延长到 6 个月” 报的不是成功率，是**无干预时长**。这是最接近 MTBF 的公开表述，而且它承认了三个月这个基线——**这条比任何“95%”都诚实**。

任务级成功率报的是整段做对的概率，而任务本身是多步的，所以它天然隐含复合衰减。举个不指名任何厂商的算术例子：**若一个任务要 5 步、整段成功率 44%，那么平均每步做对的概率是 44%^(1/5) = 85%** [算]——

看起来“只有 44%”的一个策略，单步水平其实相当高。**这是本章给的一个反向用法：从任务级成功率反推单次成功率，来判断这个系统的真实水平**。

“需要把失败率约减半，才做到连续 13 小时” 是全书最重要的一条公开证据：**它说明连续作业不是“成功率够高就行”，而是要专门做在线改进（RECAP 一类带奖励修正的后训练）**。 13 小时 ≈ 数千次动作，按 p^N 反推：若 13 小时要 1,000 次动作且成功率 90%，需要 p = 0.9^(1/1000) = 99.99% [算]。**四位数的单次成功率——这不是模型能力，是系统工程**。

这一节必须画一条边界线。 上面这些数全部来自厂商自陈或媒体报道 [查] [推]，口径互不相同，本书未复现任何一项；而它们的共同结构是：**公开的可比的数，几乎都是短任务成功率（N 由任务定义、通常几步到几十步）与自宣时长（没有失败定义、没有恢复口径）**。 银河通用 S1 在宁德时代某基地的“7×24 无遥操从 3 个月延长到 6 个月”是媒体报道级 [推]——它是全行业最接近 MTBF 的公开表述，但既没有第三方核时，也没有说明“干预”的定义。**本章教的算法（p^N）可以用来读这些数，不能用来替它们背书**。

三条让长时程失败的原因，只有一条是算法

现在把“撑不过一小时”拆开。三条原因，性质完全不同：

一、**分布外（算法）**。 遇到没见过的物体、光照、摆放。这是研究社区在攻的那一条（VLA、世界模型、数据规模化，第 29–34 章）。

二、**无记忆（架构）**。 第 31 章说过：动作分块架构对“块之后世界怎么样”没有表征。第 29 章说过：模仿学习不会从自己的失败里学。两条合起来意味着**策略不知道自己已经失败了**。

一个具体后果：机器人抓一个抽屉，第一次没抓住（抽屉没开），它的策略里没有“我刚才失败过一次，这次该换个方式”这个状态。于是第二次用同样的力、同样的位置，再失败一次。**这就是“演示里很聪明、现场很固执”的架构原因**。

三、**硬件漂移（物理）**。这一条几乎不出现在算法论文里，而它在真实部署里最先咬人。

```
皮带/腱绳张紧随使用变化          →  同样的指令，末端位置不同（第 25 章：0.027 mm 只是弹性部分）
润滑脂随温度与时间变化           →  摩擦系数变，tau_est 的偏差变大（第 14 章）
磁钢缓慢退磁                     →  k_t 永久变小（第 36 章的不可逆出口）
电池内阻随循环增长               →  同样的动作，母线塌陷更深（第 35 章）
编码器零点随装配应力漂移         →  Park 变换的 θ 系统性偏（第 10 章那张表）
脚底橡胶磨损                     →  μ 变（第 25 章那道不等式的分母）
```

六条，全部是本书前面章节已经证明过的机制。它们的共同点是：**训练时的物理与部署三个月后的物理不是同一套**。

而第 21 章的域随机化能覆盖一部分（如果你随机化了摩擦与 k_t），但**随机化覆盖不了单调漂移**——随机化假设参数在分布内反复抽样，而磨损是单向的。**这是域随机化的一个结构性盲区，而它很少被指出**。

所以“硬件漂移”的正解不是模型，是标定与监测：定期重标定零点、监测 tau_est 与温度趋势、把 k_t 的漂移当作健康指标（第 24 章那两路 temperature 与 `MotorState` 的 `tau_est` 就是为这件事准备的硬件入口[厂]）。

执行器磨损改变动力学：同一个策略为什么三个月后会退化

上面六条漂移里，最容易被算法侧忽略的是第一条与最后一条的合成效果：**磨损不改观测分布，它改的是动力学**。也就是说，相机看到的画面还是对的，而“同样的指令会产生同样的动作”这个假设不再成立。这条链可以接到第 14 章已经算过的数上。

一，齿隙变大（齿隙 = 反着推时先空掉的那一小段空程，行业里也叫背隙或回差）。第 14 章那条链是：15 bit 编码器经 20.58:1 折算到输出端是 0.000534°，而官方 Z1 页给的谐波背隙 6 弧分 = 0.1°，**齿隙是它的 187 倍**[算]——精度地板在齿隙那里，不在传感器那里。磨损动的正是地板那一块：拿末端位移换算一次，0.1° = 0.001745 rad，乘小腿长 0.4 m（第 37

章同一假设 [未]）得 0.001745 × 0.4 = 0.70 mm；**齿隙翻一倍就是 1.40 mm**，而 0.2° ÷ 0.000534° = 375 倍。**传感器一位没变，位置误差带翻倍**。1–1.4 mm 与许多抓取任务的位姿容差正好同量级。

二，摩擦变大。官方训练仓库给的关节模型是 τ_friction = Fs · tanh(q̇/Va) + Fd · q̇，N7520-22.5 这一档 Fs = 2.4 N·m、Fd = 0.24 [厂]。把它与该档的同向峰值扭矩放在一起看，2.4 ÷ 111 = 2.2% [算]——这就是力矩模式下“指令与实际输出”之间那段死区的量级（宇树未公布这些系数的施加侧口径 [未]）。润滑脂老化、磨屑进入啮合区，动的就是这一段。

三，k_t 永久变小。第 36 章那条正反馈给过数：80 °C 温升时 1 − 0.0011 × 80 = 0.912，同电流的扭矩少 8.8%；而一旦越过不可逆退磁的膝点，这个折扣就回不来了。

于是“策略退化”不需要任何玄学解释：**策略是在旧动力学上拟合出来的，部署三个月后那台机器的执行器已经换了一套参数**。第 21 章那套域随机化能覆盖一部分，但（如本节开头所说）它覆盖不了单调漂移。所以一条诚实的评测必须包含“跑了一年之后”这一栏，而这一栏在整个行业里几乎找不到 [未]。

人会检核与重试：把乘法改成平方

现在回答那个最扎眼的问题：**人单次也就 92%，为什么人能连续干八小时？**

因为人不做 p^N 这件事。人会检出失败，然后重试。**检核与重试改变的不是 p，而是“失败是否可恢复”——也就是指数律的第二条前提**。把它写成式子：设单步失败率 q = 1 − p，失败的检出率是 d，检出后允许重试一次，那么这一步最终失败的概率是

$$q_{\text{有效}} = q\cdot(1-d) + q^2\cdot d$$

第一条是“没检出来”的失败（q 乘漏检率），第二条是“检出来了、重试又失败”的。代 q = 0.01：

检出率 d	q_有效	连续 100 步	连续 28,800 步
0（不检核）	1×10^{-2}	36.6%	2×10^{-126}
0.5	5.05×10^{-3}	60.3%	4×10^{-64}
0.9	1.09×10^{-3}	89.7%	2×10^{-14}
1（完美检出）	1×10^{-4}	99.0%	5.6%

（[算]，`foc_check.py compound_success()` 与上式。）

读这张表的方式是看最左那一列：检出率不是一阶指标，它决定你能不能进入平方法则。 $q(1-d)$ 那一项是地板——$d = 0.9$ 时地板是 0.001，于是重试再多次也只能把失败率压到千分之一，而不是万分之一。**“知道自己失败了”这件事的价值，是把失败率从 q 降到 q^2；而它的全部前提是你真的检得出来。**

那检测又靠什么？靠信息，不靠意愿。可行的入口本书前面都出现过：`tau_est` 与指令扭矩的残差（第 24 章）、双编码器之差是否超出齿隙带（第 14 章）、末端力曲线在应该接触的时刻有没有台阶（第 18 章）、以及动作分块之后世界有没有按预期变（第 31 章说这正是分块架构不提供的那个表征）。**每一路检测都要一份传感器算力与一份标定，所以“闭环检测”是一项要花钱的工程，不是一个模型输出。**

这条链在哪里断，也必须写出来：**检核与重试救不了 8 小时。** 表里最右一列即使 $d = 1$ 也只剩 5.6%。指数律给出的出路只有两条，都不是“更聪明的模型”：把 q 压到 10^{-5}–10^{-6} 量级（这是系统工程：标定、健康监测、受控场景），或者**把 N 切开**——把 28,800 步的任务链拆成若干段，每段结束时由人或一个显式检查点确认，失败只在该段内重做。工厂里那条流水线之所以能跑，靠的正是检查点，不是工人的单次成功率。**这也是为什么本章认为“分段 + 检查点”在人形上会被低估：它不性感，但它是唯一一条已经被算过的路。**

一个可以带走的评估框架

看任何一个人形作业能力宣称，问五句：

> ① 成功率的口径是什么？（单次动作 / 任务级 / 含人工兜底）
> ② 连续多长时间？多少件？
> ③ 失败之后系统自己怎么处理？（停住 / 重试 / 呼叫人）恢复要多久？
> ④ 环境变化了吗？（新物体、新光照、新摆放）变化后成功率掉多少？
> ⑤ 这套系统跑了多久？（三个月和三年的差别不是数据量，是漂移）

五句里，行业公开材料通常能答 ① 和 ④。 而 ②③⑤ 才是决定它能不能进厂的值，而它们恰好是最难报的。

类比，以及它在哪儿破

类比：机器人做任务像运动员做动作——95% 成功率意味着每 20 次错一次。

这个类比在哪里破，破得最有启发：运动员失误之后会**调整**——他知道刚才为什么没接住，下次会把手放低一点。而策略函数不知道自己刚失败了（第 ② 条原因）。

人类工人之所以能在 92% 的单次成功率下连续干八小时，是因为那 8% 的失败他每一次都当场修正了，而且修正不需要重新训练。 这就是“人在环”与“自主”之间真正的差距——不在成功率，在**失败后的处理带宽**。

所以本书对“人形什么时候能顶替工人”这个问题，给一个可检验的答案：**不是等成功率到 99.99%，而是等系统获得“知道自己失败了、并且能自己换个办法”的能力。** 而这件事需要的是记忆 + 在线改进 + 硬件健康监测三样，其中只有一样是模型问题。

一个可检验的说法：连续作业能力与单次成功率之间是指数关系 P = p^N，其成立的前提是各步失败相互独立且不可恢复；因此 95% 的单次成功率在 100 次连续动作下只剩 0.6% 的全对概率，而 8 小时按每步约 1

s 折成 N = 28,800 步时，99% 的单步成功率只剩 2×10^{-126}——要在 28,800 步里有 90% 的整体成功，单步失败率必须低于 3.7×10^{-6}，即平均 27 万步才允许错一次；长时程失败有三条原因，其中“分布外”是算法问题、“无记忆”是架构问题（分块与模仿学习都不提供失败感知）、“硬件漂移”是物理问题且域随机化在结构上无法覆盖单调漂移；检核与重试把失败率从 q 压到 q^2（q_有效 = q·(1 − d) + q^2·d，d 为检出率），能把 100 步从 36.6% 提到 99.0%，但救不了 28,800 步（仍只有 5.6%），所以剩下的出路只有两条：把 q 压到 10^{-5}–10^{-6} 量级，或把 N 用检查点切开；磨损不动传感器、只动动力学——齿隙从 0.1° 到 0.2° 时末端空程从 0.70 mm 到 1.40 mm、与编码器折算分辨率的倍数从 187 升到 375，而策略是在旧动力学上拟合的。

证据等级：p^N 与表中所有数、0.99^28800 = 2×10^{-126}、p = 0.9^(1/28800) = 99.99963% 与 27 万步、q_有效 = q(1 − d) + q^2d 那张表、0.70 mm 与 1.40 mm 与 375 倍、2.4 ÷ 111 = 2.2%、44%^(1/5) = 85%（该例为假设，不指名厂商）、0.9^(1/1000) = 99.99%、400 × 68 s = 7.6 h 是 [算]（`foc_check.py compound_success()`；小腿 0.4 m 沿用第 37 章的本书假设 [未]，Fs 的施加侧口径官方未公布 [未]）；齿隙 0.000534° 与 187 倍是第 14 章的 [算]，Z1 背隙 6 弧分、N7520-22.5 的 Fs/Fd 与 Y1 = 111 N·m、τ_friction 模型式 [厂]；药房日均单量与准确率、宁德时代 3→6 个月、Helix 56%、Gemini Robotics 2 的三项、π*0.6 的 13 小时与 RECAP，全部为厂商自陈或媒体报道 [查][推]，**口径未统一、本书未复现、且多数未说明失败定义；各家公开的可比数几乎都只是短任务成功率与自宣时长**；六条漂移机制各自回指第 10、14、21、25、35、36 章的已证内容；**宇树未公布任何一款人形的连续作业时长、MTBF、失败恢复机制，也没有任何公开的关节失效率或磨损寿命数据** [未]。

可核对清单：

- ☐ 自己算 0.95^100 与 0.99^500，确认 0.6% 与 6.6%
- ☐ 自己算 0.99^28800（先算 28800 × $\log_{10}$0.99 = −125.7），确认它在 10^{-126} 量级

- ☐ 用 p = 0.9^(1/28800) 反解一次，确认单步失败率是 3.7×10^{-6}，也就是 27 万步一次
- ☐ 拿 q_有效 = q(1 − d) + q^2d 代 q = 0.01、d = 0.9，确认地板项 q(1 − d) = 0.001 而不是 q^2 = 0.0001
- ☐ 用 0.001745 rad × 0.4 m 复算 0.70 mm，再把齿隙翻倍看它变 1.40 mm
- ☐ 找一份你最近看到的机器人作业宣传，用上面五问逐条对照，记下能答几条
- ☐ 用 44%^(1/5) 反推一次，体会“任务级成功率”与“单次成功率”的差距
- ☐ 想清楚为什么域随机化覆盖不了磨损（提示：随机化是独立同分布抽样，磨损是单调函数）

下一章：本章说长时程的三条原因里只有一条是模型。下一章去看这个行业现在把人和钱放在了哪几条原因上——以及哪一格最缺人。

第 39 章

行业分工与人才缺口：谁在做什么，缺什么人

同构的技术栈，分歧的三处

先给本章最重要的一张表。三家中国头部公司，逐层对照（宇树官方项标[厂]，另两家一律 [查]——图例里 [厂] 只留给宇树；来源见附录 E）：

层	宇树	智元	银河通用
① 感知	深度相机 + 3D 雷达；自研 L1/L2	头双目 + 腕 RGBD + 鱼眼 + 雷达 + 超声；**全关节力矩传感器**	头双目 + 躯干 4 相机 + 腕深度 + 六维力 + 雷达
② 状态估计	双 IMU 字段	未公开细节	未公开细节
③ 决策	UnifoLM 一族（WMA/VLA/WLA，6B）	GO-1 / GO-2（InternVL2.5-2B + MoE / ACoT-VLA）	AstraBrain（WAM + WBC 0.5）+ GraspVLA
④ 全身控制	公开工序是 RL（rl_gym / rl_lab）	未公开	AstraBrain-WBC 0.5（2 亿帧 / 2 万小时 / ONNX 端侧）
⑤ 关节伺服	自研电机，RS-485 4–6 Mbps	自研 PowerFlow，EtherCAT 1 kHz ↔ 3×CAN-FD	**完全未公开**
⑥ 功率与机械	两级行星（拆解）/ 型号名 14.3–22.5	准直驱 + 高透明度行星 + 液冷（宣称替代谐波使 BOM ↓30%）	未公开
中间件	裸 CycloneDDS + ROS 2 适配	自研 AimRT / 灵渠 Link-U OS	自带 GalbotSDK
大脑硬件	Orin NX 16 GB / Thor T5000	AGX Orin 64 GB + x86 工控；A3 加 Thor	AGX Orin 64 GB

数据来源	开源生态 + 遥操（HF 约 200 集）	**真机数采工厂**（百万轨迹）	**仿真合成约 99%**
整机价格	G1 ¥8.5 万起 / H2 ¥19.9 万	精灵 G1 45 万 / X2 青春版 9.8 万	不公开，走行业订单

这张表最该被读出来的不是差异，是同构。

十行里，前八行三家几乎在做同一件事：同样的传感器类别、同样的 MCU+FOC+现场总线、同样的“VLM 主干 + 动作专家”、同样的 Orin/Thor。**分歧只在最后两行：数据从哪来、以及卖给谁。**

这个观察的实际价值是：如果你要选一个职业方向，“哪家用的技术更先进”是错问题；“哪一层还没有人做好”是对问题。

在 AI 领域，没有一家大公司能保证，只要有足够的人、有足够的资源，我就能永远领先。OpenAI 和 DeepSeek 已经证明了，AI 的创新永远伴随着一些随机性，伴随着更多的聪明年轻人的。

王兴兴 — 王兴兴 2025 世界机器人大会主旨演讲（新浪“机器人前瞻”文字实录） — 2025-08-09（第一人称演讲逐字）



图 39-1　六层分工与拥挤度：哪一层缺人、哪一层挤满了人

往下追一层：为什么偏偏是这三处分叉

同构表里那三行（大脑硬件、关节伺服、中间件）不是并列的三条品味差异，它们各自有一个能算的理由。而把理由算出来，才知道每一层要什么样的人。

一，大脑层被锁死，锁的是软件栈，不是芯片。 先看硬件为什么不是理由：一个 6B 参数的模型，权重字节是 $6 \times 10^9 \times 2\ \text{B} = 12\ \text{GB}$（FP16），单步推理的时间下限是

$$t \geqslant \text{权重字节} \div \text{内存带宽}$$

第 26 章按这条式子给过一整列：带宽 41.6 GB/s ⇒ 12 ÷ 41.6 = 288 ms，100 GB/s ⇒ 120 ms，273 GB/s ⇒ 44 ms [算]——**同一颗“算得快”的芯片，如果带宽不够，模型就是在等内存而不是在算**。而换芯片真正要换掉的东西在后面：训练侧，官方 `unitree_rl_lab` 的 README 把环境钉在 Isaac Sim 5.1 + Isaac Lab 2.3 [厂]，上一代 `unitree_rl_gym` 钉的是 NVIDIA 已经终止支持的 Isaac Gym Preview 4 [厂][查]，官方仓库只在 3080 / 3090 / 4090 上测过、RTX 50 系要换 Isaac Sim 5.0.0 [厂]；部署侧，模型导出走 ONNX / TensorRT（第 30 章）[厂]；生态侧，能直接接进来的开源策略是 GR00T 与 π0.5 这一族（第 32 章）[查]。**换掉 NVIDIA = 同时换掉训练环境、推理运行时、算子与量化工具链、以及所有别人已经训好的模型**。这就是“锁死”的确切含义：Orin → Thor 的升级路径之所以只能跟着它的节奏走（官方给的口径是 Thor T5000 2070 TFLOPS（FP4-Sparse）与 Orin NX 100 TOPS（INT8），两个数不可比 [厂]，第 26 章），不是因为别的芯片跑不动，是因为换过去这些数全都要重做一遍。

二，关节层是劳动密集，因为位时间不允许把环路挂在总线上。 这是本节唯一一条能一行算完的硬理由。开源参照实现 ODrive 的官方口径是环路周期恒定 125 µs（8 kHz）、CAN 2.0b 跑 1 Mbps [查]。同一时间窗里总线能搬多少？一帧下行 17 字节，按 8N1（1 起始 + 8 数据 + 1 校验/停止

位的常见串口开销，本书在第 27 章用的就是这个口径）是 17 × 10 = 170 bit，在 1 Mbps 上就是 170 µs：**170 ÷ 125 = 1.4 倍——一帧还没发完，环路已经要闭第二次。** 所以电流环在数学上就不可能由上位机通过总线闭合，它只能长在关节自己那颗 MCU 里。另一家参照实现 moteus 的官方口径把这个结论反过来写成了规格：5 Mbps CAN-FD、控制环 15–30 kHz、PWM 默认 30 kHz [查]——33 µs 一个开关周期，而 5 Mbps 的位时间是 200 ns，一帧 17 B 在这个速率上是几十微秒量级（本书用同一个 170 bit 口径粗算得 34 µs；CAN 的位填充与帧开销另计，这里只取同量级 [行]），**于是又是“一帧 ≈ 一个环”的比例**。宇树自己的数也是同一族：官方文档给 M8010 的帧长是下行 17 B、上行 16 B、总线 4 Mbps [厂]，脚本拿 23 台全双工 1 kHz 算得需要 6.07 Mbps、反过来 4 Mbps 最多撑 659 Hz（`foc_check.py bus_budget(nodes=23, hz=1000)`）[算]，而官方 Z1 页标的控制频率是 1 kHz [厂]——**这两件事只能同时成立的方式，是 1 kHz 那条给外环，内环在本地**（帧格式细节官方未公布 [未]）。

于是关节层的工程量是按“颗”计的：G1 官方 23 个自由度 [厂]，每一颗关节都要 1 台 MCU + 6 只功率管（三相半桥，第 08 章那六个矢量）+ 1 只栅极驱动 + 1 套编码器 + 1 路总线收发器，再加一整套要自己调的 FOC 参数与第 13 章那三类失效判据。23 × 6 = **138 只功率管，每只都要过一遍死区与采样时刻这一关**。这一层不缺算力、不缺融资叙事，缺的是把 23 份一模一样的板子都调到能出扭矩且不发热的耐心——**所以它是劳动密集，所以供应商遍地，所以它的门槛在器件层而不在资金层**。这也是本章标题那句“器件层决定谁能进入这行”的第一层意思。

那这一层为什么不能用钱压平？把三个数并排放就知道它卡在哪：

一帧 17 B × 10 bit = 170 bit ⇒ 在 1 Mbps 上要 170 µs，而 ODrive 的环路周期只有 125 µs——**帧比环长，内环只能就地闭**

23 台 × (17 + 16) B × 1 kHz × 8 = 6.07 Mbps ⇒ 需要官方 4 Mbps 的 1.5 倍，反解上限只有 659 Hz（`foc_check.py bus_budget(nodes=23, hz=1000)`）

23 个关节 × 6 只功率管 = 138 只 ⇒ 而大脑侧只有 1 块 SoC，**工程量的分母差 138 倍**

这三行里没有一行是算力问题：买更多 TOPS 不会让 170 µs 变成 100 µs，也不会让第 138 只功率管自己学会不直通。它们只能一份一份地调。这就是“劳动密集”四个字的算术内容。

三，**中间件在去 ROS 化，岗位含义跟着变**。第 28 章已经核实：宇树的核心 SDK 不依赖 ROS，它是裸 CycloneDDS 的主题与服务，ROS 2 只是另外提供的一层适配（还要自编 cyclonedds 0.10.2 并设 RMW）[厂]；智元自研 AimRT 与灵渠 Link-U OS [查]；银河通用的公开文档里根本没有 ROS/DDS 这一层 [厂]。**去 ROS 化不是厂商傲慢，是“关键路径是确定性的时候，抽象层要付利息”**（第 28 章原话）。它对求职者的实际含义很具体：这一层要的人得能读 DDS 的 QoS 语义、能接受“上一帧根本没到过”这个默认假设，而不是会跑 ROS 教程里的例程。

一条判据：技能要能对账，不能对口号

把上面三层换成可核对的岗位—技能对应表。**每一格都写“你拿什么证明你会”，而且那个证明是一次代数**——因为这一行的面试里，说“我熟悉 FOC”不算任何东西，把 i_q 与扭矩对上手才算。

岗位	这条技能的可复算判据	对应章
关节伺服 / 电机控制	能判手册扭矩常数的口径：0.63895 N·m/A × 40 A = 25.56 N·m，与官方峰值 23.7 N·m 差 +7.8% ⇒ 它是**输出端**口径（转子端是 0.10094）	04

同上	能算电压包络：E = (2/3)·k_t·ω = (2/3) × 0.10094 × 189.9 = 12.78 V（幅值不变约定），24 V 母线的 SVPWM 线性区给 24 ÷ √3 = 13.86 V ⇒ 空载够、满载（+40 A × 40 mΩ = 14.38 V）不够 ⇒ 峰值扭矩与最高转速是两个角点，不能同时取	08
同上	能算死区：(100 ns ÷ 50 µs) × 24 V = 48 mV，并说清它为什么在过零点最疼	13
同上	能把合成磁场讲成一句话：三相互差 120° 的正弦，合成矢量幅值恒等于相电流峰值，方向由各相导通时间比决定	07、08
嵌入式实时	能读帧与位时间：4 Mbps ⇒ 1 bit = 1 ÷ (4 × 10⁶) = 250 ns；一帧 17 B（8N1）= 170 bit ⇒ 42.5 µs	12、27
嵌入式实时	能做总线预算：15 台 × 6 kHz × (17 + 16) B = 23.76 Mbps，是官方 4 Mbps 的 5.9 倍；反解上限 1010 Hz	27
结构与传动	能找精度地板：360 ÷ 32768 ÷ 20.58 = 0.000534°，而背隙 6 弧分 = 0.1°，是它的 187 倍	14
运控 / RL	能用官方数闭环 kp：ω_n = √(kp ÷ armature) = √(99.098 ÷ 0.02510) = 62.8 rad/s = 10.0 Hz，与官方注释一致 ⇒ kp 是反推出来的不是手调的	18

热与可靠性	能算热漂移闭环：1 + 0.00393 × 80 = 1.314（铜阻）与 1 − 0.0011 × 80 = 0.912（剩磁）⇒ 同扭矩铜损涨 58%	36
电源与电池	能判断电芯化学：75.6 V ÷ 18 串 = 4.20 V/颗（终止）、0.972 kWh ÷ 15 Ah ÷ 18 = 3.60 V/颗（标称）⇒ 不是磷酸铁锂；能算母线塌陷为什么会让 MCU 复位	35
数据与工具链	能算同步误差的代价：20 ms 不同步在 1 m/s 相对运动下 ≈ 13 像素伪视差，是 0.5 像素测距精度的 27 倍	23、33
评测与安全	能算复合衰减与置信：0.99^100 = 36.6%，而 10 次里成 7 次的 95% 区间宽到 ±28%	37、38、40

这张表里没有一个数字来自招聘市场。它给的是“会”这个字的最低标准：能拿官方公布的数做一次代数，并说清这一步在算什么。**它的用法是反向的——去面试别人也照这张表问：如果对方连 k_t 的口径都判不出来，他对这台机器的所有判断都要打折**。

边界照旧要写在正文里：以上关于“谁缺、谁挤”的全部内容都是行业观察 [行]，不是事实陈述；本书不给岗位数量、薪资区间、缺口人数这一类数，因为**没有可核来源** [未]——本章宁可空着也不填，这本身就是它想教的那件事。

这条链在哪里断：门槛低不等于这层好做

上面把关节层说成“劳动密集、供应商遍地”，很容易被读成“这层没难度”。恰恰相反，它的难度都以**不可见**的形式存在，而这正是它缺人的根因。

第一，**上限不是被物理顶住的，是被文档顶住的**。你要调一颗关节，需要 L、R、k_t、齿隙、T-N 折点、热阻这一整张参数表——第 15 章已经核过：产品页给不出这些，官方训练仓库给得出摩擦与折算惯量，而电气那一半（相电阻、电感、死区、母线电容）至今未公布 [未]。**结果是这一层的工程师必须在信息不完备的条件下做判断，而且没人教他怎么标不确定度**。追问一句：那这类判断力从哪来？只能从“自己画过板、自己烧过件”来——它不可速成，也没有发表渠道，这正是第 39 章开头那条“成果不可发表”的另一面。

第二，**代价是延迟暴露的**。死区补偿没调好，演示里看不出问题，八个月后磨损与油膜变化把它放大（第 38 章那条漂移链）；齿隙当年 6 弧分、跑一年翻倍，位置误差带跟着翻倍 [算]。而这一层的失效表现是冒烟、摔机、伤人，当场兑现（第 13、37 章）——**做砸的代价高到必须实物练，实物练的门槛高到人学不会**，这是一个自锁的环，比任何薪资数字都更能解释为什么这层缺人。

第三，**“器件层决定谁能进入这行”这句话有一个反例要写出来**。如果一个人只有算力与钱，他可以买 Orin、租万卡、跑现成模型，在 ③ 层迅速产出可发表的东西；而他买不到“23 颗关节同时不抖”这件事——那 23 份调试工时只能一份一份花。所以 ⑤⑥ 层的进入门槛低（一张 STM32G4 与几颗功率管的钱，第 40 章给量级），而**天花板高且不可用钱压缩**。这两句合起来才是本章的职业结论。

六层各自的拥挤程度

把行业投入（钱与人）按本书的六层排一下：

```
① 感知        中    —— 器件成熟，缺的是同步与标定工程
② 状态估计    低    —— 学术成熟，工程岗位少
③ 决策        极挤  —— 万卡算力 + 博士密度 + 融资叙事中心
④ 全身控制    中    —— RL 路线把显式控制的人吸走了
⑤ 关节伺服    缺    —— 会 FOC + 会总线 + 会实时的人极少
⑥ 功率机械    最缺  —— 会画功率板、会算热、会做减速器的人最少
```

为什么 ⑤⑥ 缺而 ③ 挤？ 三个原因，都很实在：

一、门槛的可见性。 ③ 有论文、有开源权重、有排行榜，做了就能被看见。⑤⑥ 的成果是“这台机器连续跑了六个月没坏”，**没有发表渠道**。

二、失败的方式。 ③ 做砸了是指标低；⑤⑥ 做砸了是**冒烟、摔机、伤人**，而且是当场。这决定了它需要的是有过实物经验的人，而实物经验不可速成。

三、技能的复合性。 第 1 部十章讲的东西，在现实里分属四个专业：电磁与电机设计、电力电子、嵌入式实时、控制理论。**一个同时懂这四样的人，在任何一家公司都是稀缺资源；而只懂其中一样的人很多。**

（本章不给“缺多少人”这类数，因为公开可核的数不存在 [未]。给的是判据与原因，比数更耐用。）

三家的钱押在哪，说明他们各自认为瓶颈是什么

```
宇树：募资约 42 亿元 — 20.22 亿具身模型、11.1 亿本体、6.24 亿制造基地
      （这三项之和 37.56 亿，其余 4.44 亿未见项目级拆分，本书不硬凑）
      产能规划：7.5 万台双足 + 11.5 万台轮式
智元：量产 2025-01 破千台 → 2026-06 第 15,000 台；2026 目标 1,000 万小时有效数据
      2026-07 启动港股 IPO
银河通用：估值 >230 亿元、已完成股份制改造；客户含宁德时代、博世、丰田、北汽、
          上汽、极氪、西门子；S1 在某基地 7×24 无遥操 3→6 个月
```

（全部 [查][推]，媒体与厂商口径混合。）

读这三段的方式是问“钱换的是什么”：

- 宇树用一半的钱买**模型能力**，用另一半买**产能**——它认为自己的躯体已经够好。
- 智元把目标写在**数据小时数**上——它认为瓶颈在数据（第 33 章）。
- 银河通用把叙事写在**客户名单与无遥操时长**上——它认为瓶颈在场景落地（第 38 章）。

三种判断都合理，而它们互不兼容。 这就是这个行业还没有共识的证据。**对读者的含义：不要因为某家公司的选择而认定那是唯一的路。**

三家共同承认的两个短板

这是本章最有价值的一节，因为它是**竞争者难得一致**的地方。

一、可靠性与室外防护。 智元公开承认续航、稳定性、速度、室外防护不达标；宇树在招股书里主动“放弃工业重载与连续作业” [推]。**两家分别承认了自己最弱的那一面，而它们指向同一处：这台机器还不能在脏、湿、重、连续的环境里干活。**

第 23 章（100 Klux 边界）、第 35 章（电压塌陷）、第 36 章（热）、第 37 章（没有强制标准）、第 38 章（漂移）——**本书已经用五章证明了这不是宣传话术，是物理与制度的复合结果。**

二、数据。 三家在路线上分歧，但对“数据不够”没有异议。而“不够”的具体含义，第 33 章给过：任务异质、分布割裂、没有可信的“真机等价于多少仿真”换算比。

一个空白，和一个位置

把上面所有场景清单列一遍：厨房、桌面、货架、办公室、仓库、装配台、药房。**共同点是室内、平整、光照可控、物体人造、负载轻。**

没有任何一家公开拥有：非平整地面、坡地、作物行之间、泥、雨、正午强光下、以及 50 公斤级负载的连续作业数据。

本书不主张“所以你应该去做农业机器人”——那是建议，不是事实。本书只指出一个可核实的结构：**这是当前所有公开数据集的一个共同空洞，而填它需要的不是算力，是场景准入。**

（场景准入 = 你能连续几周在同一块地里操作一台机器，且这台机器坏了你自己能修。这件事的门槛不在技术，在**你能不能长期待在那个场景里**。）

一张“我该往哪一层走”的判据表

你现在的背景	最短路径进入的那一层	为什么	需要补的（对应章节）
嵌入式 / 单片机 / 电机控制	⑤ 关节伺服	语法相同，只是对象从风扇/云台变成机器人关节	级联三环（12）、总线预算（27）、实时性测量（22）
电力电子 / 电源	⑥ 功率与热	你已经在处理同量级的损耗与热	齿隙与传动（14）、热阻链（36）
机械 / 车辆 / 农机	⑥ 结构与传动	你已经懂刚度、模态、疲劳	准直驱取舍（14）、平方-立方（02）
软件 / 后端 / 基础设施	数据与工具链（33）	这一层缺的是工程化能力，不是算法	数据格式与质检（33）、评测协议（38）
算法 / 深度学习	③ 决策，或 ④ 运控 RL	迁移最近	**必须补 ⑤⑥**，否则你会写出物理上不能执行的策略（04–15）
测试 / 质量 / 安全	贯穿 ①–⑥ 的可靠性与合规（不是第七层）	全行业最少人、且第 37 章显示制度即将出现	全书第 5 部

这张表里唯一一条硬建议是第五行的加粗部分。一个只做 ③ 的算法工程师，如果不知道 $\tau = k_t \cdot i$ 的口径陷阱（第 04 章）、不知道电压包络让角点互斥（第 08 章）、不知道齿隙是精度的地板（第 14 章），他会持续产出**在物理上无法执行的动作指令**，而且不知道为什么被拒。**这是本书第 1 部存在的职业理由**。

类比，以及它在哪儿破

类比：这个行业像 1995 年的互联网——基础设施缺、应用不缺想法、而所有人都在赌哪一层会赢。

这个类比在哪里破，破得最重要：1995 年的互联网，硬件（TCP/IP、光纤、CPU）是**已经存在且免费可用**的，所以创业全压在应用层。而人形机器人的硬件**每一层都还没稳定**：关节参数表不完整（第 15 章）、总线要自己定协议（第 27 章）、电池化学要自己选（第 35 章）、安全标准不存在（第 37 章）。

所以这个行业不可能只靠应用层创业成功。它需要的人，是愿意去做那些“不性感、不能发表、会冒烟”的层的人。而本书第 1 部之所以用十一章讲一颗关节，就是这个原因：**那是这个行业里最不被讨论、而最决定成败的一层**。

一个可检验的说法：宇树、智元、银河通用在感知、状态估计、决策架构、关节伺服、功率机械、中间件六层上高度同构，分歧只在数据来源（开源生态 / 真机数采工厂 / 仿真合成）与客户定位；三层的分工由器件层的算术决定而不是由融资叙事决定——大脑层被锁死的是软件栈（6B × 2 B = 12 GB 权重，带宽 41.6–273 GB/s 决定单步 288–44 ms，而训练侧钉在 Isaac Sim 5.1 + Lab 2.3、部署侧走 ONNX/TensorRT、生态侧接 GR00T 与 π0.5，换芯片要同时重做这四件事），关节层是劳动密集（一帧 17 B 按 8N1 是 170 bit，在 1 Mbps 上要 170 µs，而 ODrive 的环路周期是 125 µs：**帧比环长，所以电流环只能长在关节自己的 MCU 里**，于是 G1 的 23 个自由度就是 23 份固件与 23 × 6 = 138 只功率管的调试工时），中间件在去 ROS 化（宇树核心是裸 CycloneDDS，智元自研 AimRT / Link-U OS，银河通用公开文档里没有这一层）；三家分别把资金押在模型能力、数据小时数、场景落地三件不同的事上，说明行业对瓶颈没有共识；而三家共同承认可靠性与室外防护不达标，且没有任何一家公开拥有田间/重载/非结构化环境的操作数据；从人才看 ⑤⑥ 层最缺而 ③ 层最挤，原因是成果不可发表、失败方式致命、且需要四五个专业的复合。

证据等级：三家逐层规格取自各家官方页面、文档中心、GitHub 与 arXiv：宇树的记 [厂]，另两家的记 [查]；12 GB 与 288–44 ms 沿用第 26 章的 [算]（其中内存带宽 41.6 GB/s 那档的位宽是本书假设 [未]）、Isaac Sim 5.1 / Lab 2.3 与 Isaac Gym Preview 4 终止支持 [厂][查]、GPU 实测型号与 RTX 50 需 5.0.0 [厂]、ONNX/TensorRT 导出 [厂]、GR00T 与 π0.5 生态 [查]；ODrive 环路周期 125 µs（8 kHz）与 CAN 2.0b 1 Mbps、moteus 5 Mbps CAN-FD / 15–30 kHz 控制环 / 30 kHz PWM（= 33 µs）、位时间 200 ns 与一帧 34 µs，均为参照实现的官方口径 [查]；M8010 帧长 17 B / 16 B、总线 4 Mbps、Z1 控制频率 1 kHz、G1 23 DoF [厂]；6.07 Mbps、659 Hz、23.76 Mbps、5.9 倍、1010 Hz、250 ns、42.5 µs、138 只功率管、170 ÷ 125 = 1.4 倍，以及判据表里每一行的代数，是 [算]（`foc_check.py` 的 kt_crosscheck / voltage_envelope / deadtime_error / resolution_vs_backlash / bus_budget / natural_frequency_check 与本书手核）；募资与产能分配、量产台数、估值、客户名单、无遥操时长 [查][推]，**多为媒体与厂商口径混合，本书未独立核实**；“六层拥挤度”“三个原因”“器件层决定谁能进入这行”是行业观察 [行]，**本书不提供岗位数量、薪资与缺口人数，因为无可核数据** [未]；宇树关节总线的条数与拓扑未公布 [未]，而拆解口径与官方口径在“485 还是 CAN”上直接冲突，两条都保留（第 27 章）；判据表各行的技术依据回指对应章节。

可核对清单：

- ☐ 打开三家官网，各自找一款旗舰人形，把上表十行自己填一遍，记下哪几行三家都写不出来（预期：⑤ 层细节）
- ☐ 自己算一次 17 × 10 ÷ 10^6（1 Mbps 下一帧多少微秒），再与 125 µs 比一比，体会“帧比环长”
- ☐ 跑 `python -X utf8 code/tools/foc_check.py`，把判据表里那六条代数逐条对上脚本输出
- ☐ 用 12 GB ÷ 100 GB/s 复算 120 ms，然后想清楚：这条式子里哪个量是你换芯片能改的、哪个不能

- ☐ 找 5 个机器人公司的“运动控制/执行器”岗位 JD，数一数它们要求哪些第 1 部的能力；再找 5 个“大模型算法”JD 做同样件事
- ☐ 用第 04、08、14 章的三个判据（k_t 口径、电压包络、齿隙地板）去检查一份你正在用的关节参数表
- ☐ 想清楚你选的那一层，它的成果**能不能被发表**；如果不能，你是否有别的方式让它被看见（第 40 章）

下一章：本章给了地图与判据。最后一章把它们变成一条十二个月、每步都有可测产物的路线——这是本书唯一一章，读者需要拿支笔在上面打勾。

第 40 章

一个工程师的十二个月：把这本书变成作品集

这一章的规矩：每一层只认一个可测产物

第 39 章说过 ⑤⑥ 层的成果“不可发表”。这一章要解决的就是这件事——**用一种不需要论文、不需要许可、不需要任何人批准的可见方式**。

规矩只有一条：

每一层的学习，必须以一个“能跑、能录像、能被别人复现、且自带一个测量数”的产物结课。读完一本书、看懂一段代码、跑通一个别人的例程，都不算产物。

为什么这么严？因为“我学过 FOC”和“这是我做的驱动板，这是它的力矩-给定曲线，误差 4.2% 满量程，测试脚本在这儿”——**在招聘者眼里是两种完全不同的东西**。后者不需要相信你说任何话。

（下面六个里程碑的预算，按每周 12–18 小时稳定投入估。**它们是量级，不是承诺**。预算、时间、学校资源因人而异——有人有一块空台架与一台示波器，有人只有周末与一台笔记本；**本书给路线不给承诺，也不给时间表**，每个人的起点差异比这条路线本身的作用更大。）

我觉得主要是你足够专注。你 24 小时一直在思考这个问题，我不信这个问题是解决不了的。

王兴兴 — 《晚点 LatePost》对话王兴兴（新浪转载） — 2025-02-16（首刊 2024-05）（直接引语（问答实录））

什么叫“可复算的验收”：三条判据先算给你看

这一章后面每个里程碑都带一条验收判据，而判据必须写成**读者能自己复算的形式**，不是形容词。三条样例，它们各自回答一个“为什么是这个量级”。

一，力矩误差 ≤5% FS——FS 不写出来，这条判据就没有意义。“5% 满量程”里的 FS 是满量程扭矩，而不同关节的满量程差几十倍。拿官方 G1 关节家族的两个真实档位算：

0.05 × 5.0 N·m = 0.25 N·m = 250 mN·m（W4010-25，腕部；官方 `unitree_rl_lab` 电机表里这一档的 effort 上限是 5.0 N·m [厂]）

0.05 × 139 N·m = 6.95 N·m（N7520-22.5，膝与髋滚；`unitree_rl_lab` 的官方电机配置给这一档的 effort 是 139 N·m [厂]。**注意同一个 `unitree_ros` 仓库里两个 G1 URDF 对髋滚分别写 139 与 88**，所以引用官方模型的 effort 必须指明是哪个文件——本书取 rl_lab 配置那一档；取 88 时这条判据变成 0.05 × 88 = 4.4 N·m）

同一条“5% FS”，在一个关节上要求 250 毫牛米的分辨力，在另一个关节上允许 6.95 牛米的误差——差 27.8 倍（6.95 ÷ 0.25 = 27.8）。而 6.95 N·m 比第 38 章那条静态摩擦 Fs = 2.4 N·m 还大，比很多接触任务的容差大得多：**所以“5% FS”这种写法在大关节上是一种掩护。**你的验收要写成两个数：绝对误差（mN·m）＋它对应的 FS 口径。

还有一个更硬的理由要求你自建电流环：官方 IM6014 协议把扭矩字段量化成 **2560 个刻度 = 1 N·m** [厂]，也就是 int16 的最小刻度是 1 ÷ 2560 ≈ **0.39 mN·m**、满量程 2^{15} ÷ 2560 = 12.8 N·m——**这两个数都是转子侧的**，乘上 IM6014 那 12.667 的速比才是关节侧的 **4.9 mN·m**。**先说清楚：数字刻度不是瓶颈。**拿本章最小的那档要求去比这条总线的**关节侧**刻度，250 mN·m ÷ 4.9 mN·m = **51 个刻度** [算]；按 IM6014 自己的 5% FS（0.05 × 34.4 = 1.72 N·m）来要求，则是 **348 个刻度** [算]——协议表达它绰绰有余。（**上一版这里写的是“相当于 640 个刻度”，那是拿关节**

侧的要求去除转子侧的刻度，恰好犯了本节末尾要警告的那个错；640 = 250 ÷ 0.39，把余量夸大了 12.7 倍。）真正的瓶颈在数字之外：250 mN·m 只是 34.4 N·m 峰值扭矩的 **0.73%**（250 mN·m ÷ 34.4 N·m，[算]——**注意这是一次跨档比较，分子取 W4010-25 的判据、分母取 IM6014 的峰值扭矩，本书并列它们只说明“这一族的相对量级”，不是同一颗关节的自洽除法**），而在这个量级上，第 13 章算过的死区偷电压（48 mV）、电流采样的等效噪声、以及第 05 章那张 k_t 温漂表，每一项都能吃掉可比大小的扭矩误差。**所以结论没变、理由要换准：拿总线回读当验收仪器，量的不是“协议够不够细”，而是“你能不能把模拟误差压到 0.7% 以下”——压不住，就得自己上扭矩传感器**。

这三项都要换算到牛米，才谈得上“可比大小”。以死区偷电压那一项为例（第 13 章：100 ns ÷ 50 µs × 24 V = 48 mV [算]），一路上推到关节：

```
Δi = 48 mV ÷ 0.040 Ω = 1.2 A            （相电阻取第 08 章那条，本书假设 [未]）
k_t = 0.63895 N·m/A                      （M8010 官方参数表原值，输出侧口径 [厂]）
Δτ = 1.2 A × 0.63895 = 0.77 N·m
```

这一条链只走 M8010 这一颗关节，三个数都在同一侧：它自己的官方峰值扭矩 23.7 N·m ⇒ 5% FS = 1.19 N·m，于是这笔死区误差占**自己那档容差的 64%** [算]。（上一版这里用 23.7 ÷ 40 = 0.5925 当 k_t——那是“峰值扭矩 ÷ 峰值电流”的反除法，比官方常数低 7.8%，而这条偏差第 04 章已经单独论证过一次，不该在这里被当成 k_t 再用一遍；代回官方的 0.63895，Δτ 从 0.71 变 **0.77** N·m。**同一笔误差拿去比最小的那档要求**（W4010-25 的 250 mN·m）是 3.1 倍，但那是两颗关节，本书只把它当“档位越小越致命”的量级说明。）

[算]。**但它真正的杀伤力不在大小，在它是加性的而不是乘性的**：命令 23.7 N·m 时这个固定偏差只占 3.2%，命令 1 N·m 时占 77%，命令 0.25 N·m 时它是 **307%**——一个 0.77 N·m 的恒定偏差在近零处把指令整个吞

掉。**这就是第 13 章那句“过零附近最疼”的确切算术，也是为什么做力控的关节要么上死区补偿、要么把可命令的最小扭矩抬高到一个比它大的地板。** 顺带一条同款换算，**两边都折到关节侧再比**：IM6014 那 0.39 mN·m 是**转子侧**的协议刻度 [厂]，乘上它 12.667 的速比才是关节侧的 4.9 mN·m；0.77 N·m ÷ 0.0049 N·m ≈ **155 倍** [算]，所以“瓶颈在数字之外”这句不是修辞。**但比之前先把两侧对齐**：拿关节侧的偏差去比转子侧的刻度，就又是本章前面那种跨减速比少乘一个 **i** 的错——**力矩跨减速比乘 i，惯量才乘 i^2**（12.667 与 160.5 是两回事）。

二，**周期抖动 <50 μs——为什么是这个量级，而不是 5 μs 或 500 μs。** 三个已经在书里出现的量把它夹住：

50 μs ÷ 1000 μs = 5% ⇒ 对 1 kHz 环，采样时刻的相位误差是 360° × 0.05 = 18°

1 ÷ 4 Mbps = 250 ns ⇒ 50 μs = 200 个位时间 ⇒ 按 8N1 就是 20 字节，**比一整帧（17 B）还多**

50 μs ÷ 0.259 s = 0.019% ⇒ 相对第 00 章那个倒立摆时间常数 τ，它小到不影响稳定性判据

这三行合起来就是 50 μs 的物理含义：抖动预算大约等于“总线上排队一整帧”，同时只吃掉控制周期 5% 的相位。 反过来看两个边界：取 500 μs 就是半个周期——控制器会有一半的样本在“用上一次的姿态出这一拍的力”，这正是第 27 章说的那种失效；取 5 μs 则要求把每一段中断、每一次 DMA、每一个操作系统抢占都管住，那是硬实时 RTOS 与专用定时器的活，成本与难度是另一个量级。**判据要写成你测得出的那一个，而不是听起来最严的那一个。** 而报告方式必须是直方图与 P99 / P99.9 / 最大值——**平均值在这一项上没有信息量**（第 00 章：决定成败的是最坏情况）。

三，**未见条件成功率 ⩾70%——为什么 10 次里成 7 次不等于 70%。** 成功率的报告单位是区间，不是点。按二项分布的标准误差：

$\sigma = \sqrt{(p(1-p) \div n)} = \sqrt{(0.7 \times 0.3 \div 10)} = \sqrt{0.021} = 0.145 \Rightarrow 95\%$ 置信约 $\pm 1.96\sigma = \pm 28\%$

也就是“7/10”真正说的是“真值在 42% 到 98% 之间”。这个区间大到既不能证明它可用、也不能证明它不可用。同样的算法换样本量：$n = 30 \Rightarrow \pm 16\%$；$n = 100 \Rightarrow \sqrt{(0.21/100)} = 0.046$，$\pm 9\%$；$n = 300 \Rightarrow \pm 5\%$。要半宽压到 $\pm 10\%$，反解 $n \geqslant 1.96^2 \times 0.21 \div 0.1^2 \approx 81$ 次。**所以“连续跑 100 次”不是勤奋，是把区间宽度买到够用**；而低于 80 次的成功率数字，本书建议一律不报（写了也没有分辨力）。

这条判据自己也有一个边界，而且必须写出来：上面用的是二项分布，它假设 100 次试验相互独立。而你在同一条流水线上连着跑 100 次，物体、光照、电池 SOC、关节温度都在缓慢漂移，**失败会成簇（第 38 章那条相关性讨论同样适用）**——所以 $\pm 9\%$ 是乐观的区间宽度，实际还要加上“这批试验只代表这一段工况”这一层折扣。诚实的写法是：报次数、报区间、报这批试验的时间跨度与温湿度。

里程碑 ①：一颗会听话的关节（第 1–10 周）

目标：把第 1 部十一章全部变成手上的东西。

两条路，选一条：

```
A（自研，推荐给做硬件的人）
   STM32G4 级 MCU + 三相半桥 + 磁编码器 + 一颗无框或小云台电机
   自己画板、自己写 FOC 全环

B（省时，推荐给做算法的人）
   买 2 个带 CAN/CAN-FD 协议的开源可改关节
   再自研一块“最小驱动板”做对照（哪怕只点亮一相）
```

必须交付的产物：

- ☐ 一块能出扭矩的驱动板（原理图 + PCB + 固件，全部开源在仓库里）

- ☐ **示波器截图合集**：20 kHz PWM、三相电流波形、启用的死区、以及**电流过零处的台阶**（第 13 章那个 48 mV 的效应，你亲手抓到的样子）
- ☐ **力矩-给定曲线**：用力传感器或弹簧加载，扫 0 → 满量程，报出误差百分比（这就是第 04 章 τ = k_t·i 的**你自己的**验证）
- ☐ **温升曲线**：连续满载 30 分钟，每 30 秒记一次壳温与电流（第 36 章那张图的实物版）
- ☐ 一篇技术长文：《从零做一颗机器人关节驱动器》，把上面四样写清楚

验收判据（必须自己量，不许估）：

项	目标	对应章节
力矩模式静态误差	⩽ 5% 满量程	04
PWM 频率与死区	实测值写进 README	08、13
采样同步	改变 ADC 触发角，展示电流噪声如何变化（**这条最能证明你懂**）	13
温升	有曲线、有降额点（如果有保护）	36

为什么这一项排第一：它是唯一一件“做完之后，你看所有厂商参数表的眼光都变了”的事。**而且它是四五个专业在一张板子上汇合的地方**——第 39 章说的那种复合能力，只能这样长出来。

为什么第一件作品是一颗会听话的关节，而不是一台会走路的机器人

这一章的顺序看起来反直觉：走路才是机器人的招牌，为什么先做一颗关节？

因为可测量的只有它。把“能不能验收”拆开问三句，答案就倒向关节。

一，可测量吗？ 关节只有一个待验量：输出扭矩对给定扭矩。它是一条曲线、一个误差百分比，量程与单位都明确（上一节那 250 mN·m 就是它的尺子）。而“会走路”的待验量是十来个的耦合：质心轨迹、足端触地时刻、单支撑时间、恢复步的落点散布、IMU 噪声、地面 μ——你连“这次失败该记在哪一栏”都要先发明一套口径。

二，可复现吗？ 关节的复现条件是室温、一块台架、同一给定序列，别人在你的仓库里可以重跑。走路的复现条件里有一块你搬不走的地面（第 23 章那条：同一块地在正午与阴天的相机读数能差几十倍）与一双会变的鞋（第 25 章那道摩擦不等式的分母）。

三，失败能归因吗？ 这才是决定性的一条。关节不听话，候选原因只有四类，而且每类都有本书给过的独立判据：

```
给定 5 N·m 只出 4 N·m？
  ① k_t 口径搞错（输出端 vs 转子端）       → 第 04 章：0.63895 × 40 = 25.56 vs 23.7，差 7.8%
  ② 满载时电压不够，反电势加铜损顶住工作点  → 第 08 章：12.78 + 40×0.04 = 14.38 V > 24 ÷ √3 = 13.86 V
  ③ Park 角度偏了，电流漏进 d 轴           → 第 10 章：偏 5° 扭矩只剩 cos5° = 99.6%，i_d 占 8.7%
  ④ 死区偷走电压，过零附近最疼             → 第 13 章：(100 ns ÷ 50 μs) × 24 V = 48 mV
```

四条路，每条一次代数，互斥，可判定。而“策略走两步就摔”的候选是：23 颗关节 × 上面四类，加状态估计、接触模型、延迟分布、仿真器差异（第 21、22 章那四类）——**上百个候选，而你一次实验只能观察到“摔了”这一个比特**。归因空间大小差两个数量级，这才是“先做关节”的真正理由：**它把失败变成信息**。

补一条量的对比：一颗关节的电流环周期是 50–125 μs 量级（第 00 章那个 50 μs 缝隙、ODrive 官方口径 8 kHz [查]），一次力矩阶跃响应要多久才有结论，第 09 章那个 $\tau_m = J \cdot R \div k_t^2$ **不管这件事**——它是**转速**响应的时间常数，力矩（也就是电流）响应在它的上游，由 $\tau_e = L/R$ 与电流环带宽决定。第 09 章本轮把 τ_m 也算清了：裸转子只有 0.039 ms，毫

秒级完全来自负载惯量，所以拿它当力矩阶跃的答案是错上加错 [算]。正确的算法用第 10 章那条带宽判据：ω_c ≤ 0.35 ÷ T_lag 在 20 kHz 开关与 25 µs 计算下约 1.1 kHz ⇒ 一阶等效整定时间 4 ÷ ω_c = 4 ÷ 7,000 ≈ **0.58 ms（注意除的是 rad/s，不是 Hz**：拿 1,114 Hz 去除会得到 3.6 ms，正好差 2π），再叠上死区补偿、采样与总线节拍，验收时按 **5–20 ms** 记（这一段是经验包络 [推]，不是上面那条式子导出来的——式子只给 0.58 ms）——**若你的判据是 ±2% 稳态误差，预算就得写 20 ms**，正好占一步支撑相的 10%；而一步的支撑相是 200 ms 量级（第 17 章），一轮跌倒复位是几分钟。**同样一个下午，你在关节上能做完的实验次数比在整机上多一到两个数量级**——迭代速度这件事，在自学阶段比任何硬件配置都重要。

里程碑 ②：一条有证据的实时链路（第 8–14 周，可与 ① 并行）

目标：把第 12、27 章的带宽与延迟算术，变成你自己测出来的数。

做什么：2–4 个节点，跑 1 kHz 周期，量端到端（端到端 = 从像素一路训到关节指令，中间不手工分模块）。

- ☐ **时延直方图**（不是平均值！）：从发出指令到收到状态，采 10 万次，画分布
- ☐ **周期抖动**：目标 1 kHz 下 < 50 µs；把你的实现测出来的数写出来，超了也写
- ☐ **丢帧与恢复行为**：故意拔一下线，看系统怎么表现（第 28 章的 best_effort 语义在这里现形）
- ☐ 一份《实时性测试报告》，说明测法、设备、以及**哪些环节你没能测准**

这一项最容易被跳过，而它最值钱。因为绝大多数人会写“我做了 1 kHz 控制”，而**几乎没有人能给出一张时延直方图**。第 00 章说过：决定成败的是最坏情况延迟与抖动，不是平均。**一张直方图就是你有这个判断力的证**

明。

再加一条能让你看懂自己那张直方图的算术：**如果你的总线是轮询式的，抖动里最大的一块不是噪声，是排队的顺序**。单节点往返一帧下行 17 B + 一帧上行 16 B = 33 B，按 8N1 是 330 bit，在 4 Mbps（位时间 250 ns）上就是 330 × 250 ns = 82.5 µs [算]；一条干上轮询 8 台 ⇒ 8 × 82.5 µs = 660 µs。**而 1 kHz 的周期只有 1000 µs——第一台与第八台的采样时刻天生差掉 66% 个周期（660 ÷ 1000）**。这就是第 27 章那句“轮询给不了同步”的定量版本，也是为什么你的直方图会长成 8 个峰，而不是一团。

里程碑 ③：一次恒力按压（第 12–18 周）

目标：把第 18 章的三道天花板变成手感。

做什么：用关节顶住一个弹簧或称，命令“5 牛米”，保持 60 秒。

- ☐ 力跟踪曲线：给定 vs 实测，报出稳态误差与纹波
- ☐ **过零台阶实验**：让目标力在 +2 → −2 N 之间来回穿越，抓下力曲线。**你会看到第 13 章那个死区台阶**——然后你可以打开/关闭补偿，再抓一次，把两张图放一起
- ☐ **刚度扫描**：kp 从小到大连扫，记录每个 kp 下的噪声与是否自激。**你会亲手找到第 18 章那道“ω_n 撞上电流环带宽”的边界**
- ☐ 一张图说明：你的关节在什么条件下开始抖

这一项是“接触类任务”的入场券。第 38 章说过行业公认接触与灵巧是短板，而短板的根子在**没有人真正调过力控**。做过一次恒力按压并看到过零台阶的人，对“力控”这两个字的理解与只读过论文的人不同。

验收也写成一条式子：稳态误差 = $|\hat{\tau} - \tau_cmd| \div FS \times 100\%$，其中 $\hat{\tau}$ 用外链的测力件读，FS 用你这颗关节的满量程（上一节的 250 mN·m 就是这么来的）。**并且这一项必须分低速段与额定点两档报**，因为死区那 48 mV 在两个档位上的占比完全不同：第 13 章算过低速时可用总电压只有 1

V 量级，48 mV ÷ 1 V ≈ 5%——**你的判据阈值 5%，恰好就是死区在低速段的原生误差量级**。这不是巧合，这是为什么“恒力过零”是力控里最难的一格：你在用一个 5% 的判据去考一个本身带 5% 系统性偏差的环节。补偿能把它压下去，但补偿的符号在过零附近由噪声决定（第 13 章那条自指问题），所以这一段允许你的结论是“调不到位”，只要求你把两张曲线都交出来。

里程碑 ④：让一台仿真里的机器站起来（第 16–26 周）

目标：走完第 21 章那条工序，包括 Sim2Sim。

- ☐ 在 Isaac Lab 或 MuJoCo 里训一个能走通的步态（G1/H1 的官方 URDF/MJCF 是现成的）
- ☐ **抄一遍官方配置的三行数**：sim.dt、decimation、action_scale，并算出策略频率
- ☐ **Sim2Sim**：在另一个仿真器里跑通，把两边的足端力曲线叠在一张图上（这就是第 22 章说的“接触求解器差异”的可视化）
- ☐ **域随机化消融**：把随机化关掉再训一次，比较两者在扰动测试下的表现。**这一张对比图比任何论文都更能证明你懂 RL 运控**

为什么这一项对做硬件的人也重要：它让你有和算法同事对话的能力。第 39 章说过，最缺的是能跨层的人。

那三行数怎么抄成判据：策略频率 = 1 ÷ (sim.dt × decimation) = 1 ÷ (0.005 × 4) = 50 Hz [厂]（官方 rl_lab 配置给的正是这两个数）。**再往下算一步才是这件作品的价值**：50 Hz 意味着两次指令之间隔 20 ms，而一颗关节的电流环周期是 50–125 μs 量级，两者相差 20 ms ÷ 100 μs = 200 倍——**这 200 倍的空隙全部由关节内的 kp/kd 与你自己的补偿逻辑填**。第 18 章那道 ω_n = √(kp ÷ armature) = 62.8 rad/s = 10 Hz 的天花板就住在这条空隙里：官方 mimic 配置的 kp 是按它反推出来的，不是手调的

[算]。你的消融实验（把随机化关掉再训一次）之所以有意义，就是因为这条空隙里没有任何一路监督信号告诉你真机的动力学已经变了（第 38 章那条漂移链）。

里程碑 ⑤：一份自己的数据集（第 20–36 周）

目标：理解第 33 章那些数字为什么贵。

- ☐ 用主从臂（SO-101 一类，从臂约 ¥682、主从套约 ¥1,343 [查]）或手持采集器，采 **300–500 条演示**
- ☐ 写一个质检脚本：统计时长分布、失败样本占比、**时间同步误差**（第 23 章那条在这里落地）
- ☐ 训一个基线策略（ACT 或 Diffusion Policy），报出**在未见条件下**的成功率
- ☐ 按 LeRobot 格式开源，附评测协议

这一项的关键不是模型，是那三样“没人做的脏活”：质检脚本、同步误差统计、评测协议。第 33 章说过，一个数据集宣传如果缺这几项，它报的是“储量”而不是“可采储量”。**你自己做过一次，就永远会先看这几项**。

质检脚本里最应该出数的是时间同步，而判据不是“我们同步过”，是一张直方图：把每一路传感器与关节状态的时间戳对齐到同一个 `tick` 时基（第 28 章说过宇树的 `wireless_remote` 与机体状态是同帧上报），统计残差的中位数与 P99。为什么值得为它单独出一张图：第 23 章已经算过量级——1 m/s 的相对运动加 20 ms 不同步，会产生约 13.4 像素的伪视差，而精度假设是 0.5 像素，**也就是 27 倍的测距噪声** [算]。同理，做主从臂时那 5 美元级别的相机差异不是问题，20 ms 的错位才是。（**价格口径两条都保留：SO-101 官方页面不给价，公开的是上游 SO-ARM100 的 BOM 求和——主从 US$229.88、从臂 US$121.94 [查]；而本书前面按渠道价写的从臂约 ¥682、主从套约 ¥1,343 是分销商口径 [查]。两者不是同一个东西：一个是物料成本、一个是售价，本书不合并。**）

里程碑 ⑥：一次诚实的评测（第 30–48 周）

目标：把第 38 章那五问变成你自己的表格。

- ☐ 选一个任务，定义清楚“成功”（含“抓歪了但捡起来”算不算）
- ☐ 连续跑 100 次，报 **p^N 那一栏**（不是单次成功率！）
- ☐ 记录每一次失败的**类型**（分布外 / 无记忆重犯 / 硬件漂移），做一张饼图
- ☐ 连续跑 2 小时，看有没有第 36 章那种热降额、第 35 章那种电压塌陷

这一项最不像“成果”，而它最像工程师。因为它的产出是**一张诚实的表**，而行业最缺的正是诚实的表（第 33、38 章两次说过）。

三个数必须一起报，缺一个都不算完成：**单次成功率 p 与它的 95% 区间**（100 次 ⇒ ±9%，见上一节那条 $\sigma = \sqrt{p(1-p) \div n}$）、**连续全对率 p^N**（第 38 章那条：0.99^100 = 36.6%，所以“单步 99%、一百步全对”只有三分之一，这一栏比前一栏更能说明它能不能上生产线）、**以及失败类型那张饼图里“无记忆重犯”占的比例**。第三栏最有用，因为它直接告诉你缺的是模型还是记忆：**同一个失败重复出现，说明策略里没有“我已经失败过一次”这个状态**（第 31 章那条架构判据），这不是多训练能解决的；而失败类型分散、每次都不重样，才是数据覆盖的问题（第 33 章）。这三栏合起来，就是第 38 章那五问里 ②③⑤ 三条的最小可测版本。

十二个月怎么排

```
第 1–3 月   ① 关节 + ② 实时链路（并行）          ← 这两项决定你能不能进 ⑤⑥ 层
第 3–5 月   ③ 恒力按压 + ④ Sim2Sim               ← 接触与仿真，两条腿
第 5–9 月   ⑤ 数据集                              ← 最长、最容易拖，尽早开始
第 9–12 月  ⑥ 诚实评测 + 把六项整理成一份作品集
```

每一阶段结束，对外发一次东西（一篇长文、一个仓库、一段视频）。理由不是刷存在感，是**逼自己把过程写成能被别人复现的形式**——那是第 39 章说的“不可发表的层”唯一的可见方式。

三条纪律，比路线更重要

一、每个数自己量，不许估。 本书第 1 部有十几个 [算]，每一个都跑在脚本里；你的六个里程碑，每一个都必须有你自己测出来的数。**估来的数会让你在面试里被问倒一次，而量过的数不会。**

二、失败也要交付。 抖动没压下去、力控过零台阶没补偿好、策略在未见条件下只有 40%——**把这些写出来，附上你试过的三条路和为什么都不行。** 这比一个成功演示更能证明你能干活。第 13 章那句话在这里适用：**“这个调不出来”是一个合法结论。**

三、每个产物都要能被别人复现。 仓库里有 BOM、接线图、固件版本、测试脚本、原始数据。**别人复现不出来的东西，不算你的能力，算你的运气。**

这三条纪律也有它们的代价，写清楚免得读者按最理想的那一档规划：自己量每一个数，意味着你会在某些项上花掉比“做出效果”多三倍的时间，而外行看不出区别——**这份工作的报酬是延迟支付的，它在下一次你判断别人的方案时才兑现**；失败也要交付，意味着你的作品集里有一些不好看的东西，它会筛掉一些只看演示的读者，而留下的是真需要你的人；每个产物都可复现，意味着你要写文档、整理原始数据、钉死版本——这是最不被喜欢的一种劳动，也恰好是第 39 章说的那件“不能发表但决定成败”的事在个人层面的形式。

还有一条边界要说明：**这六件作品不构成求职保证。** 本书给的判据只能证明你做过、能测、能归因，它不能替代一台真机、一段产线、或一年的运维。反过来说也成立：**没有这些判据，你手上的演示在任何一次追问下都会散架。**

类比，以及它在哪儿破

类比：这十二个月像考一张实操证——不是笔试，是“接一根线给我看”。

这个类比在哪里破：证书有考官，而这条路没有。**没有人为你的作品集打分，只有下一个项目会不会因为你做过而找你**。所以唯一的验收标准是：**你自己能不能在三个月后，看着那张时延直方图，说出它是哪来的、为什么长这样、以及要压下去该动哪一层**。

如果答得上来，第 1 部那十一章就没有白读。

一个可检验的说法：机器人行业里 ⑤⑥ 层（关节伺服、功率与热）的成果不可发表，因此其可见性只能靠“自带测量数的可复现产物”来建立；一条十二个月路线的六个里程碑（关节、实时链路、恒力、Sim2Sim、数据集、诚实评测）分别对应第 04–15、12/27、18、21/22、33、38 章，而每一个验收项都必须能写成一次代数：力矩误差 5% FS 在官方 5.0 N·m 档上是 250 mN·m、在 139 N·m 档上是 6.95 N·m（相差 27.8 倍，所以判据要同时给绝对误差与 FS 口径）；而总线的量化从来不是瓶颈——IM6014 的刻度折到关节侧是 4.9 mN·m，250 mN·m 还剩 51 格可用（上一版写成“1 N·m 刻度比 250 mN·m 粗 4 倍”，是把“2560 刻度 = 1 N·m”这句读成了“刻度 = 1 N·m”，方向正好反了）；瓶颈在模拟前端：M8010 自己那笔死区偷电压按官方 k_t = 0.63895 推到 0.77 N·m，是它自己 5% FS（1.19 N·m）的 64%；周期抖动 50 μs 的物理含义是 1 kHz 周期的 5%（相位 18°）、200 个 250 ns 位时间（约一整帧 17 B），而轮询 8 台的采样时刻差是 660 μs = 周期的 66%；未见条件成功率 70% 在 n = 10 时的 95% 区间宽达 ±28%（42%–98%），要压到 ±10% 需要约 81 次、n = 100 时才有 ±9%，且这批试验因失败成簇而仍是乐观估计；第一件作品之所以必须是一颗关节而不是一台会走路的机器，是因为它的失败候选只有四条（k_t 口径、电压包络、Park 角度、死区）而整机是上百条——**归因空间差两个数量级，这才叫可测量**。预算、时间、学校资源因人而异，本书给路线不给承诺。

证据等级：所有器件价格（SO-101 从臂约 ¥682 / 主从套约 ¥1,343 为分销商口径 [查]，而 SO-101 官方不给价、公开的是上游 BOM 求和主从 US$229.88 / 从臂 US$121.94 [查]——**两个口径并列，不取平均也不互相替换**；STM32G431 ¥21.96、LAN9252 ¥87 来自厂商店铺与分销商 [查]）；官方 effort 5.0 N·m（W4010-25）与 139 N·m（N7520-22.5）、IM6014 扭矩量化 2560 刻度 = 1 N·m（LSB ≈ 0.39 mN·m、满量程 12.8 N·m）、M8010 帧长 17 B / 16 B 与总线 4 Mbps、sim.dt 0.005 × decimation 4 = 50 Hz、mimic 配置 kp 99.098 与 armature 0.02510、N7520-22.5 的 Fs = 2.4 N·m [厂]（effort 有两个官方口径：rl_lab 配置给 139 N·m，而 `unitree_ros` 里两个 G1 URDF 对髋滚分别写 139 与 88，本书并列保留并指明取的是 rl_lab 那一档）；ODrive 环路周期 125 µs（8 kHz）[查]；27.8 倍、51 / 348 个刻度、12.7 倍、0.77 N·m、64%、3.1 倍、155 倍、0.58 ms（= 4 ÷ 7,000 rad/s）、5–20 ms、5%、18°、200 个位时间、42.5 µs、82.5 µs、660 µs、66%、σ = 0.145 与 ±28%、±9%、81 次、36.6%、48 mV ÷ 1 V ≈ 5%、20 ms ÷ 100 µs = 200 倍、ω_n = 62.8 rad/s = 10.0 Hz 全部是 [算]（`foc_check.py` 的 deadtime_error / resolution_vs_backlash / natural_frequency_check / bus_budget / compound_success，其余为本章正文写出的除法与开方，读者可用计算器逐步复核；其中“低速段可用电压 1 V 量级”沿用第 13 章的本书假设 [未]）；各里程碑的验收判据是本书根据第 1–2 部的物理推导给出的建议 [算][行]；**时间预算是量级估计，不是承诺**，本书无法为任何个人的进度负责，也不保证任何求职结果；“每一阶段对外发一次”是方法论建议 [行]；**宇树未公布任何关节的抱闸、额定扭矩、寿命与失效率数据 [未]，因此本书的个人验收判据不能替代厂商该公开而没有的那张表**。

可核对清单：

- ☐ 在上面六个里程碑里，圈出你**现在**就能开始的那一个（不是第一个，是资源允许的那个）

- ☐ 为它写一句验收判据，格式必须是“我测出 ___ = ___ （方法 ___）”
- ☐ 自己算一次 0.05 × 5.0 与 0.05 × 139，体会同一条“5% FS”在两档关节上的 27.8 倍差距
- ☐ 自己开一次方：√(0.7 × 0.3 ÷ 10) = 0.145，再乘 1.96，确认 10 次试验报不出 70%
- ☐ 用 330 bit × 250 ns 复算 82.5 µs，再乘 8 看它吃掉 1 kHz 周期的多少
- ☐ 检查你的机器、预算、可支配时间，判断 ① 该走 A 路还是 B 路
- ☐ 把这一章的十二个月排进你的日历，然后**在下一个月末回来看这一页，勾掉你已经做的**

下一章（附录 A）：本书的符号、术语与单位约定——包括那些“中文资料里各家写法不同、你必须知道自己在用哪一套”的地方。

附录 A

符号、术语与单位

> 本附录是全书的“口径声明”。凡正文出现符号、单位或术语与本表不一致，以本表为准，并请把它当作勘误提交。



图 A-1　王兴兴，2012 年天津夏季达沃斯｜Photo by China News Service via Wikimedia Commons, CC BY 3.0

A.1 符号

符号	含义	单位	首次出现	备注
τ	扭矩	N·m	04	不写 T，避免与周期/温度冲突
i, i_q, i_d	电流 / q 轴电流 / d 轴电流	A	04, 10	i 不带下标时指相电流幅值
k_t	扭矩常数	N·m/A	04	**必须标明输出端还是转子端**，见 A.4
k_e	反电势常数	V·s/rad	08	**不等于 k_t**。本书用幅值不变约定，dq 轴反电势幅值 = p·λ_m·ω = (2/3)·k_t·ω；而厂商手册里“线电压 RMS ÷ krpm”那个口径又是另一个数。两种口径与 √3、2/3 这几个系数混用，足以把电压包络算反方向（第 08 章正文就绕开了它）
λ_m	永磁体磁链	Wb（= V·s）	05	
p	极对数	无量纲	05	不是“功率”，功率写 P
L_d, L_q	d/q 轴电感	H	05, 11	IPM 有 L_d < L_q
θ, θ_e	机械角 / 电角度	rad	05, 10	θ_e = p × θ_m
V_bus	母线电压	V	08	

d	占空比	无量纲	08	与“视差”的 d 分开，视差只在 23 章出现并就地定义
f_sw	开关（载波）频率	Hz	08, 09	
T_pwm	开关周期	s	08	T_pwm = 1/f_sw
L, R	绕组电感、相电阻	H, Ω	09	
τ_e, τ_m	电、机械时间常数	s	09, 12	τ_e = L/R；τ_m = J・R/k_t²
τ（**裸用，无下标**）	倒立摆时间常数 √(l/g)；第 35 章一处指 RC 常数	s	00, 16, 17, 35, 40	本书的区分办法：带 e/m 下标＝电/机械常数；运控与平衡章节里裸 τ＝倒立摆时间常数；第 35 章那一处写作 τ = R・C 已就地标明
1/τ	倒立摆的 e-folding 率（发散率），**不是角速度也不是频率**	**s^{-1}**	00, 24	G1 髋高 0.66 m ⇒ τ = 0.259 s ⇒ 1/τ = 3.85 s^{-1} ≈ 0.61 Hz（折 Hz 要 ÷2π）
λ（导程角）、ρ（摩擦角）	第 37 章丝杠自锁判据专用	°	37	与磁链 λ_m 撞形：**这两个符号无下标且只在第 37 章的机械一节出现**；同节的 η_正 / η_反 是正/反行程效率，与热效率无关
J	转动惯量	kg・m^2	09, 18	必须标明折算到哪一侧

N	减速比	无量纲	14	不写 i_gear，避免与电流 i 混
η	效率	无量纲	14, 20	
ω	角速度	rad/s	08	全书不用 rpm，除非引用厂商原值
q, dq, ddq	关节位置、速度、加速度	rad, rad/s, rad/s²	12	与 DDS 字段同名，故意如此
kp, kd	刚度、阻尼增益	N·m/rad, N·m·s/rad	12	官方字段名是 Stiffness / Damping
l	倒立摆等效长度	m	00, 16, 17	本书取髋高
g	重力加速度	m/s²	16	取 9.80665
F_z, μ	法向力、摩擦系数	N, 无量纲	17	
CoT	代谢代价系数	**无量纲**	20	见 A.3 的常见错误
P	功率	W	15, 20	
E	能量	J 或 Wh	20, 35	1 Wh = 3600 J
R_th	热阻	K/W 或 °C/W	36	两者数值相同
H	动作块长度	步	30, 31	
Δt	每步执行时长	s	31	
T_infer	一次推理时间	s	31	
p, N（概率与次数）	单次成功率与动作次数	无量纲, 个	38	与减速比 N 用上下文区分

A.2 单位与书写约定

一，**全程 SI**，但保留三处行业习惯：电池用 Ah 与 Wh、算力用 TOPS/TFLOPS、相机分辨率用像素。凡保留处，正文第一次出现时给出换算。

二，**N·m 不写成 J**。扭矩与能量量纲相同但物理不同，混写会导致“扭矩 × 角度 = 能量”这类式子被误读。

三，**kg 是质量**。涉及“体重”时本书仍写 kg，但凡是力都换算成 N（例：35 kg → 343 N）。本书曾因此犯过一次错（把力与质量直接相减），记录在 `bible/do-not-touch.md`。

四，**转速一律 rad/s**。引用厂商 rpm 时保留原值并加括注。

五，**百分数与“倍”不混用**。“涨 58%”与“是原来的 1.58 倍”二选一，同一句里不并存。

六，**数量级词表**：几十（20–90）、几百（100–900）、千级、万级。不用“若干”“一些”修饰可数的量。

A.3 本书踩过的四个单位坑（写出来，别重复踩）

坑	错法	正解	谁抓到的
CoT 有量纲	把 CoT 写成 J/(kg·m) 之类	CoT = E/(m·g·d)，量纲为 1	自检脚本
力与质量相减	payload = F/k − m·g	payload_kg = F/(k·g) − m	自检脚本
功率与扭矩混	P = CoT·m·g	P = CoT·m·g·v（缺速度量纲就不对）	自检脚本
峰值当额定	用 23.7 N·m 做持续设计	峰值与最高转速互斥（第 08 章），且额定值未公布	第 08、36 章论证

A.4 三个必须标明“口径”的量

这是本书最高频的错误来源。**任何一处出现这三个量，都必须写明口径，否则视为未完成**。

一、**扭矩常数 k_t**：输出端 or 转子端？判据见第 04 章（k_t × 峰值电流 ≈ 峰值扭矩 ⇒ 输出端）。

二、**扭矩**：峰值 or 额定？宇树全系只公布峰值 [未]。仿真模型里的 effort 是第三种东西（约束值），见第 02 章冲突表。

三、**频率**：总线通信频率、控制环路频率、PWM 载波频率是三件事。判据见第 12 章的带宽算术。

A.5 变换约定声明

全书统一使用**幅值不变（amplitude-invariant）Clarke/Park 变换**：

```
i_α = (2/3)(i_a − ½i_b − ½i_c)
i_β = (2/3)(√3/2)(i_b − i_c)

⇒ 三相对称正弦时 |(i_α, i_β)| = 相电流峰值
⇒ 扭矩式 τ = (3/2)·p·λ_m·i_q，系数为 3/2
```

若读者参考的资料使用功率不变变换（系数 $\sqrt{(2/3)}$），则其扭矩式系数为 3/2 或 $\sqrt{3}/2$ 不等，**系数不同不代表有人算错，只代表约定不同**。判据：**同一份资料里，变换系数与扭矩系数必须成对出现**（第 07、10 章各给了一次自检）。

A.6 术语对照（一词一义表）

本书严格区分下列常被混用的词。

本书用法	不用它指	易混词
人形 / 双足	四足、轮式	“机器人”（本书凡说“机器人”指全类）
关节	电机、执行器	三者层次不同：电机 ⊂ 执行器 ⊂ 关节
电机	关节模组	模组 = 电机 + 减速器 + 编码器 + 驱动板
减速比 N	传动效率	效率写 η
齿隙	回差、背隙（同义，首次出现时并列给出）	与“弹性变形”区分：齿隙是空程，弹性是可恢复形变
刚度 kp	硬度、强度	硬度是材料属性，强度是失效界限
柔顺 / 顺从	软（口语）	统一写“低刚度”

扭矩密度	功率密度	前者 N·m/kg，后者 W/kg
额定	持续	本书用“额定（持续）”，首次出现处并列
峰值	最大	统一写“峰值”，并注明是角点值
强化学习	RL 策略、神经网络控制	统一写“强化学习（RL）”
模仿学习	行为克隆（是其最简形式）	首次出现处说明包含关系
世界模型	VLA、动作专家	四用法见第 34 章，禁止混用
策略	模型、网络	“策略”专指 观测→动作 的映射
本体感知	传感器（泛称）	专指关节与 IMU 类自身状态测量
仿真	数字孪生	本书不用“数字孪生”
落地（部署义）	落地（触地义）	触地一律写“触地”或“足端触地”，部署一律写“部署”
实时	快	实时 = 最坏情况有界，不是平均快
确定性	稳定	确定性 = 时间有界；稳定 = 不发散
损耗（总）	铜损	**总损耗 = 铜损 + 铁损 + 驱动板（导通与开关） + 减速器摩擦**；第 36 章那颗膝的 55 W 是总损耗，第 13 章的 36 W 才是铜损，两者差约 1.5 倍

A.7 引用与标注

七级来源标记：[厂] 官方页面或官方仓库文件；[拆] 第三方拆解、券商研报、逆向工程；[行] 教科书与行业通识（**不是字树事实**）；[算] 本书推算（附录 B 有脚本）；[未] 查不到。另用 [查] 表示“来自可链接的公开资料

（论文、文档、报道），本书已读到原文但未独立复现”。另用 [推] 表示**本书自己的推断，没有一手来源支撑**——凡用它处都明写这是推断，不许把它当成已核实的事实；与第 00 章图例的七个标记一一对应。

引用格式：正文用上标式标记，出处集中在附录 E，逐条给 URL 与抓取日期。

语录块：每章一段王兴兴语录，格式固定为“引文 —— 出处（媒体/书名）— 日期（类型：问答实录/演讲/书面）”。**引号内必须是逐字引文**；若只找到转述，本书写“大意，非逐字引文”并单独标注（这一条来自一次实际犯错，见 `bible/do-not-touch.md` 第 34 条）。

这条规则的实际达成度：41 个正文章里 **41 章**有题引，**0 章留空**——其中 32 章是王兴兴本人（逐字可回读），9 章按作者的决定改用埃隆·马斯克的公开言论。改用马斯克的是第 05、07、09、10、11、12、18、24、28 这九章——磁场的垂直分量、三相旋转磁场与谐波、斩波与“平均”为什么合法、Clarke 与 Park、i_q 管扭矩 i_d 管磁场、三个环的带宽递减律、刚度由三道天花板决定、IMU 与状态估计、中间件与“去 ROS 化”。换人的原因和留空的原因是同一件事：检索过问答实录、演讲与书面访谈三类材料，王兴兴本人在这些主题上**没有留下可回读的逐字技术表述**——他谈的是路线与成本判断，不是磁场定向、坐标变换、死区或零偏。硬套只会得到一句不属于该章的话。**说话人变了，本书就把它写在明处**：这九块的出处行带 `speaker` 字段，排印时说话人显示为马斯克，不冒充王兴兴。来源层级也分两档——其中 2 条出自特斯拉自己发布的股东更新 PDF（本书用 PyMuPDF 抽全文逐字核对，记 [查]）；另外 7 条是第三方报道里的逐字引句，**本书未核对录音或视频原文**，出处行的 `类型` 里明写这一点。愿景类的话（“史上最大产品”“手术级”）不参数、不当规格用，这一点与王兴兴那 32 条同规矩。

其中 **5 条**（第 17、19、20、27、37 章）是演讲整理稿、书面化微调或记者转述，本书在出处行的 `类型` 里明写，不冒充逐字。

法律口径：语录引用依《著作权法》第 24 条适当引用；本书不引用任何未公开信息；不对上市公司做业绩判断；全书不构成采购或投资建议。

A.8 图的达成度（和 A.7 同一条规矩：缺就写明）

图台账 `bible/images.md` 计划 **36 幅**，本印本里是 **33 幅**，全部为本书自绘的 SVG——推导图、结构图、时序与账目图。差的 3 幅不是“还没画”，而是**只有作者本人能提供**：需要一台真机与一颗可以拆开摆平的样机。它们是

整机全身定妆照	读者要确认“这本书讲的机器长什么样”
腿部关节逐个特写	髋 / 膝 / 踝各一颗，摆成一排
一颗 GO-M8010-6 的拆机照	转子、定子、编码器、减速级同框

为什么不用厂商的图：出版社必须能证明版权。新闻页上的照片与视频是对方的资产，本书按“**官方素材 0 张**”处理。代价就是这三格——“真机长什么样”这件事，除了拍没有第二种来路。三处在台账里各带一条 `status: 待拍`；交稿时应连同这份台账一起给编辑，让空槽是**登记过的空槽**，而不是漏掉的图。

这也顺带说明本书的图在讲什么：33 幅自绘图里没有一张是装饰。每一幅都对应正文里一条能按计算的式子或一条口径声明，图上标的每个数都能在同一个文件的正文里找到出处；找不到出处的那一类（例如“某关节内部到底几级减速”），本书画成问号而不是画成曲线。

附录 B

全书可复算的数（含脚本与输入表）

本附录的**四个数据块**（B.2、B.3 的两块誊抄、B.4）由 `python tools/gen_appendix_b.py` 从三个工具的 运行输出重写，**不要手改**——手改会在下一次重建时被无声覆盖。要改数，改 `code/tools/` 下的输入表或 函数，然后重跑生成器。这样做的理由见本附录末尾。

B.1、B.3 的主算例表与用途边界说明、B.5、B.6 的结论段是作者写的，生成器不碰它们。它跑一次会校验 小节标题、表格行与这几段的开头行没有被动过，**但它不逐字看守全部作者正文**——B.6 抬头那一段是 机器重写的，落在它后面的作者段落只受“不许消失”这一条约束。所以改稿请自己 diff，别把这道检查 当保险。`tools/appendix_drift.py` 另外会把附录里承诺过的每一个计数与工具的实际输出再比一遍。

生成时间：2026-10-09

生成命令：`python code/tools/foc_check.py`、`python code/tools/arithmetic_check.py --demo`、`python tools/check_kg.py`

B.1 为什么这张表必须由脚本生成

本书的写作纪律是“每个 [算] 都配一个读者能自己跑的脚本”。但脚本一旦与正文分开维护就会漂移—— 本书已经发生过三次：正文写 0.12 V 而脚本算出 48 mV、正文写 21.6 倍而官方帧长给出 5.9 倍、 正文写“约 190 倍”而脚本给 187。**三次都是人改了一处没改另一处**。

所以本附录里的**数字与运行结果**是跑出来的，不是誊出来的。正文里凡标 [算] 的数，都应当能在下面 找到对应行。找不到，就是正文与工具脱节，属于必须修的缺陷。

B.2 第 1 部（一颗关节）与第 2–5 部的推算链

```
=== selfcheck ===
  ok   clarke@0° alpha: got 10 want 10
  ok   clarke@0° beta: got 0 want 0
  ok   park on q-axis d: got 0 want 0
  ok   park on q-axis q: got 10 want 10
  ok   rotating_field amplitude invariant over θ
  ok   08 反电势幅值 (2/3)·k_t·ω: got 12.779 want 12.78
  ok   08 满载所需母线 √3·(E+I·R): got 24.9052 want 24.91
  ok   08 反电势（旧系数 k_t·ω 是错的）: 错答案 19.17 必须不被接受
  ok   08 满载母线（漏掉 I·R 是错的）: 错答案 24.91 必须不被接受
  ok   svpwm gain: got 1.1547 want 1.1547
  ok   deadtime fraction: got 0.002 want 0.002
  ok   15bit resolution deg: got 0.0109863 want 0.0109863
  ok   pendulum tau 0.66m: got 0.259425 want 0.259425
  ok   compound 0.99^100: got 0.366032 want 0.366032
  ok   compound 0.99^500: got 0.00657048 want 0.0065705
  ok   compound 0.95^100: got 0.00592053 want 0.0059205
  ok   compound 0.90^50: got 0.00515378 want 0.0051538
  ok   compound 0.99^28800 量级: got 125.706 want 125.706
  ok   8h 90% 所需单步失败率 q: got 3.65834e-06 want 3.658e-06


=== 正文数字（每条都是一次代数）===


[04 τ=k_t·i 口径]
k_t·I = 0.63895 × 40 = 25.56 N·m；官方峰值 23.7 N·m，差 +7.8% ⇒ 手册常数是**输出端**口径。
折算到转子：k_t = 0.63895 ÷ 6.33 = 0.10094 N·m/A。
  → PASS

[08 电压包络（角点互斥）]
转子角速度 ω = 30 × 6.33 = 189.9 rad/s
反电势幅值 E = (2/3)·k_t·ω = (2/3) × 0.10094 × 189.9 = 12.78 V （幅值不变约定，见第 05 章）
24 V 母线在 SVPWM 线性区能给相电压幅值 = V_bus/√3 = 13.86 V
空载：12.78 V < 13.86 V ⇒ 最高转速空载拿得到（用掉母线能力的 92%）
满载：E + I·R = 12.78 + 40 × 0.040 = 14.38 V > 13.86 V ⇒ 拿不到（R 是本书假设 40 mΩ，官方未公布 [未]）
至少需要母线：空载 √3·E = 22.1 V；满载 √3·(E+I·R) = 24.9 V
```

结论：**峰值扭矩 23.7 N·m 与最高转速 30 rad/s 是两个角点，不可能同时达到**——但拦路的是铜损压降，不是空载反电势。
  → PASS

[08 SPWM vs SVPWM]
SPWM 线性区相电压峰值 = V_bus/2 = 12.00 V；SVPWM = V_bus/√3 = 13.86 V；增益 1.1547 ⇒ 多 15.5%。
  → PASS

[10 角度误差 5°]
角度误差 5.0° ⇒ 扭矩只保留 cos δ = 0.9962（掉 0.38%），同时 |i_d|/|i| = sin δ = 0.0872（8.7% 的电流在'白烧'）。维持同一扭矩要把电流幅值放大 1/cos δ：电流有效值只多 0.38%，而铜损按幅值平方算，多耗 0.77%。
  → PASS

[07 三相合成幅值]
θ=37°：i=(+7.986,+1.219,-9.205) A ⇒ (α,β)=(+7.986,+6.018)，|合成|=10.0000 A，相电流峰值=10.0 A ⇒ 相等。
  → PASS

[13 死区 100ns@20kHz]
死区 100 ns ÷ 周期 50.0 µs = 0.2000% ⇒ 电压误差 ΔV = 0.2000% × 24 V = 0.048 V。
  → PASS

[14 分辨率 vs 齿隙]
15 bit 单圈 = 32768 格 ⇒ 电机端 0.01099°；经 20.58:1 折算到输出端 0.000534°。
谐波减速器背隙官方口径 6 弧分 = 0.100° ⇒ 齿隙是编码器（折算后）分辨率的 187 倍。**再精细的编码器也救不了减速器的空程。**
  → PASS

[14 准直驱扭矩换算]
k_t(转子)=0.10094，峰值 40 A ⇒ 转子扭矩 4.04 N·m；×6.33:1 = 25.6 N·m（理想），按 85% 效率 = 21.7 N·m。
  → PASS

[15 扭矩密度与平方立方]
H1 膝 360 N·m ÷ 189 N·m/kg = 1.90 kg/关节单元（官方密度口径），与 G1 拆解主承载关节 ≈1.1 kg 同数量级 ✓
平方-立方：G1→H2 线度比 1.379 ⇒ 预测扭矩需求 ×3.61，官方档位实际 ×3.0（90→360 则 ×4.0）；预测质量 ×2.62，实测 ×2.00。
  → PASS

[27 总线带宽 M8010]

```
GO-M8010-6: 15 台 × 6000 Hz × 下行 17 B = 12.24 Mbps; × 上行 16 B = 11.52
Mbps; 合计 23.76 Mbps
官方总线 4 Mbps ⇒ 需要 5.9 倍带宽。
反过来: 4 Mbps 在 15 台全双工下最多支撑 1010 Hz
     — 与官方 Z1 页标的 Control Frequency 1KHz 撞在同一个数量级。
所以官方的 6000 Hz 只能是单台或被广播帧摊薄后的指令刷新率，不是 15 台同时
6000 Hz 全量交换。
  → PASS

[27 总线带宽 IM6014]
IM6014: 15 台 × 10000 Hz × 下行 20 B = 24.00 Mbps; × 上行 26 B = 31.20
Mbps; 合计 55.20 Mbps
官方总线 6 Mbps ⇒ 需要 9.2 倍带宽。
反过来: 6 Mbps 在 15 台全双工下最多支撑 1087 Hz
     — 与官方 Z1 页标的 Control Frequency 1KHz 撞在同一个数量级。
所以官方的 10000 Hz 只能是单台或被广播帧摊薄后的指令刷新率，不是 15 台同时
10000 Hz 全量交换。
  → PASS

[15 G1 减速比一致性]
G1 官方关节电机家族（unitree_rl_lab 配置，一手）:
  N7520-14.3: 88 N·m ÷ 14.3 = 6.15 N·m（转子侧）
  N7520-22.5: 139 N·m ÷ 22.5 = 6.18 N·m（转子侧）
  N5020-16: 25 N·m ÷ 16.0 = 1.56 N·m（转子侧）
  W4010-25: 5 N·m ÷ 25.0 = 0.20 N·m（转子侧）
同一台 ϕ7520 电机、两个减速比 ⇒ 转子侧 6.15 vs 6.18 N·m，差 0.4%。
这是一条官方数据自己完成的证明：输出扭矩 ≈ 转子扭矩 × 减速比。型号名里的数字就
是减速比，而 88 / 139 两档输出只差一个齿轮。
  → PASS

[18 刚度上限]
GO-M8010-6: τ_peak 23.7 N·m ÷ kp_max 25.599 = 满扭矩下位置误差至少 0.926
rad = 53.0°
IM6014: τ_peak 34.4 N·m ÷ kp_max 410.725 = 满扭矩下位置误差至少 0.084 rad
= 4.8°
刚度上限是指令字段的量程给的，不是控制理论给的。同一个 kp 量程下，减速比越高
（输出扭矩越大），满扭矩时的位置误差越大。这正是 Z1 官方脚注自己承认的那件事：低
减速比 ⇒ 位置控制刚度低 ⇒ 不优化就会抖。
  → PASS

[18 固有频率闭环校验]
官方 mimic 配置按电机型号派生的 kp，与官方 armature 合起来算固有频率:
  N7520_14p3: kp=40.179 ÷ armature=0.010178 ⇒ ω_n=62.8 rad/s = 10.00 Hz
  N7520_22p5: kp=99.098 ÷ armature=0.025102 ⇒ ω_n=62.8 rad/s = 10.00 Hz
```

```
W4010_25: kp=16.778 ÷ armature=0.004250 ⇒ ω_n=62.8 rad/s = 10.00 Hz
四档全部落在 10 Hz 附近（最大偏差 0.0%）。
而官方逐关节 action_scale 等于 0.25 × effort ÷ kp：N7520-14p3:0.548
N7520-22p5:0.351  N5020-16:0.439  W4010-25:0.075
这是第 18 章那道『ω_n ≤ 电流环带宽/5』天花板的官方实例：宇树自己就是用 ω_n =
√(kp/J) 反推 kp 的。
  → PASS

[18 armature 折算口径校验]
同一个 kp 的两种惯量口径，只有一种对得上官方那句 10 Hz：
  N7520_14p3: 速比 14.3、kp 40.179、armature 0.010178 ⇒ 按关节侧 10.00
Hz；若按电机侧 143 Hz
  N7520_22p5: 速比 22.5、kp 99.098、armature 0.025102 ⇒ 按关节侧 10.00
Hz；若按电机侧 225 Hz
  N5020_16: 速比 16、kp 14.251、armature 0.003610 ⇒ 按关节侧 10.00 Hz；若
按电机侧 160 Hz
  W4010_25: 速比 25、kp 16.778、armature 0.004250 ⇒ 按关节侧 10.00 Hz；若
按电机侧 250 Hz
结论：armature 是折算到关节（输出）轴的等效惯量，与源码式 J2 + J1·i2² +
Jr·(i1·i2)² 一致；按电机侧读会高出整整一个速比倍（几百 Hz），与官方注释矛盾。
  → PASS

[27 485 轮询周期 vs EtherCAT 飞过即改]
4 Mbps ⇒ 位时间 250 ns；一帧 33 B = 264 bit ⇒ 66 µs/台
15 台轮询一轮 = 990 µs（< 1000 µs，正好塞进 1 kHz）
20 台轮询一轮 = 1320 µs（> 1 kHz，装不下）
EtherCAT 100 Mbps ⇒ 位时间 10 ns；20 轴过程数据 1680 B = 13440 bit ⇒
134.4 µs
轮询 20 台 ÷ 一帧飞过 = 9.8 倍。结论：'最多 15 台'既是地址限制，也是周期限
制，两个理由独立成立。
  → PASS

[26 6B 模型带宽]
6B 参数 × 2 B = 12 GB 权重；带宽 100 GB/s ⇒ 单步 ≥ 120 ms ⇒ ≤ 8.3 Hz（还
没算 KV、激活、图像）。
  → PASS

=== 结论：0 项异常 ===
```

B.3 主算例（人形）与对照数据（arithmetic_check）

先把主算例位定清楚：进入正文论证链的可复算样本全部是人形自身可算的量，函数都在 `code/tools/foc_check.py` 里，逐条输出见 B.2。

主算例	函数名（`code/tools/foc_check.py`）	用在第几章	脚本打出来的结论
满扭矩下的位置刚度上限	`stiffness_limit()`	18	23.7 N·m ÷ 25.599 ⇒ 位置误差至少 0.926 rad（53.0°）
一条总线的带宽预算	`bus_budget()`	27	合计 23.76 Mbps vs 官方 4 Mbps ⇒ 差 5.9 倍；4 Mbps 最多 1010 Hz
权重字节 ÷ 内存带宽	`llm_memory_time()`	26	12 GB ÷ 100 GB/s ⇒ 单步 ⩾ 120 ms ⇒ ⩽ 8.3 Hz
编码器分辨率对齿隙	`resolution_vs_backlash()`	14	折算到输出端 0.000534° ⇒ 齿隙是它的 187 倍

（注意一个同名陷阱：`arithmetic_check.py` 里也有一个叫 `bus_budget` 的函数，它吃的是 `calculations.yaml` 的 `j288_bus` 块——那是数字舵机总线的报文格式算例，与人形关节无关，也与四足无关。上面表里第 27 章那条是 `foc_check.py` 的 `bus_budget(model, nodes, hz)`。）

下面是 `python code/tools/arithmetic_check.py --demo` 的输出。同一次运行，两块只是**按用途分组誊抄**：行内文字与脚本逐字一致，没有改过一个字，只是重排了先后并各加了一个小标题。

脚本输出里与“人形/通用器件”相关的行（第一行是自检抬头）。注意其中两行不给任何正文章供数，它们是示范行：`bus_budget(J288 舵机总线)` 是数字舵机的报文格式算例、`resolution_vs_backlash(Z1 机械臂关节)` 打的是

“缺另一个数，不能相除”。

```
selfcheck: 3/3 算式实现正确
bus_budget(J288 舵机总线) 单个来回_bit=460  12 个单元一轮_us=920.0  控制周期 500Hz 预算_us=2000.0  总线占预算比例=46%  地址够不够=够：12 个单元，可用地址 15 个
resolution_vs_backlash(Z1 机械臂关节) 官方背隙_度=0.1000  能不能相除=不能——缺另一个数。这两个数不能跨机型混用（宇树没给 GO-M8010-6 的背隙，也没给 Z1 的编码器位数）
latency_chain       各环节_ms={'camera_exposure_ms': 20.0, 'perception_infer_ms': 30.0, 'planner_ms': 10.0, 'bus_roundtrip_ms': 2.0, 'motor_current_response_ms': 1.0}  合计_ms=63.0  占失稳时间比例=缺
hip_height
```

对照数据（四足产品：只作第 16 章的能量对照与关节器件的史料，非本书解剖对象，不进任何一章的解剖链）

```
resolution_vs_backlash(Go2 关节电机) 编码器每圈格数=32768  输出端分辨率_度=0.001736  能不能相除=不能——缺另一个数。这两个数不能跨机型混用（宇树没给 GO-M8010-6 的背隙，也没给 Z1 的编码器位数）
reverse_cot(AS2 主算例) 每公里耗电_Wh=32.4  CoT_无量纲=0.59  对比_人步行 CoT=0.34（本书换算）/ 0.37（Nature 2026 论文口径）  倍数=约为人的 1.7 倍
torque_chain        足端单腿可出力_N=111.3  四腿合计_N=445.1  按 1/2.0 静力裕度可驮_kg=7.7  官方标称负载_kg=TBD  扭矩来源=URDF 关节输出上限 + 等效力臂（厂商源码值）
energy_chain        算不了（缺输入）
frequency_chain     算不了（缺输入）
缺数 1 项（DEMO 模式已注入占位值，仅验证算式）
  TBD: rated_payload_kg
```

这几行的用途边界：`torque_chain` 吃的是 `calculations.yaml` 里 `crosscheck_go2` 那个输入块 （大腿关节上限、等效力臂、四条腿都取自 Go2 的官方源码值），所以它那一句“四腿合计”只回答 第 16 章的一个问题：四个支点与两个支点在静力与能量上差多少。`energy_chain` 与 `frequency_chain` 打印的是“算不了（缺输入）”：这台脚本的能量链与频率链至今只有 `crosscheck_go2` 一个输入块， 里面巡航速度与髋高都是空的，而**人形侧连输入块都没有**。这条“算不了”就是第 20 章那句 “任何宇树人形的 CoT 都无法用公开数据计算”在脚本里的形状。 `reverse_cot(AS2`

主算例) 里“主算例”三个字是 arithmetic_check.py 给这一行起的键名，沿用早期稿的定位，**本书不采用它当主算例**。resolution_vs_backlash(Go2 关节电机) 的唯一用途是器件史料：GO-M8010-6 这颗电机从 Go1 时代就开始卖，它的 15 bit 编码器与 1:6.33 行星减速比是这个型号的来历（附录 C 的电机表）。

B.4 概念依赖与术语预算校验（check_kg）

```
概念 158 个，分布在 43/46 章；未被任何章引入的概念 0 个（本图即权威清单）
每章新概念数：00:5  01:2  02:4  03:2  04:4  05:4  06:2  07:4  08:6  09:3
10:5  11:3  12:6  13:6  14:6  15:2  16:4  17:5  18:4  19:5  20:3  21:6
22:4  23:2  24:6  25:2  26:2  27:2  28:6  29:5  30:1  31:4  32:4  33:2
34:4  35:4  36:4  37:4  38:5  39:1  40:3  41:1  44:1

✓ 依赖顺序、术语预算、章节引用全部通过
```

B.5 输入数据表的位置

输入	文件	说明
官方器件与整机参数	code/tools/foc_check.py 内 OFFICIAL 字典	每条都带来源标记，正文引用处另给 URL
能量/扭矩/频率链的实测与拆解数据	code/tools/calculations.yaml	含 Go2、AS2、人体对照、分辨率与齿隙条目；其中 subject（AS2）与 crosscheck_go2 两个输入块只喂 B.3 末尾的对照数据，不喂本书的解剖链
概念依赖图	bible/knowledge-graph.yaml	158 个概念，v3 人形编号
来源清单	bible/sources.md 与附录 E	抓取日期逐条记录

B.6 自检用例（防脚本自己算错）

`foc_check.py selfcheck()` 当前含 **19 条**已知答案用例（其中 2 条是负对照：故意把错答案喂进去，要求它**不被接受**）；`arithmetic_check.py selfcheck()` 另含 **3 条**。合计 22 条，逐条名字由脚本从输出里取出：

`clarke@0° alpha`、`clarke@0° beta`、`park on q-axis d` `park on q-axis q`、`rotating_field amplitude invariant over θ` `08 反电势幅值 (2/3)·k_t·ω`、`08 满载所需母线 √3·(E+I·R)` `08 反电势（旧系数 k_t·ω 是错的）`、`08 满载母线（漏掉 I·R 是错的）`、`svpwm gain` `deadtime fraction`、`15bit resolution deg`、`pendulum tau 0.66m` `compound 0.99^100`、`compound 0.99^500`、`compound 0.95^100` `compound 0.90^50`、`compound 0.99^28800 量级`、`8h 90% 所需单步失败率 q`

（check_kg 侧另有 158 个概念、0 个未被任何章引入。）

这 22 条用例已经抓到过作者本人的三次算错（CoT 量纲、kg 与 N 混用、 Park 自检用例的参考角）。这就是本书坚持“工具也要被测”的理由。**还有第四处算错不是自检抓的**： 抄错一份第三方表格里的 Go2 连杆质量，靠的是逐行回读官方 `unitree_mujoco/unitree_robots/go2/go2.xml`（记录在 `bible/do-not-touch.md` 第 32 条）——脚本测不到的地方，只能靠回到一手文件。 四处里只有“Go2 连杆质量表” 一条与四足有关：它记的是器件史料（GO-M8010-6 这颗从 Go1 时代开始卖的电机的来历，见附录 C）， 不是本书的解剖对象。

附录 C

器件与人形参数表

> 本附录把全书引用过的参数集中在一处，便于查。**每一格都带来源标记**；空白格表示官方未公布（[未]），本书不填。**C.4a（关节 MCU 选型表）与 C.5 逐格回到原厂页面核对，抓取日期 2026-10-08**；核到但易腐的数（单价、库存、NRND 状态）另标 [查] 并写明日期，核不到的格子写“未核到”。

C.1 人形与类人形整机

项	H1	H1-2	G1	G1 EDU	R1	H2
身高	约 1.8 m	—	1.32 m	1.32 m	—	1.82 m
体重	约 47 kg	—	35 kg	35 kg	—	70 kg
自由度构成	腿 5×2 + 臂 4×2 + 腰 1	腿 12 + 臂 14 + 腰 1	腿 6×2 + 腰 1 + 臂 5×2	同 + 腰可 +2 + 手 7 + 腕 2	腿 12 + 臂 10 + 腰 2 + 头 2	腿 12 + 臂 14 + 腰 3 + 头 2
膝峰值扭矩	360 [厂] / 300 [厂模型]	360	90	120 [厂] / 139 [厂模型]	60（URDF）	360
髋峰值扭矩	220 / 200（模型）	220	139 或 88（两 URDF 冲突）	同	60（URDF）	360
踝峰值扭矩	59 / 40（模型）	75×2	35 / 50 / 25（多口径）	同	33 / 50	66.88（模型）
腰峰值扭矩	—	220	—	+2 自由度档	60	180
臂/肩峰值扭矩	75	肩肘 120 / 腕 30	25（N5020-16）	同	—	120；肩俯仰 130（模型）

官方扭矩密度口径	189 N・m/kg（峰值）	—	—	—	—	—
电池	15 Ah / 0.864 kWh / 最大 67.2 V（⇒ 16 串，[算]）	—	13 串 / 9000 mAh / 充 54 V 5 A	同	未公布	15 Ah / 0.972 kWh / 75.6 V（⇒ 18 串，[算]）
续航	**未公布小时数**（Endurance 行填的是 864Wh）	—	约 2 h	约 2 h	**约 1 h**	**未公布小时数**
反推平均功率	**不给**（除数读不到）	—	≈216 W	≈216 W	—	**不给**（除数读不到）
反推功率密度	**不给**	—	≈6.2 W/kg	≈6.2 W/kg	—	**不给**
感知	3D 雷达 + 深度相机	同	深度相机 + 3D 雷达 + 4 麦 + 5 W 扬声器	同	双目（R1）/ 单目（R1 AIR）	大 FOV 仿人双目（基线 60 mm、12/10 bit RAW、F2.0、焦距 1.53/2.67 mm、−20~+60 °C）
计算	Intel i5（平台）+ i7（开发）；可选 i7 或 Orin NX，最多 3 块	—	“多品牌可选，例如 Orin”	文档中心 PC2：Jetson Orin NX / 16G / 8×A78AE / 1024 Ampere + 32 TC / 918 MHz	—	可选 Jetson Thor（T5000，2070 TFLOPS FP4-Sparse）或 Orin NX 16G（100 TOPS）

关节总线	RS-485（官方对全系电机口径）	同	同	同	同	同
参考价	店铺现标 $90,000（注“联系实价”）	未公布	¥8.5 万起 / $13,500	¥16.9–30.9 万	¥9,900 起 / $4,900	¥19.9 万含税 / $29,900
IP 防护	[未]	[未]	[未]	[未]	[未]	[未]
额定（持续）扭矩	**[未]（全系无一公布）**					

C.2 一体化关节与微型执行器

型号	峰值扭矩	减速比	供电	编码器	总线 / 帧长 / CRC	质量	尺寸	价格
GO-M8010-6	23.7 N·m（40 A）	1:6.33	12–30 V（推荐 24 V）	单圈绝对值 15 bit	RS-485 4 Mbps；17 B 下 / 16 B 上；CRC16_CCITT；ID 0–14 + 15 广播	530 g	96.5×92.5×42.3 mm	**$369**

IM6014（N6014B-12.6）	34.4（反）/ 31.7（同） N·m	3:38 = 12.667	24–75 V	**双**绝对值 15 bit	RS-485 6/4 Mbps；20 B 下（0xFE 0xEE）/ 26 B 上（0xFC 0xEE）；CRC32-MPEG2	535 g	φ65×60 mm	店铺标价存疑
F-1515-108	0.76 N·m（理想值）	1:108	[未]	[未]	[未]	45 g	34.8×23.1×23.3 mm	[未]
F-1515-214	1.498 N·m（理想值）	1:214	[未]	[未]	[未]	45 g	同上	[未]
SV1-25 / J288 数字舵机	1.5 N·m	288.35:1	[未]	[未]	私有串口协议	19.5–39 g	—	$15 起 / SV1-25 $229
G1 关节电机家族（`unitree_rl_lab` 配置）								
N7520-14.3	88 N·m（模型限幅）	14.3	[未]	[未]	走整机 `rt/lowcmd`	[未]	φ75×20 级	—
N7520-22.5	139 N·m	22.5	[未]	[未]	同	[未]	同	—

N5020-16	25 N·m	16	[未]	[未]	同	[未]	φ50×20 级	—
N5020-16-parallel	35 N·m（踝）	16	[未]	[未]	同	[未]	同	—
W4010-25	5 N·m	25	[未]	[未]	同	[未]	φ40×10 级	—
M107（H1/H1-2）	膝 360 / 髋 220 / 踝 59	[未]	[未]	[未]	[未]	反推 ≈1.9 kg/单元（用官方 189 N·m/kg）	[未]	—

指令量化（一手）

```
M8010 ：位置 int32，raw = (θ/2π)×32768（q15）；kp、kd int16，raw = (k/25.6)×32768
        允许范围 0.0 ≤ kp，kd ≤ 25.599
IM6014：转矩 int16，2560 = 1 N·m（绝对上限 2¹⁵/2560 = 12.8 N·m 转子侧）
        位置 int32，32768/2π = 1 rad；k_pos int16，12800 = 1 N·m/(rad/s)
        输出侧范围：outputTor −162.133~162.128 N·m；Kp 0~410.725；Kd 0~102.681
多字节一律小端
```

C.2a 官方电机模型参数（取自 `unitree_rl_lab/source/.../assets/robots/unitree_actuators.py`，[厂]）

这一张表是本书唯一一处能拿到「宇树关节的摩擦与惯量」的地方——它不在产品页上，在训练仓库里。

模型：`UnitreeActuator`（继承 `DelayedPDActuator`），带摩擦、延迟与 T-N 曲线限力。

```
τ_friction = Fs · tanh(q̇ / Va) + Fd · q̇        Va 默认 0.01 rad/s
armature   = J2 + J1·i2² + Jr·(i1·i2)²          折算到关节（输出）轴的等效惯量
限力按 X1 / X2 分段；X1 = Maximum Speed at Full Torque（T-N 曲线折点），rad/s
```

型号	X1	X2	Y1 同向峰值	Y2 反向峰值	Fs	Fd	armature (kg·m²)
M107_15	14.0	25.6	150.0	182.8	0	0	0.063259741
M107_24	8.8	16	240	292.5	0	0	0.160478022
Go2HV	13.5	30	20.2	23.4	0	0	—
N7520-14.3	22.63	35.52	**71**	**83.3**	1.6	0.16	0.01017752
N7520-22.5	14.5	22.7	**111.0**	**131.0**	2.4	0.24	0.025101925
N5010-16	27.0	41.5	9.5	17.0	0	0	0.0021812
N5020-16	30.86	40.13	**24.8**	**31.9**	0.6	0.06	0.003609725
W4010-25	15.3	24.76	**4.8**	**8.6**	0.6	0.06	0.00425

齿轮级惯量（源码 docstring 原文，单位 kg·m²）

```
N7520-14.3  rotor 0.489e-4 / gear_1 0.098e-4 ratio 4.5 / gear_2 0.533e-4 ratio 48/22+1
N7520-22.5  rotor 0.489e-4 / gear_1 0.109e-4 ratio 4.5 / gear_2 0.738e-4 ratio 5.0
N5020-16    rotor 0.139e-4 / gear_1 0.017e-4 ratio 46/18+1 / gear_2 0.169e-4 ratio 56/16+1
W4010-25    rotor 0.068e-4 / gear_1 ratio 5 / gear_2 ratio 5
N5010-16    rotor 0.084e-4 / gear_1 0.015e-4 ratio 4 / gear_2 0.068e-4 ratio 4
```

把上面那行齿轮级惯量代进 armature 的式子，口径这件事就自己站住了（N7520-22.5：J2 = gear_2 = 0.738e-4，J1 = gear_1 = 0.109e-4 且 i2 = 5.0，Jr = rotor = 0.489e-4 且 i1·i2 = 4.5 × 5.0 = 22.5）[算]：

```
armature = 0.738e-4 + 0.109e-4 × 5.0² + 0.489e-4 × (4.5 × 5.0)² =
0.0251019 kg·m²
```

逐项是 0.0000738 + 0.002725 + 0.02476，和 0.0251019——对上官方那个 0.025101925，八位有效数字里只差最后一位。 换句话说，这个 armature 就是**把三级惯量全部搬到输出轴上**加出来的数（转子那一项被乘大了 506 倍，是它的主导项）。反过来把 0.025102 除以 22.5 的平方（= 506.25）折回电机轴，得 4.96 × 10^{-5} kg·m^2，比 rotor 单独的 4.89 × 10^{-5} 只多出两级齿轮折算下来的 0.07 × 10^{-5}——**同一批数，两种读法各自闭合，而只有“输出轴”这一读法与式子的写法一致** [算]。

口径读错会怎样：`code/tools/foc_check.py` 的 `armature_side_check()` 的四行输出（2026-10-08 跑，PASS） [算]

```
N7520_14p3: 速比 14.3、kp 40.179、armature 0.010178 ⇒ 按关节侧 10.00 Hz;
若按电机侧 143 Hz
N7520_22p5: 速比 22.5、kp 99.098、armature 0.025102 ⇒ 按关节侧 10.00 Hz;
若按电机侧 225 Hz
N5020_16:   速比 16、kp 14.251、armature 0.003610 ⇒ 按关节侧 10.00 Hz; 若
按电机侧 160 Hz
W4010_25:   速比 25、kp 16.778、armature 0.004250 ⇒ 按关节侧 10.00 Hz; 若
按电机侧 250 Hz
```

同一个 kp、同一个 armature，两种读法差整整一个速比倍（$\sqrt{(i^2)} = i$）。**四档里只有“按关节侧”这一列全部落在 10.00 Hz，而官方 mimic 配置的注释写的就是 10 Hz 固有频率**；按电机侧读给出 143 / 225 / 160 / 250 Hz，与官方那句注释差一到两个数量级。**所以只有按关节（输出）侧读才对得上官方那句 10 Hz。** 本书第一版在这里把它标成“折算到电机轴”——**式子照抄无误、标注写反**，而一句标注会把第 18 章整条 ω_n 推导读反方向，因此这条检查已被固化进脚本（脚本注释同步改正），本表按关节侧读。

读这张表的四个要点（本书正文用到处均已标注）：

一，**N7520 两档的 rotor 惯量相同（0.489e-4 kg·m²），齿轮第二级速比不同（48/22+1 ≈ 3.18 对 5.0）**——这正是第 14 章「同一颗电机、两个齿轮」那条发现的下半部分证据。

二，**M107 系列的 Fs/Fd 都是 0**，即官方仿真里 H1/H1-2 的关节**不建模摩擦**；而 G1 用的 N7520/N5020/W4010 都有非零 Fs/Fd。**这解释了两台机器在 sim2real 上的不同难度：G1 的摩擦被建模了，H1 的没有**。

三，**Y1 与 Y2 不同（同向与反向峰值）**，例如 N7520-22.5 是 111.0 与 131.0——**官方承认电机正反转的扭矩上限不对称**。而产品页只给一个“峰值扭矩”。第 08 章那条“角点值不可当工作点”在这里有了第二个层次：**连正反向都不是同一个数**。

四，**armature 就是第 18 章里那个 J**（折算到**关节/输出轴**，不是电机轴——式子里 Jr 乘的是 $(i1 \cdot i2)^2$，已经把减速比算进去了）。**因为它本来就是输出侧口径，所以第 18 章那道 ω_n = √(kp/J) 可以直接代，不必再乘或再除速比**；口径别搞错，搞错会把第 18 章整条推导读反：`code/tools/foc_check.py` 的 `armature_side_check()` 就是钉这件事的—— 同一个 kp 按关节侧读是 10.00 Hz（对上官方注释），按电机侧读是 225 Hz（差整整一个速比，四行的机器证明见本节上面）。有了它，本章那道“ω_n = √(kp/J)”的天花板**第一次可以用官方数算**：以 N7520-22.5 为例，mimic 配置给的 Kp = 99.098、armature = 0.025101925，则

```
ω_n = √(99.098 / 0.025101925) = 62.8 rad/s ⇒ 62.8 ÷ 2π = 10.00 Hz
```

正好等于官方 mimic 配置注释里的“10 Hz 固有频率” [算][厂]。也就是说：**宇树自己就是用第 18 章那条式子反推 kp 的**。 这条验证是本书第 1 部与第 2 部之间最干净的一次闭环。

C.2b 关节里那颗 MCU：选型表与“为什么要 FPU”

第 27 章说关节层是“MCU + FOC + 现场总线”，这一张表给的就是那句话的器件落点。**每一格都标了出处；核不到的格子写“未核到”，不猜。**

型号	内核 / 主频	硬件 FPU	高分辨率定时器	CORDIC / FMAC	ADC 与 PWM 硬件联动	CAN-FD	本书为什么列它
STM32F413RGT6	Cortex-M4F，主频 未核到	有（M4F 带 FPU）	未核到	未核到	未核到	未核到	**宇树官方 IM6014 通信示例所用型号** [厂]
STM32G4 系列（G431 / G474 两档）	Cortex-M4，两页都写“up to 170 MHz”、“213 DMIPS”	单精度（G474 页原文“single-precision floating-point unit (FPU)”）	**G431 页 0 次 HRTIM；G474 页原文“HRTIM … 184 ps resolution, 12 PWM”**	两档都有（页原文“(CORDIC … and FMAC unit for filter functions)”）	系列页有“A flexible interconnect matrix allows autonomous communication between peripherals and saves CPU resources.”（逐路触发路径只在其互连矩阵表里，本轮未取到 → 见 C.4a）	**G431 1× FDCAN、G474 3× FDCAN**（参数表 CAN (FD) = 1 / 3）	开源参照实现 moteus 用的就是 STM32G4；**逐格与 URL 见下面 C.4a**
GD32F50MxxG	Cortex-M33，最高 252 MHz [拆]	未核到	报道称官方称“1 路 32 位高精度定时器” [拆]	未核到	报道称 3 路 12 位 ADC、12 通道并行 [拆]	报道称**标配 2 路 CAN-FD** [拆]	国产电机专用料，报道称官方明写面向“人形机器人、协作机械臂、灵巧手” [拆]

GD32M531 系列	Cortex-M33 / 180 MHz（官方选型表）	未核到（本轮抓到的官方页段落无 FPU 字样）	选型表无 HRTM 行；“Advanced TM (16bit) 2”	无 CORDIC/FMAC；有“TMU 1”与“空间矢量PWM硬件加速器”	官方页只给 ADC/DAC/比较器集成这句 → 触发绑定未核到	**1× CAN 2.0B，无 FD**	电机专用料，官方把“机器人关节”列为应用场景；**逐格见 C.4a**
AT32M416 / M412	页原文“ARM® Cortex®-M4主频最高180MHz”	“内建单精度浮点运算单元(FPU)与数字信号处理器(DSP)”	无 HRTIM / 无移相字样；“1个16位8通道高级定时器…带死区控制和紧急双向刹车功能”	页面无 → 未核到	页面无 → 未核到	**M416“1 x CAN-FD”；M412“1个 CAN”（无 FD 字样）**	国产电机专用料的另一家；**逐格与 URL 见 C.4a**

出处：宇树官方文档 `STM32 HAL Communication Example`（doc-cdn `30/1377`，抓取 2026-10-08）； 兆易创新 GD32F50MxxG 发布报道（电子发烧友，2026-08-05，二手转述官方稿）——**这一行的口径是二手转述，本书标 [行] 不升 [厂]**。 ST 官网的 G431 / G474 / H743 / H723 / F405 / F767 六个产品页、兆易创新的 GD32M531 官方选型表与发布页、 雅特力的 AT32M416 / AT32M412 产品页，**本轮（2026-10-08）都已回读核到**，逐格出处见 C.4a 的来源列。 （本书第一版这一节里写的“ST 官网 G474 产品页抓取超时所以留空”已经不成立，已按核到的原文填上。）

那颗“建议带 FPU”值多少钱？ 官方示例原文是： IM6014 的 STM32 例程基于 STM32F413RGT6，“如果移植到其他 MCU 型号，建议使用带 FPU 的 MCU” [厂]。 这句话不是客套，是一道可以在纸上算完的账。 FOC 每个控制周期要做的事是固定的： 两次 Clarke/Park 正逆变换（每次含正弦、余弦与两三次乘法）加两路 PI， 粗算 **约 30 次浮点乘加**。按本书假设（假设，不是厂商数据）：

```
软浮点一次乘加 ≈ 100 个时钟周期；170 MHz ⇒ 100 ÷ 1.7e8 ≈ 0.59 µs
30 次 ⇒ 约 17.7 µs，占 10 kHz 环（100 µs）预算的 17.7%
带 FPU 一次乘加 ≈ 流水线 4–6 个周期 ⇒ 30 × 5 ÷ 1.7e8 ≈ 0.9 µs，占比约 0.9%
```

换句话说：没有 FPU，光是“把电流旋到正确的坐标系里”这一件事，就要吃掉你近两成的控制预算。 再叠上第 13 章那条死区与采样窗、第 27 章那条 250 ns 位时间，就不难理解为什么 “能跑 FOC 的 MCU”和“能跑点灯的 MCU”在选型上是两个世界。 （17.7% 与 0.9% 是本书按上述假设的推算 [算]，不是任何厂商的实测数据； 换主频或换浮点库，两个数都会变，变的是量级不是结论。 算法本身：100 ÷ 1.7e8 = 0.588e-6 s = 0.588 µs；0.588 µs × 30 = 17.65 µs；17.65 ÷ 100 = 17.7%。 带 FPU 取 5 个周期：5 ÷ 1.7e8 = 0.0294 µs；× 30 = 0.88 µs；0.88 ÷ 100 = 0.88%。）

这张表怎么读：先看 FPU 那一列，再看“ADC 与 PWM 硬件联动”——第 13 章讲过采样时刻错开一个死区就会引入 48 mV 量级的电压误差， 所以真正决定电流环质量的不是主频，而是**ADC 触发能不能由 PWM 计数器硬件发出**， 让采样点稳定落在 PWM 周期的中点，而不是靠软件在 ISR 里“大概什么时候读一下”。 CAN-FD 那一列对应第 27 章：宇树官方明确目前生产的电机走 RS-485， 而行业里 5 Mbps CAN-FD 是另一条路线（moteus），两者的位时间差一个数量级。

C.3 灵巧手与机械臂

型号	自由度	触觉	传动 / 电机	总线	价格
Dex1-1	[未]	可选相机版	5 N·m / 1:25 电机	[未]	[未]
Dex2-5	[未]	—	[未]	[未]	[未]

Dex3-1	主动：拇 3 / 食 2 / 中 2	**9 阵列 / 33 压力传感器**，量程 10–2500 g，不损坏最大承受 20 kg	F-1515 系列（近端齿轮）	[未]	[未]
Dex5-1 / 1P	[未]	**94 个触觉点**（需再核）	**空心杯电机**	[未]	[未]
Dex5-S	22 DOF	—	—	—	3.99 万元起（媒体口径）
Z1 AIR / PRO	6 DoF 臂	—	**谐波减速器，减速比 60+，背隙 ≈6 弧分**	RS-485	$15,999
Z1 关节电机	—	—	峰值 33 N・m，15 bit 编码器，力控精度 ≈0.2 N・m，控制频率 1 kHz，ϕ65×52 mm，405 g，24 V 推荐	同	含在整机
R1-7a	7 轴	—	IM6014（J3–J5）	[未]	¥9,900 起 / 店铺 $750 起

Z1 官方脚注（原文，本书第 14 章的支点）

Since the reduction ratio used by each joint is relatively low, the position control stiffness of the whole machine is low. If the control mode is not optimized, there will be large position control error and shaking when the manipulator moves.

C.4 感知与算力（可买到的器件）

器件	关键参数	价格
宇树 L1	360°×90°，20 m@90% / 10 m@10%，±2.0 cm，21,600 点/s，11/180 Hz，盲区 0.05 m	**¥999 / $249**
宇树 L2	360°×96°，30 m@90% / 15 m@10%，≤2.0 cm，64,000 点/s，内置 6 轴 IMU（采样率官方未公布 [未]），4.5 mm 分辨率，100 Klux，Class 1	**¥1,799 / $419**
Livox Mid-360	360°×59°，40 m@10%，265 g，IP67	¥3,999
RealSense D435i	87°×58°@90 fps + IMU	$334
Orbbec Gemini 335	0.26–3 m	$304
Jetson Orin NX 16 GB	1024 CUDA / 32 TC / 8×A78AE，100 TOPS	¥5,499（模组）
Jetson AGX Thor DevKit	T5000：2070 TFLOPS(FP4)、128 GB、14×Neoverse-V3AE、40–130 W、**模组自带 4×CAN**	$3,499 → $5,499
地瓜 RDK S100	80 TOPS BPU + 6×A78AE + **4×Cortex-R52 实时核**、板载 CAN/CAN-FD	¥2,499（预售）
地瓜 RDK X5	10 TOPS、板载 1×CAN-FD、PoE、Wi-Fi6	¥549（4G）/ ¥699（8G）
STM32G431CBT6	170 MHz M4 + CORDIC/FMAC（**无 HRTIM**，见 C.4a 的更正）	¥21.96
纳芯微 NSi6602B	隔离半桥 4 A / 6 A 峰值	¥7.61
麦歌恩 MT6816CT	14-bit SPI 磁编码器，25,000 rpm	无公开挂牌价
Microchip LAN9252TI	EtherCAT 从站控制器（SPI 接口 ESC）	¥87.42
达妙 DM-J8009-2EC	额定 20 / 峰值 40 N·m，双编码器，CAN/CAN-FD	¥1,475–2,398

小米 CyberGear 微电机	关节级无刷微电机 + 驱动器一体，市面另一条“买得到”的路线；**官方参数与价格本轮未取到可回读页面** [未]	未核到
因时 RH56DFX	6 DoF / 12 关节，6 力传感，0.5 N 分辨率，RS-485	¥22,200
灵心巧手 L20 / O6	21 DoF 腱绳 / 6 DoF	¥4.99 万 / ¥6,666（Lite ¥3,999）
六维力（坤维 / 蓝点 / 宇立）	国产已降至约 ¥5,000/台 量级	询价
帕西尼触觉阵列	单价从十万级降至 ¥199 起	¥199

C.4 这一张表的标记约定（本章开头承诺“每一格都带来源标记”，这张表按来源分组集中标，逐行标会挤爆版面）：

- 宇树 L1 / L2 与 Thor T5000、RDK 的参数与价格：**[厂]**，取自各自官方页面，抓取日期 2026-10-07/08；
- 因时 RH56DFX（¥22,200）、灵心巧手、帕西尼、坤维/蓝点/宇立六维力、达妙、麦歌恩、纳芯微、STM32G431、LAN9252 这些**第三方器件**：参数与挂牌价来自**厂商自己的页面或经销报价**，本书按 [查] 记（他方一手可回读，不是宇树官方），抓取日期 2026-10-08；
- Livox Mid-360 的 ¥3,999 取自已 404 的旧页面，**本书标 [推] 并列为待复验**，不要当现行报价引用；
- 小米 CyberGear 本轮**没有取到可回读的官方参数页**，写 [未]，价格一格留“未核到”。

C.4a（关节 MCU 选型表）与 C.5 是**逐格回原厂核对**的，标记写在格内。

C.4a 关节 MCU 选型表（本书逐格回原厂核对）

这张表存在的理由：第 13 章说过“为什么是 G4 这一档”，那句话原来只挂在三个外设名字上。下面把它变成一次可对照的判断——**每格只填原厂页面上写着的字，核不到的格子留空并写“未核到”**。本轮（2026-10-

08）逐行回读的来源是各家**产品页 / 官方选型表 / 官方发布页**；ST 的数据手册 PDF 本轮取不到可解析的原文，所以凡是只有手册里才有的表格行（例如互连矩阵的每一条触发路径）一律写“未核到”，不假引。单价例外，来自分销商页面，另标 [查] 并写抓取日期。

口径先说死：宇树关节板用的是哪颗 MCU，官方从未公布 [未]。**这张表不是“宇树的选型”，是“你要替它选型时的对照表”**。

型号	内核 / 最高主频	FPU	高分辨率定时器（没有它 = 软件移相的代价）	CORDIC / FMAC	ADC 触发能否硬件绑定 PWM	CAN / CAN-FD 外设个数	封装 / 单价	来源（抓取 2026-10-08）
STM32G431CBT6	Cortex-M4 / 170 MHz（页原文“with 213 DMIPS”）	页原文“Arm 32-bit Cortex-M4 CPU with FPU”（单精度由 M4 架构定义）	**无 HRTIM**：产品页全文 0 次“HRTIM”。定时器清单页原文“14 timers”，含“2 x 16-bit 8-channel advanced motor control timers”、“2 x 16-bit timers with up to four IC/OC/PWM”、“1 x 16-bit timer with 2 x IC/OCs”、“2 x					

STM32G431CBT6	Cortex-M4 / 170 MHz（页原文“with 213 DMIPS”）	页原文“Arm 32-bit Cortex-M4 CPU with FPU”（单精度由 M4 架构定义）	**无 HRTIM**：产品页全文 0 次“HRTIM”。定时器清单页原文“14 timers”，含“2 x 16-bit 8-channel advanced motor control timers”、“2 x 16-bit timers with up to four IC/OC/PWM”、“1 x 16-bit timer with 2 x IC/OCs”、“2 x 16-bit timers with IC/OC/OCN/PWM”、“2 x 16-bit

power timer”	**两者都有**（页原文“The devices embed peripherals allowing mathematical/arithmetic function acceleration (CORDIC for trigonometric functions … and FMAC unit for filter functions)”）	系列页原文有一句 **“A flexible interconnect matrix allows autonomous communication between peripherals and saves CPU resources.”**；但“哪一路 TIM 触发哪一路 ADC”那一行只存在于数据手册的 interconnection matrix 表里，**本轮未取到该表原文 → 该行未核到**	**1× FDCAN**（页原文“One FDCAN” / “1 x FDCAN controller supporting flexible data rate”；参数表 CAN (FD) = 1、CAN (2.0) = “-”）	ST 页“9 packages from 32-pin to 100-pin”，Budgetary Price 栏为“-” ⇒ **官方价未核到**；分销商：立创商城 item 549633 = ¥21.96（商品编号 C529355，2026-10-08）[查]	ST 产品页 https://www.st.com/en/microcontrollers-microprocessors/stm32g431cb.html 与系列页 …/stm32g4-series.html [查]

STM32G474RET6	Cortex-M4 / 170 MHz（页原文“213 DMIPS”）	页原文“features a single-precision floating-point unit (FPU)”	**有 HRTIM**：产品页原文 **“HRTIM (High-Resolution and complex waveform builder): 6 x16-bit counters, 184 ps resolution, 12 PWM”**，Description 段又写“high resolution timer with 184 ps resolution”；另有“3 x 16-bit 8-channel advanced motor control timers”	两者都有（页原文“(CORDIC for trigonometric functions and FMAC unit for filter functions).”）	同一句系列页互连矩阵原文适用；ADC 侧页原文给“5 x 12-bit ADCs 0.25 µs, up to 42 channels. Resolution up to 16-bit with hardware oversampling, 0 to 3.6 V conversion range”。**HRTIM→ADC 那一行触发路径：本轮未取到手册表原文 → 未核到**	**3×FDCAN**（页原文“3 x FDCAN controller supporting flexible data rate”；参数表 CAN (FD) = 3、CAN (2.0) = “-”）	ST 页“9 packages from 48-pin to 128-pin”、状态“Active”；分销商：立创商城 item 538858 = ¥62.4、封装 LQFP-64(10×10)、编号 C521608（2026-10-08）[查]	ST 产品页 https://www.st.com/en/microcontrollers-microprocessors/stm32g474re.html [查]

STM32H743AII6	Cortex-M7 / 480 MHz（页原文“1027 DMIPS”）	**双精度**（页原文“supports Arm double-precision (IEEE 754 compliant) and single-precision”）	**无“HRTIM”字样**，但页上明列一句**“1× high-resolution timer (2.1 ns max resolution)”**（同页还有“2× 16-bit advanced motor control timers”、“10 × 16-bit general-purpose”、“5 × 16-bit low-power”、“2× 32-bit”）⇒ **2.1 ns 对 184 ps 差 11.4 倍**，这就是 H7 这一档

	做移相时要付的步进	无（页上 0 次 CORDIC / FMAC）	页未给触发路径表 → **未核到**	**2× CAN，都带 FD，其中 1 路 TT-CAN**（页原文 “2× CAN controllers: 2 with CAN FD, 1 with time-triggered CAN (TT-CAN)”）	页给 “UFBGA 169 7x7x0.6P 0.5 mm”；Budgetary Price “-” ⇒ 单价 **未核到**	ST 产品页 https://www.st.com/en/microcontrollers-microprocessors/stm32h743ai.html [查]

STM32H723ZGT6	Cortex-M7 / 550 MHz（页未列 DMIPS ⇒ 未核到）	双精度（页原文 “supports Arm double-precision (IEEE 754 compliant) and single-precision”）	**无 HRTIM、也无 “high-resolution timer” 字样**（0 次）；定时器是 “Seventeen 16-bit (including 5 x low power 16-bit timer available in stop mode) and four 32-bit timers”	**两者都有**——H7 里只有 72x/73x 这一档把 G4 的数学加速器搬上来了（页原文 “(CORDIC coprocessor for trigonometric functions and FMAC unit for filter functions).”）	页未给触发路径表 → **未核到**	**3× CAN-FD**（特性行页原文 “1× FD/TTCAN and 2x FDCANs”；参数表 CAN (FD) = 3、CAN (2.0) = “-”）	页给 “UFBGA 144 7x7x0.6 P 0.5 mm” 与 “LQFP 144 20x20x 1.4 mm”；Budgetary Price “-” ⇒ 单价 **未核到**	ST 产品页 https://www.st.com/en/microcontrollers-microprocessors/stm32h723zg.html [查]

STM32F405RGT6	Cortex-M4 / 168 MHz（页原文“210 DMIPS/1.25 DMIPS/MHz”）	单精度（页原文“The Cortex-M4 core features a floating-point unit (FPU) single precision”）	**无 HRTIM**（0 次）；页原文“twelve general-purpose 16-bit timers including two PWM timers for motor control, two general-purpose 32-bit timers”	**都没有**（0 次）	页未给触发路径表 → **未核到**	**2× CAN 2.0，无 FD**（页原文“Up to 2 × CAN interfaces (2.0B Active)”；参数表 CAN (2.0) = 2、CAN (FD) = “-”）—— 附录 D 里 ODrive v3.x 用的就是这颗，官方口径 CAN 2.0b 1 Mbps、无 FD [查]	页给“LQFP64 10x10x1.4 mm”；单价 **未核到**	ST 产品页 https://www.st.com/en/microcontrollers-microprocessors/stm32f405rg.html [查]

STM32F767BIT6	Cortex-M7 / 216 MHz（页原文“462 DMIPS/ 2.14 DMIPS/MHz”）	双精度（页原文“features a floating point unit (FPU), which supports Arm double-precision and single-precision”）	**无 HRTIM**（0 次）；页原文“Up to 18 timers: up to thirteen 16-bit (1x low-power 16-bit timer available in Stop mode) and two 32-bit timers”	都没有（0 次）	页未给触发路径表 → **未核到**	**3× CAN 2.0，无 FD**（页原文“3 × CANs (2.0B Active)”；参数表 CAN (2.0) = 3、CAN (FD) = “-”）	页给“LQFP208 28x28x1.4 mm”；**该页状态栏“NRND — Not recommended for New Design”**；Budgetary Price “-” ⇒ 单价 **未核到**	ST 产品页 https://www.st.com/en/microcontrollers-microprocessors/stm32f767bi.html [查]

GD32M531RBT7（电机专用）	Cortex-M33 / 180 MHz（官方选型表“Core Cortex®-M33”、“Max Speed (MHz) 180”；官方发布页“Coremark® 跑分达 705、DMIPS 达 267”）	**未核到**：本轮抓到的官方选型表与发布页段落都没有 FPU 字样（要定性须查数据手册）	**官方选型表里没有 HRTM 这一行**；定时器三行是“Advanced TM (16bit) 2”、“GPTM (16bit) 4”、“CPTM (16bit) 3”；发布页口径是**“GD32M531 定位双电机控制”**，并把“机器人关节”列为应用场景	无 CORDIC/FMAC 字样；**替代物是选型表的“TMU 1”与发布页的“内置三角函数运算及空间矢量 PWM硬件加速器”**	选型表给“12bit ADC Units (CHs) 2(16)”，发布页给“集成高速、高精度 ADC、DAC、模拟比较器等电机专属模块”；**至于 ADC 触发能否由定时器硬件发出（含“硬件移相”那一层），本轮在官方页未取到原文 → 未核到**	**1× CAN 2.0B，无 FD**（选型表“CAN 2.0B 1”，页上无 CAN-FD 行）	选型表“Package LQFP64”、“Flash 128K / RAM 32K”；发布页“支持 2.7V~~5.5V宽电压供电，工作温度覆盖-40℃~~ 105℃”；单价**未核到**	兆易创新官方选型表 https://www.gigadevice.com.cn/product/mcu/mcus-product-selector/gd32m531rbt7 与官方发布页 …/about/news-and-event/media-coverage/gd32m531-motor-control-mcu-leader [查]

AT32M416 系列（电机专用；料号后缀按该页订购信息选）	页原文“ARM® Cortex®-M4主频最高180MHz”	页原文**“内建单精度浮点运算单元(FPU)与数字信号处理器(DSP)”**	**无 HRTIM、无移相字样**；页给**“1个16位8通道高级定时器，可组成4对互补通道PWM输出，带死区控制和紧急双向刹车功能”**	页上无 CORDIC/FMAC 字样 → **未核到**	页未给 ADC 触发路径 → **未核到**	**1× CAN-FD**（页原文“1 x CAN-FD”；同族 AT32M412 那一页给的是“1个CAN”，无 CAN-FD 字样）	页列“LQFP48 7 x 7 mm / QFN48 6 x 6 mm / LQFP32 7 x 7 mm / QFN32 4 x 4 mm / TSSOP24 7.8 x 4.4 mm”；单价 **未核到**	雅特力官方产品页 https://www.arterytek.com/cn/product/AT32M416.jsp 与 …/AT32M412.jsp [查]

这张表要先解决的一个错：本书上一版在 C.4 里把 STM32G431CBT6 写成“CORDIC/FMAC/**HRTIM**”。本轮回读 ST 产品页：stm32g431cb.html 全文 `HRTIM` 出现 **0 次**，而带 HRTIM 的是 G474/G473 那一档（stm32g474re.html 直接把它写进 Key features：“HRTIM … 6 x16-bit counters, 184 ps resolution, 12 PWM”）。**C.4 那一行已就地改正并指向本表**。这条更正连带一处正文待办：第 13 章把 G431 描述成“带 HRTIM 所以被选中”的那句同样需要改（本次任务不许动别的章，已登记给主控）。**本轮没能核到的两件事也写在这里，别当成已定案**：其一，ST 数据手册 PDF（DS12589 / DS12288）本轮取不到可解析的原文，所以“G431 手册全文 0 次 HRTIM”这类只有手册才能证的话，本书只写到产品页那一层；其二，开源参照实现 moteus 的官方口径是 **STM32G4**（这一条由本书附录 D 与前几轮核对记录承载，本轮未能复查 README 原文）。

为什么这五栏（FPU / 高分辨率定时器 / 数学加速器 / ADC 触发能否硬件绑定 / CAN-FD 数）值得单列？ 因为它们各自卡在一条具体的链上，每一环都能往下再问一次：

一，**高分辨率定时器买的是什么？** 买的是死区与相移的**步进**。用本书已有的两个数算一次：第 13 章的载波口径是 20 kHz，周期

```
T = 1 / 20 kHz = 50 µs
```

（[算]。）普通 16 位定时器的最小步进是

```
Δt_16bit = 50 µs ÷ 2^16 = 763 ps
```

（[算]。）HRTIM 官方给 184 ps ⇒ **763 ÷ 184 ≈ 4.15 倍分辨率**。再看它落在死区上：第 13 章的死区口径 100 ns，用 16 位定时器只有 100 ns ÷ 0.763 ns ≈ 131 个格可分配，HRTIM 有 100 ÷ 0.184 ≈ 543 格。**同一份“死区补偿表”，能写的精细度差 4 倍**——这就是没有 HRTIM 时“软件移相”要付的代价：你只能在 763 ps 的格子里挪，剩下的误差直接乘进扭矩纹波（第 13 章那条 48 mV 死区误差的另一个来源）。

H743 那一行给了第三种答案，值得单独算一次：ST 页原文 “1× high-resolution timer (2.1 ns max resolution)”。**2.1 ns ÷ 0.184 ns ≈ 11.4 倍**——它比 G474 的 HRTIM 粗一个数量级，但比 16 位定时器的 763 ps 细 **763 ÷ 2.1 ≈ 363 倍** [算]。**“有没有高分辨率定时器”不是一个布尔值，而是一列分辨率数**，选型时真正要比的是“这一档步进落在我 100 ns 的死区上给几格”。

二，**没有 HRTIM 的那几颗靠什么补？** 靠把 ADC 触发绑到定时器上的那套互连矩阵——这一条本书只核到了**系列页那一句**：ST 的 STM32G4 系列页原文 **“A flexible interconnect matrix allows autonomous communication between peripherals and saves CPU resources.”** [厂]。**具体到哪一路定时器触发哪一路 ADC 的那张表在数据手册里，本轮取不到原文，所以那一格写“未核到”**（这正是这张表留

空的原因，不是疏忽）。**为什么这件事必须是硬件的？** 因为采样窗口只有微秒量级（第 13 章）：软件在 PWM 翻转后再去启动 ADC，抖动就是纹波电流里的确定性误差，而“省 CPU 资源”那句话说到底就是**把采样时刻的确定性从软件手里拿走**。国产两家在这一栏本轮只给到外设计数：GD32M531 官方发布页写“集成高速、高精度ADC、DAC、模拟比较器等电机专属模块”、AT32M416 官方页只给高级定时器那句“带死区控制和紧急双向刹车功能”——**两家的“触发能否硬件绑定 PWM”本书都没核到，不许拿 ST 的结论替它们填空**。

三，**数学加速器买的是什么？** CORDIC 买的是 Park 与逆 Park 里那两次 sin/cos 不做浮点库调用（第 10 章那条链：读角 → Park → PI → 逆 Park），FMAC 买的是滤波器不占 CPU 循环。**GD32 这一档给了同功能的另一种命名**（TMU + SVPWM），AT32 那一档页面上则两个都没写——**这一栏空着的格子，读作“未核到”，不读作“没有”**。

四，**CAN-FD 的个数决定一条总线挂几个关节**。第 27 章已经算过 RS-485 那条链的预算；这里只补一条选型事实：**G431 只有 1 路 FDCAN**——一颗芯片做一条总线，要多关节就必须多芯片或分时；**G474 3 路、H723 3 路、H743 2 路**，而 **F405 / F767 / GD32M531 的 CAN 不支持 FD**（参数表 CAN (FD) 为空）。宇树官方对全系电机的口径是 RS-485、单总线最多 15 台、6000 Hz [厂]，智元公开的是 EtherCAT 主站转 3× CAN-FD 5 Mbps [查]——**“3 路 CAN-FD”这个外设数量，正好是那种一次选型就能用上的判据**。

单价这一栏为什么只有两个数？ 因为 ST 产品页的 Budgetary Price 栏在本书抓取的页面上是空的（“-”），分销商价属易腐信息：本书只给核到当天的两条（G431 ¥21.96、G474 ¥62.40，均为立创商城 2026-10-08 页面），**其余一律留空**。而这两条恰好是本表最有用的一格：**同一族里，“要不要 HRTIM”的差价是 2.8 倍**（62.40 ÷ 21.96 ≈ 2.84，[算]）——“为什么是 G4 这一档”于是变成一个可以摊到关节 BOM 上算的取

舍，而不是一句外设清单。第 03 章说过执行器占整机 BOM 的 40–60%，一颗 40 元差价 × 23 个关节 ≈ 920 元 [算] 就是这次取舍的绝对额度。

这张表在什么时候不管用：它只覆盖“关节里的 FOC 那一层”。主频最高的那颗（H723 550 MHz、双精度 FPU）看着最强，但它没有 HRTIM、封装是 LQFP-144（20×20 mm）——**塞不进一个 φ75×20 级的关节**（对照 C.2 里 N7520 的尺寸口径）。所以这张表的正确读法是按列问“我这一路要什么”，不是按行找“最强的那颗”。宇树自己怎么选，官方未公布 [未]。

C.5 对照：同期其他厂商的公开口径（仅列可核项）

```
智元 PowerFlow R86-3 ：60 N·m 额定 / 200 峰值 / 1.28 kg / 行星 48:1
智元 PowerFlow R86-2 ：20 / 80 N·m / 0.81 kg / 16:1
智元 精灵 G2         ：全关节力矩传感器 ≤0.02 N·m、≥5 kHz；1652 Wh / 约 4 h；185 kg
智元 灵犀 X1 DCU     ：EtherCAT 1 kHz ⇔ 3×CAN-FD 5 Mbps
AGILINK OmniHand     ：16 DoF / 510 g
AGILINK Ultra-T      ：22 主动 + 三维触觉 0.01 N
AGILINK Ultra-M      ：全直驱视触觉 0.005 N
开源 moteus r4.11    ：100 A 峰值 / 900 W@30 V / 14.2 g / STM32G4 / 5 Mbps CAN-FD / $94
开源 ODrive S1       ：12–48 V / 40 A 连续 / CAN+USB / $149
开源 LeRobot SO-101  ：6 DoF，主从遥操 + 模仿学习（模仿学习 = 看人怎么做的录像照着学，第 29 章），Apache-2.0，从臂 ¥682
```

（以上全部 [查]，来自各家公开页面或仓库；本书不比较优劣，只作量级对照。）

C.6 这张表里最该被记住的一栏

“额定（持续）扭矩”与“IP 防护”两栏，在宇树整个人形产品线上是空的。

而同期对照厂商的那几行里，智元 PowerFlow 明确写了“60 N·m **额定** / 200 峰值”。**这是一个可以直接拿去问厂商的差别**。

第 36 章解释了厂商为什么不给；第 37 章解释了制度为什么没要求它给。**两者合起来，就是这本书认为这个行业还年轻的最硬证据**。

附录 D

去哪继续学（含许可与失效风险标注）

> 本附录只列**本书引用过或明确推荐过的入口**，每条给三件事：许可、时效风险、以及“读完能不能变成产物”。抓取日期 2026-10-07/08。**链接会腐烂，日期会过期**——这是本附录最大的失效风险。

D.1 第 1 部（一颗关节）：电机与电力电子

入口	类型	许可	时效风险	能变成的产物
ST 电机控制库与应用笔记（MCSDK、AN2016 系列）	官方文档 + 源码	代码多为 SLA/受限，文档可自由读	低（ST 长期维护）	看懂 HRTIM/ADC 触发链；里程碑 ① 的参考实现
moteus（mjbots）	开源关节固件 + 硬件	**Apache-2.0**（LICENSE 首两行即证）	低	状态机与保护逻辑的工业级范本；第 27 章“两相采样”的实证
ODrive	开源驱动器	**MIT**（LICENSE.md）	**高**：最新 release 仍是 fw-v0.5.6（2023-04-29），而 main 已到 2026-01	同上；且是“开源硬件怎么标许可”的教材
SimpleFOC	开源 FOC 库	MIT	中（偏教学，工业边界少）	最快跑通一次 Clarke/Park/开环；**不能**当产品基础
电机学与电力电子教材（Krause 等）	书	商业版权	低（理论不过期）	dq 模型与坐标变换的严格出处

厂商电机规格书（任何一体化关节）	PDF	可自由读	高（参数表常不完整）	用第 04、08 章两个判据去检查它

三个开源实现的官方口径，本书逐文件核对后如下（可直接对照第 12 章）：

```
moteus r4.11  ：STM32G4，5 Mbps CAN-FD，README 的 Control Rate 行写 15–30
kHz，
               PWM Switching Rate 行写 15–60 kHz，theory.md 另给默认值 30
kHz，
               10–44 V，峰值电功率 900 W @30 V，14.2 g，Apache-2.0
               作者自述风险原文："It has not yet burned down my
               (or anyone's that I know of) house, but there are no
guarantees."

ODrive v3.x   ：STM32F405，CAN 2.0b 1 Mbps（无 CAN-FD），
               三环级联，环路周期恒定 125 µs（8 kHz），
               position(P) → velocity(PI) → current(PI)，
               current_gain 由 current_control_bandwidth 自动设定，MIT

LeRobot SO-101：官方文档不给价，BOM 在上游 TheRobotStudio/SO-ARM100：
               主从全套 US$229.88（中国 ￥1343.16）、
               从臂单只 US$121.94（￥682.23）——
               是 BOM 求和（含螺丝刀与台夹），不是官方定价
```

（全部 [厂]，取自各仓库文件；HEAD commit 见 `research/humanoid-stack-specs.md` §0。）

为什么这三个数重要：moteus 的 README 把 **Control Rate 15–30 kHz** 与 **PWM Switching Rate 15–60 kHz** 分成两行列，`theory.md` 再补一句开关频率默认 30 kHz。这三行是第 09 章与第 27 章那条区分的现成教材：**环路更新率、开关频率、电流环带宽是三件事**。README 给的是前两个，**没有给电流环带宽的数**，所以不要把 15–30 kHz 拿去和第 09 章「带宽 ≈ f_sw/10–20」那条判据直接比——那条说的是带宽。真要读就读它的 `docs/reference/theory.md`：那是少数把“为什么取这个数”写明白的开源文档，ODrive 的 `docs/control.rst` 同理（它反过来给的是恒定的 125 µs 环路周期，也是更新率，不是带宽）。而 ODrive 是**真级联三环**的

教科书实现，与宇树 IM6014 官方声明的「不是级联三环」互为反例（第 12 章已并排讲过）。**同一个问题，两个开源实现选了不同的架构，而且都工作**。

这一层最实用的建议：找一个开源工业级实现，抄完，然后自己复现它的一个指标。读完不等于会——第 40 章的六个里程碑全部以“可测产物”结课，就是为这件事设计的。

D.2 第 2 部（从关节到姿态）：控制与 RL

入口	类型	许可	时效风险	备注
MuJoCo（官方文档 + 3.x 源码）	仿真器	Apache-2.0	低（DeepMind 维护，2026-09 已到 3.14）	Sim2Sim 的那“另一个仿真器”；MJCF 格式文档必读
Isaac Lab / Isaac Sim	仿真 + 训练	NVIDIA 软件许可（**可商用但有条款，逐版本读**）	**高**（版本迭代快，API 常破坏性变更）	当前足式 RL 主流；注意 pin 版本
unitree_rl_gym	官方训练仓库	有：`LICENSE`（本书 2026-10-09 回读；13 个宇树仓库里 12 个都有许可文件，只是文件名可能是 `LICENSE` / `LICENCE` / `LICENSE.md`；四种名字全 404 的只有 `digital_servo`，它的限制写在 README 正文里）	**高**：依赖 Isaac Gym Preview 4，已 EOL	教学价值仍在，但别当生产基线

unitree_rl_lab	官方训练仓库	**Apache-2.0**，文件名叫 `LICENCE`（英式拼写）——按 `LICENSE` 扫会说“无许可”，本书第一版就被这个名字骗过（2026-10-09 直接回读 raw 页：该文件 11,362 字节，前两行就是“Apache License / Version 2.0, January 2004”）	中（Isaac Sim 5.1 + Lab 2.3）	**G1 关节电机家族与 effort 限幅的一手来源**（附录 C）
腿式机器人控制综述（2024，PMC11639468）	论文	开放获取	低	ZMP/MPC/QP 的谱系与 1 kHz 量级出处
Raibert / Pratt 的原始论文（ZMP、捕获点）	论文	需机构访问	低（经典不过期）	第 17 章两个判据的第一手

D.3 第 3 部（感知、算力、总线）

入口	类型	许可	时效风险	备注
unitree_sdk2 / **unitree_sdk2_python**	官方 SDK	见仓库	中	DDS 主题与 IDL 的第一手；第 28 章全部字段来自这里
unitree_ros2 / **unitree_ros**	ROS 适配与模型	见仓库	中	`unitree_hg/msg/*.msg` 是消息结构的权威；URDF 在 `unitree_ros/robots/`

CycloneDDS	中间件	Apache-2.0	低	**注意宇树要 0.10.2**，与新版 API 不兼容
ROS 2 REP-2000（发行版生命周期）	官方规范	开放	低	Humble 支持至 2027-05、Jazzy（Ubuntu 24.04）至 2029-05；量产选型的第一份必读
`ros2_control`	框架	Apache-2.0	低	官方只维护 humble/jazzy 分支
EtherCAT 规范（ETG.6010 CoETP / CiA402）	标准	需加入组织或购买	低	第 27 章“为什么不用 EtherCAT”的判据出处
Livox / RealSense / Orbbec 官方 SDK 与手册	器件	各异	中	第 23 章时间同步与测距算术的实测入口
NVIDIA Jetson 模组数据手册（Orin NX、Thor T5000）	官方 PDF	开放	中	第 26 章功耗与内存带宽的一手；**注意 TOPS 与 TFLOPS 的口径**

D.4 第 4 部（大脑与数据）

入口	类型	许可	时效风险	备注
LeRobot（Hugging Face）	数据格式 + 策略库	Apache-2.0	中（迭代极快）	事实上的开源数据标准；含 act/vqbet/diffusion/smolvla/pi0/pi05/groot
unitree_IL_lerobot	官方训练脚本	见仓库	中	v0.3 起支持 GR00T 与 π0.5；宇树数据接入的第一站

Isaac GR00T N1.x	模型 + 训练	**Apache-2.0**	中	端侧 16 GB+ 可跑、有 ONNX/TensorRT 导出；第 30 章那张表的锚点
unifolm-wla / unifolm-world-model-action	官方模型	**Apache-2.0 / CC BY-NC-SA**（两个不同！）	中	**务必读 LICENSE 文件本体**，见第 32 章
AgiBot World Colosseo（arXiv 2503.06669）	数据集 + 论文	开放获取（数据集另有许可）	低	百万轨迹口径的一手出处
GraspVLA（arXiv 2505.03233，CoRL 2025）	论文 + 代码	开放（代码仓库许可需核对）	中	单卡可训的 VLA 范本，第 33 章成本算术的来源
NVIDIA Cosmos	世界模型	OpenMDW-1.1	**高**（2026-05 刚合并为 Cosmos 3）	第 34 章“造数据”用法的一手
Mobile ALOHA / UMI 的论文与仓库	方法	各异	中	低成本遥操与手持采集的两条正路

D.5 第 5 部（瓶颈）：可靠性、安全与标准

入口	类型	许可	备注
ISO 10218-1/-2（工业机器人安全）	标准	需购买	第 37 章判据的原文；**注意它不直接覆盖人形**
ISO/TS 15062（协作机器人技术规范）	技术报告	需购买	28 个身体部位的接触力与压强限值表
IEC 62443（工业网络安全）	标准系列	需购买	联网机器人的网络安全基线
GB/T 5795（ISBN 格式）等出版国标	标准	公开可读部分	附录 E 之外，出书本身要守的格式规范
任何一份工业伺服样本的**降额曲线页**	PDF	可自由读	第 36 章说人形缺的那张图，长这样

这一层是全书最“不性感”、而商业上最决定成败的一层。 第 37 章那张四问清单，答案全部在这里。

D.6 一条通用判据：怎么判断一个入口值不值得花时间

> ① 它有 LICENSE 文件本体吗？（不是 README 上的徽章。
> 名字要试全：`LICENSE`、`LICENCE`（英式，宇树 rl_lab 就是这个）、`LICENSE.md`、`COPYING`；
> 本书就因只扫 `LICENSE` 误报过一次（见 D.2 那一行）。ODrive 是反例：它两种拼写都 404，
> 文件叫 `LICENSE.md`。最后用仓库页的许可接口交叉核对一次，别只信自己扫出来的名字。）
> ② 它最后一次更新是什么时候？（超过 18 个月的教程，版本坑会吃掉你一天）
> ③ 它给的是“能跑的例子”还是“能测的判据”？
> ④ 我能不能在两周内，用它做出一个带测量数的产物？

第 ④ 条是筛子。 任何入口，如果它最好的产出是“我跟着做出来一个 demo 但说不清它为什么工作”，那它给的是第 39 章说的那种“不可见的能力”——**看起来会了，问不出数。**

本书全部 42 章正文与这一条，其实只在教一件事：**把“会了”变成“我测出 ___ = ___，方法 ___”。**

D.7 三个失效风险声明

一，**版本腐烂。**宇树仓库、Isaac Lab、LeRobot、Cosmos 的 API 在 2025–2026 变动频繁。本书给出的所有版本号（cyclonedds 0.10.2、Isaac Sim 5.1 / Lab 2.3、MuJoCo 3.14、ROS 2 Humble/Jazzy）以 2026-10 为准，**发行前需复核。**

二，**许可变更。**开源项目改许可并不罕见（尤其从宽松转严格）。**任何“能商用”的判断，都要在你要用的那一天重新读一次 LICENSE 文件，**不要引用本书。

三，**厂商页面变更**。附录 C 的所有参数取自 2026-10-07/08 的官方页面与仓库；型号、价格、参数会改。**采购前以厂商当前值为准，本书不构成采购依据**。

附录 E

参考来源、抓取日期与「查不到」清单

> E.3 的 URL 清单由 `python tools/gen_appendix_e.py` 从 `drafts/`、`bible/`、`research/` 里收集生成，其余小节（取法、「查不到」清单、冲突口径、不可采信清单）是作者的判断，逐条手写。分三段：① 一手来源（官方页面 / 官方仓库文件 / 官方文档中心）；② 二手来源（拆解、研报、论文、媒体）；③ **「查不到」清单**——本书认真找过但没有拿到答案的问题。第三段是本书最重要的部分之一。

E.1 抓取日期

一手页面与仓库文件的抓取日期是 **2026-10-07 至 2026-10-08**；**2026-10-09 对个别条目做过回读复核**（`unitree_rl_lab` 的 `LICENCE` 是否存在、moteus 的 README 与 `theory.md` 那三行、以及 `LICENSE`/`LICENCE`/`LICENSE.md` 三种名字在 13 个宇树仓库与 ODrive 上的实测），逐条记在 `bible/sources.md` 与 `bible/do-not-touch.md` 第 27 条。型号、价格、开源模型版本与仓库结构在 2026 年下半年变动极快；**发行前必须再做一次全量复核**。

E.2 一手来源的取法（可复现）

```
产品与商城：unitree.com/<型号>、unitree.com/cn/<型号>、shop.unitree.com
电机文档：  unitree.com/go1/motor（GO-M8010-6）、unitree.com/IM6014/、
unitree.com/A1/motor
文档中心：  support.unitree.com 的前端是 SPA（直接抓只返回 768 字节空壳），但
正文可匿名取：
            1) GET https://robot-api.unitree.com/doc/spaces2?locale=en
            2) GET https://robot-api.unitree.com/doc/?space=
<SPACE>&locale=en
            3) GET https://doc-
cdn.unitree.com/<space>/<id>/<locale>/<space>_<id>_<locale>
               → 返回原始 Markdown
```

> 官方仓库： unitree_sdk2、unitree_ros、unitree_ros2、unitree_mujoco（MJCF 在 unitree_robots/ 下）、
> unitree_rl_gym、unitree_rl_lab、xr_teleoperate、unitree_IL_lerobot、
> unifolm-wla、unifolm-world-model-action、unilidar_sdk

仓库勘误：unitree_mjcf 与 unitree_motors_sdk 不存在（404）；unitree_model 只有 USD/DAE，不含 URDF/XML。

E.3 已引用的 URL 清单（144 条，按字典序）

这一节由 python tools/gen_appendix_e.py 生成，收集范围是 drafts/、bible/、research/， 去重后按字典序排。规则很简单：**清单必须等于正文真的引用过的集合**——正文出现而清单漏掉的会被补上， 清单里有而正文已经查不到的会被移出，两种增减都会在生成时逐条打印，不存在“悄悄少了一条”。 条目在第一个中文字符处截断，所以不会再出现 …/11_248_en（G1 这种半截的行尾。

另有 **12 条**正文里出现的地址按规则**不进这张表**，因为它们不是“某一页文献”，生成时会逐条打印理由：

- **两条本机地址**：192.168.123.164:60001、192.168.123.2:8012——是读者在自己机器人上打开的控制台；
- **两条程序接口**：api.github.com/search/…、huggingface.co/api/datasets?…——返回内容随查询串变；
- **四条只有主机名**：mjbots.com、livoxtech.com/、docs.quadruped.de、doc-cdn.unitree.com/——站点首页，不是某一页；
- **两条 raw 仓库目录前缀**：raw.githubusercontent.com/.../unitree_rl_lab/.../ 与 .../unitree_rl_gym/master/—— raw 只发文件，列目录的地址打不开（这两条是研究笔记里当“取法前缀”用的，被正文引用时混了进来）。

反过来有一条边界要说清：**以 / 结尾不等于打不开**。

`egh0bww1.com/posts/2023-10-24-unitree-go1-collection/`、`mujoco.readthedocs.io/en/stable/computation/` 这类以斜杠结尾的地址是真页面（2026-10-09 实测 200），所以生成器只按上面四类精确排除，不敢拿“结尾有没有斜杠”一刀切。它们留在各自的正文处，这里不冒充参考条目。

```
https://arxiv.org/abs/2401.02117
https://arxiv.org/abs/2403.12945
https://arxiv.org/abs/2410.00425
https://arxiv.org/abs/2505.03233
https://arxiv.org/html/2410.24164v1
https://arxiv.org/html/2501.03575v3
https://arxiv.org/html/2503.06669v1
https://arxiv.org/html/2503.20523v1
https://arxiv.org/html/2508.05635v3
https://arxiv.org/html/2510.26742v1
https://commons.wikimedia.org/w/api.php?
action=query&generator=search&gsrsearch=unitree&gsrnamespace=6
https://damodev.csdn.net/69782b327c1d88441d8fdca1.html
https://deepmind.google/discover/blog/genie-3-a-new-frontier-for-world-
models/
https://digitalassets.tesla.com/tesla-contents/image/upload/IR/TSLA-Q1-
2025-Update.pdf
https://digitalassets.tesla.com/tesla-contents/image/upload/IR/TSLA-Q4-
2024-Update.pdf
https://doc-cdn.unitree.com/10/166/en/10_166_en
https://doc-cdn.unitree.com/10/271/en/10_271_en
https://doc-cdn.unitree.com/11/223/en/11_223_en
https://doc-cdn.unitree.com/11/248/en/11_248_en
https://doc-cdn.unitree.com/11/249/en/11_249_en
https://doc-cdn.unitree.com/11/251/en/11_251_en
https://doc-cdn.unitree.com/11/252/en/11_252_en
https://doc-cdn.unitree.com/25/1061/en/25_1061_en
https://doc-cdn.unitree.com/25/1123/en/25_1123_en
https://doc-cdn.unitree.com/27/
https://doc-cdn.unitree.com/27/1135/en/27_1135_en
https://doc-cdn.unitree.com/30/1371/en/30_1371_en
https://doc-cdn.unitree.com/30/1377/en/30_1377_en
https://doc-cdn.unitree.com/30/1378/en/30_1378_en
https://doc-cdn.unitree.com/30/1382/en/30_1382_en
```

```
https://doc-cdn.unitree.com/30/1383/en/30_1383_en
https://doc-cdn.unitree.com/8/106/en/8_106_en
https://doc-cdn.unitree.com/8/107/en/8_107_en
https://doc-cdn.unitree.com/8/1484/en/8_1484_en
https://docs.isaacsim.omniverse.nvidia.com/latest/
https://egh0bww1.com/posts/2023-10-24-unitree-go1-collection/
https://finance.sina.cn/2025-02-16/detail-ineksnuy4910269.d.html
https://github.com/Huuhuuw/FAST_LIO_LOCALIZATION_HU
https://github.com/MAVProxyUser/YushuTechUnitreeGo1
https://github.com/TheRobotStudio/SO-ARM100
https://github.com/abizovnuralem/go2_omniverse
https://github.com/abizovnuralem/go2_ros2_sdk
https://github.com/huggingface/lerobot
https://github.com/jizhang-cmu/autonomy_stack_go2
https://github.com/leggedrobotics/rsl_rl.git
https://github.com/legion1581/go2_firmware_tools
https://github.com/legion1581/unitree_webrtc_connect
https://github.com/mjbots/moteus/releases/latest
https://github.com/unitreerobotics/digital_servo
https://github.com/unitreerobotics/point_lio_unilidar
https://github.com/unitreerobotics/unifolm-vla
https://github.com/unitreerobotics/unifolm-world-model-action
https://github.com/unitreerobotics/unilidar_sdk2
https://github.com/unitreerobotics/unitree_actuator_sdk
https://github.com/unitreerobotics/unitree_guide
https://github.com/unitreerobotics/unitree_legged_sdk
https://github.com/unitreerobotics/unitree_lerobot
https://github.com/unitreerobotics/unitree_mjcf
https://github.com/unitreerobotics/unitree_motors_sdk
https://github.com/unitreerobotics/unitree_mujoco
https://github.com/unitreerobotics/unitree_rl_gym
https://github.com/unitreerobotics/unitree_rl_lab
https://github.com/unitreerobotics/unitree_ros
https://github.com/unitreerobotics/unitree_ros2
https://github.com/unitreerobotics/unitree_sdk2
https://github.com/unitreerobotics/unitree_sdk2_python
https://github.com/unitreerobotics/unitree_sim_isaaclab
https://github.com/unitreerobotics/xr_teleoperate
https://huggingface.co/datasets/unitreerobotics/unitree_model
https://k.sina.cn/article_7950358917_1d9e0d98501901bnhc.html
https://k.sina.com.cn/article_7879996653_1d5af34ed02001kkd0.html
https://m.blog.csdn.net/python9snake/article/details/155368686
https://m.blog.csdn.net/u011487024/article/details/162086616
https://m.elecfans.com/article/8187284.html
https://m.hexun.com/tech/2025-08-09/220818502.html
```

```
https://m.huxiu.com/article/4863770.html
https://maniskill.readthedocs.io/en/latest/_modules/mani_skill/utils/structs/articulation_joint.html
https://mujoco.readthedocs.io/en/stable/computation/
https://mujoco.readthedocs.io/en/stable/mjx.html
https://news.qq.com/rain/a/20250218A03XNJ00
https://nvidia-omniverse.github.io/PhysX/physx/5.4.1/_api_build/classPxSceneDesc.html
https://oss-global-cdn.unitree.com/static/a0542aa67f4b4b1f9095682e4c5c3292.zip
https://raw.githubusercontent.com/Genesis-Embodied-AI/Genesis/main/README.md
https://raw.githubusercontent.com/NVIDIA/Isaac-GR00T/main/README.md
https://raw.githubusercontent.com/TheRobotStudio/SO-ARM100/main/README.md
https://raw.githubusercontent.com/haosulab/ManiSkill/main/README.md
https://raw.githubusercontent.com/isaac-sim/IsaacLab/main/source/isaaclab/isaaclab/sim/simulation_cfg.py
https://raw.githubusercontent.com/newton-physics/newton/main/README.md
https://raw.githubusercontent.com/unitreerobotics/unifolm-world-model-action/main/README.md
https://raw.githubusercontent.com/unitreerobotics/unitree_mujoco/main/unitree_robots/g1/g1_23dof.xml
https://raw.githubusercontent.com/unitreerobotics/unitree_mujoco/main/unitree_robots/g1/g1_29dof.xml
https://raw.githubusercontent.com/unitreerobotics/unitree_mujoco/main/unitree_robots/h1/h1.xml
https://raw.githubusercontent.com/unitreerobotics/unitree_mujoco/main/unitree_robots/h1_2/h1_2_handless.xml
https://raw.githubusercontent.com/unitreerobotics/unitree_mujoco/main/unitree_robots/h2/h2_mujoco.xml
https://raw.githubusercontent.com/unitreerobotics/unitree_mujoco/main/unitree_robots/r1/R1_C++.xml
https://raw.githubusercontent.com/unitreerobotics/unitree_rl_lab/main/source/unitree_rl_lab/unitree_rl_lab/assets/robots/unitree.py
https://raw.githubusercontent.com/unitreerobotics/unitree_ros/master/robots/g1_description/g1_29dof_rev_1_0.urdf
https://raw.githubusercontent.com/unitreerobotics/unitree_ros2/master/cyclonedds_ws/src/unitree/unitree_hg/msg/LowCmd.msg
https://raw.githubusercontent.com/unitreerobotics/unitree_ros2/master/cyclonedds_ws/src/unitree/unitree_hg/msg/LowState.msg
https://raw.githubusercontent.com/unitreerobotics/unitree_ros2/master/cyclonedds_ws/src/unitree/unitree_hg/msg/MotorCmd.msg
https://raw.githubusercontent.com/unitreerobotics/unitree_ros2/master/cyclonedds_ws/src/unitree/unitree_hg/msg/MotorState.msg
```

https://raw.githubusercontent.com/unitreerobotics/unitree_sdk2/main/example/g1/g1d/g1d_arm_example.cpp
https://raw.githubusercontent.com/unitreerobotics/unitree_sdk2/main/example/g1/low_level/g1_ankle_swing_example.cpp
https://raw.githubusercontent.com/unitreerobotics/unitree_sdk2/main/include/unitree/idl/hg/LowCmd_.hpp
https://raw.githubusercontent.com/unitreerobotics/unitree_sdk2/main/include/unitree/idl/hg/LowState_.hpp
https://raw.githubusercontent.com/unitreerobotics/unitree_sdk2/main/include/unitree/idl/hg/MotorCmd_.hpp
https://raw.githubusercontent.com/unitreerobotics/unitree_sdk2/main/include/unitree/idl/hg/MotorState_.hpp
https://robot-api.unitree.com/doc/?space=
https://robot-api.unitree.com/doc/?space=Motor_SDK_Dev_Guide&locale=en
https://robot-api.unitree.com/doc/spaces2?locale=en
https://sapien-sim.github.io/docs/
https://shop.unitree.com/collections/all
https://shop.unitree.com/products/go1-motor
https://shop.unitree.com/products/unitree-im6014-motor
https://shop.unitree.com/products/unitree-z1
https://shop.unitree.com/sitemap.xml
https://support.unitree.com/home/zh/developer
https://www.1x.tech/discover/1x-world-model
https://www.arterytek.com/cn/product/AT32M416.jsp
https://www.cnblogs.com/flyingsir/p/19220639
https://www.ethercat.org/en/technology.html
https://www.figure.ai/news
https://www.figure.ai/news/helix
https://www.gigadevice.com.cn/product/mcu/mcus-product-selector/gd32m531rbt7
https://www.physicalintelligence.company/research/pi-star-0.6
https://www.st.com/en/microcontrollers-microprocessors/stm32f405rg.html
https://www.st.com/en/microcontrollers-microprocessors/stm32f767bi.html
https://www.st.com/en/microcontrollers-microprocessors/stm32g431cb.html
https://www.st.com/en/microcontrollers-microprocessors/stm32g474re.html
https://www.st.com/en/microcontrollers-microprocessors/stm32h723zg.html
https://www.st.com/en/microcontrollers-microprocessors/stm32h743ai.html
https://www.unitree.com/Dex3-1
https://www.unitree.com/Dex5-1
https://www.unitree.com/H2
https://www.unitree.com/IM6014/
https://www.unitree.com/R1
https://www.unitree.com/cn/opensource/
https://www.unitree.com/g1
https://www.unitree.com/go1/motor

```
https://www.unitree.com/h1
https://www.unitree.com/motor/
https://www.unitree.com/z1
https://zhuanlan.zhihu.com/p/2069008612740031291
https://zhuanlan.zhihu.com/p/29880026964
```

四足史料分组（非本书解剖对象）：`https://www.unitree.com/go1/motor` 与 `https://shop.unitree.com/products/go1-motor` 在上面的清单里**保留**——它们讲的是 GO-M8010-6 这颗关节电机，只因宇树把电机文档挂在 Go1 目录下才带这个名字，用途是器件史料（附录 C 的电机表、 第 15 章的器件阶梯），讲的是电机不是狗。另有 7 条早期 Go1/Go2 生态的第三方开发与仿真资源， 2026-10-08 收口时从上面的清单移进这一行，只留名、不进任何一章的论证链： `egh0bww1.com/posts/2023-10-24-unitree-go1-collection/`、 `github.com/MAVProxyUser/YushuTechUnitreeGo1`、 `github.com/abizovnuralem/go2_omniverse`、 `github.com/abizovnuralem/go2_ros2_sdk`、`github.com/jizhang-cmu/autonomy_stack_go2`、 `github.com/legion1581/go2_firmware_tools`、 `github.com/Huuhuuw/FAST_LIO_LOCALIZATION_HU`。 本书的解剖对象只有人形：同一清单里的 `unitree.com/g1`、`unitree.com/H2`、`unitree.com/h1`、`unitree.com/R1` 与它们的消息、模型与文档页才是本书逐条核对过的东西。

E.4 「查不到」清单

本书认真找过、但**没有拿到官方答案**的问题。它们不是疏漏，是这本书的边界。

器件与整机

1. 全系关节电机的**额定（持续）扭矩**——一个型号都没有。
2. 关节驱动板的 **PWM 载波频率、死区时间、是否做死区补偿**。
3. 关节电机的 **绕组电感 L、相电阻 R、转动惯量 J、极对数 p、磁链 λ_m**。

4. 关节电机是 **SPM 还是 IPM**；是否实现**弱磁**。
5. 关节驱动板的 **MCU 型号**（拆解称芯片丝印被抹除）。
6. **IMU 的型号、供应商、量程、零偏稳定性、噪声密度、整机 IMU 采样率、安装位置、g 敏感度**。
7. 编码器的**单圈/多圈**、输出端位数与精度（M8010 官方为单圈；整机页只写“双编码器”）。
8. **电芯型号、化学体系、能量密度、包重、放电倍率（C 数）、循环寿命、BMS 策略**。
9. **整机功耗**（平均与峰值）、**每公里能耗、可行驶里程**——因此人形的 CoT 无法计算。
10. 各型号的 **IP 防护等级**（四足侧有 IP54/IP67 口径，人形侧未公布）。
11. **结构模态频率**、连杆截面与材料牌号、热处理与牌号。
12. G1/H1/H2 的**关节总线拓扑**（几条干、每干几台、收发器型号）。

软件与控制

13. 整机在线控制律（是否跑 WBC/MPC、求解器、频率）。
14. 出厂固件里关节环路的实际 kp/kd 默认值（仓库里的值是训练默认值）。
15. UnifoLM 的**动作专家规模、动作头形式、是否动作分块、是否双系统、部署方式、逐任务成功率**。
16. 相机是否**硬件同步**、时钟源、是否全局快门。
17. `rt/lowstate` 在人形机型上的确切频率（Go2 上本书实测 500 Hz，不可外推）。
18. ROS 2 Jazzy 的支持状态。

安全与制度

19. **急停是单通道还是双通道**、是否走硬件安全回路。
20. **断电时序**（抱闸与断动力的先后、卸扭矩耗时）。
21. **上电默认态**，以及如何检测“断电期间被搬动过”。

22. **有人模式下的力矩与功率限制值**，依据哪条标准条款。
23. 任何一款人形的 **MTBF / 连续作业时长 / 失败恢复机制**。
24. 已获安全认证清单（CE / CR / ISO 10218 / ISO/TS 15062 / IEC 62443）。

行业与商业

25. 各家 **GPU 集群规模**、数采工厂真实台数、单条数据成本、数据合格率（北京人形数据基地除外）。
26. 银河通用的 MCU / 总线 / 传感器品牌 / 双臂供应商（完全无公开资料）。
27. 智元减速器路线的最终口径（官方“行星替代谐波”与供应链“来福谐波占比 >90%”两条并存）。
28. Dex1-1 / Dex2-5 / Dex3-1 的官方售价；H1-2 的价格与完整规格。

E.5 冲突口径登记（本书一律并列，不取平均）

项	口径 A	口径 B	出处
G1 膝峰值扭矩	90（基础页）/ 120（EDU 页与文档）	139（官方模型与 rl_lab 配置）	[厂]
G1 踝峰值扭矩	35 / 50 / 25	35（N5020-16-parallel）	[厂]
G1 hip_roll effort	139	88（同仓库另一 URDF）	[厂]
G1 减速方案	两级行星 ≈20.58	官方型号名 14.3 / 22.5	[拆] vs [厂]
H1 膝峰值扭矩	360（产品页）	300（官方 MJCF/URDF）；200（rl_lab M107-24-2）	[厂]
关节总线	官方“全部电机 RS-485”	拆解“CAN 高低线菊花链”	[厂] vs [拆]
IM6014 电压上限	75 V	72 V（同页另一处）	[厂]

IM6014 电机 ID	0~14 + 15 广播	0-15（STM32 参数表）	[厂]
R1 总自由度	26-40	20/26/26；“20-26-DOF”	[厂]
智元减速器	官方“准直驱+行星替代谐波，BOM ↓30%”	供应链“来福谐波占比 >90%”	[查]

E.6 本书判定为「不可采信」的流传说法

- 网传 H1 售价“9.5 万元”——来源不可靠。
- 宇树存在型号「M107-24」——只查到 M107 与 rl_lab 里的 M107-24-2。
- 宇树「Unitree Mimics」数据集——不存在，最接近的是 UnifoLM-WBT-Dataset。
- 「宇树用国产谐波减速器降本」——在宇树自家产品上只能证到 Z1（官方谐波）与拆解口径的 G1（行星），证据不足。
- 银河通用 ET1 的两套互斥规格（38 DoF/RK3588 与 173 cm/72 DoF/Thor）——疑为自动生成内容。
- 店铺里 $100,000 一档（B2/A2/B1/H2-Plus）——判定为占位价。

附录 F

附录：语录出处一览

正文各章的语录块只写「媒体 · 日期」，完整出处集中于此，便于核校。

章	说话人	完整出处	日期	类型	链接
第 00 章	王兴兴	上海证券报《做机器人第十六年 我看到了“黎明的光”——对话宇树科技董事长王兴兴》（新浪财经转载）	2026-08-19	直接引语（问答实录）	https://finance.sina.cn/2026-08-19/detail-ininuqvc7183212.d.html
第 01 章	王兴兴	投中网《五年前，对王兴兴的一次未公开访谈》（新浪科技转载）	页面 2025-08-07；访谈本体约 2020 年前后	早年访谈转写	https://news.sina.cn/ai/2025-08-07/detail-infkcrwy5874615.d.html
第 02 章	王兴兴	《晚点 LatePost》对话王兴兴（新浪财经转载；晚点首刊 2024-05，采自 2023-10 与 2024-05 两次访谈）	2025-02-16（首刊 2024-05）	直接引语（问答实录，回答归属明确）	https://finance.sina.cn/2025-02-16/detail-ineksnuy4910269.d.html

第 03 章	王兴兴	上海证券报（同上，来源 A），谈降价靠什么	2026-08-19	直接引语	https://finance.sina.cn/2026-08-19/detail-ininuqvc7183212.d.html
第 04 章	王兴兴	凤凰卫视《X+》专访（主持人田桐），腾讯新闻转载	2025-02-21	直接引语（问答按轮次归属）	https://news.qq.com/rain/a/20250221A05P2500
第 05 章	埃隆·马斯克（Tesla / xAI；人形机器人 Optimus 的公开发言者）	Business Insider 综述报道里的马斯克原话	2026-07-24	英文原句，经第三方报道逐字引用；本书未核对录音或视频原文	https://www.businessinsider.com/everything-we-know-about-teslas-optimus-humanoid-robot-2026-7
第 06 章	王兴兴	凤凰卫视《X+》专访（主持人田桐），腾讯新闻转载	2025-02-21	直接引语（问答按轮次归属）	https://news.qq.com/rain/a/20250221A05P2500
第 07 章	埃隆·马斯克（Tesla / xAI；人形机器人 Optimus 的公开发言者）	特斯拉 2025 年第三季度财报电话会议（36 氪英文版报道）	2025-11-07	英文原句，经第三方报道逐字引用；本书未核对录音或视频原文	https://eu.36kr.com/en/p/3542648986251139
第 08 章	王兴兴	投中网（同上，来源 B），谈选型教训	页面 2025-08-07；访谈本体约 2020 年前后	早年访谈转写	https://news.sina.cn/ai/2025-08-07/detail-infkcrwy5874615.d.html

第 09 章	埃隆·马斯克（Tesla / xAI；人形机器人 Optimus 的公开发言者）	马斯克就 Optimus 早期产能爬坡对投资者的警告（Business Insider 综述）	2026-07-24	英文原句，经第三方报道逐字引用；本书未核对录音或视频原文	https://www.businessinsider.com/everything-we-know-about-teslas-optimus-humanoid-robot-2026-7
第 10 章	埃隆·马斯克（Tesla / xAI；人形机器人 Optimus 的公开发言者）	马斯克在财报电话会议上讲 Optimus 手部设计（Business Insider 综述）	2026-07-24	英文原句，经第三方报道逐字引用；本书未核对录音或视频原文	https://www.businessinsider.com/everything-we-know-about-teslas-optimus-humanoid-robot-2026-7
第 11 章	埃隆·马斯克（Tesla / xAI；人形机器人 Optimus 的公开发言者）	马斯克对量产规模下 Optimus 单台成本的估计（Business Insider 综述）	2026-07-24	英文原句，经第三方报道逐字引用；本书未核对录音或视频原文	https://www.businessinsider.com/everything-we-know-about-teslas-optimus-humanoid-robot-2026-7
第 12 章	埃隆·马斯克（Tesla / xAI；人形机器人 Optimus 的公开发言者）	特斯拉 2024 年第四季度股东更新（公司自己发布的 PDF，逐字抽取）	2025-01-29	特斯拉自己发布的股东更新原文（PDF 全文用 PyMuPDF 抽取，逐字核对）	https://digitalassets.tesla.com/tesla-contents/image/upload/IR/TSLA-Q4-2024-Update.pdf

第 13 章	王兴兴	《晚点 LatePost》对话王兴兴（问答实录），新浪财经转载	2025-02-16	直接引语（问答实录，按“王兴兴：”归属）	https://finance.sina.cn/2025-02-16/detail-ineksnuy4910269.d.html
第 14 章	王兴兴	投中网（同上，来源 B）	页面 2025-08-07；访谈本体约 2020 年前后	早年访谈转写	https://news.sina.cn/ai/2025-08-07/detail-infkcrwy5874615.d.html
第 15 章	王兴兴	投中网（同上，来源 B），谈为什么选电驱不选液压	页面 2025-08-07；访谈本体约 2020 年前后	早年访谈转写	https://news.sina.cn/ai/2025-08-07/detail-infkcrwy5874615.d.html
第 16 章	王兴兴	21世纪经济报道《宇树科技王兴兴谈机器人现状：最大挑战在哪里？为什么坚持开源？》（2025 骁龙峰会·中国嘉宾对话，东方财富转载）	2025-09-25	直接引语（圆桌发言）	https://fund.eastmoney.com/a/202509243522729937.html

第 17 章	王兴兴	王兴兴 GTC 2026 演讲文字整理（“机器人前瞻”整理；页面自陈“在不改变原意的前提下进行了一定程度的增删修改”），腾讯新闻	2026-03-17	演讲整理稿（非逐字，页面已声明增删修改）	https://news.qq.com/rain/a/20260317A06NCH00
第 18 章	埃隆·马斯克（Tesla / xAI；人形机器人 Optimus 的公开发言者）	特斯拉 2025 年第一季度股东更新（公司自己发布的 PDF，逐字抽取）	2025-04-22	特斯拉自己发布的股东更新原文（PDF 全文用 PyMuPDF 抽取，逐字核对）	https://digitalassets.tesla.com/tesla-contents/image/upload/IR/TSLA-Q1-2025-Update.pdf
第 19 章	王兴兴	王兴兴 GTC 2026 演讲文字整理（“机器人前瞻”整理；页面自陈“在不改变原意的前提下进行了一定程度的增删修改”），腾讯新闻	2026-03-17	演讲整理稿（非逐字，页面已声明增删修改）	https://news.qq.com/rain/a/20260317A06NCH00
第 20 章	王兴兴	周天财经对谈王兴兴（雪球嘉年华文字实录；页面自陈“为文气通顺，有适度书面化微调”），钛媒体	2025-01-13	对谈实录（经书面化微调，非逐字）	https://www.tmtpost.com/7418531.html

第 21 章	王兴兴	王兴兴 2025 世界机器人大会主旨演讲（新浪“机器人前瞻”文字实录）	2025-08-09	第一人称演讲逐字	https://k.sina.cn/article_7950358917_1d9e0d98501901bnhc.html
第 22 章	王兴兴	新浪科技／和讯《首发对话实录｜王兴兴，聊宇树科技上市》（2025 WRC 现场采访）	2025-08-09	口语采访逐字（只截引这一整句）	https://m.hexun.com/tech/2025-08-09/220818502.html
第 23 章	王兴兴	界面新闻《对话宇树王兴兴：做人形机器人要克制一点，别变成传统自动化公司》	2024-08-21	直接引语（专访）	https://www.jiemian.com/article/11587971.html
第 24 章	埃隆·马斯克（Tesla / xAI；人形机器人 Optimus 的公开发言者）	特斯拉 2025 年第三季度财报电话会议（36 氪英文版报道）	2025-11-07	英文原句，经第三方报道逐字引用；本书未核对录音或视频原文	https://eu.36kr.com/en/p/3542648986251139
第 25 章	王兴兴	《晚点 LatePost》对话王兴兴（新浪转载）	2025-02-16（首刊 2024-05）	直接引语（问答实录）	https://finance.sina.cn/2025-02-16/detail-ineksnuy4910269.d.html

第 26 章	王兴兴	王兴兴 2025 世界机器人大会主旨演讲（新浪“机器人前瞻”文字实录）	2025-08-09	第一人称演讲逐字	https://k.sina.cn/article_7950358917_1d9e0d98501901bnhc.html
第 27 章	王兴兴	21世纪经济报道（2025 骁龙峰会·中国嘉宾对话报道），天天基金转载	2025-09-24	记者转述（全句无引号、由记者以“在王兴兴的期待中”引出；原场为圆桌对话，无逐字稿可核）	https://fund.eastmoney.com/a/202509243522729937.html
第 28 章	埃隆·马斯克（Tesla / xAI；人形机器人 Optimus 的公开发言者）	马斯克在播客里谈 xAI 与人形机器人的协同（Business Insider 综述；原文为片段）	2026-07-24	英文原句，经第三方报道逐字引用；本书未核对录音或视频原文	https://www.businessinsider.com/everything-we-know-about-teslas-optimus-humanoid-robot-2026-7
第 29 章	王兴兴	王兴兴 2025 世界机器人大会主旨演讲（新浪“机器人前瞻”文字实录）	2025-08-09	第一人称演讲逐字	https://k.sina.cn/article_7950358917_1d9e0d98501901bnhc.html
第 30 章	王兴兴	界面新闻（同上，来源 D）	2024-08-21	直接引语（专访）	https://www.jiemian.com/article/11587971.html

第 31 章	王兴兴	王兴兴 2025 世界机器人大会主旨演讲（新浪“机器人前瞻”文字实录）	2025-08-09	第一人称演讲逐字	https://k.sina.cn/article_7950358917_1d9e0d98501901bnhc.html
第 32 章	王兴兴	《晚点 LatePost》对话王兴兴（新浪转载）	2025-02-16（首刊 2024-05）	直接引语（问答实录）	https://finance.sina.cn/2025-02-16/detail-ineksnuy4910269.d.html
第 33 章	王兴兴	王兴兴 2025 世界机器人大会主旨演讲（新浪“机器人前瞻”文字实录）	2025-08-09	第一人称演讲逐字（“是反而是”为实录原文重复字，保留）	https://k.sina.cn/article_7950358917_1d9e0d98501901bnhc.html
第 34 章	王兴兴	王兴兴 2025 世界机器人大会主旨演讲（新浪“机器人前瞻”文字实录）	2025-08-09	第一人称演讲逐字	https://k.sina.cn/article_7950358917_1d9e0d98501901bnhc.html
第 35 章	王兴兴	投中网（同上，来源 B）	页面 2025-08-07；访谈本体约 2020 年前后	早年访谈转写	https://news.sina.cn/ai/2025-08-07/detail-infkcrwy5874615.d.html
第 36 章	王兴兴	凤凰卫视《X+》专访（主持人田桐），腾讯新闻转载	2025-02-21	直接引语（问答按轮次归属）	https://news.qq.com/rain/a/20250221A05P2500

第 37 章	王兴兴	王兴兴 GTC 2026 演讲文字整理（“机器人前瞻”整理；页面自陈“在不改变原意的前提下进行了一定程度的增删修改”），腾讯新闻	2026-03-17	演讲整理稿（非逐字，页面已声明增删修改）	https://news.qq.com/rain/a/20260317A06NCH00
第 38 章	王兴兴	《中国电子报》“人形机器人高质量发展”系列访谈（腾讯新闻转载）	采访 2024-08-20；转载 2025-02-18	直接引语（记者以“王兴兴回应表示”引出）	https://news.qq.com/rain/a/20250218A03XNJ00
第 39 章	王兴兴	王兴兴 2025 世界机器人大会主旨演讲（新浪“机器人前瞻”文字实录）	2025-08-09	第一人称演讲逐字	https://k.sina.cn/article_7950358917_1d9e0d98501901bnhc.html
第 40 章	王兴兴	《晚点 LatePost》对话王兴兴（新浪转载）	2025-02-16（首刊 2024-05）	直接引语（问答实录）	https://finance.sina.cn/2025-02-16/detail-ineksnuy4910269.d.html